Quantum Secure Cryptographic library - QSC

Revision 1.4.0.0, November 6, 2026

John G. Underhill – john.underhill@protonmail.com

This document is an engineering level description of the QSC cryptographic library. In its contents, a guide to implementing QSC, an explanation of its design, as well as references to its component primitives and links to supporting documentation.

The QSC library is the property of the author John G. Underhill and the QRCS Corporation.

Contents **Page**

1 Foreword 5

2 Introduction 6

3 References 8

4 Tables 10

5 Symmetric Ciphers 14

5.1 CSX-512 14

5.2 RCS 17

5.3 ChaCha 20

5.4 AES 25

6 HASH and MAC Functions 44

6.1 SHA3 44

6.2 SHA2 50

6.3 KMAC 54

6.4 HMAC 58

6.5 QMAC 61

7 Random Providers 71

7.1 ACP 71

7.2 CSP 73

7.3 RDP 74

8 DRBG, KDF, and PRNG 77
8.1 SHAKE
45
77
8.2 cSHAKE 81
8.3 HKDF 82
8.4 CSG 84
8.5 HCG 86
8.6 SCB 89
8.7 SECRAND 91
9 Asymmetric Ciphers 97
9.1 ECDH 97
9.2 Kyber 99
9.3 McEliece 101
9.4 HQC
9.5 EDDH
10 Asymmetric Signature Schemes 147
10.1 Dilithium 147
10.2 ECDSA 149
10.3 EDDSA
10.4 Falcon 150
10.5 SPHINCS+ 152
11 TLS 1.3 212
12 X.509 PKI Toolkit 309
13 Networking and Sockets 432
13.1 IP Info 432
13.2 Queue 436
13.3 Socket 439

13.4 Socket Client 451
13.5 Socket Server 454
13.6 Network Utilities 457
14 Threads and Asynchronous Processing 474
14.1 Async Threading 100
14.2 Events 478
14.3 Thread Pool 481
15 Storage and Data Processing 490
15.1 Collection 490
15.2 List 109
15.3 QSORT 496
16 Integer and String Tools 504
16.1 Array Utilities 504
16.2 Console Utilities 506
16.3 Encoding 512
16.4 File Utilities 517
16.5 Folder Utilities 523
16.6 Integer Utilities 525
16.7 String Utilities 533
16.8 System Utilities 542
16.9 TimerEx 545
16.10 Time Stamp 547
16.11 Transpose 549
16.11 Winutils 153
17 Memory and Processor 589
17.1 CPUID 589
17.2 SIMD Memory Utilities 590
17.3 Secure Memory Functions 596
18 Validation, Build, and Wrapper AssetsDesign Considerations 597
5.5 Additional Symmetric Cipher APIs 33

6.6 Additional Hash and MAC APIs 64
7.4 Entropy Providers 75
8.7 Additional DRBG / KDF APIs 96
9.6 Additional KEM APIs 106
10.6 Additional Signature APIs 155
11.1 TLS API Reference 217
12.1 X.509 API Reference 310
13.7 Networking APIs 461
14.4 Concurrency APIs 483
15.4 Collections APIs 499
16.12 Utility APIs 556
17.4 Memory and CPU APIs 598
18.5 Optimized Compilation Profiles 603
19 Hardware Security 604
Annex A Design Considerations 627

1: Foreword

This document is intended as a formal description of the QSC library, an inventory of its contents, function definitions, and brief descriptions of the algorithms.This is a summary of the functions and capabilities of the QSC library revision identified in the document title block.

For explanations of the various algorithms implemented in the QSC library, refer to the References section of this document.

QSC has been used to implement several of our products, including the distributed key tunneling protocol SKDP, the multi party crypto-system MPDC, the asymmetric tunneling protocol QSMP, and HKDS, a distributed key management protocol. The

library now also includes a full TLS 1.3 client and server implementation and a comprehensive X.509 PKI toolkit.

2: Introduction

QSC is a compact and self-contained post-quantum-secure cryptographic functions library written in the C programming language. It has been written to the highest standards of secure programming, using the MISRA rule-set, and has been designed to be easy to read, verify, and implement. The code in this library is well structured, highly readable and well commented, and thoroughly documented throughout.

It contains only the most secure, proven, and powerful cryptographic primitives. It is focused on the future, not the past, and the future of the worlds secure crypto-systems must be post-quantum secure. We have installed some algorithms for backwards

compatibility; AES, ChaCha, SHA2, HMAC, HKDF, ECDH (P-256/P-384/P-521), ECDSA (P-256/P-384/P-521), EDDH (Curve25519/Curve448), and EDDSA (Ed25519/Ed448). But our primary design goal has been to create a set of modern tools, ones that can be used to build the security systems of the future.

The library implements the post-quantum asymmetric mechanisms exposed by the current public headers: Kyber / ML-KEM, Classic McEliece, HQC, Dilithium / ML-DSA, Falcon, and SPHINCS+. These implementations are organized under the QSC public API and include portable reference paths and SIMD-accelerated paths where enabled by the build configuration and the target processor.

QSC has a powerful set of tools; a full TLS 1.3 client and server implementation, a comprehensive X.509 PKI toolkit covering certificate creation, CSR generation, CRL management, OCSP, PKCS#12 keystores, and PEM encoding. It also includes an IPv4/IPv6 networking stack featuring synchronous and asynchronous sockets, multi-threading tools, SIMD memory functions, integer conversion and evaluation functions, BER/DER/HEX/PEM encoding utilities, ASN.1 helpers, as well as a full set of cryptographic hashes, MAC functions, DRBGs, random providers, and random number generators. It is a complete set of tools, all vetted for secure operation, and written to stringent specifications, that can be applied in the most high-security of environments.

QSC also contains two QRCS symmetric cipher designs, RCS and CSX. These authenticated stream ciphers are designed for long-term security applications and are exposed through authenticated transform APIs with associated-data support.

We have implemented the Keccak family of hash functions, and pseudo-random generators including SHA3, SHAKE, and KMAC. Keccak is probably the most well-studied set of cryptographic functions in modern times, having won the NIST SHA3 competition, after being the subject of intense academic scrutiny for more than three years.

The library contains many of the most currently used cryptographic functions, including HMAC, HKDF, SHA2, and AES, providing a window to backwards compatibility in your designs. What we haven't done, is install anything that we feel is insecure in the current context, while focusing on ciphers and protocols that provide the best guarantee of long-term security. We have made a very modular, compact and efficient library that future security products can be constructed with; our encrypted tunneling protocol QSMP, was built using this library, as well as the key distribution protocols SKDP, and multi-party crypto-system MPDC. Its small size makes it ideal for IOT devices and deployments with memory and storage constraints. QSC can just as easily be used in server/client software, or any implementation that requires a securely coded, fast and

powerful set of cryptographic tools. The modular design, makes it ideal as a base tool-set implemented in more complex communications protocols. QSC was designed to be the future of cryptographic development, lean and mean, and containing only the most powerful tools available today.

3: References

3.1 Normative References

The following documents serve as references for key components in QSC:

1. NIST FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable Output Functions
2. NIST SP 800-185: Derived Functions cSHAKE, KMAC, TupleHash and ParallelHash
3. NIST SP 800-90A: Recommendation for Random Number Generation
4. NIST SP 800-108: Recommendation for Key Derivation using Pseudorandom Functions
5. NIST FIPS 197 The Advanced Encryption Standard
6. FIPS 198.1 The AES standard

7. RFC 2104, HMAC
8. RFC 5869, HKDF

3.2 Reference Links

1. The QSC Cryptographic library: https://github.com/Steppenwolfe65/QSC
2. The Keccak Code Package: https://github.com/XKCP/XKCP
3. NIST AES FIPS 197: http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf
4. ChaCha Specification: http://cr.yp.to/chacha/chacha-20080128.pdf
1. NIST FIPS-202: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
2. SP800-185: http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf
3. NIST FIPS 202: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
4. NIST Rijndael amended: http://csrc.nist.gov/archive/aes/rijndael/Rijndael-ammended.pdf
5. FIPS 198.1 http://csrc.nist.gov/publications/fips/fips198-1/FIPS-198-1_final.pdf
6. NIST SP800-90B: http://csrc.nist.gov/publications/drafts/800-90/draft-sp800-90b.pdf
7. NIST SHA3 The Keccak digest: http://keccak.noekeon.org/Keccak-submission-3.pdf
8. ChaCha Specification: http://cr.yp.to/chacha/chacha-20080128.pdf
9. RFC 2104, HMAC: http://tools.ietf.org/html/rfc2104
10. RFC 5869, HKDF: http://tools.ietf.org/html/rfc5869
11. FIPS 198-1: http://csrc.nist.gov/publications/fips/fips198-1/FIPS-198-1_final.pdf
12. NaCl library: https://nacl.cr.yp.to
13. Kyber a CCA-secure module-lattice-based KEM: https://eprint.iacr.org/2017/634.pdf
14. Kyber, a simple, provably secure key exchange: http://eprint.iacr.org/2012/688.pdf
15. Classic McEliece: https://classic.mceliece.org/nist/mceliece-20171129.pdf
16. HQC Project: https://pqc-hqc.org/
17. Dilithium Reference: https://pq-crystals.org/dilithium/data/dilithium-specification.pdf
18. Dilithium: https://pq-crystals.org/dilithium/data/dilithium-20180114.pdf
19. The Falcon Algorithm Specification: https://falcon-sign.info/falcon.pdf
20. The SPHINCS+ Signature Scheme: https://sphincs.org/data/sphincs+-specification.pdf

4: Tables

Contents **Page**

Table 5.1a CSX-512 state structure 11
Table 5.1b CSX-512 keyparams structure 11
Table 5.1c CSX-512 constants 12
Table 5.2a CSX-512 state structure 12
Table 5.2b CSX-512 keyparams structure 13
Table 5.2c RSX constants 13
Table 5.3a ChaCha state structure 16
Table 5.3b ChaCha keyparams structure 16
Table 5.3c ChaCha constants 17

Table 5.4a AES state structure 19
Table 5.4b AES keyparams structure 19
Table 5.4c AES keyparams structure 20
Table 5.4d AES constants 20
Table 6.1a SHA3 state structure 25
Table 6.1b SHA3 constants 26
Table 6.2a SHA2-256 state structure 29
Table 6.2b SHA2-512 state structure 29
Table 6.2c SHA2 constants 29
Table 6.3a Keccak state structure 30
Table 6.4a HMAC-256 state structure 36
Table 6.4b HMAC-512 state structure 36
Table 6.5a QMAC state structure 38
Table 6.4b QMAC keyparams structure 38
Table 6.4c QMAC modes enumeration 38
Table 7.1a ACP constants 41
Table 7.2a CSP constants 42
Table 7.3a RDP constants 43
Table 8.1a Keccak state structure 45
Table 8.4a CSG state structure 45
Table 8.4b CSG constants 50
Table 8.5a HCG structure 52
Table 8.5b HCG constants 52
Table 8.6a secrand state structure 54
Table 8.6b secrand constants 54
Table 9.1a ECDH constants 61
Table 9.2a Kyber constants 62
Table 9.2b Kyber parameter sets 62
Table 9.3a McEliece constants 64
Table 9.3b McEliece parameter sets 64

Table 10.1a Dilithium constants 66
Table 10.1b Dilithium parameter sets 66
Table 10.2a ECDSA constants 68
Table 10.4a Falcon constants 71
Table 10.4b Falcon parameter sets 71
Table 10.5a SPHINCS+ constants 73
Table 10.5b SPHINCS+ parameter sets 74
Table 13.1a IPv4 address structure 75
Table 13.1b IPv6 address structure 75
Table 13.1c IPv4 address info structure 75
Table 13.1d IPv6 address info structure 75
Table 13.2 queue constants 79
Table 13.3a socket state structure 81
Table 13.3b socket async receive structure 81
Table 13.3c socket receive poll structure 82
Table 13.3d socket state structure 82
Table 13.3e socket state constants 83
Table 13.3f IPv6 prefix types enumeration 83
Table 13.3g address families enumeration 83
Table 13.3h socket states enumeration 83
Table 13.3i socket options enumeration 84
Table 13.3j socket protocols enumeration 84
Table 13.3k socket receive enumeration 85
Table 13.3l socket send enumeration 85
Table 13.3m socket shut down enumeration 85
Table 13.3n socket transports enumeration 85
Table 13.5a socket async receive structure 93
Table 13.5b socket async accept structure 93
Table 13.6a network adaptor info structure 96
Table 13.6b network utilities constants 97

Table 14.1 async type definitions 100
Table 14.2a event types 103
Table 14.2b event list enumeration 104
Table 14.2c event handler structure 104
Table 14.4a thread pool structure 105
Table 15.1 socket state constants 114
Table 15.2a font color enumeration 115
Table 15.2b font style enumeration 116
Table 16.1 BER ASN1 tag enumeration 120
Table 16.2 the BER element state structure 121
Table 17.1 cpu features structure 158

5: Symmetric Ciphers

5.1 CSX-512

Header:

csx.h

Description:

CSX is based on the ChaCha stream cipher, with some important differences. We doubled the size of the input cipher key to 512-bits. We accomplished this by changing the ciphers transform functions internal integer size from 32-bit integers, to 64-bit integers. This doubled the size of the internal state from 512-bit to 1024-bit. The integer rotation constants were also changed to align with 64-bit integers, and the number of transformation rounds have been increased from the ChaCha maximum of 20 to 40

rounds. CSX is an authenticated AEAD stream-cipher, using the Keccak message authentication code generator KMAC, to authenticate the message stream.

Structures:

The qsc_csx_state structure contains the internal cipher state used by the cipher, along with the Keccak state used by the authentication function KMAC, and a Boolean storing the cipher mode; encrypt or decrypt.

Data Name	Data Type	Bit Length	Function
state	Uint64 Array	1024	Cipher state
kstate	Uint64 Array	1600	Keccak state
encrypt	Bool	8	Mode

Table 5.1a CSX-512 state structure

The qsc_csx_keyparams structure is used to load the symmetric cipher key, the key length, nonce, and mode into the cipher through the initialization function.

Data Name	Data Type	Bit Length	Function
key	Uint8 Array	512	Cipher key
keylen	Uint64	64	Key size
nonce	Uint8 Array	128	Cipher nonce
info	Uint8 Array	variable	Optional tweak
infolen	Uint64	64	Info size

Table 5.1b CSX-512 keyparams structure

Constants:

Constant Name	Value	Purpose
QSC_CSX_AUTHENTICATED	N/A	Enables KMAC authentication mode.
QSC_CSX_BLOCK_SIZE	128	The internal block size in bytes.
QSC_CSX_INFO_SIZE	48	The maximum byte length of the info string.
QSC_CSX_KEY_SIZE	64	The size in bytes of the CSX-512 input cipher-key.

QSC_CSX_MAC_SIZE	64	The CSX-512 MAC code array length in bytes.
QSC_CSX_NONCE_SIZE	16	The byte size of the nonce array.
QSC_CSX_STATE_SIZE	16	The uint64 size of the internal state array.

Table 5.1c CSX-512 constants

Call Hierarchy:

The initialize function is called to add the keying material and prepare the CSX state. The associated data call can optionally be used to add data like packet headers, to the authentication code generator's message input. If the mode is set to encrypt, the transform call encrypts the input message, and outputs the cipher-text and MAC tag. In decrypt mode, the cipher authenticates the cipher-text and if successful, decrypts the cipher-text to the output array. The dispose function resets the cipher state to zero.

initialize(*state*, *key*, *mode*)

associated(*state*, *message*, *length*)

transform(*state*, *output*, *input*, *length*)

dispose(state)

API:

Dispose

Dispose of the CSX cipher state, erases the state and resets it to zero. The dispose function takes a pointer to the CSX state as a parameter.

```
void qsc_csx_dispose(qsc_csx_state* ctx)
```

Initialize

Initializes the cipher state with the state structure, keyparams structure, and cipher mode. The initialize function takes a pointer to the CSX state, key parameters structure, and a Boolean mode assignment as parameters.

```
void qsc_csx_initialize(qsc_csx_state* ctx, const qsc_csx_keyparams* keyparams, bool encryption)
```

Set Associated

Adds associated data to the authentication mechanism, takes the cipher state, and the data, and data length as parameters. The set associated function takes a pointer to the CSX state, the data array pointer, and the data length as parameters.

```
void qsc_csx_set_associated(qsc_csx_state* ctx, const uint8_t* data, size_t length)
```

Transform

Encrypts data and adds the authentication code to the cipher-text in encrypt mode, or authenticates the cipher-text and decrypts data in decrypt mode. Takes a pointer to the CSX state structure, a pointer to the output and input arrays, and the data length as parameters.

```
bool qsc_csx_transform(qsc_csx_state* ctx, uint8_t* output, const uint8_t* input, size_t length)
```

References:

- ChaCha Specification: http://cr.yp.to/chacha/chacha-20080128.pdf
- NIST FIPS-202: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
- SP800-185: http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf
- NIST FIPS 202: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf

5.2 RCS

Header:

rcs.h

Description:

RCS is an authenticated stream cipher based on the block cipher Rijndael, the symmetric cipher used in AES. We used the wide-block form with a 256-bit internal state, and replaced the differentially weak key-schedule with the Keccak XOF function cSHAKE. We increased the number of transformation rounds from 14 with AES-256, to 22 when using a 256-bit key, or 30 rounds when using a 512-bit key. RCS is an authenticated AEAD stream-cipher, using the Keccak message authentication code generator KMAC, to authenticate the message stream.

Structures:

The **qsc_rcs_state** structure contains the internal round-key state used by the cipher, along with the Keccak state used by the authentication function KMAC, the nonce array, an internal counter, and a Boolean storing the cipher mode; encrypt or decrypt.

Data Name	Data Type	Bit Length	Function
ctype	Enum	8	Key mode
roundkeys	Uint32 Array	variable	Key state
roundkeylen	Uint64	64	Round-key size
rounds	Uint64	64	Round count
kstate	Uint64 Array	variable	KMAC state
nonce	Uint8 Array	256	Nonce array
counter	Uint64	64	Internal counter
encrypt	Bool	8	Mode

Table 5.2a CSX-512 state structure

The **qsc_rcs_keyparams** structure is used to load the symmetric cipher key, the key length, nonce, and mode into the cipher through the initialization function.

Data Name	Data Type	Bit Length	Function
key	Uint8 Array	512	Cipher key
keylen	Uint64	64	Key size
nonce	Uint8 Array	128	Cipher nonce
info	Uint8 Array	variable	Optional tweak

infolen	Uint64	64	Info size

Table 5.2b CSX-512 keyparams structure

Constants:

Constant Name	Value	Purpose
QSC_RCS_AUTHENTICATED	N/A	Enables KMAC authentication mode.
QSC_SYSTEM_RCS_AESNI_ENABLED	N/A	Enable the use of intrinsic and the AES-NI implementation.
QSC_RCS_BLOCK_SIZE	32	The internal block size in bytes.
QSC_RCS256_KEY_SIZE	32	The size in bytes of the RCS-256 input cipher-key.
QSC_RCS256_MAC_SIZE	32	The RCS-256 MAC code array length in bytes.
QSC_RCS512_KEY_SIZE	64	The size in bytes of the RCS-512 input cipher-key.
QSC_RCS512_MAC_SIZE	64	The RCS-512 MAC code array length in bytes.
QSC_RCS_NONCE_SIZE	32	The byte size of the nonce array.

Table 5.2c RSX constants

Call Hierarchy:

The initialize function is called to add the keying material and prepare the RCS state. The associated data call can optionally be used to add data, like packet headers, to the authentication code generator's message input. If the mode is set to encrypt, the transform call encrypts the input message, and outputs the cipher-text and MAC tag. In decrypt mode, the RCS authenticates the cipher-text and if successful, decrypts the cipher-text to the output array. The dispose function resets the cipher state to zero.

initialize(*state*, *key*, *mode*)

associated(*state*, *message*, *length*)

transform(*state*, *output*, *input*, *length*)

dispose(*state*)

API:

Dispose

Dispose of the RCS cipher state, erases the state and resets it to zero. The dispose function takes a pointer to the RCS state as a parameter.

```
void qsc_rcs_dispose(qsc_rcs_state* ctx)
```

Initialize

Initializes the cipher state with the state structure, keyparams structure, and cipher mode. The initialize function takes a pointer to the RCS state, key parameters structure, and a Boolean encryption mode as parameters.

```
void qsc_rcs_initialize(qsc_rcs_state* ctx, const qsc_rcs_keyparams* keyparams, bool encryption)
```

Set Associated

Adds associated data to the authentication mechanism, takes the cipher state, and the data and data length as parameters. The set associated function takes a pointer to the RCS state, the data array pointer, and the data length as parameters.

```
void qsc_rcs_set_associated(qsc_rcs_state* ctx, const uint8_t* data, size_t length)
```

Transform

Encrypts data and adds the authentication code to the cipher-text in encrypt mode, or authenticates the cipher-text and decrypts data in decrypt mode. Takes a pointer to the RCS state structure, pointers to the output and input arrays, and the data length as parameters.

```
bool qsc_rcs_transform(qsc_rcs_state* ctx, uint8_t* output, const uint8_t* input, size_t length);
```

5.3 ChaCha

Header:

chacha.h

Description:

The ChaCha implementation, like all of our cipher implementations has been optimized with AVX instructions, including AVX2 and AVX-512 implementations. It is the standard implementation of ChaChaPoly20, set to 20 transformation rounds.

Structures:

The qsc_chacha_state structure contains the internal cipher state.

Data Name	Data Type	Bit Length	Function
key	Uint8 Array	512	Cipher key
keylen	Uint64	64	Key size
nonce	Uint8 Array	128	Cipher nonce

Table 5.3a ChaCha state structure

The qsc_chacha_keyparams structure is used to load the symmetric cipher key, the key length, and the nonce into the cipher through the initialization function.

Data Name	Data Type	Bit Length	Function
key	Uint8 Array	512	Cipher key
keylen	Uint64	64	Key size
nonce	Uint8 Array	64	Cipher nonce

Table 5.3b ChaCha keyparams structure

Constants:

Constant Name	Value	Purpose

QSC_CHACHA_BLOCK_SIZE	64	The internal block size in bytes.
QSC_CHACHA_KEY128_SIZE	16	The size in bytes of the CHACHA-128 input cipher-key.
QSC_CHACHA_KEY256_SIZE	32	The size in bytes of the CHACHA-256 input cipher-key.
QSC_CHACHA_NONCE_SIZE	12	ChaCha nonce length, in bytes.
QSC_CHACHA_ROUND_COUNT	20	The number of transformation rounds.

Table 5.3c ChaCha constants

Call Hierarchy:

The initialize function is called to add the keying material and prepare the ChaCha state. If the mode is set to encrypt, the transform call encrypts the input message, and outputs the cipher-text. In decrypt mode, the cipher decrypts the cipher-text to the output array. The dispose function resets the cipher state to zero.

initialize(*state*, *key*, *mode*)

transform(*state*, *output*, *input*, *length*)

dispose(*state*)

API:

Dispose

Dispose of the ChaCha cipher state, erases the state and resets it to zero. The dispose function takes a pointer to the ChaCha state as a parameter.

```
void qsc_chacha_dispose(qsc_chacha_state* ctx)
```

Initialize

Initializes the cipher state with the state structure, keyparams structure, and cipher mode. The initialize function takes a pointer to the ChaCha state, and a pointer to the key parameters structure as parameters.

void qsc_chacha_initialize(qsc_chacha_state* ctx, const qsc_chacha_keyparams* keyparams)

Transform

Encrypts data from the input array, and writes the cipher-text to the output array in encrypt mode. In decrypt mode, input the cipher-text, and outputs the plain-text. Takes a pointer to the ChaCha state structure, a pointer to the output and input arrays, and the input length as parameters.

void qsc_chacha_transform(qsc_chacha_state* ctx, uint8_t* output, const uint8_t* input, size_t length)

The **qsc_chacha_poly1305_state** structure contains the internal cipher state.

Data Name	Data Type	Bit Length	Function
qsc_chacha_state	State	N/A	Chacha state
qsc_poly1305_state	State	N/A	Poly1305 state
aadlen	Uint64	64	AAD length
msglen	Uint64	64	Message length

Table 5.3d ChaCha_poly1305 state structure

Call Hierarchy:

The initialize function is called to add the keying material and prepare the ChachaPoly1305 state. The associated data call can optionally be used to add data, like packet headers, to the authentication code generator's message input. If the encrypt function is called, the transform call encrypts the input message, and outputs the cipher-text and MAC tag. The decryption call authenticates the cipher-text and if successful, decrypts the cipher-text to the output array. The dispose function resets the cipher state to zero.

initialize(*state*, *key*, *mode*)

associated(*state*, *message*, *length*)

encrypt(*state*, *output*, *input*, *length*)

decrypt(*state*, *output*, *input*, *length*)

dispose(*state*)

API:

Dispose

Dispose of the ChachaPoly1305 cipher state, erases the state and resets it to zero. The dispose function takes a pointer to the ChachaPoly1305 state as a parameter.

```
void qsc_ chacha_poly1305_dispose(qsc_chacha_poly1305_state* ctx)
```

Initialize

Initializes the cipher state with the state structure and keyparams structure. The initialize function takes a pointer to the ChachaPoly1305 state and key parameters structure.

```
void qsc_chacha_poly1305_initialize(qsc_chacha_poly1305_state* ctx, const
qsc_chacha_keyparams* keyparams);
```

Set Associated

Adds associated data to the authentication mechanism, takes the cipher state, and the data and data length as parameters. The set associated function takes a pointer to the qsc_chacha_poly1305_state state, the data array pointer, and the data length as parameters.

```
void qsc_ chacha_poly1305_set_associated(qsc_chacha_poly1305_state* ctx, const
uint8_t* data, size_t length)
```

Decrypt

Verifies the MAC tag appended to the message, and if it is validated, decrypts the ciphertext and writes the plaintext to the output array. Takes a pointer to the qsc_chacha_poly1305_state state structure, pointers to the output and input arrays, and the data length as parameters.

void qsc_ chacha_poly1305_encrypt(qsc_chacha_poly1305_state* ctx, uint8_t* output, const uint8_t* input, size_t length)

Encrypt

Encrypts data and adds the authentication code to the cipher-text to the output array. Takes a pointer to the qsc_chacha_poly1305_state state structure, pointers to the output and input arrays, and the data length as parameters.

bool qsc_rcs_transform(qsc_rcs_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

References:

- ChaCha Specification: http://cr.yp.to/chacha/chacha-20080128.pdf

5.4 AES

Header:

aes.h

Description:

The Advanced Encryption Standard (AES) implementation of the Rijndael cipher. Includes ECB mode for testing, as well as the CBC and CTR cipher modes, and the authenticated stream cipher mode HBA. Implementations of AES-NI and the reference code versions, enhanced with AVX to AVX-512 instructions.

Structures:

The qsc_aes_state structure contains the internal cipher state.

Data Name	Data Type	Bit Length	Function
roundkeys	Uint32 Array	variable	Key state
roundkeylen	Uint64	64	Round-key size

rounds	Uint64	64	Round count
nonce	Uint8 Array	256	Nonce array

Table 5.4a AES state structure

The qsc_aes_keyparams structure is used to load the symmetric cipher key, the key length, and the nonce into the cipher through the initialization function.

Data Name	Data Type	Bit Length	Function
key	Uint8 Array	512	Cipher key
keylen	Uint64	64	Key size
iv	Uint8 Array	64	Cipher IV

Table 5.4b AES keyparams structure

The qsc_aes_hba256_state structure is used with the HBA authenticated AEAD mode.

Data Name	Data Type	Bit Length	Function
kstate	MAC State	Variable	HMAC or KMAC state
cstate	AES State	Variable	The AES state
mkey	Uint8 Array	256	Mac Key
cust	Uint8 Array	2048	Custom string
custlen	Uint64	64	Custom string length
encrypt	Boolean	8	Encrypt/decrypt mode

Table 5.4c AES HBA-256 state structure

The qsc_aes_gcm256_state structure is used with the GCM authenticated AEAD mode.

Data Name	Data Type	Bit Length	Function
cstate	State	Variable	The AES state
C	MAC State	Variable	Current counter block
H	AES State	Variable	Hash subkey
J0	Uint8 Array	Variable	Pre-counter block
S	Uint8 Array	Variable	GHASH accumulator

aadlen	Uint64	64	AAD length in bits
ctlen	Uint64	64	Ciphertext length in bits

Table 5.4d AES GCM-256 state structure

Constants:

Constant Name	Value	Purpose
QSC_SYSTEM_AESNI_ENABLED	N/A	Enable the use of intrinsics and the AES-NI implementation.
QSC_AES_BLOCK_SIZE	16	The internal block size in bytes, required by the encryption and decryption functions.
QSC_AES128_KEY_SIZE	16	The size in bytes of the AES-128 input cipher-key.
QSC_AES256_KEY_SIZE	32	The size in bytes of the AES-256 input cipher-key.
QSC_AES_IV_SIZE	16	AES initialization vector size, in bytes.
QSC_HBA256_MAC_SIZE	32	HBA-256 MAC output size, in bytes.
QSC_HBA_MAXAAD_SIZE	256	The HBA modes maximum AAD string length.
QSC_HBA_MAXINFO_SIZE	256	The HBA modes maximum info string length.
QSC_HBA_KMAC_EXTENSION	N/A	Enables the HBA KMAC authentication mode.

Table 5.4e AES constants

Call Hierarchy:

The initialize function is called to add the keying material and prepare the AES state. If the mode is set to encrypt, the transform call encrypts the input message, and outputs the cipher-text. In decrypt mode, the cipher decrypts the cipher-text to the output array. The dispose function resets the cipher state to zero.

initialize(*state*, *key*, *mode*, *type*)

decrypt(*state*, *output*, *input*, *length*)

encrypt(*state*, *output*, *input*, *length*)

dispose(*state*)

API:

Dispose

Dispose of the AES cipher state, erases the state and resets it to zero. The dispose function takes a pointer to the AES state as a parameter.

```
void qsc_aes_dispose(qsc_aes_state* ctx);
```

Initialize

Initializes the cipher state with the state structure, keyparams structure, and cipher mode. The initialize function takes a pointer to the AES state, key parameters structure, and a Boolean encryption mode as parameters.

```
void qsc_aes_initialize(qsc_aes_state* ctx, const qsc_aes_keyparams* keyparams, bool encryption, qsc_aes_cipher_type ctype);
```

CBC Encrypt

Encrypts data from the input array, and writes the cipher-text to the output array in encrypt mode. Takes a pointer to the AES state, pointers to the input and output arrays, and input length as parameters.

```
void qsc_aes_cbc_encrypt(qsc_aes_state* ctx, uint8_t* output, const uint8_t* input, size_t length);
```

CBC Decrypt

Decrypts cipher-text from the input array, and writes the plain-text to the output array in decrypt mode. Takes a pointer to the AES state, pointers to the input and output arrays, and length as parameters.

```
void qsc_aes_cbc_decrypt(qsc_aes_state* ctx, uint8_t* output, size_t* outputlen, const uint8_t* input, size_t length);
```

CBC Encrypt Block

Encrypt a single block of data using cipher block chaining mode. Takes a pointer to the AES state, pointers to the input and output arrays as parameters.

```
void qsc_aes_cbc_encrypt_block(qsc_aes_state* ctx, uint8_t* output, const uint8_t* input);
```

CBC Decrypt Block

Decrypt a single block of data using cipher block chaining mode. Takes a pointer to the AES state, pointers to the input and output arrays as parameters.

```
void qsc_aes_cbc_decrypt_block(qsc_aes_state* ctx, uint8_t* output, const uint8_t* input);
```

Add Padding

Add PKCS7 padding to a CBC block before encryption. The function takes the input array pointer and array length as parameters.

```
void qsc_pkcs7_add_padding(uint8_t* input, size_t length)
```

Padding Size

Calculates the padding length on a CBC block after decryption. The function takes the input array pointer as a parameter, and returns the padding size.

```
size_t qsc_pkcs7_padding_length(const uint8_t* input)
```

CTR-BE Transform

Transform data using AES with a Big-Endian counter mode. This function takes a pointer to the AES state, pointers to the input and output arrays, and the input length as parameters.

```
void qsc_aes_ctrbe_transform(qsc_aes_state* ctx, uint8_t* output, const uint8_t* input, size_t length);
```

CTR-LE Transform

Transform data using AES with a Little-Endian counter mode. This function takes a pointer to the AES state, pointers to the input and output arrays, and the length as parameters.

```
void qsc_aes_ctrle_transform(qsc_aes_state* ctx, uint8_t* output, const uint8_t* input, size_t length);
```

ECB Encrypt Block

Used for encryption in known answer tests or as a component in a more complex protocol. This function takes a pointer to the AES state, pointers to the input and output arrays as parameters.

```
void qsc_aes_ecb_decrypt_block(const qsc_aes_state* ctx, uint8_t* output, const uint8_t* input);
```

ECB Decrypt Block

Used for decryption in known answer tests or as a component in a more complex protocol. This function takes a pointer to the AES state, pointers to the input and output arrays as parameters.

```
void qsc_aes_ecb_encrypt_block(const qsc_aes_state* ctx, uint8_t* output, const uint8_t* input);
```

HBA

The HBA cipher mode, turns AES into an authenticated AEAD stream cipher, using either HMAC or KMAC message authentication code generator (MAC) functions.

Dispose

Clears the HBA state, resetting the state members to their defaults. Takes a pointer to the AES state as a parameter.

```
void qsc_aes_hba256_dispose(qsc_aes_hba256_state* ctx);
```

Initialize

Initialize the HBA mode with a keyparams structure. Takes a pointer to the HBA state, the keyparams with a key, nonce, and optional info arrays, and the encryption mode as function parameters.

```
void qsc_aes_hba256_initialize(qsc_aes_hba256_state* ctx, const qsc_aes_keyparams* keyparams, bool encrypt);
```

Set Associated

The set associated function adds additional message data to the MAC function. Takes a pointer to the HBA state, a pointer to the message data, and the message length as parameters.

```
void qsc_aes_hba256_set_associated(qsc_aes_hba256_state* ctx, const uint8_t* data, size_t datalen);
```

Transform

The transform function encrypts and MACS the cipher-text in encryption mode, or authenticates the cipher-text and decrypts the message in decryption mode. The function takes a pointer to the HBA state, pointers to the output and input arrays, and the message length as parameters.

```
bool qsc_aes_hba256_transform(qsc_aes_hba256_state* ctx, uint8_t* output, const uint8_t* input, size_t length);
```

GCM

The GCM cipher mode, turns AES into an authenticated AEAD stream cipher, using either GMC message authentication code generator (MAC) functions.

Dispose

Clears the GCM state, resetting the state members to their defaults. Takes a pointer to the AES state as a parameter.

```
void qsc_aes_gcm256_dispose(qsc_aes_gcm256_state* ctx);
```

Initialize

Initialize the GCM mode with a keyparams structure. Takes a pointer to the GCM state, the keyparams with a key, nonce, and optional info arrays, and the encryption mode as function parameters.

```
void qsc_aes_gcm256_initialize(qsc_aes_gcm256_state* ctx, const qsc_aes_keyparams* keyparams, bool encryption);
```

Set Associated

The set associated function adds additional message data to the MAC function. Takes a pointer to the GCM state, a pointer to the message data, and the message length as parameters.

```
void qsc_aes_gcm256_set_associated(qsc_aes_gcm256_state* ctx, const uint8_t* data, size_t datalen);
```

Decrypt

The function verifies the MAC of the ciphertext against the tag appended to the ciphertext. If the ciphertext is verified the message is decrypted and written to the output array. The function takes a pointer to the GCM state, pointers to the output and input arrays, and the message length as parameters.

```
bool qsc_aes_gcm256_transform(qsc_aes_gcm256_state* ctx, uint8_t* output, const uint8_t* input, size_t length);
```

Encrypt

The transform function encrypts and MACs the cipher-text and appends the tag to the ciphertext output. The function takes a pointer to the GCM state, pointers to the output and input arrays, and the message length as parameters.

```
bool qsc_aes_gcm256_transform(qsc_aes_gcm256_state* ctx, uint8_t* output, const uint8_t* input, size_t length);
```

References:

- NIST AES FIPS 197: http://csrc.nist.gov/publications/fips/fips197/fips-197.pdf
- NIST Rijndael amended: http://csrc.nist.gov/archive/aes/rijndael/Rijndael-ammended.pdf

5.5 Additional Symmetric Cipher APIs

Header: aes.h

qsc_aes_ecb_decrypt_blocks

Purpose: Decrypt several consecutive AES-ECB blocks in one call.

```
void qsc_aes_ecb_decrypt_blocks(const qsc_aes_state* ctx, uint8_t* output, const uint8_t* input, size_t length);
```

Parameters: ctx: Initialized AES decryption context; borrowed and unchanged.; output: Output buffer, at least length bytes; exact in-place use permitted.; input: Input buffer, at least length bytes; partial overlap unsupported.; length: Byte count, must be a multiple of QSC_AES_BLOCK_SIZE..

Return: None (void).

Notes: ECB is not semantically secure; use only as an internal building block. No allocations or shared state.

qsc_aes_ecb_encrypt_blocks

Purpose: Encrypt several consecutive AES-ECB blocks in one call.

void qsc_aes_ecb_encrypt_blocks(const qsc_aes_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: Initialized AES encryption context; borrowed and unchanged.; output: Output buffer, at least length bytes; exact in-place use permitted.; input: Input buffer, at least length bytes; partial overlap unsupported.; length: Byte count, must be a multiple of QSC_AES_BLOCK_SIZE..

Return: None (void).

Notes: ECB is not semantically secure; use only as an internal building block. No allocations or shared state.

qsc_aes_gcm128_decrypt

Purpose: Decrypt ciphertext and verify the authentication tag using GCM-AES-128. Authenticates the ciphertext via GHASH, then decrypts it using AES-128 in GCTR mode. The input buffer must contain the raw ciphertext immediately followed by the QSC_GCM128_MAC_SIZE-byte authentication tag. Plaintext is written provisionally during GCTR and is erased before the function returns false on authentication failure. Never expose output before a true return value, including from another thread.

bool qsc_aes_gcm128_decrypt(qsc_aes_gcm128_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [struct] Pointer to an initialized qsc_aes_gcm128_state structure.; output: [uint8_t*] Pointer to the plaintext output buffer. Must be at least (length - QSC_GCM128_MAC_SIZE) bytes.; input: [const uint8_t*] Pointer to the ciphertext buffer with the appended MAC tag. Length must equal (ciphertext length + QSC_GCM128_MAC_SIZE).; length: [size_t] Total number of bytes in input, including the QSC_GCM128_MAC_SIZE-byte tag..

Return: [bool] true if authentication and decryption succeeded; false if the tag did not match (output is zeroed).

Notes: The ctx must be initialized via qsc_aes_gcm128_initialize before use. \sa qsc_aes_gcm128_initialize, qsc_aes_gcm128_set_associated, qsc_aes_gcm128_encrypt, qsc_aes_gcm128_transform

qsc_aes_gcm128_dispose

Purpose: Dispose of a GCM-128 context. Securely erases all key material and internal state held by the context, including the cipher round keys, counter block, GHASH

subkey, pre-counter block, and accumulator. Must be called when the context is no longer needed to prevent sensitive data from remaining in memory.

void qsc_aes_gcm128_dispose(qsc_aes_gcm128_state* ctx);

Parameters: ctx: [struct] Pointer to a qsc_aes_gcm128_state structure to dispose..

Return: None (void).

Notes: Must be called before the context goes out of scope. \sa qsc_aes_gcm128_initialize

qsc_aes_gcm128_encrypt

Purpose: Encrypt plaintext and append an authentication tag using GCM-AES-128. Encrypts length bytes of input using AES-128 in GCTR mode, then computes a GHASH-based authentication tag over the ciphertext (and any previously supplied AAD) and appends the QSC_GCM128_MAC_SIZE-byte tag immediately after the ciphertext in output.

void qsc_aes_gcm128_encrypt(qsc_aes_gcm128_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [struct] Pointer to an initialized qsc_aes_gcm128_state structure.; output: [uint8_t*] Pointer to the output buffer. Must be at least (length + QSC_GCM128_MAC_SIZE) bytes.; input: [const uint8_t*] Pointer to the plaintext input buffer.; length: [size_t] Number of plaintext bytes to encrypt (excluding the tag)..

Return: None (void).

Notes: The ctx must be initialized via qsc_aes_gcm128_initialize before use. The nonce used during initialization must never be reused with the same key; doing so is a catastrophic loss of security. \sa qsc_aes_gcm128_initialize, qsc_aes_gcm128_set_associated, qsc_aes_gcm128_decrypt, qsc_aes_gcm128_transform A void encrypt call does not write ciphertext or a tag after an invalid AAD addition; prefer the checked transform when possible.

qsc_aes_gcm128_initialize

Purpose: Initialize the GCM-128 context for authenticated encryption or decryption. Performs full GCM initialization: expands the 128-bit key into AES round keys, computes the GHASH subkey H = AES(K, 0^128), and derives the pre-counter block J0 from the supplied nonce. When the nonce is exactly 12 bytes it is used directly with a 32-bit counter suffix; any other length is processed through GHASH per the GCM specification.

void qsc_aes_gcm128_initialize(qsc_aes_gcm128_state* ctx, const qsc_aes_keyparams* keyparams, bool encryption);

Parameters: ctx: [struct] Pointer to a qsc_aes_gcm128_state structure to initialize.; keyparams: [const struct] Pointer to a constant qsc_aes_keyparams structure supplying the 128-bit key (keylen == QSC_AES128_KEY_SIZE), the nonce (noncelen in [1, QSC_GCM_MAX_NONCE_SIZE]), and an optional info field (unused by GCM, may be NULL/0).; encryption: [bool] true to prepare the context for encryption; false for decryption..

Return: None (void).

Notes: Requires a 16-byte key and a nonce of 1 through QSC_GCM_MAX_NONCE_SIZE bytes. Invalid parameters invalidate ctx; subsequent transforms return false without output. Parameters and their buffers must not overlap ctx. Must be called before qsc_aes_gcm128_set_associated, qsc_aes_gcm128_encrypt, qsc_aes_gcm128_decrypt, or qsc_aes_gcm128_transform. A nonce must never be reused under the same key. \sa qsc_aes_gcm128_dispose, qsc_aes_gcm128_encrypt, qsc_aes_gcm128_decrypt

qsc_aes_gcm128_set_associated

Purpose: Supply associated additional data (AAD) to GCM-128. Feeds datalen bytes of unencrypted associated data into the GHASH accumulator so they are covered by the authentication tag. Must be called after qsc_aes_gcm128_initialize and before qsc_aes_gcm128_encrypt or qsc_aes_gcm128_decrypt. May be called multiple times; each call appends to the running AAD.

void qsc_aes_gcm128_set_associated(qsc_aes_gcm128_state* ctx, const uint8_t* data, size_t datalen);

Parameters: ctx: [struct] Pointer to an initialized qsc_aes_gcm128_state structure.; data: [const uint8_t*] Pointer to the associated data buffer.; datalen: [size_t] Length of the associated data in bytes. Cumulative AAD across all calls must not exceed QSC_GCM_MAXAAD_SIZE. Rejected AAD invalidates the context until reinitialization..

Return: None (void).

Notes: Must be called before any call to qsc_aes_gcm128_encrypt or qsc_aes_gcm128_decrypt. \sa qsc_aes_gcm128_encrypt, qsc_aes_gcm128_decrypt Rejected AAD (including cumulative limit violations) invalidates the GCM context. Subsequent operations do not produce authenticated output until reinitialization. This

void setter cannot report failure directly. Use qsc_aes_gcm128_transform to obtain an explicit failure result; the direct void encrypt functions cannot report one.

qsc_aes_gcm128_transform

Purpose: Unified GCM-128 encrypt-or-decrypt transform. Dispatches to qsc_aes_gcm128_encrypt or qsc_aes_gcm128_decrypt according to the mode set during initialization. In \b encryption mode length is the number of plaintext bytes; the output buffer receives the ciphertext followed by the QSC_GCM128_MAC_SIZE-byte tag and the function returns true. In \b decryption mode length is the number of plaintext bytes (i.e. the ciphertext length \e excluding the tag); the function internally reads (length + QSC_GCM128_MAC_SIZE) bytes from input, verifies the tag, and returns true on success or false on authentication failure (output is securely zeroed on failure).

bool qsc_aes_gcm128_transform(qsc_aes_gcm128_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [struct] Pointer to an initialized qsc_aes_gcm128_state structure.; output: [uint8_t*] Pointer to the output buffer.; input: [const uint8_t*] Pointer to the input buffer.; length: [size_t] Plaintext byte count (see above for per-mode semantics)..

Return: [bool] true on success; false if authentication failed during decryption. \sa qsc_aes_gcm128_initialize, qsc_aes_gcm128_set_associated, qsc_aes_gcm128_encrypt, qsc_aes_gcm128_decrypt

qsc_aes_gcm256_decrypt

Purpose: * Decrypt ciphertext and verify the authentication tag using GCM-AES-256. Authenticates the ciphertext via GHASH, then decrypts it using AES-256 in GCTR mode. The input buffer must contain the raw ciphertext immediately followed by the QSC_GCM256_MAC_SIZE-byte authentication tag. Plaintext is written provisionally and erased before a false return. The caller must not expose output before authentication succeeds.

Details: * Decrypt ciphertext and verify the authentication tag using GCM-AES-256. Authenticates the ciphertext via GHASH, then decrypts it using AES-256 in GCTR mode. The input buffer must contain the raw ciphertext immediately followed by the QSC_GCM256_MAC_SIZE-byte authentication tag. Plaintext is written provisionally and erased before a false return. The caller must not expose output before authentication succeeds.

bool qsc_aes_gcm256_decrypt(qsc_aes_gcm256_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [struct] Pointer to an initialized qsc_aes_gcm256_state.; output: [uint8_t*] Pointer to the plaintext output buffer. * Must be at least (length QSC_GCM256_MAC_SIZE) bytes.; input: [const uint8_t*] Pointer to the ciphertext with appended MAC. Total length must be ciphertext_len QSC_GCM256_MAC_SIZE.; length: [size_t] Total number of bytes in input including the tag..

Return: [bool] true if authentication and decryption succeeded; false otherwise.

qsc_aes_gcm256_encrypt

Purpose: Encrypt data using the GCM-AES-256 authenticated encryption mode. In encryption mode, this function encrypts the plaintext using AES-256 in CTR mode, computes a MAC over the nonce and ciphertext, and appends the MAC to the output.

void qsc_aes_gcm256_encrypt(qsc_aes_gcm256_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [struct] Pointer to an initialized qsc_aes_gcm256_state structure.; output: [uint8_t*] Pointer to the output buffer; must be large enough to hold ciphertext plus MAC.; input: [const uint8_t*] Pointer to the input data; ciphertext with appended MAC.; length: [size_t] Length of the input data in bytes (excluding the MAC for decryption). \sa qsc_aes_gcm256_initialize, qsc_aes_gcm256_set_associated.

Return: None (void).

Notes: A void encrypt call does not write ciphertext or a tag after an invalid AAD addition; prefer the checked transform when possible.

Header: chacha.h

qsc_chacha_poly1305_decrypt

Purpose: Decrypt data using the ChaCha-Poly1305 authenticated encryption mode. Authenticates the ciphertext and appended 16-byte tag before producing plaintext. On tag mismatch, no plaintext is written. The state is consumed even on authentication failure; initialize a new context for the next record. Cipher and MAC state are erased after each completed authentication attempt; reuse is rejected.

bool qsc_chacha_poly1305_decrypt(qsc_chacha_poly1305_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [struct] Pointer to an initialized qsc_chacha_poly1305_state structure.; output: [uint8_t*] Pointer to the plaintext buffer.; input: [const uint8_t*] Pointer to the ciphertext data with the appended MAC.; length: [size_t] Total input length, including the appended 16-byte MAC..

Return: [bool] Returns true if the MAC verification and transformation was successful; otherwise, false. \sa qsc_chacha_poly1305_initialize, qsc_chacha_poly1305_set_associated

qsc_chacha_poly1305_dispose

Purpose: Dispose of an ChaCha-Poly1305 state. Securely clears all internal state and keys used by the ChaCha-Poly1305 authenticated encryption mode.

void qsc_chacha_poly1305_dispose(qsc_chacha_poly1305_state* ctx);

Parameters: ctx: [struct] Pointer to a qsc_chacha_poly1305_state structure..

Return: None (void).

Notes: Must be called before the state goes out of scope. \sa qsc_chacha_poly1305_set_associated, qsc_chacha_poly1305_decrypt, qsc_chacha_poly1305_encrypt, qsc_chacha_poly1305_initialize

qsc_chacha_poly1305_encrypt

Purpose: Encrypt data using the ChaCha-Poly1305 authenticated encryption mode. In encryption mode, this function encrypts the plaintext using ChaCha20, computes a MAC over the ciphertext, and appends the MAC to the output. On success, the entire record state is erased and further operations require fresh initialization.

void qsc_chacha_poly1305_encrypt(qsc_chacha_poly1305_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [struct] Pointer to an initialized qsc_chacha_poly1305_state structure.; output: [uint8_t*] Pointer to the output buffer, must be large enough to hold ciphertext plus MAC.; input: [const uint8_t*] Pointer to the plaintext; may equal output. A NULL pointer is valid only for an empty plaintext.; length: [size_t] Length of the input data in bytes (excluding the MAC for decryption). \sa qsc_chacha_poly1305_initialize, qsc_chacha_poly1305_set_associated.

Return: None (void).

qsc_chacha_poly1305_encrypt_checked

Purpose: Encrypt and authenticate with explicit counter-exhaustion reporting.

bool qsc_chacha_poly1305_encrypt_checked(qsc_chacha_poly1305_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [qsc_chacha_poly1305_state*] Initialized cipher and authenticator.; output: [uint8_t*] Ciphertext plus 16-byte tag buffer.; input: [const uint8_t*] Plaintext buffer, may equal output.; length: [size_t] Plaintext byte count..

Return: [bool] True on complete success, false if the record cannot be processed safely.

Notes: On error the output and stream counter are not modified. The caller must provide space for length plus 16 authentication-tag bytes.

qsc_chacha_poly1305_set_associated

Purpose: Set the associated data (AAD) for ChaCha-Poly1305 authenticated encryption. The associated data authenticates headers and other data without encrypting them. Call at most once per initialized record, before encryption or decryption. The AAD is padded immediately; this is not an incremental AAD update API. A NULL data pointer is accepted only when datalen is zero. The context is single-use: initialize afresh for each record and nonce.

void qsc_chacha_poly1305_set_associated(qsc_chacha_poly1305_state* ctx, const uint8_t* data, size_t datalen);

Parameters: ctx: [struct] Pointer to the qsc_chacha_poly1305_state structure.; data: [const uint8_t*] Pointer to the associated data.; datalen: [size_t] Length of the associated data in bytes. \sa qsc_chacha_poly1305_encrypt.

Return: None (void).

qsc_chacha_transform_checked

Purpose: Transform ChaCha20 bytes with counter-exhaustion reporting.

bool qsc_chacha_transform_checked(qsc_chacha_state* ctx, uint8_t* output, const uint8_t* input, size_t length);

Parameters: ctx: [qsc_chacha_state*] Initialized cipher state.; output: [uint8_t*] Writable output, at least length bytes.; input: [const uint8_t*] Readable input, at least length bytes; may equal output.; length: [size_t] Number of bytes to transform..

Return: [bool] True if all requested bytes were processed; false on invalid parameters or counter exhaustion.

Notes: A failure leaves input/output and the cipher state unmodified. Changing the size of qsc_chacha_state requires rebuilding dependent binaries.

Header: csx.h

qsc_csx_extended_transform

Purpose: A multi-call transform for a large array of bytes, such as required by file encryption. This call can be used to transform and authenticate a very large array of bytes (+1GB). On the last call in the sequence, set the finalize parameter to true to complete authentication, and write the MAC code to the end of the output array in encryption mode, or compare to the embedded MAC code and authenticate in decryption mode. In encryption mode, the input plain-text is encrypted, then authenticated, and the MAC code is appended to the cipher-text. In decryption mode, the input cipher-text is authenticated internally and compared to the MAC code appended to the cipher-text, If the final tag does not match, the current output is unchanged and the state is invalidated. Earlier calls with finalize=false release UNAUTHENTICATED plaintext. The caller must quarantine all such output until the final call succeeds, and discard it on failure. Both peers must use identical nonzero chunk lengths and associated-data call sequences. Each call consumes whole keystream blocks and authenticates its starting nonce; changing chunk boundaries is not equivalent to a single transform. The same overlap and counter limits as qsc_csx_transform apply. A tag-sized extension is required on encryption output or decryption input only when finalize=true.

bool qsc_csx_extended_transform(qsc_csx_state* ctx, uint8_t* output, const uint8_t* input, size_t length, bool finalize);

Parameters: ctx: [struct] The cipher state structure; output: [uint8_t*] A pointer to the output array; input: [const] A pointer to the input array; length: [size_t] The number of bytes to transform; finalize: [bool] Complete authentication on a stream if set to true.

Return: : [bool] Returns true if the cipher has been transformed the data successfully, false on failure

Notes: The cipher must be initialized before this function can be called

qsc_csx_store_nonce

Purpose: Retrieves the current nonce from the state

void qsc_csx_store_nonce(const qsc_csx_state* ctx, uint8_t nonce[QSC_CSX_NONCE_SIZE]);

Parameters: ctx: [struct] The cipher state structure; nonce: [uint8_t*] The output nonce array.

Return: None (void).

Notes: Retrieve the next nonce only after a successful finalized transform and before disposal. Reinitializing with the stored nonce starts a new authentication chain; both peers must reinitialize.

Header: rcs.h

qsc_rcs_extended_transform

Purpose: A multi-call transform for a large array of bytes. This function can be used to transform and authenticate a very large array of bytes (e.g., >1GB). On the last call in the sequence, set the finalize parameter to true to complete authentication. In encryption mode, the plain-text is encrypted and the MAC code is appended. With finalize=true, a failed tag leaves the current output unchanged and disposes the context. Calls with finalize=false release UNAUTHENTICATED plaintext. Quarantine all such output until final authentication succeeds, and discard it on failure. Both peers must use identical chunk lengths and associated-data call sequences. Each call authenticates its starting nonce and consumes whole keystream blocks; changing chunk boundaries is not equivalent to a single transform. An empty final call is supported. The overlap and counter limits of qsc_rcs_transform apply. Tag space is required only when finalize=true. State objects must not be shared concurrently between threads.

bool qsc_rcs_extended_transform(qsc_rcs_state* ctx, uint8_t* output, const uint8_t* input, size_t length, bool finalize);

Parameters: ctx: [qsc_rcs_state*] A pointer to the cipher state structure.; output: [uint8_t*] A pointer to the output array.; input: [const uint8_t*] A pointer to the input array.; length: [size_t] The number of bytes to transform.; finalize: [bool] A flag to indicate if this is the final call..

Return: [bool] Returns true if the data was transformed successfully, false on failure.

qsc_rcs_store_nonce

Purpose: Retrieves the current nonce from the state

```
void qsc_rcs_store_nonce(const qsc_rcs_state* ctx, uint8_t nonce[QSC_RCS_NONCE_SIZE]);
```

Parameters: ctx: [struct] The cipher state structure; nonce: [uint8_t*] The output nonce array.

Return: None (void).

Notes: Retrieve the next nonce after successful finalization and before disposal. Reinitializing with that nonce starts a new authentication chain; both peers must reinitialize.

6: HASH and MAC Functions

6.1 SHA3

Header:

sha3.h

Description:

The SHA3 cryptographic message digest, the NIST standard in secure hash functions and supporting protocols, is based on the Keccak family of cryptographic primitives. The SHA3 hash comes in three variants; SHA3-128, SHA3-256, and SHA3-512. The 128-bit variant outputs 16 bytes of hash code, the 256-bit variant outputs a 32-byte hash code, while the 512-bit variant produces 64 bytes of code.

Structures:

The qsc_keccak_state structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
state	Uint64 Array	1600	Hash state
buffer	Uint8 Array	1600	Message buffer
position	Uint64	64	Buffer position

Table 6.1a SHA3 state structure

Constants:

Constant Name	Value	Purpose
QSC_KECCAK_CSHAKE_DOMAIN_ID	0x04	The cSHAKE domain id
QSC_KECCAK_KMAC_DOMAIN_ID	0x04	KMAC domain-separation byte.
QSC_KECCAK_KPA_DOMAIN_ID	0x41	The KPA domain id.
QSC_KECCAK_PERMUTATION_ROUNDS	24	The standard number of permutation rounds.
QSC_KECCAK_PERMUTATION_MAX_ROUNDS	48	The maximum number of permutation rounds.
QSC_KECCAK_PERMUTATION_MIN_ROUNDS	12	The minimum number of permutation rounds.
QSC_KECCAK_SHA3_DOMAIN_ID	6	The SHA3 domain id.
QSC_KECCAK_SHAKE_DOMAIN_ID	0x1F	The SHAKE domain id.
QSC_KECCAK_STATE_BYTE_SIZE	200	The Keccak state array byte size.
QSC_KECCAK_128_RATE	168	The KMAC-128 byte absorption rate.
QSC_KECCAK_256_RATE	136	The KMAC-256 byte absorption rate.
QSC_KECCAK_512_RATE	72	The KMAC-512 byte absorption rate.
QSC_KECCAK_STATE_SIZE	25	The Keccak SHA3 uint64 state array size.
QSC_KMAC_256_KEY_SIZE	32	The KMAC-256 key size in bytes.
QSC_KMAC_512_KEY_SIZE	64	The KMAC-512 key size in bytes.
QSC_SHA3_128_HASH_SIZE	16	The SHA3-128 hash size in bytes.

QSC_SHA3_256_HASH_SIZE	32	The SHA-256 hash size in bytes.
QSC_SHA3_512_HASH_SIZE	64	The SHA-512 hash size in bytes.
QSC_SHAKE_256_KEY_SIZE	32	The SHAKE-256 key size in bytes.
QSC_SHAKE512_KEY_SIZE	64	The SHAKE-512 key size in bytes.

Table 6.1b SHA3 constants

Call Hierarchy

The long form of the SHA3 function API, first requires a call to the initialize function to ready the SHA3 state, then calls to the update function to add message data to the hash. The finalize call writes the message hash to the output array, and the dispose function sets the state to zero.

initialize(*state*)

update(*state*, *message*, *length*)

finalize(*state*, *output*)

dispose(*state*)

API:

Compute

Compute the hash for a message with a single function call. Takes the message, and hashes it, and returns the hash code to the output array. Takes pointers to the output and message arrays, and the message length as parameters.

```
void qsc_sha3_compute128(uint8_t* output, const uint8_t* message, size_t msglen)
```

```
void qsc_sha3_compute256(uint8_t* output, const uint8_t* message, size_t msglen)
```

```
void qsc_sha3_compute512(uint8_t* output, const uint8_t* message, size_t msglen)
```

Dispose

Clear the Keccak state structure, setting all variables to zero. Takes a pointer to the Keccak state as a parameter.

```
void qsc_keccak_dispose(qsc_keccak_state* ctx)
```

Finalize

Process the SHA3 state and return the output hash. Takes a pointer to the Keccak state, the permutation rate, the domain separator, and a pointer to the output array as parameters.

```
void qsc_keccak_finalize(qsc_keccak_state* ctx, qsc_keccak_rate rate, uint8_t* output, size_t outlen, uint8_t domain, size_t rounds)
```

Initialize

Prepares the SHA3 state structure, must be called before any other call. Takes the Keccak state as a parameter.

```
void qsc_sha3_initialize(qsc_keccak_state* ctx)
```

Update

Update the SHA3 state with message data. Takes a pointer to the state, the permutation rate, a pointer to the message array, and the message length as parameters.

```
void qsc_sha3_update(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* message, size_t msglen)
```

Permute

The SHA3 permutation functions are exposed, for use as a function in other protocols. Takes a pointer to the Keccak state, and a round count as parameters.

```
void qsc_keccak_permute(qsc_keccak_state* ctx, size_t rounds)
```

```
void qsc_keccak_permute_p1600c(uint64_t* state, size_t rounds)
```

void qsc_keccak_permute_p1600u(uint64_t* state)

Keccak

Keccak free functions used in the hash, XOF and MAC functions, have been exposed, so that they can be used in more complex constructions.

API:

void qsc_keccak_absorb(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* message, size_t msglen, uint8_t domain, size_t rounds)

void qsc_keccak_absorb_custom(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* custom, size_t custlen, const uint8_t* name, size_t namelen, size_t rounds)

void qsc_keccak_absorb_key_custom(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* key, size_t keylen, const uint8_t* custom, size_t custlen, const uint8_t* name, size_t namelen, size_t rounds)

void qsc_keccak_finalize(qsc_keccak_state* ctx, qsc_keccak_rate rate, uint8_t* output, size_t outlen, uint8_t domain, size_t rounds)

void qsc_keccak_incremental_absorb(qsc_keccak_state* ctx, uint32_t rate, const uint8_t* message, size_t msglen)

void qsc_keccak_incremental_finalize(qsc_keccak_state* ctx, uint32_t rate, uint8_t domain)

void qsc_keccak_incremental_squeeze(qsc_keccak_state* ctx, size_t rate, uint8_t* output, size_t outlen)

void qsc_keccak_permute(qsc_keccak_state* ctx, size_t rounds)

void qsc_keccak_permute_p1600c(uint64_t* state, size_t rounds)

void qsc_keccak_squeezeblocks(qsc_keccak_state* ctx, uint8_t* output, size_t nblocks, qsc_keccak_rate rate, size_t rounds)

void qsc_keccak_initialize_state(qsc_keccak_state* ctx)

void qsc_keccak_update(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* message, size_t msglen, size_t rounds)

KPA

The Keccak Parallel Authentication domain identifier is retained as an internal Keccak-domain value. The current public QSC headers do not expose a qsc_kpa_state type or exported qsc_kpa_* API family.

API:

References:

- SHA3 Fips202: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
- SP800-185: http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf
- SHA3 Keccak Submission http://keccak.noekeon.org/Keccak-submission-3.pdf
- Keccak Reference Guide: https://keccak.team/files/Keccak-reference-3.0.pdf

6.2 SHA2

Header:

sha2.h

Description:

The SHA2 cryptographic message digest, is the NIST standard in secure hash functions and supporting protocols. The SHA2 hash comes in two variants; SHA2-256, and SHA2-512. The 256-bit variant outputs a 32-byte hash code, while the 512-bit variant produces 64 bytes of code.

Structures:

The qsc_sha256_state structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
state	Uint32 Array	256	Hash state
buffer	Uint8 Array	512	Message buffer
t	Uint64	64	Message length
position	Uint64	64	Buffer position

Table 6.2a SHA2-256 state structure

The qsc_sha512_state structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
state	Uint64 Array	512	Hash state
buffer	Uint8 Array	512	Message buffer
t	Uint64 Array	128	Message length
position	Uint64	64	Buffer position

Table 6.2b SHA2-512 state structure

Constants:

Constant Name	Value	Purpose
QSC_SHA2_SHANI_ENABLED	N/A	Enables the SHA2 permutation intrinsics.
QSC_HKDF_256_KEY_SIZE	32	The HKDF-256 key size in bytes.
QSC_HKDF_512_KEY_SIZE	64	The HKDF-512 key size in bytes.
QSC_HMAC_256_KEY_SIZE	32	The HMAC(SHA2-256) key size.
QSC_HMAC_512_KEY_SIZE	64	The HMAC(SHA2-512) key size.
QSC_HMAC_256_MAC_SIZE	32	The HMAC-256 mac-code size in bytes.
QSC_HMAC_512_MAC_SIZE	64	The HMAC-512 mac-code size in bytes.

QSC_HMAC_256_RATE	64	The HMAC-256 input rate size in bytes.
QSC_HMAC_512_RATE	128	The HMAC-512 input rate size in bytes.
QSC_SHA2_256_HASH_SIZE	32	The SHA2-256 hash size in bytes.
QSC_SHA2_384_HASH_SIZE	48	The SHA2-384 hash size in bytes.
QSC_SHA2_512_HASH_SIZE	64	The SHA2-512 hash size in bytes.
QSC_SHA2_256_RATE	64	The SHA-256 byte absorption rate.
QSC_SHA2_384_RATE	128	The SHA-384 byte absorption rate.
QSC_SHA2_512_RATE	128	The SHA2-512 byte absorption rate.
QSC_SHA2_STATE_SIZE	0x08	The SHA2-256 state array size.

Table 6.2c SHA2 constants

Call Hierarchy

The long form of the SHA2 function API, first requires a call to the initialize function to ready the SHA2 state, then calls to the update function to add message data to the hash. The finalize call writes the message hash to the output array, and the dispose function sets the state to zero.

initialize(*state*)

update(*state*, *message*, *length*)

finalize(*state*, *output*)

dispose(*state*)

API:

Compute

Compute the hash for a message with a single function call. Takes the message, and hashes it, and returns the hash code to the output array. This function takes pointers to the output and message arrays, and the message length as parameters.

void qsc_sha256_compute(uint8_t* output, const uint8_t* message, size_t msglen)

void qsc_sha384_compute(uint8_t* output, const uint8_t* message, size_t msglen)

void qsc_sha512_compute(uint8_t* output, const uint8_t* message, size_t msglen)

Dispose

Clear the SHA2 state structure, setting all variables to zero. Takes a pointer to the SHA2 state as a parameter.

void qsc_sha256_dispose(qsc_sha256_state* ctx)

void qsc_sha384_dispose(qsc_sha384_state* ctx)

void qsc_sha512_dispose(qsc_sha512_state* ctx)

Finalize

Process the SHA2 state and return the output hash. Takes a pointer to the SHA2 state, and a pointer to the output array as parameters.

void qsc_sha256_finalize(qsc_sha256_state* ctx, uint8_t* output)

void qsc_sha384_finalize(qsc_sha384_state* ctx, uint8_t* output)

void qsc_sha512_finalize(qsc_sha512_state* ctx, uint8_t* output)

Initialize

Prepares the SHA2 state structure, must be called before any other call. Takes a pointer to the SHA2 state as a parameter.

void qsc_sha256_initialize(qsc_sha256_state* ctx)

void qsc_sha384_initialize(qsc_sha384_state* ctx)

void qsc_sha512_initialize(qsc_sha512_state* ctx)

Update

Update the SHA2 state with message data. Takes a pointer to the SHA2 state, a pointer to the message array, and the message length as parameters.

```
void qsc_sha256_update(qsc_sha256_state* ctx, const uint8_t* message, size_t msglen)
```

```
void qsc_sha384_update(qsc_sha384_state* ctx, const uint8_t* message, size_t msglen)
```

```
void qsc_sha512_update(qsc_sha512_state* ctx, const uint8_t* message, size_t msglen)
```

Permute

The SHA2-256 and SHA2-512 permutation functions are exposed, for use as a function in other protocols. Takes a pointer to the output and the input arrays as parameters.

```
void qsc_sha256_permute(uint32_t* output, const uint8_t* input)
```

```
void qsc_sha512_permute(uint64_t* output, const uint8_t* input)
```

References:

- SHA3 Fips202: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
- SHA3 Keccak Submission: http://keccak.noekeon.org/Keccak-submission-3.pdf

6.3 KMAC

Header:

sha3.h

Description:

The Keccak Message Authentication Code generator (KMAC), is a keyed hash function, used to authenticate a message. It is part of the SHA3 family of functions.

Structures:

The KMAC **qsc_keccak_state** structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
state	Uint64 Array	1600	Hash state
buffer	Uint8 Array	1600	Message buffer
position	Uint64	64	Buffer position

Table 6.3 Keccak state structure

Call Hierarchy:

The long form of the KMAC function API, first requires a call to the initialize function to add the keying material and update the state, then calls to the update function to add message data to the hash. The finalize call writes the message hash to the output array, and the dispose function sets the state to zero.

initialize(*state*, *rate*, *key*, *custom*)

update(*state*, *rate*, *message*, *length*)

finalize(*state*, *rate*, *output*)

dispose(*state*)

API:

Compute

The compute function adds pointers to the key, customization string, and message, and generates the MAC code with a single function call. Takes pointers to the output, custom, and message arrays, and their lengths as parameters.

```
void qsc_kmac128_compute(uint8_t* output, size_t outlen, const uint8_t* message, size_t msglen, const uint8_t* key, size_t keylen, const uint8_t* custom, size_t custlen)
```

```
void qsc_kmac256_compute(uint8_t* output, size_t outlen, const uint8_t* message, size_t msglen, const uint8_t* key, size_t keylen, const uint8_t* custom, size_t custlen)
```

```
void qsc_kmac512_compute(uint8_t* output, size_t outlen, const uint8_t* message, size_t msglen, const uint8_t* key, size_t keylen, const uint8_t* custom, size_t custlen)
```

Dispose

The dispose function resets the MAC functions state to zero. Takes a pointer to the Keccak state as a parameter.

void qsc_keccak_dispose(qsc_keccak_state* ctx)

Finalize

The finalize function processes the state, and outputs the MAC code. The function takes a pointer to the state, the permutation rate, the output array pointer, and output length as parameters.

void qsc_kmac_finalize(qsc_keccak_state* ctx, qsc_keccak_rate rate, uint8_t* output, size_t outlen)

Initialize

The initialize function initializes the state and adds the key, permutation rate, and customization string. The function takes a pointer the state, the permutation rate, pointers to the key and custom arrays, and their lengths as parameters.

void qsc_kmac_initialize(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* key, size_t keylen, const uint8_t* custom, size_t custlen)

Update

The update function adds message data to the MAC function. The function takes a pointer the state, the permutation rate, a pointer to the output array, and the output length as parameters.

void qsc_kmac_finalize(qsc_keccak_state* ctx, qsc_keccak_rate rate, uint8_t* output, size_t outlen)

Parallel KMAC

The parallel forms of KMAC process multiple message streams concurrently using SIMD instructions. Takes pointers to multiple output, key, custom and message arrays, and their sizes, as parameters.

```
void qsc_kmac_128x4(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, size_t
outlen,
        const uint8_t* key0, const uint8_t* key1, const uint8_t* key2, const uint8_t* key3,
        size_t keylen, const uint8_t* cst0, const uint8_t* cst1, const uint8_t* cst2, const
        uint8_t* cst3, size_t cstlen, const uint8_t* msg0, const uint8_t* msg1, const
        uint8_t* msg2, const uint8_t* msg3, size_t msglen)
void qsc_kmac_256x4(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, size_t
outlen,
        const uint8_t* key0, const uint8_t* key1, const uint8_t* key2, const uint8_t* key3,
        size_t keylen, const uint8_t* cst0, const uint8_t* cst1, const uint8_t* cst2, const
        uint8_t* cst3, size_t cstlen, const uint8_t* msg0, const uint8_t* msg1, const
        uint8_t* msg2, const uint8_t* msg3, size_t msglen)
void qsc_kmac_512x4(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, size_t
outlen,
        const uint8_t* key0, const uint8_t* key1, const uint8_t* key2, const uint8_t* key3,
        size_t keylen, const uint8_t* cst0, const uint8_t* cst1, const uint8_t* cst2, const
        uint8_t* cst3, size_t cstlen, const uint8_t* msg0, const uint8_t* msg1, const
        uint8_t* msg2, const uint8_t* msg3, size_t msglen)
void qsc_kmac_128x8(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3,
        uint8_t* out4, uint8_t* out5, uint8_t* out6, uint8_t* out7, size_t outlen,
        const uint8_t* key0, const uint8_t* key1, const uint8_t* key2, const uint8_t* key3,
        const uint8_t* key4, const uint8_t* key5, const uint8_t* key6, const uint8_t* key7,
        size_t keylen, const uint8_t* cst0, const uint8_t* cst1, const uint8_t* cst2, const
        uint8_t* cst3, const uint8_t* cst4, const uint8_t* cst5, const uint8_t* cst6, const
        uint8_t* cst7, size_t cstlen, const uint8_t* msg0, const uint8_t* msg1, const
        uint8_t* msg2, const uint8_t* msg3, const uint8_t* msg4, const uint8_t* msg5,
        const uint8_t* msg6, const uint8_t* msg7, size_t msglen)
void qsc_kmac_256x8(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3,
        uint8_t* out4, uint8_t* out5, uint8_t* out6, uint8_t* out7, size_t outlen,
        const uint8_t* key0, const uint8_t* key1, const uint8_t* key2, const uint8_t* key3,
        const uint8_t* key4, const uint8_t* key5, const uint8_t* key6, const uint8_t* key7,
        size_t keylen, const uint8_t* cst0, const uint8_t* cst1, const uint8_t* cst2, const
        uint8_t* cst3, const uint8_t* cst4, const uint8_t* cst5, const uint8_t* cst6, const
        uint8_t* cst7, size_t cstlen, const uint8_t* msg0, const uint8_t* msg1, const
        uint8_t* msg2, const uint8_t* msg3, const uint8_t* msg4, const uint8_t* msg5,
        const uint8_t* msg6, const uint8_t* msg7, size_t msglen)
void qsc_kmac_512x8(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3,
        uint8_t* out4, uint8_t* out5, uint8_t* out6, uint8_t* out7, size_t outlen,
        const uint8_t* key0, const uint8_t* key1, const uint8_t* key2, const uint8_t* key3,
```

```
const uint8_t* key4, const uint8_t* key5, const uint8_t* key6, const uint8_t* key7,
size_t keylen, const uint8_t* cst0, const uint8_t* cst1, const uint8_t* cst2, const
uint8_t* cst3, const uint8_t* cst4, const uint8_t* cst5, const uint8_t* cst6, const
uint8_t* cst7, size_t cstlen, const uint8_t* msg0, const uint8_t* msg1, const
uint8_t* msg2, const uint8_t* msg3, const uint8_t* msg4, const uint8_t* msg5,
const uint8_t* msg6, const uint8_t* msg7, size_t msglen)
```

References:

- NIST FIPS-202: http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
- SP800-185: http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf

6.4 HMAC

Header:

sha2.h

Description:

The Hash-based Message Authentication Code generator (HMAC), is a keyed hash function, used to authenticate a message. It uses the SHA2 family of hash functions, SHA2-256, SHA2-384, and SHA2-512.

Structures:

The **qsc_hmac256_state** structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
pstate	SHA2-256 State	996	Hash state
ipad	Uint8 Array	512	Input buffer
opad	Uint8 Array	512	Output buffer

Table 6.4a HMAC-256 state structure

The qsc_hmac512_state structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
pstate	SHA2-512 State	1216	Hash state
ipad	Uint8 Array	1024	Input buffer
opad	Uint8 Array	1024	Output buffer

Table 6.4b HMAC-512 state structure

Call Hierarchy

The long form of the HMAC function API, first requires a call to the initialize function to add the keying material and initialize the state, then calls to the update function to add message data to the hash. The finalize call writes the message hash to the output array, and the dispose function sets the state to zero.

initialize(*state*, *key*)

update(*state*, *message*, *length*)

finalize(*state*, *output*)

dispose(*state*)

API:

Compute

The compute function adds the key and message, and generates the MAC code with a single function call. Takes pointers to the output, message, and key arrays, and their lengths as parameters.

```
void qsc_hmac256_compute(uint8_t* output, const uint8_t* message, size_t msglen, const uint8_t* key, size_t keylen)
```

```
void qsc_hmac384_compute(uint8_t* output, const uint8_t* message, size_t msglen, const uint8_t* key, size_t keylen)
```

```
void qsc_hmac512_compute(uint8_t* output, const uint8_t* message, size_t msglen, const uint8_t* key, size_t keylen)
```

Dispose

The dispose function resets the MAC functions state to zero. Takes a pointer to the HMAC state as a parameter.

```
void qsc_hmac256_dispose(qsc_hmac256_state* ctx)
```

```
void qsc_hmac384_dispose(qsc_hmac384_state* ctx)
```

```
void qsc_hmac512_dispose(qsc_hmac512_state* ctx)
```

Finalize

The finalize function finalizes the state, and outputs the MAC code. Takes pointers to the HMAC state, and the output array as parameters.

```
void qsc_hmac256_finalize(qsc_hmac256_state* ctx, uint8_t* output)
```

```
void qsc_hmac384_finalize(qsc_hmac384_state* ctx, uint8_t* output)
```

```
void qsc_hmac512_finalize(qsc_hmac512_state* ctx, uint8_t* output)
```

Initialize

The initialize function initializes the state and adds the key. Takes pointers to the HMAC state, the key array, and key length as parameters.

```
void qsc_hmac256_initialize(qsc_hmac256_state* ctx, const uint8_t* key, size_t keylen)
```

```
void qsc_hmac384_initialize(qsc_hmac384_state* ctx, const uint8_t* key, size_t keylen)
```

```
void qsc_hmac512_initialize(qsc_hmac512_state* ctx, const uint8_t* key, size_t keylen)
```

Update

The update function adds message data to the MAC function. Takes pointers to the HMAC state, the message array, and message length as parameters.

```
void qsc_hmac256_update(qsc_hmac256_state* ctx, const uint8_t* message, size_t
msglen)
```

```
void qsc_hmac384_update(qsc_hmac384_state* ctx, const uint8_t* message, size_t msglen)
```

void qsc_hmac512_update(qsc_hmac512_state* ctx, const uint8_t* message, size_t msglen)

References:

- RFC 2104, HMAC: http://tools.ietf.org/html/rfc2104
- FIPS 198-1: http://csrc.nist.gov/publications/fips/fips198-1/FIPS-198-1_final.pdf

6.5 QMAC

Header:

qmac.h

Description:

The Quantum-Safe Message Authentication Code generator (QMAC), is a keyed message authentication function used to authenticate a message. It derives hash and finalization subkeys using Keccak, and uses 256-bit carryless multiplication and reduction over GF(2^256) to process the message state.

Structures:

The qsc_qmac_state structure contains the internal QMAC state.

Data Name	Data Type	Bit Length	Function
F	Uint64 Array	256	Finalization key
H	Uint64 Array	256	Hash Key
Y	Uint64 Array	256	Intermediate State
initialized	Boolean	8	Initialization status

Table 6.5a QMAC state structure

The qsc_qmac_keyparams structure contains the QMAC key, optional nonce, optional information string, and their lengths.

Data Name	Data Type	Bit Length	Function
key	Uint8 Array	256	The secret key
keylen	Uint64 Integer	64	The key length
nonce	Uint8 Array Pointer	64	The nonce pointer
noncelen	Uint64 Integer	64	The nonce length
info	Uint8 Array Pointer	64	The info pointer
infolen	Uint64 Integer	64	The info length

Table 6.5b QMAC keyparams structure

QMAC does not expose a public mode enumeration in the current API.

Constant Name	Purpose
QSC_QMAC_BLOCK_SIZE	The natural block size of the message input in bytes.
QSC_QMAC_KEY_SIZE	The QMAC key size in bytes.
QSC_QMAC_MAC_SIZE	The QMAC MAC code size in bytes.
QSC_QMAC_NONCE_SIZE	The QMAC nonce size in bytes.
QSC_QMAC_STATE_SIZE	The QMAC state size in 64-bit words.

Table 6.5c QMAC constants

Call Hierarchy

The long form of the QMAC function API first calls initialize to add the keying material and optional parameters, then calls update to absorb message data. The finalize call writes the MAC code to the output array and disposes of the internal state.

initialize(*state*, *key*)

update(*state*, *message*, *length*)

finalize(*state*, *output*)

dispose(*state*)

API:

Compute

The compute function adds the key parameters and message, and generates the MAC code with a single function call. Takes pointers to the output, key parameters, message, and message length as parameters.

```
void qsc_qmac_compute(uint8_t* output, qsc_qmac_keyparams* keyparams, const uint8_t* message, size_t msglen)
```

Dispose

The dispose function resets the MAC functions state to zero. Takes a pointer to the QMAC state as a parameter.

```
void qsc_qmac_dispose(qsc_qmac_state* ctx)
```

Finalize

The finalize function finalizes the state, and outputs the MAC code. Takes pointers to the QMAC state, and the output array as parameters.

```
void qsc_qmac_finalize(qsc_qmac_state* ctx, uint8_t* output)
```

Initialize

The initialize function initializes the state and adds the key. Takes pointers to the QMAC state, the keyparams structure as parameters.

```
void qsc_qmac_initialize(qsc_qmac_state* ctx, qsc_qmac_keyparams* keyparams)
```

Update

The update function adds message data to the MAC function. Takes pointers to the QMAC state, the message array, and message length as parameters.

```
void qsc_qmac_update(qsc_qmac_state* ctx, const uint8_t* message, size_t msglen)
```

6.6 Additional Hash and MAC APIs

Header: pbmac1.h

qsc_pbmac1_compute

Purpose: Process a message with PBMAC1 and return the MAC code.

bool qsc_pbmac1_compute(uint8_t* output, const qsc_pbmac1_keyparams* keyparams, const uint8_t* message, size_t msglen);

Parameters: output: [uint8_t*] The output byte array; receives the MAC code.; keyparams: [const qsc_pbmac1_keyparams*] Pointer to the PBMAC1 key parameters.; message: [const uint8_t*] The message input byte array.; msglen: [size_t] The number of message bytes to process..

Return: [bool] Returns true on success.

Notes: The output array must be at least qsc_pbmac1_mac_size(keyparams->hash) bytes in length.

qsc_pbmac1_derive_key

Purpose: Derive a PBMAC1 MAC key with PBKDF2.

bool qsc_pbmac1_derive_key(uint8_t* output, size_t outlen, const qsc_pbmac1_keyparams* keyparams);

Parameters: output: [uint8_t*] The output byte array; receives the derived key.; outlen: [size_t] The requested derived key length in bytes.; keyparams: [const qsc_pbmac1_keyparams*] Pointer to the PBMAC1 key parameters..

Return: [bool] Returns true on success.

qsc_pbmac1_dispose

Purpose: Dispose of a PBMAC1 state.

void qsc_pbmac1_dispose(qsc_pbmac1_state* ctx);

Parameters: ctx: [qsc_pbmac1_state*] Pointer to the PBMAC1 state structure..

Return: None (void).

qsc_pbmac1_finalize

Purpose: Finalize the PBMAC1 state and return the MAC code.

bool qsc_pbmac1_finalize(qsc_pbmac1_state* ctx, uint8_t* output);

Parameters: ctx: [qsc_pbmac1_state*] Pointer to the PBMAC1 state structure.; output: [uint8_t*] The output byte array; receives the MAC code..

Return: [bool] Returns true on success.

Notes: Finalize calls dispose on the PBMAC1 state.

qsc_pbmac1_initialize

Purpose: Initialize a PBMAC1 state.

bool qsc_pbmac1_initialize(qsc_pbmac1_state* ctx, const qsc_pbmac1_keyparams* keyparams);

Parameters: ctx: [qsc_pbmac1_state*] Pointer to the PBMAC1 state structure.; keyparams: [const qsc_pbmac1_keyparams*] Pointer to the PBMAC1 key parameters..

Return: [bool] Returns true on success.

qsc_pbmac1_mac_size

Purpose: Return the MAC output size for a PBMAC1 hash selection.

size_t qsc_pbmac1_mac_size(qsc_pbmac1_hash_type hash);

Parameters: hash: [qsc_pbmac1_hash_type] The digest and HMAC selection..

Return: [size_t] Returns the MAC size in bytes, or zero if the hash is unsupported.

qsc_pbmac1_update

Purpose: Update PBMAC1 with message input.

bool qsc_pbmac1_update(qsc_pbmac1_state* ctx, const uint8_t* message, size_t msglen);

Parameters: ctx: [qsc_pbmac1_state*] Pointer to the PBMAC1 state structure.; message: [const uint8_t*] The input message byte array.; msglen: [size_t] The number of message bytes to process..

Return: [bool] Returns true on success.

qsc_pbmac1_verify

Purpose: Verify a PBMAC1 code using a constant-time comparison.

bool qsc_pbmac1_verify(const uint8_t* code, size_t codelen, const qsc_pbmac1_keyparams* keyparams, const uint8_t* message, size_t msglen);

Parameters: code: [const uint8_t*] The expected MAC code.; codelen: [size_t] The expected MAC code length in bytes.; keyparams: [const qsc_pbmac1_keyparams*] Pointer to the PBMAC1 key parameters.; message: [const uint8_t*] The message input byte array.; msglen: [size_t] The number of message bytes to process..

Return: [bool] Returns true if the MAC code is valid.

Header: poly1305.h

qsc_poly1305_blockupdate

Purpose: Update the poly1305 generator with a single block of message input. Absorbs exactly 16 bytes in stream order, including any pending partial block.

void qsc_poly1305_blockupdate(qsc_poly1305_state* ctx, const uint8_t* message);

Parameters: ctx: [struct] The function ctx; must be initialized; message: [const] The input message byte array.

Return: None (void).

Notes: Message length must be a single 16 byte message block. \n

qsc_poly1305_compute

Purpose: Compute the MAC code and return the result in the mac byte array.

void qsc_poly1305_compute(uint8_t* output, const uint8_t* message, size_t msglen, const uint8_t* key);

Parameters: output: The output byte array; receives the MAC code; message: [const] Message input; may be NULL when msglen is zero.; msglen: Number of message bytes to process; key: [const] The 32 byte key array.

Return: None (void).

Notes: The output array must be at least 16 bytes in length.

qsc_poly1305_dispose

Purpose: Dispose of the ctx resetting all values to zero.

void qsc_poly1305_dispose(qsc_poly1305_state* ctx);

Parameters: ctx: [struct] The function ctx.

Return: None (void).

qsc_poly1305_finalize

Purpose: Finalize the message ctx and return the MAC code. Absorb the last block, write the MAC and securely reset the state. \n

void qsc_poly1305_finalize(qsc_poly1305_state* ctx, uint8_t* mac);

Parameters: ctx: [struct] The function ctx; must be initialized; mac: The MAC byte array; receives the MAC code.

Return: None (void).

qsc_poly1305_initialize

Purpose: Initialize the ctx with the secret key. A key must be used for only one message. Key storage must not overlap ctx. Buffered data is erased; a NULL key erases the previous state in release builds.

void qsc_poly1305_initialize(qsc_poly1305_state* ctx, const uint8_t key[QSC_POLY1305_KEY_SIZE]);

Parameters: ctx: [struct] The function ctx; key: [const] The secret key byte array.

Return: None (void).

qsc_poly1305_reset

Purpose: Erase the accumulator, key, and buffered data, invalidating the context.

void qsc_poly1305_reset(qsc_poly1305_state* ctx);

Parameters: ctx: The function ctx.

Return: None (void).

qsc_poly1305_update

Purpose: Update the poly1305 generator with a length of message input. Absorbs the input message into the ctx.

void qsc_poly1305_update(qsc_poly1305_state* ctx, const uint8_t* message, size_t msglen);

Parameters: ctx: [struct] The function ctx; must be initialized; message: [const] Input message; may be NULL when msglen is zero.; msglen: Number of input bytes; zero is a no-op..

Return: None (void).

qsc_poly1305_verify

Purpose: Verify a MAC code. Compare the supplied code with the computed tag using QSC constant-time verification.

int32_t qsc_poly1305_verify(const uint8_t* code, const uint8_t* message, size_t msglen, const uint8_t* key);

Parameters: code: [const] The MAC code byte array; message: [const] Message bytes; may be NULL when msglen is zero.; msglen: Number of message bytes to process; key: [const] The secret key byte array.

Return: Zero on tag match; nonzero on mismatch or invalid parameters.

Header: sha3.h

qsc_kmac_update

Purpose: The KMAC message update function. Long form api: must be used in conjunction with the initialize and finalize functions.

void qsc_kmac_update(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* message, size_t msglen);

Parameters: ctx: [qsc_keccak_state*] A reference to the Keccak state; must be initialized; rate: [qsc_keccak_rate] The rate of absorption in bytes; message: [const uint8_t*] The message input byte array; msglen: [size_t] The number of message bytes to process.

Return: None (void).

Notes: The state must be initialized before calling.

qsc_kmac_xof128_compute

Purpose: Key a KMAC-128 XOF instance and generate an output key. Short form api: processes the keyin, x, and s inputs and generates the output key with a single call. Key the XOF generator process a message and output the key.

void qsc_kmac_xof128_compute(uint8_t* kout, size_t koutlen, const uint8_t* x, size_t xlen, const uint8_t* kin, size_t kinlen, const uint8_t* s, size_t slen);

Parameters: kout: [uint8_t*] The pointer to output key byte array; koutlen: [size_t] The number of key bytes to generate; x: [const uint8_t*] The context string containing the information related to the derived keying material.; xlen: [size_t] The number of context string bytes to process; kin: [const uint8_t*] The input key byte array; kinlen: [size_t] The number of key bytes to process; s: [const uint8_t*] The optional label customization string; slen: [size_t] The byte length of the customization string.

Return: None (void).

qsc_kmac_xof256_compute

Purpose: Key a KMAC-256 XOF instance and generate an output key. Short form api: processes the keyin, x, and s inputs and generates the output key with a single call. Key the XOF generator process a message and output the key.

void qsc_kmac_xof256_compute(uint8_t* kout, size_t koutlen, const uint8_t* x, size_t xlen, const uint8_t* kin, size_t kinlen, const uint8_t* s, size_t slen);

Parameters: kout: [uint8_t*] The pointer to output key byte array; koutlen: [size_t] The number of key bytes to generate; x: [const uint8_t*] The context string containing the information related to the derived keying material.; xlen: [size_t] The number of context string bytes to process; kin: [const uint8_t*] The input key byte array; kinlen: [size_t] The number of key bytes to process; s: [const uint8_t*] The optional label customization string; slen: [size_t] The byte length of the customization string.

Return: None (void).

qsc_kmac_xof512_compute

Purpose: Key a KMAC-512 XOF instance and generate an output key. The KMAC XOF as specified in SP800-108r1. Short form api: processes the keyin, x, and s inputs and generates the output key with a single call. Key the XOF generator process a message and output the key.

void qsc_kmac_xof512_compute(uint8_t* kout, size_t koutlen, const uint8_t* x, size_t xlen, const uint8_t* kin, size_t kinlen, const uint8_t* s, size_t slen);

Parameters: kout: [uint8_t*] The pointer to output key byte array; koutlen: [size_t] The number of key bytes to generate; x: [const uint8_t*] The context string containing the information related to the derived keying material.; xlen: [size_t] The number of context string bytes to process; kin: [const uint8_t*] The input key byte array; kinlen: [size_t] The number of key bytes to process; s: [const uint8_t*] The optional label customization string; slen: [size_t] The byte length of the customization string.

Return: None (void).

qsc_sha3_finalize

Purpose: Finalize the message state and returns the hash value in output. Long form api: must be used in conjunction with the initialize and block-update functions. Absorb the last block of message and create the hash value. Produces a 32 byte output code using QSC_KECCAK_256_RATE, 64 bytes with QSC_KECCAK_512_RATE.

void qsc_sha3_finalize(qsc_keccak_state* ctx, qsc_keccak_rate rate, uint8_t* output);

Parameters: ctx: [qsc_keccak_state*] A reference to the Keccak state; must be initialized; rate: [qsc_keccak_rate] The rate of absorption in bytes; output: [uint8_t*] The output byte array; receives the hash code.

Return: None (void).

Notes: The output array must be sized correctly corresponding to the absorption rate ((200 - rate) / 2). Finalizes the message state, can not be used in consecutive calls. The state must be initialized before calling.

7: Random Providers

7.1 ACP

Header:

acp.h

Description:

The auto entropy provider uses a hashed collection of system timers, statistics,

the RDRAND provider, and the system random provider, to seed an instance of cSHAKE-512.

Constants:

Constant Name	Value	Purpose
QSC_ACP_SEED_MAX	10240000	The maximum seed size that can be extracted from a single generate call.

Table 7.1a ACP constants

API:

Generate

Fills an output array with random bytes. Takes a pointer to the output array, and the output length as parameters.

```
bool qsc_acp_generate(uint8_t* output, size_t length)
```

ACP Uint16

Returns a random 16-bit integer.

uint16_t qsc_acp_uint16(void);

ACP Uint32

Returns a random 32-bit integer.

uint32_t qsc_acp_uint32(void);

ACP Uint64

Returns a random 64-bit integer.

uint64_t qsc_acp_uint64(void);

7.2 CSP

Header:

csp.h

Description:

The cryptographic systems entropy provider uses entropy provided by the operating systems random provider.

Constants:

Constant Name	Value	Purpose
QSC_CSP_SEED_MAX	1024000	The maximum seed size that can be extracted from a single generate call.

Table 7.2a CSP constants

API:

Generate

Fills an output array with random bytes. Takes a pointer to the output array, and the output length as parameters.

```
bool qsc_csp_generate(uint8_t* output, size_t length)
```

CSP Uint16

Returns a random 16-bit integer.

```
uint16_t qsc_csp_uint16(void);
```

CSP Uint32

Returns a random 32-bit integer.

```
uint32_t qsc_csp_uint32(void);
```

CSP Uint64

Returns a random 64-bit integer.

```
uint64_t qsc_csp_uint64(void);
```

7.3 RDP

Header:

rdp.h

Description:

The RDRAND entropy provider uses random noise generated by the CPU as a source of random bytes.

Constants:

Constant Name	Value	Purpose
QSC_RDP_SEED_MAX	1024000	The maximum seed size that can be extracted from a single generate call.

Table 7.3a RDP constants

API:

Generate

Fills an output array with random bytes. Takes the pointer to the output array, and the output length as parameters.

```
bool qsc_rdp_generate(uint8_t* output, size_t length)
```

RDP Uint16

Returns a random 16-bit integer.

```
uint16_t qsc_rdp_uint16(void);
```

RDP Uint32

Returns a random 32-bit integer.

```
uint32_t qsc_rdp_uint32(void);
```

RDP Uint64

Returns a random 64-bit integer.

```
uint64_t qsc_rdp_uint64(void);
```

7.4 Entropy Providers and Extended Random APIs

ETR adds caller-supplied entropy collection with SHAKE-512 conditioning. Inability to obtain sufficient validated source entropy must be treated as failure; this is not a silent fallback to ACP.

Header: etr.h

qsc_etr_dispose

Purpose: Dispose of the registered external true-random source.

Details: Clears the registered callback and context pointer. The context remains owned by the caller and is not released by ETR. This function must not be called concurrently with qsc_etr_generate. \sa qsc_etr_initialize

void qsc_etr_dispose(void);

Return: None (void).

qsc_etr_generate

Purpose: Generate SHAKE-512 conditioned random bytes from the external TRNG.

Details: Collects at least one byte of raw source entropy for every output byte, with a minimum raw collection of 64 bytes. Source requests are bounded to a maximum of 1024 bytes per callback invocation. Each returned block is checked before it is absorbed into SHAKE-512. Conditioned output is produced only after all source requests and validation checks have succeeded. Concurrent generation calls are permitted only when the registered external callback and its context are thread-safe. Initialization and disposal must not occur concurrently with generation.

bool qsc_etr_generate(uint8_t* output, size_t length);

Parameters: output: [uint8_t*] Pointer to the output buffer that receives the conditioned bytes.; length: [size_t] The number of conditioned bytes to generate; must not exceed QSC_ETR_SEED_MAX..

Return: [bool] Returns true on success. Returns false if ETR is not initialized, the source fails, a partial read occurs, or a raw source validation check fails. On failure, a valid output buffer is cleared. \sa qsc_etr_initialize, qsc_etr_dispose, qsc_shake512_compute

qsc_etr_initialize

Purpose: Initialize ETR with an external true-random source.

Details: Registers the callback and caller-owned context used by qsc_etr_generate. The context pointer may be NULL when the callback does not require external state. A source cannot be replaced while ETR is initialized; call qsc_etr_dispose before registering a different source. This function registers the source but does not invoke it. Device initialization and source-specific startup health tests must be completed by the caller or callback before random output is requested.

bool qsc_etr_initialize(qsc_etr_source_callback source, void* context);

Parameters: source: [qsc_etr_source_callback] The external TRNG callback.; context: [void*] Pointer to the caller-owned external source context, or NULL..

Return: [bool] Returns true if the source was registered, or false if the callback is NULL or ETR is already initialized. \sa qsc_etr_generate, qsc_etr_dispose

Header: rdp.h

qsc_rdrand_available

Purpose: Test whether the platform exposes a usable high-resolution timer.

bool qsc_rdrand_available(void);

Return: [bool] Returns true when the RDRAND functionality is available.

8: DRBG, KDF, and PRNG

8.1 SHAKE

Header:

sha3.h

Description:

SHAKE is the Keccak extended output function (XOF), a secure way to generate pseudo-random output from an input key. It can be used as a key expansion function, a pseudo-random byte generator, a hash function, and as a PRNG.

cSHAKE is the customized from of SHAKE, it takes a key, and the additional customization and name parameters.

Structures:

The SHAKE **qsc_keccak_state** structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
state	Uint64 Array	1600	Hash state
buffer	Uint8 Array	1600	Message buffer
position	Uint64	64	Buffer position

Table 8.1a Keccak state structure

Call Hierarchy:

The long form of the SHAKE function API, first requires a call to the initialize function to add the keying material. The squeeze call writes the pseudo-random to the output array, and the dispose function sets the state to zero.

initialize(*state*, *rate*, *key*, *keylen*)

squeeze (*state*, *rate*, *output*, *nblocks*)

dispose(*state*)

API:

Compute

The compute function is the short form of the API, the key is added and the output is processed using a single function call. The SHAKE compute functions take a pointer to the output array, the number of bytes to write to that array, a pointer to the key array, and the keys byte length as parameters.

void qsc_shake128_compute(uint8_t* output, size_t outlen, const uint8_t* key, size_t keylen)

void qsc_shake256_compute(uint8_t* output, size_t outlen, const uint8_t* key, size_t keylen)

void qsc_shake512_compute(uint8_t* output, size_t outlen, const uint8_t* key, size_t keylen)

Initialize

The initialize function is part of the long-form of the API. It is used to initialize the Keccak state, and inject the key. The SHAKE initialize function takes a pointer to the Keccak state, the absorption rate, a pointer to the input key, and the key length as parameters.

void qsc_shake_initialize(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* key, size_t keylen)

Squeeze

The squeeze function outputs pseudo-random in blocks. The block size corresponds to the Keccak absorption/permutation rate. Initialize must be called first, and the output array must be sized to receive a rate-sized block of bytes. The squeeze function takes a pointer to the Keccak state, the absorption rate, a pointer to the output byte array, and the number of blocks to write as parameters.

void qsc_shake_squeezeblocks(qsc_keccak_state* ctx, qsc_keccak_rate rate, uint8_t* output, size_t nblocks)

Parallel SHAKE

The parallel SHAKE functions process multiple blocks of data concurrently using SIMD instructions. Using AVX2 instructions, 4 simultaneous SHAKE calculations can be made, the ‘x4’ API. With AVX-512 instructions, 8 simultaneous SHAKE calculations are made, using the ‘x8’ API.

SHAKE-X4

The SHAKE X4 functions take 4 output array pointers, and 4 input array pointers, and the input and output arrays lengths as parameters. The functions take multiple output and input array pointers, and the input and output array sizes as parameters.

```
void qsc_shake_128x4(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, size_t outlen, const uint8_t* inp0, const uint8_t* inp1, const uint8_t* inp2, const uint8_t* inp3, size_t inplen);
```

```
void qsc_shake_256x4(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, size_t outlen, const uint8_t* inp0, const uint8_t* inp1, const uint8_t* inp2, const uint8_t* inp3, size_t inplen);
```

```
void qsc_shake_512x4(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, size_t outlen, const uint8_t* inp0, const uint8_t* inp1, const uint8_t* inp2, const uint8_t* inp3, size_t inplen);
```

SHAKE-X8

The SHAKE X8 functions take 8 output array pointers, and 8 input array pointers, and the input and output arrays lengths as parameters.

```
void qsc_shake_128x8(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, uint8_t* out4, uint8_t* out5, uint8_t* out6, uint8_t* out7, size_t outlen, const uint8_t* inp0, const uint8_t* inp1, const uint8_t* inp2, const uint8_t* inp3, const uint8_t* inp4, const uint8_t* inp5, const uint8_t* inp6, const uint8_t* inp7, size_t inplen);
```

```
void qsc_shake_256x8(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, uint8_t* out4, uint8_t* out5, uint8_t* out6, uint8_t* out7, size_t outlen, const uint8_t* inp0, const uint8_t* inp1, const uint8_t* inp2, const uint8_t* inp3, const uint8_t* inp4, const uint8_t* inp5, const uint8_t* inp6, const uint8_t* inp7, size_t inplen);
```

```
void qsc_shake_512x8(uint8_t* out0, uint8_t* out1, uint8_t* out2, uint8_t* out3, uint8_t* out4, uint8_t* out5, uint8_t* out6, uint8_t* out7, size_t outlen, const uint8_t* inp0, const uint8_t* inp1, const uint8_t* inp2, const uint8_t* inp3, const uint8_t* inp4, const uint8_t* inp5, const uint8_t* inp6, const uint8_t* inp7, size_t inplen);
```

8.2 cSHAKE

Compute

The compute function is the short form of the API, the key, custom and name arrays are added and the output is processed using a single function call. The cSHAKE compute functions take as parameters; a pointer to the output array, the number of bytes to write to that array, a pointer to the key array, and the keys byte length, pointers to the name and custom arrays, and their lengths.

```
void qsc_cshake128_compute(uint8_t* output, size_t outlen, const uint8_t* key, size_t keylen, const uint8_t* name, size_t namelen, const uint8_t* custom, size_t custlen)
```

```
void qsc_cshake256_compute(uint8_t* output, size_t outlen, const uint8_t* key, size_t keylen, const uint8_t* name, size_t namelen, const uint8_t* custom, size_t custlen)
```

```
void qsc_cshake512_compute(uint8_t* output, size_t outlen, const uint8_t* key, size_t keylen, const uint8_t* name, size_t namelen, const uint8_t* custom, size_t custlen)
```

Initialize

The initialize function is part of the long-form of the API. It is used to initialize the Keccak state, and inject the key, custom, and name arrays. The cSHAKE initialize function takes a pointer to the Keccak state, the absorption rate, a pointer to the input key array, and the key length as parameters.

```
void qsc_cshake_initialize(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* key,
size_t keylen, const uint8_t* name, size_t namelen, const uint8_t* custom, size_t custlen)
```

Squeeze

The squeeze function outputs pseudo-random in blocks. The block size corresponds to the Keccak rate. Initialize must be called first, and the output array must be sized to receive a rate-sized block of bytes. The squeeze function takes a pointer to the Keccak state, the absorption rate, a pointer to the output byte array, and the number of blocks to write as parameters.

```
void qsc_cshake_squeezeblocks(qsc_keccak_state* ctx, qsc_keccak_rate rate, uint8_t*
output, size_t nblocks)
```

Update

The update function can inject new keying or message material into the Keccak state. The update function takes the Keccak state, the absorption rate, a pointer to the key array, and the key length as parameters.

```
void qsc_cshake_update(qsc_keccak_state* ctx, qsc_keccak_rate rate, const uint8_t* key,
size_t keylen)
```

References:

- NIST SHA3 FIPS 202 http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
- SP800-185: http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf

8.3 HKDF

Header:

sha2.h

Description:

Hash-based Key Derivation Function, is a key stretching function, in this implementation it uses either the SHA2-256, or SHA2-512 hashes as the underlying pseudo-random function.

API:

Expand

The expand function is a key-expansion function. The expand function takes a pointer to the output array, the number of bytes to write to that array, a pointer to key and optional info arrays, and their lengths as parameters.

```
void qsc_hkdf256_expand(uint8_t* output, size_t otplen, const uint8_t* key, size_t keylen, const uint8_t* info, size_t infolen);
```

```
void qsc_hkdf384_expand(uint8_t* output, size_t otplen, const uint8_t* key, size_t keylen, const uint8_t* info, size_t infolen)
```

```
void qsc_hkdf512_expand(uint8_t* output, size_t otplen, const uint8_t* key, size_t keylen, const uint8_t* info, size_t infolen);
```

Extract

The extract function combines multiple inputs and extracts a pseudo-random output. The extract function takes a pointer to the output array, the output length, and pointers to a key and optional salt array, and their lengths as parameters.

```
void qsc_hkdf256_extract(uint8_t* output, size_t otplen, const uint8_t* key, size_t keylen, const uint8_t* salt, size_t saltlen);
```

```
void qsc_hkdf384_extract(uint8_t* output, size_t otplen, const uint8_t* key, size_t keylen, const uint8_t* salt, size_t saltlen)
```

```
void qsc_hkdf512_extract(uint8_t* output, size_t otplen, const uint8_t* key, size_t keylen, const uint8_t* salt, size_t saltlen);
```

References:

- RFC 2104: http://tools.ietf.org/html/rfc2104

- RFC 5869: http://tools.ietf.org/html/rfc5869

8.4 CSG

Header:

csg.h

Description:

The cSHAKE deterministic random bytes generator (CSG), is an auto re-keying DRBG, that can re-key with random seed material at specified thresholds, to provide forward secrecy and predictive resistance in a long-running DRBG instance.

Structures:

The qsc_csg_state structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
kstate	Keccak state	3264	Hash state
cache	Uint8 Array	1088	Cache buffer
bctr	Uint64	64	Buffer count
cpos	Uint64	64	Cache position
crmd	Uint64	64	Cache remainder
rate	Uint64	64	Keccak rate
pres	Bool	8	Auto rekey

Table 8.4a CSG state structure

Constants:

Constant Name	Value	Purpose
QSC_CSG_256_SEED_SIZE	32	The CSG-256 seed size.
QSC_CSG_512_SEED_SIZE	64	The CSG-512 seed size.

QSC_CSG_RESEED_THRESHHOLD	1024000	The auto re-seed threshold size, 1MB default.

Table 8.4b CSG constants

Call Hierarchy

The long form of the CSG function API, first requires a call to the initialize function to add the keying material and optional info array. The generate function populates the output array with pseudo-random. The update function can be used to add entropy to the state from an external source, and the dispose function sets the state to default values.

initialize(*state*, *seed*, *seedlen*, *infolen*, *pres*)

generate (*state*, *output*, *outlen*)

update(*state*, *seed*, *seedlen*)

dispose(*state*)

API:

Dispose

Resets the CSG state members to defaults. Takes a pointer to the CSG state as a parameter.

```
void qsc_csg_dispose(qsc_csg_state* ctx)
```

Initialize

Initializes the generator with seeding material. The initialize function takes a pointer to a seed array and the seed length, a pointer to an optional info parameter and length, and a predictive-resistance Boolean that enables automatic re-seeding.

```
void qsc_csg_initialize(qsc_csg_state* ctx, const uint8_t* seed, size_t seedlen, const uint8_t* info, size_t infolen, bool predres)
```

Generate

Generates an array of pseudo-random and writes it to the output byte array. Takes a pointer to the CSG state, a pointer to the output array, and the output length as parameters.

```
bool qsc_csg_generate(qsc_csg_state* ctx, uint8_t* output, size_t otplen);
```

Update

Updates the generator state with new keying material. The update function takes a pointer to the CSG state, a pointer to a seed array, and the seed length as parameters.

```
void qsc_csg_update(qsc_csg_state* ctx, const uint8_t* seed, size_t seedlen)
```

References:

- NIST SHA3 FIPS 202 http://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf
- NIST SP800-185: http://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf

8.5 HCG

Header:

hcg.h

Description:

HMAC based pseudo-random bytes generator (HCG), uses SHA2-512 internally as the pseudo random function. HCG is an auto re-keying DRBG, that can re-key with random seed material at specified thresholds, to provide forward secrecy and predictive resistance in a long-running DRBG instance.

Structures:

The qsc_hcg_state structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
cache	Uint8 Array	512	Cache buffer
info	Uint8 Array	448	Cache buffer
nonce	Uint8 Array	64	Nonce array
bctr	Uint64	64	Buffer count
cpos	Uint64	64	Cache position
crmd	Uint64	64	Cache remainder
pres	Bool	8	Auto rekey

Table 8.5a HCG structure

Constants:

Constant Name	Value	Purpose
QSC_HCG_MAX_INFO_SIZE	0x38	The maximum info parameter size.
QSC_HCG_NONCE_SIZE	0x08	The nonce array size.
QSC_HCG_RESEED_THRESHHOLD	65535	Automatic HCG reseed threshold.
QSC_HCG_SEED_SIZE	64	The seed array size.

Table 8.5b HCG constants

Call Hierarchy:

The long form of the HCG function API, first requires a call to the initialize function to add the keying material and optional info array. The generate function populates the output array with pseudo-random. The update function can be used to add additional externally provided entropy to the state, and the dispose function sets the state to default values.

initialize(*state*, *seed*, *seedlen*, *infolen*, *pres*)

generate (*state*, *output*, *outlen*)

update(*state*, *seed*, *seedlen*)

dispose(*state*)

API:

Dispose

Resets the HCG state members to defaults. Takes the HCG state as a parameter.

```
void qsc_hcg_dispose(qsc_hcg_state* ctx)
```

Initialize

Initializes the generator with seeding material. The initialize function takes a pointer to a seed array and the seed length, a pointer to an optional info parameter and length, and a predictive-resistance Boolean that enables automatic re-seeding.

```
void qsc_hcg_initialize(qsc_hcg_state* ctx, const uint8_t* seed, size_t seedlen, const uint8_t* info, size_t infolen, bool pres);
```

Generate

Generates an array of pseudo-random and writes it to the output. Takes a pointer to the HCG state, a pointer to the output array, and the output length as parameters.

```
bool qsc_hcg_generate(qsc_hcg_state* ctx, uint8_t* output, size_t otplen);
```

Update

Updates the generator state with new keying material. The update function takes a pointer to the HCG state, a pointer to a seed array, and the seed length as parameters.

```
void qsc_hcg_update(qsc_hcg_state* ctx, const uint8_t* seed, size_t seedlen)
```

References:

- RFC 2104, HMAC: http://tools.ietf.org/html/rfc2104
- FIPS 198-1: http://csrc.nist.gov/publications/fips/fips198-1/FIPS-198-1_final.pdf

8.6 SCB

Header:

scb.h

Description:

SCB is a cost based key derivation function (KDF) that uses the Keccak cSHAKE XOF function, to produce pseudo-random bytes from a seeded custom SHAKE generator. If a 32-byte key is used, the implementation uses the cSHAKE-256 implementation for pseudo-random generation, if a 64-byte key is used, the generator uses cSHAKE-512. The CPU cost feature is an iteration count in the cost mechanism, it determines the number of times both the state absorption and memory scattering functions execute. The Memory cost, is the maximum number of mebibytes (MiB) the internal cache uses, during execution of the memory cost mechanism. The L2 cache mechanism can be changed to a doubling of 128 KiB (128, 256, 512, 1024 KiB), doubling the cache sizes above the default 256 KiB, also means that the minimum memory cost must be doubled, L2=512 Kib, minimum memcost is 2, 1024 KiB minimum memcost is 4 etc. The generator can be updated with new seed material, which is absorbed into the Keccak state.

Structures:

The qsc_scb_state structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
key	Uint8 Array	512	Internal key
cpuc	Uint64	448	CPU cost
memc	Uint64	64	Memory cost
klen	Uint64	64	Key length
rate	enum	64	Keccak rate

Table 8.6a SCB state structure

Constants:

Constant Name	Value	Purpose

QSC_SCB_256_SEED_SIZE	32	The 256-bit key size.
QSC_SCB_512_SEED_SIZE	64	The 512-bit key size.
QSC_SCB_L2CACHE_DEFAULT_SIZE	262144	The default cache size
QSC_SCB_MEMORY_COST_SIZE	1048576	The default memory cost size.
QSC_SCB_MEMORY_MAXIMUM	128	The maximum memory size.
QSC_SCB_MEMORY_MINIMUM	1	The minimum memory size.
QSC_SCB_CPU_MINIMUM	1	The minimum CPU cost.
QSC_SCB_CPU_MAXIMUM	1000	The maximum CPU cost.

Table 8.6b SCB constants

API:

Dispose

Resets the SCB state members to defaults. Takes the SCB state as a parameter.

```
void qsc_scb_dispose(qsc_scb_state* ctx)
```

Initialize

Initializes the generator with seeding material. The initialize function takes a pointer to a seed array and the seed length, a pointer to an optional info parameter and length, and a predictive-resistance Boolean that enables automatic re-seeding.

```
void qsc_scb_initialize(qsc_scb_state* ctx, const uint8_t* seed, size_t seedlen, const uint8_t* info, size_t infolen, size_t cpucost, size_t memcost)
```

Generate

Generates an array of pseudo-random and writes it to the output. Takes a pointer to the SCB state, a pointer to the output array, and the output length as parameters.

```
bool qsc_scb_generate(qsc_scb_state* ctx, uint8_t* output, size_t otplen)
```

Update

Updates the generator state with new keying material. The update function takes a pointer to the SCB state, a pointer to a seed array, and the seed length as parameters.

void qsc_scb_update(qsc_scb_state* ctx, const uint8_t* seed, size_t seedlen)

8.7 SecRand

Header:

secrand.h

Description:

Secure number generator, that uses the CSG DRBG as the random generator. Provides access to the range of signed and unsigned integers, from floats and doubles to 8-bit and 64-bit integers. Integer sizes can be scoped to either a maximum value, or maximum and minimum values.

Structures:

The **qsc_secrand_state** structure contains the internal function state.

Data Name	Data Type	Bit Length	Function
hstate	qsc_csg_state	Variable	The CSG DRBG state
cache	Uint8 Array	0x400	The cache buffer
cpos	Uint64	0x40	The cache position
init	Boolean	0x08	The initialized flag

Table 8.7a secrand state structure

Constants:

Constant Name	Value	Purpose
QSC_SECRAND_SEED_SIZE	0x20	The expected seed size used in the initialize function
QSC_SECRAND_CACHE_SIZE	0x400	The size of the internal cache

Table 8.7b secrand constants

API:

Initialize

Initialize the random generator with a seed and optional customization array.

```
void qsc_secrand_initialize(const uint8_t* seed, size_t seedlen, const uint8_t* custom, size_t custlen)
```

Generate

Generate an array of pseudo-random bytes.

```
bool qsc_secrand_generate(uint8_t* output, size_t length)
```

Next Char

Generate a signed 8-bit random integer.

```
int8_t qsc_secrand_next_char(void);
```

Next Double

Generate a random double integer.

```
double qsc_secrand_next_double(void);
```

Next Int16

Generate a signed 16-bit random integer.

```
int16_t qsc_secrand_next_int16(void);
```

Next Int16 Max

Generate a signed 16-bit random integer of a maximum value.

int16_t qsc_secrand_next_int16_max(int16_t maximum)

Next Int16 Max Min

Generate a signed 16-bit random integer of a maximum and minimum value.

int16_t qsc_secrand_next_int16_maxmin(int16_t maximum, int16_t minimum)

Next Uint16

Generate an unsigned 16-bit random integer.

uint16_t qsc_secrand_next_uint16(void);

Next Uint16 Max

Generate an unsigned 16-bit random integer of a maximum value.

uint16_t qsc_secrand_next_uint16_max(uint16_t maximum)

Next Uint16 Max Min

Generate an unsigned 16-bit random integer of a maximum and minimum value.

uint16_t qsc_secrand_next_uint16_maxmin(uint16_t maximum, uint16_t minimum)

Next Int32

Generate a signed 32-bit random integer.

int32_t qsc_secrand_next_int32(void);

Next Int32 Max

Generate a signed 32-bit random integer of a maximum value.

int32_t qsc_secrand_next_int32_max(int32_t maximum)

Next Int32 Max Min

Generate a signed 32-bit random integer of a maximum and minimum value.

int32_t qsc_secrand_next_int32_maxmin(int32_t maximum, int32_t minimum)

Next Uint32

Generate an unsigned 32-bit random integer.

uint32_t qsc_secrand_next_uint32(void);

Next Uint32 Max

Generate an unsigned 32-bit random integer of a maximum value.

uint32_t qsc_secrand_next_uint32_max(uint32_t maximum)

Next Uint32 Max Min

Generate an unsigned 32-bit random integer of a maximum and minimum value.

uint32_t qsc_secrand_next_uint32_maxmin(uint32_t maximum, uint32_t minimum)

Next Int64

Generate a signed 64-bit random integer.

int64_t qsc_secrand_next_int64(void);

Next Int64 Max

Generate a signed 64-bit random integer of a maximum value.

int64_t qsc_secrand_next_int64_max(int64_t maximum)

Next Int64 Max Min

Generate a signed 64-bit random integer of a maximum and minimum value.

int64_t qsc_secrand_next_int64_maxmin(int64_t maximum, int64_t minimum)

Next Uint64

Generate an unsigned 64-bit random integer.

uint64_t qsc_secrand_next_uint64(void);

Next Uint64 Max

Generate an unsigned 64-bit random integer of a maximum value.

uint64_t qsc_secrand_next_uint64_max(uint64_t maximum)

Next Uint64 Max Min

Generate an unsigned 64-bit random integer of a maximum and minimum value.

uint64_t qsc_secrand_next_uint64_maxmin(uint64_t maximum, uint64_t minimum)

8.7 Additional DRBG, KDF, and PRNG APIs

Header: secrand.h

qsc_secrand_dispose

Purpose: Clear the buffer and destroy the internal state.

```
void qsc_secrand_dispose(void);
```

Return: None (void).

qsc_secrand_next_uchar

Purpose: Generate an unsigned 8-bit random integer.

```
uint8_t qsc_secrand_next_uchar(void);
```

Return: [uint8_t] Returns an unsigned 8-bit random integer.

9: Asymmetric Ciphers

9.1 ECDH

Header:

ecdh.h

Description:

The Elliptic Curve Diffie Hellman asymmetric cipher. In a Diffie Hellman key exchange, parties generate key-pairs, and exchange their public keys. A shared secret is derived by both parties combing the received public key with their private keys.

Constants:

Constant Name	Value	Purpose
QSC_ECDH_PRIVATEKEY_SIZE	32 / 48 / 66	P-256 / P-384 / P-521 raw private-key sizes, in bytes.
QSC_ECDH_PUBLICKEY_SIZE	64 / 96 / 132	P-256 / P-384 / P-521 raw public-key sizes, in bytes.
QSC_ECDH_SHAREDSECRET_SIZE	32 / 48 / 66	P-256 / P-384 / P-521 shared-secret sizes, in bytes.
QSC_ECDH_SEED_SIZE	32 / 48 / 66	P-256 / P-384 / P-521 seed sizes, in bytes.

Table 9.1a ECDH constants

API:

Key Exchange

Hosts exchange public keys, then combine them with their private keys to extract a shared secret. Takes a pointer to the secret output array, the private key, and public key, as parameters, and returns true if the shared-secret was extracted successfully, or false if the key exchange fails.

```
bool qsc_ecdh_key_exchange(uint8_t* secret, const uint8_t* privatekey, const uint8_t* publickey)
```

Generate

Generates a public/private key-pair used in the derivation of a shared secret. The generate function takes pointers to the output public key, the private key, and a pointer to the RNG function as parameters.

```
bool qsc_ecdh_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

bool qsc_ecdh_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);
```

References:

NaCl by Daniel J. Bernstein, Tanja Lange, Peter Schwabe https://nacl.cr.yp.to

9.2 Kyber

Header:

kyber.h

Description:

The NIST FIPS 203 specification of Kyber. Kyber encapsulates a shared secret with the public key, which is extracted from the cipher-text using the private key.

Constants:

Constant Name	Value	Purpose
QSC_KYBER_CIPHERTEXT_SIZE	variable	The byte size of the cipher-text array.
QSC_KYBER_PRIVATEKEY_SIZE	variable	The byte size of the secret private-key array.
QSC_KYBER_PUBLICKEY_SIZE	variable	The byte size of the public-key array.
QSC_KYBER_SEED_SIZE	32	The byte size of the seed array.
QSC_KYBER_SHAREDSECRET_SIZE	32	The byte size of the shared secret-key array.

Table 9.2a Kyber constants

Parameter Sets:

Constant Name	Value	Purpose
QSC_KYBER_S3K3P768	N/A	The Kyber S3Q3329N256K3 parameter set.

QSC_KYBER_S5K4P1024	N/A	The Kyber S5Q3329N256K4 parameter set.
QSC_KYBER_S6K5P1280	N/A	The Kyber S6Q3329N256K5 parameter set.

Table 9.2b Kyber parameter sets

API:

Decapsulate

Decapsulates the shared secret from the cipher-text. The decapsulate function takes pointers to the shared-secret array, and the cipher-text array, and a pointer to the private key as parameters, and returns true if the cipher-text was decapsulated successfully, or false if the key exchange fails.

```
bool qsc_kyber_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey)
```

Encapsulate

Encapsulates a shared secret in cipher-text, using the public key. The encapsulate function takes a pointer to the shared-secret array, the cipher-text array, and the public key, and a pointer to an RNG function.

```
bool qsc_kyber_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));
```

Generate

The generate function creates a public and private key-pair. It takes pointers to the public and private key arrays, and a pointer to an RNG function.

```
bool qsc_kyber_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));
```

References:

- Kyber a CCA-secure module-lattice-based KEM: https://eprint.iacr.org/2017/634.pdf
- Kyber, a simple, provably secure key exchange: http://eprint.iacr.org/2012/688.pdf

9.3 McEliece

Description:

The NIST Post Quantum Competition round 3 asymmetric cipher finalist Classic McEliece. Encapsulates a shared secret with the public key, which is extracted from the cipher-text using the private key.

Constants:

Constant Name	Value	Purpose
QSC_MCELIECE_CIPHERTEXT_SIZE	variable	The byte size of the cipher-text array.
QSC_MCELIECE_PRIVATEKEY_SIZE	variable	The byte size of the secret private-key array.
QSC_MCELIECE_PUBLICKEY_SIZE	variable	The byte size of the public-key array.
QSC_MCELIECE_SEED_SIZE	32	The byte size of the seed array.
QSC_MCELIECE_SHAREDSECRET_SIZE	32	The byte size of the shared secret-key array.

Table 9.3a McEliece constants

Parameter Sets:

Constant Name	Value	Purpose
QSC_MCELIECE_S3N4608T96	N/A	The McEliece S3-N4608T96 parameter set.
QSC_MCELIECE_S5N6688T128	N/A	The McEliece S5-N6688T128 parameter set.

QSC_MCELIECE_S6N6960T119	N/A	The McEliece S5-N6960T119 parameter set.
QSC_MCELIECE_S7N8192T128	N/A	The McEliece S5-N8192T128 parameter set.

Table 9.3b McEliece parameter sets

API:

Decapsulate

Decapsulates the shared secret from the cipher-text. The decapsulate function takes a pointer to the shared-secret array, and the cipher-text array, and a pointer to the private key as parameters, and returns true if the cipher-text was decapsulated successfully, or false if the key exchange fails.

```
bool qsc_mceliece_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey)
```

Encapsulate

Encapsulates a shared secret in cipher-text using the public key. The encapsulate function takes a pointer to the shared-secret array, the cipher-text array, and the public key, and a pointer to an RNG function as parameters.

```
bool qsc_mceliece_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));
```

Generate

The generate function creates a public and private key-pair. It takes pointers to the public and private key arrays, and a pointer to an RNG function.

```
bool qsc_mceliece_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));
```

References:

- Classic McEliece: https://classic.mceliece.org/nist/mceliece-20171129.pdf

9.5 HQC

Header:

hqc.h

Description:

The Hamming Quasi-Cyclic (HQC) key encapsulation mechanism is a code-based post-quantum asymmetric cipher. HQC encapsulates a shared secret with the public key; the secret is then extracted from the cipher-text using the private key. The implementation supports the public parameter sets selected by the QSC build configuration and includes AVX2-optimized paths where available.

Constants:

Constant Name	Value	Purpose
QSC_HQC_CIPHERTEXT_SIZE	variable	The byte size of the cipher-text array.
QSC_HQC_PRIVATEKEY_SIZE	variable	The byte size of the secret private-key array.
QSC_HQC_PUBLICKEY_SIZE	variable	The byte size of the public-key array.
QSC_HQC_SEED_SIZE	32	The byte size of the seed array.
QSC_HQC_SHAREDSECRET_SIZE	32	The byte size of the shared secret-key array.

Table 9.5a HQC constants

Parameter Sets:

Constant Name	Value	Purpose
QSC_HQC_S1N2321	N/A	The HQC NIST security category 1 parameter set (n=2321).
QSC_HQC_S3N4602	N/A	The HQC NIST security category 3 parameter set (n=4602).

QSC_HQC_S5N7333	N/A	The HQC NIST security category 5 parameter set (n=7333).

Table 9.5b HQC parameter sets

API:

Decapsulate

Decapsulates the shared secret from the cipher-text. Takes pointers to the shared-secret output array, the cipher-text array, and the private key as parameters. Returns true if decapsulation succeeds.

bool qsc_hqc_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey)

Encapsulate

Encapsulates a shared secret in cipher-text using the public key. Takes pointers to the shared-secret output array, the cipher-text output array, the public key, and a pointer to the RNG function as parameters. Returns true on success.

bool qsc_hqc_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t))

Generate

Generates a public and private key-pair. Takes pointers to the public key output array, the private key output array, and a pointer to the RNG function as parameters. Returns true on success.

bool qsc_hqc_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t))

References:

HQC Project: https://pqc-hqc.org/

NIST Post-Quantum Cryptography Round 4 Submissions: https://csrc.nist.gov/Projects/post-quantum-cryptography/round-4-submissions

9.6 EDDH

Header:

eddh.h

Description:

The Edwards Curve Diffie-Hellman (EDDH) key exchange uses the Curve25519 and Curve448 elliptic curves to derive a shared secret. Both parties generate key-pairs and exchange public keys; each party then combines the received public key with their own private key to derive an identical shared secret. The implementation has been designed for constant-time execution to mitigate side-channel attacks.

Constants:

Constant Name	Value	Purpose
QSC_EDDH_PRIVATEKEY_SIZE	32 or 56	The byte size of the private-key array (curve-dependent).
QSC_EDDH_PUBLICKEY_SIZE	32 or 56	The byte size of the public-key array (curve-dependent).
QSC_EDDH_SHAREDSECRET_SIZE	32 or 56	The byte size of the shared secret array (curve-dependent).
QSC_EDDH_SEED_SIZE	32 or 56	The byte size of the seed array (curve-dependent).

Table 9.6a EDDH constants

Parameter Sets:

Constant Name	Value	Purpose
QSC_EDDH_S1EC25519	N/A	The Curve25519 (128-bit security) parameter set; keys and secret are 32 bytes.
QSC_EDDH_S3EC448	N/A	The Curve448 (224-bit security) parameter set; keys and secret are 56 bytes.

Table 9.6b EDDH parameter sets

API:

Key Exchange

Derives the shared secret from the local private key and a remote public key. Takes pointers to the shared secret output array, the local private key, and the remote public key. Returns true if the shared secret was derived successfully.

bool qsc_eddh_key_exchange(uint8_t* secret, const uint8_t* privatekey, const uint8_t* publickey)

Generate

Generates a public and private key-pair. Takes pointers to the public key output, the private key output, and either an RNG function pointer or a seed array.

bool qsc_eddh_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t))

bool qsc_eddh_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed)

References:

Ed25519 Specification: https://ed25519.cr.yp.to/ed25519-20110926.pdf

Curve25519 ECDH: https://cr.yp.to/ecdh.html

9.6 Additional KEM and Key Agreement APIs

Dedicated ECDH P-256/P-384/P-521, X25519/X448, HQC, Classic McEliece and parameter-specific ML-KEM entry points extend the generic selection facades. Kyber-1280 is experimental and not a FIPS 203 ML-KEM parameter set.

Header: ecdh.h

qsc_ecdh_public_from_private

Purpose: Derives a public key from an existing private key. The private key is a big-endian scalar of QSC_ECDH_PRIVATEKEY_SIZE bytes. The public key is an uncompressed raw Qx || Qy point without a SEC1 tag. This function is intended for use when importing or reconstructing keys from external representations such as PKCS#8 or application-defined storage.

void qsc_ecdh_public_from_private(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: [uint8_t*] Pointer to the output public-key array.; privatekey: [const uint8_t*] Pointer to the input private-key array..

Return: None (void).

Notes: Arrays must be sized to QSC_ECDH_PUBLICKEY_SIZE and QSC_ECDH_PRIVATEKEY_SIZE.

Header: ecdhp256.h

qsc_ecdhp256_generate_keypair

Purpose: Generate a key pair from a fresh callback-supplied seed.

bool qsc_ecdhp256_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Caller-owned 64-byte X||Y output, cleared on failure.; privatekey: Caller-owned 32-byte raw scalar output, erased on failure.; rng_generate: Callback writing exactly its requested byte count and returning true only when all bytes are cryptographically random. Invoked once for 32 bytes; borrowed buffers must not be retained. Callback state must be thread-safe when shared. Blinding separately uses the QSC CSP and may independently fail..

Return: True on success; false on null inputs, callback failure or derivation failure.

qsc_ecdhp256_generate_seeded_keypair

Purpose: Generate a key pair from a caller-supplied seed.

bool qsc_ecdhp256_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: Caller-owned 64-byte X||Y output, cleared on failure.; privatekey: Caller-owned 32-byte raw scalar output, erased on failure.; seed: Read-only 32-byte seed; must not overlap either output..

Return: True on success; false on null inputs, zero derived scalar or CSP failure.

qsc_ecdhp256_key_exchange

Purpose: Validate the peer and derive a nonzero raw ECDH agreement value.

bool qsc_ecdhp256_key_exchange(uint8_t* secret, const uint8_t* publickey, const uint8_t* privatekey);

Parameters: secret: Caller-owned 32-byte shared output, cleared on failure.; publickey: Read-only 64-byte peer X||Y.; privatekey: Read-only 32-byte canonical scalar..

Return: True on success; false on invalid inputs, CSP failure or all-zero output.

Notes: This curve API takes the public key before the private key, unlike qsc_ecdh_key_exchange.

qsc_ecdhp256_publickey_from_privatekey

Purpose: Derive the public point of a raw private scalar.

int32_t qsc_ecdhp256_publickey_from_privatekey(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: Caller-owned 64-byte X||Y output, cleared on failure.; privatekey: Read-only 32-byte big-endian scalar in [1,n)..

Return: Zero on success; -1 for null input, -2 for zero scalar, -3 for an out-of-range scalar, -9 for unavailable blinding entropy; other negatives fail.

qsc_ecdhp256_scalar_multiply

Purpose: Multiply a validated peer point by a raw private scalar.

int32_t qsc_ecdhp256_scalar_multiply(uint8_t* secret, const uint8_t* privatekey, const uint8_t* publickey);

Parameters: secret: Caller-owned 32-byte X-coordinate output, cleared on failure.; privatekey: Read-only 32-byte big-endian scalar in [1,n).; publickey: Read-only 64-byte canonical peer X||Y point..

Return: Zero on success; -1 for null input, -2 for zero scalar, -3 for scalar range, -4 for coordinate range, -5 for an off-curve point, -9 for entropy failure. Other negative values indicate arithmetic failure. The low-level result may be all zero; use key_exchange for the established all-zero rejection policy.

Header: ecdhp384.h

qsc_ecdhp384_generate_keypair

Purpose: Generate a key pair from a fresh callback-supplied seed.

bool qsc_ecdhp384_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Caller-owned 96-byte X||Y output, cleared on failure.; privatekey: Caller-owned 48-byte raw scalar output, erased on failure.; rng_generate: Callback writing exactly its requested byte count and returning true only when all bytes are cryptographically random. Invoked once for 48 bytes; borrowed buffers must not be retained. Callback state must be thread-safe when shared. Blinding separately uses the QSC CSP and may independently fail..

Return: True on success; false on null inputs, callback failure or derivation failure.

qsc_ecdhp384_generate_seeded_keypair

Purpose: Generate a key pair from a caller-supplied seed.

bool qsc_ecdhp384_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: Caller-owned 96-byte X||Y output, cleared on failure.; privatekey: Caller-owned 48-byte raw scalar output, erased on failure.; seed: Read-only 48-byte seed; must not overlap either output..

Return: True on success; false on null inputs, zero derived scalar or CSP failure.

qsc_ecdhp384_key_exchange

Purpose: Validate the peer and derive a nonzero raw ECDH agreement value.

bool qsc_ecdhp384_key_exchange(uint8_t* secret, const uint8_t* publickey, const uint8_t* privatekey);

Parameters: secret: Caller-owned 48-byte shared output, cleared on failure.; publickey: Read-only 96-byte peer X||Y.; privatekey: Read-only 48-byte canonical scalar..

Return: True on success; false on invalid inputs, CSP failure or all-zero output.

Notes: This curve API takes the public key before the private key, unlike qsc_ecdh_key_exchange.

qsc_ecdhp384_publickey_from_privatekey

Purpose: Derive the public point of a raw private scalar.

int32_t qsc_ecdhp384_publickey_from_privatekey(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: Caller-owned 96-byte X||Y output, cleared on failure.; privatekey: Read-only 48-byte big-endian scalar in [1,n)..

Return: Zero on success; -1 for null input, -2 for zero scalar, -3 for an out-of-range scalar, -9 for unavailable blinding entropy; other negatives fail.

qsc_ecdhp384_scalar_multiply

Purpose: Multiply a validated peer point by a raw private scalar.

int32_t qsc_ecdhp384_scalar_multiply(uint8_t* secret, const uint8_t* privatekey, const uint8_t* publickey);

Parameters: secret: Caller-owned 48-byte X-coordinate output, cleared on failure.; privatekey: Read-only 48-byte big-endian scalar in [1,n).; publickey: Read-only 96-byte canonical peer X||Y point..

Return: Zero on success; -1 for null input, -2 for zero scalar, -3 for scalar range, -4 for coordinate range, -5 for an off-curve point, -9 for entropy failure. Other negative values indicate arithmetic failure. The low-level result may be all zero; use key_exchange for the established all-zero rejection policy.

Header: ecdhp521.h

qsc_ecdhp521_generate_keypair

Purpose: Generate a key pair from a fresh callback-supplied seed.

bool qsc_ecdhp521_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Caller-owned 132-byte X||Y output, cleared on failure.; privatekey: Caller-owned 66-byte raw scalar output, erased on failure.; rng_generate: Callback writing exactly its requested byte count and returning true only when all bytes are cryptographically random. Invoked once for 66 bytes; borrowed buffers must not be retained. Callback state must be thread-safe when shared. Blinding separately uses the QSC CSP and may independently fail..

Return: True on success; false on null inputs, callback failure or derivation failure.

qsc_ecdhp521_generate_seeded_keypair

Purpose: Generate a key pair from a caller-supplied seed.

bool qsc_ecdhp521_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: Caller-owned 132-byte X||Y output, cleared on failure.; privatekey: Caller-owned 66-byte raw scalar output, erased on failure.; seed: Read-only 66-byte seed; must not overlap either output..

Return: True on success; false on null inputs, zero derived scalar or CSP failure.

qsc_ecdhp521_key_exchange

Purpose: Validate the peer and derive a nonzero raw ECDH agreement value.

bool qsc_ecdhp521_key_exchange(uint8_t* secret, const uint8_t* publickey, const uint8_t* privatekey);

Parameters: secret: Caller-owned 66-byte shared output, cleared on failure.; publickey: Read-only 132-byte peer X||Y.; privatekey: Read-only 66-byte canonical scalar..

Return: True on success; false on invalid inputs, CSP failure or all-zero output.

Notes: This curve API takes the public key before the private key, unlike qsc_ecdh_key_exchange.

qsc_ecdhp521_publickey_from_privatekey

Purpose: Derive the public point of a raw private scalar.

int32_t qsc_ecdhp521_publickey_from_privatekey(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: Caller-owned 132-byte X||Y output, cleared on failure.; privatekey: Read-only 66-byte big-endian scalar in [1,n)..

Return: Zero on success; -1 for null input, -2 for zero scalar, -3 for an out-of-range scalar, -9 for unavailable blinding entropy; other negatives fail.

qsc_ecdhp521_scalar_multiply

Purpose: Multiply a validated peer point by a raw private scalar.

int32_t qsc_ecdhp521_scalar_multiply(uint8_t* secret, const uint8_t* privatekey, const uint8_t* publickey);

Parameters: secret: Caller-owned 66-byte X-coordinate output, cleared on failure.; privatekey: Read-only 66-byte big-endian scalar in [1,n).; publickey: Read-only 132-byte canonical peer X||Y point..

Return: Zero on success; -1 for null input, -2 for zero scalar, -3 for scalar range, -4 for coordinate range, -5 for an off-curve point, -9 for entropy failure. Other negative values indicate arithmetic failure. The low-level result may be all zero; use key_exchange for the established all-zero rejection policy.

Header: eddh.h

qsc_eddh_public_from_private

Purpose: Derives the selected X25519 or X448 public key from an existing private key. The private key is interpreted as a raw scalar of QSC_EDDH_PRIVATEKEY_SIZE bytes. Scalar clamping is applied internally during public key derivation, as specified by RFC 7748. This function is intended for use when importing or reconstructing keys from external representations such as PKCS#8 or application-defined storage.

void qsc_eddh_public_from_private(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: [uint8_t*] Pointer to the output public-key array.; privatekey: [const uint8_t*] Pointer to the input private-key array..

Return: None (void).

Notes: Arrays must be sized to QSC_EDDH_PUBLICKEY_SIZE and QSC_EDDH_PRIVATEKEY_SIZE.

Header: eddh25519.h

qsc_x25519_generate_keypair

Purpose: Generate an X25519 key pair using a random callback.

Details: Null input or callback failure clears each supplied output and securely erases the private output. The API returns void; no key pair is produced on failure.

void qsc_x25519_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Caller-owned 32-byte public output; distinct from privatekey.; privatekey: Caller-owned 32-byte raw private output; clamping does not modify it.; rng_generate: Borrowed callback, invoked once for 32 bytes. Return true only after filling every requested byte with cryptographically secure randomness. Do not retain the buffer; synchronize callback state when shared between threads..

Return: None (void).

qsc_x25519_generate_seeded_keypair

Purpose: Generate the established deterministic X25519 key pair from a seed.

Details: No status return; null input clears each supplied output. Clamping applies only to an internal copy. Independent calls share no mutable state.

void qsc_x25519_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: Caller-owned 32-byte public output; must not overlap the other buffers.; privatekey: Caller-owned 32-byte raw private output; receives the unmodified seed.; seed: Read-only 32-byte seed. May equal privatekey; other overlap is unsupported..

Return: None (void).

qsc_x25519_key_exchange

Purpose: Compute an X25519 agreement value and reject all-zero output.

Details: Null inputs clear a supplied output. Rejected low-order peers produce zero. The entire output is checked without early exit. Apply a protocol KDF before use as a key; this function does not authenticate the peer. Independent calls are thread-safe.

bool qsc_x25519_key_exchange(uint8_t* secret, const uint8_t* publickey, const uint8_t* privatekey);

Parameters: secret: Caller-owned 32-byte shared output; may alias an input.; publickey: Read-only 32-byte peer u-coordinate.; privatekey: Read-only 32-byte raw private scalar; clamped internally..

Return: True for an accepted nonzero result; false for null inputs or a rejected result.

qsc_x25519_public_from_private

Purpose: Derive an X25519 public key from a raw private scalar.

Details: No status return; null inputs clear a supplied output. No allocation or mutable global state; calls with independent output buffers are thread-safe.

void qsc_x25519_public_from_private(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: Caller-owned 32-byte public output; may alias privatekey.; privatekey: Read-only 32-byte scalar; clamping uses an internal erased copy..

Return: None (void).

Header: eddh448.h

qsc_crypto_sc448_clamp

Purpose: Clamp an RFC 7748 X448 scalar in place.

Details: Clear the two low bits and set bit 447. This is the X448 scalar format, not the 57-byte Ed448 signing format.

void qsc_crypto_sc448_clamp(uint8_t* k);

Parameters: k: Caller-owned 56-byte writable scalar; null is ignored in release builds..

Return: None (void).

qsc_x448_generate_keypair

Purpose: Generate an X448 key pair using a random callback.

Details: Null input or callback failure clears each supplied output and securely erases the private output. The API returns void; no key pair is produced on failure.

void qsc_x448_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Caller-owned 56-byte public output; distinct from privatekey.; privatekey: Caller-owned 56-byte raw private output; clamping does not modify it.; rng_generate: Borrowed callback, invoked once for 56 bytes. Return true only after filling every requested byte with cryptographically secure randomness. Do not retain the buffer; synchronize callback state when shared between threads..

Return: None (void).

qsc_x448_generate_seeded_keypair

Purpose: Generate the established deterministic X448 key pair from a seed.

Details: No status return; null input clears each supplied output. Clamping applies only to an internal copy. Independent calls share no mutable state.

void qsc_x448_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: Caller-owned 56-byte public output; must not overlap the other buffers.; privatekey: Caller-owned 56-byte raw private output; receives the unmodified seed.; seed: Read-only 56-byte seed. May equal privatekey; other overlap is unsupported..

Return: None (void).

qsc_x448_key_exchange

Purpose: Compute an X448 agreement value and reject all-zero output.

Details: Null inputs clear a supplied output. Rejected low-order peers produce zero. The entire output is checked without early exit. Apply a protocol KDF before use as a key; this function does not authenticate the peer. Independent calls are thread-safe.

bool qsc_x448_key_exchange(uint8_t* secret, const uint8_t* publickey, const uint8_t* privatekey);

Parameters: secret: Caller-owned 56-byte shared output; may alias an input.; publickey: Read-only 56-byte peer u-coordinate.; privatekey: Read-only 56-byte raw private scalar; clamped internally..

Return: True for an accepted nonzero result; false for null inputs or a rejected result.

qsc_x448_public_from_private

Purpose: Derive an X448 public key from a raw private scalar.

Details: No status return; null inputs clear a supplied output. No allocation or mutable global state; calls with independent output buffers are thread-safe.

void qsc_x448_public_from_private(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: Caller-owned 56-byte public output; may alias privatekey.; privatekey: Read-only 56-byte scalar; clamping uses an internal erased copy..

Return: None (void).

Header: hqc.h

qsc_hqc_generate_seeded_keypair

Purpose: Generates a seed-based HQC-2026 keypair deterministically.

void qsc_hqc_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* seed);

Parameters: publickey: [uint8_t*] Pointer to output public key (an already allocated array of QSC_HQC_PUBLICKEY_SIZE bytes); privatekey: [uint8_t*] Pointer to output private key (an already allocated array of QSC_HQC_PRIVATEKEY_SIZE bytes); seed: [uint8_t*] Input random seed, QSC_HQC_SEED_SIZE bytes; must not alias the outputs..

Return: None (void).

Notes: Intended for deterministic testing. This does not claim NIST ACVP conformance.

Header: hqc_multi.h

qsc_hqc1_decapsulate

Purpose: Decapsulate HQC-1 with the QSC compact 32-byte private seed.

bool qsc_hqc1_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Caller-owned shared secret output of 32 bytes.; ciphertext: Ciphertext input of QSC_HQC1_CIPHERTEXT_SIZE bytes.; privatekey: QSC compact private seed input of 32 bytes..

Return: True when backend decapsulation completed; follow documented implicit rejection semantics.

Notes: All buffers are caller-owned. On invalid arguments, no backend call occurs.

qsc_hqc1_encapsulate

Purpose: Encapsulate a 32-byte secret and HQC-1 ciphertext.

bool qsc_hqc1_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Caller-owned output of QSC_HQC_SHAREDSECRET_BYTES bytes.; ciphertext: Caller-owned output of QSC_HQC1_CIPHERTEXT_SIZE bytes.; publickey: Verification input of QSC_HQC1_PUBLICKEY_SIZE bytes.; rng_generate: RNG callback, which must supply secure entropy; thread safety depends on it..

Return: True on success; false on invalid pointers or random source failure.

qsc_hqc1_generate_keypair

Purpose: Generate a QSC HQC-1 compact-key pair using the supplied RNG.

bool qsc_hqc1_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Caller-owned output of QSC_HQC1_PUBLICKEY_SIZE bytes.; privatekey: Caller-owned secret seed output of 32 bytes; caller must erase it.; rng_generate: RNG callback; thread safety depends on its implementation..

Return: True on success; false for invalid arguments or RNG failure.

qsc_hqc1_generate_seeded_keypair

Purpose: Generate a QSC HQC-1 key pair from the supplied 32-byte seed.

void qsc_hqc1_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* seed);

Parameters: publickey: Caller-owned output of QSC_HQC1_PUBLICKEY_SIZE bytes.; privatekey: Caller-owned 32-byte compact seed output; erase after use.; seed: Caller-owned seed, at least QSC_HQC_SEED_BYTES bytes..

Return: None (void).

Notes: No output is produced for null pointers; caller owns all buffers.

qsc_hqc1_seeded_encapsulate

Purpose: Deterministically encapsulate a shared secret for HQC-1.

void qsc_hqc1_seeded_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, const uint8_t seed[QSC_HQC_SEED_BYTES]);

Parameters: secret: Caller-owned 32-byte secret output.; ciphertext: Caller-owned ciphertext output of QSC_HQC1_CIPHERTEXT_SIZE bytes.; publickey: Public key input, QSC_HQC1_PUBLICKEY_SIZE bytes.; seed: Caller-provided 32-byte reproducibility seed..

Return: None (void).

Notes: For tests/controlled entropy only; no output if any pointer is null.

qsc_hqc3_decapsulate

Purpose: Decapsulate HQC-3 with the QSC compact 32-byte private seed.

bool qsc_hqc3_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Caller-owned shared secret output of 32 bytes.; ciphertext: Ciphertext input of QSC_HQC3_CIPHERTEXT_SIZE bytes.; privatekey: QSC compact private seed input of 32 bytes..

Return: True when backend decapsulation completed; follow documented implicit rejection semantics.

Notes: All buffers are caller-owned. On invalid arguments, no backend call occurs.

qsc_hqc3_encapsulate

Purpose: Encapsulate a 32-byte secret and HQC-3 ciphertext.

bool qsc_hqc3_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Caller-owned output of QSC_HQC_SHAREDSECRET_BYTES bytes.; ciphertext: Caller-owned output of QSC_HQC3_CIPHERTEXT_SIZE bytes.; publickey: Verification input of QSC_HQC3_PUBLICKEY_SIZE bytes.; rng_generate: RNG callback, which must supply secure entropy; thread safety depends on it..

Return: True on success; false on invalid pointers or random source failure.

qsc_hqc3_generate_keypair

Purpose: Generate a QSC HQC-3 compact-key pair using the supplied RNG.

bool qsc_hqc3_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Caller-owned output of QSC_HQC3_PUBLICKEY_SIZE bytes.; privatekey: Caller-owned secret seed output of 32 bytes; caller must erase it.; rng_generate: RNG callback; thread safety depends on its implementation..

Return: True on success; false for invalid arguments or RNG failure.

qsc_hqc3_generate_seeded_keypair

Purpose: Generate a QSC HQC-3 key pair from the supplied 32-byte seed.

void qsc_hqc3_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* seed);

Parameters: publickey: Caller-owned output of QSC_HQC3_PUBLICKEY_SIZE bytes.; privatekey: Caller-owned 32-byte compact seed output; erase after use.; seed: Caller-owned seed, at least QSC_HQC_SEED_BYTES bytes..

Return: None (void).

Notes: No output is produced for null pointers; caller owns all buffers.

qsc_hqc3_seeded_encapsulate

Purpose: Deterministically encapsulate a shared secret for HQC-3.

void qsc_hqc3_seeded_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, const uint8_t seed[QSC_HQC_SEED_BYTES]);

Parameters: secret: Caller-owned 32-byte secret output.; ciphertext: Caller-owned ciphertext output of QSC_HQC3_CIPHERTEXT_SIZE bytes.; publickey: Public key input, QSC_HQC3_PUBLICKEY_SIZE bytes.; seed: Caller-provided 32-byte reproducibility seed..

Return: None (void).

Notes: For tests/controlled entropy only; no output if any pointer is null.

qsc_hqc5_decapsulate

Purpose: Decapsulate HQC-5 with the QSC compact 32-byte private seed.

bool qsc_hqc5_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Caller-owned shared secret output of 32 bytes.; ciphertext: Ciphertext input of QSC_HQC5_CIPHERTEXT_SIZE bytes.; privatekey: QSC compact private seed input of 32 bytes..

Return: True when backend decapsulation completed; follow documented implicit rejection semantics.

Notes: All buffers are caller-owned. On invalid arguments, no backend call occurs.

qsc_hqc5_encapsulate

Purpose: Encapsulate a 32-byte secret and HQC-5 ciphertext.

bool qsc_hqc5_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Caller-owned output of QSC_HQC_SHAREDSECRET_BYTES bytes.; ciphertext: Caller-owned output of QSC_HQC5_CIPHERTEXT_SIZE bytes.; publickey: Verification input of QSC_HQC5_PUBLICKEY_SIZE bytes.; rng_generate: RNG callback, which must supply secure entropy; thread safety depends on it..

Return: True on success; false on invalid pointers or random source failure.

qsc_hqc5_generate_keypair

Purpose: Generate a QSC HQC-5 compact-key pair using the supplied RNG.

bool qsc_hqc5_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Caller-owned output of QSC_HQC5_PUBLICKEY_SIZE bytes.; privatekey: Caller-owned secret seed output of 32 bytes; caller must erase it.; rng_generate: RNG callback; thread safety depends on its implementation..

Return: True on success; false for invalid arguments or RNG failure.

qsc_hqc5_generate_seeded_keypair

Purpose: Generate a QSC HQC-5 key pair from the supplied 32-byte seed.

void qsc_hqc5_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* seed);

Parameters: publickey: Caller-owned output of QSC_HQC5_PUBLICKEY_SIZE bytes.; privatekey: Caller-owned 32-byte compact seed output; erase after use.; seed: Caller-owned seed, at least QSC_HQC_SEED_BYTES bytes..

Return: None (void).

Notes: No output is produced for null pointers; caller owns all buffers.

qsc_hqc5_seeded_encapsulate

Purpose: Deterministically encapsulate a shared secret for HQC-5.

void qsc_hqc5_seeded_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, const uint8_t seed[QSC_HQC_SEED_BYTES]);

Parameters: secret: Caller-owned 32-byte secret output.; ciphertext: Caller-owned ciphertext output of QSC_HQC5_CIPHERTEXT_SIZE bytes.; publickey: Public key input, QSC_HQC5_PUBLICKEY_SIZE bytes.; seed: Caller-provided 32-byte reproducibility seed..

Return: None (void).

Notes: For tests/controlled entropy only; no output if any pointer is null.

Header: kyber.h

qsc_kyber_generate_seeded_keypair

Purpose: Generates public and private key for the CCA-Secure Kyber key encapsulation mechanism using input seeds.

void qsc_kyber_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* d, uint8_t* z);

Parameters: publickey: Caller-provided output for the selected parameter-set public key.; privatekey: Caller-provided output for the selected parameter-set private key.; d: [uint8_t*] Pointer to the random d coin (a populated random array of QSC_KYBER_SYMBYTES bytes); z: [uint8_t*] Pointer to the random z coin (a populated random array of QSC_KYBER_SYMBYTES bytes).

Return: None (void).

Notes: Used exclusively for the NIST ACVP KAT tests, use the other call to generate the key-pair.

Header: kyber1024.h

qsc_kyber1024_decapsulate

Purpose: Validate the private key and decapsulate a ciphertext.

bool qsc_kyber1024_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output shared secret (QSC_KYBER1024_SHAREDSECRET_SIZE bytes).; ciphertext: Input ciphertext (QSC_KYBER1024_CIPHERTEXT_SIZE bytes).; privatekey: Input private key (QSC_KYBER1024_PRIVATEKEY_SIZE bytes)..

Return: True for a valid private key and completed decapsulation. An invalid ciphertext uses implicit rejection and still returns true.

Notes: Invalid secret key erases the output. Inputs must not overlap.

qsc_kyber1024_encapsulate

Purpose: Encapsulate a shared secret for a validated public key.

bool qsc_kyber1024_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output shared secret (QSC_KYBER1024_SHAREDSECRET_SIZE bytes).; ciphertext: Output ciphertext (QSC_KYBER1024_CIPHERTEXT_SIZE bytes).; publickey: Input public key (QSC_KYBER1024_PUBLICKEY_SIZE bytes).; rng_generate: Caller-supplied CSPRNG callback; must fill requested bytes..

Return: True on success; false if the key, callback or pointer is invalid.

Notes: An invalid public key clears the output buffers. Inputs must not overlap.

qsc_kyber1024_generate_keypair

Purpose: Generate a keypair for the independent 1024 profile.

bool qsc_kyber1024_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output public key (QSC_KYBER1024_PUBLICKEY_SIZE bytes).; privatekey: Output private key (QSC_KYBER1024_PRIVATEKEY_SIZE bytes).; rng_generate: Caller-supplied CSPRNG callback..

Return: True on successful generation; false on invalid pointers or RNG failure.

Notes: Output buffers must not overlap.

qsc_kyber1024_generate_seeded_keypair

Purpose: Deterministically generate a keypair for regression testing.

void qsc_kyber1024_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* d, uint8_t* z);

Parameters: publickey: Output public key (QSC_KYBER1024_PUBLICKEY_SIZE bytes).; privatekey: Output private key (QSC_KYBER1024_PRIVATEKEY_SIZE bytes).; d: Input fixed 32-byte key-generation seed (may be modified by backend).; z: Input fixed 32-byte implicit-rejection seed..

Return: None (void).

Notes: For test vectors only; production callers must use generate_keypair.

qsc_kyber1024_seeded_encapsulate

Purpose: Deterministically encapsulate for known-answer testing.

void qsc_kyber1024_seeded_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, const uint8_t* m);

Parameters: secret: Output shared secret (QSC_KYBER1024_SHAREDSECRET_SIZE bytes).; ciphertext: Output ciphertext (QSC_KYBER1024_CIPHERTEXT_SIZE bytes).; publickey: Input public key (QSC_KYBER1024_PUBLICKEY_SIZE bytes).; m: Input fixed 32-byte test coin..

Return: None (void).

Notes: For tests only; production callers must use encapsulate with a CSPRNG.

Header: kyber1280.h

qsc_kyber1280_decapsulate

Purpose: Validate the private key and decapsulate a ciphertext.

bool qsc_kyber1280_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output shared secret (QSC_KYBER1280_SHAREDSECRET_SIZE bytes).; ciphertext: Input ciphertext (QSC_KYBER1280_CIPHERTEXT_SIZE bytes).; privatekey: Input private key (QSC_KYBER1280_PRIVATEKEY_SIZE bytes)..

Return: True for a valid private key and completed decapsulation. An invalid ciphertext uses implicit rejection and still returns true.

Notes: Invalid secret key erases the output. Inputs must not overlap.

qsc_kyber1280_encapsulate

Purpose: Encapsulate a shared secret for a validated public key.

bool qsc_kyber1280_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output shared secret (QSC_KYBER1280_SHAREDSECRET_SIZE bytes).; ciphertext: Output ciphertext (QSC_KYBER1280_CIPHERTEXT_SIZE bytes).; publickey: Input public key (QSC_KYBER1280_PUBLICKEY_SIZE bytes).; rng_generate: Caller-supplied CSPRNG callback; must fill requested bytes..

Return: True on success; false if the key, callback or pointer is invalid.

Notes: An invalid public key clears the output buffers. Inputs must not overlap.

qsc_kyber1280_generate_keypair

Purpose: Generate a keypair for the independent 1280 profile.

bool qsc_kyber1280_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output public key (QSC_KYBER1280_PUBLICKEY_SIZE bytes).; privatekey: Output private key (QSC_KYBER1280_PRIVATEKEY_SIZE bytes).; rng_generate: Caller-supplied CSPRNG callback..

Return: True on successful generation; false on invalid pointers or RNG failure.

Notes: Output buffers must not overlap.

qsc_kyber1280_generate_seeded_keypair

Purpose: Deterministically generate a keypair for regression testing.

void qsc_kyber1280_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* d, uint8_t* z);

Parameters: publickey: Output public key (QSC_KYBER1280_PUBLICKEY_SIZE bytes).; privatekey: Output private key (QSC_KYBER1280_PRIVATEKEY_SIZE bytes).; d: Input fixed 32-byte key-generation seed (may be modified by backend).; z: Input fixed 32-byte implicit-rejection seed..

Return: None (void).

Notes: For test vectors only; production callers must use generate_keypair.

qsc_kyber1280_seeded_encapsulate

Purpose: Deterministically encapsulate for known-answer testing.

void qsc_kyber1280_seeded_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, const uint8_t* m);

Parameters: secret: Output shared secret (QSC_KYBER1280_SHAREDSECRET_SIZE bytes).; ciphertext: Output ciphertext (QSC_KYBER1280_CIPHERTEXT_SIZE bytes).; publickey: Input public key (QSC_KYBER1280_PUBLICKEY_SIZE bytes).; m: Input fixed 32-byte test coin..

Return: None (void).

Notes: For tests only; production callers must use encapsulate with a CSPRNG.

Header: kyber512.h

qsc_kyber512_decapsulate

Purpose: Validate the private key and decapsulate a ciphertext.

bool qsc_kyber512_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output shared secret (QSC_KYBER512_SHAREDSECRET_SIZE bytes).; ciphertext: Input ciphertext (QSC_KYBER512_CIPHERTEXT_SIZE bytes).; privatekey: Input private key (QSC_KYBER512_PRIVATEKEY_SIZE bytes)..

Return: True for a valid private key and completed decapsulation. An invalid ciphertext uses implicit rejection and still returns true.

Notes: Invalid secret key erases the output. Inputs must not overlap.

qsc_kyber512_encapsulate

Purpose: Encapsulate a shared secret for a validated public key.

bool qsc_kyber512_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output shared secret (QSC_KYBER512_SHAREDSECRET_SIZE bytes).; ciphertext: Output ciphertext (QSC_KYBER512_CIPHERTEXT_SIZE bytes).; publickey: Input

public key (QSC_KYBER512_PUBLICKEY_SIZE bytes).; rng_generate: Caller-supplied CSPRNG callback; must fill requested bytes..

Return: True on success; false if the key, callback or pointer is invalid.

Notes: An invalid public key clears the output buffers. Inputs must not overlap.

qsc_kyber512_generate_keypair

Purpose: Generate a keypair for the independent 512 profile.

bool qsc_kyber512_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output public key (QSC_KYBER512_PUBLICKEY_SIZE bytes).; privatekey: Output private key (QSC_KYBER512_PRIVATEKEY_SIZE bytes).; rng_generate: Caller-supplied CSPRNG callback..

Return: True on successful generation; false on invalid pointers or RNG failure.

Notes: Output buffers must not overlap.

qsc_kyber512_generate_seeded_keypair

Purpose: Deterministically generate a keypair for regression testing.

void qsc_kyber512_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* d, uint8_t* z);

Parameters: publickey: Output public key (QSC_KYBER512_PUBLICKEY_SIZE bytes).; privatekey: Output private key (QSC_KYBER512_PRIVATEKEY_SIZE bytes).; d: Input fixed 32-byte key-generation seed (may be modified by backend).; z: Input fixed 32-byte implicit-rejection seed..

Return: None (void).

Notes: For test vectors only; production callers must use generate_keypair.

qsc_kyber512_seeded_encapsulate

Purpose: Deterministically encapsulate for known-answer testing.

void qsc_kyber512_seeded_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, const uint8_t* m);

Parameters: secret: Output shared secret (QSC_KYBER512_SHAREDSECRET_SIZE bytes).; ciphertext: Output ciphertext (QSC_KYBER512_CIPHERTEXT_SIZE bytes).; publickey: Input public key (QSC_KYBER512_PUBLICKEY_SIZE bytes).; m: Input fixed 32-byte test coin..

Return: None (void).

Notes: For tests only; production callers must use encapsulate with a CSPRNG.

Header: kyber768.h

qsc_kyber768_decapsulate

Purpose: Validate the private key and decapsulate a ciphertext.

bool qsc_kyber768_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output shared secret (QSC_KYBER768_SHAREDSECRET_SIZE bytes).; ciphertext: Input ciphertext (QSC_KYBER768_CIPHERTEXT_SIZE bytes).; privatekey: Input private key (QSC_KYBER768_PRIVATEKEY_SIZE bytes)..

Return: True for a valid private key and completed decapsulation. An invalid ciphertext uses implicit rejection and still returns true.

Notes: Invalid secret key erases the output. Inputs must not overlap.

qsc_kyber768_encapsulate

Purpose: Encapsulate a shared secret for a validated public key.

bool qsc_kyber768_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output shared secret (QSC_KYBER768_SHAREDSECRET_SIZE bytes).; ciphertext: Output ciphertext (QSC_KYBER768_CIPHERTEXT_SIZE bytes).; publickey: Input public key (QSC_KYBER768_PUBLICKEY_SIZE bytes).; rng_generate: Caller-supplied CSPRNG callback; must fill requested bytes..

Return: True on success; false if the key, callback or pointer is invalid.

Notes: An invalid public key clears the output buffers. Inputs must not overlap.

qsc_kyber768_generate_keypair

Purpose: Generate a keypair for the independent 768 profile.

bool qsc_kyber768_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output public key (QSC_KYBER768_PUBLICKEY_SIZE bytes).; privatekey: Output private key (QSC_KYBER768_PRIVATEKEY_SIZE bytes).; rng_generate: Caller-supplied CSPRNG callback..

Return: True on successful generation; false on invalid pointers or RNG failure.

Notes: Output buffers must not overlap.

qsc_kyber768_generate_seeded_keypair

Purpose: Deterministically generate a keypair for regression testing.

void qsc_kyber768_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, uint8_t* d, uint8_t* z);

Parameters: publickey: Output public key (QSC_KYBER768_PUBLICKEY_SIZE bytes).; privatekey: Output private key (QSC_KYBER768_PRIVATEKEY_SIZE bytes).; d: Input fixed 32-byte key-generation seed (may be modified by backend).; z: Input fixed 32-byte implicit-rejection seed..

Return: None (void).

Notes: For test vectors only; production callers must use generate_keypair.

qsc_kyber768_seeded_encapsulate

Purpose: Deterministically encapsulate for known-answer testing.

void qsc_kyber768_seeded_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, const uint8_t* m);

Parameters: secret: Output shared secret (QSC_KYBER768_SHAREDSECRET_SIZE bytes).; ciphertext: Output ciphertext (QSC_KYBER768_CIPHERTEXT_SIZE bytes).; publickey: Input public key (QSC_KYBER768_PUBLICKEY_SIZE bytes).; m: Input fixed 32-byte test coin..

Return: None (void).

Notes: For tests only; production callers must use encapsulate with a CSPRNG.

Header: mceliece_multi.h

qsc_mceliece_460896_decapsulate

Purpose: Decapsulates using the ISO 460896 construction.

bool qsc_mceliece_460896_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_460896_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_460896_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_460896_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_460896_encapsulate

Purpose: Encapsulates using ISO 460896.

bool qsc_mceliece_460896_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_460896_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_460896_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_460896_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_460896_generate_keypair

Purpose: Generates a public and secret key pair for ISO 460896.

bool qsc_mceliece_460896_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_460896_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_460896_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_460896f_decapsulate

Purpose: Decapsulates using the ISO 460896f construction.

bool qsc_mceliece_460896f_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_460896F_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_460896F_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_460896F_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_460896f_encapsulate

Purpose: Encapsulates using ISO 460896f.

bool qsc_mceliece_460896f_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_460896F_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_460896F_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_460896F_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_460896f_generate_keypair

Purpose: Generates a public and secret key pair for ISO 460896f.

bool qsc_mceliece_460896f_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_460896F_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_460896F_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_460896pc_decapsulate

Purpose: Decapsulates using the ISO 460896pc construction.

bool qsc_mceliece_460896pc_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_460896PC_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_460896PC_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_460896PC_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_460896pc_encapsulate

Purpose: Encapsulates using ISO 460896pc.

bool qsc_mceliece_460896pc_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_460896PC_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_460896PC_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_460896PC_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_460896pc_generate_keypair

Purpose: Generates a public and secret key pair for ISO 460896pc.

bool qsc_mceliece_460896pc_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_460896PC_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_460896PC_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_460896pcf_decapsulate

Purpose: Decapsulates using the ISO 460896pcf construction.

bool qsc_mceliece_460896pcf_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_460896PCF_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_460896PCF_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_460896PCF_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_460896pcf_encapsulate

Purpose: Encapsulates using ISO 460896pcf.

bool qsc_mceliece_460896pcf_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_460896PCF_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_460896PCF_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_460896PCF_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_460896pcf_generate_keypair

Purpose: Generates a public and secret key pair for ISO 460896pcf.

bool qsc_mceliece_460896pcf_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_460896PCF_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_460896PCF_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_6688128_decapsulate

Purpose: Decapsulates using the ISO 6688128 construction.

bool qsc_mceliece_6688128_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_6688128_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_6688128_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_6688128_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_6688128_encapsulate

Purpose: Encapsulates using ISO 6688128.

bool qsc_mceliece_6688128_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_6688128_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_6688128_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_6688128_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_6688128_generate_keypair

Purpose: Generates a public and secret key pair for ISO 6688128.

bool qsc_mceliece_6688128_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_6688128_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_6688128_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_6688128f_decapsulate

Purpose: Decapsulates using the ISO 6688128f construction.

bool qsc_mceliece_6688128f_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_6688128F_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_6688128F_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_6688128F_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_6688128f_encapsulate

Purpose: Encapsulates using ISO 6688128f.

bool qsc_mceliece_6688128f_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_6688128F_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_6688128F_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_6688128F_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_6688128f_generate_keypair

Purpose: Generates a public and secret key pair for ISO 6688128f.

bool qsc_mceliece_6688128f_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_6688128F_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_6688128F_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_6688128pc_decapsulate

Purpose: Decapsulates using the ISO 6688128pc construction.

bool qsc_mceliece_6688128pc_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_6688128PC_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_6688128PC_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_6688128PC_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_6688128pc_encapsulate

Purpose: Encapsulates using ISO 6688128pc.

bool qsc_mceliece_6688128pc_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_6688128PC_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_6688128PC_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_6688128PC_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_6688128pc_generate_keypair

Purpose: Generates a public and secret key pair for ISO 6688128pc.

bool qsc_mceliece_6688128pc_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_6688128PC_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_6688128PC_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_6688128pcf_decapsulate

Purpose: Decapsulates using the ISO 6688128pcf construction.

bool qsc_mceliece_6688128pcf_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_6688128PCF_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_6688128PCF_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_6688128PCF_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_6688128pcf_encapsulate

Purpose: Encapsulates using ISO 6688128pcf.

bool qsc_mceliece_6688128pcf_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_6688128PCF_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_6688128PCF_CIPHERTEXT_SIZE bytes.; publickey:

Input, QSC_MCELIECE_6688128PCF_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_6688128pcf_generate_keypair

Purpose: Generates a public and secret key pair for ISO 6688128pcf.

bool qsc_mceliece_6688128pcf_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_6688128PCF_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_6688128PCF_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_6960119_decapsulate

Purpose: Decapsulates using the ISO 6960119 construction.

bool qsc_mceliece_6960119_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_6960119_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_6960119_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_6960119_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_6960119_encapsulate

Purpose: Encapsulates using ISO 6960119.

bool qsc_mceliece_6960119_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_6960119_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_6960119_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_6960119_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_6960119_generate_keypair

Purpose: Generates a public and secret key pair for ISO 6960119.

bool qsc_mceliece_6960119_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_6960119_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_6960119_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_6960119f_decapsulate

Purpose: Decapsulates using the ISO 6960119f construction.

bool qsc_mceliece_6960119f_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_6960119F_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_6960119F_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_6960119F_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119f, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_6960119f_encapsulate

Purpose: Encapsulates using ISO 6960119f.

bool qsc_mceliece_6960119f_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_6960119F_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_6960119F_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_6960119F_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_6960119f_generate_keypair

Purpose: Generates a public and secret key pair for ISO 6960119f.

bool qsc_mceliece_6960119f_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_6960119F_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_6960119F_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_6960119pc_decapsulate

Purpose: Decapsulates using the ISO 6960119pc construction.

bool qsc_mceliece_6960119pc_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_6960119PC_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_6960119PC_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_6960119PC_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_6960119pc_encapsulate

Purpose: Encapsulates using ISO 6960119pc.

bool qsc_mceliece_6960119pc_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_6960119PC_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_6960119PC_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_6960119PC_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_6960119pc_generate_keypair

Purpose: Generates a public and secret key pair for ISO 6960119pc.

bool qsc_mceliece_6960119pc_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_6960119PC_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_6960119PC_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_6960119pcf_decapsulate

Purpose: Decapsulates using the ISO 6960119pcf construction.

bool qsc_mceliece_6960119pcf_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_6960119PCF_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_6960119PCF_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_6960119PCF_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_6960119pcf_encapsulate

Purpose: Encapsulates using ISO 6960119pcf.

bool qsc_mceliece_6960119pcf_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_6960119PCF_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_6960119PCF_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_6960119PCF_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_6960119pcf_generate_keypair

Purpose: Generates a public and secret key pair for ISO 6960119pcf.

bool qsc_mceliece_6960119pcf_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_6960119PCF_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_6960119PCF_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_8192128_decapsulate

Purpose: Decapsulates using the ISO 8192128 construction.

bool qsc_mceliece_8192128_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_8192128_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_8192128_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_8192128_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_8192128_encapsulate

Purpose: Encapsulates using ISO 8192128.

bool qsc_mceliece_8192128_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_8192128_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_8192128_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_8192128_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_8192128_generate_keypair

Purpose: Generates a public and secret key pair for ISO 8192128.

bool qsc_mceliece_8192128_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_8192128_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_8192128_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_8192128f_decapsulate

Purpose: Decapsulates using the ISO 8192128f construction.

bool qsc_mceliece_8192128f_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_8192128F_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_8192128F_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_8192128F_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_8192128f_encapsulate

Purpose: Encapsulates using ISO 8192128f.

bool qsc_mceliece_8192128f_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_8192128F_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_8192128F_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_8192128F_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_8192128f_generate_keypair

Purpose: Generates a public and secret key pair for ISO 8192128f.

bool qsc_mceliece_8192128f_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_8192128F_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_8192128F_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_8192128pc_decapsulate

Purpose: Decapsulates using the ISO 8192128pc construction.

bool qsc_mceliece_8192128pc_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_8192128PC_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_8192128PC_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_8192128PC_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_8192128pc_encapsulate

Purpose: Encapsulates using ISO 8192128pc.

bool qsc_mceliece_8192128pc_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_8192128PC_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_8192128PC_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_8192128PC_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_8192128pc_generate_keypair

Purpose: Generates a public and secret key pair for ISO 8192128pc.

bool qsc_mceliece_8192128pc_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_8192128PC_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_8192128PC_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

qsc_mceliece_8192128pcf_decapsulate

Purpose: Decapsulates using the ISO 8192128pcf construction.

bool qsc_mceliece_8192128pcf_decapsulate(uint8_t* secret, const uint8_t* ciphertext, const uint8_t* privatekey);

Parameters: secret: Output, QSC_MCELIECE_8192128PCF_SHAREDSECRET_SIZE bytes.; ciphertext: Input, QSC_MCELIECE_8192128PCF_CIPHERTEXT_SIZE bytes.; privatekey: Input, QSC_MCELIECE_8192128PCF_PRIVATEKEY_SIZE bytes..

Return: True after implicit-rejection processing; false on invalid inputs or, for 6960119, nonzero ciphertext padding.

Notes: Reentrant with disjoint input/output arrays; the caller owns memory.

qsc_mceliece_8192128pcf_encapsulate

Purpose: Encapsulates using ISO 8192128pcf.

bool qsc_mceliece_8192128pcf_encapsulate(uint8_t* secret, uint8_t* ciphertext, const uint8_t* publickey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: secret: Output, QSC_MCELIECE_8192128PCF_SHAREDSECRET_SIZE bytes.; ciphertext: Output, QSC_MCELIECE_8192128PCF_CIPHERTEXT_SIZE bytes.; publickey: Input, QSC_MCELIECE_8192128PCF_PUBLICKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false.

Notes: Caller owns all buffers; reentrant for independent RNG and buffers.

qsc_mceliece_8192128pcf_generate_keypair

Purpose: Generates a public and secret key pair for ISO 8192128pcf.

bool qsc_mceliece_8192128pcf_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output, QSC_MCELIECE_8192128PCF_PUBLICKEY_SIZE bytes.; privatekey: Output, QSC_MCELIECE_8192128PCF_PRIVATEKEY_SIZE bytes.; rng_generate: Secure RNG callback; shared state must be synchronized..

Return: True on success, otherwise false, including RNG failure.

Notes: Requires a sufficiently large stack; recommended minimum is 4 MiB. Caller owns key buffers. Reentrant for independent RNG and buffers.

10: Asymmetric Signature Schemes

10.1 Dilithium

Header:

dilithium.h

Description:

The NIST FIPS 204 specification of the asymmetric signature-scheme Dilithium. A message is signed using the private key, and verified using the public key.

Constants:

Constant Name	Value	Purpose
QSC_DILITHIUM_PRIVATEKEY_SIZE	variable	The byte size of the secret private-key array.
QSC_DILITHIUM_PUBLICKEY_SIZE	variable	The byte size of the public-key array.
QSC_DILITHIUM_SIGNATURE_SIZE	variable	The byte size of the signature array.

Table 10.1a Dilithium constants

Parameter Sets:

Constant Name	Value	Purpose
QSC_DILITHIUM_S1P44	N/A	The Dilithium S1N256Q8380417 parameter set.
QSC_DILITHIUM_S3P65	N/A	The Dilithium S2N256Q8380417 parameter set.
QSC_DILITHIUM_S5P87	N/A	The Dilithium S3N256Q8380417 parameter set.

Table 10.1b Dilithium parameter sets

API:

Generate

The generate function creates the public verification key and private signing key.

The generate function takes pointers to the public and private key arrays, and a pointer to the RNG function as parameters.

```
bool qsc_dilithium_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t))
```

Sign

The sign function signs a message. It takes pointers to the signed message array and signed message length, a pointer to the message array, the message size, a pointer to the private key, and a pointer to the RNG function as parameters.

```
bool qsc_dilithium_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t))
```

Verify

The verify function, authenticates the signature attached to the message. The verify function takes a pointer to the output message array and message length, a pointer to the signed message, and public key array as parameters, and returns true if the signature is verified, false if it has failed.

```
bool qsc_dilithium_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey)
```

References:

- Dilithium Reference Paper : https://pq-crystals.org/dilithium/data/dilithium-specification.pdf

- Dilithium, A Lattice-Based Digital Signature Scheme: https://pq-crystals.org/dilithium/data/dilithium-20180114.pdf

10.2 ECDSA

Header:

ecdsa.h

Description:

The Elliptic Curve Digital Signature Algorithm, signs a message that can be verified with a public key.

Constants:

Constant Name	Value	Purpose
QSC_ECDSA_SIGNATURE_SIZE	64 / 96 / 132	P-256 / P-384 / P-521 raw signature sizes, in bytes.
QSC_ECDSA_PRIVATEKEY_SIZE	96 / 144 / 198	P-256 / P-384 / P-521 combined private-key sizes, in bytes.
QSC_ECDSA_PUBLICKEY_SIZE	64 / 96 / 132	P-256 / P-384 / P-521 public-key sizes, in bytes.
QSC_ECDSA_SEED_SIZE	32 / 48 / 66	P-256 / P-384 / P-521 seed sizes, in bytes.

Table 10.2a ECDSA constants

API:

Generate

The generate function creates the public verification key and private signing key. The function takes pointers to the public and private key arrays, and a pointer to the RNG function as parameters.

```
bool qsc_ecdsa_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool
(*rng_generate)(uint8_t*, size_t));
```

Sign

The sign function signs a message. It takes pointers to the signed message array and signed message length, a pointer to the message array, the message size, and a pointer to the private key as parameters.

```
bool qsc_ecdsa_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t
msglen, const uint8_t* privatekey);
```

Verify

The verify function, authenticates the signature attached to the message. The verify function takes a pointer to the output message array and message length, a pointer to the signed message, and public key array as parameters, and returns true if the signature is verified, false if it has failed.

```
bool qsc_ecdsa_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t
smsglen, const uint8_t* publickey)
```

References:

NaCl by Daniel J. Bernstein, Tanja Lange, Peter Schwabe https://nacl.cr.yp.to

10.4 Falcon

Header:

falcon.h

Description:

The NIST Post Quantum Competition round 3 asymmetric signature-scheme finalist Falcon. A message is signed using the private key, and verified using the public key.

Constants:

Constant Name	Value	Purpose
QSC_FALCON_PRIVATEKEY_SIZE	variable	The byte size of the secret private-key array.
QSC_FALCON_PUBLICKEY_SIZE	variable	The byte size of the public-key array.
QSC_FALCON_SIGNATURE_SIZE	variable	The byte size of the signature array.

Table 10.4a Falcon constants

Parameter Sets:

Constant Name	Value	Purpose
QSC_FALCON_S3SHAKE256F512	N/A	The Falcon S3SHAKE256F512 parameter set.
QSC_FALCON_S5SHAKE256F1024	N/A	The Falcon S5SHAKE256F1024 parameter set.

Table 10.4b Falcon parameter sets

API:

Generate

The generate function creates the public verification key and private signing key.

The function takes pointers to the public and private key arrays, and a pointer to the RNG function.

```
bool qsc_falcon_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t))
```

Sign

The sign function signs a message. It takes pointers to the output signed message array and signed message length, a pointer to the message array, the message size, a pointer to the private key, and a pointer to the RNG function as parameters.

```
bool qsc_falcon_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t))
```

Verify

The verify function, authenticates the signature attached to the message. The verify function takes a pointer to the output message array and message length, pointers to the signed message, and public key array as parameters, and returns true if the signature is verified, false if it has failed.

```
bool qsc_falcon_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey)
```

References:

The Falcon Algorithm Specification: https://falcon-sign.info/falcon.pdf

10.5 SPHINCS+

Header:

sphincsplus.h

Description:

The NIST FIPS 205 specification of the asymmetric signature-scheme candidate SPHINCS+. A message is signed using the private key, and verified using the public key.

Constants:

Constant Name	Value	Purpose

QSC_SPHINCSPLUS_SIGNATURE_SIZE	variable	The byte size of the secret private-key array.
QSC_SPHINCSPLUS_PRIVATEKEY_SIZE	variable	The byte size of the public-key array.
QSC_SPHINCSPLUS_PUBLICKEY_SIZE	variable	The byte size of the signature array.

Table 10.4a SPHINCS+ constants

Parameter Sets:

Constant Name	Value	Purpose
QSC_SPHINCSPLUS_S3S192SHAKERS	N/A	The SphincsPlus S3S192SHAKERS robust small parameter set.
QSC_SPHINCSPLUS_S5S256SHAKERS	N/A	The SphincsPlus S5S256SHAKERS robust small parameter set.

Table 10.5b SPHINCS+ parameter sets

API:

Generate

The generate function creates the public verification key and private signing key. The function takes pointers to the public and private key arrays, and a pointer to the RNG function as parameters.

bool qsc_sphincsplus_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t))

Sign

The sign function signs a message. It takes pointers to the output signed message array and signed message length, a pointer to the message array, the message size, a pointer to the private key, and a pointer to the RNG function as parameters.

bool qsc_sphincsplus_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t))

Verify

The verify function, authenticates the signature attached to the message. The verify function takes a pointer to the output message array and message length, a pointer to the signed message, and public key array as parameters, and returns true if the signature is verified, false if it has failed.

bool qsc_sphincsplus_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey)

References:

- The SPHINCS+ Signature Scheme: https://sphincs.org/data/sphincs+-specification.pdf

10.6 Additional Digital Signature APIs

Public ML-DSA-44/65/87, Ed25519/Ed448, NIST P-curve ECDSA and six SHAKE-based SLH-DSA parameter-set families are listed with their actual exported signatures.

Header: dilithium.h

qsc_dilithium_seeded_generate_keypair

Purpose: Generates a Dilithium public/private key-pair using a random input seed.

void qsc_dilithium_seeded_generate_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: [uint8_t*] Pointer to the public verification-key array.; privatekey: [uint8_t*] Pointer to the private signature-key array.; seed: [const uint8_t*] Pointer to the random seed..

Return: None (void).

Notes: Used exclusively for the NIST ACVP KAT tests, use the other call to generate the key-pair. Arrays must be sized to QSC_DILITHIUM_PUBLICKEY_SIZE, QSC_DILITHIUM_PRIVATEKEY_SIZE, and the seed to DILITHIUM_SEEDBYTES.

qsc_dilithium_sign_ex

Purpose: Takes the message as input with the additional context parameter, and returns an array containing the signature followed by the message.

bool qsc_dilithium_sign_ex(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signedmsg: [uint8_t*] Pointer to the signed-message array.; smsglen: [size_t*] Pointer to the signed message length.; message: [const uint8_t*] Pointer to the message array.; msglen: [size_t] The message array length.; context: [const uint8_t*] Pointer to the context array.; ctxlen: [size_t] The context array length.; privatekey: [const uint8_t*] Pointer to the private signature-key.; rng_generate: [bool (*)(uint8_t*, size_t)] Pointer to the random generator..

Return: [bool] Returns true if the message was signed successfully.

Notes: The signed-message array must be sized to the size of the message plus QSC_DILITHIUM_SIGNATURE_SIZE.

qsc_dilithium_verify_ex

Purpose: Verifies a signature-message pair and context parameter with the public key.

bool qsc_dilithium_verify_ex(uint8_t* message, size_t* msglen, const uint8_t* context, size_t ctxlen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: [uint8_t*] Pointer to the message output array.; msglen: [size_t*] Pointer to the length of the message array.; context: [const uint8_t*] Pointer to the context array.; ctxlen: [size_t] The context array length.; signedmsg: [const uint8_t*]

Pointer to the signed message array.; smsglen: [size_t] The signed message length.; publickey: [const uint8_t*] Pointer to the public verification-key array..

Return: [bool] Returns true if the signature is valid.

Header: ecdsa.h

qsc_ecdsa_generate_seeded_keypair

Purpose: Generate a P-256 public/private key pair from a 32-byte seed.

Details: Derives the private scalar d = SHA-256(seed) mod n, computes the public key Q = d*G, and stores both. This function is deterministic: the same seed always produces the same key pair.

bool qsc_ecdsa_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: [uint8_t*] Pointer to the output public verification-key array.; privatekey: [uint8_t*] Pointer to the output private signature-key array.; seed: [const uint8_t*] Pointer to the random 32-byte seed array..

Return: [bool] Returns true on success.

Notes: Arrays must be sized to QSC_ECDSA_PUBLICKEY_SIZE and QSC_ECDSA_PRIVATEKEY_SIZE respectively.

qsc_ecdsa_publickey_from_privatekey

Purpose: Derive a P-256 public key from a raw private scalar.

Details: This function derives the affine public point Q = dG from a 32-byte big-endian private scalar and serializes the result as the raw public-key form Qx || Qy. The private scalar must be in the range [1, n - 1], where n is the order of the P-256 base point.

int32_t qsc_ecdsa_publickey_from_privatekey(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: [uint8_t*] Output buffer receiving the 64-byte public key.; privatekey: [const uint8_t*] Input 32-byte private scalar..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsa_publickey_from_sec1

Purpose: Convert a SEC 1 uncompressed point to the library raw public-key form.

bool qsc_ecdsa_publickey_from_sec1(uint8_t* publickey, const uint8_t* secpub);

Parameters: publickey: [uint8_t*] Output raw public key of 64 bytes.; secpub: [const uint8_t*] Input SEC 1 public key of 65 bytes..

Return: [bool] Returns true on success.

qsc_ecdsa_publickey_to_sec1

Purpose: Convert a raw public key (Qx || Qy) to SEC 1 uncompressed point form.

void qsc_ecdsa_publickey_to_sec1(uint8_t* secpub, const uint8_t* publickey);

Parameters: secpub: [uint8_t*] Output buffer of 65 bytes.; publickey: [const uint8_t*] Input raw public key of 64 bytes..

Return: None (void).

qsc_ecdsa_sign_scalar

Purpose: Sign a message with a P-256 private key, used primarily for RFC KAT testing).

Details: Computes a deterministic ECDSA signature using the private key scalar d, (RFC KATs) then writes the 64-byte signature followed by the message into signedmsg. On success *smsglen = msglen + 64.

bool qsc_ecdsa_sign_scalar(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: [uint8_t*] Pointer to the signed-message output array.; smsglen: [size_t*] Pointer to the signed-message length output.; message: [const uint8_t*] Pointer to the message to sign.; msglen: [size_t] Message length in bytes.; privatekey: [const uint8_t*] Pointer to the 96-byte private key array..

Return: [bool] Returns true on success.

Notes: signedmsg must be at least msglen + QSC_ECDSA_SIGNATURE_SIZE bytes.

Header: ecdsap256.h

qsc_ecdsap256_comb_verify_context_clear

Purpose: Clear and invalidate a reusable comb context.

void qsc_ecdsap256_comb_verify_context_clear(qsc_ecdsap256_comb_verify_context* context);

Parameters: context: Caller-owned context; NULL is allowed. Exclusive access required..

Return: None (void).

qsc_ecdsap256_comb_verify_context_initialize

Purpose: Validate a P-256 public key and construct its reusable comb table.

Details: Clears context on failure when non-NULL. No retained pointers or heap allocation; thread-safe with distinct contexts. Initialization uses public data.

bool qsc_ecdsap256_comb_verify_context_initialize(qsc_ecdsap256_comb_verify_context* context, const uint8_t* publickey);

Parameters: context: Caller-owned destination; exclusive access required.; publickey: Exactly EC_NISTP256_PUBLICKEY_SIZE readable bytes, disjoint from context..

Return: True on success; false for NULL or noncanonical/off-curve/identity keys.

qsc_ecdsap256_joint_verify_context_clear

Purpose: Erase a caller-owned four-bit joint verification context.

void qsc_ecdsap256_joint_verify_context_clear(qsc_ecdsap256_joint_verify_context* context);

Parameters: context: Context to clear, or NULL..

Return: None (void).

Notes: Do not erase while another thread is verifying with this context.

qsc_ecdsap256_joint_verify_context_initialize

Purpose: Validate a public key and initialize its four-bit joint precomputation.

bool qsc_ecdsap256_joint_verify_context_initialize(qsc_ecdsap256_joint_verify_context* context, const uint8_t* publickey);

Parameters: context: Caller-owned, writable context, cleared on failure.; publickey: Public P-256 X||Y, 64 bytes, big-endian..

Return: True on successful validation and initialization, otherwise false.

Notes: No heap, no global state; context initialization is not concurrent-safe.

qsc_ecdsap256_keypair

Purpose: Generate a P-256 public/private key pair from a 32-byte seed.

Details: Derives a private scalar from the seed via SHA-256, reduces it modulo n, rejects zero, computes Q = d*G using the P-256 base point, and stores both keys. The private key layout is seed[32] || Qx[32] || Qy[32].

```
int32_t qsc_ecdsap256_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);
```

Parameters: publickey: [uint8_t*] Output public key (64 bytes: Qx || Qy, big-endian).; privatekey: [uint8_t*] Output private key (96 bytes: seed || Qx || Qy).; seed: [const uint8_t*] 32-byte random seed..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsap256_privatekey_validate

Purpose: Validate an externally imported P-256 seed/public-key bundle.

Details: Independently regenerates the public point from the seed and compares all 64 embedded public-key bytes in constant time. Invoke at key ingestion and after any modification to imported key material. This is a separate validation API to avoid changing frozen signing performance.

```
bool qsc_ecdsap256_privatekey_validate(const uint8_t* privatekey);
```

Parameters: privatekey: [const uint8_t*] 96-byte seed || publickey bundle..

Return: True if the bundle is internally consistent; otherwise false.

Notes: No shared state is retained. Caller owns and erases the private key.

qsc_ecdsap256_publickey_from_privatekey

Purpose: Derive a P-256 public key from a raw private scalar.

Details: This function derives the affine public point Q = dG from a 32-byte big-endian private scalar and serializes the result as the raw public-key form Qx || Qy. The private scalar must be in the range [1, n - 1], where n is the order of the P-256 base point.

int32_t qsc_ecdsap256_publickey_from_privatekey(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: [uint8_t*] Output buffer receiving the 64-byte public key.; privatekey: [const uint8_t*] Input 32-byte private scalar..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsap256_sign

Purpose: Sign a message using a P-256 private key.

Details: Produces a 64-byte signature (r || s) prepended to the message in the signedmsg buffer. The nonce k is derived deterministically from the private key and message hash per RFC 6979 using HMAC-SHA256. Fresh entropy is still required for blinded scalar arithmetic; signing fails closed when the blinding random-number generator is unavailable.

int32_t qsc_ecdsap256_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: [uint8_t*] Output signed-message buffer (msglen + 64 bytes).; smsglen: [size_t*] Set to msglen + EC_NISTP256_SIGNATURE_SIZE on success, 0 on failure.; message: [const uint8_t*] Message to sign.; msglen: [size_t] Message length in bytes.; privatekey: [const uint8_t*] 96-byte private key (seed || pubkey)..

Return: [int32_t] 0 on success, -1 for general failure, -2 for a zero derived scalar, or -3 when blinding entropy is unavailable.

Notes: The embedded public key is not checked during signing. Imported bundles MUST be checked with qsc_ecdsap256_privatekey_validate at ingestion; otherwise the signature may not verify against the supplied public key. On failure the caller-owned signedmsg buffer (msglen + 64 bytes) is cleared, and smsglen is reset to zero. Inputs and output must not overlap.

qsc_ecdsap256_sign_scalar

Purpose: Sign a message using a P-256 private key (RFC 6979 deterministic k using scalar d).

Details: Produces a 64-byte signature (r || s) prepended to the message in the signedmsg buffer. The nonce k is derived deterministically from the private key and message hash per RFC 6979 using HMAC-SHA256. Fresh entropy is still required for blinded scalar arithmetic; signing fails closed when the blinding random-number generator is unavailable.

int32_t qsc_ecdsap256_sign_scalar(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: [uint8_t*] Output signed-message buffer (msglen + 64 bytes).; smsglen: [size_t*] Set to msglen + EC_NISTP256_SIGNATURE_SIZE on success, 0 on failure.; message: [const uint8_t*] Message to sign.; msglen: [size_t] Message length in bytes.; privatekey: [const uint8_t*] 32-byte big-endian scalar in [1, n-1]..

Return: [int32_t] 0 on success, -1 if scalar is out of range, -2 for zero scalar, or -3 when the blinding random-number generator fails. On failure the caller-owned signedmsg buffer (msglen + 64 bytes) is cleared, and smsglen is reset to zero. Inputs and output must not overlap.

qsc_ecdsap256_verify

Purpose: Verify a P-256 signed message and recover the message bytes.

Details: Verifies the 64-byte (r || s) signature prepended to signedmsg against the 64-byte public key. On success the message bytes are copied into message and msglen is set. On failure message is zeroed and msglen is set to 0.

bool qsc_ecdsap256_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: [uint8_t*] Output message buffer (at least smsglen - 64 bytes).; msglen: [size_t*] Set to the recovered message length on success.; signedmsg: [const uint8_t*] Signed-message buffer (signature || message).; smsglen: [size_t] Total signed-message length.; publickey: [const uint8_t*] 64-byte public key (Qx || Qy, big-endian)..

Return: [int32_t] 0 on success, -1 on failure.

qsc_ecdsap256_verify_context_clear

Purpose: Clear all public-key precomputation from the context.

void qsc_ecdsap256_verify_context_clear(qsc_ecdsap256_verify_context* context);

Parameters: context: Context to clear, NULL is permitted..

Return: None (void).

Notes: Do not race with other threads reading the context.

qsc_ecdsap256_verify_context_initialize

Purpose: Validate a public key and prepare a reusable verifier context.

bool qsc_ecdsap256_verify_context_initialize(qsc_ecdsap256_verify_context* context, const uint8_t* publickey);

Parameters: context: Writable caller-owned context, erased on failure.; publickey: Raw 64-byte P-256 public key, big-endian X || Y..

Return: True for a valid public key; false otherwise.

Notes: No heap or shared mutable data. Call once before concurrent verifications.

qsc_ecdsap256_verify_prehashed

Purpose: Verify a signature against a separately computed SHA-256, SHA-384 or SHA-512 digest.

Details: This API does not hash the recovered message. The caller must ensure the digest belongs to that message; otherwise signature-to-message binding is not established. All buffers remain caller-owned, must not overlap, and do not transfer ownership. This operation does not modify shared state and is thread-safe for independent buffers.

bool qsc_ecdsap256_verify_prehashed(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey, const uint8_t* digest, size_t digestlen);

Parameters: message: Receives the recovered message on success (smsglen minus signature bytes capacity).; msglen: Receives recovered message length; zero on verification failure.; signedmsg: Signature-prefixed message input.; smsglen: Input length in bytes.; publickey: Raw P-256 public key.; digest: SHA-2 digest computed over the original message by the caller.; digestlen: Exactly 32, 48 or 64 bytes; hash truncation or left-zero-extension follows FIPS 186-5..

Return: True if the signature is mathematically valid for the supplied digest.

qsc_ecdsap256_verify_prehashed_with_comb_context

Purpose: Verify with a reusable comb and an independently supplied SHA-2 digest.

Details: Does not hash or bind the appended message to the digest; the caller must establish that relationship. Buffers must be disjoint; output contents are unspecified on failure. Public-input verification only. Concurrent readers may share context with independent writable buffers. No input is retained.

bool qsc_ecdsap256_verify_prehashed_with_comb_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap256_comb_verify_context* context, const uint8_t* digest, size_t digestlen);

Parameters: message: Caller-owned output, capacity smsglen minus the signature size; non-NULL.; msglen: Caller-owned length output; zero on failure when non-NULL.; signedmsg: Read-only raw r||s followed by the associated message.; smsglen: Total signed-message size, at least EC_NISTP256_SIGNATURE_SIZE.; context: Initialized, immutable context from the matching initializer.; digest: Caller-owned SHA-256, SHA-384 or SHA-512 digest.; digestlen: Exactly 32, 48 or 64 bytes; normalized as in the prehashed API..

Return: True only if the signature matches the supplied digest.

qsc_ecdsap256_verify_with_comb_context

Purpose: Verify and recover a signed message using a reusable public-key comb.

Details: Uses the curve's standard SHA-2 digest. All buffers must be disjoint. Output contents are unspecified on failure. Public-input verification only; never supply secret scalars. Thread-safe with independent output buffers.

bool qsc_ecdsap256_verify_with_comb_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap256_comb_verify_context* context);

Parameters: message: Caller-owned output, capacity smsglen minus the signature size; non-NULL.; msglen: Caller-owned length output; zero on failure when non-NULL.; signedmsg: Read-only raw r||s followed by the message.; smsglen: Total signed-message size, at least EC_NISTP256_SIGNATURE_SIZE.; context: Initialized, immutable context from the matching initializer..

Return: True only for a valid signature; false on invalid arguments or signature.

qsc_ecdsap256_verify_with_context

Purpose: Verify a signature with one immutable precomputed public-key context.

bool qsc_ecdsap256_verify_with_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap256_verify_context* context);

Parameters: message: Recovered-message destination of at least smsglen minus 64 bytes.; msglen: Recovered length output, zero on failure.; signedmsg: Input r||s signature plus message bytes.; smsglen: Length of signedmsg in bytes.; context: Initialized, immutable context for the verifying public key..

Return: True on a valid signature, false on error or invalid signature.

Notes: Thread safe for concurrent calls with independent output buffers and an immutable context. On compilers without 128-bit words the portable verifier remains functional but does not accelerate with precomputation.

qsc_ecdsap256_verify_with_joint_context

Purpose: Verify and recover a message using a precomputed joint-window key.

bool qsc_ecdsap256_verify_with_joint_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap256_joint_verify_context* context);

Parameters: message: Recovered-message output, capacity at least signedlen-64.; msglen: Result length output, set to zero on failure.; signedmsg: Input 64-byte r||s followed by message.; smsglen: Total input length in bytes.; context: Previously initialized, immutable joint verification context..

Return: True only for a valid signature under the cached public key.

Notes: Public-input-only algorithm; the signing routines do not use this API or any joint-context arithmetic. Thread safe with immutable context and independent input/output buffers.

Header: ecdsap384.h

qsc_ecdsap384_comb_verify_context_clear

Purpose: Clear and invalidate a reusable comb context.

void qsc_ecdsap384_comb_verify_context_clear(qsc_ecdsap384_comb_verify_context* context);

Parameters: context: Caller-owned context; NULL is allowed. Exclusive access required..

Return: None (void).

qsc_ecdsap384_comb_verify_context_initialize

Purpose: Validate a P-384 public key and construct its reusable comb table.

Details: Clears context on failure when non-NULL. No retained pointers or heap allocation; thread-safe with distinct contexts. Initialization uses public data.

```
bool
qsc_ecdsap384_comb_verify_context_initialize(qsc_ecdsap384_comb_verify_context*
context, const uint8_t* publickey);
```

Parameters: context: Caller-owned destination; exclusive access required.; publickey: Exactly EC_NISTP384_PUBLICKEY_SIZE readable bytes, disjoint from context..

Return: True on success; false for NULL or noncanonical/off-curve/identity keys.

qsc_ecdsap384_keypair

Purpose: Generate a P-384 public/private key pair from a 48-byte seed.

Details: Derives a private scalar from the seed via SHA-384, reduces it into [1, n-1], computes Q = d*G using the P-384 base point, and stores both keys. The private key layout is seed[48] || Qx[48] || Qy[48].

```
int32_t qsc_ecdsap384_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t*
seed);
```

Parameters: publickey: [uint8_t*] Output public key (96 bytes: Qx || Qy, big-endian).; privatekey: [uint8_t*] Output private key (144 bytes: seed || Qx || Qy).; seed: [const uint8_t*] 48-byte random seed..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsap384_publickey_from_privatekey

Purpose: Derive a P-384 public key from a raw private scalar.

Details: This function derives the affine public point Q = dG from a 48-byte big-endian private scalar and serializes the result as the raw public-key form Qx || Qy. The private scalar must be in the range [1, n - 1], where n is the order of the P-384 base point.

```
int32_t qsc_ecdsap384_publickey_from_privatekey(uint8_t* publickey, const uint8_t*
privatekey);
```

Parameters: publickey: [uint8_t*] Output buffer receiving the 96-byte public key.; privatekey: [const uint8_t*] Input 48-byte private scalar..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsap384_sign

Purpose: Sign a message using a P-384 private key.

Details: Produces a 96-byte signature (r || s) prepended to the message in the signedmsg buffer. The nonce k is derived deterministically from the private key and message hash per RFC 6979 using HMAC-SHA384. Signing still requires CSP entropy for scalar, projective, and inversion blinding and fails if entropy generation fails. Blinding does not change the deterministic signature bytes.

int32_t qsc_ecdsap384_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: [uint8_t*] Output signed-message buffer (msglen + 96 bytes).; smsglen: [size_t*] Set to msglen + EC_NISTP384_SIGNATURE_SIZE on success, 0 on failure.; message: [const uint8_t*] Message to sign.; msglen: [size_t] Message length in bytes.; privatekey: [const uint8_t*] 144-byte private key (seed || pubkey)..

Return: [int32_t] 0 on success, -1 on failure.

qsc_ecdsap384_sign_scalar

Purpose: Sign a message using a P-384 private key scalar.

Details: Produces a 96-byte signature (r || s) prepended to the message in the signedmsg buffer. The nonce k is derived deterministically from the private key scalar and message hash per RFC 6979 using HMAC-SHA384. CSP entropy is required for blinding; entropy failure causes signing to fail without changing the deterministic nonce construction.

int32_t qsc_ecdsap384_sign_scalar(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: [uint8_t*] Output signed-message buffer (msglen + 96 bytes).; smsglen: [size_t*] Set to msglen + EC_NISTP384_SIGNATURE_SIZE on success, 0 on failure.; message: [const uint8_t*] Message to sign.; msglen: [size_t] Message length in bytes.; privatekey: [const uint8_t*] 48-byte private scalar..

Return: [int32_t] 0 on success, -1 on failure.

qsc_ecdsap384_verify

Purpose: Verify a P-384 signed message and recover the message bytes.

Details: Verifies the 96-byte (r || s) signature prepended to signedmsg against the 96-byte public key. On success the message bytes are copied into message and msglen is set. On failure message is zeroed and msglen is set to 0.

bool qsc_ecdsap384_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: [uint8_t*] Output message buffer (at least smsglen - 96 bytes).; msglen: [size_t*] Set to the recovered message length on success.; signedmsg: [const uint8_t*] Signed-message buffer (signature || message).; smsglen: [size_t] Total signed-message length.; publickey: [const uint8_t*] 96-byte public key (Qx || Qy, big-endian)..

Return: [bool] Returns true on success, false on failure.

qsc_ecdsap384_verify_context_clear

Purpose: Clear a verification context, making subsequent verification fail.

Details: Does not release storage. Requires exclusive context access.

void qsc_ecdsap384_verify_context_clear(qsc_ecdsap384_verify_context* context);

Parameters: context: Caller-owned context to clear; NULL is permitted..

Return: None (void).

qsc_ecdsap384_verify_context_initialize

Purpose: Validate a P-384 public key and build its reusable verification table.

Details: Rejects noncanonical coordinates and off-curve points. No heap or RNG. Reinitialization replaces the previous key. Requires exclusive context access.

bool qsc_ecdsap384_verify_context_initialize(qsc_ecdsap384_verify_context* context, const uint8_t* publickey);

Parameters: context: Writable caller-owned context, cleared on failure; NULL fails.; publickey: Read-only 96-byte big-endian X||Y, disjoint from context..

Return: True on success; false for NULL arguments or an invalid public key.

qsc_ecdsap384_verify_prehashed

Purpose: Verify a signature against a separately computed SHA-256, SHA-384 or SHA-512 digest.

Details: This API does not hash the recovered message. The caller must ensure the digest belongs to that message; otherwise signature-to-message binding is not established. All buffers remain caller-owned, must not overlap, and do not transfer ownership. This operation does not modify shared state and is thread-safe for independent buffers.

bool qsc_ecdsap384_verify_prehashed(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey, const uint8_t* digest, size_t digestlen);

Parameters: message: Receives the recovered message on success (smsglen minus signature bytes capacity).; msglen: Receives recovered message length; zero on verification failure.; signedmsg: Signature-prefixed message input.; smsglen: Input length in bytes.; publickey: Raw P-384 public key.; digest: SHA-2 digest computed over the original message by the caller.; digestlen: Exactly 32, 48 or 64 bytes; hash truncation or left-zero-extension follows FIPS 186-5..

Return: True if the signature is mathematically valid for the supplied digest.

qsc_ecdsap384_verify_prehashed_with_comb_context

Purpose: Verify with a reusable comb and an independently supplied SHA-2 digest.

Details: Does not hash or bind the appended message to the digest; the caller must establish that relationship. Buffers must be disjoint; output contents are unspecified on failure. Public-input verification only. Concurrent readers may share context with independent writable buffers. No input is retained.

bool qsc_ecdsap384_verify_prehashed_with_comb_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap384_comb_verify_context* context, const uint8_t* digest, size_t digestlen);

Parameters: message: Caller-owned output, capacity smsglen minus the signature size; non-NULL.; msglen: Caller-owned length output; zero on failure when non-NULL.; signedmsg: Read-only raw r||s followed by the associated message.; smsglen: Total signed-message size, at least EC_NISTP384_SIGNATURE_SIZE.; context: Initialized, immutable context from the matching initializer.; digest: Caller-owned SHA-256, SHA-384 or SHA-512 digest.; digestlen: Exactly 32, 48 or 64 bytes; normalized as in the prehashed API..

Return: True only if the signature matches the supplied digest.

qsc_ecdsap384_verify_prehashed_with_context

Purpose: Verify a supplied SHA-2 digest using a reusable public-key table.

Details: Does not hash or bind the recovered message: the caller must ensure digest belongs to that message. All storage remains caller-owned and must not overlap. With valid pointer and length arguments, an invalid signature or cleared context clears the output. Public-input variable-time operation, no heap or RNG. An immutable context may be shared concurrently when each call uses independent output buffers.

bool qsc_ecdsap384_verify_prehashed_with_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap384_verify_context* context, const uint8_t* digest, size_t digestlen);

Parameters: message: Caller-owned output of at least smsglen minus 96 bytes when smsglen is at least 96; NULL permitted only for an empty recovered message.; msglen: Required output length pointer; set to zero on failure.; signedmsg: Read-only 96-byte r||s signature followed by message bytes.; smsglen: Total signed-message size in bytes, at least 96.; context: Read-only context initialized for the verifying public key.; digest: Read-only digest of the original message, computed by the caller.; digestlen: Exactly 32, 48 or 64 bytes for SHA-256, SHA-384 or SHA-512..

Return: True when the signature is valid for the supplied digest; false otherwise.

qsc_ecdsap384_verify_with_comb_context

Purpose: Verify and recover a signed message using a reusable public-key comb.

Details: Uses the curve's standard SHA-2 digest. All buffers must be disjoint. Output contents are unspecified on failure. Public-input verification only; never supply secret scalars. Thread-safe with independent output buffers.

bool qsc_ecdsap384_verify_with_comb_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap384_comb_verify_context* context);

Parameters: message: Caller-owned output, capacity smsglen minus the signature size; non-NULL.; msglen: Caller-owned length output; zero on failure when non-NULL.; signedmsg: Read-only raw r||s followed by the message.; smsglen: Total signed-message size, at least EC_NISTP384_SIGNATURE_SIZE.; context: Initialized, immutable context from the matching initializer..

Return: True only for a valid signature; false on invalid arguments or signature.

qsc_ecdsap384_verify_with_context

Purpose: Verify and recover a SHA-384 signed message using a reusable table.

Details: The context and buffers must not overlap. With valid pointer and length arguments, an invalid signature or cleared context clears the output. No heap or RNG. Variable-time public-input arithmetic; concurrent use requires independent output buffers.

```
bool qsc_ecdsap384_verify_with_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap384_verify_context* context);
```

Parameters: message: Caller-owned output of at least smsglen minus 96 bytes when smsglen is at least 96; NULL permitted only for an empty recovered message.; msglen: Required output length pointer; set to zero on failure.; signedmsg: Read-only 96-byte r||s signature followed by message bytes.; smsglen: Total signed-message size in bytes, at least 96.; context: Read-only context initialized for the verifying public key..

Return: True on a valid signature; false for invalid arguments, context or signature.

Header: ecdsap521.h

qsc_ecdsap521_comb_verify_context_clear

Purpose: Clear and invalidate a reusable comb context.

```
void qsc_ecdsap521_comb_verify_context_clear(qsc_ecdsap521_comb_verify_context* context);
```

Parameters: context: Caller-owned context; NULL is allowed. Exclusive access required..

Return: None (void).

qsc_ecdsap521_comb_verify_context_initialize

Purpose: Validate a P-521 public key and construct its reusable comb table.

Details: Clears context on failure when non-NULL. No retained pointers or heap allocation; thread-safe with distinct contexts. Initialization uses public data.

```
bool qsc_ecdsap521_comb_verify_context_initialize(qsc_ecdsap521_comb_verify_context* context, const uint8_t* publickey);
```

Parameters: context: Caller-owned destination; exclusive access required.; publickey: Exactly EC_NISTP521_PUBLICKEY_SIZE readable bytes, disjoint from context..

Return: True on success; false for NULL or noncanonical/off-curve/identity keys.

qsc_ecdsap521_keypair

Purpose: Generate a P-521 public/private key pair from a 66-byte seed.

Details: The seed is hashed with SHA-512 and reduced modulo the P-521 subgroup order to derive the signing scalar. The public key is then computed as Q = dG. The output private key layout is seed[66] || Qx[66] || Qy[66].

int32_t qsc_ecdsap521_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: [uint8_t*] Output public key (132 bytes).; privatekey: [uint8_t*] Output private key (198 bytes).; seed: [const uint8_t*] Input 66-byte seed..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsap521_publickey_from_privatekey

Purpose: Derive a P-521 public key from a raw private scalar.

Details: This function derives the affine public point Q = dG from a 66-byte big-endian private scalar and serializes the result as Qx || Qy.

int32_t qsc_ecdsap521_publickey_from_privatekey(uint8_t* publickey, const uint8_t* privatekey);

Parameters: publickey: [uint8_t*] Output buffer receiving the 132-byte public key.; privatekey: [const uint8_t*] Input 66-byte private scalar..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsap521_sign

Purpose: Sign a message using a P-521 private key.

Details: The message is hashed with SHA-512. The deterministic nonce is generated using RFC 6979 with HMAC-SHA512. The output signed-message buffer contains the 132-byte signature prepended to the message.

int32_t qsc_ecdsap521_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: [uint8_t*] Output signed-message buffer.; smsglen: [size_t*] Set to msglen + EC_NISTP521_SIGNATURE_SIZE on success.; message: [const uint8_t*] Message to sign.; msglen: [size_t] Message length in bytes.; privatekey: [const uint8_t*] 198-byte private key (seed || pubkey)..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsap521_sign_scalar

Purpose: Sign a message using a raw P-521 private scalar.

Details: The input private key is the 66-byte big-endian signing scalar d. This function exists so the implementation can be tested directly against RFC 6979 known-answer vectors, which specify the scalar explicitly.

int32_t qsc_ecdsap521_sign_scalar(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: [uint8_t*] Output signed-message buffer.; smsglen: [size_t*] Set to msglen + EC_NISTP521_SIGNATURE_SIZE on success.; message: [const uint8_t*] Message to sign.; msglen: [size_t] Message length in bytes.; privatekey: [const uint8_t*] 66-byte private scalar..

Return: [int32_t] Returns 0 on success, or a negative error code on failure.

qsc_ecdsap521_verify

Purpose: Verify a P-521 signed message and recover the message bytes.

bool qsc_ecdsap521_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: [uint8_t*] Output message buffer.; msglen: [size_t*] Set to the recovered message length on success.; signedmsg: [const uint8_t*] Signed-message buffer (signature || message).; smsglen: [size_t] Total signed-message length.; publickey: [const uint8_t*] 132-byte raw public key (Qx || Qy)..

Return: [bool] Returns true on success, false on failure.

qsc_ecdsap521_verify_prehashed

Purpose: Verify a signature against a separately computed SHA-256, SHA-384 or SHA-512 digest.

Details: This API does not hash the recovered message. The caller must ensure the digest belongs to that message; otherwise signature-to-message binding is not established. All buffers remain caller-owned, must not overlap, and do not transfer ownership. This operation does not modify shared state and is thread-safe for independent buffers.

bool qsc_ecdsap521_verify_prehashed(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey, const uint8_t* digest, size_t digestlen);

Parameters: message: Receives the recovered message on success (smsglen minus signature bytes capacity).; msglen: Receives recovered message length; zero on verification failure.; signedmsg: Signature-prefixed message input.; smsglen: Input length in bytes.; publickey: Raw P-521 public key.; digest: SHA-2 digest computed over the original message by the caller.; digestlen: Exactly 32, 48 or 64 bytes; hash truncation or left-zero-extension follows FIPS 186-5..

Return: True if the signature is mathematically valid for the supplied digest.

qsc_ecdsap521_verify_prehashed_with_comb_context

Purpose: Verify with a reusable comb and an independently supplied SHA-2 digest.

Details: Does not hash or bind the appended message to the digest; the caller must establish that relationship. Buffers must be disjoint; output contents are unspecified on failure. Public-input verification only. Concurrent readers may share context with independent writable buffers. No input is retained.

bool qsc_ecdsap521_verify_prehashed_with_comb_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap521_comb_verify_context* context, const uint8_t* digest, size_t digestlen);

Parameters: message: Caller-owned output, capacity smsglen minus the signature size; non-NULL.; msglen: Caller-owned length output; zero on failure when non-NULL.; signedmsg: Read-only raw r||s followed by the associated message.; smsglen: Total signed-message size, at least EC_NISTP521_SIGNATURE_SIZE.; context: Initialized, immutable context from the matching initializer.; digest: Caller-owned SHA-256, SHA-384 or SHA-512 digest.; digestlen: Exactly 32, 48 or 64 bytes; normalized as in the prehashed API..

Return: True only if the signature matches the supplied digest.

qsc_ecdsap521_verify_with_comb_context

Purpose: Verify and recover a signed message using a reusable public-key comb.

Details: Uses the curve's standard SHA-2 digest. All buffers must be disjoint. Output contents are unspecified on failure. Public-input verification only; never supply secret scalars. Thread-safe with independent output buffers.

bool qsc_ecdsap521_verify_with_comb_context(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const qsc_ecdsap521_comb_verify_context* context);

Parameters: message: Caller-owned output, capacity smsglen minus the signature size; non-NULL.; msglen: Caller-owned length output; zero on failure when non-NULL.; signedmsg: Read-only raw r||s followed by the message.; smsglen: Total signed-message size, at least EC_NISTP521_SIGNATURE_SIZE.; context: Initialized, immutable context from the matching initializer..

Return: True only for a valid signature; false on invalid arguments or signature.

Header: eddsa.h

qsc_eddsa_generate_keypair

Purpose: Generates a EDDSA public/private key-pair.

void qsc_eddsa_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: [uint8_t*] Pointer to the public verification-key array; privatekey: [uint8_t*] Pointer to the private signature-key array; rng_generate: [uint8_t*, size_t] Pointer to the random generator.

Return: None (void).

Notes: Arrays must be sized to QSC_EDDSA_PUBLICKEY_SIZE and QSC_EDDSA_PRIVATEKEY_SIZE.

qsc_eddsa_generate_seeded_keypair

Purpose: Generates a EDDSA public/private key-pair.

void qsc_eddsa_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: [uint8_t*] Pointer to the public verification-key array; privatekey: [uint8_t*] Pointer to the private signature-key array; seed: [const uint8_t*] Pointer to the random QSC_EDDSA_SEED_SIZE-byte seed array.

Return: None (void).

Notes: Arrays must be sized to QSC_EDDSA_PUBLICKEY_SIZE and QSC_EDDSA_PRIVATEKEY_SIZE.

qsc_eddsa_sign

Purpose: Takes the message as input and returns an array containing the signature followed by the message.

void qsc_eddsa_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: [uint8_t*] Pointer to the signed-message array; smsglen: [size_t*] Pointer to the signed message length; message: [const uint8_t*] Pointer to the message array; msglen: [size_t] The message length; privatekey: [const uint8_t*] Pointer to the private signature-key array.

Return: None (void).

Notes: Signature array must be sized to the size of the message plus QSC_EDDSA_SIGNATURE_SIZE.

qsc_eddsa_verify

Purpose: Verifies a signature-message pair with the public key.

bool qsc_eddsa_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: [uint8_t*] Pointer to the message array to be signed; msglen: [size_t*]Pointer to the message length; signedmsg: [const uint8_t*] Pointer to the signed message array; smsglen: [size_t] The signed message length; publickey: [const uint8_t*] Pointer to the public verification-key array.

Return: [bool] Returns true for success

Header: eddsa25519.h

qsc_ed25519_generate_keypair

Purpose: Generate an Ed25519 key pair from a caller-supplied RNG.

Details: Key buffers must not overlap. RNG failure clears both output buffers. No dynamic ownership is transferred. The callback's thread safety is the caller's responsibility.

void qsc_ed25519_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output array, QSC_ED25519_PUBLICKEY_SIZE bytes.; privatekey: Output array, QSC_ED25519_PRIVATEKEY_SIZE bytes.; rng_generate: Non-NULL callback writing the requested number of bytes; returns true on success..

Return: None (void).

qsc_ed25519_generate_seeded_keypair

Purpose: Generate an Ed25519 key pair deterministically from a seed.

Details: Caller retains ownership. Inputs and outputs must not overlap.

void qsc_ed25519_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: Output array, QSC_ED25519_PUBLICKEY_SIZE bytes.; privatekey: Output array, QSC_ED25519_PRIVATEKEY_SIZE bytes.; seed: Input array, QSC_ED25519_SEED_SIZE bytes..

Return: None (void).

qsc_ed25519_sign

Purpose: Sign a message and prepend its Ed25519 signature.

Details: Buffers are caller-owned; use disjoint signedmsg and message buffers.

bool qsc_ed25519_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: Output array, at least msglen + QSC_ED25519_SIGNATURE_SIZE bytes.; smsglen: Output length of signed message in bytes; set on success.; message:

Input message, msglen bytes.; msglen: Message length in bytes.; privatekey: Input array, QSC_ED25519_PRIVATEKEY_SIZE bytes..

Return: true on successful signing; false on failure.

qsc_ed25519_verify

Purpose: Verify a signature-prefixed message and recover its plaintext.

Details: Caller owns all buffers; recovery output must not overlap signedmsg.

bool qsc_ed25519_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: Output array, at least smsglen - QSC_ED25519_SIGNATURE_SIZE bytes on valid input.; msglen: Output recovered-message length in bytes.; signedmsg: Input signature-prefixed message of smsglen bytes.; smsglen: Signed-message length including QSC_ED25519_SIGNATURE_SIZE bytes of signature.; publickey: Input array, QSC_ED25519_PUBLICKEY_SIZE bytes..

Return: true when signature is valid; false when invalid or malformed.

Header: eddsa448.h

qsc_ed448_generate_keypair

Purpose: Generate an Ed448 key pair from a caller-supplied RNG.

Details: Key buffers must not overlap. RNG failure clears both output buffers. No dynamic ownership is transferred. The callback's thread safety is the caller's responsibility.

void qsc_ed448_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output array, QSC_ED448_PUBLICKEY_SIZE bytes.; privatekey: Output array, QSC_ED448_PRIVATEKEY_SIZE bytes.; rng_generate: Non-NULL callback writing the requested number of bytes; returns true on success..

Return: None (void).

qsc_ed448_generate_seeded_keypair

Purpose: Generate an Ed448 key pair deterministically from a seed.

Details: Caller retains ownership. Inputs and outputs must not overlap.

void qsc_ed448_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t* seed);

Parameters: publickey: Output array, QSC_ED448_PUBLICKEY_SIZE bytes.; privatekey: Output array, QSC_ED448_PRIVATEKEY_SIZE bytes.; seed: Input array, QSC_ED448_SEED_SIZE bytes..

Return: None (void).

qsc_ed448_sign

Purpose: Sign a message and prepend its Ed448 signature.

Details: Buffers are caller-owned; use disjoint signedmsg and message buffers.

bool qsc_ed448_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey);

Parameters: signedmsg: Output array, at least msglen + QSC_ED448_SIGNATURE_SIZE bytes.; smsglen: Output length of signed message in bytes; set on success.; message: Input message, msglen bytes.; msglen: Message length in bytes.; privatekey: Input array, QSC_ED448_PRIVATEKEY_SIZE bytes..

Return: true on successful signing; false on failure.

qsc_ed448_verify

Purpose: Verify a signature-prefixed message and recover its plaintext.

Details: Caller owns all buffers; recovery output must not overlap signedmsg.

bool qsc_ed448_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: Output array, at least smsglen - QSC_ED448_SIGNATURE_SIZE bytes on valid input.; msglen: Output recovered-message length in bytes.; signedmsg: Input signature-prefixed message of smsglen bytes.; smsglen: Signed-message length including QSC_ED448_SIGNATURE_SIZE bytes of signature.; publickey: Input array, QSC_ED448_PUBLICKEY_SIZE bytes..

Return: true when signature is valid; false when invalid or malformed.

Header: falcon1024.h

qsc_falcon1024_generate_keypair

Purpose: Generate a randomized Falcon-1024 keypair.

bool qsc_falcon1024_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output array of QSC_FALCON1024_PUBLICKEY_SIZE bytes.; privatekey: Output array of QSC_FALCON1024_PRIVATEKEY_SIZE bytes.; rng_generate: Non-NULL RNG callback; returns false on failure..

Return: True on success; false on invalid arguments or backend failure.

Notes: The caller owns all buffers. No global mutable state is used.

qsc_falcon1024_sign

Purpose: Create a legacy Falcon-1024 signed message.

bool qsc_falcon1024_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signedmsg: Caller-allocated output of at least msglen + QSC_FALCON1024_SIGNATURE_SIZE bytes.; smsglen: Output actual signed-message length in bytes; reset to zero on failure.; message: Non-NULL input message buffer containing msglen bytes.; msglen: Message length in bytes; the addition of overhead must not overflow size_t.; privatekey: Input array of QSC_FALCON1024_PRIVATEKEY_SIZE bytes.; rng_generate: Non-NULL RNG callback used for nonce and signing randomness..

Return: True on success; false on malformed key, RNG failure or invalid arguments.

Notes: The caller owns the buffers; no global mutable state is used.

qsc_falcon1024_verify

Purpose: Verify and recover a message from a Falcon-1024 signed message.

bool qsc_falcon1024_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: Caller-allocated output buffer of at least smsglen bytes.; msglen: Output recovered length in bytes; reset to zero on failure.; signedmsg: Input signed message of smsglen bytes.; smsglen: Input signed-message length.; publickey: Input array of QSC_FALCON1024_PUBLICKEY_SIZE bytes..

Return: True on successful verification; false for malformed or invalid signatures.

Notes: Message data is only copied on success; no global mutable state is used.

Header: falcon512.h

qsc_falcon512_generate_keypair

Purpose: Generate a randomized Falcon-512 keypair.

bool qsc_falcon512_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output array of QSC_FALCON512_PUBLICKEY_SIZE bytes.; privatekey: Output array of QSC_FALCON512_PRIVATEKEY_SIZE bytes.; rng_generate: Non-NULL RNG callback; returns false on failure..

Return: True on success; false on invalid arguments or backend failure.

Notes: The caller owns all buffers. No global mutable state is used.

qsc_falcon512_sign

Purpose: Create a legacy Falcon-512 signed message.

bool qsc_falcon512_sign(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signedmsg: Caller-allocated output of at least msglen + QSC_FALCON512_SIGNATURE_SIZE bytes.; smsglen: Output actual signed-message length in bytes; reset to zero on failure.; message: Non-NULL input message buffer containing msglen bytes.; msglen: Message length in bytes; the addition of overhead must not overflow size_t.; privatekey: Input array of QSC_FALCON512_PRIVATEKEY_SIZE bytes.; rng_generate: Non-NULL RNG callback used for nonce and signing randomness..

Return: True on success; false on malformed key, RNG failure or invalid arguments.

Notes: The caller owns the buffers; no global mutable state is used.

qsc_falcon512_verify

Purpose: Verify and recover a message from a Falcon-512 signed message.

bool qsc_falcon512_verify(uint8_t* message, size_t* msglen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: Caller-allocated output buffer of at least smsglen bytes.; msglen: Output recovered length in bytes; reset to zero on failure.; signedmsg: Input signed message of smsglen bytes.; smsglen: Input signed-message length.; publickey: Input array of QSC_FALCON512_PUBLICKEY_SIZE bytes..

Return: True on successful verification; false for malformed or invalid signatures.

Notes: Message data is only copied on success; no global mutable state is used.

Header: mldsa.h

qsc_mldsa44_generate_keypair

Purpose: Generate an ML-DSA-44 key pair with caller RNG.

bool qsc_mldsa44_generate_keypair(uint8_t* pk, uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: pk: Caller-owned output, QSC_MLDSA44_PUBLICKEY_SIZE bytes.; sk: Caller-owned secret output, QSC_MLDSA44_PRIVATEKEY_SIZE bytes.; rng_generate: Random-byte callback, true on success; must be thread safe if called concurrently..

Return: True on successful generation; false on invalid argument or RNG failure.

Notes: Non-thread-safe if the supplied RNG is non-thread-safe; caller must erase the secret key.

qsc_mldsa44_seeded_generate_keypair

Purpose: Deterministically derive an ML-DSA-44 key pair from a 32-byte seed.

void qsc_mldsa44_seeded_generate_keypair(uint8_t* pk, uint8_t* sk, const uint8_t* seed);

Parameters: pk: Output, QSC_MLDSA44_PUBLICKEY_SIZE bytes.; sk: Secret output, QSC_MLDSA44_PRIVATEKEY_SIZE bytes.; seed: Input array of QSC_MLDSA_SEED_SIZE bytes..

Return: None (void).

Notes: Not thread-sensitive; caller owns and erases keys; invalid pointers perform no operation.

qsc_mldsa44_sign

Purpose: Sign a message with ML-DSA-44; append message to signature.

bool qsc_mldsa44_sign(uint8_t* sm, size_t* smlen, const uint8_t* msg, size_t msglen, const uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: sm: Caller-owned output, at least QSC_MLDSA44_SIGNATURE_SIZE + msglen bytes.; smlen: Receives output length.; msg: Input message, msglen bytes.; msglen: Message length in bytes.; sk: Secret signing key, QSC_MLDSA44_PRIVATEKEY_SIZE bytes.; rng_generate: RNG callback needed for hedged signing; ignored in deterministic mode..

Return: True on signing success; false on invalid arguments, rejected length or RNG failure.

Notes: Use sign_ex for context binding. Caller owns buffers, protect secret and output aliasing.

qsc_mldsa44_sign_ex

Purpose: Sign using explicit ML-DSA-44 context binding.

bool qsc_mldsa44_sign_ex(uint8_t* sm, size_t* smlen, const uint8_t* msg, size_t msglen, const uint8_t* ctx, size_t ctxlen, const uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: sm: Output, QSC_MLDSA44_SIGNATURE_SIZE + msglen bytes.; smlen: Receives signed output length.; msg: Message input, msglen bytes.; msglen: Message length.; ctx: Optional context string, at most 255 bytes; non-null when ctxlen is nonzero.; ctxlen: Context length (0 to 255 bytes).; sk: Signing key, QSC_MLDSA44_PRIVATEKEY_SIZE bytes.; rng_generate: RNG callback for randomized signing, or null in deterministic builds..

Return: True on successful signing; false on invalid parameters, RNG or backend failure.

Notes: Caller owns outputs; thread safety depends on the supplied RNG.

qsc_mldsa44_verify

Purpose: Verify an ML-DSA-44 signature and recover its attached message.

bool qsc_mldsa44_verify(uint8_t* msg, size_t* msglen, const uint8_t* sm, size_t smlen, const uint8_t* pk);

Parameters: msg: Caller-owned message output, at least smlen - signature-size bytes when valid.; msglen: Receives verified message length, zero on failure.; sm: Input signature followed by message.; smlen: Total input length.; pk: Public verification key, QSC_MLDSA44_PUBLICKEY_SIZE bytes..

Return: True on a valid signature, otherwise false.

Notes: No output size is inferred from pointer; caller must allocate appropriately.

qsc_mldsa44_verify_ex

Purpose: Verify an ML-DSA-44 signed message with context binding.

bool qsc_mldsa44_verify_ex(uint8_t* msg, size_t* msglen, const uint8_t* ctx, size_t ctxlen, const uint8_t* sm, size_t smlen, const uint8_t* pk);

Parameters: msg: Caller-owned recovered message output.; msglen: Receives output length; zero on failure.; ctx: Context input, non-null when ctxlen is nonzero.; ctxlen: Context length in the range 0 to 255 bytes.; sm: Signed message input.; smlen: Signed-message length.; pk: Public verification key, QSC_MLDSA44_PUBLICKEY_SIZE bytes..

Return: True if signature, message and context match; otherwise false.

Notes: No mutable shared state; output owned by caller.

qsc_mldsa65_generate_keypair

Purpose: Generate an ML-DSA-65 key pair with caller RNG.

bool qsc_mldsa65_generate_keypair(uint8_t* pk, uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: pk: Caller-owned output, QSC_MLDSA65_PUBLICKEY_SIZE bytes.; sk: Caller-owned secret output, QSC_MLDSA65_PRIVATEKEY_SIZE bytes.; rng_generate: Random-byte callback, true on success; must be thread safe if called concurrently..

Return: True on successful generation; false on invalid argument or RNG failure.

Notes: Non-thread-safe if the supplied RNG is non-thread-safe; caller must erase the secret key.

qsc_mldsa65_seeded_generate_keypair

Purpose: Deterministically derive an ML-DSA-65 key pair from a 32-byte seed.

void qsc_mldsa65_seeded_generate_keypair(uint8_t* pk, uint8_t* sk, const uint8_t* seed);

Parameters: pk: Output, QSC_MLDSA65_PUBLICKEY_SIZE bytes.; sk: Secret output, QSC_MLDSA65_PRIVATEKEY_SIZE bytes.; seed: Input array of QSC_MLDSA_SEED_SIZE bytes..

Return: None (void).

Notes: Not thread-sensitive; caller owns and erases keys; invalid pointers perform no operation.

qsc_mldsa65_sign

Purpose: Sign a message with ML-DSA-65; append message to signature.

bool qsc_mldsa65_sign(uint8_t* sm, size_t* smlen, const uint8_t* msg, size_t msglen, const uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: sm: Caller-owned output, at least QSC_MLDSA65_SIGNATURE_SIZE + msglen bytes.; smlen: Receives output length.; msg: Input message, msglen bytes.; msglen: Message length in bytes.; sk: Secret signing key, QSC_MLDSA65_PRIVATEKEY_SIZE bytes.; rng_generate: RNG callback needed for hedged signing; ignored in deterministic mode..

Return: True on signing success; false on invalid arguments, rejected length or RNG failure.

Notes: Use sign_ex for context binding. Caller owns buffers, protect secret and output aliasing.

qsc_mldsa65_sign_ex

Purpose: Sign using explicit ML-DSA-65 context binding.

bool qsc_mldsa65_sign_ex(uint8_t* sm, size_t* smlen, const uint8_t* msg, size_t msglen, const uint8_t* ctx, size_t ctxlen, const uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: sm: Output, QSC_MLDSA65_SIGNATURE_SIZE + msglen bytes.; smlen: Receives signed output length.; msg: Message input, msglen bytes.; msglen: Message length.; ctx: Optional context string, at most 255 bytes; non-null when ctxlen is nonzero.; ctxlen: Context length (0 to 255 bytes).; sk: Signing key, QSC_MLDSA65_PRIVATEKEY_SIZE bytes.; rng_generate: RNG callback for randomized signing, or null in deterministic builds..

Return: True on successful signing; false on invalid parameters, RNG or backend failure.

Notes: Caller owns outputs; thread safety depends on the supplied RNG.

qsc_mldsa65_verify

Purpose: Verify an ML-DSA-65 signature and recover its attached message.

bool qsc_mldsa65_verify(uint8_t* msg, size_t* msglen, const uint8_t* sm, size_t smlen, const uint8_t* pk);

Parameters: msg: Caller-owned message output, at least smlen - signature-size bytes when valid.; msglen: Receives verified message length, zero on failure.; sm: Input signature followed by message.; smlen: Total input length.; pk: Public verification key, QSC_MLDSA65_PUBLICKEY_SIZE bytes..

Return: True on a valid signature, otherwise false.

Notes: No output size is inferred from pointer; caller must allocate appropriately.

qsc_mldsa65_verify_ex

Purpose: Verify an ML-DSA-65 signed message with context binding.

bool qsc_mldsa65_verify_ex(uint8_t* msg, size_t* msglen, const uint8_t* ctx, size_t ctxlen, const uint8_t* sm, size_t smlen, const uint8_t* pk);

Parameters: msg: Caller-owned recovered message output.; msglen: Receives output length; zero on failure.; ctx: Context input, non-null when ctxlen is nonzero.; ctxlen: Context length in the range 0 to 255 bytes.; sm: Signed message input.; smlen: Signed-message length.; pk: Public verification key, QSC_MLDSA65_PUBLICKEY_SIZE bytes..

Return: True if signature, message and context match; otherwise false.

Notes: No mutable shared state; output owned by caller.

qsc_mldsa87_generate_keypair

Purpose: Generate an ML-DSA-87 key pair with caller RNG.

bool qsc_mldsa87_generate_keypair(uint8_t* pk, uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: pk: Caller-owned output, QSC_MLDSA87_PUBLICKEY_SIZE bytes.; sk: Caller-owned secret output, QSC_MLDSA87_PRIVATEKEY_SIZE bytes.; rng_generate: Random-byte callback, true on success; must be thread safe if called concurrently..

Return: True on successful generation; false on invalid argument or RNG failure.

Notes: Non-thread-safe if the supplied RNG is non-thread-safe; caller must erase the secret key.

qsc_mldsa87_seeded_generate_keypair

Purpose: Deterministically derive an ML-DSA-87 key pair from a 32-byte seed.

void qsc_mldsa87_seeded_generate_keypair(uint8_t* pk, uint8_t* sk, const uint8_t* seed);

Parameters: pk: Output, QSC_MLDSA87_PUBLICKEY_SIZE bytes.; sk: Secret output, QSC_MLDSA87_PRIVATEKEY_SIZE bytes.; seed: Input array of QSC_MLDSA_SEED_SIZE bytes..

Return: None (void).

Notes: Not thread-sensitive; caller owns and erases keys; invalid pointers perform no operation.

qsc_mldsa87_sign

Purpose: Sign a message with ML-DSA-87; append message to signature.

bool qsc_mldsa87_sign(uint8_t* sm, size_t* smlen, const uint8_t* msg, size_t msglen, const uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: sm: Caller-owned output, at least QSC_MLDSA87_SIGNATURE_SIZE + msglen bytes.; smlen: Receives output length.; msg: Input message, msglen bytes.; msglen: Message length in bytes.; sk: Secret signing key, QSC_MLDSA87_PRIVATEKEY_SIZE bytes.; rng_generate: RNG callback needed for hedged signing; ignored in deterministic mode..

Return: True on signing success; false on invalid arguments, rejected length or RNG failure.

Notes: Use sign_ex for context binding. Caller owns buffers, protect secret and output aliasing.

qsc_mldsa87_sign_ex

Purpose: Sign using explicit ML-DSA-87 context binding.

bool qsc_mldsa87_sign_ex(uint8_t* sm, size_t* smlen, const uint8_t* msg, size_t msglen, const uint8_t* ctx, size_t ctxlen, const uint8_t* sk, bool (*rng_generate)(uint8_t*, size_t));

Parameters: sm: Output, QSC_MLDSA87_SIGNATURE_SIZE + msglen bytes.; smlen: Receives signed output length.; msg: Message input, msglen bytes.; msglen: Message length.; ctx: Optional context string, at most 255 bytes; non-null when ctxlen is nonzero.; ctxlen: Context length (0 to 255 bytes).; sk: Signing key,

QSC_MLDSA87_PRIVATEKEY_SIZE bytes.; rng_generate: RNG callback for randomized signing, or null in deterministic builds..

Return: True on successful signing; false on invalid parameters, RNG or backend failure.

Notes: Caller owns outputs; thread safety depends on the supplied RNG.

qsc_mldsa87_verify

Purpose: Verify an ML-DSA-87 signature and recover its attached message.

bool qsc_mldsa87_verify(uint8_t* msg, size_t* msglen, const uint8_t* sm, size_t smlen, const uint8_t* pk);

Parameters: msg: Caller-owned message output, at least smlen - signature-size bytes when valid.; msglen: Receives verified message length, zero on failure.; sm: Input signature followed by message.; smlen: Total input length.; pk: Public verification key, QSC_MLDSA87_PUBLICKEY_SIZE bytes..

Return: True on a valid signature, otherwise false.

Notes: No output size is inferred from pointer; caller must allocate appropriately.

qsc_mldsa87_verify_ex

Purpose: Verify an ML-DSA-87 signed message with context binding.

bool qsc_mldsa87_verify_ex(uint8_t* msg, size_t* msglen, const uint8_t* ctx, size_t ctxlen, const uint8_t* sm, size_t smlen, const uint8_t* pk);

Parameters: msg: Caller-owned recovered message output.; msglen: Receives output length; zero on failure.; ctx: Context input, non-null when ctxlen is nonzero.; ctxlen: Context length in the range 0 to 255 bytes.; sm: Signed message input.; smlen: Signed-message length.; pk: Public verification key, QSC_MLDSA87_PUBLICKEY_SIZE bytes..

Return: True if signature, message and context match; otherwise false.

Notes: No mutable shared state; output owned by caller.

Header: slhdsa.h

qsc_slhdsa_shake128f_generate_keypair

Purpose: Generates the SLH-DSA-SHAKE-128f key pair with a random generator.

bool qsc_slhdsa_shake128f_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output buffer, at least 32 bytes.; privatekey: Output buffer, at least 64 bytes; contains secret material.; rng_generate: Cryptographically secure generator, return true on success..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake128f_generate_seeded_keypair

Purpose: Generates a repeatable SLH-DSA-SHAKE-128f key pair from 3*n bytes.

bool qsc_slhdsa_shake128f_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t seed[QSC_SLHDSA_SHAKE128F_KEYGEN_SEED_SIZE]);

Parameters: publickey: Output buffer, at least 32 bytes.; privatekey: Output buffer, at least 64 bytes.; seed: Exactly 48 bytes, caller-owned and not modified..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake128f_sign

Purpose: Creates a FIPS 205 pure hedged detached signature with RNG-supplied randomizer.

bool qsc_slhdsa_shake128f_sign(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer, at least 17088 bytes.; siglen: Receives 17088 on success, zero on failure.; message: Message pointer; NULL allowed only if msglen is zero.; msglen: Message length in bytes.; context: Optional context, at most 255 bytes; NULL iff ctxlen is zero.; ctxlen: Context length in bytes.; privatekey: Input 64-byte secret key.; rng_generate: Secure randomizer generator, must return true..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake128f_sign_deterministic

Purpose: Creates a FIPS 205 pure deterministic detached signature using PK.seed as randomizer.

bool qsc_slhdsa_shake128f_sign_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey);

Parameters: signature: Output buffer, at least 17088 bytes.; siglen: Receives 17088 on success, zero on failure.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Context pointer; NULL permitted only when ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; privatekey: Input 64-byte secret key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake128f_sign_prehash

Purpose: HashSLH-DSA-SHAKE-128f hedged signing with FIPS 205 prehash domain.

bool qsc_slhdsa_shake128f_sign_prehash(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned plaintext, NULL only for zero length.; msglen: Input message length.; context: Caller-owned context, NULL only if ctxlen zero.; ctxlen: Context length, maximum 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key.; rng_generate: Secure n-byte randomizer generator..

Return: True only when signing succeeds.

Notes: Reentrant; internal prehash/digest and randomizer securely erased.

qsc_slhdsa_shake128f_sign_prehash_deterministic

Purpose: HashSLH-DSA-SHAKE-128f deterministic prehash signing.

bool qsc_slhdsa_shake128f_sign_prehash_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey);

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned input, NULL only when msglen zero.; msglen: Input message length.; context: Caller-owned context, NULL only for empty context.; ctxlen: Context length, at most 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key..

Return: True on success, false on invalid inputs or signing failure.

Notes: Reentrant; deterministic signing uses private key's PK.seed.

qsc_slhdsa_shake128f_verify

Purpose: Verifies an exactly 17088-byte pure detached SLH-DSA-SHAKE-128f signature.

bool qsc_slhdsa_shake128f_verify(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* publickey);

Parameters: signature: Exactly 17088 bytes, caller-owned.; siglen: Must equal 17088.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Optional context; NULL iff ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; publickey: Input 32-byte public key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake128f_verify_prehash

Purpose: Verify a FIPS 205 HashSLH-DSA-SHAKE-128f signature.

bool qsc_slhdsa_shake128f_verify_prehash(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* publickey);

Parameters: signature: Fixed-size caller-owned detached signature.; siglen: Signature length; must match the selected parameter set.; message: Caller-owned message; NULL only for empty message.; msglen: Input message length.; context: Caller-owned context; NULL only for empty context.; ctxlen: Context length, maximum 255 bytes.; algorithm: Hash function used by signer to construct digest and OID.; publickey: Caller-owned public key..

Return: True only if the prehash-mode signature verifies.

Notes: Reentrant; pure-mode signatures are never accepted as prehash signatures.

qsc_slhdsa_shake128s_generate_keypair

Purpose: Generates the SLH-DSA-SHAKE-128s key pair with a random generator.

bool qsc_slhdsa_shake128s_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output buffer, at least 32 bytes.; privatekey: Output buffer, at least 64 bytes; contains secret material.; rng_generate: Cryptographically secure generator, return true on success..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake128s_generate_seeded_keypair

Purpose: Generates a repeatable SLH-DSA-SHAKE-128s key pair from 3*n bytes.

bool qsc_slhdsa_shake128s_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t seed[QSC_SLHDSA_SHAKE128S_KEYGEN_SEED_SIZE]);

Parameters: publickey: Output buffer, at least 32 bytes.; privatekey: Output buffer, at least 64 bytes.; seed: Exactly 48 bytes, caller-owned and not modified..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake128s_sign

Purpose: Creates a FIPS 205 pure hedged detached signature with RNG-supplied randomizer.

bool qsc_slhdsa_shake128s_sign(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer, at least 7856 bytes.; siglen: Receives 7856 on success, zero on failure.; message: Message pointer; NULL allowed only if msglen is zero.; msglen: Message length in bytes.; context: Optional context, at most 255 bytes;

NULL iff ctxlen is zero.; ctxlen: Context length in bytes.; privatekey: Input 64-byte secret key.; rng_generate: Secure randomizer generator, must return true..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake128s_sign_deterministic

Purpose: Creates a FIPS 205 pure deterministic detached signature using PK.seed as randomizer.

bool qsc_slhdsa_shake128s_sign_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey);

Parameters: signature: Output buffer, at least 7856 bytes.; siglen: Receives 7856 on success, zero on failure.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Context pointer; NULL permitted only when ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; privatekey: Input 64-byte secret key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake128s_sign_prehash

Purpose: HashSLH-DSA-SHAKE-128s hedged signing with FIPS 205 prehash domain.

bool qsc_slhdsa_shake128s_sign_prehash(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned plaintext, NULL only for zero length.; msglen: Input message length.; context: Caller-owned context, NULL only if ctxlen zero.; ctxlen: Context length, maximum 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key.; rng_generate: Secure n-byte randomizer generator..

Return: True only when signing succeeds.

Notes: Reentrant; internal prehash/digest and randomizer securely erased.

qsc_slhdsa_shake128s_sign_prehash_deterministic

Purpose: HashSLH-DSA-SHAKE-128s deterministic prehash signing.

bool qsc_slhdsa_shake128s_sign_prehash_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey);

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned input, NULL only when msglen zero.; msglen: Input message length.; context: Caller-owned context, NULL only for empty context.; ctxlen: Context length, at most 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key..

Return: True on success, false on invalid inputs or signing failure.

Notes: Reentrant; deterministic signing uses private key's PK.seed.

qsc_slhdsa_shake128s_verify

Purpose: Verifies an exactly 7856-byte pure detached SLH-DSA-SHAKE-128s signature.

bool qsc_slhdsa_shake128s_verify(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* publickey);

Parameters: signature: Exactly 7856 bytes, caller-owned.; siglen: Must equal 7856.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Optional context; NULL iff ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; publickey: Input 32-byte public key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake128s_verify_prehash

Purpose: Verify a FIPS 205 HashSLH-DSA-SHAKE-128s signature.

bool qsc_slhdsa_shake128s_verify_prehash(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* publickey);

Parameters: signature: Fixed-size caller-owned detached signature.; siglen: Signature length; must match the selected parameter set.; message: Caller-owned message; NULL only for empty message.; msglen: Input message length.; context: Caller-owned context; NULL only for empty context.; ctxlen: Context length, maximum 255 bytes.; algorithm: Hash function used by signer to construct digest and OID.; publickey: Caller-owned public key..

Return: True only if the prehash-mode signature verifies.

Notes: Reentrant; pure-mode signatures are never accepted as prehash signatures.

qsc_slhdsa_shake192f_generate_keypair

Purpose: Generates the SLH-DSA-SHAKE-192f key pair with a random generator.

bool qsc_slhdsa_shake192f_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output buffer, at least 48 bytes.; privatekey: Output buffer, at least 96 bytes; contains secret material.; rng_generate: Cryptographically secure generator, return true on success..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake192f_generate_seeded_keypair

Purpose: Generates a repeatable SLH-DSA-SHAKE-192f key pair from 3*n bytes.

bool qsc_slhdsa_shake192f_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t seed[QSC_SLHDSA_SHAKE192F_KEYGEN_SEED_SIZE]);

Parameters: publickey: Output buffer, at least 48 bytes.; privatekey: Output buffer, at least 96 bytes.; seed: Exactly 72 bytes, caller-owned and not modified..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake192f_sign

Purpose: Creates a FIPS 205 pure hedged detached signature with RNG-supplied randomizer.

bool qsc_slhdsa_shake192f_sign(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer, at least 35664 bytes.; siglen: Receives 35664 on success, zero on failure.; message: Message pointer; NULL allowed only if msglen is zero.; msglen: Message length in bytes.; context: Optional context, at most 255 bytes; NULL iff ctxlen is zero.; ctxlen: Context length in bytes.; privatekey: Input 96-byte secret key.; rng_generate: Secure randomizer generator, must return true..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake192f_sign_deterministic

Purpose: Creates a FIPS 205 pure deterministic detached signature using PK.seed as randomizer.

bool qsc_slhdsa_shake192f_sign_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey);

Parameters: signature: Output buffer, at least 35664 bytes.; siglen: Receives 35664 on success, zero on failure.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Context pointer; NULL permitted only when ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; privatekey: Input 96-byte secret key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake192f_sign_prehash

Purpose: HashSLH-DSA-SHAKE-192f hedged signing with FIPS 205 prehash domain.

bool qsc_slhdsa_shake192f_sign_prehash(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned plaintext, NULL only for zero length.; msglen: Input message length.; context: Caller-owned context, NULL only if ctxlen zero.; ctxlen: Context length, maximum 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key.; rng_generate: Secure n-byte randomizer generator..

Return: True only when signing succeeds.

Notes: Reentrant; internal prehash/digest and randomizer securely erased.

qsc_slhdsa_shake192f_sign_prehash_deterministic

Purpose: HashSLH-DSA-SHAKE-192f deterministic prehash signing.

bool qsc_slhdsa_shake192f_sign_prehash_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey);

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned input, NULL only when msglen zero.; msglen: Input message length.; context: Caller-owned context, NULL only for empty context.; ctxlen: Context length, at most 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key..

Return: True on success, false on invalid inputs or signing failure.

Notes: Reentrant; deterministic signing uses private key's PK.seed.

qsc_slhdsa_shake192f_verify

Purpose: Verifies an exactly 35664-byte pure detached SLH-DSA-SHAKE-192f signature.

bool qsc_slhdsa_shake192f_verify(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* publickey);

Parameters: signature: Exactly 35664 bytes, caller-owned.; siglen: Must equal 35664.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Optional context; NULL iff ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; publickey: Input 48-byte public key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake192f_verify_prehash

Purpose: Verify a FIPS 205 HashSLH-DSA-SHAKE-192f signature.

bool qsc_slhdsa_shake192f_verify_prehash(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* publickey);

Parameters: signature: Fixed-size caller-owned detached signature.; siglen: Signature length; must match the selected parameter set.; message: Caller-owned message; NULL only for empty message.; msglen: Input message length.; context: Caller-owned context; NULL only for empty context.; ctxlen: Context length, maximum 255 bytes.; algorithm: Hash function used by signer to construct digest and OID.; publickey: Caller-owned public key..

Return: True only if the prehash-mode signature verifies.

Notes: Reentrant; pure-mode signatures are never accepted as prehash signatures.

qsc_slhdsa_shake192s_generate_keypair

Purpose: Generates the SLH-DSA-SHAKE-192s key pair with a random generator.

bool qsc_slhdsa_shake192s_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output buffer, at least 48 bytes.; privatekey: Output buffer, at least 96 bytes; contains secret material.; rng_generate: Cryptographically secure generator, return true on success..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake192s_generate_seeded_keypair

Purpose: Generates a repeatable SLH-DSA-SHAKE-192s key pair from 3*n bytes.

bool qsc_slhdsa_shake192s_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t seed[QSC_SLHDSA_SHAKE192S_KEYGEN_SEED_SIZE]);

Parameters: publickey: Output buffer, at least 48 bytes.; privatekey: Output buffer, at least 96 bytes.; seed: Exactly 72 bytes, caller-owned and not modified..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake192s_sign

Purpose: Creates a FIPS 205 pure hedged detached signature with RNG-supplied randomizer.

bool qsc_slhdsa_shake192s_sign(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer, at least 16224 bytes.; siglen: Receives 16224 on success, zero on failure.; message: Message pointer; NULL allowed only if msglen is zero.; msglen: Message length in bytes.; context: Optional context, at most 255 bytes; NULL iff ctxlen is zero.; ctxlen: Context length in bytes.; privatekey: Input 96-byte secret key.; rng_generate: Secure randomizer generator, must return true..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake192s_sign_deterministic

Purpose: Creates a FIPS 205 pure deterministic detached signature using PK.seed as randomizer.

bool qsc_slhdsa_shake192s_sign_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey);

Parameters: signature: Output buffer, at least 16224 bytes.; siglen: Receives 16224 on success, zero on failure.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Context pointer; NULL permitted only when ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; privatekey: Input 96-byte secret key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake192s_sign_prehash

Purpose: HashSLH-DSA-SHAKE-192s hedged signing with FIPS 205 prehash domain.

bool qsc_slhdsa_shake192s_sign_prehash(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned plaintext, NULL only for zero length.; msglen: Input message length.; context: Caller-owned context, NULL only if ctxlen zero.; ctxlen: Context length, maximum 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key.; rng_generate: Secure n-byte randomizer generator..

Return: True only when signing succeeds.

Notes: Reentrant; internal prehash/digest and randomizer securely erased.

qsc_slhdsa_shake192s_sign_prehash_deterministic

Purpose: HashSLH-DSA-SHAKE-192s deterministic prehash signing.

bool qsc_slhdsa_shake192s_sign_prehash_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey);

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned input, NULL only when msglen zero.; msglen: Input message length.; context: Caller-owned context, NULL only for empty context.; ctxlen: Context length, at most 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key..

Return: True on success, false on invalid inputs or signing failure.

Notes: Reentrant; deterministic signing uses private key's PK.seed.

qsc_slhdsa_shake192s_verify

Purpose: Verifies an exactly 16224-byte pure detached SLH-DSA-SHAKE-192s signature.

bool qsc_slhdsa_shake192s_verify(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* publickey);

Parameters: signature: Exactly 16224 bytes, caller-owned.; siglen: Must equal 16224.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message

length.; context: Optional context; NULL iff ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; publickey: Input 48-byte public key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake192s_verify_prehash

Purpose: Verify a FIPS 205 HashSLH-DSA-SHAKE-192s signature.

bool qsc_slhdsa_shake192s_verify_prehash(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* publickey);

Parameters: signature: Fixed-size caller-owned detached signature.; siglen: Signature length; must match the selected parameter set.; message: Caller-owned message; NULL only for empty message.; msglen: Input message length.; context: Caller-owned context; NULL only for empty context.; ctxlen: Context length, maximum 255 bytes.; algorithm: Hash function used by signer to construct digest and OID.; publickey: Caller-owned public key..

Return: True only if the prehash-mode signature verifies.

Notes: Reentrant; pure-mode signatures are never accepted as prehash signatures.

qsc_slhdsa_shake256f_generate_keypair

Purpose: Generates the SLH-DSA-SHAKE-256f key pair with a random generator.

bool qsc_slhdsa_shake256f_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output buffer, at least 64 bytes.; privatekey: Output buffer, at least 128 bytes; contains secret material.; rng_generate: Cryptographically secure generator, return true on success..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake256f_generate_seeded_keypair

Purpose: Generates a repeatable SLH-DSA-SHAKE-256f key pair from 3*n bytes.

bool qsc_slhdsa_shake256f_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t seed[QSC_SLHDSA_SHAKE256F_KEYGEN_SEED_SIZE]);

Parameters: publickey: Output buffer, at least 64 bytes.; privatekey: Output buffer, at least 128 bytes.; seed: Exactly 96 bytes, caller-owned and not modified..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake256f_sign

Purpose: Creates a FIPS 205 pure hedged detached signature with RNG-supplied randomizer.

bool qsc_slhdsa_shake256f_sign(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer, at least 49856 bytes.; siglen: Receives 49856 on success, zero on failure.; message: Message pointer; NULL allowed only if msglen is zero.; msglen: Message length in bytes.; context: Optional context, at most 255 bytes; NULL iff ctxlen is zero.; ctxlen: Context length in bytes.; privatekey: Input 128-byte secret key.; rng_generate: Secure randomizer generator, must return true..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake256f_sign_deterministic

Purpose: Creates a FIPS 205 pure deterministic detached signature using PK.seed as randomizer.

bool qsc_slhdsa_shake256f_sign_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey);

Parameters: signature: Output buffer, at least 49856 bytes.; siglen: Receives 49856 on success, zero on failure.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Context pointer; NULL permitted only when ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; privatekey: Input 128-byte secret key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake256f_sign_prehash

Purpose: HashSLH-DSA-SHAKE-256f hedged signing with FIPS 205 prehash domain.

bool qsc_slhdsa_shake256f_sign_prehash(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned plaintext, NULL only for zero length.; msglen: Input message length.; context: Caller-owned context, NULL only if ctxlen zero.; ctxlen: Context length, maximum 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key.; rng_generate: Secure n-byte randomizer generator..

Return: True only when signing succeeds.

Notes: Reentrant; internal prehash/digest and randomizer securely erased.

qsc_slhdsa_shake256f_sign_prehash_deterministic

Purpose: HashSLH-DSA-SHAKE-256f deterministic prehash signing.

bool qsc_slhdsa_shake256f_sign_prehash_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey);

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned input, NULL only when msglen zero.; msglen: Input message length.; context: Caller-owned context, NULL only for empty context.; ctxlen: Context length, at most 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key..

Return: True on success, false on invalid inputs or signing failure.

Notes: Reentrant; deterministic signing uses private key's PK.seed.

qsc_slhdsa_shake256f_verify

Purpose: Verifies an exactly 49856-byte pure detached SLH-DSA-SHAKE-256f signature.

bool qsc_slhdsa_shake256f_verify(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* publickey);

Parameters: signature: Exactly 49856 bytes, caller-owned.; siglen: Must equal 49856.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Optional context; NULL iff ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; publickey: Input 64-byte public key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake256f_verify_prehash

Purpose: Verify a FIPS 205 HashSLH-DSA-SHAKE-256f signature.

bool qsc_slhdsa_shake256f_verify_prehash(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* publickey);

Parameters: signature: Fixed-size caller-owned detached signature.; siglen: Signature length; must match the selected parameter set.; message: Caller-owned message; NULL only for empty message.; msglen: Input message length.; context: Caller-owned context; NULL only for empty context.; ctxlen: Context length, maximum 255 bytes.; algorithm: Hash function used by signer to construct digest and OID.; publickey: Caller-owned public key..

Return: True only if the prehash-mode signature verifies.

Notes: Reentrant; pure-mode signatures are never accepted as prehash signatures.

qsc_slhdsa_shake256s_generate_keypair

Purpose: Generates the SLH-DSA-SHAKE-256s key pair with a random generator.

bool qsc_slhdsa_shake256s_generate_keypair(uint8_t* publickey, uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: publickey: Output buffer, at least 64 bytes.; privatekey: Output buffer, at least 128 bytes; contains secret material.; rng_generate: Cryptographically secure generator, return true on success..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake256s_generate_seeded_keypair

Purpose: Generates a repeatable SLH-DSA-SHAKE-256s key pair from 3*n bytes.

bool qsc_slhdsa_shake256s_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t seed[QSC_SLHDSA_SHAKE256S_KEYGEN_SEED_SIZE]);

Parameters: publickey: Output buffer, at least 64 bytes.; privatekey: Output buffer, at least 128 bytes.; seed: Exactly 96 bytes, caller-owned and not modified..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake256s_sign

Purpose: Creates a FIPS 205 pure hedged detached signature with RNG-supplied randomizer.

bool qsc_slhdsa_shake256s_sign(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer, at least 29792 bytes.; siglen: Receives 29792 on success, zero on failure.; message: Message pointer; NULL allowed only if msglen is zero.; msglen: Message length in bytes.; context: Optional context, at most 255 bytes; NULL iff ctxlen is zero.; ctxlen: Context length in bytes.; privatekey: Input 128-byte secret key.; rng_generate: Secure randomizer generator, must return true..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake256s_sign_deterministic

Purpose: Creates a FIPS 205 pure deterministic detached signature using PK.seed as randomizer.

bool qsc_slhdsa_shake256s_sign_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey);

Parameters: signature: Output buffer, at least 29792 bytes.; siglen: Receives 29792 on success, zero on failure.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Context pointer; NULL permitted only when ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; privatekey: Input 128-byte secret key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. Signing reads secret material; never share writable buffers across calls.

qsc_slhdsa_shake256s_sign_prehash

Purpose: HashSLH-DSA-SHAKE-256s hedged signing with FIPS 205 prehash domain.

bool qsc_slhdsa_shake256s_sign_prehash(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned plaintext, NULL only for zero length.; msglen: Input message length.; context: Caller-owned context, NULL only if ctxlen zero.; ctxlen: Context length, maximum 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key.; rng_generate: Secure n-byte randomizer generator..

Return: True only when signing succeeds.

Notes: Reentrant; internal prehash/digest and randomizer securely erased.

qsc_slhdsa_shake256s_sign_prehash_deterministic

Purpose: HashSLH-DSA-SHAKE-256s deterministic prehash signing.

bool qsc_slhdsa_shake256s_sign_prehash_deterministic(uint8_t* signature, size_t* siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* privatekey);

Parameters: signature: Output buffer of the set's signature size.; siglen: Output signature length or zero on error.; message: Caller-owned input, NULL only when msglen zero.; msglen: Input message length.; context: Caller-owned context, NULL only for empty context.; ctxlen: Context length, at most 255 bytes.; algorithm: Supported prehash function.; privatekey: Caller-owned private key..

Return: True on success, false on invalid inputs or signing failure.

Notes: Reentrant; deterministic signing uses private key's PK.seed.

qsc_slhdsa_shake256s_verify

Purpose: Verifies an exactly 29792-byte pure detached SLH-DSA-SHAKE-256s signature.

bool qsc_slhdsa_shake256s_verify(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* publickey);

Parameters: signature: Exactly 29792 bytes, caller-owned.; siglen: Must equal 29792.; message: Message pointer; NULL permitted only when msglen is zero.; msglen: Message length.; context: Optional context; NULL iff ctxlen is zero.; ctxlen: Context length, at most 255 bytes.; publickey: Input 64-byte public key..

Return: True on success, false on invalid input or cryptographic failure.

Notes: Reentrant. All buffers are caller-owned. No shared mutable state.

qsc_slhdsa_shake256s_verify_prehash

Purpose: Verify a FIPS 205 HashSLH-DSA-SHAKE-256s signature.

bool qsc_slhdsa_shake256s_verify_prehash(const uint8_t* signature, size_t siglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, qsc_slhdsa_prehash_algorithm algorithm, const uint8_t* publickey);

Parameters: signature: Fixed-size caller-owned detached signature.; siglen: Signature length; must match the selected parameter set.; message: Caller-owned message; NULL only for empty message.; msglen: Input message length.; context: Caller-owned context; NULL only for empty context.; ctxlen: Context length, maximum 255 bytes.; algorithm: Hash function used by signer to construct digest and OID.; publickey: Caller-owned public key..

Return: True only if the prehash-mode signature verifies.

Notes: Reentrant; pure-mode signatures are never accepted as prehash signatures.

Header: sphincsplus.h

qsc_sphincsplus_generate_seeded_keypair

Purpose: Generates a Sphincs+ public/private key-pair using a seed.

bool qsc_sphincsplus_generate_seeded_keypair(uint8_t* publickey, uint8_t* privatekey, const uint8_t seed[QSC_SPHINCSPLUS_GENERATE_SEED_SIZE]);

Parameters: publickey: [uint8_t*] Pointer to the public verification-key array; privatekey: [uint8_t*] Pointer to the private signature-key array; seed: [const uint8_t*] Pointer to the random generator.

Return: [bool] Returns true for success

Notes: Arrays must be sized to QSC_SPHINCSPLUS_PUBLICKEY_SIZE and QSC_SPHINCSPLUS_SECRETKEY_SIZE, seed to QSC_SPHINCSPLUS_GENERATE_SEED_SIZE.

qsc_sphincsplus_seeded_sign_ex

Purpose: Takes the message and a seed as input and returns an array containing the signature followed by the message.

bool qsc_sphincsplus_seeded_sign_ex(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, const uint8_t* seed);

Parameters: signedmsg: [uint8_t*] Pointer to the signed-message array; smsglen: [size_t*] Pointer to the signed message length; message: [const uint8_t*] Pointer to the message array; msglen: [size_t] The message length; context: [const uint8_t*] Pointer to the context array.; ctxlen: [size_t] The context array length.; privatekey: [const uint8_t*] Pointer to the private signature-key array; seed: [const uint8_t*] Pointer to the random seed.

Return: [bool] Returns true for success

Notes: Signature array must be sized to the size of the message plus QSC_SPHINCSPLUS_SIGNATURE_SIZE.

qsc_sphincsplus_sign_ex

Purpose: Takes the message as input with the additional context parameter and returns an array containing the signature followed by the message.

bool qsc_sphincsplus_sign_ex(uint8_t* signedmsg, size_t* smsglen, const uint8_t* message, size_t msglen, const uint8_t* context, size_t ctxlen, const uint8_t* privatekey, bool (*rng_generate)(uint8_t*, size_t));

Parameters: signedmsg: [uint8_t*] Pointer to the signed-message array.; smsglen: [size_t*] Pointer to the signed message length.; message: [const uint8_t*] Pointer to the message array.; msglen: [size_t] The message array length.; context: [const uint8_t*] Pointer to the context array.; ctxlen: [size_t] The context array length.; privatekey: [const uint8_t*] Pointer to the private signature-key.; rng_generate: Caller-provided secure random generator callback used by signing..

Return: [bool] Returns true if the message was signed successfully.

Notes: The signed-message array must be sized to the size of the message plus QSC_SPHINCSPLUS_SIGNATURE_SIZE.

qsc_sphincsplus_verify_ex

Purpose: Verifies a signature-message pair and context parameter with the public key.

bool qsc_sphincsplus_verify_ex(uint8_t* message, size_t* msglen, const uint8_t* context, size_t ctxlen, const uint8_t* signedmsg, size_t smsglen, const uint8_t* publickey);

Parameters: message: [uint8_t*] Pointer to the message output array.; msglen: [size_t*] Pointer to the length of the message array.; context: [const uint8_t*] Pointer to the context array.; ctxlen: [size_t] The context array length.; signedmsg: [const uint8_t*] Pointer to the signed message array.; smsglen: [size_t] The signed message length.; publickey: [const uint8_t*] Pointer to the public verification-key array..

Return: [bool] Returns true if the signature is valid.

11: TLS 1.3

The QSC TLS 1.3 implementation provides a complete, record-layer-based TLS 1.3 client and server stack. The engine does not own sockets or transport resources; callers supply inbound byte buffers and outbound scratch buffers. The engine drives handshake progression, frames encrypted application records, decrypts inbound records, dispatches post-handshake messages, and handles session tickets and closure alerts. All cipher suites and signature algorithms are negotiated according to RFC 8446.

11.1 TLS Socket Interface

Header:

tlssocket.h

Description:

The TLS socket interface is the primary SDK-facing TLS API. It combines TLS policy, certificate and trust-store configuration, socket transport, application send and receive operations, connection status queries, ALPN selection, session-ticket handling, key update, shutdown, and listener/server helpers. The lower-level client, server, engine, I/O, record, transcript, codec, extension, and key-schedule headers are implementation and advanced integration interfaces.

Selected API:

void qsc_tls_socket_context_initialize(qsc_tls_socket_context* context)

qsc_tls_socket_status
qsc_tls_socket_context_set_default_client_policy(qsc_tls_socket_context* context)

qsc_tls_socket_status
qsc_tls_socket_context_set_default_server_policy(qsc_tls_socket_context* context)

qsc_tls_socket_status qsc_tls_socket_client_connect_host(qsc_tls_socket_connection* connection, const qsc_tls_socket_context* context, const char* hostname, const char* service)

qsc_tls_socket_status qsc_tls_socket_send(qsc_tls_socket_connection* connection, const uint8_t* input, size_t inlen, size_t* written)

qsc_tls_socket_status qsc_tls_socket_receive(qsc_tls_socket_connection* connection, uint8_t* output, size_t outlen, size_t* read)

qsc_tls_socket_status qsc_tls_socket_shutdown(qsc_tls_socket_connection* connection)

qsc_tls_socket_status qsc_tls_socket_get_peer_info(const qsc_tls_socket_connection* connection, qsc_tls_socket_peer_info* peerinfo)

qsc_tls_socket_status qsc_tls_socket_listener_accept(qsc_tls_socket_listener* listener, qsc_tls_socket_connection* connection)

qsc_tls_socket_status qsc_tls_socket_server_start(qsc_tls_socket_server* server)

11.5 TLS I/O

Header:

tlsio.h

Description:

The TLS I/O adapter binds an initialized TLS engine connection to a QSC socket and provides handshake, send, receive, and shutdown helpers for integrations that require explicit engine and transport ownership.

qsc_tls_status qsc_tls_io_attach(qsc_tls_io_connection* io, qsc_tls_connection* engine, qsc_socket* socket)

qsc_tls_status qsc_tls_io_handshake(qsc_tls_io_connection* io)

qsc_tls_status qsc_tls_io_send(qsc_tls_io_connection* io, const uint8_t* input, size_t inlen, size_t* written)

qsc_tls_status qsc_tls_io_receive(qsc_tls_io_connection* io, uint8_t* output, size_t outlen, size_t* read)

11.2 TLS Client Engine

Header:

tlsclient.h

Description:

The TLS 1.3 client API provides functions to initialize the client state from a caller-supplied configuration, transmit the ClientHello, process inbound TLS records (including the full handshake exchange), and query the negotiated cipher suite after the handshake completes.

API:

Initialize

Initializes a TLS client state from a caller-supplied configuration structure. Takes a pointer to the client state and a pointer to the configuration as parameters. Returns a TLS status code.

qsc_tls_status qsc_tls_client_initialize(qsc_tls_client_state* state, const qsc_tls_client_config* config)

Send Hello

Serializes and writes the ClientHello record to the output buffer. Takes pointers to the client state, the output buffer, the output buffer length, and a pointer to receive the number of bytes written.

qsc_tls_status qsc_tls_client_send_hello(qsc_tls_client_state* state, uint8_t* output, size_t outlen, size_t* written)

Process Record

Consumes inbound TLS records and drives the handshake state machine. Takes pointers to the client state, the input buffer and length, a consumed counter, the output buffer and length, and a written counter.

qsc_tls_status qsc_tls_client_process_record(qsc_tls_client_state* state, const uint8_t* input, size_t inlen, size_t* consumed, uint8_t* output, size_t outlen, size_t* written)

Handshake Complete

Returns true if the TLS handshake has been completed successfully.

bool qsc_tls_client_is_handshake_complete(const qsc_tls_client_state* state)

Dispose

Releases all resources held by the client state.

void qsc_tls_client_dispose(qsc_tls_client_state* state)

11.3 TLS Server Engine

Header:

tlsserver.h

Description:

The TLS 1.3 server API provides functions to initialize the server state from a configuration structure containing the server certificate chain and private key, process inbound client records through the full TLS 1.3 handshake, and send application data after the handshake completes.

API:

Initialize

Initializes the TLS server state. Takes a pointer to the server state and a pointer to the server configuration as parameters. Returns a TLS status code.

```
qsc_tls_status qsc_tls_server_initialize(qsc_tls_server_state* state, const qsc_tls_server_config* config)
```

Process Record

Processes an inbound TLS record and drives the server-side handshake state machine. Parameters mirror those of the client process record function.

```
qsc_tls_status qsc_tls_server_process_record(qsc_tls_server_state* state, const uint8_t* input, size_t inlen, size_t* consumed, uint8_t* output, size_t outlen, size_t* written)
```

Handshake Complete

```
bool qsc_tls_server_is_handshake_complete(const qsc_tls_server_state* state)
```

Dispose

```
void qsc_tls_server_dispose(qsc_tls_server_state* state)
```

11.4 TLS Engine

Header:

tlsengine.h

Description:

The TLS engine provides a unified client and server connection handle and a bounded record-processing layer. It drives handshake progression, frames encrypted application records, decrypts inbound records, dispatches post-handshake messages, and constructs closure alerts. The engine stores the active TLS endpoint state, the endpoint role, and bounded scratch buffers used for inbound handshake reassembly and application record processing.

References:

RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3: https://tools.ietf.org/html/rfc8446

11.1 TLS 1.3 Implementor API Reference

The TLS 1.3 reference below covers exposed socket, engine, handshake, record, transcript, groups, signatures and extension interfaces. APIs that operate on an established TLS context require the caller to maintain the documented session lifecycle.

Header: tlsalert.h

qsc_tls_alert_decode

Purpose: Decode a TLS alert record payload.

Details: Parses exactly one standard two-byte TLS alert payload and extracts the alert description field. In TLS 1.3, AlertLevel is a legacy field and is ignored on receipt. Unknown alert-description values are returned to the caller so they can be treated as error alerts as required by RFC 9846.

qsc_tls_status qsc_tls_alert_decode(const uint8_t* input, size_t inlen, qsc_tls_alert_description* description);

Parameters: input: [const uint8_t*] The encoded alert payload.; inlen: [size_t] The length of the encoded input in bytes.; description: [enum*] Receives the decoded TLS alert description..

Return: [qsc_tls_status] Returns the operation status.

qsc_tls_alert_encode

Purpose: Encode a TLS alert record payload.

Details: Writes the standard two-byte TLS alert payload to the destination buffer. close_notify and user_canceled are encoded with warning level; all defined error alerts are encoded with fatal level, as required by TLS 1.3.

qsc_tls_status qsc_tls_alert_encode(uint8_t* output, size_t outlen, qsc_tls_alert_description description);

Parameters: output: [uint8_t*] The destination buffer receiving the encoded alert payload.; outlen: [size_t] The length of the destination buffer in bytes.; description: [enum] The TLS alert description to encode..

Return: [qsc_tls_status] Returns the operation status.

qsc_tls_alert_encode_record

Purpose: Brief Encode a plaintext TLS alert record. Builds the two-byte alert payload and wraps it in a TLSPlaintext record of content type alert. This is used for alert transmission before encrypted traffic keys are available.

Details: Brief Encode a plaintext TLS alert record. Builds the two-byte alert payload and wraps it in a TLSPlaintext record of content type alert. This is used for alert transmission before encrypted traffic keys are available.

qsc_tls_status qsc_tls_alert_encode_record(uint8_t* output, size_t outlen, size_t* written, qsc_tls_alert_description description);

Parameters: output: [uint8_t*] The destination buffer receiving the encoded record.; outlen: [size_t] The length of the destination buffer in bytes.; written: [size_t*] Receives the number of bytes written.; description: [enum] The TLS alert description to encode..

Return: [qsc_tls_status] Returns the operation status.

qsc_tls_alert_from_status

Purpose: Brief Map an internal TLS status value to an RFC alert description. Converts internal parser, state-machine, and authentication failures to the closest TLS alert description used on the wire. This helper is used when the wrapper layer needs to emit a protocol alert in response to a local failure.

Details: Brief Map an internal TLS status value to an RFC alert description. Converts internal parser, state-machine, and authentication failures to the closest TLS alert description used on the wire. This helper is used when the wrapper layer needs to emit a protocol alert in response to a local failure.

qsc_tls_alert_description qsc_tls_alert_from_status(qsc_tls_status status);

Parameters: status: [enum] The internal TLS status value..

Return: [enum] Returns the mapped TLS alert description.

qsc_tls_alert_is_close_notify

Purpose: Check if alert is close_notify

bool qsc_tls_alert_is_close_notify(qsc_tls_alert_description description);

Parameters: description: [enum] Alert description.

Return: : true if close_notify

qsc_tls_alert_is_fatal_level

Purpose: Check if alert level is fatal

bool qsc_tls_alert_is_fatal_level(uint8_t level);

Parameters: level: [uint8_t] Alert level.

Return: : true if fatal

qsc_tls_alert_is_valid

Purpose: Validate alert description

bool qsc_tls_alert_is_valid(qsc_tls_alert_description description);

Parameters: description: [enum] Alert description.

Return: : true if valid

qsc_tls_alert_to_string

Purpose: Convert alert description to string (debug only)

const char* qsc_tls_alert_to_string(qsc_tls_alert_description description);

Parameters: description: [qsc_tls_alert_description] Alert description enumerator.

Return: : const string name

Header: tlscert.h

qsc_tls_certificate_decode_message

Purpose: Decode a TLS Certificate message

Details: Parses a TLS 1.3 Certificate handshake message and extracts the certificate request context and certificate chain entries as spans into the input buffer. The current QSC profile does not negotiate CertificateEntry extensions, so a non-empty per-certificate extension vector is rejected as unsupported.

qsc_tls_status qsc_tls_certificate_decode_message(const uint8_t* input, size_t inlen, const uint8_t** requestcontext, size_t* requestcontextlen, qsc_tls_certificate_view* chain, size_t chaincapacity, size_t* chainlength);

Parameters: input: [const uint8_t*] Pointer to encoded message buffer; inlen: [size_t] Length of input buffer in bytes; requestcontext: [const uint8_t**] Pointer to decoded request context span; requestcontextlen: [size_t*] Length of request context; chain: [struct] Output array of certificate views; chaincapacity: [size_t] Maximum number of entries in chain array; chainlength: [size_t*] Number of decoded certificates.

Return: qsc_tls_status: Operation status code

qsc_tls_certificate_encode_message

Purpose: Encode a TLS Certificate message

Details: Serializes a TLS 1.3 Certificate handshake message containing a certificate request context and a certificate chain. Each certificate entry is encoded as a vector24 with an empty extensions block.

qsc_tls_status qsc_tls_certificate_encode_message(const uint8_t* requestcontext, size_t requestcontextlen, const qsc_tls_certificate_view* chain, size_t chainlength, uint8_t* output, size_t outlen, size_t* offset);

Parameters: requestcontext: [const uint8_t*] Pointer to the certificate request context buffer; requestcontextlen: [size_t] Length of the request context in bytes (<= 255); chain: [const struct] Pointer to an array of certificate views; chainlength: [size_t] Number of certificates in the chain; output: [uint8_t*] Output buffer for encoded message; outlen: [size_t] Size of the output buffer in bytes; offset: [size_t*] Pointer to current write offset in output buffer.

Return: qsc_tls_status: Operation status code

qsc_tls_certificate_interface_get_last_alert

Purpose: Query the most recent alert reason from a certificate interface.

Details: Returns a certificate-specific TLS alert when the interface uses the built-in QSC X.509 bridge. For custom callbacks, this helper falls back to a generic certificate or CertificateVerify alert according to the verification phase.

qsc_tls_alert_description qsc_tls_certificate_interface_get_last_alert(const qsc_tls_certificate_interface* iface, bool verifyphase);

Parameters: iface: [struct*] The certificate interface.; verifyphase: [bool] Set true when the failure occurred during CertificateVerify processing..

Return: [enum] Returns the mapped TLS alert description.

qsc_tls_certificate_interface_initialize

Purpose: Initialize a certificate callback interface.

void qsc_tls_certificate_interface_initialize(qsc_tls_certificate_interface* iface, qsc_tls_certificate_chain_validate_callback validatechain, qsc_tls_certificate_verify_callback verifycertificateverify, void* state);

Parameters: iface: [struct] The interface structure to initialize.; validatechain: [function] The chain validation callback.; verifycertificateverify: [function] The CertificateVerify callback.; state: [void*] The caller-supplied callback state..

Return: None (void).

qsc_tls_certificate_interface_initialize_qsc_x509

Purpose: Initialize a certificate callback interface using the QSC X.509 bridge.

qsc_tls_status
qsc_tls_certificate_interface_initialize_qsc_x509(qsc_tls_certificate_interface* iface, qsc_tls_qsc_x509_context* context);

Parameters: iface: [struct] The interface structure to initialize.; context: [struct] The X.509 bridge context..

Return: [qsc_tls_status] Returns the operation status.

qsc_tls_certificate_interface_is_valid

Purpose: Determine whether a certificate callback interface is complete.

bool qsc_tls_certificate_interface_is_valid(const qsc_tls_certificate_interface* iface);

Parameters: iface: [struct*] The interface to inspect..

Return: [bool] Returns true if the required callbacks are present.

qsc_tls_certificate_request_decode

Purpose: Decode a TLS CertificateRequest message

Details: Parses a TLS 1.3 CertificateRequest handshake message and extracts the request context and extensions block as spans into the input buffer.

qsc_tls_status qsc_tls_certificate_request_decode(const uint8_t* input, size_t inlen, const uint8_t** requestcontext, size_t* requestcontextlen, const uint8_t** extensionsblock, size_t* extensionsblocklen);

Parameters: input: [const uint8_t*] Pointer to encoded message buffer; inlen: [size_t] Length of input buffer; requestcontext: [const uint8_t**] Pointer to request context span; requestcontextlen: [size_t*] Length of request context; extensionsblock: [const uint8_t**] Pointer to extensions block span; extensionsblocklen: [size_t*] Length of extensions block.

Return: qsc_tls_status: Operation status code

qsc_tls_certificate_request_encode

Purpose: Encode a TLS CertificateRequest message

Details: Serializes a TLS 1.3 CertificateRequest handshake message containing a request context and a pre-encoded extensions block.

qsc_tls_status qsc_tls_certificate_request_encode(const uint8_t* requestcontext, size_t requestcontextlen, const uint8_t* extensionsblock, size_t extensionsblocklen, uint8_t* output, size_t outlen, size_t* offset);

Parameters: requestcontext: [const uint8_t*] Pointer to request context buffer; requestcontextlen: [size_t] Length of request context in bytes (<= 255); extensionsblock: [const uint8_t*] Pointer to extensions block buffer; extensionsblocklen: [size_t] Length of extensions block in bytes; output: [uint8_t*] Output buffer for encoded message; outlen: [size_t] Size of output buffer; offset: [size_t*] Pointer to current write offset.

Return: qsc_tls_status: Operation status code

qsc_tls_x509_alert_from_verify_status

Purpose: Map a QSC X.509 verification result to a TLS alert description.

qsc_tls_alert_description qsc_tls_x509_alert_from_verify_status(qsc_x509_verify_status status);

Parameters: status: [enum] The QSC X.509 verification status..

Return: [enum] Returns the mapped TLS alert description.

qsc_tls_x509_context_clone

Purpose: Clone a configured QSC X.509 TLS context for one TLS connection.

Details: Copies borrowed validation configuration from source, clears mutable result state, and deliberately does not share the source verification scratch buffer.

qsc_tls_status qsc_tls_x509_context_clone(qsc_tls_qsc_x509_context* destination, const qsc_tls_qsc_x509_context* source);

Parameters: destination: [struct*] Destination connection-owned context.; source: [const struct*] Configured source/template context..

Return: [enum] Returns qsc_tls_status_success on success.

qsc_tls_x509_context_initialize

Purpose: Initialize a QSC X.509 bridge context.

qsc_tls_status qsc_tls_x509_context_initialize(qsc_tls_qsc_x509_context* context, const qsc_x509_store* truststore, const qsc_x509_certificate* intermediates, size_t intermediatecount, const qsc_x509_time* validationtime, uint8_t* verifybuffer, size_t verifybufferlen);

Parameters: context: [struct] The context structure to initialize.; truststore: [struct*] The trust store containing the trust anchors.; intermediates: [struct*] Optional intermediate certificates.; intermediatecount: [size_t] The number of intermediate certificates supplied.; validationtime: [struct*] Validation time used during certificate verification.; verifybuffer: [uint8_t*] Scratch buffer used during verification.; verifybufferlen: [size_t] The length of the scratch buffer in bytes..

Return: qsc_tls_status; the header does not specify additional failure semantics.

qsc_tls_x509_signature_algorithm_from_tls

Purpose: Map a TLS signature scheme to a QSC X.509 signature algorithm identifier.

qsc_x509_signature_algorithm
qsc_tls_x509_signature_algorithm_from_tls(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme..

Return: [qsc_x509_signature_algorithm] Returns the mapped X.509 algorithm identifier.

qsc_tls_x509_validate_chain

Purpose: Validate a peer certificate chain using the QSC X.509 bridge.

bool qsc_tls_x509_validate_chain(const qsc_tls_certificate_view* chain, size_t chainlength, const qsc_tls_certificate_validation_context* context, void* state);

Parameters: chain: [struct*] The certificate chain entries in leaf-first order.; chainlength: [size_t] The number of certificate entries.; context: [struct*] The certificate validation context.; state: [void*] The caller-supplied bridge context..

Return: [bool] Returns true if the chain is accepted.

qsc_tls_x509_verify_certificate_verify

Purpose: Verify a TLS CertificateVerify signature using the QSC X.509 bridge.

bool qsc_tls_x509_verify_certificate_verify(qsc_tls_signature_scheme scheme, const uint8_t* input, size_t inputlen, const uint8_t* signature, size_t signaturelen, const qsc_tls_certificate_view* signer, void* state);

Parameters: scheme: [enum] The TLS signature scheme.; input: CertificateVerify signed-message input bytes.; inputlen: Length of input in bytes.; signature: [const uint8_t*] The encoded signature bytes.; signaturelen: [size_t] The length of the signature in bytes.; signer: [struct*] The signer certificate view.; state: [void*] The caller-supplied bridge context..

Return: [bool] Returns true if the signature is valid.

Header: tlscertx509.h

qsc_tls_cert_x509_bind

Purpose: Bind an X.509-backed state to a TLS certificate interface.

void qsc_tls_cert_x509_bind(qsc_tls_certificate_interface* iface, qsc_tls_cert_x509_state* state);

Parameters: iface: [struct*] Destination interface with callbacks populated.; state: [struct*] Caller-owned state; must outlive the interface use..

Return: None (void).

qsc_tls_cert_x509_state_initialize

Purpose: Initialize an X.509 interface state with sensible defaults.

void qsc_tls_cert_x509_state_initialize(qsc_tls_cert_x509_state* state, const qsc_x509_store* truststore);

Parameters: state: [struct*] Destination state, cleared to zero.; truststore: [const*] Trust anchors; NULL causes certificate validation to fail closed..

Return: None (void).

Header: tlsclient.h

qsc_tls_client_config_set_certificate_interface

Purpose: Copy a certificate-validation interface into a TLS client configuration.

Details: This setter is the TLS-side attachment point for certificate validation. X.509 helpers prepare the qsc_tls_certificate_interface, but do not mutate TLS state objects directly. The hostname pointer is borrowed from the caller and must remain valid for the lifetime of configurations initialized from this object.

qsc_tls_status qsc_tls_client_config_set_certificate_interface(qsc_tls_client_config* config, const qsc_tls_certificate_interface* iface, const char* hostname);

Parameters: config: [struct*] The client configuration to update.; iface: [struct*] The certificate-validation interface to copy.; hostname: [const char*] Optional server hostname for SNI and certificate validation..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_client_config_set_local_certificate

Purpose: Configure a local client certificate chain and private signing key.

Details: The configured identity is used only when the server sends a main-handshake CertificateRequest whose signature_algorithms extension permits verifyscheme. If no compatible identity is configured, the client sends the required empty Certificate message and continues without certificate authentication.

qsc_tls_status qsc_tls_client_config_set_local_certificate(qsc_tls_client_config* config, const qsc_tls_certificate_view* chain, size_t chainlength, qsc_tls_signature_scheme verifyscheme, const uint8_t* privatekeydata, size_t privatekeylen);

Parameters: config: [struct*] The client configuration to update.; chain: [const struct*] Client certificate chain views in leaf-first order.; chainlength: [size_t] Number of entries in the certificate chain.; verifyscheme: [enum] Signature scheme used for client CertificateVerify.; privatekeydata: [const uint8_t*] Raw private signing key bytes.; privatekeylen: [size_t] Length of the private signing key in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_client_get_negotiated_cipher_suite

Purpose: Get the cipher suite negotiated by the server.

qsc_tls_cipher_suite qsc_tls_client_get_negotiated_cipher_suite(const qsc_tls_client_state* state);

Parameters: state: [const struct*] The client state to query..

Return: [qsc_tls_cipher_suite] Returns the negotiated cipher suite, or an unspecified value if negotiation has not completed.

Header: tlscodec.h

qsc_tls_codec_read_bytes

Purpose: Read raw bytes from a TLS buffer.

qsc_tls_status qsc_tls_codec_read_bytes(const uint8_t* input, size_t inplen, size_t* offset, uint8_t* output, size_t outlen);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length..

Return: The operation status.

qsc_tls_codec_read_u16

Purpose: rief Read an unsigned 16-bit value from a TLS buffer.

Details: rief Read an unsigned 16-bit value from a TLS buffer.

qsc_tls_status qsc_tls_codec_read_u16(const uint8_t* input, size_t inplen, size_t* offset, uint16_t* value);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; value: [uint16_t*] The decoded value..

Return: The operation status.

qsc_tls_codec_read_u24

Purpose: Read an unsigned 24-bit value from a TLS buffer.

qsc_tls_status qsc_tls_codec_read_u24(const uint8_t* input, size_t inplen, size_t* offset, uint32_t* value);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; value: [uint32_t*] The decoded value..

Return: The operation status.

qsc_tls_codec_read_u32

Purpose: Read an unsigned 32-bit value from a TLS buffer.

qsc_tls_status qsc_tls_codec_read_u32(const uint8_t* input, size_t inplen, size_t* offset, uint32_t* value);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; value: [uint32_t*] The decoded value..

Return: The operation status.

qsc_tls_codec_read_u64

Purpose: Read an unsigned 64-bit value from a TLS buffer in network byte order.

qsc_tls_status qsc_tls_codec_read_u64(const uint8_t* input, size_t inplen, size_t* offset, uint64_t* value);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; value: [uint64_t*] The decoded value..

Return: The operation status.

qsc_tls_codec_read_u8

Purpose: Read an unsigned 8-bit value from a TLS buffer.

qsc_tls_status qsc_tls_codec_read_u8(const uint8_t* input, size_t inplen, size_t* offset, uint8_t* value);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; value: [uint8_t*] The decoded value..

Return: The operation status.

qsc_tls_codec_read_vector16_span

Purpose: Read a 16-bit length-prefixed TLS vector as a span.

qsc_tls_status qsc_tls_codec_read_vector16_span(const uint8_t* input, size_t inplen, size_t* offset, const uint8_t** span, size_t* spanlen);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; span: [const uint8_t**] The returned span pointer.; spanlen: [size_t*] The returned span length..

Return: The operation status.

qsc_tls_codec_read_vector24_span

Purpose: Read a 24-bit length-prefixed TLS vector as a span.

qsc_tls_status qsc_tls_codec_read_vector24_span(const uint8_t* input, size_t inplen, size_t* offset, const uint8_t** span, size_t* spanlen);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; span: [const uint8_t**] The returned span pointer.; spanlen: [size_t*] The returned span length..

Return: The operation status.

qsc_tls_codec_read_vector8_span

Purpose: Read an 8-bit length-prefixed TLS vector as a span.

qsc_tls_status qsc_tls_codec_read_vector8_span(const uint8_t* input, size_t inplen, size_t* offset, const uint8_t** span, size_t* spanlen);

Parameters: input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length.; offset: [size_t*] The input and output read offset.; span: [const uint8_t**] The returned span pointer.; spanlen: [size_t*] The returned span length..

Return: The operation status.

qsc_tls_codec_vector_begin_u16

Purpose: Reserve a 16-bit length-prefixed vector header.

qsc_tls_status qsc_tls_codec_vector_begin_u16(uint8_t* output, size_t outlen, size_t* offset, size_t* headerposition);

Parameters: output: Writable TLS codec output buffer.; outlen: Capacity in bytes of output.; offset: In/out cursor recording the current encoded position.; headerposition: Receives the position reserved for the vector length..

Return: qsc_tls_status; the header does not specify additional failure semantics.

qsc_tls_codec_vector_begin_u24

Purpose: Reserve a 24-bit length-prefixed vector header.

qsc_tls_status qsc_tls_codec_vector_begin_u24(uint8_t* output, size_t outlen, size_t* offset, size_t* headerposition);

Parameters: output: Writable TLS codec output buffer.; outlen: Capacity in bytes of output.; offset: In/out cursor recording the current encoded position.; headerposition: Receives the position reserved for the vector length..

Return: qsc_tls_status; the header does not specify additional failure semantics.

qsc_tls_codec_vector_begin_u8

Purpose: Reserve space for a length-prefixed vector header and return its position.

Details: Used for encoding nested TLS structures where the length is unknown until the body has been written. Advance the write offset past the reserved header, write the body using other codec helpers, then call qsc_tls_codec_vector_end_u8 / _u16 / _u24 with the returned header position to backpatch the final length.

qsc_tls_status qsc_tls_codec_vector_begin_u8(uint8_t* output, size_t outlen, size_t* offset, size_t* headerposition);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; headerposition: [size_t*] Receives the reserved header byte position..

Return: The operation status.

qsc_tls_codec_vector_end_u16

Purpose: Backpatch the 16-bit length of a reserved vector.

qsc_tls_status qsc_tls_codec_vector_end_u16(uint8_t* output, size_t outlen, const size_t* offset, size_t headerposition);

Parameters: output: Writable TLS codec output buffer.; outlen: Capacity in bytes of output.; offset: In/out cursor recording the current encoded position.; headerposition: Position of the previously reserved vector length header..

Return: qsc_tls_status; the header does not specify additional failure semantics.

qsc_tls_codec_vector_end_u24

Purpose: Backpatch the 24-bit length of a reserved vector.

qsc_tls_status qsc_tls_codec_vector_end_u24(uint8_t* output, size_t outlen, const size_t* offset, size_t headerposition);

Parameters: output: Writable TLS codec output buffer.; outlen: Capacity in bytes of output.; offset: In/out cursor recording the current encoded position.; headerposition: Position of the previously reserved vector length header..

Return: qsc_tls_status; the header does not specify additional failure semantics.

qsc_tls_codec_vector_end_u8

Purpose: Backpatch the 8-bit length of a reserved vector.

qsc_tls_status qsc_tls_codec_vector_end_u8(uint8_t* output, size_t outlen, const size_t* offset, size_t headerposition);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The current write offset; used to compute the body length.; headerposition: [size_t] The header position returned from qsc_tls_codec_vector_begin_u8..

Return: qsc_tls_status; the header does not specify additional failure semantics.

qsc_tls_codec_write_bytes

Purpose: Write raw bytes to a TLS buffer.

qsc_tls_status qsc_tls_codec_write_bytes(uint8_t* output, size_t outlen, size_t* offset, const uint8_t* input, size_t inplen);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length..

Return: The operation status.

qsc_tls_codec_write_u16

Purpose: Write an unsigned 16-bit value to a TLS buffer.

qsc_tls_status qsc_tls_codec_write_u16(uint8_t* output, size_t outlen, size_t* offset, uint16_t value);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; value: [uint16_t] The value to encode..

Return: The operation status.

qsc_tls_codec_write_u24

Purpose: Write an unsigned 24-bit value to a TLS buffer.

qsc_tls_status qsc_tls_codec_write_u24(uint8_t* output, size_t outlen, size_t* offset, uint32_t value);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; value: [uint32_t] The value to encode. The upper byte must be zero..

Return: The operation status.

qsc_tls_codec_write_u32

Purpose: Write an unsigned 32-bit value to a TLS buffer.

qsc_tls_status qsc_tls_codec_write_u32(uint8_t* output, size_t outlen, size_t* offset, uint32_t value);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; value: [uint32_t] The value to encode..

Return: The operation status.

qsc_tls_codec_write_u8

Purpose: Write an unsigned 8-bit value to a TLS buffer.

qsc_tls_status qsc_tls_codec_write_u8(uint8_t* output, size_t outlen, size_t* offset, uint8_t value);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; value: [uint8_t] The value to encode..

Return: The operation status.

qsc_tls_codec_write_vector16

Purpose: Write a 16-bit length-prefixed TLS vector.

qsc_tls_status qsc_tls_codec_write_vector16(uint8_t* output, size_t outlen, size_t* offset, const uint8_t* input, size_t inplen);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length..

Return: The operation status.

qsc_tls_codec_write_vector24

Purpose: Write a 24-bit length-prefixed TLS vector.

qsc_tls_status qsc_tls_codec_write_vector24(uint8_t* output, size_t outlen, size_t* offset, const uint8_t* input, size_t inplen);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length..

Return: The operation status.

qsc_tls_codec_write_vector8

Purpose: Write an 8-bit length-prefixed TLS vector.

qsc_tls_status qsc_tls_codec_write_vector8(uint8_t* output, size_t outlen, size_t* offset, const uint8_t* input, size_t inplen);

Parameters: output: [uint8_t*] The destination buffer.; outlen: [size_t] The destination buffer length.; offset: [size_t*] The input and output write offset.; input: [const uint8_t*] The source buffer.; inplen: [size_t] The source buffer length..

Return: The operation status.

Header: tlsecdsader.h

qsc_tls_ecdsa_der_decode

Purpose: Decode a DER-encoded ECDSA signature into raw r||s form.

qsc_tls_status qsc_tls_ecdsa_der_decode(const uint8_t* der, size_t derlen, size_t componentsize, uint8_t* output, size_t outlen);

Parameters: der: [const uint8_t*] DER-encoded signature.; derlen: [size_t] DER signature length.; componentsize: [size_t] Expected size of each component (defines output width).; output: [uint8_t*] Destination r||s buffer; must be at least 2*componentsize bytes.; outlen: [size_t] Capacity of output..

Return: qsc_tls_status; the header does not specify additional failure semantics.

qsc_tls_ecdsa_der_encode

Purpose: DER-encode a raw r||s ECDSA signature.

qsc_tls_status qsc_tls_ecdsa_der_encode(const uint8_t* rs, size_t componentsize, uint8_t* output, size_t outlen, size_t* written);

Parameters: rs: [const uint8_t*] Raw r||s buffer, each component width bytes long.; componentsize: [size_t] Size of each component in bytes (32 for P-256, 48 for P-384, 66 for P-521).; output: [uint8_t*] Destination buffer for DER-encoded signature.; outlen: [size_t] Capacity of output.; written: [size_t*] On success, receives the DER length (variable, typically 70-72 for P-256)..

Return: qsc_tls_status; the header does not specify additional failure semantics.

Header: tlsengine.h

qsc_tls_engine_close

Purpose: Build an encrypted close_notify alert record.

Details: Constructs a TLS closure alert for the active connection and writes the encrypted close_notify record to the caller-supplied output buffer. The caller is responsible for transmitting the generated record and then closing or draining the underlying transport according to its own I/O policy.

qsc_tls_status qsc_tls_engine_close(qsc_tls_connection* connection, uint8_t* output, size_t outlen, size_t* written);

Parameters: connection: [qsc_tls_connection*] Pointer to an initialized TLS engine connection.; output: [uint8_t*] Pointer to the output buffer receiving the close_notify record.; outlen: [size_t] Size, in bytes, of the output buffer.; written: [size_t*] Pointer receiving the number of bytes written to output..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid state, insufficient output capacity, or encryption failure.

qsc_tls_engine_consume_session_ticket

Purpose: Consume a TLS 1.3 NewSessionTicket record.

Details: Compatibility wrapper over the normal established-connection receive dispatcher. The input record is processed through the same post-handshake reassembly and dispatch path used by qsc_tls_engine_read_application_data_ex. When a complete NewSessionTicket is retained by the configured resumption policy, its parsed metadata and derived resumption PSK are returned in ticketout. Fragmented post-handshake messages may require multiple calls before a ticket becomes available.

qsc_tls_status qsc_tls_engine_consume_session_ticket(qsc_tls_connection* connection, const uint8_t* input, size_t inlen, size_t* consumed, qsc_tls_session_ticket* ticketout);

Parameters: connection: [qsc_tls_connection*] Pointer to a client-role TLS engine connection in the established state.; input: [const uint8_t*] Pointer to the encrypted NewSessionTicket record.; inlen: [size_t] Number of inbound bytes available in input.; consumed: [size_t*] Pointer receiving the number of inbound bytes consumed.; ticketout: [qsc_tls_session_ticket*] Pointer receiving the parsed and derived ticket metadata..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid role, invalid state, malformed input, authentication failure, or ticket derivation failure.

qsc_tls_engine_dispose

Purpose: Dispose of a TLS engine connection.

Details: Releases the active TLS engine state and zeroizes transient buffers, traffic secrets, key schedules, cipher states, and other sensitive material held by the connection context.

void qsc_tls_engine_dispose(qsc_tls_connection* connection);

Parameters: connection: [qsc_tls_connection*] Pointer to the TLS engine connection context to dispose..

Return: None (void).

qsc_tls_engine_emit_session_ticket

Purpose: Emit a TLS 1.3 NewSessionTicket record.

Details: Builds a NewSessionTicket handshake message for a server-role connection in the established state. The function generates the ticket nonce and ticket identifier, derives the per-ticket resumption secret from the resumption_master_secret, encrypts the handshake message under the server application write traffic keys, and returns the generated ticket metadata to the caller.

qsc_tls_status qsc_tls_engine_emit_session_ticket(qsc_tls_connection* connection, uint32_t lifetime_seconds, uint8_t* output, size_t outlen, size_t* written, qsc_tls_session_ticket* ticketout);

Parameters: connection: [qsc_tls_connection*] Pointer to a server-role TLS engine connection in the established state.; lifetime_seconds: [uint32_t] Ticket lifetime value, in seconds, to advertise to the client.; output: [uint8_t*] Pointer to the output buffer receiving the encrypted NewSessionTicket record.; outlen: [size_t] Size, in bytes, of the output buffer.; written: [size_t*] Pointer receiving the number of bytes written to output.; ticketout: [qsc_tls_session_ticket*] Pointer receiving the generated ticket metadata, including nonce, ticket value, lifetime, age-add value, cipher suite, and derived resumption secret..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid role, invalid state, insufficient output capacity, or cryptographic failure.

qsc_tls_engine_handshake

Purpose: Advance the TLS handshake state machine.

Details: Processes inbound handshake or record-layer bytes and writes any generated outbound TLS flight to the supplied output buffer. The caller should invoke this function repeatedly as network data becomes available until qsc_tls_engine_is_handshake_complete() returns true. The function reports how many inbound bytes were consumed and how many outbound bytes were produced. The engine does not perform socket I/O.

qsc_tls_status qsc_tls_engine_handshake(qsc_tls_connection* connection, const uint8_t* input, size_t inlen, size_t* consumed, uint8_t* output, size_t outlen, size_t* written);

Parameters: connection: [qsc_tls_connection*] Pointer to an initialized TLS engine connection.; input: [const uint8_t*] Pointer to inbound TLS bytes. This value may be NULL when inlen is zero.; inlen: [size_t] Number of bytes available in input.; consumed: [size_t*] Pointer receiving the number of inbound bytes consumed by the engine.; output: [uint8_t*] Pointer to the caller-supplied outbound buffer.; outlen: [size_t] Size, in bytes, of the outbound buffer.; written: [size_t*] Pointer receiving the number of outbound bytes written to output..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on malformed input, insufficient output capacity, invalid state, or cryptographic failure.

qsc_tls_engine_initialize_client

Purpose: Initialize a TLS engine connection as a client.

Details: Initializes the supplied connection object for client-side TLS operation using the caller-provided client configuration. The function clears and prepares the connection state, selects the client role, and initializes the embedded client protocol context.

qsc_tls_status qsc_tls_engine_initialize_client(qsc_tls_connection* connection, const qsc_tls_client_config* config);

Parameters: connection: [qsc_tls_connection*] Pointer to the TLS engine connection context to initialize.; config: [const qsc_tls_client_config*] Pointer to the client configuration used to initialize the client protocol state..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid input or initialization failure.

qsc_tls_engine_initialize_server

Purpose: Initialize a TLS engine connection as a server.

Details: Initializes the supplied connection object for server-side TLS operation using the caller-provided server configuration. The function clears and prepares the connection state, selects the server role, and initializes the embedded server protocol context.

qsc_tls_status qsc_tls_engine_initialize_server(qsc_tls_connection* connection, const qsc_tls_server_config* config);

Parameters: connection: [qsc_tls_connection*] Pointer to the TLS engine connection context to initialize.; config: [const qsc_tls_server_config*] Pointer to the server configuration used to initialize the server protocol state..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid input or initialization failure.

qsc_tls_engine_is_handshake_complete

Purpose: Test whether the TLS handshake is complete.

Details: Returns the handshake-completion status of the active client or server state. When this function returns true, the connection may send and receive protected application data.

bool qsc_tls_engine_is_handshake_complete(const qsc_tls_connection* connection);

Parameters: connection: [const qsc_tls_connection*] Pointer to an initialized TLS engine connection..

Return: [bool] Returns true when the handshake is complete; otherwise returns false.

qsc_tls_engine_read_application_data

Purpose: Decrypt inbound application records.

Details: Consumes inbound TLS records, decrypts application-data records under the active read traffic keys, and writes plaintext application bytes to the output buffer. The function reports the number of inbound bytes consumed and plaintext bytes written.

qsc_tls_status qsc_tls_engine_read_application_data(qsc_tls_connection* connection, const uint8_t* input, size_t inlen, size_t* consumed, uint8_t* output, size_t outlen, size_t* written);

Parameters: connection: [qsc_tls_connection*] Pointer to an established TLS engine connection.; input: [const uint8_t*] Pointer to inbound TLS record bytes.; inlen: [size_t] Number of inbound bytes available in input.; consumed: [size_t*] Pointer receiving the number of inbound bytes consumed.; output: [uint8_t*] Pointer to the plaintext output

buffer.; outlen: [size_t] Size, in bytes, of the plaintext output buffer.; written: [size_t*] Pointer receiving the number of plaintext bytes written to output..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid state, malformed records, authentication failure, insufficient output capacity, or closure alert processing.

qsc_tls_engine_read_application_data_ex

Purpose: Decrypt inbound records and process post-handshake messages.

Details: Consumes inbound TLS records, decrypts application-data records, and dispatches supported post-handshake Handshake messages. KeyUpdate is processed immediately and NewSessionTicket is decoded and retained for client-role connections. Unsupported or role-invalid post-handshake Handshake messages are rejected with unexpected_message. If an inbound KeyUpdate requests a reciprocal update, the generated response record is written to responseoutput when a response buffer is supplied. When no response buffer is supplied, the reciprocal KeyUpdate is queued and emitted before the next outbound Application Data record. A retained ticket can be retrieved with qsc_tls_engine_take_session_ticket().

qsc_tls_status qsc_tls_engine_read_application_data_ex(qsc_tls_connection* connection, const uint8_t* input, size_t inlen, size_t* consumed, uint8_t* output, size_t outlen, size_t* written, uint8_t* responseoutput, size_t responseoutlen, size_t* responsewritten);

Parameters: connection: [qsc_tls_connection*] Pointer to an established TLS engine connection.; input: [const uint8_t*] Pointer to inbound TLS record bytes.; inlen: [size_t] Number of inbound bytes available in input.; consumed: [size_t*] Pointer receiving the number of inbound bytes consumed.; output: [uint8_t*] Pointer to the plaintext application output buffer.; outlen: [size_t] Size, in bytes, of the plaintext output buffer.; written: [size_t*] Pointer receiving the number of plaintext bytes written to output.; responseoutput: [uint8_t*] Optional pointer to a buffer receiving any automatically generated post-handshake response records.; responseoutlen: [size_t] Size, in bytes, of responseoutput when non-NULL.; responsewritten: [size_t*] Optional pointer receiving the number of bytes written to responseoutput..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid state, malformed records, authentication failure, unsupported post-handshake input, or insufficient buffer capacity.

qsc_tls_engine_request_key_update

Purpose: Initiate a TLS 1.3 KeyUpdate operation.

Details: Builds and encrypts a KeyUpdate post-handshake record using the active application write traffic keys, advances the local write traffic secret, enforces the RFC 9846 outbound KeyUpdate epoch limit, prevents a second update_requested message until a subsequent peer KeyUpdate is received, and writes the resulting record to the caller-supplied output buffer.

qsc_tls_status qsc_tls_engine_request_key_update(qsc_tls_connection* connection, bool requestpeerupdate, uint8_t* output, size_t outlen, size_t* written);

Parameters: connection: [qsc_tls_connection*] Pointer to an established TLS engine connection.; requestpeerupdate: [bool] If true, request that the peer also update its sending keys.; output: [uint8_t*] Pointer to the output buffer receiving the encrypted KeyUpdate record.; outlen: [size_t] Size, in bytes, of the output buffer.; written: [size_t*] Pointer receiving the number of bytes written to output..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid state, insufficient output capacity, or cryptographic failure.

qsc_tls_engine_take_session_ticket

Purpose: Take the most recently received TLS 1.3 session ticket.

Details: Copies the most recently processed NewSessionTicket metadata to ticketout and clears the engine-owned copy. This function is intended for callers using the normal application-data receive path, where post-handshake NewSessionTicket messages are dispatched automatically.

qsc_tls_status qsc_tls_engine_take_session_ticket(qsc_tls_connection* connection, qsc_tls_session_ticket* ticketout);

Parameters: connection: [qsc_tls_connection*] Pointer to an established client-role TLS engine connection.; ticketout: [qsc_tls_session_ticket*] Pointer receiving the processed session ticket..

Return: [qsc_tls_status] Returns qsc_tls_status_success when a ticket was available, or qsc_tls_status_invalid_state otherwise.

qsc_tls_engine_write_application_data

Purpose: Encrypt and frame application data.

Details: Encrypts plaintext application data under the active application write traffic keys and writes one or more TLS application-data records to the output buffer. The handshake must be complete before application data is written.

qsc_tls_status qsc_tls_engine_write_application_data(qsc_tls_connection* connection, const uint8_t* input, size_t inlen, uint8_t* output, size_t outlen, size_t* written);

Parameters: connection: [qsc_tls_connection*] Pointer to an established TLS engine connection.; input: [const uint8_t*] Pointer to plaintext application data.; inlen: [size_t] Number of plaintext bytes to encrypt.; output: [uint8_t*] Pointer to the caller-supplied output buffer.; outlen: [size_t] Size, in bytes, of the output buffer.; written: [size_t*] Pointer receiving the number of encrypted record bytes written to output..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid state, invalid input, insufficient output capacity, or encryption failure.

Header: tlserrors.h

qsc_tls_error_to_string

Purpose: Convert a TLS status code to a descriptive diagnostic string.

const char* qsc_tls_error_to_string(qsc_tls_status status);

Parameters: status: [enum] The TLS status code..

Return: : A constant descriptive string for the status code.

Header: tlsextensions.h

qsc_tls_extensions_bitmap_initialize

Purpose: Reset a TLS extension appearance bitmap.

Details: Clears all extension tracking masks in the supplied bitmap. The bitmap is used while parsing a single handshake message to detect duplicate extensions.

void qsc_tls_extensions_bitmap_initialize(qsc_tls_extension_bitmap* bitmap);

Parameters: bitmap: [qsc_tls_extension_bitmap*] Pointer to the extension bitmap to initialize..

Return: None (void).

qsc_tls_extensions_bitmap_set

Purpose: Mark an extension type as present in an appearance bitmap.

Details: Sets the bit associated with an extension type and reports whether the extension was newly recorded. This function is used to enforce the TLS 1.3 rule that an extension shall not appear more than once in the same extension block.

bool qsc_tls_extensions_bitmap_set(qsc_tls_extension_bitmap* bitmap, uint16_t extensiontype);

Parameters: bitmap: [qsc_tls_extension_bitmap*] Pointer to the extension appearance bitmap.; extensiontype: [uint16_t] The numeric TLS extension type to mark..

Return: [bool] Returns true if the extension was newly added; otherwise returns false if the extension was already present or the bitmap is NULL.

qsc_tls_extensions_decode_alpn

Purpose: Decode the application_layer_protocol_negotiation extension body.

Details: Parses an ALPN ProtocolNameList extension body into a bounded output policy container. Zero-length protocol identifiers, overlong identifiers, duplicate identifiers, and excess protocol entries are rejected.

qsc_tls_status qsc_tls_extensions_decode_alpn(const uint8_t* input, size_t inplen, qsc_tls_alpn_protocols* alpn);

Parameters: input: [const uint8_t*] Pointer to the ALPN extension body.; inplen: [size_t] Length, in bytes, of the ALPN extension body.; alpn: [struct*] Pointer to the output ALPN protocol-list container..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_cookie

Purpose: Decode a TLS cookie extension.

qsc_tls_status qsc_tls_extensions_decode_cookie(const uint8_t* input, size_t inplen, const uint8_t** cookie, size_t* cookielen);

Parameters: input: [const uint8_t*] Extension body.; inplen: [size_t] Extension body length.; cookie: [const uint8_t**] Receives a borrowed pointer to the cookie bytes.; cookielen: [size_t*] Receives the cookie length..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_early_data_max

Purpose: Decode an early_data maximum-size extension body.

Details: Parses a NewSessionTicket early_data extension body and returns the max_early_data_size value.

qsc_tls_status qsc_tls_extensions_decode_early_data_max(const uint8_t* input, size_t inplen, uint32_t* maxearlydatasize);

Parameters: input: [const uint8_t*] Pointer to the encoded early_data extension body.; inplen: [size_t] Length, in bytes, of the encoded extension body.; maxearlydatasize: [uint32_t*] Pointer receiving the decoded maximum early-data size..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_key_share_client_hello

Purpose: Decode a ClientHello key_share extension body.

Details: Parses a ClientHello key_share vector into non-owning group and public-share spans. The share pointers refer directly to the supplied input buffer.

qsc_tls_status qsc_tls_extensions_decode_key_share_client_hello(const uint8_t* input, size_t inplen, qsc_tls_named_group* groups, const uint8_t** shares, size_t* sharelens, size_t capacity, size_t* count);

Parameters: input: [const uint8_t*] Pointer to the encoded key_share extension body.; inplen: [size_t] Length, in bytes, of the encoded body.; groups: [qsc_tls_named_group*] Pointer to the output named-group array.; shares: [const uint8_t**] Pointer to the output public-share pointer array.; sharelens: [size_t*] Pointer to the output public-share length array.; capacity: [size_t] Maximum number of key-share entries that may be written to the output arrays.; count: [size_t*] Pointer receiving the number of decoded key-share entries..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_key_share_hello_retry

Purpose: Decode a HelloRetryRequest key_share extension body.

Details: Parses the requested named group from a HelloRetryRequest key_share extension body.

qsc_tls_status qsc_tls_extensions_decode_key_share_hello_retry(const uint8_t* input, size_t inplen, qsc_tls_named_group* requestedgroup);

Parameters: input: [const uint8_t*] Pointer to the encoded key_share extension body.; inplen: [size_t] Length, in bytes, of the encoded body.; requestedgroup: [qsc_tls_named_group*] Pointer receiving the requested named group..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_key_share_server_hello

Purpose: Decode a ServerHello key_share extension body.

Details: Parses the selected named group and server public-share span from a ServerHello key_share extension body. The returned share pointer refers directly to the supplied input buffer.

qsc_tls_status qsc_tls_extensions_decode_key_share_server_hello(const uint8_t* input, size_t inplen, qsc_tls_named_group* selectedgroup, const uint8_t** share, size_t* sharelen);

Parameters: input: [const uint8_t*] Pointer to the encoded key_share extension body.; inplen: [size_t] Length, in bytes, of the encoded body.; selectedgroup: [qsc_tls_named_group*] Pointer receiving the selected named group.; share: [const uint8_t**] Pointer receiving the public-share span.; sharelen: [size_t*] Pointer receiving the public-share length, in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_pre_shared_key_offer

Purpose: Decode a ClientHello pre_shared_key extension body.

Details: Parses an OfferedPsks body into PSK identity views and binder spans. The views point directly into the supplied input buffer and remain valid only while that buffer remains valid. The function also returns the offset of the binders-list length prefix so the caller can recompute binders over the correctly truncated ClientHello transcript.

qsc_tls_status qsc_tls_extensions_decode_pre_shared_key_offer(const uint8_t* input, size_t inplen, qsc_tls_psk_identity_view* identities, const uint8_t** binders, size_t* binderlens, size_t capacity, size_t* count, size_t* binderblockoffset);

Parameters: input: [const uint8_t*] Pointer to the encoded pre_shared_key extension body.; inplen: [size_t] Length, in bytes, of the encoded extension body.; identities:

[qsc_tls_psk_identity_view*] Pointer to the output identity view array.; binders: [const uint8_t**] Pointer to the output binder span array.; binderlens: [size_t*] Pointer to the output binder-length array.; capacity: [size_t] Maximum number of identities and binders that can be written to the output arrays.; count: [size_t*] Pointer receiving the number of identities and binders parsed.; binderblockoffset: [size_t*] Pointer receiving the offset of the binders-list length prefix within input..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_pre_shared_key_server

Purpose: Decode a ServerHello pre_shared_key extension body.

Details: Parses a ServerHello pre_shared_key extension body and returns the selected PSK identity index.

qsc_tls_status qsc_tls_extensions_decode_pre_shared_key_server(const uint8_t* input, size_t inplen, uint16_t* selidentity);

Parameters: input: [const uint8_t*] Pointer to the encoded pre_shared_key extension body.; inplen: [size_t] Length, in bytes, of the encoded extension body.; selidentity: [uint16_t*] Pointer receiving the selected PSK identity index..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_server_name

Purpose: Decode a server_name extension body.

Details: Parses a server_name extension containing a host_name entry. The returned hostname pointer refers directly to the supplied input buffer and is not NULL-terminated.

qsc_tls_status qsc_tls_extensions_decode_server_name(const uint8_t* input, size_t inplen, const char** hostname, size_t* hostnamelen);

Parameters: input: [const uint8_t*] Pointer to the encoded server_name extension body.; inplen: [size_t] Length, in bytes, of the encoded body.; hostname: [const char**] Pointer receiving the hostname span.; hostnamelen: [size_t*] Pointer receiving the hostname length, in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_signature_algorithms

Purpose: Decode a signature_algorithms extension body.

Details: Parses a signature-scheme vector and writes the decoded schemes to the caller-provided output array. This decoder is suitable for both signature_algorithms and signature_algorithms_cert bodies when the wire format is identical.

qsc_tls_status qsc_tls_extensions_decode_signature_algorithms(const uint8_t* input, size_t inplen, qsc_tls_signature_scheme* schemes, size_t schemecapacity, size_t* schemecount);

Parameters: input: [const uint8_t*] Pointer to the encoded signature-scheme vector body.; inplen: [size_t] Length, in bytes, of the encoded body.; schemes: [qsc_tls_signature_scheme*] Pointer to the output scheme array.; schemecapacity: [size_t] Maximum number of scheme entries that may be written to schemes.; schemecount: [size_t*] Pointer receiving the number of decoded schemes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_supported_groups

Purpose: Decode a supported_groups extension body.

Details: Parses the named-group vector from a supported_groups extension body and writes the decoded groups to the caller-provided output array.

qsc_tls_status qsc_tls_extensions_decode_supported_groups(const uint8_t* input, size_t inplen, qsc_tls_named_group* groups, size_t groupcapacity, size_t* groupcount);

Parameters: input: [const uint8_t*] Pointer to the encoded supported_groups body.; inplen: [size_t] Length, in bytes, of the encoded body.; groups: [qsc_tls_named_group*] Pointer to the output named-group array.; groupcapacity: [size_t] Maximum number of group entries that may be written to groups.; groupcount: [size_t*] Pointer receiving the number of decoded groups..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_supported_versions_client

Purpose: Decode a ClientHello supported_versions extension body.

Details: Parses the ClientHello supported_versions vector and reports whether TLS 1.3 is present in the advertised version list.

qsc_tls_status qsc_tls_extensions_decode_supported_versions_client(const uint8_t* input, size_t inplen, bool* acceptstls13);

Parameters: input: [const uint8_t*] Pointer to the encoded supported_versions body.; inplen: [size_t] Length, in bytes, of the encoded body.; acceptstls13: [bool*] Pointer receiving true if TLS 1.3 is advertised..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_decode_supported_versions_server

Purpose: Decode a ServerHello supported_versions extension body.

Details: Parses the selected protocol version from a ServerHello supported_versions extension body.

qsc_tls_status qsc_tls_extensions_decode_supported_versions_server(const uint8_t* input, size_t inplen, uint16_t* selectedversion);

Parameters: input: [const uint8_t*] Pointer to the encoded supported_versions body.; inplen: [size_t] Length, in bytes, of the encoded body.; selectedversion: [uint16_t*] Pointer receiving the selected protocol version..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_alpn

Purpose: Encode the application_layer_protocol_negotiation extension.

Details: Writes an ALPN extension containing the ordered protocol identifiers stored in the supplied bounded policy. Each protocol identifier is encoded as an opaque ProtocolName vector and must have non-zero length.

qsc_tls_status qsc_tls_extensions_encode_alpn(uint8_t* output, size_t outlen, size_t* offset, const qsc_tls_alpn_protocols* alpn);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; alpn: [const struct*] Pointer to the bounded ALPN protocol-list policy..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_cookie

Purpose: Encode a TLS cookie extension.

qsc_tls_status qsc_tls_extensions_encode_cookie(uint8_t* output, size_t outlen, size_t* offset, const uint8_t* cookie, size_t cookielen);

Parameters: output: [uint8_t*] Destination extension block.; outlen: [size_t] Destination capacity.; offset: [size_t*] In/out write offset.; cookie: [const uint8_t*] Cookie bytes.; cookielen: [size_t] Cookie length; must be non-zero and at most 65535 bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_early_data_empty

Purpose: Encode an empty early_data extension.

Details: Writes an early_data extension with a zero-length body. This form is used in ClientHello and EncryptedExtensions contexts.

qsc_tls_status qsc_tls_extensions_encode_early_data_empty(uint8_t* output, size_t outlen, size_t* offset);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_early_data_max

Purpose: Encode an early_data maximum-size extension.

Details: Writes an early_data extension containing the max_early_data_size value used in a NewSessionTicket context.

qsc_tls_status qsc_tls_extensions_encode_early_data_max(uint8_t* output, size_t outlen, size_t* offset, uint32_t maxearlydatasize);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; maxearlydatasize: [uint32_t] Maximum permitted early-data size, in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_key_share_client

Purpose: Encode the ClientHello key_share extension.

Details: Writes a ClientHello key_share extension containing one KeyShareEntry. The entry contains the named group identifier and the public key-exchange share.

qsc_tls_status qsc_tls_extensions_encode_key_share_client(uint8_t* output, size_t outlen, size_t* offset, qsc_tls_named_group group, const uint8_t* publicshare, size_t publicsharelen);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; group: [qsc_tls_named_group] The named group associated with the public share.; publicshare: [const uint8_t*] Pointer to the public key-exchange share.; publicsharelen: [size_t] Length, in bytes, of the public share..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_key_share_hello_retry

Purpose: Encode the HelloRetryRequest key_share extension.

Details: Writes a key_share extension in HelloRetryRequest form. The encoded body contains only the requested named group identifier.

qsc_tls_status qsc_tls_extensions_encode_key_share_hello_retry(uint8_t* output, size_t outlen, size_t* offset, qsc_tls_named_group group);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; group: [qsc_tls_named_group] The named group requested by the server..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_key_share_server

Purpose: Encode the ServerHello key_share extension.

Details: Writes a ServerHello key_share extension containing the selected named group and the server public key-exchange share.

qsc_tls_status qsc_tls_extensions_encode_key_share_server(uint8_t* output, size_t outlen, size_t* offset, qsc_tls_named_group group, const uint8_t* publicshare, size_t publicsharelen);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write

offset; updated on success.; group: [qsc_tls_named_group] The selected named group.; publicshare: [const uint8_t*] Pointer to the server public share.; publicsharelen: [size_t] Length, in bytes, of the public share..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_pre_shared_key_offer

Purpose: Encode a ClientHello pre_shared_key extension.

Details: Writes a ClientHello pre_shared_key extension body containing OfferedPsks. The identities vector is encoded first, followed by a binders vector. Binder entries are emitted as zero-filled placeholders of the requested length. The caller must compute and backpatch the real binder values after hashing the truncated ClientHello transcript.

qsc_tls_status qsc_tls_extensions_encode_pre_shared_key_offer(uint8_t* output, size_t outlen, size_t* offset, const qsc_tls_psk_identity_view* identities, size_t identitycount, size_t binderlen, size_t* binderoffset);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; identities: [const qsc_tls_psk_identity_view*] Pointer to the PSK identity view array.; identitycount: [size_t] Number of PSK identities to encode.; binderlen: [size_t] Length, in bytes, of each binder placeholder.; binderoffset: [size_t*] Pointer receiving the absolute output offset of the first binder byte..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_pre_shared_key_server

Purpose: Encode a ServerHello pre_shared_key extension.

Details: Writes a ServerHello pre_shared_key extension containing the selected PSK identity index.

qsc_tls_status qsc_tls_extensions_encode_pre_shared_key_server(uint8_t* output, size_t outlen, size_t* offset, uint16_t selidentity);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; selidentity: [uint16_t] The selected PSK identity index..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_psk_key_exchange_modes

Purpose: Encode the psk_key_exchange_modes extension.

Details: Writes a psk_key_exchange_modes extension containing the supplied list of PSK key exchange mode identifiers.

qsc_tls_status qsc_tls_extensions_encode_psk_key_exchange_modes(uint8_t* output, size_t outlen, size_t* offset, const uint8_t* modes, size_t modecount);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; modes: [const uint8_t*] Pointer to the PSK key exchange mode list.; modecount: [size_t] Number of mode identifiers in the list..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_server_name

Purpose: Encode the server_name extension.

Details: Writes a server_name extension containing a single host_name entry. The hostname is copied without a terminating NULL byte.

qsc_tls_status qsc_tls_extensions_encode_server_name(uint8_t* output, size_t outlen, size_t* offset, const char* hostname);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; hostname: [const char*] Pointer to the NULL-terminated hostname string..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_signature_algorithms

Purpose: Encode the signature_algorithms extension.

Details: Writes a signature_algorithms extension containing the supplied ordered list of supported signature schemes.

qsc_tls_status qsc_tls_extensions_encode_signature_algorithms(uint8_t* output, size_t outlen, size_t* offset, const qsc_tls_signature_scheme* schemes, size_t schemecount);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write

offset; updated on success.; schemes: [const qsc_tls_signature_scheme*] Pointer to the signature scheme list.; schemecount: [size_t] Number of entries in the signature scheme list..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_signature_algorithms_cert

Purpose: Encode the signature_algorithms_cert extension.

Details: Writes a signature_algorithms_cert extension containing the supplied ordered list of supported certificate signature schemes.

qsc_tls_status qsc_tls_extensions_encode_signature_algorithms_cert(uint8_t* output, size_t outlen, size_t* offset, const qsc_tls_signature_scheme* schemes, size_t schemecount);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; schemes: [const qsc_tls_signature_scheme*] Pointer to the certificate signature scheme list.; schemecount: [size_t] Number of entries in the certificate signature scheme list..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_supported_groups

Purpose: Encode the supported_groups extension.

Details: Writes a supported_groups extension containing the supplied ordered list of named groups.

qsc_tls_status qsc_tls_extensions_encode_supported_groups(uint8_t* output, size_t outlen, size_t* offset, const qsc_tls_named_group* groups, size_t groupcount);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success.; groups: [const qsc_tls_named_group*] Pointer to the named-group list.; groupcount: [size_t] Number of entries in the named-group list..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_encode_supported_versions_client

Purpose: Encode the ClientHello supported_versions extension.

Details: Writes a TLS supported_versions extension in ClientHello format. The encoded extension advertises TLS 1.3 as the supported protocol version.

qsc_tls_status qsc_tls_extensions_encode_supported_versions_client(uint8_t* output, size_t outlen, size_t* offset);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success,.

qsc_tls_extensions_encode_supported_versions_server

Purpose: Encode the ServerHello supported_versions extension.

Details: Writes a TLS supported_versions extension in ServerHello format. The encoded extension contains the selected TLS 1.3 protocol version.

qsc_tls_status qsc_tls_extensions_encode_supported_versions_server(uint8_t* output, size_t outlen, size_t* offset);

Parameters: output: [uint8_t*] Pointer to the destination extension buffer.; outlen: [size_t] Size, in bytes, of the destination buffer.; offset: [size_t*] Pointer to the current write offset; updated on success..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_is_permitted

Purpose: Test whether an extension is permitted in a handshake message.

Details: Applies TLS 1.3 extension placement rules for the extension types supported by this implementation. The function returns true only when the specified extension type is valid for the supplied handshake message type.

bool qsc_tls_extensions_is_permitted(qsc_tls_handshake_type message, qsc_tls_extension_type extensiontype);

Parameters: message: [qsc_tls_handshake_type] The TLS handshake message type being parsed or constructed.; extensiontype: [qsc_tls_extension_type] The TLS extension type to test..

Return: [bool] Returns true if the extension is permitted in the specified message; otherwise returns false.

qsc_tls_extensions_select_alpn

Purpose: Select a mutually supported ALPN protocol.

Details: Selects the first server-preferred protocol that appears in the client protocol list.

qsc_tls_status qsc_tls_extensions_select_alpn(const qsc_tls_alpn_protocols* clientalpn, const qsc_tls_alpn_protocols* serveralpn, uint8_t* selected, size_t selectedcap, size_t* selectedlen);

Parameters: clientalpn: [const struct*] Pointer to the decoded client ALPN list.; serveralpn: [const struct*] Pointer to the server ALPN policy.; selected: [uint8_t*] Pointer to the selected protocol output buffer.; selectedcap: [size_t] Size, in bytes, of the selected protocol output buffer.; selectedlen: [size_t*] Receives the selected protocol length in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_select_cipher_suite

Purpose: Select a mutually supported cipher suite.

Details: Selects the first server-preferred cipher suite that appears in the serialized client cipher-suite vector. The client vector is expected to contain the inner cipher-suite list bytes, encoded as two-byte suite identifiers.

qsc_tls_status qsc_tls_extensions_select_cipher_suite(const uint8_t* clientsuites, size_t clientsuiteslen, const qsc_tls_cipher_suite* serverpreference, size_t serverpreferencecount, qsc_tls_cipher_suite* selected);

Parameters: clientsuites: [const uint8_t*] Pointer to the serialized client cipher-suite list.; clientsuiteslen: [size_t] Length, in bytes, of the client cipher-suite list.; serverpreference: [const qsc_tls_cipher_suite*] Pointer to the ordered server cipher-suite preference list.; serverpreferencecount: [size_t] Number of entries in the server preference list.; selected: [qsc_tls_cipher_suite*] Pointer receiving the selected cipher suite..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_extensions_select_key_share

Purpose: Select a mutually supported key-share group.

Details: Selects the first server-preferred named group that appears in the client group list.

qsc_tls_status qsc_tls_extensions_select_key_share(const qsc_tls_named_group* groups, size_t groupcount, const qsc_tls_named_group* serverpreference, size_t serverpreferencecount, qsc_tls_named_group* selected);

Parameters: groups: [const qsc_tls_named_group*] Pointer to the client named-group list.; groupcount: [size_t] Number of entries in the client named-group list.; serverpreference: [const qsc_tls_named_group*] Pointer to the ordered server named-group preference list.; serverpreferencecount: [size_t] Number of entries in the server preference list.; selected: [qsc_tls_named_group*] Pointer receiving the selected named group..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

Header: tlsgroups.h

qsc_tls_groups_client_derive_shared_secret

Purpose: Derive the client-side shared secret from the peer server key share.

Details: Validates the server key-share length against the descriptor for the active named group and derives the shared secret or hybrid secret concatenation into the caller-supplied output buffer.

qsc_tls_status qsc_tls_groups_client_derive_shared_secret(qsc_tls_key_exchange_state* state, const uint8_t* serverkeyshare, size_t serverkeysharelen, uint8_t* sharedsecret, size_t sharedsecretlen, size_t* written);

Parameters: state: [struct*] The initialized client key-exchange state.; serverkeyshare: [const uint8_t*] The peer server key-share bytes.; serverkeysharelen: [size_t] The server key-share length in bytes.; sharedsecret: [uint8_t*] The destination buffer for the derived shared secret.; sharedsecretlen: [size_t] The destination buffer length in bytes.; written: [size_t*] Receives the number of bytes written to sharedsecret..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_groups_descriptor_get

Purpose: Get the descriptor for a named group.

const qsc_tls_group_descriptor* qsc_tls_groups_descriptor_get(qsc_tls_named_group group);

Parameters: group: [enum] The TLS named group..

Return: [const qsc_tls_group_descriptor*] Returns a pointer to the descriptor, or NULL if the group is unknown.

qsc_tls_groups_generate_client_keypair

Purpose: Generate a client ephemeral key pair for a named group.

Details: Clears the state structure, generates the ephemeral private key material, emits the corresponding client key share into the publicshare buffer, and records the selected group and lengths for later shared-secret derivation.

qsc_tls_status qsc_tls_groups_generate_client_keypair(qsc_tls_key_exchange_state* state, qsc_tls_named_group group);

Parameters: state: [struct*] The key-exchange state to initialize.; group: [enum] The named group to generate..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_groups_is_supported

Purpose: Determine whether a named group is supported in the current build.

bool qsc_tls_groups_is_supported(qsc_tls_named_group group);

Parameters: group: [enum] The TLS named group..

Return: [bool] Returns true if the named group is available.

qsc_tls_groups_key_exchange_state_dispose

Purpose: Dispose of a key-exchange state and zeroize retained key material.

void qsc_tls_groups_key_exchange_state_dispose(qsc_tls_key_exchange_state* state);

Parameters: state: [struct*] The key-exchange state to clear..

Return: None (void).

qsc_tls_groups_server_respond

Purpose: Generate the server response key share and shared secret for an offered client key share.

Details: Implements the server-side half of the named-group primitive. Depending on the selected group, the function may perform Diffie-Hellman key generation, KEM encapsulation, or a hybrid composition that concatenates multiple component outputs.

qsc_tls_status qsc_tls_groups_server_respond(qsc_tls_named_group group, const uint8_t* clientkeyshare, size_t clientkeysharelen, uint8_t* serverkeyshare, size_t serverkeysharelen, size_t* serverkeysharewritten, uint8_t* sharedsecret, size_t sharedsecretlen, size_t* sharedsecretwritten);

Parameters: group: [enum] The negotiated named group.; clientkeyshare: [const uint8_t*] The client key-share bytes.; clientkeysharelen: [size_t] The client key-share length in bytes.; serverkeyshare: [uint8_t*] The destination buffer for the encoded server response key share.; serverkeysharelen: [size_t] The destination buffer length in bytes.; serverkeysharewritten: [size_t*] Receives the number of bytes written to serverkeyshare.; sharedsecret: [uint8_t*] The destination buffer for the derived shared secret.; sharedsecretlen: [size_t] The destination buffer length in bytes.; sharedsecretwritten: [size_t*] Receives the number of bytes written to sharedsecret..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

Header: tlshandshake.h

qsc_tls_handshake_decode_certificate_verify

Purpose: Decode the body of a CertificateVerify handshake message.

Details: The returned signature span aliases the input buffer.

qsc_tls_status qsc_tls_handshake_decode_certificate_verify(const uint8_t* input, size_t inlen, qsc_tls_signature_scheme* scheme, const uint8_t** signature, size_t* signaturelen);

Parameters: input: [const uint8_t*] The CertificateVerify message body.; inlen: [size_t] The body length in bytes.; scheme: [enum*] Receives the signature scheme identifier.; signature: [const uint8_t**] Receives a pointer to the encoded signature span.; signaturelen: [size_t*] Receives the signature length in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_decode_finished

Purpose: Decode the body of a Finished handshake message.

Details: The returned verify-data span aliases the input buffer.

qsc_tls_status qsc_tls_handshake_decode_finished(const uint8_t* input, size_t inlen, const uint8_t** verifydata, size_t* verifydatalen);

Parameters: input: [const uint8_t*] The Finished message body.; inlen: [size_t] The body length in bytes.; verifydata: [const uint8_t**] Receives a pointer to the verify_data span.; verifydatalen: [size_t*] Receives the verify_data length in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_decode_key_update

Purpose: Decode the body of a KeyUpdate handshake message.

Details: Requires the exact one-byte TLS 1.3 KeyUpdate body and accepts only the update_not_requested and update_requested values.

qsc_tls_status qsc_tls_handshake_decode_key_update(const uint8_t* input, size_t inlen, bool* requestupdate);

Parameters: input: [const uint8_t*] The KeyUpdate message body.; inlen: [size_t] The body length in bytes.; requestupdate: [bool*] Receives true when the peer requests a reciprocal key update..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_encode_certificate_verify

Purpose: Encode the body of a CertificateVerify handshake message.

qsc_tls_status qsc_tls_handshake_encode_certificate_verify(uint8_t* output, size_t outlen, size_t* offset, qsc_tls_signature_scheme scheme, const uint8_t* signature, size_t signaturelen);

Parameters: output: [uint8_t*] The destination output buffer.; outlen: [size_t] The destination buffer length in bytes.; offset: [size_t*] On input, the starting write offset; on success, advanced past the encoded body.; scheme: [enum] The signature scheme identifier.; signature: [const uint8_t*] The encoded signature bytes.; signaturelen: [size_t] The signature length in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_encode_encrypted_extensions

Purpose: Encode the body of an EncryptedExtensions handshake message.

Details: Writes the supplied extensions block as the TLS vector carried by the message body. The caller is responsible for constructing the extension block contents.

qsc_tls_status qsc_tls_handshake_encode_encrypted_extensions(uint8_t* output, size_t outlen, size_t* offset, const uint8_t* extensions, size_t extensionslen);

Parameters: output: [uint8_t*] The destination output buffer.; outlen: [size_t] The destination buffer length in bytes.; offset: [size_t*] On input, the starting write offset; on success, advanced past the encoded body.; extensions: [const uint8_t*] The raw encoded extension block.; extensionslen: [size_t] The extension block length in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_encode_finished

Purpose: Encode the body of a Finished handshake message.

qsc_tls_status qsc_tls_handshake_encode_finished(uint8_t* output, size_t outlen, size_t* offset, const uint8_t* verifydata, size_t verifydatalen);

Parameters: output: [uint8_t*] The destination output buffer.; outlen: [size_t] The destination buffer length in bytes.; offset: [size_t*] On input, the starting write offset; on success, advanced past the encoded body.; verifydata: [const uint8_t*] The computed Finished verify_data bytes.; verifydatalen: [size_t] The verify_data length in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_encode_key_update

Purpose: Encode the body of a KeyUpdate handshake message.

qsc_tls_status qsc_tls_handshake_encode_key_update(uint8_t* output, size_t outlen, size_t* offset, bool requestupdate);

Parameters: output: [uint8_t*] The destination output buffer.; outlen: [size_t] The destination buffer length in bytes.; offset: [size_t*] On input, the starting write offset; on success, advanced past the encoded body.; requestupdate: [bool] True to request that the peer also update its sending traffic keys..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_read_header

Purpose: Read a TLS handshake header.

qsc_tls_status qsc_tls_handshake_read_header(const uint8_t* input, size_t inlen, size_t* offset, qsc_tls_handshake_type* type, size_t* bodylen);

Parameters: input: [const uint8_t*] The input buffer containing the handshake header.; inlen: [size_t] The number of bytes available in input.; offset: [size_t*] On input, the starting read offset; on success, advanced by four bytes.; type: [enum*] Receives the decoded handshake type.; bodylen: [size_t*] Receives the decoded handshake body length in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_send_change_cipher_spec_compat

Purpose: Encode the compatibility ChangeCipherSpec record used by TLS 1.3 middlebox mode.

Details: Emits a plaintext TLS record carrying a one-byte ChangeCipherSpec fragment with value 0x01. This helper produces the full TLSPlaintext record rather than a handshake message body.

qsc_tls_status qsc_tls_handshake_send_change_cipher_spec_compat(uint8_t* output, size_t outlen, size_t* written);

Parameters: output: [uint8_t*] The destination output buffer.; outlen: [size_t] The destination buffer length in bytes.; written: [size_t*] Receives the number of bytes written..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_handshake_write_header

Purpose: Write a TLS handshake header.

Details: Writes the 1-byte handshake type followed by the 24-bit body length field. The function emits only the header and does not append the message body.

qsc_tls_status qsc_tls_handshake_write_header(uint8_t* output, size_t outlen, size_t* offset, qsc_tls_handshake_type type, size_t bodylen);

Parameters: output: [uint8_t*] The destination output buffer.; outlen: [size_t] The destination buffer length in bytes.; offset: [size_t*] On input, the starting write offset; on success, advanced by four bytes.; type: [enum] The handshake message type.; bodylen: [size_t] The handshake body length in bytes..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

Header: tlsio.h

qsc_tls_io_handshake_ex

Purpose: Drive the TLS handshake with an explicit cumulative wall-clock deadline.

Details: The timeout bounds the entire handshake independently of socket receive/send timeouts. Before each blocking socket operation, the adapter waits only for the time remaining in this budget. A timeout value of zero disables the cumulative deadline and preserves transport-level timeout behavior.

qsc_tls_status qsc_tls_io_handshake_ex(qsc_tls_io_connection* io, uint32_t timeoutms);

Parameters: io: [struct*] The attached I/O adapter.; timeoutms: [uint32_t] The cumulative handshake deadline in milliseconds; zero disables it..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success or qsc_tls_status_timeout when the deadline expires.

qsc_tls_io_shutdown

Purpose: Emit and send a close_notify alert for the attached connection.

qsc_tls_status qsc_tls_io_shutdown(qsc_tls_io_connection* io);

Parameters: io: [struct*] The attached I/O adapter..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

Header: tlskeyschedule.h

qsc_tls_keyschedule_advance_traffic_secret

Purpose: Advance an application traffic secret for TLS KeyUpdate.

Details: Computes the next traffic secret from the current traffic secret using the TLS 1.3 "traffic upd" label. The caller is responsible for replacing the active read or write traffic secret and deriving new record protection keys.

qsc_tls_status qsc_tls_keyschedule_advance_traffic_secret(qsc_tls_hash_algorithm hash, const uint8_t* currenttrafficsecret, size_t trafficsecretlen, uint8_t* nexttrafficsecret);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; currenttrafficsecret: [const uint8_t*] Pointer to the current traffic secret.; trafficsecretlen: [size_t] Length, in

bytes, of the current traffic secret.; nexttrafficsecret: [uint8_t*] Pointer to the destination buffer for the next traffic secret..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_build_certificate_verify_input

Purpose: Build the TLS 1.3 CertificateVerify signature input.

Details: Constructs the exact CertificateVerify input defined by TLS 1.3: 64 space characters, followed by the role-specific context string, followed by a single zero byte separator, followed by the transcript hash. The resulting buffer is the message that is signed or verified by the certificate authentication layer.

qsc_tls_status qsc_tls_keyschedule_build_certificate_verify_input(const char* contextstring, const uint8_t* transcripthash, size_t transcripthashlen, uint8_t* output, size_t outlen, size_t* written);

Parameters: contextstring: [const char*] Pointer to the role-specific context string, for example "TLS 1.3, server CertificateVerify".; transcripthash: [const uint8_t*] Pointer to the transcript hash at the CertificateVerify boundary.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash.; output: [uint8_t*] Pointer to the destination buffer.; outlen: [size_t] Length, in bytes, of the destination buffer.; written: [size_t*] Pointer receiving the number of bytes written..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_compute_finished

Purpose: Compute a TLS 1.3 Finished verify_data value.

Details: Derives finished_key from basekey using HKDF-Expand-Label with the "finished" label and computes verify_data as HMAC(finished_key, transcript_hash). The output length is the selected hash digest size.

qsc_tls_status qsc_tls_keyschedule_compute_finished(qsc_tls_hash_algorithm hash, const uint8_t* basekey, size_t basekeylen, const uint8_t* transcripthash, size_t transcripthashlen, uint8_t* output, size_t outlen, size_t* written);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; basekey: [const uint8_t*] Pointer to the base traffic secret used to derive finished_key.; basekeylen: [size_t] Length, in bytes, of the base key.; transcripthash: [const uint8_t*] Pointer to the transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash.; output: [uint8_t*] Pointer to the verify_data output buffer.; outlen: [size_t] Length, in

bytes, of the output buffer.; written: [size_t*] Pointer receiving the number of bytes written..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_compute_psk_binder

Purpose: Compute a TLS 1.3 PSK binder MAC.

Details: Computes the PSK binder as HMAC(finished_key, transcript_hash_up_to_binders), where finished_key is derived from the binder key by HKDF-Expand-Label using the "finished" label. The transcript hash must cover the ClientHello through the PSK identities vector and must exclude the binder values themselves.

qsc_tls_status qsc_tls_keyschedule_compute_psk_binder(qsc_tls_hash_algorithm hash, const uint8_t* binderkey, size_t binderkeylen, const uint8_t* partialtranshash, size_t transcripthashlen, uint8_t* output, size_t outlen, size_t* written);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; binderkey: [const uint8_t*] Pointer to the derived binder key.; binderkeylen: [size_t] Length, in bytes, of the binder key.; partialtranshash: [const uint8_t*] Pointer to the ClientHello transcript hash up to the binders.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash.; output: [uint8_t*] Pointer to the binder output buffer.; outlen: [size_t] Length, in bytes, of the output buffer.; written: [size_t*] Pointer receiving the number of binder bytes written..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_application_traffic_secrets

Purpose: Derive the client and server application traffic secrets.

Details: Derives c ap traffic 0 and s ap traffic 0 from the Main Secret using the transcript hash that includes the server Finished message. These secrets are used to derive application-data record protection keys.

qsc_tls_status
qsc_tls_keyschedule_derive_application_traffic_secrets(qsc_tls_key_schedule_state* state, const uint8_t* transcripthash, size_t transcripthashlen);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with the Main Secret derived.; transcripthash: [const uint8_t*] Pointer to the

application traffic transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_binder_key

Purpose: Derive the TLS 1.3 PSK binder key.

Details: Derives the binder key from early_secret for PSK binder computation. The external flag selects the TLS 1.3 "ext binder" label for externally provisioned PSKs or the "res binder" label for resumption PSKs. The early secret must already have been extracted.

qsc_tls_status qsc_tls_keyschedule_derive_binder_key(qsc_tls_key_schedule_state* state, bool external);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with early_secret derived.; external: [bool] Set to true for external PSKs; set to false for resumption PSKs..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, or an error status on invalid state or derivation failure.

qsc_tls_keyschedule_derive_client_early_traffic_secret

Purpose: Derive the client early traffic secret for 0-RTT data.

Details: Derives client_early_traffic_secret from early_secret using the ClientHello transcript hash up to, but not including, the PSK binder values. The early secret must already have been extracted.

qsc_tls_status
qsc_tls_keyschedule_derive_client_early_traffic_secret(qsc_tls_key_schedule_state* state, const uint8_t* transcripthash, size_t transcripthashlen);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with early_secret derived.; transcripthash: [const uint8_t*] Pointer to the partial ClientHello transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_early_exporter_secret

Purpose: Derive the RFC 9846 early exporter secret.

Details: Derives the RFC 9846 early exporter secret from early_secret using the supplied ClientHello transcript hash. This secret is used by exporter interfaces that are valid during early-data processing.

qsc_tls_status
qsc_tls_keyschedule_derive_early_exporter_secret(qsc_tls_key_schedule_state* state, const uint8_t* transcripthash, size_t transcripthashlen);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with early_secret derived.; transcripthash: [const uint8_t*] Pointer to the relevant ClientHello transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_exporter_master_secret

Purpose: Derive the RFC 9846 exporter secret.

Details: Derives the RFC 9846 exporter secret from the Main Secret and the supplied transcript hash. The resulting secret may be used by exporter interfaces that bind external application keys to the TLS session.

qsc_tls_status
qsc_tls_keyschedule_derive_exporter_master_secret(qsc_tls_key_schedule_state* state, const uint8_t* transcripthash, size_t transcripthashlen);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with the Main Secret derived.; transcripthash: [const uint8_t*] Pointer to the exporter transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_handshake_traffic_secrets

Purpose: Derive the client and server handshake traffic secrets.

Details: Derives c hs traffic and s hs traffic from handshake_secret using the ClientHello through ServerHello transcript hash. These secrets are used to derive the handshake record protection keys.

qsc_tls_status
qsc_tls_keyschedule_derive_handshake_traffic_secrets(qsc_tls_key_schedule_state* state, const uint8_t* transcripthash, size_t transcripthashlen);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with handshake_secret derived.; transcripthash: [const uint8_t*] Pointer to the ClientHello through ServerHello transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_resumption_master_secret

Purpose: Derive the RFC 9846 resumption secret.

Details: Derives the RFC 9846 resumption secret from the Main Secret and the supplied transcript hash. The resulting secret is used with per-ticket nonces to derive session resumption PSKs.

qsc_tls_status
qsc_tls_keyschedule_derive_resumption_master_secret(qsc_tls_key_schedule_state* state, const uint8_t* transcripthash, size_t transcripthashlen);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with the Main Secret derived.; transcripthash: [const uint8_t*] Pointer to the resumption transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_resumption_psk

Purpose: Derive a resumption PSK from the RFC 9846 resumption secret.

Details: Computes PSK = HKDF-Expand-Label(resumption_secret, "resumption", ticket_nonce, Hash.length). The caller must derive the resumption secret before invoking this function.

qsc_tls_status qsc_tls_keyschedule_derive_resumption_psk(const qsc_tls_key_schedule_state* state, const uint8_t* nonce, size_t noncelen, uint8_t* output, size_t outlen);

Parameters: state: [const qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with the resumption secret derived.; nonce: [const uint8_t*] Pointer to the

per-ticket nonce from the NewSessionTicket message.; noncelen: [size_t] Length, in bytes, of the ticket nonce.; output: [uint8_t*] Pointer to the destination PSK buffer.; outlen: [size_t] Number of PSK bytes to derive, typically Hash.length..

Return: [qsc_tls_status] Returns qsc_tls_status_success.

qsc_tls_keyschedule_derive_secret_empty

Purpose: Derive a TLS 1.3 secret using the hash of the empty string.

Details: Computes Derive-Secret(secret, label, "") by using the digest of the empty transcript as the HKDF-Expand-Label context. This operation is used for the TLS 1.3 derived-secret boundary between extraction stages.

qsc_tls_status qsc_tls_keyschedule_derive_secret_empty(qsc_tls_hash_algorithm hash, const uint8_t* secret, size_t secretlen, const char* label, size_t labellen, uint8_t* output, size_t outlen);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; secret: [const uint8_t*] Pointer to the base secret.; secretlen: [size_t] Length, in bytes, of the base secret.; label: [const char*] Pointer to the derivation label.; labellen: [size_t] Length, in bytes, of label.; output: [uint8_t*] Pointer to the destination secret buffer.; outlen: [size_t] Length, in bytes, of the destination buffer..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_secret_with_hash

Purpose: Derive a TLS 1.3 secret using a supplied transcript hash.

Details: Implements Derive-Secret(secret, label, transcript_hash) by invoking HKDF-Expand-Label, with the supplied transcript hash as the context and an output length equal to the selected hash digest size unless otherwise constrained by outlen.

qsc_tls_status qsc_tls_keyschedule_derive_secret_with_hash(qsc_tls_hash_algorithm hash, const uint8_t* secret, size_t secretlen, const char* label, size_t labellen, const uint8_t* transcripthash, size_t transcripthashlen, uint8_t* output, size_t outlen);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; secret: [const uint8_t*] Pointer to the base secret.; secretlen: [size_t] Length, in bytes, of the base secret.; label: [const char*] Pointer to the derivation label.; labellen: [size_t] Length, in bytes, of label.; transcripthash: [const uint8_t*] Pointer to the transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash.; output: [uint8_t*]

Pointer to the destination secret buffer.; outlen: [size_t] Length, in bytes, of the destination buffer..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_derive_traffic_keys

Purpose: Derive record protection key and IV material from a traffic secret.

Details: Expands a traffic secret into the AEAD record protection key and base IV using the TLS 1.3 "key" and "iv" labels. The caller supplies the key and IV lengths associated with the negotiated cipher suite.

qsc_tls_status qsc_tls_keyschedule_derive_traffic_keys(qsc_tls_hash_algorithm hash, const uint8_t* trafficsecret, size_t trafficsecretlen, size_t keylen, size_t ivlen, uint8_t* keyoutput, uint8_t* ivoutput);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; trafficsecret: [const uint8_t*] Pointer to the traffic secret.; trafficsecretlen: [size_t] Length, in bytes, of the traffic secret.; keylen: [size_t] Required record protection key length, in bytes.; ivlen: [size_t] Required record protection IV length, in bytes.; keyoutput: [uint8_t*] Pointer to the destination key buffer.; ivoutput: [uint8_t*] Pointer to the destination IV buffer..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_extract_early_secret

Purpose: Extract the TLS 1.3 early secret.

Details: Computes early_secret = HKDF-Extract(0, PSK). When no PSK is used, the caller supplies NULL with a zero length and the implementation performs the no-PSK TLS 1.3 flow.

qsc_tls_status qsc_tls_keyschedule_extract_early_secret(qsc_tls_key_schedule_state* state, const uint8_t* psk, size_t psklen);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state.; psk: [const uint8_t*] Pointer to the optional PSK, or NULL when psklen is zero.; psklen: [size_t] Length, in bytes, of the PSK..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_extract_handshake_secret

Purpose: Extract the TLS 1.3 handshake secret.

Details: Computes the handshake secret from the derived early-secret boundary value and the supplied DHE or hybrid shared secret. The early secret must already have been extracted.

qsc_tls_status qsc_tls_keyschedule_extract_handshake_secret(qsc_tls_key_schedule_state* state, const uint8_t* dhe, size_t dhelen);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with early_secret derived.; dhe: [const uint8_t*] Pointer to the DHE, KEM, or hybrid shared secret.; dhelen: [size_t] Length, in bytes, of the shared secret..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_extract_master_secret

Purpose: Extract the TLS 1.3 Main Secret.

Details: Computes the RFC 9846 Main Secret from the derived handshake-secret boundary value and an all-zero input keying material value. The handshake secret must already have been extracted.

qsc_tls_status qsc_tls_keyschedule_extract_master_secret(qsc_tls_key_schedule_state* state);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to an initialized key schedule state with handshake_secret derived..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_hkdf_expand

Purpose: Perform HKDF-Expand for the selected TLS hash algorithm.

Details: Expands a pseudorandom key into output keying material using the supplied HKDF info value. This is the generic HKDF-Expand primitive used internally by the TLS 1.3 label expansion functions.

qsc_tls_status qsc_tls_keyschedule_hkdf_expand(qsc_tls_hash_algorithm hash, const uint8_t* prk, size_t prklen, const uint8_t* info, size_t infolen, uint8_t* output, size_t outlen);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; prk: [const uint8_t*] Pointer to the pseudorandom key.; prklen: [size_t] Length, in bytes, of the pseudorandom key.; info: [const uint8_t*] Pointer to the HKDF info field, or NULL when infolen is zero.; infolen: [size_t] Length, in bytes, of the HKDF info field.; output: [uint8_t*]

Pointer to the destination output keying material.; outlen: [size_t] Number of bytes to derive..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_hkdf_expand_label

Purpose: Perform TLS 1.3 HKDF-Expand-Label.

Details: Encodes the TLS 1.3 HkdfLabel structure as: uint16 length, opaque label vector containing the literal prefix "tls13 " followed by label, and an opaque context vector. The encoded label is then supplied to HKDF-Expand.

qsc_tls_status qsc_tls_keyschedule_hkdf_expand_label(qsc_tls_hash_algorithm hash, const uint8_t* secret, size_t secretlen, const char* label, size_t labellen, const uint8_t* context, size_t contextlen, uint8_t* output, size_t outlen);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; secret: [const uint8_t*] Pointer to the input secret.; secretlen: [size_t] Length, in bytes, of the input secret.; label: [const char*] Pointer to the TLS label string without the "tls13 " prefix.; labellen: [size_t] Length, in bytes, of label.; context: [const uint8_t*] Pointer to the context value, or NULL when contextlen is zero.; contextlen: [size_t] Length, in bytes, of the context value.; output: [uint8_t*] Pointer to the destination output buffer.; outlen: [size_t] Number of bytes to derive..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_hkdf_extract

Purpose: Perform HKDF-Extract for the selected TLS hash algorithm.

Details: Computes an HKDF pseudorandom key from the supplied salt and input keying material. The output length must match the digest size of the selected hash algorithm.

qsc_tls_status qsc_tls_keyschedule_hkdf_extract(qsc_tls_hash_algorithm hash, const uint8_t* salt, size_t saltlen, const uint8_t* ikm, size_t ikmlen, uint8_t* output, size_t outlen);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; salt: [const uint8_t*] Pointer to the salt buffer, or NULL when saltlen is zero.; saltlen: [size_t] Length, in bytes, of the salt buffer.; ikm: [const uint8_t*] Pointer to the input keying material.; ikmlen: [size_t] Length, in bytes, of the input keying material.; output: [uint8_t*] Pointer to the destination pseudorandom key buffer.; outlen: [size_t] Length, in bytes, of the destination buffer..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_state_dispose

Purpose: Dispose of a TLS key schedule state.

Details: Zeroizes all stored secrets, clears state flags, and returns the state object to an inert value. This function should be called for every initialized key schedule state before the associated connection is released.

void qsc_tls_keyschedule_state_dispose(qsc_tls_key_schedule_state* state);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to the key schedule state to dispose..

Return: None (void).

qsc_tls_keyschedule_state_initialize

Purpose: Initialize a TLS key schedule state.

Details: Clears the supplied state object, records the negotiated hash algorithm, and resolves the digest size used by all subsequent key schedule operations.

qsc_tls_status qsc_tls_keyschedule_state_initialize(qsc_tls_key_schedule_state* state, qsc_tls_hash_algorithm hash);

Parameters: state: [qsc_tls_key_schedule_state*] Pointer to the key schedule state to initialize.; hash: [qsc_tls_hash_algorithm] Negotiated TLS hash algorithm..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_suite_hash

Purpose: Resolve the hash algorithm associated with a TLS cipher suite.

Details: Returns the transcript and HKDF hash algorithm used by the specified cipher suite. Unsupported suites return the implementation-defined invalid or none hash selector.

qsc_tls_hash_algorithm qsc_tls_keyschedule_suite_hash(qsc_tls_cipher_suite suite);

Parameters: suite: [qsc_tls_cipher_suite] TLS cipher suite selector..

Return: [qsc_tls_hash_algorithm] Returns the hash algorithm associated with the cipher suite.

qsc_tls_keyschedule_suite_record_sizes

Purpose: Resolve record protection key and IV lengths for a TLS cipher suite.

Details: Maps the negotiated cipher suite to the AEAD key length and base IV length required by TLS record protection.

qsc_tls_status qsc_tls_keyschedule_suite_record_sizes(qsc_tls_cipher_suite suite, size_t* keylen, size_t* ivlen);

Parameters: suite: [qsc_tls_cipher_suite] TLS cipher suite selector.; keylen: [size_t*] Pointer receiving the record protection key length.; ivlen: [size_t*] Pointer receiving the record protection IV length..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_keyschedule_verify_finished

Purpose: Verify a TLS 1.3 Finished verify_data value.

Details: Recomputes the expected Finished MAC and compares it to the supplied candidate using constant-time comparison. The candidate length must match the selected hash digest size.

qsc_tls_status qsc_tls_keyschedule_verify_finished(qsc_tls_hash_algorithm hash, const uint8_t* basekey, size_t basekeylen, const uint8_t* transcripthash, size_t transcripthashlen, const uint8_t* candidate, size_t candidatelen);

Parameters: hash: [qsc_tls_hash_algorithm] Hash algorithm selector.; basekey: [const uint8_t*] Pointer to the base traffic secret used to derive finished_key.; basekeylen: [size_t] Length, in bytes, of the base key.; transcripthash: [const uint8_t*] Pointer to the transcript hash.; transcripthashlen: [size_t] Length, in bytes, of the transcript hash.; candidate: [const uint8_t*] Pointer to the received verify_data value.; candidatelen: [size_t] Length, in bytes, of the candidate value..

Return: [qsc_tls_status] Returns qsc_tls_status_success.

Header: tlsrecord.h

qsc_tls_record_decode_plaintext

Purpose: Decode a plaintext TLS record.

Details: Applies the RFC 9846 record-layer wire bounds. TLSPlaintext fragments are limited to 2^14 bytes and protected TLSCiphertext fragments, identified by the outer application_data type, are limited to 2^14 + 256 bytes.

qsc_tls_status qsc_tls_record_decode_plaintext(const uint8_t* input, size_t inlen, qsc_tls_record_content_type* type, const uint8_t** payload, size_t* payloadlen);

Parameters: input: [const uint8_t*] The source record buffer.; inlen: [size_t] The source buffer length in bytes.; type: [enum] Receives the decoded outer record content type.; payload: [const uint8_t**] Receives a pointer to the decoded payload span.; payloadlen: [size_t*] Receives the decoded payload length in bytes..

Return: Returns a TLS status code.

qsc_tls_record_decrypt

Purpose: Decrypt a protected TLSCiphertext record.

Details: Accepts RFC 9846 record padding up to the TLSCiphertext wire limit, removes the inner content-type byte and zero padding, and rejects authenticated content greater than 2^14 bytes with qsc_tls_status_record_overflow.

qsc_tls_status qsc_tls_record_decrypt(qsc_tls_record_state* state, uint8_t* output, size_t outlen, size_t* written, qsc_tls_record_content_type* inner_type, const uint8_t* input, size_t inlen);

Parameters: state: [struct] The active read-side record protection state.; output: [uint8_t*] The destination plaintext buffer.; outlen: [size_t] The destination buffer length in bytes.; written: [size_t*] Receives the number of plaintext bytes written.; inner_type: [enum] Receives the decoded inner content type.; input: [const uint8_t*] The protected record buffer.; inlen: [size_t] The protected record length in bytes..

Return: Returns a TLS status code.

qsc_tls_record_encode_plaintext

Purpose: Encode a plaintext TLS record.

Details: Enforces the RFC 9846 Section 5.1 TLSPlaintext 2^14-byte fragment limit. Handshake records must be non-empty, Alert records must contain exactly one two-byte Alert message, and compatibility CCS records must contain 0x01.

qsc_tls_status qsc_tls_record_encode_plaintext(uint8_t* output, size_t outlen, size_t* written, qsc_tls_record_content_type type, const uint8_t* input, size_t inlen);

Parameters: output: [uint8_t*] The destination record buffer.; outlen: [size_t] The destination buffer length in bytes.; written: [size_t*] Receives the number of bytes written.; type: [enum] The outer record content type.; input: [const uint8_t*] The plaintext payload buffer.; inlen: [size_t] The plaintext payload length in bytes..

Return: Returns a TLS status code.

qsc_tls_record_encrypt

Purpose: Protect a TLSInnerPlaintext payload as a TLSCiphertext record.

Details: Enforces the per-key outbound AEAD usage limits required by RFC 9846 Section 5.5. AES-GCM counts every protected record conservatively as a full-size record.

qsc_tls_status qsc_tls_record_encrypt(qsc_tls_record_state* state, uint8_t* output, size_t outlen, size_t* written, qsc_tls_record_content_type inner_type, const uint8_t* input, size_t inlen);

Parameters: state: [struct] The active write-side record protection state.; output: [uint8_t*] The destination record buffer.; outlen: [size_t] The destination buffer length in bytes.; written: [size_t*] Receives the number of bytes written.; inner_type: [enum] The inner content type trailer.; input: [const uint8_t*] The plaintext payload buffer.; inlen: [size_t] The plaintext payload length in bytes..

Return: Returns a TLS status code.

qsc_tls_record_state_dispose

Purpose: Dispose of a TLS record protection state.

void qsc_tls_record_state_dispose(qsc_tls_record_state* state);

Parameters: state: [struct] The record state to clear..

Return: None (void).

qsc_tls_record_state_get_sequence

Purpose: Read the current sequence number. Returns 0 when state is NULL or uninitialized.

uint64_t qsc_tls_record_state_get_sequence(const qsc_tls_record_state* state);

Parameters: state: [const struct*] The record state..

Return: [uint64_t] Current 64-bit sequence counter.

qsc_tls_record_state_initialize

Purpose: Initialize a TLS record protection state.

void qsc_tls_record_state_initialize(qsc_tls_record_state* state, qsc_tls_cipher_suite suite, const uint8_t* key, size_t keylen, const uint8_t* iv, size_t ivlen);

Parameters: state: [struct] The record state to initialize.; suite: Negotiated TLS cipher-suite identifier used to initialize record protection.; key: [const uint8_t*] The traffic key buffer.; keylen: [size_t] The traffic key length in bytes.; iv: [const uint8_t*] The static traffic IV buffer.; ivlen: [size_t] The static traffic IV length in bytes..

Return: None (void).

qsc_tls_record_state_install_keys

Purpose: Install or replace traffic keys on an existing record state, resetting the sequence.

Details: Intended for epoch transitions (handshake to application keys) and for KeyUpdate. The existing key material is zeroized via qsc_memutils_secure_erase before the new key and IV are installed. Equivalent in effect to qsc_tls_record_state_dispose followed by qsc_tls_record_state_initialize but presented as a single atomic operation so callers cannot observe a partially-cleared state.

qsc_tls_status qsc_tls_record_state_install_keys(qsc_tls_record_state* state, qsc_tls_cipher_suite suite, const uint8_t* key, size_t keylen, const uint8_t* iv, size_t ivlen);

Parameters: state: [struct*] The record state to update.; suite: [enum] The cipher suite.; key: [const uint8_t*] The new traffic key.; keylen: [size_t] The new key length; must match the suite.; iv: [const uint8_t*] The new 12-byte static IV.; ivlen: [size_t] The IV length; must be 12..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_record_try_get_span_length

Purpose: Determine the full span length of a TLS record.

Details: Rejects wire fragment lengths above the RFC 9846 TLSCiphertext maximum with qsc_tls_status_record_overflow before buffering the complete record.

qsc_tls_status qsc_tls_record_try_get_span_length(const uint8_t* input, size_t inlen, size_t* recordlen, bool* complete);

Parameters: input: [const uint8_t*] The source buffer.; inlen: [size_t] The source buffer length in bytes.; recordlen: [size_t*] Receives the full record span length in bytes.; complete: [bool*] Receives true if the full record is present in the source buffer..

Return: Returns a TLS status code.

Header: tlsserver.h

qsc_tls_server_authorize_client_certificate

Purpose: Validate and authorize a presented mTLS client certificate chain.

Details: This helper performs the server-side mTLS chain validation step, prepares a bounded authorization information structure, and then evaluates the configured application authorization callback. CertificateVerify possession checking is performed separately by the TLS handshake state machine.

qsc_tls_status qsc_tls_server_authorize_client_certificate(qsc_tls_server_state* state, const qsc_tls_certificate_view* chain, size_t chainlength);

Parameters: state: [struct*] The server handshake state.; chain: [const struct*] Client certificate chain views in leaf-first order.; chainlength: [size_t] Number of certificate views in the chain..

Return: [qsc_tls_status] Returns qsc_tls_status_success when the certificate is valid and authorized.

qsc_tls_server_config_add_certificate_identity

Purpose: Add an SNI-selectable certificate identity to a TLS server configuration.

qsc_tls_status qsc_tls_server_config_add_certificate_identity(qsc_tls_server_config* config, const char* hostname, const qsc_tls_local_certificate_config* localcert);

Parameters: config: [struct*] The server configuration to update.; hostname: [const char*] The DNS name or wildcard pattern for this identity.; localcert: [const struct*] The local certificate configuration for the identity..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_server_config_set_certificate_interface

Purpose: Copy a client-certificate validation interface into a TLS server configuration.

Details: This setter is the TLS-side attachment point for mutual-TLS peer validation. X.509 helpers prepare the qsc_tls_certificate_interface, but do not mutate TLS server state directly.

qsc_tls_status qsc_tls_server_config_set_certificate_interface(qsc_tls_server_config* config, const qsc_tls_certificate_interface* iface, bool requestclientauth, bool requireclientauth);

Parameters: config: [struct*] The server configuration to update.; iface: [struct*] Optional certificate-validation interface. Required when client authentication is requested or required.; requestclientauth: [bool] Send CertificateRequest when true.; requireclientauth: [bool] Reject an empty client Certificate when true..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_server_config_set_client_authorization

Purpose: Configure the server-side mTLS application authorization callback.

Details: During the TLS handshake the callback is evaluated only after certificate-chain validation and CertificateVerify possession proof have both succeeded. If required is true, a missing callback or a callback rejection denies the peer. If required is false, validated certificate possession remains the authentication boundary.

qsc_tls_status qsc_tls_server_config_set_client_authorization(qsc_tls_server_config* config, qsc_tls_client_authorization_callback callback, void* state, bool required);

Parameters: config: [struct*] The server configuration to update.; callback: [function] Optional application authorization callback.; state: [void*] Caller-owned state passed to the callback.; required: [bool] Require callback acceptance when true..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_server_config_set_local_certificate

Purpose: Copy a local certificate chain and private signing key into a TLS server configuration.

Details: The certificate views are copied as non-owning DER pointers. The pointed-to DER buffers must remain valid for the lifetime of handshakes initialized from this configuration. The private key bytes are copied into the configuration and are used by the TLS CertificateVerify signing callback at handshake time.

qsc_tls_status qsc_tls_server_config_set_local_certificate(qsc_tls_server_config* config, const qsc_tls_certificate_view* chain, size_t chainlength, qsc_tls_signature_scheme verifyscheme, const uint8_t* privatekeydata, size_t privatekeylen);

Parameters: config: [struct*] The server configuration to update.; chain: [struct*] Certificate chain views in leaf-first order.; chainlength: [size_t] Number of valid chain entries.; verifyscheme: [enum] TLS CertificateVerify signature scheme.; privatekeydata: [const uint8_t*] Raw private key bytes for the signing scheme.; privatekeylen: [size_t] Length of the private key..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_server_config_set_sni_required

Purpose: Configure whether the server requires a recognized SNI hostname.

qsc_tls_status qsc_tls_server_config_set_sni_required(qsc_tls_server_config* config, bool required);

Parameters: config: [struct*] The server configuration to update.; required: [bool] Set to true to reject absent or unmatched SNI names..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_server_get_negotiated_cipher_suite

Purpose: Return the TLS cipher suite selected for the supplied server handshake state. If state is NULL, the implementation returns qsc_tls_cipher_suite_none.

qsc_tls_cipher_suite qsc_tls_server_get_negotiated_cipher_suite(const qsc_tls_server_state* state);

Parameters: state: Server handshake state to inspect; NULL returns qsc_tls_cipher_suite_none..

Return: qsc_tls_cipher_suite; the header does not specify additional failure semantics.

Header: tlssession.h

qsc_tls_session_ticket_decode

Purpose: Decode a TLS session ticket structure.

Details: Parses an RFC 9846 NewSessionTicket message body into a qsc_tls_session_ticket structure. Local-only fields are cleared and must be populated by the caller from the established connection context. The input buffer must contain a complete and correctly formatted encoded ticket. Partial or malformed input will result in a failure status. A structurally valid ticket whose opaque identity exceeds QSC_TLS_TICKET_MAX_SIZE is fully validated and returns qsc_tls_status_not_supported without retaining the oversized identity.

qsc_tls_status qsc_tls_session_ticket_decode(const uint8_t* input, size_t inplen, qsc_tls_session_ticket* ticket);

Parameters: input: [const uint8_t*] Pointer to the encoded session ticket buffer.; inplen: [size_t] Length, in bytes, of the encoded ticket buffer.; ticket: [qsc_tls_session_ticket*] Pointer to the structure that will receive the decoded ticket contents..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success, qsc_tls_status_not_supported when a valid ticket exceeds local retention capacity, or an error status for malformed input.

qsc_tls_session_ticket_dispose

Purpose: Dispose of a TLS session ticket structure.

Details: Clears and zeroizes all sensitive fields within a qsc_tls_session_ticket structure, including the resumption secret, ticket identity, nonce, and any associated metadata. This function should be called when the ticket is no longer required to prevent retention of sensitive material in memory. The structure remains valid for reuse after disposal but must be reinitialized before use.

void qsc_tls_session_ticket_dispose(qsc_tls_session_ticket* ticket);

Parameters: ticket: [qsc_tls_session_ticket*] Pointer to the session ticket structure to dispose..

Return: None (void).

qsc_tls_session_ticket_encode

Purpose: Encode a TLS session ticket structure.

Details: Serializes the RFC 9846 NewSessionTicket message body represented by a qsc_tls_session_ticket. Only wire fields are encoded; local resumption metadata and PSK bytes are intentionally not serialized. The caller is responsible for providing a buffer of

sufficient size. The function does not perform dynamic allocation and will fail if the output buffer is too small to contain the serialized ticket.

qsc_tls_status qsc_tls_session_ticket_encode(const qsc_tls_session_ticket* ticket, uint8_t* output, size_t outlen, size_t* written);

Parameters: ticket: [const qsc_tls_session_ticket*] Pointer to the session ticket structure to encode.; output: [uint8_t*] Pointer to the destination buffer that will receive the encoded ticket.; outlen: [size_t] Size, in bytes, of the destination buffer.; written: [size_t*] Pointer receiving the number of bytes written to output..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

Header: tlssigalgs.h

qsc_tls_signature_scheme_descriptor_get

Purpose: Get the registry descriptor for a TLS signature scheme.

const qsc_tls_signature_scheme_descriptor* qsc_tls_signature_scheme_descriptor_get(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: A pointer to the constant descriptor for the selected scheme, or NULL if the scheme is unknown to the registry.

qsc_tls_signature_scheme_hash

Purpose: Get the transcript hash algorithm associated with a TLS signature scheme.

Details: For signature schemes that bind a specific transcript hash or pre-hash mode, this function returns the corresponding TLS hash identifier. For schemes with no valid mapping, the return value may indicate an unset or null hash.

qsc_tls_hash_algorithm qsc_tls_signature_scheme_hash(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: The associated TLS hash algorithm identifier.

qsc_tls_signature_scheme_is_certificate_verify_capable

Purpose: Determine whether a TLS signature scheme may be used in CertificateVerify.

Details: This query reports whether the scheme is valid for use in the TLS CertificateVerify message under the local registry policy. A scheme may be recognized by the registry but still not be eligible for CertificateVerify.

bool qsc_tls_signature_scheme_is_certificate_verify_capable(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: Returns true if the scheme is CertificateVerify-capable.

qsc_tls_signature_scheme_is_mldsa

Purpose: Determine whether a TLS signature scheme belongs to the ML-DSA family.

bool qsc_tls_signature_scheme_is_mldsa(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: Returns true if the scheme is an ML-DSA variant.

qsc_tls_signature_scheme_is_pq

Purpose: Determine whether a TLS signature scheme is post-quantum.

bool qsc_tls_signature_scheme_is_pq(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: Returns true if the scheme is classified as post-quantum.

qsc_tls_signature_scheme_is_supported

Purpose: Determine whether a TLS signature scheme is supported by the registry.

bool qsc_tls_signature_scheme_is_supported(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: Returns true if the scheme is implemented and available for use.

qsc_tls_signature_scheme_matches_x509_algorithm

Purpose: Determine whether a TLS signature scheme matches an X.509 signature algorithm.

bool qsc_tls_signature_scheme_matches_x509_algorithm(qsc_tls_signature_scheme scheme, qsc_x509_signature_algorithm algorithm);

Parameters: scheme: [enum] The TLS signature scheme identifier.; algorithm: [enum] The X.509 signature algorithm identifier..

Return: Returns true if the TLS scheme and X.509 algorithm correspond.

qsc_tls_signature_scheme_name

Purpose: Get the human-readable name of a TLS signature scheme.

```
const char* qsc_tls_signature_scheme_name(qsc_tls_signature_scheme scheme);
```

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: A constant string naming the scheme, or NULL if the scheme is not known.

qsc_tls_signature_scheme_private_key_size

Purpose: Get the private key size associated with a TLS signature scheme.

Details: The returned value is the implementation-defined private key size in bytes for the selected scheme. For schemes that are unknown or unsupported, the function may return zero.

```
size_t qsc_tls_signature_scheme_private_key_size(qsc_tls_signature_scheme scheme);
```

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: The private key size in bytes.

qsc_tls_signature_scheme_public_key_size

Purpose: Get the public key size associated with a TLS signature scheme.

Details: The returned value is the implementation-defined public key size in bytes for the selected scheme. For schemes that are unknown or unsupported, the function may return zero.

```
size_t qsc_tls_signature_scheme_public_key_size(qsc_tls_signature_scheme scheme);
```

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: The public key size in bytes.

qsc_tls_signature_scheme_signature_size

Purpose: Get the maximum encoded signature size for a TLS signature scheme.

Details: This value represents the maximum encoded signature size expected by the TLS implementation for the specified scheme. It may be used to size working buffers or to validate received signature lengths.

size_t qsc_tls_signature_scheme_signature_size(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: The maximum signature size in bytes.

qsc_tls_signature_scheme_validate_signature_length

Purpose: Validate a received or generated signature length for a TLS signature scheme.

bool qsc_tls_signature_scheme_validate_signature_length(qsc_tls_signature_scheme scheme, size_t signaturelen);

Parameters: scheme: [enum] The TLS signature scheme identifier.; signaturelen: [size_t] The signature length in bytes to validate..

Return: Returns true if the length is valid for the selected scheme.

qsc_tls_signature_scheme_x509_algorithm

Purpose: Get the X.509 signature algorithm corresponding to a TLS signature scheme.

Details: This function maps a TLS signature-scheme identifier to the corresponding X.509 signature algorithm identifier used by the certificate subsystem.

qsc_x509_signature_algorithm
qsc_tls_signature_scheme_x509_algorithm(qsc_tls_signature_scheme scheme);

Parameters: scheme: [enum] The TLS signature scheme identifier..

Return: The mapped X.509 signature algorithm identifier.

Header: tlssignerdefault.h

qsc_tls_signer_default_sign

Purpose: Produce a TLS CertificateVerify signature.

Details: Matches the qsc_tls_certificate_sign_callback contract so this function can be installed directly as qsc_tls_local_certificate_config::signcallback.

bool qsc_tls_signer_default_sign(qsc_tls_signature_scheme scheme, const uint8_t* input, size_t inputlen, uint8_t* signature, size_t* signaturelen, void* state);

Parameters: scheme: [enum] The TLS signature scheme to use.; input: [const uint8_t*] The formatted TLS 1.3 CertificateVerify input bytes.; inputlen: [size_t] Length of input in bytes.; signature: [uint8_t*] Destination buffer for the signature.; signaturelen: [size_t*] On input, the available buffer size; on success, the encoded length.; state: [void*] Must point to a qsc_tls_signer_default_context..

Return: [bool] Returns true if the signature was produced successfully.

qsc_tls_signer_default_verify

Purpose: Verify a TLS CertificateVerify signature against the supplied public-key view.

Details: Matches the qsc_tls_certificate_verify_callback contract so this function can be installed directly as qsc_tls_certificate_interface::verifycertificateverify. The \a signer view is expected to expose the raw public-key bytes (not an X.509 DER blob); for SPKI-aware verification wrap this signer through tlscert_x509 which decodes the certificate and extracts the public key before dispatching here.

bool qsc_tls_signer_default_verify(qsc_tls_signature_scheme scheme, const uint8_t* input, size_t inputlen, const uint8_t* signature, size_t signaturelen, const qsc_tls_certificate_view* signer, void* state);

Parameters: scheme: [enum] The negotiated TLS signature scheme.; input: [const uint8_t*] The formatted TLS 1.3 CertificateVerify input bytes covered by the signature.; inputlen: [size_t] Length of input in bytes.; signature: [const uint8_t*] The encoded signature bytes (DER for ECDSA, raw for Ed25519 / ML-DSA).; signaturelen: [size_t] Length of the signature in bytes.; signer: [const struct*] View exposing the peer's raw public-key bytes.; state: [void*] Unused; pass NULL..

Return: [bool] Returns true if the signature verifies.

Header: tlssocket.h

qsc_tls_socket_client_connect_host_ex

Purpose: Connect to a remote host with an explicit session ticket and early-data preference.

qsc_tls_socket_status qsc_tls_socket_client_connect_host_ex(qsc_tls_socket_connection* connection, const qsc_tls_socket_context* context, const char* hostname, const char* service, const qsc_tls_session_ticket* ticket, bool enableearlydata);

Parameters: connection: [struct*] A pointer to an initialized connection structure.; context: [const struct*] A pointer to an initialized client context.; hostname: [const char*] The null-terminated DNS hostname used for connection and verification.; service: [const char*] The null-terminated service or port string.; ticket: [const struct*] A pointer to the session ticket to offer, or NULL for a full handshake.; enableearlydata: [bool] Reserved 0-RTT switch; true is rejected by the QSC TLS socket profile..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_client_connect_ipv4

Purpose: Connect to an IPv4 address and complete a TLS client handshake.

qsc_tls_socket_status qsc_tls_socket_client_connect_ipv4(qsc_tls_socket_connection* connection, const qsc_tls_socket_context* context, const qsc_ipinfo_ipv4_address* address, uint16_t port, const char* hostname);

Parameters: connection: [struct*] A pointer to an initialized connection structure.; context: [const struct*] A pointer to an initialized client context.; address: [const struct*] A pointer to the IPv4 address.; port: [uint16_t] The remote TCP port.; hostname: [const char*] The verification hostname, or NULL when policy permits..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_client_connect_ipv6

Purpose: Connect to an IPv6 address and complete a TLS client handshake.

qsc_tls_socket_status qsc_tls_socket_client_connect_ipv6(qsc_tls_socket_connection* connection, const qsc_tls_socket_context* context, const qsc_ipinfo_ipv6_address* address, uint16_t port, const char* hostname);

Parameters: connection: [struct*] A pointer to an initialized connection structure.; context: [const struct*] A pointer to an initialized client context.; address: [const struct*] A pointer to the IPv6 address.; port: [uint16_t] The remote TCP port.; hostname: [const char*] The verification hostname, or NULL when policy permits..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_connection_cancel

Purpose: Request cancellation of a TLS socket connection.

Details: Cancellation atomically records the request and shuts down the socket to unblock pending I/O. The thread that owns the connection remains responsible for final TLS and socket disposal. The connection must not be disposed concurrently with this cancellation request.

qsc_tls_socket_status qsc_tls_socket_connection_cancel(qsc_tls_socket_connection* connection);

Parameters: connection: [struct*] A pointer to the TLS socket connection..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_connection_clear_session_ticket

Purpose: Clear the most recent session ticket retained by a connection.

void qsc_tls_socket_connection_clear_session_ticket(qsc_tls_socket_connection* connection);

Parameters: connection: [struct*] A pointer to the TLS socket connection..

Return: None (void).

qsc_tls_socket_connection_dispose

Purpose: Dispose of a TLS socket connection and clear owned sensitive state.

void qsc_tls_socket_connection_dispose(qsc_tls_socket_connection* connection);

Parameters: connection: [struct*] A pointer to the connection structure to dispose..

Return: None (void).

qsc_tls_socket_connection_get_session_ticket

Purpose: Retrieve the most recent session ticket associated with a connection.

qsc_tls_socket_status qsc_tls_socket_connection_get_session_ticket(const qsc_tls_socket_connection* connection, qsc_tls_session_ticket* ticketout);

Parameters: connection: [const struct*] A pointer to the TLS socket connection.; ticketout: [struct*] A pointer to the ticket structure to receive the ticket copy..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_connection_initialize

Purpose: Initialize a TLS socket connection structure.

void qsc_tls_socket_connection_initialize(qsc_tls_socket_connection* connection);

Parameters: connection: [struct*] A pointer to the connection structure to initialize..

Return: None (void).

qsc_tls_socket_connection_set_log_callback

Purpose: Set the connection-level structured logging callback.

qsc_tls_socket_status
qsc_tls_socket_connection_set_log_callback(qsc_tls_socket_connection* connection, qsc_tls_socket_log_callback callback, void* state);

Parameters: connection: [struct*] A pointer to the TLS socket connection.; callback: [function] The logging callback pointer, or NULL to disable connection logging.; state: [void*] The user-defined callback state pointer..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_connection_set_socket_options

Purpose: Apply socket options to an initialized or connected TLS socket connection.

qsc_tls_socket_status
qsc_tls_socket_connection_set_socket_options(qsc_tls_socket_connection* connection, const qsc_tls_socket_options* options);

Parameters: connection: [struct*] A pointer to the TLS socket connection.; options: [const struct*] A pointer to the socket options to apply..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_add_server_identity_files

Purpose: Load an additional SNI-selectable server identity from certificate-chain and private-key files.

qsc_tls_socket_status
qsc_tls_socket_context_add_server_identity_files(qsc_tls_socket_context* context, const char* hostname, const char* certificatechainpath, const char* privatekeypath, qsc_tls_signature_scheme verifyscheme);

Parameters: context: [struct*] A pointer to the initialized context.; hostname: [const char*] The DNS name or wildcard pattern used for SNI selection.; certificatechainpath: [const char*] The certificate-chain file path.; privatekeypath: [const char*] The private-key file path.; verifyscheme: [enum] The TLS CertificateVerify signature scheme for the identity..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_clear_alpn_protocols

Purpose: Clear the ordered ALPN protocol list from the context.

qsc_tls_socket_status
qsc_tls_socket_context_clear_alpn_protocols(qsc_tls_socket_context* context);

Parameters: context: [struct*] A pointer to the initialized context..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_clear_session_ticket

Purpose: Clear the client session ticket stored in the context.

void qsc_tls_socket_context_clear_session_ticket(qsc_tls_socket_context* context);

Parameters: context: [struct*] A pointer to the initialized context..

Return: None (void).

qsc_tls_socket_context_dispose

Purpose: Dispose of a TLS socket context and clear owned sensitive state.

void qsc_tls_socket_context_dispose(qsc_tls_socket_context* context);

Parameters: context: [struct*] A pointer to the context structure to dispose..

Return: None (void).

qsc_tls_socket_context_load_client_identity_files

Purpose: Load a client certificate chain and private key into the context for mutual TLS.

qsc_tls_socket_status
qsc_tls_socket_context_load_client_identity_files(qsc_tls_socket_context* context, const char* certificatechainpath, const char* privatekeypath, qsc_tls_signature_scheme verifyscheme);

Parameters: context: [struct*] A pointer to the initialized context.; certificatechainpath: [const char*] The null-terminated file path to the client certificate chain.; privatekeypath: [const char*] The null-terminated file path to the client private key.; verifyscheme: [enum] The TLS signature scheme used by the client CertificateVerify operation..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_load_crl_file

Purpose: Load a certificate revocation list file into the context trust store.

qsc_tls_socket_status qsc_tls_socket_context_load_crl_file(qsc_tls_socket_context* context, const char* path);

Parameters: context: [struct*] A pointer to the initialized context.; path: [const char*] The null-terminated file path to the CRL file..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_load_server_identity_files

Purpose: Load a server certificate chain and private key into the context.

qsc_tls_socket_status
qsc_tls_socket_context_load_server_identity_files(qsc_tls_socket_context* context, const char* certificatechainpath, const char* privatekeypath, qsc_tls_signature_scheme verifyscheme);

Parameters: context: [struct*] A pointer to the initialized context.; certificatechainpath: [const char*] The null-terminated file path to the server certificate chain.; privatekeypath: [const char*] The null-terminated file path to the server private key.; verifyscheme: [enum] The TLS signature scheme used by the server CertificateVerify operation..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_load_trust_anchor_bundle_file

Purpose: Load a trust anchor bundle file into the context trust store.

qsc_tls_socket_status
qsc_tls_socket_context_load_trust_anchor_bundle_file(qsc_tls_socket_context* context, const char* path, bool selfsigned);

Parameters: context: [struct*] A pointer to the initialized context.; path: [const char*] The null-terminated file path to the trust anchor bundle.; selfsigned: [bool] Indicates whether the bundle contains self-signed trust anchors..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_load_trust_anchor_file

Purpose: Load a trust anchor certificate file into the context trust store.

qsc_tls_socket_status
qsc_tls_socket_context_load_trust_anchor_file(qsc_tls_socket_context* context, const char* path, bool selfsigned);

Parameters: context: [struct*] A pointer to the initialized context.; path: [const char*] The null-terminated file path to the trust anchor certificate.; selfsigned: [bool] Indicates whether the trust anchor is self-signed..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_alpn_protocols

Purpose: Set the ordered ALPN protocol list for the context.

qsc_tls_socket_status qsc_tls_socket_context_set_alpn_protocols(qsc_tls_socket_context* context, const char* const* protocols, size_t protocolcount, bool required);

Parameters: context: [struct*] A pointer to the initialized context.; protocols: [const char**] A pointer to an ordered list of null-terminated protocol strings.; protocolcount: [size_t] The number of protocol strings in the list.; required: [bool] Require a mutually supported ALPN protocol when true..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_cipher_suites

Purpose: Set the ordered cipher suite preference list for the context.

qsc_tls_socket_status qsc_tls_socket_context_set_cipher_suites(qsc_tls_socket_context* context, const qsc_tls_cipher_suite* suites, size_t suitecount);

Parameters: context: [struct*] A pointer to the initialized context.; suites: [const enum*] A pointer to the cipher suite preference array.; suitecount: [size_t] The number of cipher suites in the array..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_client_auth

Purpose: Configure server-side client certificate authentication policy.

qsc_tls_socket_status qsc_tls_socket_context_set_client_auth(qsc_tls_socket_context* context, bool requestclientauth, bool requireclientauth);

Parameters: context: [struct*] A pointer to the initialized context.; requestclientauth: [bool] Request a client certificate from peers when true.; requireclientauth: [bool] Require a verified client certificate when true..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_client_authorization

Purpose: Configure the server-side mTLS application authorization callback.

Details: The callback is invoked after client-certificate chain validation and CertificateVerify possession proof succeed. If required is true, the handshake policy rejects the client when no callback is configured or when the callback returns false.

qsc_tls_socket_status
qsc_tls_socket_context_set_client_authorization(qsc_tls_socket_context* context, qsc_tls_client_authorization_callback callback, void* state, bool required);

Parameters: context: [struct*] The TLS socket context to update.; callback: [function] Optional application authorization callback.; state: [void*] Caller-owned state passed to the callback.; required: [bool] Require callback acceptance when true..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_development_policy

Purpose: Configure the context with the development TLS policy.

Details: The development policy is intended for local testing. It may permit unverified peers depending on the lower TLS and X.509 configuration. It must not be used for production deployments.

qsc_tls_socket_status
qsc_tls_socket_context_set_development_policy(qsc_tls_socket_context* context);

Parameters: context: [struct*] A pointer to the initialized context..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_experimental_pqc_policy

Purpose: Configure the context with the experimental ML-KEM and ML-DSA policy.

Details: This policy enables experimental post-quantum key-establishment and signature preferences. It requires a peer that supports the same experimental TLS named groups and signature schemes.

qsc_tls_socket_status
qsc_tls_socket_context_set_experimental_pqc_policy(qsc_tls_socket_context* context);

Parameters: context: [struct*] A pointer to the initialized context..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_log_callback

Purpose: Set the context-level structured logging callback.

qsc_tls_socket_status qsc_tls_socket_context_set_log_callback(qsc_tls_socket_context* context, qsc_tls_socket_log_callback callback, void* state);

Parameters: context: [struct*] A pointer to the initialized context.; callback: [function] The logging callback pointer, or NULL to disable context logging.; state: [void*] The user-defined callback state pointer..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_mlkem_hybrid_policy

Purpose: Configure the context with an explicit ML-KEM hybrid interoperability policy.

Details: The ML-KEM hybrid policy enables the hybrid ML-KEM named-group preference while retaining conventional TLS 1.3 cipher suites and classical certificate signature schemes for peers that support hybrid key establishment.

qsc_tls_socket_status
qsc_tls_socket_context_set_mlkem_hybrid_policy(qsc_tls_socket_context* context);

Parameters: context: [struct*] A pointer to the initialized context..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_named_groups

Purpose: Set the ordered named group preference list for the context.

qsc_tls_socket_status qsc_tls_socket_context_set_named_groups(qsc_tls_socket_context* context, const qsc_tls_named_group* groups, size_t groupcount);

Parameters: context: [struct*] A pointer to the initialized context.; groups: [const enum*] A pointer to the named group preference array.; groupcount: [size_t] The number of named groups in the array..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_psk_lookup_callback

Purpose: Set the server-side PSK session-ticket lookup callback.

Details: The callback is invoked when a client offers a TLS 1.3 PSK identity. It must recover the complete server-side qsc_tls_session_ticket metadata associated with that opaque ticket identity. Passing NULL disables PSK resumption lookup while leaving NewSessionTicket issuance policy unchanged.

qsc_tls_socket_status
qsc_tls_socket_context_set_psk_lookup_callback(qsc_tls_socket_context* context, qsc_tls_psk_lookup_callback callback, void* state);

Parameters: context: [struct*] A pointer to the initialized context.; callback: [function] The ticket lookup callback, or NULL to disable server-side resumption lookup.; state: [void*] Caller-owned state forwarded to callback..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_session_ticket

Purpose: Set the client session ticket used for resumption attempts.

qsc_tls_socket_status qsc_tls_socket_context_set_session_ticket(qsc_tls_socket_context* context, const qsc_tls_session_ticket* ticket);

Parameters: context: [struct*] A pointer to the initialized context.; ticket: [const struct*] A pointer to the session ticket to copy into the context..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_session_ticket_policy

Purpose: Set the context-level session ticket policy.

qsc_tls_socket_status
qsc_tls_socket_context_set_session_ticket_policy(qsc_tls_socket_context* context, const qsc_tls_socket_ticket_policy* policy);

Parameters: context: [struct*] A pointer to the initialized context.; policy: [const struct*] A pointer to the session ticket policy to copy into the context..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_signature_schemes

Purpose: Set the ordered signature scheme preference list for the context.

qsc_tls_socket_status
qsc_tls_socket_context_set_signature_schemes(qsc_tls_socket_context* context, const qsc_tls_signature_scheme* schemes, size_t schemecount);

Parameters: context: [struct*] A pointer to the initialized context.; schemes: [const enum*] A pointer to the signature scheme preference array.; schemecount: [size_t] The number of signature schemes in the array..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_sni_required

Purpose: Configure whether server handshakes require a recognized SNI hostname.

qsc_tls_socket_status qsc_tls_socket_context_set_sni_required(qsc_tls_socket_context* context, bool required);

Parameters: context: [struct*] A pointer to the initialized context.; required: [bool] Set to true to reject absent or unmatched SNI names..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_socket_options

Purpose: Set the default socket options for the context.

qsc_tls_socket_status qsc_tls_socket_context_set_socket_options(qsc_tls_socket_context* context, const qsc_tls_socket_options* options);

Parameters: context: [struct*] A pointer to the initialized context.; options: [const struct*] A pointer to the socket options to copy into the context..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_context_set_strict_policy

Purpose: Configure the context with the strict TLS policy.

qsc_tls_socket_status qsc_tls_socket_context_set_strict_policy(qsc_tls_socket_context* context);

Parameters: context: [struct*] A pointer to the initialized context..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_get_selected_alpn

Purpose: Retrieve the selected ALPN protocol for a TLS socket connection.

qsc_tls_socket_status qsc_tls_socket_get_selected_alpn(const qsc_tls_socket_connection* connection, char* protocol, size_t protocolcap, size_t* protocollen);

Parameters: connection: [const struct*] A pointer to the TLS socket connection.; protocol: [char*] A pointer to the destination string buffer.; protocolcap: [size_t] Size, in bytes, of the destination string buffer.; protocollen: [size_t*] Receives the selected protocol length excluding the null terminator..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_is_connected

Purpose: Test whether a TLS socket connection owns a connected socket.

bool qsc_tls_socket_is_connected(const qsc_tls_socket_connection* connection);

Parameters: connection: [const struct*] A pointer to the TLS socket connection..

Return: [bool] Returns true when the connection is socket-connected.

qsc_tls_socket_is_handshake_complete

Purpose: Test whether the TLS handshake has completed.

bool qsc_tls_socket_is_handshake_complete(const qsc_tls_socket_connection* connection);

Parameters: connection: [const struct*] A pointer to the TLS socket connection..

Return: [bool] Returns true when the TLS handshake has completed.

qsc_tls_socket_key_update

Purpose: Request a TLS 1.3 KeyUpdate operation on a completed connection.

qsc_tls_socket_status qsc_tls_socket_key_update(qsc_tls_socket_connection* connection, bool requestpeerupdate);

Parameters: connection: [struct*] A pointer to the established TLS socket connection.; requestpeerupdate: [bool] Request that the peer also update its sending keys when true..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_listener_bind

Purpose: Bind a TLS listener to a local address and port.

qsc_tls_socket_status qsc_tls_socket_listener_bind(qsc_tls_socket_listener* listener, const qsc_tls_socket_context* context, const char* address, uint16_t port, qsc_socket_address_families family);

Parameters: listener: [struct*] A pointer to the initialized listener.; context: [const struct*] A pointer to the initialized server context.; address: [const char*] The null-terminated local address string.; port: [uint16_t] The local TCP port.; family: [enum] The socket address family..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_listener_close

Purpose: Close a TLS socket listener.

void qsc_tls_socket_listener_close(qsc_tls_socket_listener* listener);

Parameters: listener: [struct*] A pointer to the listener to close..

Return: None (void).

qsc_tls_socket_listener_initialize

Purpose: Initialize a TLS socket listener structure.

void qsc_tls_socket_listener_initialize(qsc_tls_socket_listener* listener);

Parameters: listener: [struct*] A pointer to the listener structure to initialize..

Return: None (void).

qsc_tls_socket_listener_set_options

Purpose: Set basic listener socket options.

qsc_tls_socket_status qsc_tls_socket_listener_set_options(qsc_tls_socket_listener* listener, bool reuseaddress, bool nodelay, uint32_t recvtimeoutms, uint32_t sendtimeoutms);

Parameters: listener: [struct*] A pointer to the initialized listener.; reuseaddress: [bool] Enable address reuse when true.; nodelay: [bool] Enable TCP no-delay when true.; recvtimeoutms: [uint32_t] The receive timeout in milliseconds.; sendtimeoutms: [uint32_t] The send timeout in milliseconds..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_listener_set_socket_options

Purpose: Set the full listener socket option structure.

qsc_tls_socket_status qsc_tls_socket_listener_set_socket_options(qsc_tls_socket_listener* listener, const qsc_tls_socket_options* options);

Parameters: listener: [struct*] A pointer to the initialized listener.; options: [const struct*] A pointer to the socket options to copy into the listener..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_negotiated_cipher_suite

Purpose: Get the negotiated TLS cipher suite.

qsc_tls_cipher_suite qsc_tls_socket_negotiated_cipher_suite(const qsc_tls_socket_connection* connection);

Parameters: connection: [const struct*] A pointer to the TLS socket connection..

Return: [qsc_tls_cipher_suite] Returns the negotiated cipher suite, or the lower-layer default value when unavailable.

qsc_tls_socket_negotiated_group

Purpose: Get the negotiated TLS named group.

qsc_tls_named_group qsc_tls_socket_negotiated_group(const qsc_tls_socket_connection* connection);

Parameters: connection: [const struct*] A pointer to the TLS socket connection..

Return: [qsc_tls_named_group] Returns the negotiated named group, or the lower-layer default value when unavailable.

qsc_tls_socket_negotiated_signature_scheme

Purpose: Get the negotiated TLS signature scheme.

qsc_tls_signature_scheme qsc_tls_socket_negotiated_signature_scheme(const qsc_tls_socket_connection* connection);

Parameters: connection: [const struct*] A pointer to the TLS socket connection..

Return: [qsc_tls_signature_scheme] Returns the negotiated signature scheme, or the lower-layer default value when unavailable.

qsc_tls_socket_options_initialize_default

Purpose: Initialize socket options to the wrapper default values.

void qsc_tls_socket_options_initialize_default(qsc_tls_socket_options* options);

Parameters: options: [struct*] A pointer to the socket options structure to initialize..

Return: None (void).

qsc_tls_socket_receive_frame

Purpose: Receive a length-prefixed framed application message.

Details: The frame format is a four-byte big-endian unsigned payload length followed by the payload bytes. If the encoded frame length exceeds the output capacity, the call returns an error. Zero-length frames are accepted and set read to zero.

qsc_tls_socket_status qsc_tls_socket_receive_frame(qsc_tls_socket_connection* connection, uint8_t* output, size_t outlen, size_t* read);

Parameters: connection: [struct*] A pointer to the established TLS socket connection.; output: [uint8_t*] A pointer to the application payload output buffer. This parameter may be NULL only when outlen is zero.; outlen: [size_t] The output buffer length in bytes.; read: [size_t*] Receives the number of payload bytes written to output..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_result_clear

Purpose: Clear a TLS socket structured result.

void qsc_tls_socket_result_clear(qsc_tls_socket_result* result);

Parameters: result: [struct*] A pointer to the result structure to clear..

Return: None (void).

qsc_tls_socket_send_frame

Purpose: Send a length-prefixed framed application message.

Details: The frame format is a four-byte big-endian unsigned payload length followed by the payload bytes. The payload length must not exceed QSC_TLS_SOCKET_FRAME_SIZE_MAX. Zero-length frames are permitted and encode only the four-byte length header.

qsc_tls_socket_status qsc_tls_socket_send_frame(qsc_tls_socket_connection* connection, const uint8_t* input, size_t inlen);

Parameters: connection: [struct*] A pointer to the established TLS socket connection.; input: [const uint8_t*] A pointer to the application payload. This parameter may be NULL only when inlen is zero.; inlen: [size_t] The payload length in bytes..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_server_configure

Purpose: Configure a TLS socket server listener and context.

qsc_tls_socket_status qsc_tls_socket_server_configure(qsc_tls_socket_server* server, const qsc_tls_socket_context* context, const char* address, uint16_t port, qsc_socket_address_families family);

Parameters: server: [struct*] A pointer to the initialized server.; context: [const struct*] A pointer to the initialized server context.; address: [const char*] The null-terminated local address string.; port: [uint16_t] The local TCP port.; family: [enum] The socket address family..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_server_dispose

Purpose: Dispose of a TLS socket server and clear owned connection state.

Details: Disposal requests shutdown, waits for the accept loop and connection owners to become quiescent, reaps remaining worker handles, destroys the pool mutex, and clears

the server. It must be called by an external owner and not from a server callback executing on a worker or blocking server thread.

void qsc_tls_socket_server_dispose(qsc_tls_socket_server* server);

Parameters: server: [struct*] A pointer to the server to dispose..

Return: None (void).

qsc_tls_socket_server_initialize

Purpose: Initialize a TLS socket server structure.

void qsc_tls_socket_server_initialize(qsc_tls_socket_server* server);

Parameters: server: [struct*] A pointer to the server structure to initialize..

Return: None (void).

qsc_tls_socket_server_send_session_ticket

Purpose: Send a TLS server session ticket over a completed server connection.

qsc_tls_socket_status
qsc_tls_socket_server_send_session_ticket(qsc_tls_socket_connection* connection, uint32_t lifetime_seconds, qsc_tls_session_ticket* ticketout);

Parameters: connection: [struct*] A pointer to the established server-side TLS socket connection.; lifetime_seconds: [uint32_t] The ticket lifetime hint in seconds.; ticketout: [struct*] Receives a copy of the emitted ticket, or NULL..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_server_set_callbacks

Purpose: Set the application callbacks for a TLS socket server.

qsc_tls_socket_status qsc_tls_socket_server_set_callbacks(qsc_tls_socket_server* server,
qsc_tls_socket_server_connect_callback onconnect,
qsc_tls_socket_server_receive_callback onreceive,
qsc_tls_socket_server_disconnect_callback ondisconnect,
qsc_tls_socket_server_error_callback onerror, void* state);

Parameters: server: [struct*] A pointer to the initialized server.; onconnect: [function] The callback invoked after a successful TLS handshake, or NULL.; onreceive: [function] The callback invoked when application data is received, or NULL.; ondisconnect: [function]

The callback invoked when a connection is closed, or NULL.; onerror: [function] The callback invoked when an error occurs, or NULL.; state: [void*] The user-defined callback state pointer..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_server_set_log_callback

Purpose: Set the server-level structured logging callback.

qsc_tls_socket_status qsc_tls_socket_server_set_log_callback(qsc_tls_socket_server* server, qsc_tls_socket_log_callback callback, void* state);

Parameters: server: [struct*] A pointer to the initialized server.; callback: [function] The logging callback pointer, or NULL to disable server logging.; state: [void*] The user-defined callback state pointer..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_server_set_max_clients

Purpose: Set the maximum number of concurrent client connections accepted by the server.

qsc_tls_socket_status qsc_tls_socket_server_set_max_clients(qsc_tls_socket_server* server, size_t maxclients);

Parameters: server: [struct*] A pointer to the initialized server.; maxclients: [size_t] The maximum number of clients. The value must not exceed QSC_TLS_SOCKET_SERVER_CONNECTIONS_MAX..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on success.

qsc_tls_socket_server_start_concurrent

Purpose: Start the server in bounded concurrent mode.

Details: Concurrent mode accepts inbound TLS connections into a fixed pool of connection slots and dispatches one worker per accepted connection, up to the configured maximum client count.

qsc_tls_socket_status qsc_tls_socket_server_start_concurrent(qsc_tls_socket_server* server);

Parameters: server: [struct*] A pointer to the configured server..

Return: [qsc_tls_socket_status] Returns qsc_tls_socket_status_success on normal completion.

qsc_tls_socket_server_stop

Purpose: Stop a running TLS socket server.

Details: The stop operation closes the listener, atomically requests cancellation of active connections, shuts down their sockets to unblock pending I/O, and joins worker threads other than the calling worker itself. Connection disposal remains owned by the worker or blocking accept loop. The running flag is cleared by the server start routine when its accept loop has actually exited. This function may be called from a server callback. qsc_tls_socket_server_dispose must instead be called by an external owner after callback execution has returned; disposing the server from one of its own callbacks is not supported.

void qsc_tls_socket_server_stop(qsc_tls_socket_server* server);

Parameters: server: [struct*] A pointer to the running server..

Return: None (void).

qsc_tls_socket_session_ticket_is_valid

Purpose: Test whether a session ticket is valid and unexpired for resumption.

bool qsc_tls_socket_session_ticket_is_valid(const qsc_tls_session_ticket* ticket);

Parameters: ticket: [const struct*] A pointer to the session ticket to validate..

Return: [bool] Returns true when the ticket metadata, KDF binding, and local lifetime are valid.

qsc_tls_socket_status_string

Purpose: Return a constant string for a TLS socket wrapper status value.

const char* qsc_tls_socket_status_string(qsc_tls_socket_status status);

Parameters: status: [enum] The TLS socket wrapper status value..

Return: [const char*] Returns a constant null-terminated status string.

qsc_tls_socket_ticket_policy_initialize_default

Purpose: Initialize a session ticket policy to the wrapper default values.

void qsc_tls_socket_ticket_policy_initialize_default(qsc_tls_socket_ticket_policy* policy);

Parameters: policy: [struct*] A pointer to the ticket policy structure to initialize..

Return: None (void).

Header: tlstranscript.h

qsc_tls_transcript_digest_size

Purpose: Get the digest size in bytes for a TLS transcript hash algorithm.

size_t qsc_tls_transcript_digest_size(qsc_tls_hash_algorithm hash);

Parameters: hash: [enum] The transcript hash algorithm..

Return: [size_t] Returns the digest size in bytes, or zero if the algorithm is unsupported.

qsc_tls_transcript_dispose

Purpose: Dispose of a transcript hash state.

Details: Securely erases the active transcript state and any retained intermediate hash context.

void qsc_tls_transcript_dispose(qsc_tls_transcript_state* state);

Parameters: state: [struct*] The transcript state to clear..

Return: None (void).

qsc_tls_transcript_initialize

Purpose: Initialize a transcript hash state.

qsc_tls_status qsc_tls_transcript_initialize(qsc_tls_transcript_state* state,
qsc_tls_hash_algorithm hash);

Parameters: state: [struct*] The transcript state to initialize.; hash: [enum] The hash algorithm to activate..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_transcript_replace_with_message_hash

Purpose: Replace the transcript with the synthetic message_hash form used after HelloRetryRequest.

Details: Replaces the current transcript contents with the synthetic handshake message defined by RFC 9846 Section 4.1, consisting of the message_hash handshake type and the digest of the previous ClientHello.

qsc_tls_status qsc_tls_transcript_replace_with_message_hash(qsc_tls_transcript_state* state);

Parameters: state: [struct*] The active transcript state..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_transcript_snapshot

Purpose: Snapshot the current transcript digest without disturbing the ongoing hash state.

Details: Clones the underlying hash context, finalizes the clone into the caller-supplied output buffer, and leaves the live transcript state unchanged.

qsc_tls_status qsc_tls_transcript_snapshot(const qsc_tls_transcript_state* state, uint8_t* output, size_t outlen, size_t* written);

Parameters: state: [const struct*] The active transcript state.; output: [uint8_t*] The destination buffer for the transcript digest.; outlen: [size_t] The destination buffer length in bytes.; written: [size_t*] Receives the number of digest bytes written..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

qsc_tls_transcript_update

Purpose: Append serialized handshake bytes to the transcript hash.

qsc_tls_status qsc_tls_transcript_update(qsc_tls_transcript_state* state, const uint8_t* input, size_t inplen);

Parameters: state: [struct*] The active transcript state.; input: [const uint8_t*] The serialized handshake bytes to append.; inplen: [size_t] The number of bytes to append..

Return: [qsc_tls_status] Returns qsc_tls_status_success on success.

12: X.509 PKI Toolkit

The QSC library includes a comprehensive X.509 Public Key Infrastructure toolkit. The toolkit provides functions for creating, encoding, decoding, verifying, and managing

X.509 certificates and related structures including Certificate Signing Requests (CSRs), Certificate Revocation Lists (CRLs), OCSP responses, PKCS#12 keystores, and PEM-encoded key and certificate files. All major operations support post-quantum signature algorithms (Dilithium, Falcon, SPHINCS+) in addition to classical ECDSA.

12.1 X.509 Wrapper Interface

Header:

x509wrap.h

Description:

The X.509 wrapper interface provides high-level profile, trust-store, server-identity, peer-verification, import/export, CSR, certificate issuance, and TLS bridge helpers. It is the preferred entry point for SDK consumers. Lower-level certificate, CSR, CRL, SPKI, PEM, OCSP, PKCS#12, extension, name, key, and verification headers remain available for advanced or subsystem use.

Selected API:

void qsc_x509w_profile_initialize(qsc_x509w_profile* profile)

void qsc_x509w_profile_set_tls_server_defaults(qsc_x509w_profile* profile)

void qsc_x509w_trust_store_initialize(qsc_x509w_trust_store* store)

qsc_x509w_status qsc_x509w_certificate_load_file(const char* path, qsc_x509_certificate* certificate)

qsc_x509w_status qsc_x509w_trust_store_add_anchor_file(qsc_x509w_trust_store* store, const char* path, bool selfsigned)

qsc_x509w_status qsc_x509w_verify_peer_certificates(const qsc_x509_certificate* certificates, size_t certificatecount, const qsc_x509w_trust_store* store, const qsc_x509w_profile* profile, qsc_x509w_result* result)

qsc_x509w_status qsc_x509w_csr_create(qsc_x509_csr* csr, const qsc_x509_name* subject, const qsc_x509_subject_public_key_info* spki, const qsc_x509_algorithm_identifier* signaturealgorithm)

qsc_x509w_status qsc_x509w_certificate_issue_from_csr(const qsc_x509_csr* csr, const qsc_x509_certificate* issuer, const qsc_x509_algorithm_identifier* signaturealgorithm,

const uint8_t* serialnumber, size_t serialnumberlen, const qsc_x509_validity* validity, uint32_t profile, uint32_t policyflags, qsc_x509_certificate_sign_callback signcallback, void* context, uint8_t* output, size_t* outputlen)

qsc_x509w_status qsc_x509w_tls_bridge_configure(qsc_x509w_tls_bridge* bridge, const qsc_x509w_trust_store* store, const qsc_x509w_profile* profile)

12.2 X.509 Certificates

Header:

x509cert.h, x509certwrite.h, x509verify.h, x509spki.h, x509pem.h

Description:

Functions for parsing, verifying, and writing DER and PEM encoded X.509 v3 certificates. Certificate construction is performed through the qsc_x509_certificate_builder_* API family, including subject, issuer, validity, public-key, signature-algorithm, extension, profile, CSR-extension, and DER/PEM output support.

12.3 Certificate Signing Requests

Header:

x509csr.h

Description:

Functions for generating, encoding, decoding, signing, and verifying PKCS#10 Certificate Signing Requests using the qsc_x509_csr_* API family. The implementation includes subject and SPKI assignment, extension requests, SAN DNS/IP helpers, ML-DSA and ML-KEM SPKI helpers, DER/PEM encoding, and CSR signature verification.

12.4 Certificate Revocation Lists

Header:

x509crl.h, x509crlwrite.h

Description:

Functions for parsing, creating, and signing X.509 Certificate Revocation Lists (CRLs). Supports revocation entry management, CRL extension handling, and OCSP stapling.

12.4 OCSP

Header:

x509ocsp.h

Description:

Functions for parsing OCSP responses, validating a certificate against an issuer through a caller supplied fetch callback, and verifying OCSP responder certificates.

12.5 PKCS#12 Keystores

Header:

x509pkcs12.h

Description:

Functions for initializing and parsing PKCS#12 bundles and decrypting encrypted private-key information from password-protected containers.

12.6 PEM Encoding

Header:

x509pem.h

Description:

Functions for encoding and decoding PEM-formatted certificates, keys, CSRs, and CRLs, providing interoperability with standard TLS infrastructure.

12.1 X.509 and ASN.1 Implementor API Reference

The certificate toolkit includes high-level X.509 wrappers, parsing, verification, CSR/CRL/OCSP/PKCS#12, DER encoding and trust-store operations. Contracts are taken from each module header and are not an independent security approval.

Header: x509aia.h

qsc_x509_aia_get_ocsp_url

Purpose: Extract the OCSP responder URI from an Authority Information Access extension.

Details: Parses the DER-encoded Authority Information Access extension value and locates an id-ad-ocsp access descriptor. When present, the URI is copied to the caller-supplied buffer and NUL-terminated. This function does not allocate memory. The caller must provide an output buffer and set \a urllen to the capacity of that buffer on entry. On success, \a urllen receives the URI byte length excluding the terminating NUL.

```
bool qsc_x509_aia_get_ocsp_url(const uint8_t* ext, size_t extlen, char* url, size_t* urllen);
```

Parameters: ext: [const] Pointer to the DER-encoded Authority Information Access extension value.; extlen: The extension length in bytes.; url: Output buffer receiving the URI string.; urllen: [in,out] On input, the output buffer capacity; on output, the URI length excluding the terminating NUL..

Return: Returns true on success; otherwise returns false.

Header: x509cert.h

qsc_x509_certificate_clear

Purpose: Clears a decoded certificate structure.

```
void qsc_x509_certificate_clear(qsc_x509_certificate* certificate);
```

Parameters: certificate: [qsc_x509_certificate*] The certificate structure..

Return: None (void).

qsc_x509_certificate_decode_der

Purpose: Decodes a DER encoded X.509 certificate.

qsc_asn1_status qsc_x509_certificate_decode_der(const uint8_t* der, size_t derlen, qsc_x509_certificate* certificate);

Parameters: der: [const uint8_t*] The DER encoded certificate.; derlen: [size_t] The length of the DER certificate in octets.; certificate: [qsc_x509_certificate*] Receives the decoded certificate..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_x509_certificate_find_extension

Purpose: Finds the first raw extension entry of a given type.

const qsc_x509_extension* qsc_x509_certificate_find_extension(const qsc_x509_certificate* certificate, qsc_x509_extension_type type);

Parameters: certificate: [const qsc_x509_certificate*] The decoded certificate.; type: [qsc_x509_extension_type] The extension type to search for..

Return: [const qsc_x509_extension*] Returns a pointer to the first matching extension, or NULL if the requested extension is not present.

qsc_x509_certificate_free

Purpose: Releases certificate-owned DER storage and clears the structure.

void qsc_x509_certificate_free(qsc_x509_certificate* certificate);

Parameters: certificate: [qsc_x509_certificate*] The certificate structure..

Return: None (void).

Header: x509certwrite.h

qsc_x509_cert_issuance_apply_csr_extensions

Purpose: Apply CSR-requested extensions to a certificate builder.

Details: Filters and copies CSR extensions into the certificate builder according to the supplied issuance policy flags.

qsc_asn1_status
qsc_x509_cert_issuance_apply_csr_extensions(qsc_x509_certificate_builder* builder, const qsc_x509_csr* csr, uint32_t policyflags);

Parameters: builder: [struct] The destination certificate builder.; csr: [const][struct] The source certificate signing request.; policyflags: The CSR extension propagation policy bitmask..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_cert_issuance_filter_requested_extensions

Purpose: Filter CSR-requested extensions under issuer policy.

Details: Examines the requested CSR extensions and emits only those extensions permitted by the supplied policy flag mask.

qsc_asn1_status qsc_x509_cert_issuance_filter_requested_extensions(const qsc_x509_csr* csr, uint32_t policyflags, qsc_x509_extensions* filteredextensions);

Parameters: csr: [const][struct] The source certificate signing request.; policyflags: The CSR extension propagation policy bitmask.; filteredextensions: [struct] The destination filtered extension set..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_cert_issuance_validate_csr

Purpose: Validate a certificate signing request for issuance use.

Details: Performs structural and issuance-related validation of a CSR before its contents are used to generate a certificate.

qsc_asn1_status qsc_x509_cert_issuance_validate_csr(const qsc_x509_csr* csr);

Parameters: csr: [const][struct] The certificate signing request to validate..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_add_extension

Purpose: Add a raw extension to the builder.

Details: Appends a caller-supplied extension object to the set of certificate extensions being assembled by the builder.

qsc_asn1_status
qsc_x509_certificate_builder_add_extension(qsc_x509_certificate_builder* builder, const qsc_x509_extension* extension);

Parameters: builder: [struct] The destination certificate builder.; extension: [const][struct] The extension to add..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_add_subject_alt_name_dns

Purpose: Add a DNS subject alternative name entry.

Details: Appends a dNSName general-name entry to the Subject Alternative Name extension content held by the builder.

```
qsc_asn1_status
qsc_x509_certificate_builder_add_subject_alt_name_dns(qsc_x509_certificate_builder*
builder, const char* dnsname, size_t dsnamelen);
```

Parameters: builder: [struct] The destination certificate builder.; dnsname: [const] The DNS host name string.; dsnamelen: The length of the DNS host name string in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_add_subject_alt_name_ip

Purpose: Add an IP-address subject alternative name entry.

Details: Appends an iPAddress general-name entry to the Subject Alternative Name extension content held by the builder.

```
qsc_asn1_status
qsc_x509_certificate_builder_add_subject_alt_name_ip(qsc_x509_certificate_builder*
builder, const uint8_t* address, size_t addresslen);
```

Parameters: builder: [struct] The destination certificate builder.; address: [const] The binary IP address.; addresslen: The length of the IP address in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_apply_generated_identifiers

Purpose: Apply generated key identifiers to the builder.

Details: Computes and assigns identifier extensions derived from the subject public key and, when supplied, the issuer certificate.

qsc_asn1_status
qsc_x509_certificate_builder_apply_generated_identifiers(qsc_x509_certificate_builder* builder, const qsc_x509_certificate* issuer);

Parameters: builder: [struct] The destination certificate builder.; issuer: [const][struct] The optional issuer certificate used for AKI derivation..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_apply_profile

Purpose: Apply a predefined certificate profile to the builder.

Details: Adjusts builder state to conform to a named built-in issuance profile such as root CA, intermediate CA, TLS server, or TLS client.

qsc_asn1_status qsc_x509_certificate_builder_apply_profile(qsc_x509_certificate_builder* builder, uint32_t profile);

Parameters: builder: [struct] The destination certificate builder.; profile: The predefined profile selector..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_clear

Purpose: Clear a certificate builder instance.

Details: Clears all builder state and releases or resets any internally held certificate-construction data. This function is used to erase or reinitialize a builder after use.

void qsc_x509_certificate_builder_clear(qsc_x509_certificate_builder* builder);

Parameters: builder: [struct] The certificate builder to clear..

Return: None (void).

qsc_x509_certificate_builder_encode_tbs_der

Purpose: Encode the TBSCertificate portion as DER.

Details: Serializes the builder contents into the DER representation of the TBSCertificate structure without applying a signature. The caller may pass a null output buffer to query the required size through outputlen.

qsc_asn1_status qsc_x509_certificate_builder_encode_tbs_der(const qsc_x509_certificate_builder* builder, uint8_t* output, size_t* outputlen);

Parameters: builder: [const][struct] The source certificate builder.; output: The destination buffer receiving the DER encoding.; outputlen: The input capacity of output and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_initialize

Purpose: Initialize a certificate builder instance.

Details: Resets the builder to a clean default state suitable for certificate construction. This function shall be called before any other builder mutator is used on the object.

void qsc_x509_certificate_builder_initialize(qsc_x509_certificate_builder* builder);

Parameters: builder: [struct] The certificate builder to initialize..

Return: None (void).

qsc_x509_certificate_builder_set_authority_key_identifier

Purpose: Set the Authority Key Identifier extension content.

Details: Applies an Authority Key Identifier extension to the builder.

qsc_asn1_status
qsc_x509_certificate_builder_set_authority_key_identifier(qsc_x509_certificate_builder* builder, const qsc_x509_authority_key_identifier* authoritykeyidentifier);

Parameters: builder: [struct] The destination certificate builder.; authoritykeyidentifier: [const][struct] The Authority Key Identifier value..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_basic_constraints

Purpose: Set the Basic Constraints extension content.

Details: Applies a Basic Constraints extension definition to the builder.

qsc_asn1_status
qsc_x509_certificate_builder_set_basic_constraints(qsc_x509_certificate_builder* builder, const qsc_x509_basic_constraints* basicconstraints);

Parameters: builder: [struct] The destination certificate builder.; basicconstraints: [const][struct] The Basic Constraints value..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_extended_key_usage

Purpose: Set the Extended Key Usage extension content.

Details: Applies an Extended Key Usage extension definition to the builder.

qsc_asn1_status
qsc_x509_certificate_builder_set_extended_key_usage(qsc_x509_certificate_builder* builder, const qsc_x509_extended_key_usage* extendedkeyusage);

Parameters: builder: [struct] The destination certificate builder.; extendedkeyusage: [const][struct] The Extended Key Usage value..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_issuer

Purpose: Set the issuer distinguished name.

Details: Copies the issuer name into the builder for later certificate encoding.

qsc_asn1_status qsc_x509_certificate_builder_set_issuer(qsc_x509_certificate_builder* builder, const qsc_x509_name* issuer);

Parameters: builder: [struct] The destination certificate builder.; issuer: [const][struct] The issuer distinguished name..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_issuer_from_certificate

Purpose: Set the issuer name from an issuer certificate.

Details: Extracts the issuer certificate subject name and applies it as the issuer distinguished name of the certificate being built.

qsc_asn1_status
qsc_x509_certificate_builder_set_issuer_from_certificate(qsc_x509_certificate_builder* builder, const qsc_x509_certificate* issuer);

Parameters: builder: [struct] The destination certificate builder.; issuer: [const][struct] The issuer certificate..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_key_usage

Purpose: Set the Key Usage extension content.

Details: Applies a Key Usage extension definition to the builder.

qsc_asn1_status qsc_x509_certificate_builder_set_key_usage(qsc_x509_certificate_builder* builder, const qsc_x509_key_usage* keyusage);

Parameters: builder: [struct] The destination certificate builder.; keyusage: [const][struct] The Key Usage value..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_serial

Purpose: Set the certificate serial number.

Details: Assigns the serial number that will be encoded into the certificate. The caller supplies the serial number as a raw byte string.

qsc_asn1_status qsc_x509_certificate_builder_set_serial(qsc_x509_certificate_builder* builder, const uint8_t* serialnumber, size_t serialnumberlen);

Parameters: builder: [struct] The destination certificate builder.; serialnumber: [const] The serial number byte array.; serialnumberlen: The length of the serial number in bytes..

Return: [enum] Returns a qsc_asn1_status code indicating success or the operation failure condition.

qsc_x509_certificate_builder_set_signature_algorithm

Purpose: Set the certificate signature algorithm identifier.

Details: Assigns the outer certificate signature algorithm identifier and the corresponding TBSCertificate signature field used during signing.

qsc_asn1_status
qsc_x509_certificate_builder_set_signature_algorithm(qsc_x509_certificate_builder* builder, const qsc_x509_algorithm_identifier* signaturealgorithm);

Parameters: builder: [struct] The destination certificate builder.; signaturealgorithm: [const][struct] The signature algorithm identifier..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_spki

Purpose: Set the subject public key information.

Details: Assigns the SubjectPublicKeyInfo structure that identifies the public-key algorithm and embeds the subject public key value.

qsc_asn1_status qsc_x509_certificate_builder_set_spki(qsc_x509_certificate_builder* builder, const qsc_x509_subject_public_key_info* spki);

Parameters: builder: [struct] The destination certificate builder.; spki: [const][struct] The subject public key information structure..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_subject

Purpose: Set the subject distinguished name.

Details: Copies the subject name into the builder for later certificate encoding.

qsc_asn1_status qsc_x509_certificate_builder_set_subject(qsc_x509_certificate_builder* builder, const qsc_x509_name* subject);

Parameters: builder: [struct] The destination certificate builder.; subject: [const][struct] The subject distinguished name..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_subject_key_identifier

Purpose: Set the Subject Key Identifier extension content.

Details: Applies a Subject Key Identifier extension to the builder.

qsc_asn1_status
qsc_x509_certificate_builder_set_subject_key_identifier(qsc_x509_certificate_builder* builder, const qsc_x509_subject_key_identifier* subjectkeyidentifier);

Parameters: builder: [struct] The destination certificate builder.; subjectkeyidentifier: [const][struct] The Subject Key Identifier value..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_set_validity

Purpose: Set the certificate validity interval.

Details: Copies the not-before and not-after validity values into the builder. The interval is rejected when notBefore is later than notAfter.

qsc_asn1_status qsc_x509_certificate_builder_set_validity(qsc_x509_certificate_builder* builder, const qsc_x509_validity* validity);

Parameters: builder: [struct] The destination certificate builder.; validity: [const][struct] The validity interval to apply..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_sign

Purpose: Sign and encode a complete certificate.

Details: Encodes the TBSCertificate, invokes the caller-supplied signing callback to produce the certificate signature, and then emits the final DER-encoded Certificate structure.

qsc_asn1_status qsc_x509_certificate_builder_sign(const qsc_x509_certificate_builder* builder, qsc_x509_certificate_sign_callback signcallback, void* context, uint8_t* output, size_t* outputlen);

Parameters: builder: [const][struct] The source certificate builder.; signcallback: The signing callback used to produce the certificate signature.; context: The opaque caller-defined signing context passed to the callback.; output: The destination buffer receiving the DER certificate.; outputlen: The input capacity of output and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_builder_validate_profile

Purpose: Validate builder contents against a predefined profile.

Details: Checks whether the populated builder state is consistent with the selected certificate profile and, when applicable, with the supplied issuer certificate.

qsc_asn1_status qsc_x509_certificate_builder_validate_profile(const qsc_x509_certificate_builder* builder, const qsc_x509_certificate* issuer, uint32_t profile);

Parameters: builder: [const][struct] The source certificate builder.; issuer: [const][struct] The optional issuer certificate.; profile: The predefined profile selector..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_certificate_encode_pem

Purpose: Encode a DER certificate into PEM.

Details: Converts a DER-encoded certificate into the textual PEM representation, including the BEGIN CERTIFICATE and END CERTIFICATE encapsulation markers.

qsc_asn1_status qsc_x509_certificate_encode_pem(const uint8_t* der, size_t derlen, char* output, size_t* outputlen);

Parameters: der: [const] The DER-encoded certificate input.; derlen: The length of the DER input in bytes.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of output and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_compute_authority_key_identifier

Purpose: Compute an Authority Key Identifier from an issuer certificate.

Details: Derives an AKI value from the issuer certificate information and stores the result in the destination identifier object.

qsc_asn1_status qsc_x509_compute_authority_key_identifier(const qsc_x509_certificate* issuer, qsc_x509_authority_key_identifier* authoritykeyidentifier);

Parameters: issuer: [const][struct] The issuer certificate.; authoritykeyidentifier: [struct] The destination Authority Key Identifier..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_compute_subject_key_identifier

Purpose: Compute a Subject Key Identifier from subject public key information.

Details: Derives an SKI value from the supplied SubjectPublicKeyInfo structure and stores the result in the destination identifier object.

qsc_asn1_status qsc_x509_compute_subject_key_identifier(const qsc_x509_subject_public_key_info* spki, qsc_x509_subject_key_identifier* subjectkeyidentifier);

Parameters: spki: [const][struct] The subject public key information source.; subjectkeyidentifier: [struct] The destination Subject Key Identifier..

Return: [enum] Returns a qsc_asn1_status code.

Header: x509crl.h

qsc_x509_certificate_is_revoked_by_crl

Purpose: Test whether a certificate is revoked by a CRL.

Details: Compares the serial number of the supplied certificate against the revoked entries contained in the CRL and returns whether the certificate appears in the CRL revocation list.

bool qsc_x509_certificate_is_revoked_by_crl(const qsc_x509_certificate* certificate, const qsc_x509_crl* crl);

Parameters: certificate: [const][struct] The certificate to test.; crl: [const][struct] The CRL to query..

Return: Returns true if the certificate serial number is present in the CRL; otherwise returns false.

qsc_x509_crl_check_algorithms

Purpose: Check CRL signature algorithm consistency.

Details: Verifies that the TBSCertList signature algorithm identifier and the outer CertificateList signature algorithm identifier are mutually consistent.

qsc_x509_crl_verify_status qsc_x509_crl_check_algorithms(const qsc_x509_crl* crl);

Parameters: crl: [const][struct] The decoded CRL to inspect..

Return: [enum] Returns a qsc_x509_crl_verify_status result code.

qsc_x509_crl_clear

Purpose: Clear a decoded CRL object.

Details: Resets all fields of the CRL object to their default state and clears any decoded content retained from a prior parse operation.

void qsc_x509_crl_clear(qsc_x509_crl* crl);

Parameters: crl: [struct] The CRL object to clear..

Return: None (void).

qsc_x509_crl_decode_der

Purpose: Decode a DER encoded X.509 CRL.

Details: Parses a DER encoded CertificateList structure and populates the destination

qsc_asn1_status qsc_x509_crl_decode_der(const uint8_t* der, size_t derlen, qsc_x509_crl* crl);

Parameters: der: [const] The input DER encoded CRL buffer.; derlen: The length of the input DER buffer in bytes.; crl: [struct] The destination decoded CRL object..

Return: [enum] Returns a qsc_asn1_status code indicating decode success or failure.

qsc_x509_crl_encode_der

Purpose: Encode a decoded CRL object as DER.

Details: Serializes the supplied CRL object into DER encoded CertificateList form. This routine is intended for CRL emission and round-trip testing when the CRL object model contains sufficient information to reconstruct the output.

qsc_asn1_status qsc_x509_crl_encode_der(const qsc_x509_crl* crl, uint8_t* output, size_t* outputlen);

Parameters: crl: [const][struct] The source CRL object.; output: The destination buffer receiving the DER encoded CRL.; outputlen: The input capacity of output and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code indicating encode success or failure.

qsc_x509_crl_find_serial

Purpose: Find a revoked entry by certificate serial number.

Details: Searches the decoded revoked entry set for a serial number match and returns a pointer to the matching entry when found.

const qsc_x509_crl_entry* qsc_x509_crl_find_serial(const qsc_x509_crl* crl, const uint8_t* serial, size_t seriallen);

Parameters: crl: [const][struct] The decoded CRL to search.; serial: [const] The certificate serial number to locate.; seriallen: The length of the serial number in bytes..

Return: Returns a pointer to the matching revoked entry, or NULL if no match is found.

qsc_x509_crl_is_current

Purpose: Test whether a CRL is current at a supplied evaluation time.

Details: Compares the supplied time against the CRL thisUpdate and optional nextUpdate bounds and returns whether the CRL is valid for use at that time.

bool qsc_x509_crl_is_current(const qsc_x509_crl* crl, const qsc_asn1_time* now);

Parameters: crl: [const][struct] The decoded CRL.; now: [const][struct] The evaluation time..

Return: Returns true if the CRL is current at the supplied time; otherwise returns false.

qsc_x509_crl_is_revoked

Purpose: Test whether a certificate is revoked by a CRL.

Details: This function provides the inverse argument ordering of

bool qsc_x509_crl_is_revoked(const qsc_x509_crl* crl, const qsc_x509_certificate* certificate);

Parameters: crl: [const][struct] The CRL to query.; certificate: [const][struct] The certificate to test..

Return: Returns true if the certificate serial number is present in the CRL; otherwise returns false.

qsc_x509_crl_verify

Purpose: Verify a decoded CRL against an issuer certificate.

Details: Performs CRL validation checks including input validation, algorithm consistency, issuer matching, current-time validity checks, issuer policy checks, and callback-driven cryptographic signature verification.

qsc_x509_crl_verify_status qsc_x509_crl_verify(const qsc_x509_crl* crl, const qsc_x509_certificate* issuer, const qsc_asn1_time* now, qsc_x509_crl_signature_verify_callback callback, void* state);

Parameters: crl: [const][struct] The decoded CRL to verify.; issuer: [const][struct] The issuer certificate expected to have signed the CRL.; now: [const][struct] The evaluation time used for thisUpdate and nextUpdate checks.; callback: The caller supplied CRL signature verification callback.; state: Caller-defined opaque state passed through to callback..

Return: [enum] Returns a qsc_x509_crl_verify_status result code.

Header: x509crlwrite.h

qsc_x509_crl_builder_add_extension

Purpose: Add a CRL extension to the builder.

Details: Appends a caller supplied extension object to the set of CRL extensions being assembled by the builder. Duplicate extension types or duplicate extension object identifiers are rejected.

qsc_asn1_status qsc_x509_crl_builder_add_extension(qsc_x509_crl_builder* builder, const qsc_x509_extension* extension);

Parameters: builder: [struct] The destination CRL builder.; extension: [const][struct] The extension to add..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

qsc_x509_crl_builder_add_revoked_serial

Purpose: Add a revoked certificate entry by serial number.

Details: Appends a revoked-certificate entry to the builder using the supplied serial number and revocation date. Leading zero octets in the supplied serial number are normalized before storage, and duplicate serial numbers are rejected.

qsc_asn1_status qsc_x509_crl_builder_add_revoked_serial(qsc_x509_crl_builder* builder, const uint8_t* serialnumber, size_t serialnumberlen, const qsc_asn1_time* revocationdate);

Parameters: builder: [struct] The destination CRL builder.; serialnumber: [const] The revoked certificate serial number.; serialnumberlen: The length of the serial number in bytes.; revocationdate: [const][struct] The revocation date for the entry..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

qsc_x509_crl_builder_clear

Purpose: Clear a CRL builder.

Details: Clears all builder state and resets any accumulated CRL construction data. This function is used to erase or reinitialize a builder after use.

void qsc_x509_crl_builder_clear(qsc_x509_crl_builder* builder);

Parameters: builder: [struct] The CRL builder to clear..

Return: None (void).

qsc_x509_crl_builder_encode_tbs_der

Purpose: Encode the TBSCertList portion as DER.

Details: Serializes the builder contents into the DER representation of the TBSCertList structure without applying a signature. The caller may pass a null output buffer to query the required size through outputlen.

qsc_asn1_status qsc_x509_crl_builder_encode_tbs_der(const qsc_x509_crl_builder* builder, uint8_t* output, size_t* outputlen);

Parameters: builder: [const][struct] The source CRL builder.; output: The destination buffer receiving the DER encoding.; outputlen: The input capacity of output and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

qsc_x509_crl_builder_initialize

Purpose: Initialize a CRL builder.

Details: Resets the CRL builder to a clean default state suitable for CRL construction. This function shall be called before any other builder mutator is used on the object.

void qsc_x509_crl_builder_initialize(qsc_x509_crl_builder* builder);

Parameters: builder: [struct] The CRL builder to initialize..

Return: None (void).

qsc_x509_crl_builder_set_issuer

Purpose: Set the CRL issuer distinguished name.

Details: Copies the issuer name into the builder for subsequent TBSCertList or CertificateList encoding.

qsc_asn1_status qsc_x509_crl_builder_set_issuer(qsc_x509_crl_builder* builder, const qsc_x509_name* issuer);

Parameters: builder: [struct] The destination CRL builder.; issuer: [const][struct] The issuer distinguished name..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

qsc_x509_crl_builder_set_signature_algorithm

Purpose: Set the CRL signature algorithm identifier.

Details: Assigns the AlgorithmIdentifier used in the TBSCertList signature field and the outer CertificateList signature field when the CRL is signed.

qsc_asn1_status qsc_x509_crl_builder_set_signature_algorithm(qsc_x509_crl_builder* builder, const qsc_x509_algorithm_identifier* signaturealgorithm);

Parameters: builder: [struct] The destination CRL builder.; signaturealgorithm: [const][struct] The signature AlgorithmIdentifier..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

qsc_x509_crl_builder_set_update_times

Purpose: Set the CRL update times.

Details: Assigns the thisUpdate and nextUpdate values that define the CRL validity interval. The interval is rejected when thisUpdate is later than nextUpdate.

qsc_asn1_status qsc_x509_crl_builder_set_update_times(qsc_x509_crl_builder* builder, const qsc_asn1_time* thisupdate, const qsc_asn1_time* nextupdate);

Parameters: builder: [struct] The destination CRL builder.; thisupdate: [const][struct] The CRL thisUpdate time.; nextupdate: [const][struct] The CRL nextUpdate time..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

qsc_x509_crl_builder_sign

Purpose: Sign and encode a complete CRL.

Details: Encodes the TBSCertList, invokes the caller supplied signing callback to produce the CRL signature, and emits the final DER encoded CertificateList structure.

qsc_asn1_status qsc_x509_crl_builder_sign(const qsc_x509_crl_builder* builder, qsc_x509_certificate_sign_callback signcallback, void* context, uint8_t* output, size_t* outputlen);

Parameters: builder: [const][struct] The source CRL builder.; signcallback: The signing callback used to produce the CRL signature.; context: Caller defined opaque signing context passed to the callback.; output: The destination buffer receiving the DER encoded CRL.; outputlen: The input capacity of output and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

qsc_x509_crl_builder_validate_issuer

Purpose: Validate the builder issuer against an issuer certificate.

Details: Checks that the builder issuer state is compatible with the supplied issuer certificate before CRL signing or issuance.

qsc_asn1_status qsc_x509_crl_builder_validate_issuer(const qsc_x509_crl_builder* builder, const qsc_x509_certificate* issuer);

Parameters: builder: [const][struct] The source CRL builder.; issuer: [const][struct] The issuer certificate expected to sign the CRL..

Return: [enum] Returns a qsc_asn1_status code indicating validation success or failure.

qsc_x509_crl_der_encode_pem

Purpose: Encode a DER CRL into PEM.

Details: Converts a DER encoded CRL into textual PEM form including the BEGIN X509 CRL and END X509 CRL encapsulation markers.

qsc_asn1_status qsc_x509_crl_der_encode_pem(const uint8_t* der, size_t derlen, char* output, size_t* outputlen);

Parameters: der: [const] The DER encoded CRL input.; derlen: The length of the DER input in bytes.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of output and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

qsc_x509_crl_encode_pem

Purpose: Encode a decoded CRL object into PEM.

Details: Serializes a decoded CRL object and converts it into textual PEM form. This function provides object-to-PEM conversion for CRL instances already held in decoded representation.

qsc_asn1_status qsc_x509_crl_encode_pem(const qsc_x509_crl* crl, char* output, size_t* outputlen);

Parameters: crl: [const][struct] The decoded CRL object.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of output and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code indicating success or failure.

Header: x509csr.h

qsc_x509_csr_add_attribute

Purpose: Add a generic non-extension CSR attribute.

Details: Adds a generic CSR attribute identified by the supplied OID and storing the DER encoding of the first attribute value.

qsc_asn1_status qsc_x509_csr_add_attribute(qsc_x509_csr* csr, const qsc_asn1_oid* oid, const uint8_t* value, size_t valuelen);

Parameters: csr: [struct] The CSR object to update.; oid: [const][struct] The attribute object identifier.; value: [const] The DER encoded first attribute value.; valuelen: The length of the value in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_add_san_dns

Purpose: Add a DNS name to the requested Subject Alternative Name extension.

qsc_asn1_status qsc_x509_csr_add_san_dns(qsc_x509_csr* csr, const char* dnsname, size_t dnsnamelen);

Parameters: csr: [struct] The CSR object to update.; dnsname: [const] The DNS name string.; dnsnamelen: The length of the DNS name string in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_add_san_ip

Purpose: Add an IP address to the requested Subject Alternative Name extension.

qsc_asn1_status qsc_x509_csr_add_san_ip(qsc_x509_csr* csr, const uint8_t* address, size_t addresslen);

Parameters: csr: [struct] The CSR object to update.; address: [const] The binary IP address.; addresslen: The length of the binary IP address in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_clear

Purpose: Clear a CSR object.

Details: Clears all mutable and decoded state held in the CSR object.

void qsc_x509_csr_clear(qsc_x509_csr* csr);

Parameters: csr: [struct] The CSR object to clear..

Return: None (void).

qsc_x509_csr_copy_extension_request

Purpose: Copy the normalized extensionRequest payload.

qsc_asn1_status qsc_x509_csr_copy_extension_request(const qsc_x509_csr* csr, qsc_x509_extensions* extensions);

Parameters: csr: [const][struct] The CSR object to inspect.; extensions: [struct] The destination extension set..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_decode_der

Purpose: Decode a DER encoded CSR.

qsc_asn1_status qsc_x509_csr_decode_der(qsc_x509_csr* csr, const uint8_t* input, size_t inputlen);

Parameters: csr: [struct] The destination CSR object.; input: [const] The DER encoded CSR input buffer.; inputlen: The length of the input buffer in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_decode_pem

Purpose: Decode a PEM encoded CSR.

qsc_asn1_status qsc_x509_csr_decode_pem(qsc_x509_csr* csr, const char* input, size_t inputlen);

Parameters: csr: [struct] The destination CSR object.; input: [const] The PEM encoded CSR input buffer.; inputlen: The length of the input buffer in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_encode_der

Purpose: Encode and sign a complete CSR as DER.

Details: Encodes the CertificationRequestInfo structure, invokes the supplied signing callback, and emits the complete DER encoded CertificationRequest object.

qsc_asn1_status qsc_x509_csr_encode_der(const qsc_x509_csr* csr, qsc_x509_certificate_sign_callback signcallback, void* context, uint8_t* output, size_t* outputlen);

Parameters: csr: [const][struct] The CSR object to encode.; signcallback: The CSR signing callback.; context: Caller-defined opaque signing context.; output: The destination buffer receiving the DER CSR.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_encode_info_der

Purpose: Encode the CertificationRequestInfo structure as DER.

Details: Serializes only the unsigned CertificationRequestInfo portion of the CSR.

qsc_asn1_status qsc_x509_csr_encode_info_der(const qsc_x509_csr* csr, uint8_t* output, size_t* outputlen);

Parameters: csr: [const][struct] The CSR object to encode.; output: The destination buffer receiving the DER output.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_encode_pem

Purpose: Encode a DER CSR as PEM.

qsc_asn1_status qsc_x509_csr_encode_pem(const uint8_t* der, size_t derlen, char* output, size_t* outputlen);

Parameters: der: [const] The DER encoded CSR input.; derlen: The length of the DER input in bytes.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of the output buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_find_extension

Purpose: Find a requested extension by type.

const qsc_x509_extension* qsc_x509_csr_find_extension(const qsc_x509_csr* csr, qsc_x509_extension_type type);

Parameters: csr: [const][struct] The CSR object to inspect.; type: [enum] The extension type to locate..

Return: Returns a pointer to the matching extension, or NULL if no match is present.

qsc_x509_csr_get_attribute

Purpose: Get a generic CSR attribute by OID.

const qsc_x509_csr_attribute* qsc_x509_csr_get_attribute(const qsc_x509_csr* csr, const qsc_asn1_oid* oid);

Parameters: csr: [const][struct] The CSR object to inspect.; oid: [const][struct] The attribute object identifier to locate..

Return: Returns a pointer to the matching attribute, or NULL if no match is present.

qsc_x509_csr_get_extension_request

Purpose: Get the normalized extensionRequest payload.

const qsc_x509_extensions* qsc_x509_csr_get_extension_request(const qsc_x509_csr* csr);

Parameters: csr: [const][struct] The CSR object to inspect..

Return: Returns a pointer to the requested extension set.

qsc_x509_csr_initialize

Purpose: Initialize a CSR object.

Details: Initializes a CSR object to a clean default state suitable for first use. For an object that may already hold decoded or owned DER state, call qsc_x509_csr_clear() instead.

void qsc_x509_csr_initialize(qsc_x509_csr* csr);

Parameters: csr: [struct] The CSR object to initialize..

Return: None (void).

qsc_x509_csr_set_extended_key_usage

Purpose: Set the requested Extended Key Usage extension.

qsc_asn1_status qsc_x509_csr_set_extended_key_usage(qsc_x509_csr* csr, const qsc_x509_extended_key_usage* extendedkeyusage);

Parameters: csr: [struct] The CSR object to update.; extendedkeyusage: [const][struct] The Extended Key Usage value..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_extension_request

Purpose: Set the normalized extensionRequest payload.

Details: Replaces the CSR extensionRequest contents with the supplied X.509 extension set.

qsc_asn1_status qsc_x509_csr_set_extension_request(qsc_x509_csr* csr, const qsc_x509_extensions* extensions);

Parameters: csr: [struct] The CSR object to update.; extensions: [const][struct] The requested extension set..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_ml_dsa_signature_algorithm

Purpose: Set the CSR signature algorithm to an ML-DSA variant.

Details: Selects the ML-DSA parameter set corresponding to the supplied level and updates the CSR signature algorithm identifier accordingly.

qsc_asn1_status qsc_x509_csr_set_ml_dsa_signature_algorithm(qsc_x509_csr* csr, uint32_t level);

Parameters: csr: [struct] The CSR object to update.; level: The ML-DSA parameter-set level selector..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_ml_dsa_spki

Purpose: Set the CSR subject public key information to an ML-DSA key.

Details: Populates the CSR SPKI field using an ML-DSA public key and the supplied ML-DSA parameter-set level.

qsc_asn1_status qsc_x509_csr_set_ml_dsa_spki(qsc_x509_csr* csr, uint32_t level, const uint8_t* publickey, size_t publickeylen);

Parameters: csr: [struct] The CSR object to update.; level: The ML-DSA parameter-set level selector.; publickey: [const] The ML-DSA public key bytes.; publickeylen: The length of the public key in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_ml_kem_spki

Purpose: Set the CSR subject public key information to an ML-KEM key.

Details: Populates the CSR SPKI field using an ML-KEM public key and the supplied ML-KEM parameter-set level.

qsc_asn1_status qsc_x509_csr_set_ml_kem_spki(qsc_x509_csr* csr, uint32_t level, const uint8_t* publickey, size_t publickeylen);

Parameters: csr: [struct] The CSR object to update.; level: The ML-KEM parameter-set level selector.; publickey: [const] The ML-KEM public key bytes.; publickeylen: The length of the public key in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_signature_algorithm

Purpose: Set the CSR signature algorithm identifier.

qsc_asn1_status qsc_x509_csr_set_signature_algorithm(qsc_x509_csr* csr, const qsc_x509_algorithm_identifier* signaturealgorithm);

Parameters: csr: [struct] The CSR object to update.; signaturealgorithm: [const][struct] The signature algorithm identifier..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_spki

Purpose: Set the CSR subject public key information.

qsc_asn1_status qsc_x509_csr_set_spki(qsc_x509_csr* csr, const qsc_x509_subject_public_key_info* spki);

Parameters: csr: [struct] The CSR object to update.; spki: [const][struct] The subject public key information..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_subject

Purpose: Set the CSR subject distinguished name.

qsc_asn1_status qsc_x509_csr_set_subject(qsc_x509_csr* csr, const qsc_x509_name* subject);

Parameters: csr: [struct] The CSR object to update.; subject: [const][struct] The subject distinguished name..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_subject_alt_name

Purpose: Set the requested Subject Alternative Name extension.

qsc_asn1_status qsc_x509_csr_set_subject_alt_name(qsc_x509_csr* csr, const qsc_x509_subject_alt_name* subjectaltname);

Parameters: csr: [struct] The CSR object to update.; subjectaltname: [const][struct] The Subject Alternative Name value..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_set_subject_key_identifier

Purpose: Set the requested Subject Key Identifier extension.

qsc_asn1_status qsc_x509_csr_set_subject_key_identifier(qsc_x509_csr* csr, const qsc_x509_subject_key_identifier* subjectkeyidentifier);

Parameters: csr: [struct] The CSR object to update.; subjectkeyidentifier: [const][struct] The Subject Key Identifier value..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_sign

Purpose: Sign and encode a CSR as DER.

Details: This function performs CSR signing and final DER emission. It provides a semantically explicit signing entry point alongside qsc_x509_csr_encode_der.

qsc_asn1_status qsc_x509_csr_sign(const qsc_x509_csr* csr, qsc_x509_certificate_sign_callback signcallback, void* context, uint8_t* output, size_t* outputlen);

Parameters: csr: [const][struct] The CSR object to sign.; signcallback: The CSR signing callback.; context: Caller-defined opaque signing context.; output: The destination buffer receiving the DER CSR.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_verify

Purpose: Verify a CSR using its contained subject public key information.

bool qsc_x509_csr_verify(const qsc_x509_csr* csr);

Parameters: csr: [const][struct] The CSR to verify..

Return: Returns true if the CSR signature is valid; otherwise returns false.

qsc_x509_csr_verify_ex

Purpose: Verify a CSR using a caller-supplied verification callback.

bool qsc_x509_csr_verify_ex(const qsc_x509_csr* csr, qsc_x509_csr_signature_verify_callback verifycallback, void* state);

Parameters: csr: [const][struct] The CSR to verify.; verifycallback: The verification callback.; state: Caller-defined opaque verification context..

Return: Returns true if the CSR signature is valid; otherwise returns false.

qsc_x509_csr_verify_with_spki

Purpose: Verify a CSR signature against a caller-supplied signer SPKI.

Details: This function verifies the CSR signature using the supplied subject public key information structure rather than the SPKI carried inside the CSR. It is intended for cases where signature verification is performed against an external or previously normalized public key representation.

bool qsc_x509_csr_verify_with_spki(const qsc_x509_csr* csr, const qsc_x509_subject_public_key_info* signerspki);

Parameters: csr: [const][struct] The CSR to verify.; signerspki: [const][struct] The signer subject public key information used for verification..

Return: Returns true if the CSR signature is valid; otherwise returns false.

Header: x509ext.h

qsc_x509_ext_authority_key_identifier_decode

Purpose: Decode an AuthorityKeyIdentifier extension payload.

Details: Parses the DER encoded extnValue contents of an AuthorityKeyIdentifier extension and extracts the supported authority key identifier fields. The ASN.1 definition is: AuthorityKeyIdentifier ::= SEQUENCE { keyIdentifier [0] KeyIdentifier OPTIONAL, authorityCertIssuer [1] GeneralNames OPTIONAL, authorityCertSerialNumber [2] CertificateSerialNumber OPTIONAL }

qsc_asn1_status qsc_x509_ext_authority_key_identifier_decode(const uint8_t* data, size_t datalen, qsc_x509_authority_key_identifier* aki);

Parameters: data: [const] The DER encoded Authority Key Identifier payload.; datalen: The length of the encoded payload in bytes.; aki: [struct] The destination Authority Key Identifier object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_authority_key_identifier_encode

Purpose: Encode an AuthorityKeyIdentifier extension payload.

Details: Serializes a qsc_x509_authority_key_identifier object into the DER representation of an AuthorityKeyIdentifier extension payload.

qsc_asn1_status qsc_x509_ext_authority_key_identifier_encode(const qsc_x509_authority_key_identifier* aki, uint8_t* output, size_t* outputlen);

Parameters: aki: [const][struct] The source Authority Key Identifier object.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_basic_constraints_decode

Purpose: Decode a BasicConstraints extension payload.

Details: Parses the DER encoded extnValue contents of a BasicConstraints extension and writes the result to the supplied qsc_x509_basic_constraints structure. The ASN.1 definition is: BasicConstraints ::= SEQUENCE { cA BOOLEAN DEFAULT FALSE, pathLenConstraint INTEGER OPTIONAL } The cA field indicates whether the subject may act as a certification authority. The optional pathLenConstraint limits the number of non-self-issued CA certificates that may follow this certificate in a certification path.

qsc_asn1_status qsc_x509_ext_basic_constraints_decode(const uint8_t* data, size_t datalen, qsc_x509_basic_constraints* bc);

Parameters: data: [const] The DER encoded BasicConstraints payload.; datalen: The length of the encoded payload in bytes.; bc: [struct] The destination Basic Constraints object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_basic_constraints_encode

Purpose: Encode a BasicConstraints extension payload.

Details: Serializes a qsc_x509_basic_constraints object into the DER representation of a BasicConstraints extension payload suitable for placement inside extnValue.

qsc_asn1_status qsc_x509_ext_basic_constraints_encode(const qsc_x509_basic_constraints* bc, uint8_t* output, size_t* outputlen);

Parameters: bc: [const][struct] The source Basic Constraints object.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_extended_key_usage_decode

Purpose: Decode an ExtendedKeyUsage extension payload.

Details: Parses the DER encoded extnValue contents of an ExtendedKeyUsage extension and records the contained key purpose identifiers. The ASN.1 definition is: ExtKeyUsageSyntax ::= SEQUENCE SIZE (1..MAX) OF KeyPurposeId KeyPurposeId ::= OBJECT IDENTIFIER

qsc_asn1_status qsc_x509_ext_extended_key_usage_decode(const uint8_t* data, size_t datalen, qsc_x509_extended_key_usage* eku);

Parameters: data: [const] The DER encoded Extended Key Usage payload.; datalen: The length of the encoded payload in bytes.; eku: [struct] The destination Extended Key Usage object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_extended_key_usage_encode

Purpose: Encode an ExtendedKeyUsage extension payload.

Details: Serializes a qsc_x509_extended_key_usage object into the DER representation of an ExtendedKeyUsage extension payload.

qsc_asn1_status qsc_x509_ext_extended_key_usage_encode(const qsc_x509_extended_key_usage* eku, uint8_t* output, size_t* outputlen);

Parameters: eku: [const][struct] The source Extended Key Usage object.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_has_eku

Purpose: Test whether an Extended Key Usage set contains a requested usage bit.

Details: Evaluates the supplied implementation-defined bit mask against the decoded or constructed Extended Key Usage object and reports whether the requested usage is present.

bool qsc_x509_ext_has_eku(const qsc_x509_extended_key_usage* eku, uint32_t bitmask);

Parameters: eku: [const][struct] The Extended Key Usage object to inspect.; bitmask: The implementation-defined usage bit mask to test..

Return: Returns true if the requested usage is present; otherwise returns false.

qsc_x509_ext_issuer_alt_name_decode

Purpose: Decode an IssuerAltName extension payload.

Details: Parses the DER encoded extnValue contents of an Issuer Alternative Name extension and records the supported GeneralName entries.

qsc_asn1_status qsc_x509_ext_issuer_alt_name_decode(const uint8_t* data, size_t datalen, qsc_x509_issuer_alt_name* ian);

Parameters: data: [const] The DER encoded Issuer Alternative Name payload.; datalen: The length of the encoded payload in bytes.; ian: [struct] The destination Issuer Alternative Name object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_issuer_alt_name_encode

Purpose: Encode an IssuerAltName extension payload.

Details: Serializes a qsc_x509_issuer_alt_name object into the DER representation of an Issuer Alternative Name extension payload.

qsc_asn1_status qsc_x509_ext_issuer_alt_name_encode(const qsc_x509_issuer_alt_name* ian, uint8_t* output, size_t* outputlen);

Parameters: ian: [const][struct] The source Issuer Alternative Name object.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_key_usage_decode

Purpose: Decode a KeyUsage extension payload.

Details: Parses the DER encoded extnValue contents of a KeyUsage extension and converts the BIT STRING representation into the implementation-defined usage mask stored in the caller supplied output variable. The ASN.1 definition is: KeyUsage ::= BIT STRING

qsc_asn1_status qsc_x509_ext_key_usage_decode(const uint8_t* data, size_t datalen, uint16_t* usage);

Parameters: data: [const] The DER encoded KeyUsage payload.; datalen: The length of the encoded payload in bytes.; usage: The destination usage bit mask..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_key_usage_encode

Purpose: Encode a KeyUsage extension payload.

Details: Serializes a qsc_x509_key_usage object into the DER representation of a KeyUsage extension payload.

qsc_asn1_status qsc_x509_ext_key_usage_encode(const qsc_x509_key_usage* keyusage, uint8_t* output, size_t* outputlen);

Parameters: keyusage: [const][struct] The source Key Usage object.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_subject_alt_name_add_dns

Purpose: Add a DNS name entry to a Subject Alternative Name object.

Details: Appends a dNSName GeneralName entry to the supplied Subject Alternative Name container.

qsc_asn1_status qsc_x509_ext_subject_alt_name_add_dns(qsc_x509_subject_alt_name* san, const char* dnsname, size_t dnsnamelen);

Parameters: san: [struct] The Subject Alternative Name object to update.; dnsname: [const] The DNS host name string.; dnsnamelen: The length of the DNS host name string in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_subject_alt_name_add_ip

Purpose: Add an IP address entry to a Subject Alternative Name object.

Details: Appends an iPAddress GeneralName entry to the supplied Subject Alternative Name container.

qsc_asn1_status qsc_x509_ext_subject_alt_name_add_ip(qsc_x509_subject_alt_name* san, const uint8_t* address, size_t addresslen);

Parameters: san: [struct] The Subject Alternative Name object to update.; address: [const] The binary IP address.; addresslen: The length of the binary IP address in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_subject_alt_name_decode

Purpose: Decode a SubjectAltName extension payload.

Details: Parses the DER encoded extnValue contents of a Subject Alternative Name extension and records the supported GeneralName entries.

qsc_asn1_status qsc_x509_ext_subject_alt_name_decode(const uint8_t* data, size_t datalen, qsc_x509_subject_alt_name* san);

Parameters: data: [const] The DER encoded Subject Alternative Name payload.; datalen: The length of the encoded payload in bytes.; san: [struct] The destination Subject Alternative Name object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_subject_alt_name_encode

Purpose: Encode a SubjectAltName extension payload.

Details: Serializes a qsc_x509_subject_alt_name object into the DER representation of a Subject Alternative Name extension payload.

qsc_asn1_status qsc_x509_ext_subject_alt_name_encode(const qsc_x509_subject_alt_name* san, uint8_t* output, size_t* outputlen);

Parameters: san: [const][struct] The source Subject Alternative Name object.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_subject_key_identifier_decode

Purpose: Decode a SubjectKeyIdentifier extension payload.

Details: Parses the DER encoded extnValue contents of a SubjectKeyIdentifier extension and copies the identifier bytes to the supplied output structure. The ASN.1 definition is: SubjectKeyIdentifier ::= OCTET STRING

qsc_asn1_status qsc_x509_ext_subject_key_identifier_decode(const uint8_t* data, size_t datalen, qsc_x509_subject_key_identifier* ski);

Parameters: data: [const] The DER encoded Subject Key Identifier payload.; datalen: The length of the encoded payload in bytes.; ski: [struct] The destination Subject Key Identifier object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_ext_subject_key_identifier_encode

Purpose: Encode a SubjectKeyIdentifier extension payload.

Details: Serializes a qsc_x509_subject_key_identifier object into the DER representation of a SubjectKeyIdentifier extension payload.

qsc_asn1_status qsc_x509_ext_subject_key_identifier_encode(const qsc_x509_subject_key_identifier* ski, uint8_t* output, size_t* outputlen);

Parameters: ski: [const][struct] The source Subject Key Identifier object.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_extension_decode

Purpose: Decode a single X.509 Extension sequence.

Details: Parses an ASN.1 DER encoded Extension object and converts it to the normalized qsc_x509_extension representation. The decoded extension includes the extension object identifier, the optional critical flag, and the raw extnValue OCTET STRING contents. The ASN.1 definition is: Extension ::= SEQUENCE { extnID OBJECT IDENTIFIER, critical BOOLEAN DEFAULT FALSE, extnValue OCTET STRING } This function does not interpret the inner extnValue payload beyond extracting it from the outer OCTET STRING wrapper. Typed interpretation is performed by the extension-specific decode routines declared below.

qsc_asn1_status qsc_x509_extension_decode(const qsc_encoding_ber_element* element, qsc_x509_extension* ext);

Parameters: element: [const][struct] The ASN.1 sequence element containing the encoded Extension structure.; ext: [struct] The destination normalized extension object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_extension_initialize

Purpose: brief Initialize a normalized extension entry.

Details: brief Initialize a normalized extension entry.

void qsc_x509_extension_initialize(qsc_x509_extension* ext);

Parameters: ext: [struct] The extension object to initialize..

Return: None (void).

qsc_x509_extension_validate

Purpose: brief Validate a normalized extension entry. Performs structural and payload-adjacent validation on a decoded extension entry. This routine does not replace object-level certificate, CSR, or CRL policy validation, but it rejects malformed critical fields, missing extnValue content, and inconsistent normalized state.

Details: brief Validate a normalized extension entry. Performs structural and payload-adjacent validation on a decoded extension entry. This routine does not replace object-level certificate, CSR, or CRL policy validation, but it rejects malformed critical fields, missing extnValue content, and inconsistent normalized state.

qsc_asn1_status qsc_x509_extension_validate(const qsc_x509_extension* ext);

Parameters: ext: [const][struct] The decoded extension entry. eturn [enum] Returns a qsc_asn1_status code..

Return: qsc_asn1_status; the header does not specify additional failure semantics.

qsc_x509_extensions_decode

Purpose: Decode an X.509 Extensions sequence.

Details: Parses an ASN.1 Extensions container and decodes each contained Extension entry into the normalized qsc_x509_extensions output structure. The ASN.1 definition is: Extensions ::= SEQUENCE OF Extension Each entry is first normalized by qsc_x509_extension_decode. Typed payload interpretation, when required, is then performed separately by the extension-specific decode functions.

qsc_asn1_status qsc_x509_extensions_decode(const qsc_encoding_ber_element* element, qsc_x509_extensions* extensions);

Parameters: element: [const][struct] The ASN.1 sequence element containing the Extensions collection.; extensions: [struct] The destination normalized extension set..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_extensions_initialize

Purpose: brief Initialize a normalized extension set.

Details: brief Initialize a normalized extension set.

void qsc_x509_extensions_initialize(qsc_x509_extensions* extensions);

Parameters: extensions: [struct] The extension set object to initialize..

Return: None (void).

qsc_x509_extensions_validate

Purpose: brief Validate a normalized extension set. Performs set-level structural checks such as duplicate extension rejection and consistency checks across already-decoded typed extension state.

Details: brief Validate a normalized extension set. Performs set-level structural checks such as duplicate extension rejection and consistency checks across already-decoded typed extension state.

qsc_asn1_status qsc_x509_extensions_validate(const qsc_x509_extensions* extensions);

Parameters: extensions: [const][struct] The decoded extension set. eturn [enum] Returns a qsc_asn1_status code..

Return: qsc_asn1_status; the header does not specify additional failure semantics.

Header: x509host.h

qsc_x509_certificate_match_dns_name

Purpose: Match a hostname against certificate DNS identifiers.

Details: Evaluates the supplied hostname against dNSName identifiers contained in the certificate Subject Alternative Name extension. Certificate DNS identifiers are processed using their explicit X.509 lengths and are not interpreted as NUL-terminated strings. Matching is ASCII case-insensitive. A wildcard is permitted only as the complete left-most label and matches exactly one hostname label. Subject commonName values are not used as a fallback when no matching dNSName Subject Alternative Name is present.

bool qsc_x509_certificate_match_dns_name(const qsc_x509_certificate* certificate, const char* hostname);

Parameters: certificate: [const][struct] The certificate to evaluate.; hostname: [const] The NUL-terminated hostname to match..

Return: Returns true if the hostname matches a dNSName Subject Alternative Name; otherwise returns false.

qsc_x509_certificate_match_endpoint

Purpose: Match a network endpoint against a certificate.

Details: Evaluates both hostname and IP address inputs against the certificate. The function first attempts DNS name matching when a hostname is supplied, then attempts IP address matching when a binary address is supplied. Either input may be NULL, but at least one identity input should be provided by the caller.

bool qsc_x509_certificate_match_endpoint(const qsc_x509_certificate* certificate, const char* hostname, const uint8_t* address, size_t addresslen);

Parameters: certificate: [const][struct] The certificate to evaluate.; hostname: [const] The hostname to match, or NULL.; address: [const] The binary IP address, or NULL.; addresslen: The length of the IP address in bytes..

Return: Returns true if the endpoint matches the certificate identity; otherwise returns false.

qsc_x509_certificate_match_hostname

Purpose: Match a hostname against a certificate.

Details: Performs DNS service-identity validation against the dNSName entries in the certificate Subject Alternative Name extension. The subject commonName is not used for hostname validation. Certificate DNS identifiers are evaluated using their explicit X.509 lengths and the DNS matching rules implemented by qsc_x509_certificate_match_dns_name().

bool qsc_x509_certificate_match_hostname(const qsc_x509_certificate* certificate, const char* hostname);

Parameters: certificate: [const][struct] The certificate to evaluate.; hostname: [const] The NUL-terminated hostname to match..

Return: Returns true if the hostname matches a dNSName Subject Alternative Name; otherwise returns false.

qsc_x509_certificate_match_ip_address

Purpose: Match an IP address against certificate IP identifiers.

Details: Compares a binary IPv4 or IPv6 address against the iPAddress entries contained in the certificate Subject Alternative Name extension.

bool qsc_x509_certificate_match_ip_address(const qsc_x509_certificate* certificate, const uint8_t* address, size_t addresslen);

Parameters: certificate: [const][struct] The certificate to evaluate.; address: [const] The binary IP address.; addresslen: The address length in bytes. Supported values are 4 and 16..

Return: Returns true if the address matches a certificate IP identifier; otherwise returns false.

qsc_x509_dns_name_match

Purpose: Match a DNS hostname against a certificate DNS pattern.

Details: Compares a hostname against a DNS identifier using ASCII case-insensitive comparison. A wildcard is permitted only as the complete left-most label, in the form "*.example.com", and matches exactly one hostname label. Partial-label wildcards and multiple wildcards are rejected. Both inputs to this function are NUL-terminated strings. Certificate identifiers stored as length-delimited X.509 values are validated through qsc_x509_certificate_match_dns_name().

bool qsc_x509_dns_name_match(const char* pattern, const char* hostname);

Parameters: pattern: [const] The NUL-terminated DNS pattern.; hostname: [const] The NUL-terminated hostname to evaluate..

Return: Returns true if the hostname matches the DNS pattern; otherwise returns false.

Header: x509key.h

qsc_x509_certificate_key_match

Purpose: Test whether a certificate matches a private key object.

Details: Compares the subject public key information contained in the certificate with the public key material associated with the supplied private-key object.

bool qsc_x509_certificate_key_match(const qsc_x509_certificate* certificate, const qsc_x509_private_key* key);

Parameters: certificate: [const][struct] The certificate to evaluate.; key: [const][struct] The private-key object to compare..

Return: Returns true if the certificate corresponds to the supplied private key; otherwise returns false.

qsc_x509_private_key_decode_pkcs8_der

Purpose: Decode a PKCS #8 DER private key.

Details: Parses a DER encoded PKCS #8 private key object and populates the destination private-key container.

qsc_asn1_status qsc_x509_private_key_decode_pkcs8_der(const uint8_t* data, size_t datalen, qsc_x509_private_key* key);

Parameters: data: [const] The DER encoded input buffer.; datalen: The length of the input buffer in bytes.; key: [struct] The destination private-key container..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_pkcs8_der_ex

Purpose: Decode a PKCS #8 DER private key into caller-supplied buffers.

Details: Parses a DER encoded PKCS #8 private key object and writes the decoded algorithm identifier, private key bytes, optional public key bytes, and presence flag into caller-managed storage. On entry, the caller supplies the output buffer capacities. On return, the length outputs are set to the number of bytes written, or reset to zero when decoding fails before a value is produced.

qsc_asn1_status qsc_x509_private_key_decode_pkcs8_der_ex(const uint8_t* data, size_t datalen, qsc_x509_algorithm_identifier* algorithm, uint8_t* privatekey, size_t privatekeycapacity, size_t* privatekeylen, uint8_t* publickey, size_t publickeycapacity, size_t* publickeylen, bool* publickeypresent);

Parameters: data: [const] The DER encoded input buffer.; datalen: The length of the input buffer in bytes.; algorithm: [struct] The destination algorithm identifier.; privatekey: The destination private key buffer.; privatekeycapacity: The capacity of the private key

buffer in bytes.; privatekeylen: The number of bytes written to the private key buffer.; publickey: The destination public key buffer.; publickeycapacity: The capacity of the public key buffer in bytes.; publickeylen: The number of bytes written to the public key buffer.; publickeypresent: Indicates whether a public key was present in the decoded object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_pkcs8_pem

Purpose: Decode a PKCS #8 PEM private key.

Details: Parses a PEM encoded PKCS #8 private key object and populates the destination private-key container.

qsc_asn1_status qsc_x509_private_key_decode_pkcs8_pem(const char* pem, size_t pemlen, qsc_x509_private_key* key);

Parameters: pem: [const] The NULL-terminated PEM text.; pemlen: The length of the PEM text.; key: [struct] The destination private-key container..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_pkcs8_pem_ex

Purpose: Decode a PKCS #8 PEM private key into caller-supplied buffers.

Details: Parses a PEM encoded PKCS #8 private key object and writes the decoded algorithm identifier, private key bytes, optional public key bytes, and presence flag into caller-managed storage. The helper first base64-decodes the PEM object into an internal DER buffer and then applies the same strict PKCS #8 decode rules as the DER interface.

qsc_asn1_status qsc_x509_private_key_decode_pkcs8_pem_ex(const char* pem, size_t pemlen, qsc_x509_algorithm_identifier* algorithm, uint8_t* privatekey, size_t privatekeycapacity, size_t* privatekeylen, uint8_t* publickey, size_t publickeycapacity, size_t* publickeylen, bool* publickeypresent);

Parameters: pem: [const] The NULL-terminated PEM text.; pemlen: The length of the PEM text.; algorithm: [struct] The destination algorithm identifier.; privatekey: The destination private key buffer.; privatekeycapacity: The capacity of the private key buffer in bytes.; privatekeylen: The number of bytes written to the private key buffer.; publickey: The destination public key buffer.; publickeycapacity: The capacity of the public key buffer in bytes.; publickeylen: The number of bytes written to the public key

buffer.; publickeypresent: Indicates whether a public key was present in the decoded object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_sec1_der

Purpose: Decode a SEC 1 DER private key.

Details: Parses a DER encoded SEC 1 private key object and populates the destination private-key container.

qsc_asn1_status qsc_x509_private_key_decode_sec1_der(const uint8_t* data, size_t datalen, qsc_x509_private_key* key);

Parameters: data: [const] The DER encoded input buffer.; datalen: The length of the input buffer in bytes.; key: [struct] The destination private-key container..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_sec1_pem

Purpose: Decode a SEC 1 PEM private key.

Details: Parses a PEM encoded SEC 1 private key object and populates the destination private-key container.

qsc_asn1_status qsc_x509_private_key_decode_sec1_pem(const char* pem, size_t pemlen, qsc_x509_private_key* key);

Parameters: pem: [const] The NULL-terminated PEM text.; pemlen: The length of the pem array.; key: [struct] The destination private-key container..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_expected_private_size

Purpose: Return the expected private-key byte length for the supplied X.509 algorithm identifier, or zero when the algorithm is unsupported.

size_t qsc_x509_private_key_expected_private_size(const qsc_x509_algorithm_identifier* algorithm);

Parameters: algorithm: Decoded algorithm identifier used to select the expected private-key length..

Return: size_t; the header does not specify additional failure semantics.

qsc_x509_private_key_expected_public_size

Purpose: Get the expected public key size for an algorithm identifier.

Details: Returns the implementation-defined public key length associated with the supplied algorithm identifier. This helper is used for strict parameter-set validation during decode and import operations.

size_t qsc_x509_private_key_expected_public_size(const qsc_x509_algorithm_identifier* algorithm);

Parameters: algorithm: [const][struct] The algorithm identifier to evaluate..

Return: Returns the expected public key size in bytes, or zero if the algorithm is unsupported.

qsc_x509_private_key_initialize

Purpose: Initialize a private-key container.

Details: Resets the private-key object to a clean default state before decoding or reuse.

void qsc_x509_private_key_initialize(qsc_x509_private_key* key);

Parameters: key: [struct] The private-key container to initialize..

Return: None (void).

qsc_x509_private_key_validate

Purpose: brief Validate a decoded private-key object. Performs strict algorithm, parameter-set, and key-size validation on a normalized private-key object. When an embedded public key is present, the helper also validates its size against the decoded algorithm identifier.

Details: brief Validate a decoded private-key object. Performs strict algorithm, parameter-set, and key-size validation on a normalized private-key object. When an embedded public key is present, the helper also validates its size against the decoded algorithm identifier.

qsc_asn1_status qsc_x509_private_key_validate(const qsc_x509_private_key* key);

Parameters: key: [const][struct] The private-key object to validate. eturn [enum] Returns a qsc_asn1_status code..

Return: qsc_asn1_status; the header does not specify additional failure semantics.

Header: x509keywrite.h

qsc_x509_private_key_encode_pkcs8_der

Purpose: Encode a normalized private key object as PKCS #8 DER.

Details: Serializes the supplied private-key container to DER encoded PKCS #8 format. The caller may request inclusion of the optional embedded public key.

qsc_asn1_status qsc_x509_private_key_encode_pkcs8_der(const qsc_x509_private_key* key, bool includepublickey, uint8_t* output, size_t* outputlen);

Parameters: key: [const][struct] The source private-key container.; includepublickey: Includes the optional embedded public key when set to true.; output: The destination buffer receiving the DER encoding.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_pkcs8_der_ex

Purpose: Encode raw private key material as PKCS #8 DER.

Details: Serializes caller-supplied private key bytes, algorithm identifier, and optional public key bytes to DER encoded PKCS #8 format without requiring a qsc_x509_private_key container.

qsc_asn1_status qsc_x509_private_key_encode_pkcs8_der_ex(const qsc_x509_algorithm_identifier* algorithm, const uint8_t* privatekey, size_t privatekeylen, const uint8_t* publickey, size_t publickeylen, bool publickeypresent, uint8_t* output, size_t* outputlen);

Parameters: algorithm: [const][struct] The algorithm identifier to encode.; privatekey: [const] The raw private key bytes.; privatekeylen: The length of the private key in bytes.; publickey: [const] The optional raw public key bytes.; publickeylen: The length of the public key in bytes.; publickeypresent: Indicates whether the public key input is present.; output: The destination buffer receiving the DER encoding.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_pkcs8_ml_dsa_der

Purpose: Encode an ML-DSA private key as PKCS #8 DER.

Details: Serializes caller-supplied ML-DSA private key material using the selected ML-DSA parameter set and emits DER encoded PKCS #8 output.

qsc_asn1_status
qsc_x509_private_key_encode_pkcs8_ml_dsa_der(qsc_x509_pqc_parameter_set parameter, const uint8_t* privatekey, size_t privatekeylen, const uint8_t* publickey, size_t publickeylen, bool publickeypresent, uint8_t* output, size_t* outputlen);

Parameters: parameter: [enum] The ML-DSA parameter set identifier.; privatekey: [const] The raw ML-DSA private key bytes.; privatekeylen: The length of the private key in bytes.; publickey: [const] The optional raw ML-DSA public key bytes.; publickeylen: The length of the public key in bytes.; publickeypresent: Indicates whether the public key input is present.; output: The destination buffer receiving the DER encoding.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_pkcs8_ml_dsa_pem

Purpose: Encode an ML-DSA private key as PKCS #8 PEM.

Details: Serializes caller-supplied ML-DSA private key material using the selected ML-DSA parameter set and converts the resulting PKCS #8 object to PEM text.

qsc_asn1_status
qsc_x509_private_key_encode_pkcs8_ml_dsa_pem(qsc_x509_pqc_parameter_set parameter, const uint8_t* privatekey, size_t privatekeylen, const uint8_t* publickey, size_t publickeylen, bool publickeypresent, char* output, size_t* outputlen);

Parameters: parameter: [enum] The ML-DSA parameter set identifier.; privatekey: [const] The raw ML-DSA private key bytes.; privatekeylen: The length of the private key in bytes.; publickey: [const] The optional raw ML-DSA public key bytes.; publickeylen: The length of the public key in bytes.; publickeypresent: Indicates whether the public key input is present.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of the output buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_pkcs8_ml_kem_der

Purpose: Encode an ML-KEM private key as PKCS #8 DER.

Details: Serializes caller-supplied ML-KEM private key material using the selected ML-KEM parameter set and emits DER encoded PKCS #8 output.

qsc_asn1_status
qsc_x509_private_key_encode_pkcs8_ml_kem_der(qsc_x509_pqc_parameter_set parameter, const uint8_t* privatekey, size_t privatekeylen, const uint8_t* publickey, size_t publickeylen, bool publickeypresent, uint8_t* output, size_t* outputlen);

Parameters: parameter: [enum] The ML-KEM parameter set identifier.; privatekey: [const] The raw ML-KEM private key bytes.; privatekeylen: The length of the private key in bytes.; publickey: [const] The optional raw ML-KEM public key bytes.; publickeylen: The length of the public key in bytes.; publickeypresent: Indicates whether the public key input is present.; output: The destination buffer receiving the DER encoding.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_pkcs8_ml_kem_pem

Purpose: Encode an ML-KEM private key as PKCS #8 PEM.

Details: Serializes caller-supplied ML-KEM private key material using the selected ML-KEM parameter set and converts the resulting PKCS #8 object to PEM text.

qsc_asn1_status
qsc_x509_private_key_encode_pkcs8_ml_kem_pem(qsc_x509_pqc_parameter_set parameter, const uint8_t* privatekey, size_t privatekeylen, const uint8_t* publickey, size_t publickeylen, bool publickeypresent, char* output, size_t* outputlen);

Parameters: parameter: [enum] The ML-KEM parameter set identifier.; privatekey: [const] The raw ML-KEM private key bytes.; privatekeylen: The length of the private key in bytes.; publickey: [const] The optional raw ML-KEM public key bytes.; publickeylen: The length of the public key in bytes.; publickeypresent: Indicates whether the public key input is present.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of the output buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_pkcs8_pem

Purpose: Encode a normalized private key object as PKCS #8 PEM.

Details: Serializes the supplied private-key container to PKCS #8 DER and converts the result to PEM text form.

qsc_asn1_status qsc_x509_private_key_encode_pkcs8_pem(const qsc_x509_private_key* key, bool includepublickey, char* output, size_t* outputlen);

Parameters: key: [const][struct] The source private-key container.; includepublickey: Includes the optional embedded public key when set to true.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of the output buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_pkcs8_pem_ex

Purpose: Encode raw private key material as PKCS #8 PEM.

Details: Serializes caller-supplied private key bytes, algorithm identifier, and optional public key bytes to PKCS #8 DER and converts the result to PEM text form.

qsc_asn1_status qsc_x509_private_key_encode_pkcs8_pem_ex(const qsc_x509_algorithm_identifier* algorithm, const uint8_t* privatekey, size_t privatekeylen, const uint8_t* publickey, size_t publickeylen, bool publickeypresent, char* output, size_t* outputlen);

Parameters: algorithm: [const][struct] The algorithm identifier to encode.; privatekey: [const] The raw private key bytes.; privatekeylen: The length of the private key in bytes.; publickey: [const] The optional raw public key bytes.; publickeylen: The length of the public key in bytes.; publickeypresent: Indicates whether the public key input is present.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of the output buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_sec1_der

Purpose: Encode a normalized private key object as SEC 1 DER.

Details: Serializes the supplied private-key container to DER encoded SEC 1 format. The caller may request inclusion of algorithm parameters and an embedded public key when supported by the underlying key representation.

qsc_asn1_status qsc_x509_private_key_encode_sec1_der(const qsc_x509_private_key* key, bool includeparameters, bool includepublickey, uint8_t* output, size_t* outputlen);

Parameters: key: [const][struct] The source private-key container.; includeparameters: Includes algorithm parameters when set to true.; includepublickey: Includes the optional embedded public key when set to true.; output: The destination buffer receiving the DER encoding.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_encode_sec1_pem

Purpose: Encode a normalized private key object as SEC 1 PEM.

Details: Serializes the supplied private-key container to SEC 1 DER and converts the result to PEM text form.

qsc_asn1_status qsc_x509_private_key_encode_sec1_pem(const qsc_x509_private_key* key, bool includeparameters, bool includepublickey, char* output, size_t* outputlen);

Parameters: key: [const][struct] The source private-key container.; includeparameters: Includes algorithm parameters when set to true.; includepublickey: Includes the optional embedded public key when set to true.; output: The destination character buffer receiving the PEM text.; outputlen: The input capacity of the output buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

Header: x509name.h

qsc_x509_name_attribute_long_name

Purpose: Gets the descriptive registry name of a distinguished name attribute type.

const char* qsc_x509_name_attribute_long_name(qsc_x509_name_attribute_type type);

Parameters: type: [qsc_x509_name_attribute_type] The distinguished name attribute type..

Return: [const char*] Returns the descriptive registry name, or NULL if the type is invalid.

qsc_x509_name_attribute_short_name

Purpose: Gets the short display name of a distinguished name attribute type.

const char* qsc_x509_name_attribute_short_name(qsc_x509_name_attribute_type type);

Parameters: type: [qsc_x509_name_attribute_type] The distinguished name attribute type..

Return: [const char*] Returns the short display name, or NULL if the type is invalid.

qsc_x509_name_attribute_type_from_oid

Purpose: Maps an object identifier registry identifier to a normalized distinguished name attribute type.

qsc_x509_name_attribute_type qsc_x509_name_attribute_type_from_oid(qsc_oid_id id);

Parameters: id: [qsc_oid_id] The object identifier registry identifier..

Return: [qsc_x509_name_attribute_type] Returns the mapped distinguished name attribute type.

qsc_x509_name_clear

Purpose: Clears a decoded distinguished name structure.

void qsc_x509_name_clear(qsc_x509_name* name);

Parameters: name: [qsc_x509_name*] The decoded distinguished name structure..

Return: None (void).

qsc_x509_name_equals

Purpose: Tests two decoded distinguished names for deterministic equality.

bool qsc_x509_name_equals(const qsc_x509_name* a, const qsc_x509_name* b);

Parameters: a: [const qsc_x509_name*] The first decoded distinguished name.; b: [const qsc_x509_name*] The second decoded distinguished name..

Return: [bool] Returns true if the decoded distinguished names are equal.

qsc_x509_name_find_first

Purpose: Finds the first attribute of a given type within a decoded distinguished name.

const qsc_x509_name_attribute* qsc_x509_name_find_first(const qsc_x509_name* name, qsc_x509_name_attribute_type type);

Parameters: name: [const qsc_x509_name*] The decoded distinguished name.; type: [qsc_x509_name_attribute_type] The requested distinguished name attribute type..

Return: [const qsc_x509_name_attribute*] Returns a pointer to the first matching attribute, or NULL if no match exists.

qsc_x509_name_parse

Purpose: Parses an X.509 Name element.

qsc_asn1_status qsc_x509_name_parse(const qsc_encoding_ber_element* element, qsc_x509_name* name);

Parameters: element: [const qsc_encoding_ber_element*] The ASN.1 element representing the Name value.; name: [qsc_x509_name*] Receives the decoded distinguished name..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_x509_name_to_string

Purpose: Formats a decoded distinguished name as a compact zero-terminated string.

Details: The formatter emits a comma-separated sequence of short-name assignments in the order stored in the decoded object, for example CN=example, O=Org. Unknown attribute types are rendered using the dotted-decimal attribute OID.

qsc_asn1_status qsc_x509_name_to_string(const qsc_x509_name* name, char* output, size_t otplen, size_t* outlen);

Parameters: name: [const qsc_x509_name*] The decoded distinguished name.; output: [char*] The destination character array.; otplen: [size_t] The size of the destination character array.; outlen: [size_t*] Receives the number of characters written excluding the terminating zero..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

Header: x509ocsp.h

qsc_x509_ocsp_parse_response

Purpose: Parse an OCSP response from DER.

Details: Decodes a DER encoded OCSP response using strict DER parsing and extracts the normalized certificate status, optional revocation time, and optional nonce fields into the supplied response object.

bool qsc_x509_ocsp_parse_response(const uint8_t* der, size_t derlen, qsc_x509_ocsp_response* response);

Parameters: der: [const] The DER encoded OCSP response buffer.; derlen: The length of the DER buffer in bytes.; response: [struct] The destination parsed OCSP response object..

Return: Returns true if the OCSP response was successfully parsed; otherwise returns false.

qsc_x509_ocsp_validate

Purpose: Validate a certificate using OCSP.

Details: Builds an OCSP request for the supplied certificate and issuer certificate, retrieves the responder output through the caller-supplied fetch callback, validates and parses the result, verifies the BasicOCSPResponse using the preserved signed bytes, and stores the normalized OCSP status in the destination response object.

bool qsc_x509_ocsp_validate(const qsc_x509_certificate* certificate, const qsc_x509_certificate* issuer, const char* url, qsc_x509_ocsp_fetch_callback fetch, void* context, const qsc_asn1_time* now, qsc_x509_ocsp_response* response);

Parameters: certificate: [const][struct] The certificate whose revocation status is being queried.; issuer: [const][struct] The issuer certificate for the queried certificate.; url: [const] The OCSP responder URL.; fetch: The caller-supplied OCSP transport callback.; context: Caller-defined opaque transport context.; now: [const][struct] The caller-supplied validation time used for responder and response freshness checks.; response: [struct] The destination parsed OCSP response object..

Return: Returns true if OCSP validation completed successfully; otherwise returns false.

qsc_x509_ocsp_verify_responder

Purpose: Verify an OCSP responder certificate.

Details: Validates that the supplied responder certificate is acceptable for signing OCSP BasicOCSPResponse objects for the supplied issuer. The function checks the responder validity interval, enforces the OCSP signing extended key usage for delegated responders, optionally accepts an explicitly trusted responder present in the supplied trust store, and otherwise verifies the responder certificate against the issuer certificate using the library's signature verification path. When the responder certificate is the issuer certificate itself, the OCSP signing extended key usage is not required. An explicitly trusted responder present in the supplied trust store is accepted under the caller's local trust policy. Otherwise, a delegated responder must include the explicit id-kp-OCSPSigning extended key usage. If the KeyUsage extension is present, the digitalSignature bit must be set.

bool qsc_x509_ocsp_verify_responder(const qsc_x509_certificate* responder, const qsc_x509_certificate* issuer, const qsc_x509_store* store, const qsc_asn1_time* now);

Parameters: responder: [const][struct] The responder certificate extracted from the OCSP response.; issuer: [const][struct] The issuer certificate for the certificate whose status is being checked.; store: [const][struct] Optional trust store containing explicitly trusted responder certificates.; now: [const][struct] The caller-supplied validation time..

Return: Returns true if the responder certificate is acceptable for OCSP response signing; otherwise returns false.

Header: x509pem.h

qsc_x509_certificate_decode_pem

Purpose: Decode a PEM encoded certificate.

Details: Parses a PEM encoded X.509 certificate and decodes the contained DER Certificate object into the supplied certificate structure.

qsc_asn1_status qsc_x509_certificate_decode_pem(const char* pem, size_t pemlen, qsc_x509_certificate* certificate);

Parameters: pem: [const] The PEM encoded certificate text.; pemlen: The length of the PEM text in bytes.; certificate: [struct] The destination certificate object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_chain_decode_pem_bundle

Purpose: Decode a PEM certificate bundle into a chain object.

Details: Parses a concatenated PEM bundle containing one or more certificates and loads the decoded certificates into the caller-supplied chain object using the provided certificate storage array.

qsc_asn1_status qsc_x509_chain_decode_pem_bundle(const char* pem, size_t pemlen, qsc_x509_certificate* certificates, size_t certcount, qsc_x509_chain* chain);

Parameters: pem: [const] The PEM bundle text.; pemlen: The length of the PEM bundle in bytes.; certificates: [struct] The caller-supplied certificate storage array.; certcount: The number of certificate elements available in certificates.; chain: [struct] The destination certificate chain object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_chain_free

Purpose: Free the resources associated with a decoded certificate chain.

Details: Releases or resets any internal state held by the supplied chain object after PEM bundle loading or chain construction.

void qsc_x509_chain_free(qsc_x509_chain* chain);

Parameters: chain: [struct] The certificate chain object to free..

Return: None (void).

qsc_x509_crl_decode_pem

Purpose: Decode a CRL from a PEM bundle.

Details: Searches a PEM bundle for a certificate revocation list object and decodes the first matching CRL into the destination structure.

qsc_asn1_status qsc_x509_crl_decode_pem(const char* pem, size_t pemlen, qsc_x509_crl* crl);

Parameters: pem: [const] The PEM bundle text.; pemlen: The length of the PEM bundle in bytes.; crl: [struct] The destination CRL object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_csr_decode_pem_from_bundle

Purpose: Decode a CSR from a PEM bundle.

Details: Searches a PEM bundle for a certificate signing request object and decodes the first matching CSR into the destination structure.

qsc_asn1_status qsc_x509_csr_decode_pem_from_bundle(const char* pem, size_t pemlen, qsc_x509_csr* csr);

Parameters: pem: [const] The PEM bundle text.; pemlen: The length of the PEM bundle in bytes.; csr: [struct] The destination CSR object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_pem_encode_certificate

Purpose: Convert a DER encoded certificate to PEM.

qsc_asn1_status qsc_x509_pem_encode_certificate(const uint8_t* der, size_t derlen, char* pem, size_t* pemlen);

Parameters: der: [const] The DER encoded certificate.; derlen: The length of the DER input in bytes.; pem: The destination PEM buffer.; pemlen: The input capacity of the PEM buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_pem_encode_crl

Purpose: Convert a DER encoded CRL to PEM.

qsc_asn1_status qsc_x509_pem_encode_crl(const uint8_t* der, size_t derlen, char* pem, size_t* pemlen);

Parameters: der: [const] The DER encoded CRL.; derlen: The length of the DER input in bytes.; pem: The destination PEM buffer.; pemlen: The input capacity of the PEM buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_pem_encode_csr

Purpose: Convert a DER encoded CSR to PEM.

qsc_asn1_status qsc_x509_pem_encode_csr(const uint8_t* der, size_t derlen, char* pem, size_t* pemlen);

Parameters: der: [const] The DER encoded CSR.; derlen: The length of the DER input in bytes.; pem: The destination PEM buffer.; pemlen: The input capacity of the PEM buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_pem_encode_private_key_pkcs8

Purpose: Convert a DER encoded PKCS #8 private key to PEM.

qsc_asn1_status qsc_x509_pem_encode_private_key_pkcs8(const uint8_t* der, size_t derlen, char* pem, size_t* pemlen);

Parameters: der: [const] The DER encoded private key.; derlen: The length of the DER input in bytes.; pem: The destination PEM buffer.; pemlen: The input capacity of the PEM buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_pem_encode_private_key_pkcs8_from_key

Purpose: Encode a normalized private key object as PKCS #8 PEM.

Details: Serializes the supplied private-key container as PKCS #8 and converts the resulting DER object to PEM text form.

qsc_asn1_status qsc_x509_pem_encode_private_key_pkcs8_from_key(const qsc_x509_private_key* key, bool includepublickey, char* pem, size_t* pemlen);

Parameters: key: [const][struct] The source private-key container.; includepublickey: Includes the optional embedded public key when set to true.; pem: The destination PEM buffer.; pemlen: The input capacity of the PEM buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_pem_encode_private_key_sec1

Purpose: Convert a DER encoded SEC 1 private key to PEM.

qsc_asn1_status qsc_x509_pem_encode_private_key_sec1(const uint8_t* der, size_t derlen, char* pem, size_t* pemlen);

Parameters: der: [const] The DER encoded private key.; derlen: The length of the DER input in bytes.; pem: The destination PEM buffer.; pemlen: The input capacity of the PEM buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_pem_encode_private_key_sec1_from_key

Purpose: Encode a normalized private key object as SEC 1 PEM.

Details: Serializes the supplied private-key container as SEC 1 and converts the resulting DER object to PEM text form.

qsc_asn1_status qsc_x509_pem_encode_private_key_sec1_from_key(const qsc_x509_private_key* key, bool includeparameters, bool includepublickey, char* pem, size_t* pemlen);

Parameters: key: [const][struct] The source private-key container.; includeparameters: Includes algorithm parameters when set to true.; includepublickey: Includes the optional embedded public key when set to true.; pem: The destination PEM buffer.; pemlen: The input capacity of the PEM buffer and, on success, the number of characters written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_pem_from_bundle

Purpose: Decode a private key from a PEM bundle.

Details: Searches a PEM bundle for a supported private-key object and decodes the first matching key into the normalized private-key container.

qsc_asn1_status qsc_x509_private_key_decode_pem_from_bundle(const char* pem, size_t pemlen, qsc_x509_private_key* key);

Parameters: pem: [const] The PEM bundle text.; pemlen: The length of the PEM bundle in bytes.; key: [struct] The destination private-key object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_pkcs8_pem_ex_from_bundle

Purpose: Decode a PKCS #8 private key from a PEM bundle into caller-supplied buffers.

Details: Searches a PEM bundle for a PEM encoded PKCS #8 private key and decodes the first matching object into caller-managed algorithm, private-key, and public- key buffers.

qsc_asn1_status qsc_x509_private_key_decode_pkcs8_pem_ex_from_bundle(const char* pem, size_t pemlen, qsc_x509_algorithm_identifier* algorithm, uint8_t* privatekey, size_t privatekeycapacity, size_t* privatekeylen, uint8_t* publickey, size_t publickeycapacity, size_t* publickeylen, bool* publickeypresent);

Parameters: pem: [const] The PEM bundle text.; pemlen: The length of the PEM bundle in bytes.; algorithm: [struct] The destination algorithm identifier.; privatekey: The destination private key buffer.; privatekeycapacity: The capacity of the private key buffer in bytes.; privatekeylen: The number of bytes written to the private key buffer.; publickey: The destination public key buffer.; publickeycapacity: The capacity of the public key buffer in bytes.; publickeylen: The number of bytes written to the public key buffer.; publickeypresent: Indicates whether a public key was present in the decoded object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_pkcs8_pem_from_bundle

Purpose: Decode a PKCS #8 private key from a PEM bundle.

Details: Searches a PEM bundle for a PEM encoded PKCS #8 private key and decodes the first matching object into the normalized private-key container.

qsc_asn1_status qsc_x509_private_key_decode_pkcs8_pem_from_bundle(const char* pem, size_t pemlen, qsc_x509_private_key* key);

Parameters: pem: [const] The PEM bundle text.; pemlen: The length of the PEM bundle in bytes.; key: [struct] The destination private-key object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_private_key_decode_sec1_pem_from_bundle

Purpose: Decode a SEC 1 private key from a PEM bundle.

Details: Searches a PEM bundle for a PEM encoded SEC 1 private key and decodes the first matching object into the normalized private-key container.

qsc_asn1_status qsc_x509_private_key_decode_sec1_pem_from_bundle(const char* pem, size_t pemlen, qsc_x509_private_key* key);

Parameters: pem: [const] The PEM bundle text.; pemlen: The length of the PEM bundle in bytes.; key: [struct] The destination private-key object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_store_free

Purpose: Free the resources associated with a certificate store.

Details: Releases or resets any internal state held by the supplied trust store object after PEM bundle loading or verification use.

void qsc_x509_store_free(qsc_x509_store* store);

Parameters: store: [struct] The certificate store object to free..

Return: None (void).

qsc_x509_store_load_pem_bundle

Purpose: Load a PEM trust-anchor bundle into a certificate store.

Details: Parses a concatenated PEM bundle containing one or more trust-anchor certificates and populates the destination store using the provided anchor storage array.

qsc_asn1_status qsc_x509_store_load_pem_bundle(const char* pem, size_t pemlen, qsc_x509_trust_anchor* anchors, size_t anchorcount, qsc_x509_store* store);

Parameters: pem: [const] The PEM bundle text.; pemlen: The length of the PEM bundle in bytes.; anchors: [struct] The caller-supplied trust-anchor storage array.; anchorcount: The number of trust-anchor elements available in anchors.; store: [struct] The destination certificate store object..

Return: [enum] Returns a qsc_asn1_status code.

Header: x509pkcs12.h

qsc_x509_pkcs12_clear

Purpose: Clear a PKCS #12 bundle and release certificate backing storage.

Details: Releases all certificate DER storage owned by a successfully parsed bundle, securely erases the decoded private-key object, and resets the container. Call this function before reusing or discarding a bundle that has been populated by qsc_x509_pkcs12_parse.

void qsc_x509_pkcs12_clear(qsc_x509_pkcs12_bundle* bundle);

Parameters: bundle: [struct] The initialized PKCS #12 bundle to clear..

Return: None (void).

qsc_x509_pkcs12_decrypt_encrypted_private_key_info

Purpose: Decrypt an EncryptedPrivateKeyInfo object.

Details: Decrypts a DER encoded EncryptedPrivateKeyInfo structure using the supplied password and writes the recovered PrivateKeyInfo bytes to the caller-supplied output buffer.

bool qsc_x509_pkcs12_decrypt_encrypted_private_key_info(const uint8_t* data, size_t datalen, const char* password, uint8_t* privatekeyinfo, size_t privatekeyinfocapacity, size_t* privatekeyinfolen);

Parameters: data: [const] The DER encoded EncryptedPrivateKeyInfo input buffer.; datalen: The length of the input buffer in bytes.; password: [const] The password used to decrypt the private key object.; privatekeyinfo: The destination buffer receiving the decrypted PrivateKeyInfo bytes.; privatekeyinfocapacity: The capacity of the destination buffer in bytes.; privatekeyinfolen: The number of bytes written to the destination buffer..

Return: Returns true if decryption completed successfully; otherwise returns false.

qsc_x509_pkcs12_encode_mac_data_der

Purpose: Encode a PKCS #12 MacData object over an AuthSafe OCTET STRING value.

Details: Computes the PKCS #12 whole-file integrity MAC over the supplied AuthSafe contents and encodes the resulting MacData object. When usepbmac1 is false, the function emits a classic RFC 7292 MacData object using the PKCS #12 Appendix B MAC KDF with HMAC-SHA2-256. When usepbmac1 is true, the function emits an RFC 9879 PBMAC1 AlgorithmIdentifier using PBKDF2-HMAC-SHA2-256 and HMAC-SHA2-256.

bool qsc_x509_pkcs12_encode_mac_data_der(uint8_t* output, size_t outcap, size_t* outlen, const uint8_t* authsafe, size_t authsafelen, const char* password, const uint8_t* salt, size_t saltlen, uint64_t iterations, bool usepbmac1);

Parameters: output: The destination buffer receiving DER encoded MacData.; outcap: The capacity of the destination buffer in bytes.; outlen: The number of bytes written to the destination buffer.; authsafe: [const] The AuthSafe OCTET STRING contents to authenticate.; authsafelen: The length of the AuthSafe contents in bytes.; password: [const] The PKCS #12 password.; salt: [const] The MAC salt.; saltlen: The MAC salt length

in bytes.; iterations: The MAC KDF iteration count.; usepbmac1: When true, encode PBMAC1; otherwise encode classic MacData..

Return: Returns true if MacData encoding completed successfully; otherwise returns false.

qsc_x509_pkcs12_initialize

Purpose: Initialize a PKCS #12 bundle container.

Details: Resets a fresh bundle object to a clean default state before first use. A bundle populated by qsc_x509_pkcs12_parse owns certificate backing storage and must be released with qsc_x509_pkcs12_clear rather than reinitialized.

void qsc_x509_pkcs12_initialize(qsc_x509_pkcs12_bundle* bundle);

Parameters: bundle: [struct] The PKCS #12 bundle container to initialize..

Return: None (void).

qsc_x509_pkcs12_parse

Purpose: Parse a PKCS #12 bundle from DER.

Details: Decodes a PKCS #12 container, validates the outer MAC, processes the AuthenticatedSafe ContentInfo sequence, applies password-based decryption where required, extracts at most one private key, and loads decoded X.509 certificates into owned backing storage. Recognized SafeBag parse/decryption failures abort the import instead of returning a partial bundle. When both a private key and certificate material are present, the key must cryptographically match a certificate; localKeyId attributes are used as an additional association check when both sides provide them. The destination is initialized before parsing. A previously populated bundle must first be released with qsc_x509_pkcs12_clear before reuse.

bool qsc_x509_pkcs12_parse(const uint8_t* data, size_t datalen, const char* password, qsc_x509_pkcs12_bundle* bundle);

Parameters: data: [const] The DER encoded PKCS #12 input buffer.; datalen: The length of the input buffer in bytes.; password: [const] The password used to decrypt protected contents.; bundle: [struct] The destination parsed PKCS #12 bundle object..

Return: Returns true if parsing completed successfully; otherwise returns false.

qsc_x509_pkcs12_verify_mac_data

Purpose: Verify the outer PKCS #12 MacData without extracting bundle contents.

Details: Decodes the PFX wrapper, locates the AuthSafe OCTET STRING, and verifies the outer MacData. Both classic RFC 7292 MacData and RFC 9879 PBMAC1 MacData are supported. This function does not decrypt safe contents or extract certificates or private keys; use qsc_x509_pkcs12_parse for full bundle import.

bool qsc_x509_pkcs12_verify_mac_data(const uint8_t* data, size_t datalen, const char* password);

Parameters: data: [const] The DER encoded PKCS #12 input buffer.; datalen: The length of the input buffer in bytes.; password: [const] The PKCS #12 MAC password..

Return: Returns true if the outer PKCS #12 MacData verifies; otherwise returns false.

Header: x509rev.h

qsc_x509_certificate_check_revocation

Purpose: Check certificate revocation status using resolver-driven CRL acquisition.

Details: Resolves a CRL for the target certificate using the configured resolver callback and applies revocation processing according to the supplied revocation policy options.

qsc_x509_revocation_status qsc_x509_certificate_check_revocation(const qsc_x509_certificate* certificate, const qsc_x509_certificate* issuer, const qsc_x509_revocation_options* options, const qsc_x509_time* validationtime);

Parameters: certificate: [const][struct] The certificate whose revocation status is being checked.; issuer: [const][struct] The issuer certificate for the target certificate.; options: [const][struct] The revocation policy and callback options.; validationtime: [const][struct] The time used to evaluate CRL validity..

Return: [enum] Returns a qsc_x509_revocation_status code.

qsc_x509_certificate_check_revocation_with_crl

Purpose: Check certificate revocation status using a supplied CRL.

Details: Evaluates the target certificate against the supplied CRL, verifies the CRL against the issuer certificate using the caller-supplied verification callback, and returns a normalized revocation status result.

qsc_x509_revocation_status qsc_x509_certificate_check_revocation_with_crl(const qsc_x509_certificate* certificate, const qsc_x509_certificate* issuer, const qsc_x509_crl* crl, qsc_x509_crl_signature_verify_callback verifycallback, void* verifycontext, const qsc_x509_time* validationtime);

Parameters: certificate: [const][struct] The certificate whose revocation status is being checked.; issuer: [const][struct] The issuer certificate for the target certificate.; crl: [const][struct] The CRL to use for revocation checking.; verifycallback: The callback used to verify the CRL signature.; verifycontext: Caller-defined opaque context passed to the verification callback.; validationtime: [const][struct] The time used to evaluate CRL validity..

Return: [enum] Returns a qsc_x509_revocation_status code.

qsc_x509_revocation_options_initialize

Purpose: Initialize a revocation options structure.

Details: Resets the revocation options object to a clean default state suitable for later policy and callback configuration.

void qsc_x509_revocation_options_initialize(qsc_x509_revocation_options* options);

Parameters: options: [struct] The revocation options structure to initialize..

Return: None (void).

Header: x509revext.h

qsc_x509_apply_delta_crl

Purpose: Apply a delta CRL to a base CRL and produce a merged CRL view.

Details: Validates the supplied base CRL and delta CRL against the issuer certificate and evaluation time, then applies the delta revocation updates to the base CRL and writes the resulting merged revocation state to the destination CRL object. The caller supplies the CRL signature verification callback so that cryptographic signature checking remains aligned with the surrounding X.509 verification layer and supported algorithm set.

qsc_x509_crl_verify_status qsc_x509_apply_delta_crl(qsc_x509_crl* mergedcrl, const qsc_x509_crl* basecrl, const qsc_x509_crl* deltacrl, const qsc_x509_certificate* issuer, const qsc_asn1_time* now, qsc_x509_crl_signature_verify_callback callback, void* state);

Parameters: mergedcrl: [struct] The destination CRL object receiving the merged result.; basecrl: [const][struct] The base CRL to update.; deltacrl: [const][struct] The delta CRL containing incremental revocation changes.; issuer: [const][struct] The issuer certificate expected to have signed the CRLs.; now: [const][struct] The evaluation time used for CRL validity checks.; callback: The caller-supplied CRL signature verification callback.; state: Caller-defined opaque context passed to the verification callback..

Return: [enum] Returns a qsc_x509_crl_verify_status code indicating success or the reason the merge failed.

qsc_x509_crl_check_certificate_scope

Purpose: Check whether a CRL can be used to determine the status of a certificate.

Details: Validates the supported issuingDistributionPoint scope restrictions for the supplied CRL and target certificate. CRLs using scope forms that are not implemented by QSC, including named distribution points, reason-partitioned CRLs, indirect CRLs, and attribute-certificate-only CRLs, are rejected fail-closed rather than being treated as complete status information.

qsc_x509_crl_verify_status qsc_x509_crl_check_certificate_scope(const qsc_x509_crl* crl, const qsc_x509_certificate* certificate, const qsc_x509_certificate* issuer);

Parameters: crl: [const][struct] The CRL whose scope is being evaluated.; certificate: [const][struct] The certificate whose status is being checked.; issuer: [const][struct] The issuer certificate for the target certificate..

Return: [enum] Returns QSC_X509_CRL_VERIFY_STATUS_SUCCESS only when the CRL can safely determine status for the supplied certificate.

qsc_x509_ocsp_stapled_verify

Purpose: Verify a stapled OCSP response for a certificate.

Details: Parses and validates a stapled OCSP response associated with the supplied certificate and issuer certificate. This helper is intended for use in protocols that carry an OCSP response out-of-band from direct responder retrieval, such as TLS certificate status stapling.

bool qsc_x509_ocsp_stapled_verify(const uint8_t* stapled, size_t stapledlen, const qsc_x509_certificate* certificate, const qsc_x509_certificate* issuer);

Parameters: stapled: [const] The stapled OCSP response bytes.; stapledlen: The length of the stapled OCSP response in bytes.; certificate: [const][struct] The certificate whose

status is being verified.; issuer: [const][struct] The issuer certificate for the target certificate..

Return: Returns true if the stapled OCSP response is accepted for the certificate; otherwise returns false.

Header: x509sig.h

qsc_x509_signature_algorithm_decode

Purpose: Decodes a signature AlgorithmIdentifier structure.

qsc_asn1_status qsc_x509_signature_algorithm_decode(const qsc_encoding_ber_element* element, qsc_x509_algorithm_identifier* algorithm);

Parameters: element: [const qsc_encoding_ber_element*] The ASN.1 SEQUENCE element containing the AlgorithmIdentifier structure.; algorithm: [qsc_x509_algorithm_identifier*] Receives the decoded algorithm identifier..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_x509_signature_algorithm_equal

Purpose: Compares two decoded algorithm identifiers for semantic equality.

bool qsc_x509_signature_algorithm_equal(const qsc_x509_algorithm_identifier* left, const qsc_x509_algorithm_identifier* right);

Parameters: left: [const qsc_x509_algorithm_identifier*] The left algorithm value.; right: [const qsc_x509_algorithm_identifier*] The right algorithm value..

Return: [bool] Returns true if the two algorithm identifiers are equivalent.

qsc_x509_signature_algorithm_is_ecdsa

Purpose: Tests whether a decoded signature algorithm is an ECDSA variant.

bool qsc_x509_signature_algorithm_is_ecdsa(qsc_x509_signature_algorithm algorithm);

Parameters: algorithm: [qsc_x509_signature_algorithm] The decoded signature algorithm..

Return: [bool] Returns true for ECDSA-with-SHA1/SHA256/SHA384/SHA512.

qsc_x509_signature_algorithm_is_ml_dsa

Purpose: Tests whether a decoded signature algorithm is an ML-DSA variant.

bool qsc_x509_signature_algorithm_is_ml_dsa(qsc_x509_signature_algorithm algorithm);

Parameters: algorithm: [qsc_x509_signature_algorithm] The decoded signature algorithm..

Return: [bool] Returns true for ML-DSA-44, ML-DSA-65, or ML-DSA-87.

qsc_x509_signature_algorithm_matches_spki

Purpose: Tests whether a decoded signature algorithm is compatible with the supplied subject public key information.

bool qsc_x509_signature_algorithm_matches_spki(qsc_x509_signature_algorithm algorithm, const qsc_x509_subject_public_key_info* spki);

Parameters: algorithm: [qsc_x509_signature_algorithm] The decoded signature algorithm.; spki: [const qsc_x509_subject_public_key_info*] The public key information..

Return: [bool] Returns true if the signature algorithm and SPKI describe a compatible signing primitive and parameter set.

qsc_x509_signature_component_size

Purpose: Gets the expected maximum signature component size in octets for a named curve.

size_t qsc_x509_signature_component_size(qsc_x509_named_curve curve);

Parameters: curve: [qsc_x509_named_curve] The named curve identifier..

Return: [size_t] Returns the signature component size in octets, or zero if the curve is not recognized.

qsc_x509_signature_expected_size

Purpose: Gets the expected raw signature size in octets for a signature algorithm.

Details: For ECDSA this returns the size of the raw concatenated r || s form used by the internal QSC verification path. For ML-DSA this returns the compiled signature size when available. For unknown or unsupported algorithms, zero is returned.

size_t qsc_x509_signature_expected_size(qsc_x509_signature_algorithm algorithm, qsc_x509_named_curve curve);

Parameters: algorithm: [qsc_x509_signature_algorithm] The decoded signature algorithm.; curve: [qsc_x509_named_curve] The named curve for ECDSA signatures..

Return: [size_t] Returns the expected raw signature size in octets, or zero.

qsc_x509_signature_value_decode_ecdsa

Purpose: Decodes a certificate ECDSA signature BIT STRING.

qsc_asn1_status qsc_x509_signature_value_decode_ecdsa(const qsc_encoding_ber_element* element, qsc_x509_named_curve curve, qsc_x509_ecdsa_signature* signature);

Parameters: element: [const qsc_encoding_ber_element*] The BIT STRING element containing the DER encoded ECDSA signature value.; curve: [qsc_x509_named_curve] The named curve associated with the signing key.; signature: [qsc_x509_ecdsa_signature*] Receives the decoded signature components..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_x509_signature_value_decode_raw

Purpose: Decodes a certificate signature BIT STRING as a raw octet sequence.

qsc_asn1_status qsc_x509_signature_value_decode_raw(const qsc_encoding_ber_element* element, uint8_t* signature, size_t signaturelen, size_t* outlen);

Parameters: element: [const qsc_encoding_ber_element*] The BIT STRING element containing the signature value.; signature: [uint8_t*] Receives the raw signature octets.; signaturelen: [size_t] The size of the output array in octets.; outlen: [size_t*] Receives the number of octets written..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

Header: x509sigver.h

qsc_x509_qsc_crl_signature_verify

Purpose: Verify a CRL signature using the issuer certificate and QSC back-end.

Details: Verifies the signature on the supplied certificate revocation list using the issuer certificate subject public key information and the CRL signature algorithm metadata.

Verification uses the preserved TBSCertList DER bytes carried by the decoded CRL object and rejects inconsistent algorithm pairings before invoking the cryptographic back-end.

bool qsc_x509_qsc_crl_signature_verify(const qsc_x509_crl* crl, const qsc_x509_certificate* issuer, void* state);

Parameters: crl: [const][struct] The certificate revocation list whose signature is to be verified.; issuer: [const][struct] The issuer certificate providing the verification public key.; state: Caller-supplied verification state, typically a qsc_x509_verify_state object..

Return: Returns true if the CRL signature is valid; otherwise returns false.

qsc_x509_qsc_csr_signature_verify

Purpose: Verify a CSR signature using the CSR subject key and QSC back-end.

Details: Verifies the signature on the supplied certificate signing request using the subject public key information carried inside the CSR and the declared CSR signature algorithm. Verification uses the preserved CertificationRequestInfo DER bytes carried by the decoded CSR object and does not reconstruct the signed region from normalized fields.

bool qsc_x509_qsc_csr_signature_verify(const qsc_x509_csr* csr, void* state);

Parameters: csr: [const][struct] The certificate signing request whose signature is to be verified.; state: Caller-supplied verification state, typically a qsc_x509_verify_state object..

Return: Returns true if the CSR signature is valid; otherwise returns false.

qsc_x509_qsc_signature_verify

Purpose: Verify a certificate signature using the issuer certificate and QSC back-end.

Details: Verifies the signature on the supplied certificate using the issuer certificate subject public key information and the signature algorithm declared by the certificate. Verification uses the preserved TBSCertificate DER bytes carried by the decoded certificate object and rejects inconsistent algorithm pairings before invoking the cryptographic back-end.

bool qsc_x509_qsc_signature_verify(const qsc_x509_certificate* certificate, const qsc_x509_certificate* issuer, void* state);

Parameters: certificate: [const][struct] The certificate whose signature is to be verified.; issuer: [const][struct] The issuer certificate providing the verification public key.; state: Caller-supplied verification state, typically a qsc_x509_verify_state object..

Return: Returns true if the certificate signature is valid; otherwise returns false.

qsc_x509_qsc_verify_signed_data

Purpose: Verify a generic signed-data object using explicit signature metadata.

Details: Verifies a raw signed-data buffer using the supplied signature bytes, signature algorithm selector, and signer subject public key information. This helper provides the algorithm-dispatch core used by the higher-level certificate, CRL, and CSR verification wrappers. The object-specific wrappers enforce signed-region preservation and AlgorithmIdentifier consistency before delegating to this routine.

bool qsc_x509_qsc_verify_signed_data(const uint8_t* data, size_t datalen, const uint8_t* signature, size_t signaturelen, uint8_t unusedbits, qsc_x509_signature_algorithm signaturealgorithm, const qsc_x509_subject_public_key_info* spki, void* state);

Parameters: data: [const] The signed-data message buffer.; datalen: The length of the signed-data buffer in bytes.; signature: [const] The signature bytes to verify.; signaturelen: The length of the signature in bytes.; unusedbits: The number of unused bits in the final signature octet when the signature originated from an ASN.1 BIT STRING.; signaturealgorithm: [enum] The normalized signature algorithm identifier.; spki: [const][struct] The signer subject public key information used for verification.; state: Caller-supplied verification state, typically a qsc_x509_verify_state object..

Return: Returns true if the signature is valid for the supplied data and public key; otherwise returns false.

qsc_x509_qsc_verify_state_initialize

Purpose: Initialize a QSC X.509 verification state object.

Details: Associates the verification state with a caller-managed scratch buffer and records the available buffer capacity. This function shall be called before using the state object with the QSC-backed verification helpers.

void qsc_x509_qsc_verify_state_initialize(qsc_x509_verify_state* state, uint8_t* buffer, size_t buflen);

Parameters: state: [struct] The verification state object to initialize.; buffer: The caller-supplied scratch buffer.; buflen: The capacity of the scratch buffer in bytes..

Return: None (void).

Header: x509spki.h

qsc_x509_algorithm_identifier_decode

Purpose: Decode an AlgorithmIdentifier object.

Details: Parses an ASN.1 DER encoded AlgorithmIdentifier sequence and populates the normalized qsc_x509_algorithm_identifier structure.

qsc_asn1_status qsc_x509_algorithm_identifier_decode(const qsc_encoding_ber_element* element, qsc_x509_algorithm_identifier* algorithm);

Parameters: element: [const][struct] The ASN.1 element containing the AlgorithmIdentifier sequence.; algorithm: [struct] The destination algorithm identifier object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_algorithm_identifier_initialize

Purpose: Initialize an AlgorithmIdentifier object.

Details: Resets the supplied algorithm identifier object to a clean default state.

void qsc_x509_algorithm_identifier_initialize(qsc_x509_algorithm_identifier* algorithm);

Parameters: algorithm: [struct] The algorithm identifier object to initialize..

Return: None (void).

qsc_x509_algorithm_identifier_initialize_mldsa

Purpose: Initialize an AlgorithmIdentifier for ML-DSA.

Details: Populates the supplied algorithm identifier object with the OID and parameter-set metadata corresponding to the selected ML-DSA parameter set.

qsc_asn1_status qsc_x509_algorithm_identifier_initialize_mldsa(qsc_x509_algorithm_identifier* algorithm, qsc_x509_pqc_parameter_set parameterset);

Parameters: algorithm: [struct] The algorithm identifier object to initialize.; parameterset: [enum] The ML-DSA parameter-set identifier..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_algorithm_identifier_initialize_mlkem

Purpose: Initialize an AlgorithmIdentifier for ML-KEM.

Details: Populates the supplied algorithm identifier object with the OID and parameter-set metadata corresponding to the selected ML-KEM parameter set.

qsc_asn1_status
qsc_x509_algorithm_identifier_initialize_mlkem(qsc_x509_algorithm_identifier* algorithm, qsc_x509_pqc_parameter_set parameterset);

Parameters: algorithm: [struct] The algorithm identifier object to initialize.; parameterset: [enum] The ML-KEM parameter-set identifier..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_algorithm_identifier_is_mldsa

Purpose: Test whether an AlgorithmIdentifier denotes ML-DSA.

Details: Examines the object identifier and parameter-set metadata in the supplied algorithm identifier and reports whether it represents an ML-DSA algorithm.

bool qsc_x509_algorithm_identifier_is_mldsa(const qsc_x509_algorithm_identifier* algorithm);

Parameters: algorithm: [const][struct] The algorithm identifier to inspect..

Return: Returns true if the algorithm identifier represents ML-DSA; otherwise returns false.

qsc_x509_algorithm_identifier_is_mlkem

Purpose: Test whether an AlgorithmIdentifier denotes ML-KEM.

Details: Examines the object identifier and parameter-set metadata in the supplied algorithm identifier and reports whether it represents an ML-KEM algorithm.

bool qsc_x509_algorithm_identifier_is_mlkem(const qsc_x509_algorithm_identifier* algorithm);

Parameters: algorithm: [const][struct] The algorithm identifier to inspect..

Return: Returns true if the algorithm identifier represents ML-KEM; otherwise returns false.

qsc_x509_algorithm_identifier_validate

Purpose: brief Validate a normalized AlgorithmIdentifier object. Performs strict OID-driven validation of an AlgorithmIdentifier, including parameter presence or absence rules, named-curve consistency, and ML-DSA or ML-KEM parameter-set consistency. This helper is intended for callers that need to validate decoded algorithm metadata independently of a full SPKI decode.

Details: brief Validate a normalized AlgorithmIdentifier object. Performs strict OID-driven validation of an AlgorithmIdentifier, including parameter presence or absence rules, named-curve consistency, and ML-DSA or ML-KEM parameter-set consistency. This helper is intended for callers that need to validate decoded algorithm metadata independently of a full SPKI decode.

qsc_asn1_status qsc_x509_algorithm_identifier_validate(const qsc_x509_algorithm_identifier* algorithm);

Parameters: algorithm: [const][struct] The algorithm identifier object to validate..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_named_curve_coordinate_size

Purpose: Get the coordinate size for a named elliptic curve.

Details: Returns the coordinate width in bytes for the supplied named curve. This helper is used when validating or extracting affine coordinates from EC public keys.

size_t qsc_x509_named_curve_coordinate_size(qsc_x509_named_curve curve);

Parameters: curve: [enum] The named elliptic curve identifier..

Return: Returns the coordinate size in bytes, or zero if the curve is unsupported.

qsc_x509_named_curve_public_key_size

Purpose: Get the encoded public-key size for a named elliptic curve.

Details: Returns the expected byte length of an uncompressed EC public key for the supplied named curve.

size_t qsc_x509_named_curve_public_key_size(qsc_x509_named_curve curve);

Parameters: curve: [enum] The named elliptic curve identifier..

Return: Returns the encoded public-key size in bytes, or zero if the curve is unsupported.

qsc_x509_pqc_public_key_size

Purpose: Get the expected public-key size for a PQC parameter set.

Details: Returns the implementation-defined public-key size associated with the supplied ML-DSA or ML-KEM parameter set.

size_t qsc_x509_pqc_public_key_size(qsc_x509_pqc_parameter_set parameterset);

Parameters: parameterset: [enum] The post-quantum parameter-set identifier..

Return: Returns the expected public-key size in bytes, or zero if the parameter set is unsupported.

qsc_x509_spki_get_ec_coordinates

Purpose: Extract affine EC coordinates from an SPKI object.

Details: Reads the uncompressed elliptic-curve public-key payload from the supplied SPKI object and writes the affine x and y coordinates to the caller-supplied output buffers.

qsc_asn1_status qsc_x509_spki_get_ec_coordinates(const qsc_x509_subject_public_key_info* spki, uint8_t* x, size_t xlen, uint8_t* y, size_t ylen);

Parameters: spki: [const][struct] The subject public key information object containing the EC public key.; x: The destination buffer receiving the x-coordinate bytes.; xlen: The capacity of the x-coordinate buffer in bytes.; y: The destination buffer receiving the y-coordinate bytes.; ylen: The capacity of the y-coordinate buffer in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_spki_initialize_ec

Purpose: Initialize an SPKI object for an elliptic-curve public key.

Details: Populates the supplied SubjectPublicKeyInfo object using the selected named curve and the supplied EC public-key bytes.

qsc_asn1_status qsc_x509_spki_initialize_ec(qsc_x509_subject_public_key_info* spki, qsc_x509_named_curve curve, const uint8_t* publickey, size_t publickeylen);

Parameters: spki: [struct] The destination subject public key information object.; curve: [enum] The named elliptic curve identifier.; publickey: [const] The raw encoded EC public-key bytes.; publickeylen: The length of the public key in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_spki_initialize_ml_dsa

Purpose: Initialize an SPKI object for an ML-DSA public key.

Details: Populates the supplied SubjectPublicKeyInfo object using the selected ML-DSA parameter set and the supplied public-key bytes. This helper accepts only the ML-DSA parameter set supported by the current build.

qsc_asn1_status qsc_x509_spki_initialize_ml_dsa(qsc_x509_subject_public_key_info* spki, qsc_x509_pqc_parameter_set parameterset, const uint8_t* publickey, size_t publickeylen);

Parameters: spki: [struct] The destination subject public key information object.; parameterset: [enum] The ML-DSA parameter-set identifier.; publickey: [const] The raw ML-DSA public-key bytes.; publickeylen: The length of the public key in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_spki_initialize_ml_kem

Purpose: Initialize an SPKI object for an ML-KEM public key.

Details: Populates the supplied SubjectPublicKeyInfo object using the selected ML-KEM parameter set and the supplied public-key bytes. This helper accepts only the ML-KEM parameter set supported by the current build.

qsc_asn1_status qsc_x509_spki_initialize_ml_kem(qsc_x509_subject_public_key_info* spki, qsc_x509_pqc_parameter_set parameterset, const uint8_t* publickey, size_t publickeylen);

Parameters: spki: [struct] The destination subject public key information object.; parameterset: [enum] The ML-KEM parameter-set identifier.; publickey: [const] The raw ML-KEM public-key bytes.; publickeylen: The length of the public key in bytes..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_spki_is_uncompressed_ec_point

Purpose: Test whether an SPKI contains an uncompressed EC point.

Details: Evaluates the subject public key payload and associated algorithm metadata to determine whether the encoded elliptic-curve public key uses the standard uncompressed point format.

bool qsc_x509_spki_is_uncompressed_ec_point(const qsc_x509_subject_public_key_info* spki);

Parameters: spki: [const][struct] The subject public key information object to inspect..

Return: Returns true if the SPKI contains an uncompressed EC point; otherwise returns false.

qsc_x509_spki_validate

Purpose: Validate a normalized SubjectPublicKeyInfo object.

Details: Performs structural and algorithm-specific validation of the supplied SPKI object, including parameter-set and public-key size consistency checks where applicable.

qsc_asn1_status qsc_x509_spki_validate(const qsc_x509_subject_public_key_info* spki);

Parameters: spki: [const][struct] The subject public key information object to validate..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_subject_public_key_info_decode

Purpose: Decode a SubjectPublicKeyInfo object.

Details: Parses an ASN.1 DER encoded SubjectPublicKeyInfo sequence and populates the normalized qsc_x509_subject_public_key_info structure with the decoded algorithm identifier and subject public key bytes.

qsc_asn1_status qsc_x509_subject_public_key_info_decode(const qsc_encoding_ber_element* element, qsc_x509_subject_public_key_info* spki);

Parameters: element: [const][struct] The ASN.1 element containing the SubjectPublicKeyInfo sequence.; spki: [struct] The destination subject public key information object..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_subject_public_key_info_initialize

Purpose: Initialize a SubjectPublicKeyInfo object.

Details: Resets the supplied subject public key information object to a clean default state.

void qsc_x509_subject_public_key_info_initialize(qsc_x509_subject_public_key_info* spki);

Parameters: spki: [struct] The subject public key information object to initialize..

Return: None (void).

Header: x509store.h

qsc_x509_chain_build

Purpose: Build a certification chain from a leaf certificate to a trust anchor.

Details: Attempts to construct a certificate path beginning at the supplied leaf certificate, proceeding through the provided intermediate certificates, and terminating at a trusted anchor in the store. The resulting ordered path is written to the caller-supplied output certificate array and summarized in the destination chain object.

qsc_x509_verify_status qsc_x509_chain_build(const qsc_x509_certificate* leaf, const qsc_x509_certificate* intermediates, size_t intermediatecount, const qsc_x509_store* store, qsc_x509_certificate* output, size_t outputcount, qsc_x509_chain* chain);

Parameters: leaf: [const][struct] The leaf certificate from which path construction begins.; intermediates: [const][struct] The intermediate certificate array available for path building.; intermediatecount: The number of certificates in intermediates.; store: [const][struct] The trust store containing candidate trust anchors.; output: [struct] The destination certificate array receiving the constructed path.; outputcount: The number of certificate elements available in output.; chain: [struct] The destination chain descriptor object..

Return: [enum] Returns a qsc_x509_verify_status code describing the chain build result.

qsc_x509_store_add_anchor

Purpose: Add a trust anchor to a store from a certificate.

Details: Converts the supplied certificate into a trust-anchor representation and adds it to the store, subject to the specified anchor storage capacity. The caller indicates whether the certificate should be treated as self-signed for anchor construction purposes.

qsc_asn1_status qsc_x509_store_add_anchor(qsc_x509_store* store, const qsc_x509_certificate* certificate, bool selfsigned);

Parameters: store: [struct] The destination trust store.; certificate: [const][struct] The certificate to add as a trust anchor.; selfsigned: Indicates whether the certificate is self-signed..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_store_contains_anchor

Purpose: Test whether a store contains a certificate as a trust anchor.

Details: Compares the supplied certificate against the anchors present in the store and reports whether an equivalent anchor is already available.

bool qsc_x509_store_contains_anchor(const qsc_x509_store* store, const qsc_x509_certificate* certificate);

Parameters: store: [const][struct] The trust store to inspect.; certificate: [const][struct] The certificate to test..

Return: Returns true if the store contains a matching trust anchor; otherwise returns false.

qsc_x509_store_find_anchor_by_subject

Purpose: Find a trust anchor by subject name.

Details: Searches the store for a trust anchor whose subject distinguished name matches the supplied X.509 name object.

const qsc_x509_trust_anchor* qsc_x509_store_find_anchor_by_subject(const qsc_x509_store* store, const qsc_x509_name* subject);

Parameters: store: [const][struct] The trust store to search.; subject: [const][struct] The subject distinguished name to match..

Return: Returns a pointer to the matching trust anchor, or NULL if no match is found.

qsc_x509_store_find_anchor_by_subject_key_identifier

Purpose: Find a trust anchor by Subject Key Identifier.

Details: Searches the store for a trust anchor whose Subject Key Identifier matches the supplied key identifier byte string.

const qsc_x509_trust_anchor*
qsc_x509_store_find_anchor_by_subject_key_identifier(const qsc_x509_store* store, const uint8_t* keyidentifier, size_t keyidentifierlen);

Parameters: store: [const][struct] The trust store to search.; keyidentifier: [const] The Subject Key Identifier bytes to match.; keyidentifierlen: The length of the key identifier in bytes..

Return: Returns a pointer to the matching trust anchor, or NULL if no match is found.

qsc_x509_store_find_anchor_for_certificate

Purpose: Find a trust anchor applicable to a certificate.

Details: Searches the store for a trust anchor whose subject matches the certificate issuer. AuthorityKeyIdentifier selectors are used as a preference when they can be evaluated; issuer-name matching remains the fallback so that AKI/SKI metadata is not promoted into an independent path-validation requirement.

const qsc_x509_trust_anchor* qsc_x509_store_find_anchor_for_certificate(const qsc_x509_store* store, const qsc_x509_certificate* certificate);

Parameters: store: [const][struct] The trust store to search.; certificate: [const][struct] The certificate for which a matching anchor is sought..

Return: Returns a pointer to the matching trust anchor, or NULL if no suitable anchor is found.

qsc_x509_store_find_issuer

Purpose: Find an issuer certificate in the trust store.

Details: Searches the trust store for a certificate that can act as the issuer of the supplied certificate, typically by subject and authority key identifier matching.

const qsc_x509_certificate* qsc_x509_store_find_issuer(const qsc_x509_store* store, const qsc_x509_certificate* certificate);

Parameters: store: [const][struct] The trust store to search.; certificate: [const][struct] The certificate whose issuer is sought..

Return: Returns a pointer to the matching issuer certificate, or NULL if no issuer is found.

qsc_x509_store_initialize

Purpose: Initialize a certificate store.

Details: Initializes a certificate store with a caller-provided buffer. The store starts empty. Anchors must be added using qsc_x509_store_add_anchor().

void qsc_x509_store_initialize(qsc_x509_store* store, qsc_x509_trust_anchor* anchors, size_t capacity);

Parameters: store: [struct] Store instance.; anchors: [array] Caller-allocated array of anchor pointers.; capacity: [size_t] Number of entries the array can hold..

Return: None (void).

Header: x509time.h

qsc_x509_time_compare

Purpose: Compare two X.509 time values.

Details: Performs an ordered comparison of two normalized time values.

int32_t qsc_x509_time_compare(const qsc_x509_time* a, const qsc_x509_time* b);

Parameters: a: [const][struct] The first time value.; b: [const][struct] The second time value..

Return: Returns a negative value if a is earlier than b, zero if the values are equal, or a positive value if a is later than b.

qsc_x509_time_decode

Purpose: Decode an ASN.1 X.509 time element.

Details: Decodes an ASN.1 time element and normalizes the result into the supplied X.509 time object. The input element may represent either a UTCTime or a GeneralizedTime value.

bool qsc_x509_time_decode(qsc_x509_time* out, const qsc_encoding_ber_element* elem);

Parameters: out: [struct] The destination decoded time object.; elem: [const][struct] The ASN.1 element containing the encoded time value..

Return: Returns true if decoding completed successfully; otherwise returns false.

qsc_x509_time_is_valid

Purpose: Test whether a normalized X.509 time value is structurally valid.

Details: Checks that the supplied time object contains a valid normalized calendar and clock representation suitable for X.509 validity evaluation.

```
bool qsc_x509_time_is_valid(const qsc_x509_time* time);
```

Parameters: time: [const][struct] The time object to validate..

Return: Returns true if the time object is valid; otherwise returns false.

qsc_x509_time_parse_generalizedtime

Purpose: Parse a GeneralizedTime string.

Details: Parses a character string encoded in ASN.1 GeneralizedTime textual form and writes the normalized result to the supplied X.509 time object. The accepted form is exactly YYYYMMDDHHMMSSZ as required by DER.

```
bool qsc_x509_time_parse_generalizedtime(const char* s, size_t len, qsc_x509_time* out);
```

Parameters: s: [const] The input GeneralizedTime character buffer.; len: The length of the input buffer in bytes.; out: [struct] The destination parsed time object..

Return: Returns true if parsing completed successfully; otherwise returns false.

qsc_x509_time_parse_utctime

Purpose: Parse a UTCTime string.

Details: Parses a character string encoded in ASN.1 UTCTime textual form and writes the normalized result to the supplied X.509 time object. The accepted form is exactly YYMMDDHHMMSSZ as required by DER.

```
bool qsc_x509_time_parse_utctime(const char* s, size_t len, qsc_x509_time* out);
```

Parameters: s: [const] The input UTCTime character buffer.; len: The length of the input buffer in bytes.; out: [struct] The destination parsed time object..

Return: Returns true if parsing completed successfully; otherwise returns false.

qsc_x509_validity_decode

Purpose: Decode a certificate Validity sequence.

Details: Decodes an ASN.1 Validity sequence and writes the notBefore and notAfter values to the supplied X.509 validity structure. The function accepts only a two-element DER Validity sequence and rejects intervals where notBefore is later than notAfter.

qsc_asn1_status qsc_x509_validity_decode(qsc_x509_validity* validity, const qsc_encoding_ber_element* elem);

Parameters: validity: [struct] The destination validity object.; elem: [const][struct] The ASN.1 element containing the encoded Validity sequence..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_validity_is_valid

Purpose: Test whether a validity interval is current at a supplied time.

Details: Evaluates whether the supplied reference time falls within the certificate validity interval described by the notBefore and notAfter fields.

bool qsc_x509_validity_is_valid(const qsc_x509_validity* validity, const qsc_x509_time* tnow);

Parameters: validity: [const][struct] The certificate validity interval.; tnow: [const][struct] The reference time used for evaluation..

Return: Returns true if the reference time is within the validity interval; otherwise returns false.

Header: x509verify.h

qsc_x509_certificate_allows_client_auth

Purpose: Test whether a certificate allows TLS client authentication.

Details: Evaluates the certificate key usage and extended key usage constraints for TLS client-auth applicability.

bool qsc_x509_certificate_allows_client_auth(const qsc_x509_certificate* certificate);

Parameters: certificate: [const][struct] The certificate to inspect..

Return: Returns true if the certificate permits TLS client authentication; otherwise returns false.

qsc_x509_certificate_allows_server_auth

Purpose: Test whether a certificate allows TLS server authentication.

Details: Evaluates the certificate key usage and extended key usage constraints for TLS server-auth applicability.

bool qsc_x509_certificate_allows_server_auth(const qsc_x509_certificate* certificate);

Parameters: certificate: [const][struct] The certificate to inspect..

Return: Returns true if the certificate permits TLS server authentication; otherwise returns false.

qsc_x509_certificate_check_algorithms

Purpose: Check certificate algorithm consistency.

Details: Validates the internal consistency of certificate signature algorithm metadata, signature encoding constraints, and related algorithm fields.

qsc_x509_verify_status qsc_x509_certificate_check_algorithms(const qsc_x509_certificate* certificate);

Parameters: certificate: [const][struct] The certificate to inspect..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_certificate_check_hostname

Purpose: Check whether a certificate matches a hostname.

Details: Evaluates the certificate identity information against the supplied DNS host name.

qsc_x509_verify_status qsc_x509_certificate_check_hostname(const qsc_x509_certificate* certificate, const char* hostname);

Parameters: certificate: [const][struct] The certificate to inspect.; hostname: [const] The hostname to match..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_certificate_check_ip_address

Purpose: Check whether a certificate matches an IP address.

Details: Evaluates the certificate identity information against the supplied binary IP address.

qsc_x509_verify_status qsc_x509_certificate_check_ip_address(const qsc_x509_certificate* certificate, const uint8_t* address, size_t addresslen);

Parameters: certificate: [const][struct] The certificate to inspect.; address: [const] The binary IP address to match.; addresslen: The length of the IP address in bytes..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_certificate_check_issuer

Purpose: Check whether one certificate may issue another.

Details: Evaluates issuer-subject name relationships, CA status, path-length constraints, and related issuer policy requirements. Authority key identifiers are path-construction hints and are not treated as independent certificate-validity requirements by this function.

qsc_x509_verify_status qsc_x509_certificate_check_issuer(const qsc_x509_certificate* issuer, const qsc_x509_certificate* subject, size_t remainingdepth);

Parameters: issuer: [const][struct] The candidate issuer certificate.; subject: [const][struct] The candidate subject certificate.; remainingdepth: The remaining allowable certification-path depth..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_certificate_check_purpose

Purpose: Check certificate suitability for a requested purpose.

Details: Evaluates the certificate against the requested application purpose, including usage and purpose constraints where applicable.

qsc_x509_verify_status qsc_x509_certificate_check_purpose(const qsc_x509_certificate* certificate, qsc_x509_verify_purpose purpose);

Parameters: certificate: [const][struct] The certificate to inspect.; purpose: [enum] The requested verification purpose..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_certificate_check_structure

Purpose: Check RFC-aligned certificate structural invariants.

Details: Validates certificate-local structural rules that do not require issuer, trust-store, or time context. This includes version and extension compatibility, empty-subject handling, CA and key-usage coherence, and subject public-key algorithm suitability.

qsc_x509_verify_status qsc_x509_certificate_check_structure(const qsc_x509_certificate* certificate);

Parameters: certificate: [const][struct] The certificate to inspect..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_certificate_check_validity

Purpose: Check certificate validity at a supplied time.

Details: Evaluates the certificate notBefore and notAfter fields relative to the supplied evaluation time.

qsc_x509_verify_status qsc_x509_certificate_check_validity(const qsc_x509_certificate* certificate, const qsc_asn1_time* ascnow);

Parameters: certificate: [const][struct] The certificate to inspect.; ascnow: [const][struct] The evaluation time..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_certificate_is_ca

Purpose: Test whether a certificate is authorized to act as a CA.

Details: Evaluates the Basic Constraints and related policy indicators to determine whether the certificate may act as a certification authority.

bool qsc_x509_certificate_is_ca(const qsc_x509_certificate* certificate);

Parameters: certificate: [const][struct] The certificate to inspect..

Return: Returns true if the certificate is a CA certificate; otherwise returns false.

qsc_x509_certificate_is_self_issued

Purpose: Test whether a certificate is self-issued.

Details: Determines whether the certificate subject and issuer names are equivalent, indicating that the certificate is self-issued.

bool qsc_x509_certificate_is_self_issued(const qsc_x509_certificate* certificate);

Parameters: certificate: [const][struct] The certificate to inspect..

Return: Returns true if the certificate is self-issued; otherwise returns false.

qsc_x509_certificate_is_self_signed

Purpose: Test whether a certificate is self-signed.

Details: Determines whether the certificate is self-issued and whether its signature validates under its own subject public key through the caller-supplied verification callback.

bool qsc_x509_certificate_is_self_signed(const qsc_x509_certificate* certificate, qsc_x509_signature_verify_callback callback, void* state);

Parameters: certificate: [const][struct] The certificate to inspect.; callback: The caller-supplied signature verification callback.; state: Caller-defined opaque verification context..

Return: Returns true if the certificate is self-signed; otherwise returns false.

qsc_x509_certificate_verify

Purpose: Verify a certificate against its issuer.

Details: Performs core certificate validation, including algorithm checks, validity checks, issuer relationship validation, and cryptographic signature verification through the supplied callback.

qsc_x509_verify_status qsc_x509_certificate_verify(const qsc_x509_certificate* certificate, const qsc_x509_certificate* issuer, const qsc_asn1_time* now, qsc_x509_signature_verify_callback callback, void* state);

Parameters: certificate: [const][struct] The certificate to verify.; issuer: [const][struct] The issuer certificate.; now: [const][struct] The evaluation time.; callback: The caller-supplied signature verification callback.; state: Caller-defined opaque verification context..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_certificate_verify_ex

Purpose: Verify a certificate against its issuer using extended options.

Details: Performs certificate verification with additional policy controls, including requested purpose validation, optional revocation processing, and optional unsupported-critical-extension rejection.

qsc_x509_verify_status qsc_x509_certificate_verify_ex(const qsc_x509_certificate* certificate, const qsc_x509_certificate* issuer, const qsc_asn1_time* now, qsc_x509_signature_verify_callback callback, void* state, const qsc_x509_verify_options* options);

Parameters: certificate: [const][struct] The certificate to verify.; issuer: [const][struct] The issuer certificate.; now: [const][struct] The evaluation time.; callback: The caller-supplied signature verification callback.; state: Caller-defined opaque verification context.; options: [const][struct] Optional extended verification controls..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_chain_is_anchored

Purpose: Test whether a chain terminates at a trusted anchor.

Details: Determines whether the supplied certification chain is anchored in the provided trust store.

bool qsc_x509_chain_is_anchored(const qsc_x509_chain* chain, const qsc_x509_store* store);

Parameters: chain: [const][struct] The certification chain to inspect.; store: [const][struct] The trust store containing candidate anchors..

Return: Returns true if the chain is trust-anchored; otherwise returns false.

qsc_x509_chain_verify

Purpose: Verify a certification chain.

Details: Performs ordered validation of the certificates in the chain, checks that the path terminates at a trust anchor in the supplied store, and applies cryptographic signature verification through the caller-supplied callback.

qsc_x509_verify_status qsc_x509_chain_verify(const qsc_x509_chain* chain, const qsc_x509_store* store, const qsc_asn1_time* now, qsc_x509_signature_verify_callback callback, void* state);

Parameters: chain: [const][struct] The certification chain to verify.; store: [const][struct] The trust store containing candidate anchors.; now: [const][struct] The evaluation time.; callback: The caller-supplied signature verification callback.; state: Caller-defined opaque verification context..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_chain_verify_ex

Purpose: Verify a certification chain using extended options.

Details: Performs certification-path verification with additional policy controls, including requested purpose validation, optional revocation processing, and optional unsupported-critical-extension rejection.

qsc_x509_verify_status qsc_x509_chain_verify_ex(const qsc_x509_chain* chain, const qsc_x509_store* store, const qsc_asn1_time* now, qsc_x509_signature_verify_callback callback, void* state, const qsc_x509_verify_options* options);

Parameters: chain: [const][struct] The certification chain to verify.; store: [const][struct] The trust store containing candidate anchors.; now: [const][struct] The evaluation time.; callback: The caller-supplied signature verification callback.; state: Caller-defined opaque verification context.; options: [const][struct] Optional extended verification controls..

Return: [enum] Returns a qsc_x509_verify_status code.

qsc_x509_verify_options_initialize

Purpose: Initialize a verification options structure.

Details: Resets the verification options object to a clean default state suitable for subsequent policy configuration.

void qsc_x509_verify_options_initialize(qsc_x509_verify_options* options);

Parameters: options: [struct] The verification options structure to initialize..

Return: None (void).

Header: x509wrap.h

qsc_x509w_availability_string

Purpose: Convert an availability indicator to a constant display string.

const char* qsc_x509w_availability_string(qsc_x509w_availability availability);

Parameters: availability: The availability indicator..

Return: Returns a constant string describing the availability state.

qsc_x509w_certificate_chain_load_file

Purpose: Load a certificate chain from a file into caller-supplied certificate storage and a chain descriptor.

qsc_x509w_status qsc_x509w_certificate_chain_load_file(const char* path, qsc_x509_certificate* certificates, size_t certificatecount, qsc_x509_chain* chain);

Parameters: path: The input file path.; certificates: The caller-supplied certificate storage array.; certificatecount: The capacity of the certificate storage array.; chain: The destination chain descriptor..

Return: Returns the wrapper status code.

qsc_x509w_certificate_chain_load_memory

Purpose: Load a certificate chain from a memory buffer into caller-supplied certificate storage and a chain descriptor.

qsc_x509w_status qsc_x509w_certificate_chain_load_memory(const uint8_t* data, size_t datalen, qsc_x509_certificate* certificates, size_t certificatecount, qsc_x509_chain* chain);

Parameters: data: The input buffer.; datalen: The length of the input buffer in bytes.; certificates: The caller-supplied certificate storage array.; certificatecount: The capacity of the certificate storage array.; chain: The destination chain descriptor..

Return: Returns the wrapper status code.

qsc_x509w_certificate_check_role

Purpose: Evaluate whether a certificate is suitable for a requested role.

qsc_x509w_status qsc_x509w_certificate_check_role(const qsc_x509_certificate* certificate, qsc_x509w_certificate_role role, const char* hostname, qsc_x509w_result* result);

Parameters: certificate: The certificate to test.; role: The requested certificate role.; hostname: An optional hostname used for TLS server role evaluation.; result: The destination diagnostic result object..

Return: Returns the wrapper status code.

qsc_x509w_certificate_export_der

Purpose: Export a certificate as DER.

qsc_x509w_status qsc_x509w_certificate_export_der(const qsc_x509_certificate* certificate, uint8_t* output, size_t outputlen, size_t* written);

Parameters: certificate: The certificate to export.; output: The destination byte buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written..

Return: Returns the wrapper status code.

qsc_x509w_certificate_export_pem

Purpose: Export a certificate as PEM.

qsc_x509w_status qsc_x509w_certificate_export_pem(const qsc_x509_certificate* certificate, char* output, size_t outputlen, size_t* written);

Parameters: certificate: The certificate to export.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_certificate_issuer_string

Purpose: Format a certificate issuer distinguished name into a normalized string.

qsc_x509w_status qsc_x509w_certificate_issuer_string(const qsc_x509_certificate* certificate, char* output, size_t outputlen, size_t* written);

Parameters: certificate: The source certificate.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_certificate_load_memory

Purpose: Load a certificate from a memory buffer.

Details: DER input is decoded using the low-level borrowed-buffer model; the caller must keep the DER input alive while the certificate is in use. PEM input is decoded into owned DER storage and is released by qsc_x509_certificate_clear. APIs that retain a certificate, such as the wrapper trust store, make an independent owned copy before returning.

qsc_x509w_status qsc_x509w_certificate_load_memory(const uint8_t* data, size_t datalen, qsc_x509_certificate* certificate);

Parameters: data: The input buffer.; datalen: The length of the input buffer in bytes.; certificate: The destination certificate object..

Return: Returns the wrapper status code.

qsc_x509w_certificate_subject_common_name

Purpose: Extract the first common-name attribute from a certificate subject.

qsc_x509w_status qsc_x509w_certificate_subject_common_name(const qsc_x509_certificate* certificate, char* output, size_t outputlen, size_t* written);

Parameters: certificate: The source certificate.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_certificate_subject_dns_name

Purpose: Retrieve a DNS subjectAltName entry by index.

qsc_x509w_status qsc_x509w_certificate_subject_dns_name(const qsc_x509_certificate* certificate, size_t index, char* output, size_t outputlen, size_t* written);

Parameters: certificate: The source certificate.; index: The zero-based DNS name index.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_certificate_subject_dns_name_count

Purpose: Get the number of DNS subjectAltName entries in a certificate.

size_t qsc_x509w_certificate_subject_dns_name_count(const qsc_x509_certificate* certificate);

Parameters: certificate: The source certificate..

Return: Returns the number of DNS subjectAltName entries.

qsc_x509w_certificate_subject_string

Purpose: Format a certificate subject distinguished name into a normalized string.

qsc_x509w_status qsc_x509w_certificate_subject_string(const qsc_x509_certificate* certificate, char* output, size_t outputlen, size_t* written);

Parameters: certificate: The source certificate.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_crl_load_file

Purpose: Load a CRL from a file.

qsc_x509w_status qsc_x509w_crl_load_file(const char* path, qsc_x509_crl* crl);

Parameters: path: The input file path.; crl: The destination CRL object..

Return: Returns the wrapper status code.

qsc_x509w_crl_load_memory

Purpose: Load a CRL from a memory buffer.

qsc_x509w_status qsc_x509w_crl_load_memory(const uint8_t* data, size_t datalen, qsc_x509_crl* crl);

Parameters: data: The input buffer.; datalen: The length of the input buffer in bytes.; crl: The destination CRL object..

Return: Returns the wrapper status code.

qsc_x509w_csr_add_dns_name

Purpose: Add a DNS subjectAltName entry to a CSR.

qsc_x509w_status qsc_x509w_csr_add_dns_name(qsc_x509_csr* csr, const char* dnsname);

Parameters: csr: The CSR to modify.; dnsname: The DNS name to add..

Return: Returns the wrapper status code.

qsc_x509w_csr_export_der

Purpose: Export a CSR as DER.

qsc_x509w_status qsc_x509w_csr_export_der(const qsc_x509_csr* csr, uint8_t* output, size_t outputlen, size_t* written);

Parameters: csr: The CSR to export.; output: The destination byte buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written..

Return: Returns the wrapper status code.

qsc_x509w_csr_export_pem

Purpose: Export a CSR as PEM.

qsc_x509w_status qsc_x509w_csr_export_pem(const qsc_x509_csr* csr, char* output, size_t outputlen, size_t* written);

Parameters: csr: The CSR to export.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_csr_load_file

Purpose: Load a CSR from a file.

qsc_x509w_status qsc_x509w_csr_load_file(const char* path, qsc_x509_csr* csr);

Parameters: path: The input file path.; csr: The destination CSR object..

Return: Returns the wrapper status code.

qsc_x509w_csr_load_memory

Purpose: Load a CSR from a memory buffer.

qsc_x509w_status qsc_x509w_csr_load_memory(const uint8_t* data, size_t datalen, qsc_x509_csr* csr);

Parameters: data: The input buffer.; datalen: The length of the input buffer in bytes.; csr: The destination CSR object..

Return: Returns the wrapper status code.

qsc_x509w_csr_sign_der

Purpose: Sign a CSR and export the result as DER.

qsc_x509w_status qsc_x509w_csr_sign_der(const qsc_x509_csr* csr, qsc_x509_certificate_sign_callback signcallback, void* context, uint8_t* output, size_t* outputlen);

Parameters: csr: The CSR to sign.; signcallback: The caller-supplied signing callback.; context: The opaque callback context.; output: The destination DER buffer.; outputlen: On input, the output-buffer capacity; on output, the number of bytes written..

Return: Returns the wrapper status code.

qsc_x509w_csr_sign_pem

Purpose: Sign a CSR and export the result as PEM.

qsc_x509w_status qsc_x509w_csr_sign_pem(const qsc_x509_csr* csr, qsc_x509_certificate_sign_callback signcallback, void* context, char* output, size_t* outputlen);

Parameters: csr: The CSR to sign.; signcallback: The caller-supplied signing callback.; context: The opaque callback context.; output: The destination PEM buffer.; outputlen: On input, the output-buffer capacity; on output, the number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_csr_verify

Purpose: Verify a CSR signature and structure.

qsc_x509w_status qsc_x509w_csr_verify(const qsc_x509_csr* csr);

Parameters: csr: The CSR to verify..

Return: Returns the wrapper status code.

qsc_x509w_current_utc_time

Purpose: Acquire the current UTC time in X.509 time form.

qsc_x509w_status qsc_x509w_current_utc_time(qsc_x509_time* currenttime);

Parameters: currenttime: The destination time object..

Return: Returns the wrapper status code.

qsc_x509w_deployment_config_initialize

Purpose: Initialize a deployment configuration object.

void qsc_x509w_deployment_config_initialize(qsc_x509w_deployment_config* config);

Parameters: config: The configuration object to initialize..

Return: None (void).

qsc_x509w_locator_policy_string

Purpose: Convert a locator policy to a constant display string.

const char* qsc_x509w_locator_policy_string(qsc_x509w_locator_policy policy);

Parameters: policy: The locator policy..

Return: Returns a constant string describing the policy.

qsc_x509w_name_get_attribute_first

Purpose: Extract the first matching attribute value from a distinguished name.

qsc_x509w_status qsc_x509w_name_get_attribute_first(const qsc_x509_name* name, qsc_x509_name_attribute_type type, char* output, size_t outputlen, size_t* written);

Parameters: name: The name to search.; type: The requested attribute type.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_name_string

Purpose: Format a distinguished name into a normalized string.

qsc_x509w_status qsc_x509w_name_string(const qsc_x509_name* name, char* output, size_t outputlen, size_t* written);

Parameters: name: The name to format.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_private_key_export_pkcs8_der

Purpose: Export a private key as PKCS#8 DER.

qsc_x509w_status qsc_x509w_private_key_export_pkcs8_der(const qsc_x509_private_key* privatekey, bool includepublickey, uint8_t* output, size_t outputlen, size_t* written);

Parameters: privatekey: The private key to export.; includepublickey: Set true to include public-key data when supported.; output: The destination byte buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written..

Return: Returns the wrapper status code.

qsc_x509w_private_key_export_pkcs8_pem

Purpose: Export a private key as PKCS#8 PEM.

qsc_x509w_status qsc_x509w_private_key_export_pkcs8_pem(const qsc_x509_private_key* privatekey, bool includepublickey, char* output, size_t outputlen, size_t* written);

Parameters: privatekey: The private key to export.; includepublickey: Set true to include public-key data when supported.; output: The destination character buffer.; outputlen: The capacity of the destination buffer in bytes.; written: The optional number of bytes written excluding the terminator..

Return: Returns the wrapper status code.

qsc_x509w_private_key_load_file

Purpose: Load a private key from a file.

qsc_x509w_status qsc_x509w_private_key_load_file(const char* path, qsc_x509_private_key* privatekey);

Parameters: path: The input file path.; privatekey: The destination private-key object..

Return: Returns the wrapper status code.

qsc_x509w_private_key_load_memory

Purpose: Load a private key from a memory buffer.

qsc_x509w_status qsc_x509w_private_key_load_memory(const uint8_t* data, size_t datalen, qsc_x509_private_key* privatekey);

Parameters: data: The input buffer.; datalen: The length of the input buffer in bytes.; privatekey: The destination private-key object..

Return: Returns the wrapper status code.

qsc_x509w_profile_apply_preset

Purpose: Apply a predefined validation-profile preset.

void qsc_x509w_profile_apply_preset(qsc_x509w_profile* profile, qsc_x509w_profile_preset preset);

Parameters: profile: The profile to update.; preset: The preset to apply..

Return: None (void).

qsc_x509w_profile_set_ca_defaults

Purpose: Set CA-validation defaults in a profile.

void qsc_x509w_profile_set_ca_defaults(qsc_x509w_profile* profile);

Parameters: profile: The profile to update..

Return: None (void).

qsc_x509w_profile_set_development_defaults

Purpose: Set relaxed development defaults in a profile.

void qsc_x509w_profile_set_development_defaults(qsc_x509w_profile* profile);

Parameters: profile: The profile to update..

Return: None (void).

qsc_x509w_profile_set_strict_revocation_defaults

Purpose: Set strict revocation defaults in a profile.

void qsc_x509w_profile_set_strict_revocation_defaults(qsc_x509w_profile* profile);

Parameters: profile: The profile to update..

Return: None (void).

qsc_x509w_profile_set_tls_client_defaults

Purpose: Set TLS client validation defaults in a profile.

void qsc_x509w_profile_set_tls_client_defaults(qsc_x509w_profile* profile);

Parameters: profile: The profile to update..

Return: None (void).

qsc_x509w_result_initialize

Purpose: Initialize a wrapper result object to its default state.

void qsc_x509w_result_initialize(qsc_x509w_result* result);

Parameters: result: The result object to initialize..

Return: None (void).

qsc_x509w_result_message

Purpose: Get the current diagnostic message stored in a wrapper result.

const char* qsc_x509w_result_message(const qsc_x509w_result* result);

Parameters: result: The source result object..

Return: Returns the fixed diagnostic message buffer.

qsc_x509w_revocation_mode_string

Purpose: Convert a revocation mode to a constant display string.

const char* qsc_x509w_revocation_mode_string(qsc_x509w_revocation_mode mode);

Parameters: mode: The revocation mode..

Return: Returns a constant string describing the mode.

qsc_x509w_revocation_source_string

Purpose: Convert a revocation source to a constant display string.

const char* qsc_x509w_revocation_source_string(qsc_x509w_revocation_source source);

Parameters: source: The revocation source..

Return: Returns a constant string describing the source.

qsc_x509w_server_identity_clear

Purpose: Clear a server identity and release its stored material.

void qsc_x509w_server_identity_clear(qsc_x509w_server_identity* identity);

Parameters: identity: The server identity to clear..

Return: None (void).

qsc_x509w_server_identity_get_chain

Purpose: Build a chain descriptor from a loaded server identity.

qsc_x509w_status qsc_x509w_server_identity_get_chain(const qsc_x509w_server_identity* identity, qsc_x509_chain* chain);

Parameters: identity: The source server identity.; chain: The destination chain descriptor..

Return: Returns the wrapper status code.

qsc_x509w_server_identity_initialize

Purpose: Initialize a server identity object.

void qsc_x509w_server_identity_initialize(qsc_x509w_server_identity* identity);

Parameters: identity: The server identity to initialize..

Return: None (void).

qsc_x509w_server_identity_load_configuration

Purpose: Load a server identity using a deployment configuration.

qsc_x509w_status
qsc_x509w_server_identity_load_configuration(qsc_x509w_server_identity* identity, const qsc_x509w_deployment_config* config, qsc_x509w_result* result);

Parameters: identity: The destination server identity.; config: The deployment configuration.; result: The optional diagnostic result object..

Return: Returns the wrapper status code.

qsc_x509w_server_identity_load_files

Purpose: Load a server identity from certificate-chain and private-key files.

qsc_x509w_status qsc_x509w_server_identity_load_files(qsc_x509w_server_identity* identity, const char* certificatechainpath, const char* privatekeypath);

Parameters: identity: The destination server identity.; certificatechainpath: The path to the certificate chain file.; privatekeypath: The path to the private-key file..

Return: Returns the wrapper status code.

qsc_x509w_server_identity_validate

Purpose: Validate a loaded server identity against a wrapper profile.

qsc_x509w_status qsc_x509w_server_identity_validate(const qsc_x509w_server_identity* identity, const qsc_x509w_profile* profile, qsc_x509w_result* result);

Parameters: identity: The server identity to validate.; profile: The validation profile.; result: The destination diagnostic result object..

Return: Returns the wrapper status code.

qsc_x509w_server_identity_verify

Purpose: Verify a loaded server identity against a trust store and profile.

qsc_x509w_status qsc_x509w_server_identity_verify(const qsc_x509w_server_identity* identity, const qsc_x509w_trust_store* store, const qsc_x509w_profile* profile, qsc_x509w_result* result);

Parameters: identity: The server identity to verify.; store: The trust store supplying anchors and CRLs.; profile: The validation profile.; result: The destination diagnostic result object..

Return: Returns the wrapper status code.

qsc_x509w_stage_string

Purpose: Convert a wrapper stage identifier to a constant display string.

const char* qsc_x509w_stage_string(qsc_x509w_stage stage);

Parameters: stage: The wrapper stage identifier..

Return: Returns a constant string describing the stage.

qsc_x509w_status_string

Purpose: Convert a wrapper status code to a constant display string.

const char* qsc_x509w_status_string(qsc_x509w_status status);

Parameters: status: The wrapper status code..

Return: Returns a constant string describing the status.

qsc_x509w_tls_bridge_get_interface

Purpose: Get the prepared TLS certificate interface from a configured bridge.

Details: The returned interface is a borrowed view into \a bridge and is intended as a configuration template. When the built-in QSC X.509 callbacks are installed in a TLS client or server, TLS initialization clones the validation context into that connection so mutable peer-summary, alert, and verification state are not shared across concurrent handshakes. The bridge itself retains only read-only trust/profile/CRL configuration for callback execution.

const qsc_tls_certificate_interface* qsc_x509w_tls_bridge_get_interface(const qsc_x509w_tls_bridge* bridge);

Parameters: bridge: The configured bridge..

Return: Returns a pointer to the prepared TLS certificate interface.

qsc_x509w_tls_bridge_initialize

Purpose: Initialize a TLS bridge object.

void qsc_x509w_tls_bridge_initialize(qsc_x509w_tls_bridge* bridge);

Parameters: bridge: The bridge object to initialize..

Return: None (void).

qsc_x509w_tls_bridge_is_ready

Purpose: Determine whether a TLS bridge has been configured and is ready for use.

bool qsc_x509w_tls_bridge_is_ready(const qsc_x509w_tls_bridge* bridge);

Parameters: bridge: The bridge to test..

Return: Returns true if the bridge is ready.

qsc_x509w_tls_local_certificate_from_identity

Purpose: Export a server identity into TLS local-certificate form. Extracts the certificate chain and private key from the identity object and populates a

qsc_x509w_tls_local_certificate ready for use by the TLS-layer server configuration local-certificate setter.

qsc_x509w_status qsc_x509w_tls_local_certificate_from_identity(const qsc_x509w_server_identity* identity, qsc_tls_signature_scheme verifyscheme, qsc_x509w_tls_local_certificate* localcert);

Parameters: identity: The source server identity (must contain a valid private key).; verifyscheme: The TLS CertificateVerify signature scheme identifier.; localcert: The destination TLS local-certificate export object..

Return: Returns the wrapper status code.

Notes: C6 fix: the previous API accepted a pre-computed signature; this function now extracts the private key directly from identity->privatekey so the signature can be produced at CertificateVerify build time.

qsc_x509w_tls_local_certificate_initialize

Purpose: Initialize a TLS local-certificate export object.

void qsc_x509w_tls_local_certificate_initialize(qsc_x509w_tls_local_certificate* localcert);

Parameters: localcert: The local certificate container to initialize..

Return: None (void).

qsc_x509w_tls_local_certificate_is_ready

Purpose: Determine whether a TLS local-certificate export object is ready for use.

bool qsc_x509w_tls_local_certificate_is_ready(const qsc_x509w_tls_local_certificate* localcert);

Parameters: localcert: The local-certificate export object to test..

Return: Returns true if the object is ready.

qsc_x509w_trust_store_add_anchor

Purpose: Add a certificate to the trust store as an anchor.

Details: The wrapper store retains an independent copy of the certificate state. When preserved DER is available, the store owns a private DER copy and does not depend on the lifetime of the source certificate or its input buffer.

qsc_x509w_status qsc_x509w_trust_store_add_anchor(qsc_x509w_trust_store* store, const qsc_x509_certificate* certificate, bool selfsigned);

Parameters: store: The destination trust store.; certificate: The certificate to add.; selfsigned: Set true when the anchor is expected to be self-signed..

Return: Returns the wrapper status code.

qsc_x509w_trust_store_add_anchor_bundle_file

Purpose: Decode and add one or more trust anchors from a file.

qsc_x509w_status qsc_x509w_trust_store_add_anchor_bundle_file(qsc_x509w_trust_store* store, const char* path, bool selfsigned);

Parameters: store: The destination trust store.; path: The input file path.; selfsigned: Set true when the loaded anchors are expected to be self-signed..

Return: Returns the wrapper status code.

qsc_x509w_trust_store_add_anchor_bundle_memory

Purpose: Decode and add one or more trust anchors from a memory buffer.

qsc_x509w_status
qsc_x509w_trust_store_add_anchor_bundle_memory(qsc_x509w_trust_store* store, const uint8_t* data, size_t datalen, bool selfsigned);

Parameters: store: The destination trust store.; data: The input buffer.; datalen: The length of the input buffer in bytes.; selfsigned: Set true when the loaded anchors are expected to be self-signed..

Return: Returns the wrapper status code.

qsc_x509w_trust_store_add_anchor_memory

Purpose: Load and add a single trust anchor from a memory buffer.

qsc_x509w_status qsc_x509w_trust_store_add_anchor_memory(qsc_x509w_trust_store* store, const uint8_t* data, size_t datalen, bool selfsigned);

Parameters: store: The destination trust store.; data: The input buffer.; datalen: The length of the input buffer in bytes.; selfsigned: Set true when the loaded anchor is expected to be self-signed..

Return: Returns the wrapper status code.

qsc_x509w_trust_store_add_crl

Purpose: Add a decoded CRL to the trust store.

Details: The wrapper store reparses the source CRL DER into independently owned backing storage. The source CRL may therefore be cleared after this function returns successfully.

qsc_x509w_status qsc_x509w_trust_store_add_crl(qsc_x509w_trust_store* store, const qsc_x509_crl* crl);

Parameters: store: The destination trust store.; crl: The CRL to add..

Return: Returns the wrapper status code.

qsc_x509w_trust_store_add_crl_file

Purpose: Load and add a CRL from a file.

qsc_x509w_status qsc_x509w_trust_store_add_crl_file(qsc_x509w_trust_store* store, const char* path);

Parameters: store: The destination trust store.; path: The input file path..

Return: Returns the wrapper status code.

qsc_x509w_trust_store_add_crl_memory

Purpose: Load and add a CRL from a memory buffer.

qsc_x509w_status qsc_x509w_trust_store_add_crl_memory(qsc_x509w_trust_store* store, const uint8_t* data, size_t datalen);

Parameters: store: The destination trust store.; data: The input buffer.; datalen: The length of the input buffer in bytes..

Return: Returns the wrapper status code.

qsc_x509w_trust_store_clear

Purpose: Clear a wrapper trust store and release its stored material.

void qsc_x509w_trust_store_clear(qsc_x509w_trust_store* store);

Parameters: store: The trust store to clear..

Return: None (void).

qsc_x509w_trust_store_load_configuration

Purpose: Load trust material using a deployment configuration.

qsc_x509w_status qsc_x509w_trust_store_load_configuration(qsc_x509w_trust_store* store, const qsc_x509w_deployment_config* config, qsc_x509w_result* result);

Parameters: store: The destination trust store.; config: The deployment configuration.; result: The optional diagnostic result object..

Return: Returns the wrapper status code.

qsc_x509w_verify_status_string

Purpose: Convert an underlying verification status to a constant display string.

const char* qsc_x509w_verify_status_string(qsc_x509_verify_status status);

Parameters: status: The underlying verification status..

Return: Returns a constant string describing the verification status.

Header: x509write.h

qsc_x509_compute_authority_key_identifier_from_issuer

Purpose: Computes the Authority Key Identifier (AKI) from an issuer certificate.

Details: This function derives an Authority Key Identifier structure from the provided issuer certificate. The AKI is constructed according to standard X.509 practices: - If the issuer certificate contains a Subject Key Identifier (SKI) extension, that value is used directly as the key identifier. - If no SKI is present, the key identifier is derived from the issuer's Subject Public Key Information (SPKI), typically using a SHA-1 hash over the public key bit string as specified in RFC 5280. The resulting structure is fully populated with the key identifier and marked as present. The caller may then pass the result to qsc_x509_crl_builder_set_authority_key_identifier() to include the extension in a CRL. This function does not modify the CRL builder directly.

qsc_asn1_status qsc_x509_compute_authority_key_identifier_from_issuer(const qsc_x509_certificate* issuer, qsc_x509_authority_key_identifier* authoritykeyidentifier);

Parameters: issuer: [const struct] Pointer to the issuer certificate.; authoritykeyidentifier: [struct] Pointer to the output AKI structure..

Return: Returns QSC_ASN1_STATUS_SUCCESS on success, QSC_ASN1_STATUS_INVALID_INPUT if parameters are invalid or QSC_ASN1_STATUS_FAILURE if AKI derivation fails.

qsc_x509_crl_builder_set_authority_key_identifier

Purpose: Sets the Authority Key Identifier (AKI) extension in the CRL builder.

Details: This function assigns a fully populated Authority Key Identifier structure to the CRL builder. The AKI extension identifies the public key corresponding to the private key used to sign the CRL, and is typically derived from the issuer certificate. When present, it enables relying parties to match the CRL to the correct issuer certificate during validation. The provided structure must be fully initialized by the caller. At minimum, the key identifier field should be present and contain the issuer's key identifier. Optional fields such as issuer name and serial number may also be included if required. If this function is not called, and no AKI is otherwise set, the CRL will be generated without an Authority Key Identifier extension.

qsc_asn1_status qsc_x509_crl_builder_set_authority_key_identifier(qsc_x509_crl_builder* builder, const qsc_x509_authority_key_identifier* authoritykeyidentifier);

Parameters: builder: [struct] Pointer to the CRL builder instance.; authoritykeyidentifier: [const struct] Pointer to a populated AKI structure..

Return: Returns QSC_ASN1_STATUS_SUCCESS on success or QSC_ASN1_STATUS_INVALID_INPUT if parameters are invalid.

qsc_x509_crl_builder_set_crl_number

Purpose: Sets the CRL Number extension in the CRL builder.

Details: This function assigns a CRL Number extension to the CRL builder. The CRL Number is a monotonically increasing integer used to identify and order CRLs issued by a given authority. It is encoded as a DER INTEGER within the extension value. The caller provides the CRL number as a big-endian byte array. The value is normalized during encoding to ensure proper ASN.1 INTEGER representation, including removal of leading zeros and insertion of a leading 0x00 byte if required to enforce a positive integer. The critical flag determines whether the extension is marked critical in the CRL. In most deployments, CRL Number is non-critical. If this function is not called, the CRL will be generated without a CRL Number extension.

qsc_asn1_status qsc_x509_crl_builder_set_crl_number(qsc_x509_crl_builder* builder, const uint8_t* value, size_t valuelen, bool critical);

Parameters: builder: [struct] Pointer to the CRL builder instance.; value: [const] Pointer to the big-endian CRL number byte array.; valuelen: [size_t] Length of the CRL number in bytes.; critical: [bool] Set to true to mark the extension as critical..

Return: Returns QSC_ASN1_STATUS_SUCCESS on success, QSC_ASN1_STATUS_INVALID_INPUT if parameters are invalid, QSC_ASN1_STATUS_OUT_OF_RANGE if the value length exceeds supported limits.

qsc_x509_write_algorithm_identifier

Purpose: Write a normalized AlgorithmIdentifier object.

qsc_asn1_status qsc_x509_write_algorithm_identifier(const qsc_x509_algorithm_identifier* algorithm, uint8_t* output, size_t* outputlen);

Parameters: algorithm: [const][struct] The algorithm identifier to encode.; output: The destination buffer receiving the DER AlgorithmIdentifier object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_algorithm_identifier_for_public_key

Purpose: Write an AlgorithmIdentifier object for a public-key algorithm.

Details: Encodes the canonical AlgorithmIdentifier corresponding to the supplied public-key algorithm selector and associated curve or PQC parameter set.

qsc_asn1_status qsc_x509_write_algorithm_identifier_for_public_key(qsc_x509_public_key_algorithm publickey, qsc_x509_named_curve curve, qsc_x509_pqc_parameter_set pqcparameter, uint8_t* output, size_t* outputlen);

Parameters: publickey: [enum] The public-key algorithm selector.; curve: [enum] The named elliptic curve when applicable.; pqcparameter: [enum] The post-quantum parameter set when applicable.; output: The destination buffer receiving the DER AlgorithmIdentifier object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_algorithm_identifier_for_signature

Purpose: Write an AlgorithmIdentifier object for a signature algorithm.

Details: Encodes the canonical AlgorithmIdentifier corresponding to the supplied normalized X.509 signature algorithm selector.

qsc_asn1_status
qsc_x509_write_algorithm_identifier_for_signature(qsc_x509_signature_algorithm signature, uint8_t* output, size_t* outputlen);

Parameters: signature: [enum] The signature algorithm selector.; output: The destination buffer receiving the DER AlgorithmIdentifier object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_authority_key_identifier

Purpose: Write an AuthorityKeyIdentifier extension payload.

qsc_asn1_status qsc_x509_write_authority_key_identifier(const qsc_x509_authority_key_identifier* authoritykeyidentifier, uint8_t* output, size_t* outputlen);

Parameters: authoritykeyidentifier: [const][struct] The Authority Key Identifier value to encode.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_basic_constraints

Purpose: Write a BasicConstraints extension payload.

qsc_asn1_status qsc_x509_write_basic_constraints(const qsc_x509_basic_constraints* basicconstraints, uint8_t* output, size_t* outputlen);

Parameters: basicconstraints: [const][struct] The Basic Constraints value to encode.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_bit_string

Purpose: Write an ASN.1 DER BIT STRING object.

Details: Encodes the supplied bit-string contents together with the number of unused bits in the final content octet.

qsc_asn1_status qsc_x509_write_bit_string(const uint8_t* value, size_t valuelen, uint8_t unusedbits, uint8_t* output, size_t* outputlen);

Parameters: value: [const] The bit-string contents.; valuelen: The length of the bit-string contents in bytes.; unusedbits: The number of unused bits in the final content octet.; output: The destination buffer receiving the DER BIT STRING object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_boolean

Purpose: Write an ASN.1 DER BOOLEAN object.

qsc_asn1_status qsc_x509_write_boolean(bool value, uint8_t* output, size_t* outputlen);

Parameters: value: The boolean value to encode.; output: The destination buffer receiving the DER BOOLEAN object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_explicit

Purpose: Write an explicitly tagged ASN.1 DER object from pre-encoded contents.

qsc_asn1_status qsc_x509_write_explicit(uint32_t tagnumber, const uint8_t* content, size_t contentlen, uint8_t* output, size_t* outputlen);

Parameters: tagnumber: The explicit context-specific tag number.; content: [const] The pre-encoded inner contents.; contentlen: The length of the inner contents in bytes.; output: The destination buffer receiving the explicitly tagged object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_extended_key_usage

Purpose: Write an ExtendedKeyUsage extension payload.

qsc_asn1_status qsc_x509_write_extended_key_usage(const qsc_x509_extended_key_usage* extendedkeyusage, uint8_t* output, size_t* outputlen);

Parameters: extendedkeyusage: [const][struct] The Extended Key Usage value to encode.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_extension

Purpose: Write a normalized X.509 Extension object.

qsc_asn1_status qsc_x509_write_extension(const qsc_x509_extension* extension, uint8_t* output, size_t* outputlen);

Parameters: extension: [const][struct] The extension object to encode.; output: The destination buffer receiving the DER Extension object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_extensions

Purpose: Write a normalized Extensions collection.

qsc_asn1_status qsc_x509_write_extensions(const qsc_x509_extensions* extensions, uint8_t* output, size_t* outputlen);

Parameters: extensions: [const][struct] The extension set to encode.; output: The destination buffer receiving the DER Extensions object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_general_name

Purpose: Write a normalized GeneralName object.

qsc_asn1_status qsc_x509_write_general_name(const qsc_x509_general_name* name, uint8_t* output, size_t* outputlen);

Parameters: name: [const][struct] The general name to encode.; output: The destination buffer receiving the DER GeneralName object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_generalized_time

Purpose: Write an ASN.1 DER GeneralizedTime object.

qsc_asn1_status qsc_x509_write_generalized_time(const qsc_asn1_time* value, uint8_t* output, size_t* outputlen);

Parameters: value: [const][struct] The time value to encode.; output: The destination buffer receiving the DER GeneralizedTime object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_ia5_string

Purpose: Write an ASN.1 DER IA5String object.

qsc_asn1_status qsc_x509_write_ia5_string(const char* value, size_t valuelen, uint8_t* output, size_t* outputlen);

Parameters: value: [const] The IA5String bytes.; valuelen: The length of the string in bytes.; output: The destination buffer receiving the DER IA5String object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_integer

Purpose: Write an ASN.1 DER INTEGER object.

Details: Encodes the supplied integer value bytes as a DER INTEGER object.

qsc_asn1_status qsc_x509_write_integer(const uint8_t* value, size_t valuelen, uint8_t* output, size_t* outputlen);

Parameters: value: [const] The integer value bytes in big-endian form.; valuelen: The length of the integer value in bytes.; output: The destination buffer receiving the DER INTEGER object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_issuer_alt_name

Purpose: Write an IssuerAltName extension payload.

qsc_asn1_status qsc_x509_write_issuer_alt_name(const qsc_x509_issuer_alt_name* issueraltname, uint8_t* output, size_t* outputlen);

Parameters: issueraltname: [const][struct] The Issuer Alternative Name value to encode.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_key_usage

Purpose: Write a KeyUsage extension payload.

qsc_asn1_status qsc_x509_write_key_usage(const qsc_x509_key_usage* keyusage, uint8_t* output, size_t* outputlen);

Parameters: keyusage: [const][struct] The Key Usage value to encode.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_length

Purpose: Write an ASN.1 DER length field.

Details: Encodes the supplied content length using DER definite-length form and writes the result to the output buffer.

qsc_asn1_status qsc_x509_write_length(size_t length, uint8_t* output, size_t* outputlen);

Parameters: length: The content length to encode.; output: The destination buffer receiving the encoded length field.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_name

Purpose: Write a normalized X.509 Name object.

qsc_asn1_status qsc_x509_write_name(const qsc_x509_name* name, uint8_t* output, size_t* outputlen);

Parameters: name: [const][struct] The distinguished name to encode.; output: The destination buffer receiving the DER Name object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_null

Purpose: Write an ASN.1 DER NULL object.

qsc_asn1_status qsc_x509_write_null(uint8_t* output, size_t* outputlen);

Parameters: output: The destination buffer receiving the DER NULL object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_octet_string

Purpose: Write an ASN.1 DER OCTET STRING object.

qsc_asn1_status qsc_x509_write_octet_string(const uint8_t* value, size_t valuelen, uint8_t* output, size_t* outputlen);

Parameters: value: [const] The octet-string contents.; valuelen: The length of the octet-string contents in bytes.; output: The destination buffer receiving the DER OCTET STRING object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_oid

Purpose: Write an ASN.1 DER OBJECT IDENTIFIER object.

qsc_asn1_status qsc_x509_write_oid(const qsc_asn1_oid* oid, uint8_t* output, size_t* outputlen);

Parameters: oid: [const][struct] The object identifier to encode.; output: The destination buffer receiving the DER OBJECT IDENTIFIER object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_printable_string

Purpose: Write an ASN.1 DER PrintableString object.

qsc_asn1_status qsc_x509_write_printable_string(const char* value, size_t valuelen, uint8_t* output, size_t* outputlen);

Parameters: value: [const] The PrintableString bytes.; valuelen: The length of the string in bytes.; output: The destination buffer receiving the DER PrintableString object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_raw

Purpose: Write a raw ASN.1 DER object from tag metadata and pre-encoded contents.

qsc_asn1_status qsc_x509_write_raw(uint8_t tagclass, bool constructed, uint32_t tagnumber, const uint8_t* content, size_t contentlen, uint8_t* output, size_t* outputlen);

Parameters: tagclass: The ASN.1 tag class.; constructed: Indicates whether the encoded object is constructed.; tagnumber: The ASN.1 tag number.; content: [const] The pre-encoded content bytes.; contentlen: The length of the content in bytes.; output: The destination buffer receiving the encoded object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_sequence

Purpose: Write an ASN.1 DER SEQUENCE object from pre-encoded contents.

qsc_asn1_status qsc_x509_write_sequence(const uint8_t* content, size_t contentlen, uint8_t* output, size_t* outputlen);

Parameters: content: [const] The pre-encoded content bytes.; contentlen: The length of the content in bytes.; output: The destination buffer receiving the DER SEQUENCE object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_set

Purpose: Write an ASN.1 DER SET object from pre-encoded contents.

qsc_asn1_status qsc_x509_write_set(const uint8_t* content, size_t contentlen, uint8_t* output, size_t* outputlen);

Parameters: content: [const] The pre-encoded content bytes.; contentlen: The length of the content in bytes.; output: The destination buffer receiving the DER SET object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_spki

Purpose: Write a normalized SubjectPublicKeyInfo object.

qsc_asn1_status qsc_x509_write_spki(const qsc_x509_subject_public_key_info* spki, uint8_t* output, size_t* outputlen);

Parameters: spki: [const][struct] The subject public key information object to encode.; output: The destination buffer receiving the DER SubjectPublicKeyInfo object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_spki_ec

Purpose: Write an elliptic-curve SubjectPublicKeyInfo object.

qsc_asn1_status qsc_x509_write_spki_ec(qsc_x509_named_curve curve, const uint8_t* publickey, size_t publickeylen, uint8_t* output, size_t* outputlen);

Parameters: curve: [enum] The named elliptic curve identifier.; publickey: [const] The encoded EC public-key bytes.; publickeylen: The length of the public key in bytes.; output: The destination buffer receiving the DER SubjectPublicKeyInfo object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_spki_ml_dsa

Purpose: Write an ML-DSA SubjectPublicKeyInfo object.

qsc_asn1_status qsc_x509_write_spki_ml_dsa(qsc_x509_pqc_parameter_set parameterset, const uint8_t* publickey, size_t publickeylen, uint8_t* output, size_t* outputlen);

Parameters: parameterset: [enum] The ML-DSA parameter-set identifier.; publickey: [const] The ML-DSA public-key bytes.; publickeylen: The length of the public key in bytes.; output: The destination buffer receiving the DER SubjectPublicKeyInfo object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_spki_ml_kem

Purpose: Write an ML-KEM SubjectPublicKeyInfo object.

qsc_asn1_status qsc_x509_write_spki_ml_kem(qsc_x509_pqc_parameter_set parameterset, const uint8_t* publickey, size_t publickeylen, uint8_t* output, size_t* outputlen);

Parameters: parameterset: [enum] The ML-KEM parameter-set identifier.; publickey: [const] The ML-KEM public-key bytes.; publickeylen: The length of the public key in bytes.; output: The destination buffer receiving the DER SubjectPublicKeyInfo object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_subject_alt_name

Purpose: Write a SubjectAltName extension payload.

qsc_asn1_status qsc_x509_write_subject_alt_name(const qsc_x509_subject_alt_name* subjectaltname, uint8_t* output, size_t* outputlen);

Parameters: subjectaltname: [const][struct] The Subject Alternative Name value to encode.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_subject_key_identifier

Purpose: Write a SubjectKeyIdentifier extension payload.

qsc_asn1_status qsc_x509_write_subject_key_identifier(const qsc_x509_subject_key_identifier* subjectkeyidentifier, uint8_t* output, size_t* outputlen);

Parameters: subjectkeyidentifier: [const][struct] The Subject Key Identifier value to encode.; output: The destination buffer receiving the DER payload.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_tag

Purpose: Write an ASN.1 DER tag field.

Details: Encodes the supplied tag class, constructed flag, and tag number as a DER tag field and writes the result to the output buffer.

qsc_asn1_status qsc_x509_write_tag(uint8_t tagclass, bool constructed, uint32_t tagnumber, uint8_t* output, size_t* outputlen);

Parameters: tagclass: The ASN.1 tag class.; constructed: Indicates whether the encoded object is constructed.; tagnumber: The ASN.1 tag number.; output: The destination buffer receiving the encoded tag field.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_utctime

Purpose: Write an ASN.1 DER UTCTime object.

qsc_asn1_status qsc_x509_write_utctime(const qsc_asn1_time* value, uint8_t* output, size_t* outputlen);

Parameters: value: [const][struct] The time value to encode.; output: The destination buffer receiving the DER UTCTime object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_utf8_string

Purpose: Write an ASN.1 DER UTF8String object.

qsc_asn1_status qsc_x509_write_utf8_string(const char* value, size_t valuelen, uint8_t* output, size_t* outputlen);

Parameters: value: [const] The UTF-8 string bytes.; valuelen: The length of the string in bytes.; output: The destination buffer receiving the DER UTF8String object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

qsc_x509_write_validity

Purpose: Write a normalized X.509 Validity object.

qsc_asn1_status qsc_x509_write_validity(const qsc_x509_validity* validity, uint8_t* output, size_t* outputlen);

Parameters: validity: [const][struct] The validity interval to encode.; output: The destination buffer receiving the DER Validity object.; outputlen: The input capacity of the output buffer and, on success, the number of bytes written..

Return: [enum] Returns a qsc_asn1_status code.

13: Networking and Sockets

13.1 IP Info

Header:

ipinfo.h

Description:

A set of network address functions, used by the TCP/IP sockets implementation.

Structures:

The qsc_ipinfo_ipv4_address structure contains the IPv4 address information.

Data Name	Data Type	Bit Length	Function
ipv4	Uint8 Array	32	IPv4 Address

Table 13.1a IPv4 address structure

The qsc_ipinfo_ipv6_address structure contains the IPv4 address information.

Data Name	Data Type	Bit Length	Function
Ipv6	Uint8 Array	128	IPv6 Address

Table 13.1b IPv6 address structure

The qsc_ipinfo_ipv4_info structure contains the IPv4 address information.

Data Name	Data Type	Bit Length	Function
address	IPv4 address	32	IPv4 Address
port	Uint16	16	Port number
mask	Uint8	8	Prefix mask

Table 13.1c IPv4 address info structure

The qsc_ipinfo_ipv6_info structure contains the IPv6 address information.

Data Name	Data Type	Bit Length	Function
address	IPv6 address	128	IPv4 Address
port	Uint16	16	Port number
mask	Uint8	0x08	Prefix mask

Table 13.1d IPv6 address info structure

Constants:

Constant Name	Value	Purpose
QSC_IPINFO_IPV4_BYTELEN	0x04	The ipv4 byte array length.
QSC_IPINFO_IPV4_MINLEN	0x08	The minimum ipv4 string length.
QSC_IPINFO_IPV4_STRNLEN	0x16	The ipv4 string length.
QSC_IPINFO_IPV6_BYTELEN	16	The ipv6 byte array length.
QSC_IPINFO_IPV6_STRNLEN	0x41	The ipv6 string length.

Table 13.1e address info constants

API:

Address Any

Returns an address structure set with the 'any' address.

```
qsc_ipinfo_ipv4_address qsc_ipinfo_ipv4_address_any(void);
```

```
qsc_ipinfo_ipv6_address qsc_ipinfo_ipv6_address_any(void);
```

Address Clear

Resets an address structure to zero.

```
void qsc_ipinfo_ipv4_address_clear(qsc_ipinfo_ipv4_address* address)
```

```
void qsc_ipinfo_ipv6_address_clear(qsc_ipinfo_ipv6_address* address)
```

Address From Array

Instantiate an IP address structure using a serialized 8-bit integer array.

qsc_ipinfo_ipv4_address qsc_ipinfo_ipv4_address_from_array(const uint8_t* address)

qsc_ipinfo_ipv6_address qsc_ipinfo_ipv6_address_from_array(const uint8_t* address)

Address From Bytes

Instantiate an ipv4 address structure using a set of 8-bit integers.

qsc_ipinfo_ipv4_address qsc_ipinfo_ipv4_address_from_bytes(uint8_t a1, uint8_t a2, uint8_t a3, uint8_t a4)

Address From String

Instantiate an IP address structure using a serialized address string.

qsc_ipinfo_ipv4_address qsc_ipinfo_ipv4_address_from_string(const char input[QSC_IPINFO_IPV4_STRNLEN])

qsc_ipinfo_ipv6_address qsc_ipinfo_ipv6_address_from_string(const char input[QSC_IPINFO_IPV6_STRNLEN])

Address Is Equal

Compares two IP address structures for equivalence.

bool qsc_ipinfo_ipv4_address_is_equal(const qsc_ipinfo_ipv4_address* a, const qsc_ipinfo_ipv4_address* b)

bool qsc_ipinfo_ipv6_address_is_equal(const qsc_ipinfo_ipv6_address* a, const qsc_ipinfo_ipv6_address* b)

Address Is Routable

Tests that the IP address is a valid publicly routable address.

bool qsc_ipinfo_ipv4_address_is_routable(const qsc_ipinfo_ipv4_address* address)

bool qsc_ipinfo_ipv6_address_is_routable(const qsc_ipinfo_ipv6_address* address)

Address Is Valid

Tests the IP address for validity and proper format.

bool qsc_ipinfo_ipv4_address_is_valid(const qsc_ipinfo_ipv4_address* address)

bool qsc_ipinfo_ipv6_address_is_valid(const qsc_ipinfo_ipv6_address* address)

Address Is Zeroed

Tests that the IP address is in an un-initialized state.

bool qsc_ipinfo_ipv4_address_is_zeroed(const qsc_ipinfo_ipv4_address* address)

bool qsc_ipinfo_ipv6_address_is_zeroed(const qsc_ipinfo_ipv6_address* address)

Address Loopback

Returns a copy of the IP loopback address.

qsc_ipinfo_ipv4_address qsc_ipinfo_ipv4_address_loopback(void);

qsc_ipinfo_ipv6_address qsc_ipinfo_ipv6_address_loopback(void);

Address To Array

Serializes an IP address structure to a byte array.

void qsc_ipinfo_ipv4_address_to_array(uint8_t* output, const qsc_ipinfo_ipv4_address* address)

void qsc_ipinfo_ipv6_address_to_array(uint8_t* output, const qsc_ipinfo_ipv6_address* address)

Address From String

Serializes an IP address structure to a string.

void qsc_ipinfo_ipv4_address_to_string(char output[QSC_IPINFO_IPV4_STRNLEN], const qsc_ipinfo_ipv4_address* address)

void qsc_ipinfo_ipv6_address_to_string(char output[QSC_IPINFO_IPV6_STRNLEN], const qsc_ipinfo_ipv6_address* address)

IPv6 Address Type

Returns the IPv6 address routing prefix-type.

qsc_ipv6_address_prefix_types qsc_ipinfo_ipv6_address_type(const qsc_ipinfo_ipv6_address* address)

13.2 Queue

Header:

queue.h

Description:

A set of queuing functions, used to queue packets in a network stream.

Structures:

The **qsc_queue_state** structure contains the queue state variables.

Data Name	Data Type	Bit Length	Function
queue	Uint8 2-D Array	Variable	The queue array
tags	Uint64	1600	The tag array
count	Uint64	64	The queue count
depth	Uint64	64	The queue depth
position	Uint64	64	The queue position

Table 13.2a queue state structure

Constants:

Constant Name	Value	Purpose
QSC_QUEUE_ALIGNMENT	64	The memory alignment constant.
QSC_QUEUE_MAX_DEPTH	64	The maximum queue depth.

Table 13.2b queue constants

API:

Destroy

Resets the queue state structure to zero.

Historical function name: qsc_queue_destroy. Revision 1.4 uses qsc_queue_dispose; refer to its source-exact declaration and contract in this chapter.

Flush

Flush the contents of the queue to an array.

void qsc_queue_flush(qsc_queue_state* ctx, uint8_t* output)

Initialize

Initialize the queue state.

void qsc_queue_initialize(qsc_queue_state* ctx, size_t depth, size_t width)

Items

Returns the number of items in the queue.

size_t qsc_queue_items(const qsc_queue_state* ctx)

IsFull

Returns the ‘full’ status from the queue.

Historical function name: qsc_queue_isfull. Revision 1.4 uses qsc_queue_full; refer to its source-exact declaration and contract in this chapter.

IsEmpty

Returns the 'empty' status from the queue.

Historical function name: qsc_queue_isempty. Revision 1.4 uses qsc_queue_empty; refer to its source-exact declaration and contract in this chapter.

Pop

Returns the first member of the queue, and erases that item from the queue.

```
uint64_t qsc_queue_pop(qsc_queue_state* ctx, uint8_t* output, size_t otplen);
```

Push

Add an item to the queue.

```
void qsc_queue_push(qsc_queue_state* ctx, const uint8_t* input, size_t inplen, uint64_t tag)
```

13.3 Socket

Headers:

socket.h, socketbase.h, socketflags.h

Description:

The TCP/IP network sockets functions.

Structures:

The qsc_socket structure contains the IP socket information.

Data Name	Data Type	Bit Length	Function
connection	Uint8	0x08	The packet flag
address	Int8	0x08	The address string
instance	Uint32	0x20	The instance number
port	Uint64	0x40	The port number
position	Uint64	0x40	The queue position
address_family	enum	0x08	The address family
connection_ status	enum	0x08	The connection state
socket_protocol	enum	0x08	The socket protocol
socket_transport	enum	0x08	The socket transport

Table 13.3a socket state structure

The qsc_socket_receive_async structure.

Data Name	Data Type	Bit Length	Function
callback	Pointer	0x40	The return callback
error	Pointer	0x40	The error callback
source	Socket	0x40	The socket pointer
buffer	Uint8 Array	0x800	The message buffer

Table 13.3b socket async receive structure

The qsc_receive_poll_state structure.

Data Name	Data Type	Bit Length	Function
sockarr	Pointer Array	0x40	The socket array
callback	Pointer	0x40	The return callback
error	Pointer	0x40	The error callback
count	Uint64	0x40	The active sockets

Table 13.3c socket receive poll structure

The qsc_socket_state structure.

Data Name	Data Type	Bit Length	Function
connection	Socket	0x40	The socket pointer
address	Int8	0x41	The address string
instance	Uint32	0x20	The instance number
port	Uint64	0x10	The port number
position	Uint64	0x40	The queue position
address_family	enum	0x08	The address family
connection_ status	enum	0x08	The connection state
socket_protocol	enum	0x08	The socket protocol
socket_transport	enum	0x08	The socket transport

Table 13.3d socket state structure

Constants:

Constant Name	Value	Purpose
QSC_UNINITIALIZED_SOCKET	-1	An unitialized socket handle.
QSC_SOCKET_ADDRESS_MAX_SIZE	0x41	The address string length.
QSC_SOCKET_MAX_CONN	0x7FFFFFFF	The maximum connections.
QSC_SOCKET_RET_ERROR	-1	Socket error sentinel.
QSC_SOCKET_RET_SUCCESS	0x00	The base socket success flag.
QSC_SOCKET_TERMINATOR_SIZE	0x01	The terminator size.
QSC_SOCKET_TIMEOUT_MSEC	10000	Default socket timeout in milliseconds.

QSC_UNINITIALIZED_SOCKET	-1	An unitialized socket handle.

Table 13.3e socket state constants

The qsc_ipv6_address_prefix_types enumeration.

Enumeration	Purpose
qsc_ipv6_prefix_none	No prefix is set.
qsc_ipv6_prefix_link_local	An link local address type, not globally routable.
qsc_ipv6_prefix_multicast	A qsc_ipv6_prefix_multicast address type, prefix.
qsc_ipv6_prefix_global	A globally routable address type, prefix.
qsc_ipv6_prefix_unique_local	A unique local address type, not globally routable, prefix.

Table 13.3f IPv6 prefix types enumeration

The qsc_socket_address_families enumeration.

Enumeration	Purpose
qsc_socket_address_family_none	No address family is specified AF_UNSPEC.
qsc_socket_address_family_unix	Unix local to host (pipes, portals) AF_UNIX.
qsc_socket_address_family_ipv4	The Internet Protocol 4 address family AF_INET.
qsc_socket_address_family_ipv6	The Internet Protocol 6 address family AF_INET6.

Table 13.3g address families enumeration

The qsc_socket_states enumeration.

Enumeration	Purpose
qsc_socket_address_family_none	No address family is specified AF_UNSPEC.
qsc_socket_address_family_unix	Unix local to host (pipes, portals) AF_UNIX.
qsc_socket_address_family_ipv4	The Internet Protocol 4 address family AF_INET.
qsc_socket_address_family_ipv6	The Internet Protocol 6 address family AF_INET6.

Table 13.3h socket states enumeration

The qsc_socket_options enumeration.

Enumeration	Purpose
qsc_socket_option_none	No flag is used.
qsc_socket_option_broadcast	Configures a socket for sending broadcast data SO_BROADCAST.
qsc_socket_option_ipv6_only	Flag used to enable a dual stack configuration IPV6_V6ONLY.
qsc_socket_option_keepalive	Enables sending keep-alive packets for a socket connection SO_KEEPALIVE.
qsc_socket_option_linger	Lingers on close if unsent data is present SO_LINGER.
qsc_socket_option_no_route	Sets whether outgoing data should be sent on interface the socket is bound to and not a routed on some other interface SO_DONTROUTE.
qsc_socket_option_out_of_band	Indicates that out-of-bound data should be returned in-line with regular data SO_OOBINLINE.
qsc_socket_option_reuse_address	Enables or disables the reuse of a bound address.
qsc_socket_option_receive_time_out	The timeout, in milliseconds, for blocking received calls SO_RCVTIMEO.
qsc_socket_option_send_time_out	The timeout, in milliseconds, for blocking send calls SO_SNDTIMEO.
qsc_socket_option_tcp_no_delay	Enables or disables the Nagle algorithm for TCP sockets. This option is disabled (set to FALSE) by default TCP_NODELAY.

Table 13.3i socket options enumeration

The qsc_socket_protocols enumeration.

Enumeration	Purpose
qsc_socket_protocol_none	No protocol type specified.
qsc_socket_protocol_ipv4	Internet Protocol version 4 IPPROTO_IPV4.

qsc_socket_protocol_socket	Enables or disables a socket level option.
qsc_socket_protocol_tcp	Transport Control Protocol IPPROTO_TCP.
qsc_socket_protocol_udp	Unreliable Delivery Protocol IPPROTO_UDP.
qsc_socket_protocol_ipv6	IPv6 header IPPROTO_IPV6.
qsc_socket_protocol_ipv6_routing	IPv6 Routing header IPPROTO_ROUTING.
qsc_socket_protocol_ipv6_fragment	IPv6 fragmentation header IPPROTO_FRAGMENT.
qsc_socket_protocol_icmpv6	ICMPv6 IPPROTO_ICMPV6.
qsc_socket_protocol_ipv6_no_header	IPv6 no next header IPPROTO_NONE.
qsc_socket_protocol_dstopts	IPv6 Destination options IPPROTO_DSTOPTS.
qsc_socket_protocol_raw	Raw Packet IPPROTO_RAW.

Table 13.3j socket protocols enumeration

The qsc_socket_receive_flags enumeration.

Enumeration	Purpose
qsc_socket_receive_flag_none	No flag is used.
qsc_socket_receive_flag_out_of_band	Process out of band data MSG_OOB.
qsc_socket_receive_flag_peek	Peeks at the incoming data MSG_PEEK.
qsc_socket_receive_flag_wait_all	Request completes only when buffer is full MSG_WAITALL.

Table 13.3k socket receive enumeration

The qsc_socket_send_flags enumeration.

Enumeration	Purpose
qsc_socket_send_flag_none	No flag is used.
qsc_socket_send_flag_send_oob	Sends OOB data on a stream type socket MSG_OOB.
qsc_socket_send_flag_peek_message	Sends a partial message.
qsc_socket_send_flag_no_routing	The data packets should not be routed MSG_DONTROUTE.

Table 13.3l socket send enumeration

The qsc_socket_shut_down_flags enumeration.

Enumeration	Purpose
qsc_socket_shut_down_flag_receive	Shut down the receiving channel QSC_SOCKET_SD_RECEIVE.
qsc_socket_shut_down_flag_send	Shut down the sending channel QSC_SOCKET_SD_SEND.
qsc_socket_shut_down_flag_both	Shut down both channels QSC_SOCKET_SD_BOTH.

Table 13.3m socket shut down enumeration

The qsc_socket_transports enumeration.

Enumeration	**Purpose**
qsc_socket_transport_none	No flag is used.
qsc_socket_transport_stream	Streaming connection SOCK_STREAM.
qsc_socket_transport_datagram	Datagram connection SOCK_DGRAM.
qsc_socket_transport_raw	TCP Raw socket SOCK_RAW.
qsc_socket_transport_reliable	Reliable protocol SOCK_RDM.
qsc_socket_transport_sequenced	Sequenced packets SOCK_SEQPACKET.

Table 13.3n socket transports enumeration

API:

Exception Callback

The socket exception callback prototype.

This historical callback declaration is not exported as a callable function. Use the active socket callback types defined by the public header.

Receive Async Callback

The socket asynchronous receive callback prototype.

This historical callback declaration is not exported as a callable function. Use the active socket callback types defined by the public header.

Receive Poll Callback

The receive polling callback prototype.

This historical callback declaration is not exported as a callable function. Use the active socket callback types defined by the public header.

IPv4 Valid Address

Detects if the string contains a valid IPv4 address.

```
bool qsc_socket_ipv4_valid_address(const char* address)
```

IPv6 Valid Address

Detects if the string contains a valid IPv6 address.

```
bool qsc_socket_ipv6_valid_address(const char* address)
```

Is Blocking

Determines if the socket is in blocking mode.

```
bool qsc_socket_is_blocking(const qsc_socket* sock)
```

Is Connected

Determines if the socket is connected.

```
bool qsc_socket_is_connected(const qsc_socket* sock)
```

Accept

Accept function handles an incoming connection attempt on the socket.

qsc_socket_exceptions qsc_socket_accept(const qsc_socket* source, qsc_socket* target)

Attach

Copy a socket to the target socket.

void qsc_socket_attach(qsc_socket* source, qsc_socket* target)

Bind

The Bind function associates an IP address with a socket.

qsc_socket_exceptions qsc_socket_bind(qsc_socket* sock, const char* address, uint16_t port)

qsc_socket_exceptions qsc_socket_bind_ipv4(qsc_socket* sock, const qsc_ipinfo_ipv4_address* address, uint16_t port)

qsc_socket_exceptions qsc_socket_bind_ipv6(qsc_socket* sock, const qsc_ipinfo_ipv6_address* address, uint16_t port)

Close

The Close function closes and disposes of the socket.

qsc_socket_exceptions qsc_socket_close_socket(qsc_socket* sock);

Connect

The Connect function establishes a connection to a remote host.

qsc_socket_exceptions qsc_socket_connect(qsc_socket* sock, const char* address, uint16_t port)

qsc_socket_exceptions qsc_socket_connect_ipv4(qsc_socket* sock, const qsc_ipinfo_ipv4_address* address, uint16_t port)

qsc_socket_exceptions qsc_socket_connect_ipv6(qsc_socket* sock, const qsc_ipinfo_ipv6_address* address, uint16_t port)

Create

The Create function creates a socket that is bound to a specific transport provider.

qsc_socket_exceptions qsc_socket_create(qsc_socket* sock, qsc_socket_address_families family, qsc_socket_transports transport, qsc_socket_protocols protocol)

Listen

Places the socket in the listening state, waiting for a connection.

qsc_socket_exceptions qsc_socket_listen(const qsc_socket* sock, int32_t backlog)

Receive

Receive data from a synchronous connected socket or a bound connectionless socket.

size_t qsc_socket_receive(const qsc_socket* sock, uint8_t* output, size_t otplen, qsc_socket_receive_flags flag);

Receive Async

Receive data from a connected socket asynchronously.

qsc_socket_exceptions qsc_socket_receive_async(qsc_socket_receive_async_state* state)

Receive All

Receive a block of data from a synchronous connected socket or a bound connectionless socket, and returns when the buffer is full.

size_t qsc_socket_receive_all(const qsc_socket* sock, uint8_t* output, size_t otplen, qsc_socket_receive_flags flag);

Receive From

Receive data from a synchronous connected socket or a bound connectionless socket.

size_t qsc_socket_receive_from(qsc_socket* sock, char* dest, size_t destlen, uint16_t port, uint8_t* output, size_t otplen, qsc_socket_receive_flags flag);

Receive Poll

Polls an array of sockets, and fires a callback if a socket is ready to receive data, or an error if socket is disconnected.

uint32_t qsc_socket_receive_poll(const qsc_socket_receive_poll_state* state)

Send

Sends data on a TCP connected socket.

size_t qsc_socket_send(const qsc_socket* sock, const uint8_t* input, size_t inplen, qsc_socket_send_flags flag);

Send To

Sends data on a UDP socket.

size_t qsc_socket_send_to(const qsc_socket* sock, const uint8_t* input, size_t inplen, qsc_socket_send_flags flag);

Send All

Sends a block of data larger than a single packet size, on a TCP socket and returns when sent.

size_t qsc_socket_send_all(const qsc_socket* sock, const uint8_t* input, size_t inplen, qsc_socket_send_flags flag);

Shut Down

Shuts down a socket.

qsc_socket_exceptions qsc_socket_shut_down(qsc_socket* sock, qsc_socket_shut_down_flags params);

Error To String

Returns the error string associated with the exception code.

```
const char* qsc_socket_error_to_string(qsc_socket_exceptions code)
```

Last Error

The last error generated by the internal socket library.

```
qsc_socket_exceptions qsc_socket_get_last_error(void);
```

IOCTL

Sets the IO mode of the socket.

```
qsc_socket_exceptions qsc_socket_ioctl(const qsc_socket* sock, int32_t command,
uint32_t* arguments)
```

Receive Ready

Tests the socket to see if it is ready to receive data.

```
bool qsc_socket_receive_ready(const qsc_socket* sock, const struct timeval* timeout)
```

Send Ready

Tests the socket to see if it is ready to send data.

```
bool qsc_socket_send_ready(const qsc_socket* sock, const struct timeval* timeout)
```

Set Last Error

Set the last error generated by the socket library.

```
void qsc_socket_set_last_error(qsc_socket_exceptions error)
```

Shut Down Sockets

Shut down the sockets library.

```
qsc_socket_exceptions qsc_socket_shut_down_sockets(void);
```

Socket Set Option

Send an option command to the socket.

```
qsc_socket_exceptions qsc_socket_set_option(const qsc_socket* sock,
qsc_socket_protocols level, qsc_socket_options option, int32_t optval)
```

Start Sockets

Start the sockets library.

```
bool qsc_socket_start_sockets(void);
```

13.4 Socket Client

Headers:

socketclient.h, socket.h, socketbase.h, socketflags.h

Description:

The socket client functions.

API:

Address Family

Returns the sockets address family, IPv4 or IPv6.

```
qsc_socket_address_families qsc_socket_client_address_family(const qsc_socket* sock)
```

Socket Protocol

Returns the socket protocol type.

qsc_socket_protocols qsc_socket_client_socket_protocol(const qsc_socket* sock)

Connect Host

Connect to a remote host using the network host name and service name.

qsc_socket_exceptions qsc_socket_client_connect_host(qsc_socket* sock, const char* host, const char* service)

qsc_socket_exceptions qsc_socket_client_connect_ipv4(qsc_socket* sock, const qsc_ipinfo_ipv4_address* address, uint16_t port)

qsc_socket_exceptions qsc_socket_client_connect_ipv6(qsc_socket* sock, const qsc_ipinfo_ipv6_address* address, uint16_t port)

Socket Transport

Get the socket transport type.

qsc_socket_transports qsc_socket_client_socket_transport(const qsc_socket* sock)

Client Initialize

Initialize the server socket.

void qsc_socket_client_initialize(qsc_socket* sock)

Client Receive

Receive data from a synchronous connected socket or a bound connectionless socket.

size_t qsc_socket_client_receive(const qsc_socket* sock, uint8_t* output, size_t otplen, qsc_socket_receive_flags flag);

Client Receive From

Receive UDP data from a remote host.

size_t qsc_socket_client_receive_from(qsc_socket* sock, char* address, size_t addlen, uint16_t port, uint8_t* output, size_t otplen, qsc_socket_receive_flags flag);

Client Send

Sends data on a connected socket.

size_t qsc_socket_client_send(const qsc_socket* sock, const uint8_t* input, size_t inplen, qsc_socket_send_flags flag);

Client Send To

Sends UDP data to a remote host.

size_t qsc_socket_client_send_to(const qsc_socket* sock, const uint8_t* input, size_t inplen, qsc_socket_send_flags flag);

Client Shut Down

Shut down the socket.

void qsc_socket_client_shut_down(qsc_socket* sock)

13.5 Socket Server

Headers:

socketserver.h, socket.h, socketbase.h, socketflags.h

Description:

The socket server function definitions.

Structures:

The qsc_socket_server_accept_result structure

Data Name	Data Type	Bit Length	Function

target	Socket	64	The socket pointer

Table 13.5a socket async receive structure

The qsc_socket_server_async_accept_state structure

Data Name	Data Type	Bit Length	Function
callback	Pointer	64	The return callback
error	Pointer	64	The error callback
source	Socket	64	The socket pointer

Table 13.5b socket async accept structure

API:

Accept Callback

The socket server accept callback prototype.

```
void qsc_socket_server_accept_callback(qsc_socket_server_accept_result* ares)
```

Error Callback

The socket server error callback prototype.

```
void qsc_socket_server_error_callback(const qsc_socket* source, qsc_socket_exceptions error);
```

Address Family

Returns the sockets address family, IPv4 or IPv6.

```
qsc_socket_address_families qsc_socket_server_address_family(const qsc_socket* sock)
```

Socket Protocol

Returns the socket protocol type.

```
qsc_socket_protocols qsc_socket_server_socket_protocol(const qsc_socket* sock)
```

Socket Transport

Returns the socket transport type.

qsc_socket_transports qsc_socket_server_socket_transport(const qsc_socket* sock)

Close Socket

Close the socket.

void qsc_socket_server_close_socket(qsc_socket* sock)

Initialize

Initialize the server socket.

void qsc_socket_server_initialize(qsc_socket* sock)

Listen

Places the source socket in a blocking listening state, and waits for a connection.

qsc_socket_exceptions qsc_socket_server_listen(qsc_socket* source, qsc_socket* target, const char* address, uint16_t port, qsc_socket_address_families family)

qsc_socket_exceptions qsc_socket_server_listen_ipv4(qsc_socket* source, qsc_socket* target, const qsc_ipinfo_ipv4_address* address, uint16_t port)

qsc_socket_exceptions qsc_socket_server_listen_ipv6(qsc_socket* source, qsc_socket* target, const qsc_ipinfo_ipv6_address* address, uint16_t port)

Listen Async

Places the socket in an asynchronous listening state.

qsc_socket_exceptions
qsc_socket_server_listen_async(qsc_socket_server_async_accept_state* state, const char* address, uint16_t port, qsc_socket_address_families family)

qsc_socket_exceptions
qsc_socket_server_listen_async_ipv4(qsc_socket_server_async_accept_state* state, const qsc_ipinfo_ipv4_address* address, uint16_t port)

qsc_socket_exceptions
qsc_socket_server_listen_async_ipv6(qsc_socket_server_async_accept_state* state, const qsc_ipinfo_ipv6_address* address, uint16_t port);

Set Option

Send an option command to the socket.

void qsc_socket_server_set_options(const qsc_socket* sock, qsc_socket_protocols level, qsc_socket_options option, int32_t optval)

Shut Down

Shut down the server.

void qsc_socket_server_shut_down(qsc_socket* sock)

13.6 Network Utilities

Header:

netutils.h

Description:

The network utilities functions, are used to access network services and information.

Structures:

The qsc_netutils_adaptor_info structure

Data Name	Data Type	Bit Length	Function

desc	char	0x420	The description string
dhcp	char	0x41	The dhcp string
gateway	char	32	The gateway string
ip	char	16	The port number
mac	char	64	The queue position
name	char	0x08	The address family
subnet	char	0x08	The connection state

Table 13.6a network adaptor info structure

Constants:

Constant Name	Value	Purpose
QSC_NETUTILS_ADAPTOR_NAME_SIZE	0x104	The network adaptors info string.
QSC_NETUTILS_ADAPTOR_DESCRIPTION_SIZE	0x84	The network adaptors description string.
QSC_NETUTILS_ADAPTOR_INFO_ARRAY_SIZE	0x08	The network adaptors info array size.
QSC_NETUTILS_IP_STRING_SIZE	0x80	The ip address string size.
QSC_NETUTILS_HOSTS_NAME_SIZE	0x104	The size of the hosts name buffer.
QSC_NETUTILS_MAC_ADDRESS_SIZE	0x12	MAC address string buffer capacity, in bytes.
QSC_NETUTILS_NAME_BUFFER_SIZE	0x80	The size of the protocol name buffer.
QSC_NETUTILS_SERVICE_NAME_BUFFER_SIZE	0x80	The size of the service name buffer.

Table 13.6b network utilities constants

API:

Get Adaptor Info

Retrieves the address information and the first addressable interface. Takes the netutils_adaptor_info structure as the parameter.

void qsc_netutils_get_adaptor_info(qsc_netutils_adaptor_info* info, const char* infname)

Get Adaptor Info Array

The current public API exposes single-adapter retrieval by interface name through qsc_netutils_get_adaptor_info().

Get Domain Name

Retrieves the hosts domain name. Takes a pointer to a char array as the parameter, and returns the domain name size.

size_t qsc_netutils_get_domain_name(char output[QSC_NETUTILS_DOMAIN_NAME_SIZE]);

Get IPv4 Address

Gets the first IPv4 addressable interface address. Returns the IPv4 address structure.

bool qsc_netutils_get_ipv4_address(qsc_ipinfo_ipv4_address* padd)

Get IPv6 Address

Gets the first IPv6 addressable interface address. Returns the IPv6 address structure.

bool qsc_netutils_get_ipv6_address(qsc_ipinfo_ipv6_address* padd)

Get IPv4 Info

Retrieves the IPv4 address information for a remote host. Takes the host name array, and the service name string as parameters, and returns an IPv4 address info structure.

void qsc_netutils_get_ipv4_info(qsc_ipinfo_ipv4_info* pinfo, const char* host, const char* service)

Get IPv6 Info

Retrieves the IPv6 address information for a remote host. Takes the host name array, and the service name string as parameters, and returns an IPv6 address info structure.

void qsc_netutils_get_ipv6_info(qsc_ipinfo_ipv6_info* pinfo, const char* host, const char* service)

Get MAC Address

Retrieves the MAC address of the first addressable interface. Takes the MAC address array as the parameter.

void qsc_netutils_get_mac_address(uint8_t mac[QSC_NETUTILS_MAC_ADDRESS_SIZE])

Get Peer Name

Retrieves the host name of the connected peer. Takes the output char array, and a pointer to the socket structure as parameters.

void qsc_netutils_get_peer_name(char output[QSC_NETUTILS_HOSTS_NAME_SIZE], const qsc_socket* sock);

Get Socket Name

Retrieves the socket name of the connected peer. Takes the output array and a pointer to the connected socket structure as parameters.

void qsc_netutils_get_socket_name(char output[QSC_NETUTILS_NAME_BUFFER_SIZE], const qsc_socket* sock);

Port Name To Number

Get the port number from the service name. Takes the port name, and the protocol as parameters, and returns the port number.

uint16_t qsc_netutils_port_name_to_number(const char* portname, const char* protocol);

13.7 Additional Networking APIs

Header: ipinfo.h

qsc_ipinfo_ipv4_address_get_cidr_mask

Purpose: Get the IPv4 network subnet CIDR length.

uint8_t qsc_ipinfo_ipv4_address_get_cidr_mask(const qsc_ipinfo_ipv4_address* address);

Parameters: address: [const qsc_ipinfo_ipv4_address*] A pointer to the IPv4 address structure..

Return: [uint8_t] Returns the mask length in bits.

qsc_ipinfo_ipv4_address_get_mask

Purpose: Get the IPv4 network subnet mask string.

void qsc_ipinfo_ipv4_address_get_mask(char mask[QSC_IPINFO_IPV4_MASK_STRNLEN], const qsc_ipinfo_ipv4_address* address);

Parameters: mask: [char*] The output mask string.; address: [const qsc_ipinfo_ipv4_address*] A pointer to the IPv4 address structure..

Return: None (void).

qsc_ipinfo_ipv4_address_string_is_valid

Purpose: Test if the IPv4 address string is valid.

bool qsc_ipinfo_ipv4_address_string_is_valid(const char* address);

Parameters: address: [const char*] A pointer to the IPv4 address string..

Return: [bool] Returns true if the address string is valid.

qsc_ipinfo_ipv4_array_to_string

Purpose: Convert an IPv4 address array to a string.

void qsc_ipinfo_ipv4_array_to_string(char output[QSC_IPINFO_IPV4_STRNLEN], const uint8_t* address);

Parameters: output: [char*] The serialized address string output array.; address: [const uint8_t*] A pointer to the IPv4 address array..

Return: None (void).

qsc_ipinfo_ipv4_cidr_to_mask

Purpose: Convert a CIDR mask to a subnet mask string.

void qsc_ipinfo_ipv4_cidr_to_mask(char mask[QSC_IPINFO_IPV4_MASK_STRNLEN], uint8_t cidr);

Parameters: mask: [char*] The output mask string.; cidr: [uint8_t] The input CIDR mask..

Return: None (void).

qsc_ipinfo_ipv4_mask_to_cidr

Purpose: Convert a subnet mask string to a CIDR mask.

uint8_t qsc_ipinfo_ipv4_mask_to_cidr(const char mask[QSC_IPINFO_IPV4_MASK_STRNLEN]);

Parameters: mask: [const char*] The subnet mask string..

Return: [uint8_t] Returns the mask length in bits.

qsc_ipinfo_ipv4_prefix_contains

Purpose: Check whether an IPv4 address belongs to a CIDR network prefix.

bool qsc_ipinfo_ipv4_prefix_contains(const qsc_ipinfo_ipv4_address* address, const qsc_ipinfo_ipv4_address* network, uint8_t prefix);

Parameters: address: Address to test (four bytes in network order).; network: Network address (host bits are ignored).; prefix: Prefix length in bits, 0 through 32..

Return: true for matching prefixes; false for invalid arguments or prefix.

Notes: No allocation or shared state; thread-safe.

qsc_ipinfo_ipv6_address_get_cidr_mask

Purpose: Get the IPv6 network subnet CIDR length.

uint8_t qsc_ipinfo_ipv6_address_get_cidr_mask(const qsc_ipinfo_ipv6_address* address);

Parameters: address: [const qsc_ipinfo_ipv6_address*] A pointer to the IPv6 address structure..

Return: [uint8_t] Returns the mask length in bits.

qsc_ipinfo_ipv6_address_get_mask

Purpose: Get the IPv6 network subnet mask string.

void qsc_ipinfo_ipv6_address_get_mask(char mask[QSC_IPINFO_IPV6_MASK_STRNLEN], const qsc_ipinfo_ipv6_address* address);

Parameters: mask: [char*] The output mask string.; address: [const qsc_ipinfo_ipv6_address*] A pointer to the IPv6 address structure..

Return: None (void).

qsc_ipinfo_ipv6_address_string_is_valid

Purpose: Test if the IPv6 address string is valid.

bool qsc_ipinfo_ipv6_address_string_is_valid(const char* address);

Parameters: address: [const char*] A pointer to the IPv6 address string..

Return: [bool] Returns true if the address string is valid.

qsc_ipinfo_ipv6_array_to_string

Purpose: Convert an IPv6 address array to a string.

void qsc_ipinfo_ipv6_array_to_string(char output[QSC_IPINFO_IPV6_STRNLEN], const uint8_t* address);

Parameters: output: [char*] The address string output array.; address: [const uint8_t*] A pointer to the IPv6 address array..

Return: None (void).

qsc_ipinfo_ipv6_prefix_contains

Purpose: Check whether an IPv6 address belongs to a CIDR network prefix.

bool qsc_ipinfo_ipv6_prefix_contains(const qsc_ipinfo_ipv6_address* address, const qsc_ipinfo_ipv6_address* network, uint8_t prefix);

Parameters: address: Address to test (16 bytes in network order).; network: Network address (host bits are ignored).; prefix: Prefix length in bits, 0 through 128..

Return: true for matching prefixes; false for invalid arguments or prefix.

Notes: No allocation or shared state; thread-safe.

Header: netutils.h

qsc_netutils_atoi

Purpose: Parse a string for a number

```
uint32_t qsc_netutils_atoi(const char* source);
```

Parameters: source: [const char*] The string to convert.

Return: [uint32_t] The number found in the string

qsc_netutils_get_host_name

Purpose: Retrieves the host name of the local machine

```
bool qsc_netutils_get_host_name(char host[QSC_NETUTILS_HOSTS_NAME_SIZE]);
```

Parameters: host: [char*] The host-name string.

Return: [bool] Returns true if the call succeeded

qsc_netutils_get_name_from_ipv4_address

Purpose: Retrieves fully qualified name from an IPv4 address

```
void qsc_netutils_get_name_from_ipv4_address(const qsc_ipinfo_ipv4_address* address,
char host[QSC_NETUTILS_HOSTS_NAME_SIZE]);
```

Parameters: address: [const qsc_ipinfo_ipv4_address*] The input IPv4 address string; host: [char*] The output host name.

Return: None (void).

qsc_netutils_values_print

Purpose: Print the output of network function calls

```
void qsc_netutils_values_print(void);
```

Return: None (void).

Header: socketbase.h

qsc_socket_clear_socket

Purpose: Erases the socket struture

void qsc_socket_clear_socket(qsc_socket* sock);

Parameters: sock: [qsc_socket*] The socket instance.

Return: None (void).

qsc_socket_event_loop_create

Purpose: Create a socket readiness loop using epoll (Linux), kqueue (BSD/macOS), or poll fallback.

Details: On success release with qsc_socket_event_loop_destroy(). Does not own sockets. Registration, waiting and destruction must not run concurrently; wakeup is thread-safe.

qsc_socket_event_loop* qsc_socket_event_loop_create(size_t capacity);

Parameters: capacity: Maximum registered socket count; must be nonzero and at most INT32_MAX - 1..

Return: Newly allocated loop, or NULL on allocation or operating-system failure.

qsc_socket_event_loop_destroy

Purpose: Destroy the loop and internal wake resources; never closes registered sockets.

Details: Call only after all active waiters and other users have stopped.

void qsc_socket_event_loop_destroy(qsc_socket_event_loop* loop);

Parameters: loop: Loop to free; NULL is allowed..

Return: None (void).

qsc_socket_event_loop_modify

Purpose: Replace the read/write interest mask for a previously registered socket.

bool qsc_socket_event_loop_modify(qsc_socket_event_loop* loop, const qsc_socket* sock, uint32_t events);

Parameters: loop: Initialized loop.; sock: Previously registered caller-owned socket.; events: New interest mask, read and/or write..

Return: true on success, false on invalid state or operating-system failure.

qsc_socket_event_loop_register

Purpose: Register a socket for level-triggered readiness events.

Details: An active socket must be unregistered before close or descriptor reuse.

bool qsc_socket_event_loop_register(qsc_socket_event_loop* loop, const qsc_socket* sock, uint32_t events);

Parameters: loop: Initialized loop, not NULL.; sock: Valid caller-owned socket, not NULL.; events: Interest flags, read and/or write only..

Return: true if registered, otherwise false (including duplicate registration or full loop).

qsc_socket_event_loop_unregister

Purpose: Remove a socket from the loop without closing it.

Details: Callers must synchronize this operation with waits and socket destruction.

bool qsc_socket_event_loop_unregister(qsc_socket_event_loop* loop, const qsc_socket* sock);

Parameters: loop: Initialized loop.; sock: Previously registered socket..

Return: true on success, false otherwise.

qsc_socket_event_loop_wait

Purpose: Collect available readiness notifications and cross-thread wake requests.

Details: Readiness is level triggered. Handle read/write without blocking until EAGAIN on nonblocking sockets. EOF may be reported as read readiness without the hangup flag. Wake events use QSC_UNINITIALIZED_SOCKET and the wakeup flag. A single waiter is supported per loop. Output remains valid until the caller overwrites it.

int32_t qsc_socket_event_loop_wait(qsc_socket_event_loop* loop, qsc_socket_event* output, size_t capacity, int32_t timeout_ms);

Parameters: loop: Initialized loop.; output: Caller-owned array of at least capacity events.; capacity: Number of writable output entries, from 1 through INT32_MAX - 1.; timeout_ms: Wait limit in milliseconds: -1 forever, 0 nonblocking, positive bounded wait..

Return: Number of events from 0 to capacity, or -1 on invalid input/operating-system failure.

qsc_socket_event_loop_wake

Purpose: Wake a blocked waiter from any thread without registering a socket.

Details: Consecutive wake requests may coalesce into a single notification.

bool qsc_socket_event_loop_wake(qsc_socket_event_loop* loop);

Parameters: loop: Initialized loop that has not begun destruction..

Return: true when a wake signal is queued; false on failure.

qsc_socket_get_option

Purpose: Get the value of a socket option.

Details: Retrieves the current value of a socket option from the operating system. Socket timeout options are returned in milliseconds.

qsc_socket_exceptions qsc_socket_get_option(const qsc_socket* sock, qsc_socket_protocols level, qsc_socket_options option, int32_t* optval);

Parameters: sock: [const] A pointer to the socket structure.; level: The socket protocol level.; option: The socket option to query.; optval: A pointer receiving the option value..

Return: Returns the socket exception status.

qsc_socket_iocp_attach

Purpose: Associate a caller-owned overlapped Winsock socket with a completion port.

Details: A socket cannot be reassociated with a different completion port.

bool qsc_socket_iocp_attach(qsc_socket_iocp* port, const qsc_socket* sock);

Parameters: port: Initialized completion-port context.; sock: Socket created for overlapped I/O; caller retains its lifetime..

Return: true on successful association; false on failure.

qsc_socket_iocp_cancel

Purpose: Request cancellation of pending I/O on an attached socket.

Details: Windows CancelIoEx cancels ALL outstanding I/O issued on this socket, including operations submitted outside this module. Each cancelled request still produces a completion that MUST be consumed before buffers or IOCP can be freed. Do not call concurrently with socket destruction.

bool qsc_socket_iocp_cancel(qsc_socket_iocp* port, const qsc_socket* sock);

Parameters: port: Valid IOCP context (used for API validation, not ownership).; sock: Valid socket associated with the port; caller retains ownership..

Return: true if cancellation was requested or no pending kernel requests remained.

qsc_socket_iocp_create

Purpose: Create a Windows IO completion port.

Details: Requires Winsock startup. Supports multiple simultaneous waiting consumer threads.

qsc_socket_iocp* qsc_socket_iocp_create(uint32_t max_pending);

Parameters: max_pending: Maximum simultaneous submitted operations, between 1 and LONG_MAX..

Return: Owned IOCP object, or NULL on failure; release with qsc_socket_iocp_destroy().

qsc_socket_iocp_destroy

Purpose: Destroy an idle IOCP; refuses to free live overlapped I/O requests.

Details: For outstanding requests, call qsc_socket_iocp_cancel() and drain ALL resulting completions before retrying. Callers must have stopped submissions and waiting threads. Never closes attached sockets. No automatic cancellation is performed.

bool qsc_socket_iocp_destroy(qsc_socket_iocp* port);

Parameters: port: Context to destroy, or NULL..

Return: true if destroyed or NULL, false if operations remain outstanding.

qsc_socket_iocp_submit_receive

Purpose: Submit a receive that completes through qsc_socket_iocp_wait().

Details: buffer and sock MUST remain alive until a completion is consumed, even if the submission completes immediately. No request is pending on a false return. Do not enable FILE_SKIP_COMPLETION_PORT_ON_SUCCESS on attached sockets.

bool qsc_socket_iocp_submit_receive(qsc_socket_iocp* port, const qsc_socket* sock, uint8_t* buffer, size_t length, uint64_t context);

Parameters: port: Initialized IOCP context.; sock: Previously associated socket.; buffer: Caller-owned writable buffer, at least length bytes.; length: Maximum receive size, from 1 through ULONG_MAX bytes.; context: Caller-chosen 64-bit identifier returned on completion..

Return: true if submitted (including immediate completion), otherwise false.

qsc_socket_iocp_submit_send

Purpose: Submit a send that completes through qsc_socket_iocp_wait().

Details: buffer and sock MUST remain alive until completion. Partial sends are possible. Do not enable FILE_SKIP_COMPLETION_PORT_ON_SUCCESS on attached sockets.

bool qsc_socket_iocp_submit_send(qsc_socket_iocp* port, const qsc_socket* sock, const uint8_t* buffer, size_t length, uint64_t context);

Parameters: port: Initialized IOCP context.; sock: Previously associated socket.; buffer: Caller-owned read-only buffer, at least length bytes.; length: Maximum send size, from 1 through ULONG_MAX bytes.; context: Caller-chosen 64-bit completion identifier..

Return: true on successful submission (including immediate completion), otherwise false.

qsc_socket_iocp_wait

Purpose: Wait for one completed operation and reclaim its private OVERLAPPED state.

Details: A completed operation with an error returns 1 and sets result->error. May be used by multiple consumer threads. The caller must serialize destruction.

int32_t qsc_socket_iocp_wait(qsc_socket_iocp* port, qsc_socket_iocp_result* result, int32_t timeout_ms);

Parameters: port: Initialized IOCP context.; result: Writable completion result, not NULL.; timeout_ms: Milliseconds, -1 for indefinite, or a nonnegative timeout..

Return: 1 for completion (including failed transfers), 0 on timeout, -1 on API failure.

qsc_socket_iocp_wake

Purpose: Enqueue one wakeup completion from another thread.

bool qsc_socket_iocp_wake(qsc_socket_iocp* port);

Parameters: port: Initialized completion-port context..

Return: true if queued, false on failure.

qsc_socket_max_send_buffer_size

Purpose: Get the maximum send buffer size for a socket

size_t qsc_socket_max_send_buffer_size(const qsc_socket* sock);

Parameters: sock: [const qsc_socket*] The socket instance.

Return: [size_t] Returns the maximum length of a send buffer

qsc_socket_peek

Purpose: Receive data from a synchronous connected socket or a bound connection-less socket without downloading the entire message

size_t qsc_socket_peek(const qsc_socket* sock, uint8_t* output, size_t otplen);

Parameters: sock: [const qsc_socket*] The socket instance; output: [uint8_t*] The output buffer that receives data; otplen: [size_t] The length of the output received.

Return: [size_t] Returns the number of bytes received from the remote host

qsc_socket_receive_progress

Purpose: Receive exactly a requested byte count while exposing partial progress on failure.

Details: Blocking sockets may wait indefinitely; nonblocking sockets return would_block and progress. Buffers are caller-owned; simultaneous reads on the same socket require external synchronization.

qsc_socket_exceptions qsc_socket_receive_progress(const qsc_socket* sock, uint8_t* output, size_t otplen, qsc_socket_receive_flags flag, size_t* received);

Parameters: sock: Connected socket; must remain valid for the entire operation.; output: Writable destination of at least otplen bytes.; otplen: Exact number of bytes to receive.; flag: Receive flags passed unchanged to the underlying socket API.; received: Non-NULL output pointer; receives the number of bytes read even on failure..

Return: Success when all bytes were received; not_connected on orderly EOF; otherwise an error code.

qsc_socket_receive_timeout

Purpose: Receive data from a synchronous connected socket with a temporary receive timeout. The function saves the socket's current receive timeout, applies the specified

timeout, performs the receive operation, and restores the original timeout before returning. The timeout value is specified in milliseconds. A timeout value of zero disables the receive timeout and uses normal blocking behavior.

size_t qsc_socket_receive_timeout(const qsc_socket* sock, uint8_t* output, size_t otplen, qsc_socket_receive_flags flag, int32_t timeout);

Parameters: sock: [const qsc_socket*] The socket instance; output: [uint8_t*] The output buffer that receives data; otplen: [size_t] The length of the output buffer; flag: [qsc_socket_receive_flags] Flags that influence the behavior of the receive function; timeout: [int32_t] The temporary receive timeout in milliseconds.

Return: [size_t] Returns the number of bytes received from the remote host

qsc_socket_receive_vectored

Purpose: Receive into multiple caller-owned buffers using one scatter I/O operation.

Details: The call may return after a partial transfer. The caller owns all buffers; synchronize concurrent reads.

size_t qsc_socket_receive_vectored(const qsc_socket* sock, const qsc_socket_buffer* buffers, size_t count, qsc_socket_receive_flags flag);

Parameters: sock: Socket receiving data.; buffers: Array of count writable byte spans (each of at most INT32_MAX bytes).; count: Number of spans, from 1 to QSC_SOCKET_IOV_MAX_BUFFERS.; flag: Platform receive flags..

Return: Bytes received, or zero for EOF/failure. Inspect qsc_socket_get_last_error on failure.

qsc_socket_send_progress

Purpose: Send exactly a requested byte count while exposing partial progress on failure.

Details: Blocking sockets may wait indefinitely; nonblocking sockets return would_block with progress. The caller retains buffer ownership; serialize writes if preserving stream message order matters.

qsc_socket_exceptions qsc_socket_send_progress(const qsc_socket* sock, const uint8_t* input, size_t inplen, qsc_socket_send_flags flag, size_t* sent);

Parameters: sock: Connected socket; must remain valid for the entire operation.; input: Readable buffer containing at least inplen bytes.; inplen: Exact number of bytes to

transmit.; flag: Send flags passed to the platform socket API.; sent: Non-NULL output pointer; receives the number of bytes sent even on failure..

Return: Success when the entire buffer is sent, not_connected for zero-byte send, or the platform error.

qsc_socket_send_vectored

Purpose: Send from multiple caller-owned buffers using one gather I/O operation.

Details: Short writes are possible. The caller owns all buffers; synchronize concurrent writes as needed.

size_t qsc_socket_send_vectored(const qsc_socket* sock, const qsc_socket_const_buffer* buffers, size_t count, qsc_socket_send_flags flag);

Parameters: sock: Socket transmitting data.; buffers: Array of count readable byte spans (each of at most INT32_MAX bytes).; count: Number of spans, from 1 to QSC_SOCKET_IOV_MAX_BUFFERS.; flag: Platform send flags..

Return: Bytes sent, or zero on failure; inspect qsc_socket_get_last_error on failure.

qsc_socket_set_blocking

Purpose: Set the socket blocking mode. This function enables or disables blocking behavior on an initialized socket. A blocking socket waits for send, receive, accept, and connect operations to complete. A non-blocking socket returns immediately when an operation would block.

qsc_socket_exceptions qsc_socket_set_blocking(const qsc_socket* sock, bool enabled);

Parameters: sock: [const qsc_socket*] The socket instance; enabled: [bool] Enable blocking mode when true, or non-blocking mode when false.

Return: [qsc_socket_exceptions] Returns an exception code on failure, or success(0)

Header: socketserver.h

qsc_socket_server_async_dispose

Purpose: Stops the listener and joins its accept worker before freeing resources.

Details: This call blocks until an in-flight callback completes when called from another thread. From the worker callback itself, cleanup is deferred to worker exit to prevent a

self-join. The public state and source socket must remain valid through callback return. Repeated disposal after completion is safe. Concurrent disposals of the same state must be externally serialized.

void qsc_socket_server_async_dispose(qsc_socket_server_async_accept_state* state);

Parameters: state: [qsc_socket_server_async_accept_state*] The asynchronous server state.

Return: None (void).

qsc_socket_server_async_stop

Purpose: Request nonblocking shutdown of the asynchronous accept loop.

Details: Safe to call from an accept or error callback. Does not wait for callbacks to complete; the caller must subsequently call async_dispose. A callback already being dispatched may still execute after this call; async_dispose waits for that callback when invoked from another thread. The public state and socket must remain valid until disposal completes.

bool qsc_socket_server_async_stop(qsc_socket_server_async_accept_state* state);

Parameters: state: [qsc_socket_server_async_accept_state*] Previously started listener state..

Return: [bool] True if a registered listener was found and stopped.

Notes: Thread safe for a single caller coordinating shutdown. No ownership of accepted sockets is transferred to this function.

14: Threads and Asynchronous Processing

14.1 Asynchronous Threading

Header:

async.h

Description:

Asynchronous threading and mutex functions.

Types:

Type Name	Purpose
qsc_async_mutex	The thread mutex type.
qsc_thread	The thread instance type.

Table 14.1 async type definitions

API:

Launch Thread

Launch a function on a new thread. Takes a pointer to the function, and a state containing the function parameters.

```
void qsc_async_launch_thread(void (*func)( void*), void* state)
```

Launch Parallel Threads

Launch a series of threads, using variadic function arguments. Takes a pointer to the function, the parameter count, and a variadic argument for parameters.

```
void qsc_async_launch_parallel_threads(void (*func) (void*), size_t count, ...)
```

Mutex Create

Create a new mutex instance. Takes a pointer to the mutex as a parameter, and returns true if the mutex was acquired successfully.

```
qsc_mutex qsc_async_mutex_create(void);
```

Mutex Destroy

Destroy a mutex instance, and set the value to zero. Takes a pointer to the mutex as a parameter, and returns true if the mutex was destroyed successfully.

```
bool qsc_async_mutex_destroy(qsc_mutex mtx);
```

Mutex Lock

Lock a mutex, creating the thread barrier. Takes a pointer to the mutex as a parameter.

```
void qsc_async_mutex_lock(qsc_mutex mtx);
```

Mutex Lock Ex

Initializes and locks a mutex in a single operation. Takes a pointer to the mutex as a parameter.

```
qsc_mutex qsc_async_mutex_lock_ex(void);
```

Mutex Unlock

Unlock a mutex, releasing the thread barrier. Takes a pointer to the mutex as a parameter.

```
void qsc_async_mutex_unlock(qsc_mutex mtx);
```

Mutex Unlock Ex

Unlocks a mutex, releasing the thread barrier, and destroys the mutex with a single operation. Takes a pointer to the mutex as a parameter.

```
void qsc_async_mutex_unlock_ex(qsc_mutex mtx);
```

Parallel For

A parallel for loop implementation. Processes each function call on a unique thread and waits for all threads before returning.

```
bool qsc_async_parallel_for(void (*task)(void *context, size_t index), void *context, size_t nthreads)
```

Processor Count

Get the number of virtual processors on the system.

```
size_t qsc_async_processor_count(void);
```

Thread Create

Create a thread with a single parameter. Takes a function pointer, and a pointer to the function state structure as parameters, and returns the thread handle, or zero on failure.

```
qsc_thread qsc_async_thread_create(void (*func)(void*), void* state)
```

Thread CreateEx

Create a thread with multiple parameters using a variadic expression. Takes a pointer to the function, the parameter count, and a variadic argument for parameters, and returns the thread handle, or zero on failure.

```
qsc_thread qsc_async_thread_create_ex(void (*func)(void**), void** args)
```

Thread Resume

Resume a suspended thread. Takes the thread as a parameter and returns zero on success.

```
int32_t qsc_async_thread_resume(qsc_thread handle);
```

Thread Sleep

Pause the thread for a number of milliseconds. Takes the number of milliseconds as a parameter.

```
void qsc_async_thread_sleep(uint32_t msec)
```

Thread Suspend

Suspend a thread. Takes the thread as a parameter and returns zero on success.

```
int32_t qsc_async_thread_suspend(qsc_thread handle);
```

Thread Terminate

Terminate a thread, and release the memory. Takes the thread handle as a parameter.

```
bool qsc_async_thread_terminate(qsc_thread handle);
```

Thread Wait

Wait for a thread to complete execution. Takes the thread handle as a parameter.

```
void qsc_async_thread_wait(qsc_thread handle);
```

Thread Wait Time

Wait for a thread for a maximum length of time. Takes the thread handle, and the number of milliseconds as parameters.

```
void qsc_async_thread_wait_all(qsc_thread* handles, size_t count);
```

Thread Wait All

Wait for an array of threads to complete execution. Takes the pointer to an array of thread handles, and the number of handles as parameters.

```
void qsc_async_thread_wait_all(qsc_thread* handles, size_t count);
```

14.2 Events

Header:

event.h

Description:

A set of functions used for event-style callback interfaces.

Types:

Type Name	Purpose
qsc_event_callback	The event callback function prototype.

Table 14.2a event types

Enumerations:

The qsc_event_list enumeration.

Enumeration	Purpose
qsc_event_receive_callback	A receive function callback.
qsc_event_send_callback	A send function callback.
qsc_event_connection_request	A connection request callback.
qsc_event_connection_shutdown	A shutdown request callback.

Table 14.2b event list enumeration

Structures:

The qsc_event_handlers structure

Data Name	Data Type	Bit Length	Function
callback	qsc_event_callback	0x40	The callback function
next	qsc_event_handlers	0x40	The dhcp string

Table 14.2c event handler structure

API:

Event Register

Register an event and callback. Takes the event enumerator, and the event callback as parameters, returns zero for success.

```
int32_t qsc_event_register(event_state* ctx, const char name[QSC_EVENT_NAME_SIZE],
qsc_event_callback callback)
```

Initialize Listeners

Initialize the event handler array. Takes an array of event handler structures as a parameter.

void qsc_event_initialize(event_state* ctx)

Get Callback

Retrieve an event callback by name. Takes the event callback name and returns the callback function handle.

qsc_event_callback qsc_event_get_callback(event_state* ctx, const char name[QSC_EVENT_NAME_SIZE])

Destroy Listeners

Destroy the event handler array. Takes an array of event handler structures as a parameter.

void qsc_event_dispose(event_state* ctx)

12.3 Thread Pool

Header:

threadpool.h

Description:

Thread-pool creation and management functions.

Structures:

The qsc_threadpool_state structure

Data Name	Data Type	Bit Length	Function
tpool	pthread_t Array	0x512	The thread array

tcount	Uint64	0x40	The thread count

Table 14.3 thread pool structure

API:

Add Task

Add a task to the threadpool. Takes a pointer to the threadpool, and a function pointer as parameters, and returns true on success.

```
bool qsc_threadpool_add_task(qsc_threadpool_state* ctx, void (*func)(void*), void* state);
```

Clear

Clear all threads from the threadpool. Takes a pointer to threadpool as a parameter.

```
void qsc_threadpool_destroy(qsc_threadpool_state* ctx)
```

Initialize

Initializes the threadpool structure. Takes a pointer to the threadpool as a parameter.

```
void qsc_threadpool_initialize(qsc_threadpool_state* ctx)
```

Remove Task

Remove a task from the threadpool. Takes a pointer to the threadpool and the thread index as parameters.

```
void qsc_threadpool_remove_task(qsc_threadpool_state* ctx, size_t index)
```

14.4 Additional Concurrency APIs

Header: async.h

qsc_async_atomic_bool_compare_exchange

Purpose: Atomically compare a boolean and conditionally store a new value.

Details: Reads target and compares it to expected. If they are equal, desired is written to target and the function returns true. If they differ, the target is left unchanged and the function returns false. The comparison and the conditional store are performed as a single indivisible operation.

bool qsc_async_atomic_bool_compare_exchange(volatile bool* target, bool expected, bool desired);

Parameters: target: [volatile bool*] Pointer to the boolean to test and update. Must not be NULL.; expected: [bool] The value the caller expects target to currently hold.; desired: [bool] The value to store if the comparison succeeds..

Return: [bool] True if the store was performed; false if target did not equal expected.

qsc_async_atomic_bool_exchange

Purpose: Atomically swap a boolean with a new value.

Details: Stores value into target and returns the value that target held immediately before the swap. The read and the write occur as a single indivisible operation; no other thread can observe an intermediate state.

bool qsc_async_atomic_bool_exchange(volatile bool* target, bool value);

Parameters: target: [volatile bool*] Pointer to the boolean to swap. Must not be NULL.; value: [bool] The new value to store..

Return: [bool] The previous value of target before the swap.

qsc_async_atomic_bool_load

Purpose: Atomically load the current value of a boolean.

Details: Reads the value at target using a sequentially consistent atomic load. The operation is equivalent to acquiring the value without the possibility of observing a partial write from a concurrent store.

bool qsc_async_atomic_bool_load(volatile bool* target);

Parameters: target: [volatile bool*] Pointer to the boolean to read. Must not be NULL..

Return: [bool] The value held by target at the time of the load.

qsc_async_atomic_bool_store

Purpose: Atomically store a value into a boolean.

Details: Writes value to target using a sequentially consistent atomic store. Any thread that subsequently calls qsc_async_atomic_bool_load on the same target is guaranteed to observe value or any later write.

void qsc_async_atomic_bool_store(volatile bool* target, bool value);

Parameters: target: [volatile bool*] Pointer to the boolean to write. Must not be NULL.; value: [bool] The value to store..

Return: None (void).

qsc_async_atomic_int32_add

Purpose: Atomically add a value to an int32_t. Adds value to *target and returns the value AFTER the addition.

```
int32_t qsc_async_atomic_int32_add(volatile int32_t* target, int32_t value);
```

Parameters: target: [volatile int32_t*] Pointer to the atomic integer.; value: [int32_t] The amount to add.

Return: [int32_t] The new value of *target after the addition.

qsc_async_atomic_int32_compare_exchange

Purpose: Atomically compare and conditionally exchange an int32_t value. If *target equals expected, stores desired into *target. The comparison and store occur as a single atomic operation.

```
bool qsc_async_atomic_int32_compare_exchange(volatile int32_t* target, int32_t
expected, int32_t desired);
```

Parameters: target: [volatile int32_t*] Pointer to the atomic integer.; expected: [int32_t] The value the caller expects *target to hold.; desired: [int32_t] The value to store if the comparison succeeds..

Return: [bool] True if the exchange was performed; false otherwise.

qsc_async_atomic_int32_decrement

Purpose: Atomically decrement an int32_t by one. Decrements *target by one and returns the value AFTER the decrement.

```
int32_t qsc_async_atomic_int32_decrement(volatile int32_t* target);
```

Parameters: target: [volatile int32_t*] Pointer to the atomic integer..

Return: [int32_t] The new value of *target after the decrement.

qsc_async_atomic_int32_exchange

Purpose: Atomically exchange an int32_t value. Writes value to *target and returns the previous value.

int32_t qsc_async_atomic_int32_exchange(volatile int32_t* target, int32_t value);

Parameters: target: [volatile int32_t*] Pointer to the atomic integer.; value: [int32_t] The new value to store..

Return: [int32_t] The previous value of *target.

qsc_async_atomic_int32_increment

Purpose: Atomically increment an int32_t by one. Increments *target by one and returns the value AFTER the increment.

int32_t qsc_async_atomic_int32_increment(volatile int32_t* target);

Parameters: target: [volatile int32_t*] Pointer to the atomic integer..

Return: [int32_t] The new value of *target after the increment.

qsc_async_atomic_int32_load

Purpose: Atomically load an int32_t value. Reads the current value of *target with sequential consistency.

int32_t qsc_async_atomic_int32_load(volatile int32_t* target);

Parameters: target: [volatile int32_t*] Pointer to the atomic integer..

Return: [int32_t] The current value.

qsc_async_atomic_int32_store

Purpose: Atomically store an int32_t value. Writes value to *target with sequential consistency.

void qsc_async_atomic_int32_store(volatile int32_t* target, int32_t value);

Parameters: target: [volatile int32_t*] Pointer to the atomic integer.; value: [int32_t] The value to store..

Return: None (void).

qsc_async_atomic_int32_subtract

Purpose: Atomically subtract a value from an int32_t. Subtracts value from *target and returns the value AFTER the subtraction.

int32_t qsc_async_atomic_int32_subtract(volatile int32_t* target, int32_t value);

Parameters: target: [volatile int32_t*] Pointer to the atomic integer.; value: [int32_t] The amount to subtract..

Return: [int32_t] The new value of *target after the subtraction.

qsc_async_thread_create_detached

Purpose: Launch a detached thread that automatically releases its thread handle.

Details: Creates a single-argument thread and immediately releases its join handle. The callback continues independently. The caller must retain the callback argument until execution completes and must not attempt to join the worker. On POSIX, cancellation cleanup frees the private callback wrapper.

bool qsc_async_thread_create_detached(void (*func)(void*), void* state);

Parameters: func: [void (*)(void*)] Callback to run on the detached thread.; state: [void*] Caller-owned state; may be NULL, not automatically freed..

Return: [bool] True when creation and handle release succeed.

Notes: A false result after creation may still leave a running worker.

qsc_async_thread_create_noargs

Purpose: Create a thread with no arguments parameter. Creates a new thread that executes the specified function with multiple arguments.

qsc_thread qsc_async_thread_create_noargs(void (*func)(void));

Parameters: func: [void (*)(void**)] Pointer to the function to execute in the new thread..

Return: [qsc_thread] Returns a handle to the created thread, or NULL on failure.

qsc_async_thread_wait_time

Purpose: Wait for a thread to complete execution with a timeout. Blocks until the specified thread has finished executing or the timeout expires. On successful completion, consumes the joinable thread handle. On timeout, the handle remains valid and must be joined later. On unsupported POSIX platforms, this function returns without joining (no portable timed join).

void qsc_async_thread_wait_time(qsc_thread handle, uint32_t msec);

Parameters: handle: [qsc_thread] The thread handle to wait on.; msec: [uint32_t] The maximum number of milliseconds to wait..

Return: None (void).

Header: event.h

qsc_event_clear_listener

Purpose: Clear a listener for a specified event.

Details: Clears the listener associated with the specified event name.

void qsc_event_clear_listener(event_state* ctx, const char name[QSC_EVENT_NAME_SIZE]);

Parameters: ctx: The event handler context.; name: [const char[QSC_EVENT_NAME_SIZE]] The name of the event..

Return: None (void).

qsc_event_listener_name_exists

Purpose: Initialize the event state.

Details: Searches the registered listeners using the context mutex.

bool qsc_event_listener_name_exists(const event_state* ctx, const char name[QSC_EVENT_NAME_SIZE]);

Parameters: ctx: The initialized event handler context.; name: Zero-terminated or fixed-width listener name..

Return: True when the name is registered; false otherwise.

Notes: Thread-safe except during initialization or disposal.

Header: threadpool.h

qsc_threadpool_sort

Purpose: Sort the threads in the pool, placing active threads at the start of the array

void qsc_threadpool_sort(qsc_threadpool_state* ctx);

Parameters: ctx: [qsc_threadpool_state*] The thread pool state.

Return: None (void).

qsc_threadpool_thread_active

Purpose: Check if a thread is active

bool qsc_threadpool_thread_active(const qsc_threadpool_state* ctx, size_t index);

Parameters: ctx: [const qsc_threadpool_state*] The thread pool state; index: [size_t] The thread index.

Return: [bool] Returns true if the thread is currently used

qsc_threadpool_worker_create

Purpose: Create a bounded pool of reusable worker threads.

qsc_threadpool_worker_state* qsc_threadpool_worker_create(size_t worker_count, size_t queue_capacity);

Parameters: worker_count: Number of workers, in [1, QSC_THREADPOOL_THREADS_MAX].; queue_capacity: Maximum number of waiting tasks; must be nonzero..

Return: An owned pool pointer, or NULL for invalid sizes/allocation/thread failure.

Notes: No task argument ownership is transferred. Destroy the pool to release resources.

qsc_threadpool_worker_destroy

Purpose: Stop accepting new work, execute all accepted tasks, join workers, and free pool.

void qsc_threadpool_worker_destroy(qsc_threadpool_worker_state* pool);

Parameters: pool: Pool to release, or NULL (no operation). Invalid after return..

Return: None (void).

Notes: Not safe concurrently with callers using the same pool; not callable from a worker.

qsc_threadpool_worker_submit

Purpose: Submit a task without blocking when the bounded queue is full.

bool qsc_threadpool_worker_submit(qsc_threadpool_worker_state* pool, void (*func)(void*), void* state);

Parameters: pool: Valid initialized pool.; func: Non-NULL task callback; invoked by a worker outside pool locks.; state: Caller-owned callback argument; must remain valid through execution..

Return: True if accepted, false for invalid arguments, full queue, or shutdown.

Notes: Thread-safe across independent and concurrent producers.

qsc_threadpool_worker_wait

Purpose: Wait until all previously accepted tasks and currently queued tasks finish.

void qsc_threadpool_worker_wait(qsc_threadpool_worker_state* pool);

Parameters: pool: Valid pool, or NULL (no operation)..

Return: None (void).

Notes: Concurrent submissions can extend the wait. Do not call from a pool callback.

15: Storage and Data Processing

15.1 Collection

Header:

collection.h

Description:

A keyed collection implementation.

Structures:

The qsc_collection_state structure

Data Name	Data Type	Bit Length	Function
items	Uint8 pointer	Width x Count	The items array
keys	Uint8 pointer	0x80 x Count	The key array
count	Uint32	32	The item count
width	Uint32	32	The maximum item width

Table 15.1a the collection state structure

Constants:

Constant Name	Value	Purpose
QSC_COLLECTION_KEY_WIDTH	16	The key byte length

Table 15.1b the collection constants

API:

Collection Add

Add an item to the collection. Pass in the initialized state, the item array pointer, and the pointer to the key to be associated with this item.

```
void qsc_collection_add(qsc_collection_state* ctx, const uint8_t* item, const uint8_t* key)
```

Collection Deserialize

Convert a serialized collection stream into a collection state.

```
void qsc_collection_deserialize(qsc_collection_state* ctx, const uint8_t* input)
```

Collection Dispose

Destroy the state and release the memory of a collection state.

```
void qsc_collection_dispose(qsc_collection_state* ctx)
```

Collection Erase

Erase the collection and set all state members to zero.

```
void qsc_collection_erase(qsc_collection_state* ctx)
```

Collection Initialize

Initialize the connection state. The width parameter defines the maximum width of an item.

```
void qsc_collection_initialize(qsc_collection_state* ctx, uint32_t width);
```

Collection Item Exists

Check if the item associated with the key is in the collection. Returns true if the item was found.

```
bool qsc_collection_item_exists(const qsc_collection_state* ctx, const uint8_t* key)
```

Collection Item Find

Find an item associated with a key in the collection, and copy it to the item parameter. Returns true if the item was found.

```
bool qsc_collection_find(const qsc_collection_state* ctx, uint8_t* item, const uint8_t* key)
```

Collection Item

Retrieve a copy of a collection item from its index number. The index allows iteration through the internal items. The item at the index is copied to the item parameter.

```
void qsc_collection_item(qsc_collection_state* ctx, uint8_t* item , size_t index)
```

Collection Item Remove

Remove an item from the collection. The key is used to find the item, and the key and item are removed.

```
void qsc_collection_remove(qsc_collection_state* ctx, const uint8_t* key)
```

Collection Serialize

Serialize the collection state to an array. The collection state is serialized to the output array. The size of the output array can be determined using the collection size call.

```
size_t qsc_collection_serialize(uint8_t* output, const qsc_collection_state* ctx);
```

Collection Size

Returns the size of the serialized collection in its current state.

```
size_t qsc_collection_size(const qsc_collection_state* ctx)
```

13.2 List

Header:

list.h

Description:

A dynamic items list implementation.

Structures:

The qsc_list_state structure

Data Name	Data Type	Bit Length	Function
items	Uint8 pointer	Width x Count	The items array
count	Uint64	64	The item count
width	Uint64	64	The maximum item width

Table 15.2a the collection state structure

Constants:

Constant Name	Value	Purpose
QSC_LIST_ALIGNMENT	64	The list memory alignment
QSC_LIST_MAX_DEPTH	10000	Maximum list depth.

Table 15.2b the collection constants

API:

List Add

Add an item to the list. Pass in the initialized state, and the item pointer.

```
void qsc_list_add(qsc_list_state* ctx, const void* item);
```

List Copy

Copy an item from the list using the items index position. Pass in the initialized state, the index position, and the output item pointer.

```
void qsc_list_copy(const qsc_list_state* ctx, size_t index, void* item);
```

List Count

Get the number of items in the list.

```
size_t qsc_list_count(const qsc_list_state* ctx)
```

List Deserialize

Deserialize a list state stream, and populate a list state. Pass the empty list state and the pointer to the input stream.

```
void qsc_list_deserialize(qsc_list_state* ctx, const uint8_t* input)
```

List Deserialize

Deserialize a list state stream, and populate a list state. Pass the empty list state and the pointer to the input stream.

```
void qsc_list_deserialize(qsc_list_state* ctx, const uint8_t* input)
```

List Dispose

Erase the state and release memory used by the list.

```
void qsc_list_dispose(qsc_list_state* ctx)
```

List Initialize

Initialize the list state. Pass the empty state and the byte-size width of each item.

```
void qsc_list_initialize(qsc_list_state* ctx, size_t width)
```

List Empty

Returns true if there are no items in the array.

```
bool qsc_list_empty(const qsc_list_state* ctx)
```

List Full

Returns true if the list has reached the maximum number of items.

```
bool qsc_list_full(const qsc_list_state* ctx);
```

List Item

Retrieve a copy of a list item from its index number. The index allows iteration through the internal items. The item at the index is copied to the item parameter.

```
void qsc_list_item(const qsc_list_state* ctx, uint8_t* item, size_t index)
```

List Item

Retrieve a copy of a list item from its index number. The index allows iteration through the internal items. The item at the index is copied to the item parameter.

```
void qsc_list_item(const qsc_list_state* ctx, uint8_t* item, size_t index)
```

List Item Remove

Remove an item from the list at the specified index. Pass the initialized list state and the index of the item to remove.

```
void qsc_collection_remove(qsc_collection_state* ctx, const uint8_t* key);
```

List Serialize

Serialize the list state to an array. The list state is serialized to the output array. The size of the output array can be determined using the list size call.

```
size_t qsc_list_serialize(uint8_t* output, const qsc_list_state* ctx);
```

List Size

Returns the size of the serialized list in its current state.

```
size_t qsc_list_size(const qsc_list_state* ctx)
```

List Sort

Sort the items in the list.

```
void qsc_list_sort(qsc_list_state* ctx);
```

15.3 QSORT

Header:

qsort.h

Description:

An implementation of QSORT sorting functions.

Sort i8

Sort an array of signed 8-bit integers by numerical value. Pass in the array to be sorted, and the start and end indexes within the array to sort.

```
void qsc_qsort_sort_i8(int8_t* arr, int32_t start, int32_t end);
```

Sort i16

Sort an array of signed 16-bit integers by numerical value. Pass in the array to be sorted, and the start and end indexes within the array to sort.

```
void qsc_qsort_sort_i16(int16_t* arr, int32_t start, int32_t end);
```

Sort i32

Sort an array of signed 32-bit integers by numerical value. Pass in the array to be sorted, and the start and end indexes within the array to sort.

```
void qsc_qsort_sort_i32(int32_t* arr, int32_t start, int32_t end);
```

Sort i64

Sort an array of signed 64-bit integers by numerical value. Pass in the array to be sorted, and the start and end indexes within the array to sort.

```
void qsc_qsort_sort_i64(int64_t* arr, int64_t start, int64_t end);
```

15.4 Additional Collections and Sorting APIs

Header: collection.h

qsc_collection_deserialize_sized

Purpose: Transactionally replace a collection from a size-checked wire image.

```
bool qsc_collection_deserialize_sized(qsc_collection_state* ctx, const uint8_t* input,
size_t inputlen);
```

Parameters: ctx: Initialized collection. Old contents survive invalid/truncated input or allocation failure.; input: Input bytes, caller owned and retained.; inputlen: Actual accessible input byte count including eight-byte header..

Return: True for valid complete data, false otherwise.

Notes: Same serialization format and item ordering as the legacy interface; thread-safe per state.

qsc_collection_disable_index

Purpose: Disable the optional indexed lookup accelerator.

```
bool qsc_collection_disable_index(qsc_collection_state* ctx);
```

Parameters: ctx: Initialized collection..

Return: True for a valid state, false otherwise.

Notes: Safe under concurrent collection operations; frees private index memory.

qsc_collection_enable_index

Purpose: Opt into an indexed lookup accelerator for a collection with stable external key data.

bool qsc_collection_enable_index(qsc_collection_state* ctx);

Parameters: ctx: Initialized collection..

Return: True if the index was built, false on allocation failure.

Notes: Caller must not modify ctx->keys directly while indexing is enabled. Duplicate keys resolve to earliest live index. Index lookup is not constant-time. The index is released when the last entry is removed. Thread-safe per state.

qsc_collection_reserve

Purpose: Reserve storage for at least capacity entries without altering count.

bool qsc_collection_reserve(qsc_collection_state* ctx, size_t capacity);

Parameters: ctx: Initialized collection; caller retains ownership.; capacity: Entry count up to QSC_COLLECTION_MAX_ENTRIES..

Return: True on success; false on invalid input, overflow or allocation failure.

Notes: Thread-safe for initialized state; may invalidate cached item/key pointers.

qsc_collection_serialize_sized

Purpose: Serialize a complete collection into a checked caller-owned output buffer.

bool qsc_collection_serialize_sized(uint8_t* output, size_t outputlen, const qsc_collection_state* ctx, size_t* written);

Parameters: output: Destination at least outputlen bytes long.; outputlen: Writable destination size in bytes.; ctx: Initialized source collection.; written: Receives byte count on success; unchanged on error..

Return: True if all bytes fit, false without modifying output for insufficient space.

Notes: No allocation or ownership transfer; thread-safe per collection.

Header: list.h

qsc_list_deserialize_sized

Purpose: Deserialize a checked complete list image in the existing wire format.

bool qsc_list_deserialize_sized(qsc_list_state* ctx, const uint8_t* input, size_t inputlen);

Parameters: ctx: Initialized destination list; replaced only on success.; input: Caller-owned serialized data, at least inputlen bytes long.; inputlen: Available input byte count, including the eight-byte header..

Return: True on success; false for malformed, truncated or unallocatable data.

Notes: Thread-safe per destination state. No input ownership transfer.

qsc_list_remove

Purpose: Remove an item from the list.

void qsc_list_remove(qsc_list_state* ctx, size_t index);

Parameters: ctx: [qsc_list_state*] Pointer to the list state structure.; index: [size_t] The index number of the item to remove..

Return: None (void).

qsc_list_reserve

Purpose: Reserve storage for at least capacity elements without changing count.

bool qsc_list_reserve(qsc_list_state* ctx, size_t capacity);

Parameters: ctx: Initialized list, owned by the caller.; capacity: Required element capacity, up to QSC_LIST_MAX_DEPTH..

Return: True on success; false for invalid state, oversized request or allocation failure.

Notes: Thread-safe for calls using the same initialized state; invalidates cached item pointers if storage moves.

qsc_list_rshuffle

Purpose: Randomly shuffle the items in the list.

void qsc_list_rshuffle(qsc_list_state* ctx);

Parameters: ctx: [qsc_list_state*] Pointer to the list state structure..

Return: None (void).

qsc_list_serialize_sized

Purpose: Serialize the existing little-endian list format into a sized buffer.

bool qsc_list_serialize_sized(uint8_t* output, size_t outputlen, const qsc_list_state* ctx, size_t* written);

Parameters: output: Caller-owned output, at least outputlen bytes accessible.; outputlen: Writable output capacity in bytes.; ctx: Initialized source list.; written: Receives complete serialized size on success; unchanged on failure..

Return: True on success, false for invalid state, size overflow or short buffer.

Notes: No ownership transfer or allocation; thread-safe per initialized list.

qsc_list_trim

Purpose: Shrink list allocation to the current count without changing item order.

bool qsc_list_trim(qsc_list_state* ctx);

Parameters: ctx: Initialized list, owned by the caller..

Return: True on success; false for invalid state or allocation failure.

Notes: Thread-safe; may invalidate externally cached item pointers.

Header: queue.h

qsc_queue_dispose

Purpose: Destroy the queue state.

void qsc_queue_dispose(qsc_queue_state* ctx);

Parameters: ctx: [struct] A pointer to the queue state structure..

Return: None (void).

qsc_queue_empty

Purpose: Get the empty status from the queue.

bool qsc_queue_empty(const qsc_queue_state* ctx);

Parameters: ctx: [const qsc_queue_state*] A pointer to the queue state structure..

Return: [bool] Returns true if the queue is empty.

qsc_queue_flush_bounded

Purpose: Flush all items only if the destination has sufficient capacity.

Details: A short destination leaves the queue unchanged. Thread-safe except concurrent dispose.

size_t qsc_queue_flush_bounded(qsc_queue_state* ctx, uint8_t* output, size_t outlen);

Parameters: ctx: Initialized queue.; output: Output byte array.; outlen: Capacity of output in bytes..

Return: Number of removed items; zero if empty or insufficient output capacity.

qsc_queue_full

Purpose: Get the full status from the queue.

bool qsc_queue_full(const qsc_queue_state* ctx);

Parameters: ctx: [const qsc_queue_state*] A pointer to the queue state structure..

Return: [bool] Returns true if the queue is full.

qsc_queue_pop_batch

Purpose: Remove a batch of items in FIFO order with one mutex acquisition.

Details: Thread-safe for concurrent operations, not concurrent dispose.

size_t qsc_queue_pop_batch(qsc_queue_state* ctx, uint8_t* output, size_t capacity, size_t otplen, uint64_t* tags);

Parameters: ctx: Initialized queue.; output: Buffer holding capacity times otplen bytes.; capacity: Number of destination item slots.; otplen: Number of bytes to copy per item (not exceeding queue width).; tags: Optional buffer for capacity tags..

Return: Actual number removed, or zero on empty queue or invalid input.

qsc_queue_push_batch

Purpose: Append a packed sequence of items using one mutex acquisition.

Details: Thread-safe for concurrent operations, not concurrent dispose.

size_t qsc_queue_push_batch(qsc_queue_state* ctx, const uint8_t* input, size_t count, size_t inlen, const uint64_t* tags);

Parameters: ctx: Initialized queue.; input: Packed input containing count times inlen bytes.; count: Number of available input items.; inlen: Bytes per item; must be between one and queue width.; tags: Optional array containing count tags, or NULL for zero tags..

Return: Number of successfully appended items (can be less than count).

qsc_queue_try_pop

Purpose: Pop an item, distinguishing empty state from a valid zero tag.

Details: Thread-safe for concurrent operations, not concurrent dispose.

bool qsc_queue_try_pop(qsc_queue_state* ctx, uint8_t* output, size_t otplen, uint64_t* tag);

Parameters: ctx: Initialized queue; must remain alive throughout the operation.; output: Receives otplen bytes on success; unchanged on failure.; otplen: Nonzero byte length, at most ctx->width.; tag: Optional output for the associated tag..

Return: True if an item was removed; false on empty or invalid arguments.

qsc_queue_try_push

Purpose: Push one item with an explicit success indicator.

Details: Thread-safe for concurrent operations, not concurrent dispose.

bool qsc_queue_try_push(qsc_queue_state* ctx, const uint8_t* input, size_t inlen, uint64_t tag);

Parameters: ctx: Initialized queue; must remain alive throughout the operation.; input: Buffer with at least inlen bytes.; inlen: Nonzero byte length, at most ctx->width.; tag: Associated 64-bit value (zero is permitted)..

Return: True when accepted, false on full queue or invalid arguments.

16: Integer and String Tools

16.1 Array Utilities

Header:

arrayutils.h

Description:

Array utilities; supporting string to integer functions

Constants:

Constant Name	Value	Purpose
QSC_ARRAYUTILS_NPOS	-1	The find string not found return value.

Table 16.1 socket state constants

API:

Find String

Find the first instance of a token in a string, and return the char position. Takes the string, the search token, and the string length as parameters, and returns the tokens position within the string.

```
size_t qsc_arrayutils_find_string(const char* str, size_t slen, const char* token)
```

Hex To Uint8

Converts a hexadecimal encoded string to a byte value. Takes the string, and the string length as parameters, and returns the byte value.

```
uint8_t qsc_arrayutils_hex_to_uint8(const char* str, size_t slen)
```

Uint8 To Hex

Converts a byte value to hexadecimal and writes to a string. Takes a pointer to the char output array, the output length, and the value as parameters.

```
void qsc_arrayutils_uint8_to_hex(char* output, size_t otplen, uint8_t value);
```

String To Uint8

Parse a 8-bit unsigned integer from a string. Takes a pointer to a char array, and the array length as parameters, and returns an 8-bit unsigned integer.

uint8_t qsc_arrayutils_string_to_uint8(const char* str, size_t slen)

String To Uint16

Parse a 16-bit unsigned integer from a string. Takes a pointer to a char array, and the array length as parameters, and returns a 16-bit unsigned integer.

uint16_t qsc_arrayutils_string_to_uint16(const char* str, size_t slen)

String To Uint32

Parse a 32-bit unsigned integer from a string. Takes a pointer to a char array, and the array length as parameters, and returns a 32-bit unsigned integer.

uint32_t qsc_arrayutils_string_to_uint32(const char* str, size_t slen)

String To Uint64

Parse a 64-bit unsigned integer from a string. Takes a pointer to a char array, and the array length as parameters, and returns a 64-bit unsigned integer.

uint64_t qsc_arrayutils_string_to_uint64(const char* str, size_t slen)

16.2 Console Utilities

Header:

consoleutils.h

Description:

The console utilities class contains function used in creating inter-active console-based tools.

Enumerations:

The qsc_console_font_color enumeration.

Enumeration	Purpose
white	White font displayed on the console.
blue	Blue font displayed on the console.
green	Green font displayed on the console.
red	Red font displayed on the console.

Table 16.2a font color enumeration

The qsc_console_font_style enumeration

Enumeration	Purpose
regular	Regular font displayed on the console.
bold	Bold font displayed on the console.
italic	Italic font displayed on the console.
bolditalic	Bold and italic font displayed on the console.

Table 16.2b font style enumeration

Constants:

Constant Name	Value	Purpose
QSC_CONSOLE_MAX_LINE	128	The maximum length of a console line.

Table 16.2c console constants

API:

Colored Message

Color a line of console text. Takes the message string, and a font color enumeral as parameters.

```
void qsc_consoleutils_colored_message(const char* message, qsc_console_font_color color)
```

Get Char

A blocking wait that returns a single character from console input. Returns the character value.

```
char qsc_consoleutils_get_char(void);
```

Get Line

A blocking wait that returns a string from console input after a carriage return is detected. Returns the character string. Returns the size of the string, and takes a pointer to a character array, and the maximum line length as parameters.

```
size_t qsc_consoleutils_get_line(char* line, size_t maxlen)
```

Get Formatted Line

A blocking wait that returns a string from console input after a carriage return is detected. Removes the new-line, null, and carriage return characters. Returns the size of the string, and takes a pointer to the output character array, and the maximum line length as parameters.

```
size_t qsc_consoleutils_get_formatted_line(char* line, size_t maxlen)
```

Get Wait

Pause the console until user-input is detected.

```
char qsc_consoleutils_get_wait(void);
```

Hex To Bin

Convert a hexadecimal character string to a character byte array. Takes a pointer to the input hex string, a pointer to the output byte array, and the number of characters to convert as parameters.

```
void qsc_consoleutils_hex_to_bin(const char* hexstr, uint8_t* output, size_t length)
```

Line Contains

Find a set of characters in a line of console text. Takes the input line, and the search token as parameters, and returns true if the line contains the token.

```
bool qsc_consoleutils_line_contains(const char* line, const char* token)
```

Masked Password

Gets a password masked on the console screen. Takes a pointer to an output byte array, and the output array length as parameters, and returns the size of the password.

```
size_t qsc_consoleutils_masked_password(char* output, size_t otplen);
```

Message Confirm

User confirmation that and action can continue (Y/N y/n). Takes a pointer to the message array as the parameter, and returns true/false.

```
bool qsc_consoleutils_message_confirm(const char* message)
```

Print Hex

Convert a byte array to a hexadecimal string and print to the console. Takes a pointer to the input byte array, the input array length, and the line length as parameters. The output will include new line characters inserted at line length.

```
void qsc_consoleutils_print_hex(const uint8_t* input, size_t inplen, size_t linelen);
```

Print Formatted

Print a string to the console, ignoring special characters. Takes the input message and the input length as parameters.

```
void qsc_consoleutils_print_formatted(const char* input, size_t inplen);
```

Print Formatted Line

Print a string to the console, ignoring special characters, and add a line break. Takes the input message and the input length as parameters.

```
void qsc_consoleutils_print_formatted_line(const char* input, size_t inplen);
```

Print Safe

Prints an array of characters to the console. Takes the input message as a parameter.

```
void qsc_consoleutils_print_safe(const char* input)
```

Print Line

Print an array of characters to the console with a line break.

```
void qsc_consoleutils_print_line(const char* input)
```

Print Concatenated Line

Print a concatenated set of character arrays, to the console with a line break between each. Takes a pointer to a multi-dimensional array of strings, and the number of sub-arrays as parameters..

```
void qsc_consoleutils_print_concatenated_line(const char** input, size_t count)
```

Print Uint

Print a 32-bit unsigned integer to the console. Takes the integer to print as a parameter.

```
void qsc_consoleutils_print_uint(uint32_t digit)
```

Print Ulong

Print an unsigned 64-bit integer to the console. Takes the integer to print as a parameter.

```
void qsc_consoleutils_print_ulong(uint64_t digit)
```

Print Double

Print a double floating point integer to the console. Takes the double to print as a parameter.

```
void qsc_consoleutils_print_double(double digit)
```

Progress Counter

Prints a small spinning counter. Takes the duration in seconds as a parameter.

```
void qsc_consoleutils_progress_counter(int32_t seconds)
```

Set Window Buffer

Set the size of the window scroll buffers. Takes the desired horizontal and vertical sizes as parameters.

```
void qsc_consoleutils_set_window_buffer(size_t width, size_t height)
```

Set Window Clear

Clear the text from the window.

```
void qsc_consoleutils_set_window_clear(void);
```

Set Window Prompt

Set the window prompt string.

```
void qsc_consoleutils_set_window_prompt(const char* prompt)
```

Set Window Size

Set the initial size of the console window. Takes the desired window width and height as parameters.

```
void qsc_consoleutils_set_window_size(size_t width, size_t height)
```

Set Window Title

Set the window title string. Takes a pointer to the window title string as a parameter.

```
void qsc_consoleutils_set_window_title(const char* title)
```

Set Virtual Terminal

Enable virtual terminal mode.

```
void qsc_consoleutils_set_virtual_terminal(void);
```

16.3 Encoding

Header:

encoding.h

Description:

Base64, BER, DER, HEX, and PEM encoding utilities.

Enumerations:

The qsc_encoding_ber_asn1_tag_t enumeration

Enumeration	Purpose
BER_ASN1_BOOLEAN	BOOLEAN
BER_ASN1_EOC	End-of-Contents (EOC) marker

BER_ASN1_INTEGER	INTEGER
BER_ASN1_BIT_STRING	BIT STRING
BER_ASN1_OCTET_STRING	OCTET STRING
BER_ASN1_NULL	NULL
BER_ASN1_OBJECT_IDENTIFIER	OBJECT IDENTIFIER
BER_ASN1_OBJECT_DESCRIPTOR	Object Descriptor
BER_ASN1_EXTERNAL	EXTERNAL (or Instance-of)
BER_ASN1_REAL	REAL (floating-point)
BER_ASN1_ENUMERATED	ENUMERATED
BER_ASN1_EMBEDDED_PDV	Embedded PDV
BER_ASN1_UTF8_STRING	UTF8String
BER_ASN1_RELATIVE_OID	Relative Object Identifier
BER_ASN1_SEQUENCE	SEQUENCE and SEQUENCE OF
BER_ASN1_SET	SET and SET OF
BER_ASN1_NUMERIC_STRING	NumericString
BER_ASN1_PRINTABLE_STRING	PrintableString
BER_ASN1_T61_STRING	TeletexString (T61String)
BER_ASN1_VIDEOTEX_STRING	VideotexString
BER_ASN1_IA5_STRING	IA5String
BER_ASN1_UTCTIME	UTCTime
BER_ASN1_GENERALIZEDTIME	GeneralizedTime
BER_ASN1_GRAPHIC_STRING	GraphicString
BER_ASN1_VISIBLE_STRING	VisibleString (ISO646String)
BER_ASN1_GENERAL_STRING	GeneralString
BER_ASN1_UNIVERSAL_STRING	UniversalString
BER_ASN1_CHARACTER_STRING	CharacterString
BER_ASN1_BMP_STRING	BMPString

Table 16.3a BER ASN1 tag enumeration

Structures:

The qsc_encoding_ber_element structure

Data Name	Data Type	Bit Length	Function
tagclass	uint8_t	8	Tag class identifier
constructed	bool	8	Element is constructed
tagnumber	uint32_t	32	Tag number
indefinite	bool	8	Indefinate form
length	size_t	64	definite-length encoding
value	uint8_t*	8	element's raw value
children	qsc_encoding_ber_element**	variable	child pointers
ccount	size_t	64	number of child elements

Table 16.3b the BER element state structure

API:

Base64 Decode

Decodes a base64 string to a byte array. Takes a pointer to the output byte array and input character arrays, the output and input array lengths as parameters.

```
bool qsc_encoding_base64_decode(uint8_t* output, size_t otplen, const char* input, size_t inplen);
```

Base64 Decoded Size

Gets the expected size of an array required by decoding. Takes a pointer to the input character array, and the array length as parameters, and returns the expected size of the decoded string.

```
size_t qsc_encoding_base64_decoded_size(const char* input, size_t length)
```

Base64 Encode

Encode a byte array to a base64 string. Takes pointers to the output character array, and input byte arrays, and the input and output array lengths as parameters.

bool qsc_encoding_base64_encode(char* output, size_t otplen, const uint8_t* input, size_t inplen);

Base64 Encoded Size

Gets the expected size of an array required by encoding. Takes a pointer to the input character array, and the array length as parameters, and returns the expected size of the encoded string.

size_t qsc_encoding_base64_decoded_size(const char* input, size_t length)

Is Valid Char

Tests if an encoded character is a valid base64 encoding. Takes the character value as a parameter, and returns true/false.

bool qsc_encoding_base64_is_valid_char(char value)

BER Decode Element

Decodes a BER element from encoded data.

qsc_encoding_ber_element* qsc_encoding_ber_decode_element(const uint8_t* buffer, size_t buflen, size_t* consumed)

BER Decode Length

Decodes a BER-encoded length value.

size_t qsc_encoding_ber_decode_length(const uint8_t* buffer, size_t buflen, size_t* length, bool* indef)

BER Decode Tag

Decodes an ASN.1 tag from BER-encoded data.

size_t qsc_encoding_ber_decode_tag(const uint8_t* buffer, size_t buflen, uint8_t* tagclass, bool* construct, uint32_t* tagnum)

BER Encode Element

Encodes a complete BER element.

size_t qsc_encoding_ber_encode_element(qsc_encoding_ber_element* element, uint8_t* buffer, size_t buflen)

BER Encode Length

Encodes a length value into BER format.

size_t qsc_encoding_ber_encode_length(size_t length, uint8_t* buffer, size_t buflen)

BER Encode Tag

Encodes an ASN.1 tag into BER format.

size_t qsc_encoding_ber_encode_tag(uint8_t tagclass, bool construct, uint32_t tagnum, uint8_t* buffer, size_t buflen)

BER Free Element

Free a BER element from the array.

void encoding_ber_free_element(qsc_encoding_ber_element* element)

DER Decode Element

Decodes an ASN.1 element encoded in DER format.

qsc_encoding_ber_element* qsc_encoding_der_decode_element(const uint8_t* buffer, size_t buflen, size_t* consumed)

DER Encode Element

Encodes an ASN.1 element using DER (Distinguished Encoding Rules).

size_t qsc_encoding_der_encode_element(qsc_encoding_ber_element* element, uint8_t* buffer, size_t buflen)

HEX Decode

Decodes a hexadecimal string into binary data.

bool qsc_encoding_hex_decode(const char* input, size_t inplen, uint8_t* output, size_t otplen, size_t* declen)

HEX Encode

Encodes binary data into a hexadecimal string.

bool qsc_encoding_hex_encode(const uint8_t* input, size_t inplen, char* output, size_t otplen)

PEM Decode

Decodes a PEM-formatted string into binary data.

bool qsc_encoding_pem_decode(const char* input, size_t inplen, uint8_t* output, size_t otplen, size_t* declen);

PEM Encode

Encodes binary data in PEM format.

bool qsc_encoding_pem_encode(const char* label, char* output, size_t otplen, const uint8_t* data, size_t datalen)

16.4 File Utilities

Header:

fileutils.h

Description:

File utilities; contains common file related functions.

API:

Working Directory

Get the working directory path. Takes a pointer to a character array that receives the path, and returns true if the directory is loaded, false if the path array is too small.

Historical function name: qsc_fileutils_working_directory. Revision 1.4 uses qsc_fileutils_get_working_directory; refer to its source-exact declaration and contract in this chapter.

File Exists

Test to see if a file exists. Takes a pointer to the search path as a parameter, and returns true/false.

Historical function name: qsc_fileutils_file_exists. Revision 1.4 uses qsc_fileutils_exists; refer to its source-exact declaration and contract in this chapter.

File Size

Returns the files size in bytes. Takes a pointer to the file path as a parameter, and returns the file size.

Historical function name: qsc_fileutils_file_size. Revision 1.4 uses qsc_fileutils_get_size; refer to its source-exact declaration and contract in this chapter.

Get Line

Reads a line of text from a formatted file. Takes a pointer to the output char array, a pointer to the output length, and a pointer to the file structure as parameters, and returns the line size, or zero.

Historical function name: qsc_fileutils_getline. Revision 1.4 uses qsc_fileutils_get_line; refer to its source-exact declaration and contract in this chapter.

Get Directory

Get the directory portion of a full file path. The directory receives the directory portion of the fpath. The function returns the length of the string.

```
size_t qsc_fileutils_get_directory(char* directory, size_t dirlen, const char* fpath)
```

Get Extension

Get the file extension portion of a full file path. The directory receives the directory portion of the fpath. The function returns the length of the string.

```
size_t qsc_fileutils_get_name(char* name, size_t namelen, const char* fpath)
```

Get Name

Get the file name portion of a full file path. The directory receives the directory portion of the fpath. The function returns the length of the string.

```
size_t qsc_fileutils_get_name(char* name, size_t namelen, const char* fpath)
```

Append To File

Append an array of characters to a file. Writes new data to the end of a binary file. Takes a pointer to the file path and the input stream, and the length of the input as parameters, and returns true for success.

```
bool qsc_fileutils_append_to_file(const char* fpath, const char* stream, size_t length);
```

Create File

Create a new file. Takes a pointer to the full file path as a parameter, and returns true if the file is created.

Historical function name: qsc_fileutils_create_file. Revision 1.4 uses qsc_fileutils_create; refer to its source-exact declaration and contract in this chapter.

Copy File

Create a copy of a file. Takes a pointer to the full file inpath, and the new file path as parameters., and returns true if the file is copied.

```
bool qsc_fileutils_file_copy(const char* inpath, const char* outpath)
```

Copy Object To File

Copy an object to a file. Takes pointers to the file path, and the object to copy, and the object size as parameters, and returns true if the object is copied.

```
bool qsc_fileutils_copy_object_to_file(const char* fpath, const void* obj, size_t length);
```

Copy Stream To File

Copy a character array to a file. Takes pointers to the file path, and the array to copy, and the array size as parameters, and returns true if the array is copied.

```
bool qsc_fileutils_copy_stream_to_file(const char* fpath, const char* stream, size_t length);
```

Copy File To Object

Copy a file to an object. Takes a pointer to the file path, a pointer to the object, and the object size as parameters, and returns the size in bytes that were copied.

```
size_t qsc_fileutils_copy_file_to_object(const char* fpath, void* obj, size_t length);
```

Copy File To Stream

Copy elements from a file to a byte array. Takes a pointer to the file path, a pointer to the input array, and the array length as parameters, and returns the size in bytes that were copied.

```
size_t qsc_fileutils_copy_file_to_stream(const char* fpath, char* stream, size_t length);
```

Delete File

Delete a file. Takes a pointer to the file path as a parameter, and returns true on success.

Historical function name: qsc_fileutils_delete_file. Revision 1.4 uses qsc_fileutils_delete; refer to its source-exact declaration and contract in this chapter.

Erase File

Erase a files content. Takes a pointer to the file path as a parameter, and returns true on success.

Historical function name: qsc_fileutils_erase_file. Revision 1.4 uses qsc_fileutils_erase; refer to its source-exact declaration and contract in this chapter.

List Files

Lists the files in a directory and their attributes. The result string receives each file name, attribute, creation and modified time in a formatted string. The function returns the length of the string.

size_t qsc_fileutils_list_files(char* result, size_t reslen, const char* directory)

Read Line

Read a line of text from a file. Takes a pointer to the file path, a pointer to the output buffer, and a line number as parameters, and returns the number of characters read.

int64_t qsc_fileutils_read_line(const char* fpath, char* buffer, size_t buflen, size_t linenum);

Safe Read

Read a number of characters from a file. Takes a pointer to the file path, the starting position, the output array, and the length of bytes to read. The function returns the number of bytes read..

size_t qsc_fileutils_safe_read(const char* fpath, size_t position, char* output, size_t length)

Safe Write

Write a number of characters to a file. Takes a pointer to the file path, the starting position, the output array, and the length of bytes to read. The function returns the number of bytes read..

```
size_t qsc_fileutils_safe_write(const char* fpath, size_t position, const char* input, size_t length)
```

Seek To

Set the file pointer position. Pass in the file pointer, and the position from the start of the file. The function returns true if successful.

```
bool qsc_fileutils_seekto(FILE* fp, size_t position)
```

Truncate File

Truncate a file to a specified length. Pass the file handle and the truncated length of the file. The function returns true if successful.

```
bool qsc_fileutils_truncate_file(FILE* fp, size_t length)
```

Valid Path

Checks a file path for valid formatting. The function returns true if successful.

```
bool qsc_fileutils_valid_path(const char* fpath)
```

File Write

Writes a number of bytes to a file. Pass the input parameter and input length, the starting position in the file, and the file handle. The function returns the number of bytes written to the file.

```
size_t qsc_fileutils_write(const char* input, size_t inplen, size_t position, FILE* fp)
```

File Write Line

Writes a line of text to a file. Pass the file path, the input parameter and input length. The function returns true if successful.

```
bool qsc_fileutils_write_line(const char* fpath, const char* input, size_t inplen)
```

File Zeroise

Erase a file and set it to zero bytes length. Pass the file path parameter.

```
void qsc_fileutils_zeroise(const char* fpath)
```

16.5 Folder Utilities

Header:

folderutils.h

Description:

Folder utilities; common folder support functions.

Enumerations:

The qsc_folderutils_directories enumeration

Enumeration	Purpose
qsc_folderutils_directories_user_app_data	User App Data directory.
qsc_folderutils_directories_user_desktop	User Desktop directory.
qsc_folderutils_directories_user_documents	User Documents directory.
qsc_folderutils_directories_user_downloads	User Downloads directory.
qsc_folderutils_directories_user_favourites	User Favourites directory
qsc_folderutils_directories_user_music	User Music directory
qsc_folderutils_directories_user_pictures	User Pictures directory
qsc_folderutils_directories_user_programs	User Programs directory
qsc_folderutils_directories_user_shortcuts	User Shortcuts directory
qsc_folderutils_directories_user_videos	User Video directory

Table 16.5 common directories enumeration

API:

Append Delimiter

Append a delimiter to a directory path. Pass the path parameter into the function.

```
void qsc_folderutils_append_delimiter(char path[QSC_SYSTEM_MAX_PATH])
```

Create Directory

Create a new folder. Takes the directory path as a parameter, and returns true if the folder is created.

```
bool qsc_folderutils_create_directory(const char path[QSC_SYSTEM_MAX_PATH])
```

Delete Folder

Delete a folder. Takes the directory path as a parameter, and returns true if the folder is deleted.

```
bool qsc_folderutils_delete_directory(const char path[QSC_SYSTEM_MAX_PATH])
```

Directory Exists

Check if a folder exists. Takes the directory path as a parameter, and returns true if the folder exists.

```
bool qsc_folderutils_directory_exists(const char path[QSC_SYSTEM_MAX_PATH])
```

Directory List

Lists the sub directories in a directory and their attributes. The result string receives each directory name, creation and modified time in a formatted string. The function returns true if successful

```
bool qsc_folderutils_directory_exists(const char path[QSC_SYSTEM_MAX_PATH])
```

Get Directory

Get the full path to a special system folder. Takes a directory enumerator and a pointer to an output array as parameters.

```
void qsc_folderutils_get_directory(qsc_folderutils_directories directory, char output[QSC_SYSTEM_MAX_PATH])
```

Directory Has Delimiter

Checks if the directory path ends in a delimiter. The function returns true if the path has the delimiter.

```
bool qsc_folderutils_directory_has_delimiter(const char path[QSC_SYSTEM_MAX_PATH])
```

16.6 Integer Utilities

Header:

intutils.h

Description:

Integer utilities; supporting integer related functions.

API:

Are Equal8

Compares two 8-bit integers arrays for equality. Parameters are pointers to both arrays, and the number of bytes to compare. Returns true if the arrays are equal.

```
bool qsc_intutils_are_equal8(const uint8_t* a, const uint8_t* b, size_t length)
```

Be8To16

Convert an 8-bit integer array to a 16-bit big-endian integer. Takes the input byte array as a parameter, and returns a 16-bit unsigned integer.

uint16_t qsc_intutils_be8to16(const uint8_t* input)

Be8To32

Convert an 8-bit integer array to a 32-bit big-endian integer. Takes the input byte array as a parameter, and returns a 32-bit unsigned integer.

uint32_t qsc_intutils_be8to32(const uint8_t* input)

Be8To64

Convert an 8-bit integer array to a 64-bit big-endian integer. Takes the input byte array as a parameter, and returns a 64-bit unsigned integer.

uint64_t qsc_intutils_be8to64(const uint8_t* input)

Be16To8

Convert a 16-bit big-endian integer to a 8-bit unsigned integer array. Takes a pointer to the output array, and the 16-bit value as parameters.

void qsc_intutils_be16to8(uint8_t* output, uint16_t value)

Be32To8

Convert a 32-bit big-endian integer to a 8-bit unsigned integer array. Takes a pointer to the output array, and the 32-bit value as parameters.

void qsc_intutils_be32to8(uint8_t* output, uint32_t value)

Be64To8

Convert a 64-bit big-endian integer to a 8-bit unsigned integer array. Takes a pointer to the output array, and the 64-bit value as parameters.

```
void qsc_intutils_be64to8(uint8_t* output, uint64_t value)
```

Be8Increment

Increment an 8-bit integer array as a segmented big-endian integer. Takes a pointer to the output byte array, and the arrays length as parameters.

```
void qsc_intutils_be8increment(uint8_t* output, size_t otplen);
```

BSwap32

Byte reverse an array of 32-bit integers. Takes pointers to the destination and source arrays, and the number of 32-bit integers to convert.

```
void qsc_intutils_bswap32(uint32_t* dest, const uint32_t* source, size_t length);
```

BSwap64

Byte reverse an array of 64-bit integers. Takes pointers to the destination and source arrays, and the number of 64-bit integers to convert.

```
void qsc_intutils_bswap64(uint64_t* dest, const uint64_t* source, size_t length);
```

Clear8

Set an unsigned 8-bit integer array to zeroes. Takes a pointer to the 8-bit unsigned integer array, and the number of bytes to clear as parameters.

```
void qsc_intutils_clear8(uint8_t* a, size_t count)
```

Clear16

Set an unsigned 16-bit integer array to zeroes. Takes a pointer to the 16-bit integer array, and the number of integers to clear as parameters.

```
void qsc_intutils_clear16(uint16_t* a, size_t count)
```

Clear32

Set an unsigned 32-bit integer array to zeroes. Takes a pointer to the 32-bit integer array, and the number of integers to clear as parameters.

```
void qsc_intutils_clear32(uint32_t* a, size_t count)
```

Clear64

Set an unsigned 64-bit integer array to zeroes. Takes a pointer to the 64-bit integer array, and the number of integers to clear as parameters.

```
void qsc_intutils_clear64(uint64_t* a, size_t count)
```

CMov

Constant-time conditional move function. Takes pointers to the destination and source arrays, the number of bytes to move, and the condition as parameters.

```
void qsc_intutils_cmov(uint8_t* dest, const uint8_t* source, size_t length, uint8_t cond)
```

Expand Mask

Expand an integer mask in constant time. Takes the value, and returns the mask value.

```
size_t qsc_intutils_expand_mask(size_t x)
```

Are Equal

Check if an integer is equal to a second integer. Takes the two integer values as parameters, and returns true if they are equal.

```
bool qsc_intutils_are_equal(size_t x, size_t y)
```

IsGte

Check if an integer (x) is greater or equal to a second integer (y). Takes the integers as parameters, and returns true if x is greater or equal to y.

```
bool qsc_intutils_is_gte(size_t x, size_t y)
```

Hex To Bin

Convert a hex string to an array. Takes pointers to the input hex string and output arrays, and the number of characters to process.

```
void qsc_intutils_hex_to_bin(const char* hexstr, uint8_t* output, size_t otplen);
```

Bin To Hex

Convert an array to a hex string. Takes pointers to the input byte array, and output hex character arrays, and the number of bytes to convert as parameters.

```
void qsc_intutils_bin_to_hex(const uint8_t* input, char* hexstr, size_t inplen);
```

Le8Increment

Increment an 8-bit integer array as a segmented little-endian integer. Takes a pointer to the output byte array, and the array's length as parameters.

```
void qsc_intutils_le8increment(uint8_t* output, size_t otplen);
```

Le8Increment x128

Increment the low 64-bit integer of a little-endian array by one. Takes a pointer to the 128-bit integer counter.

```
void qsc_intutils_leincrement_x128(__m128i* counter)
```

Le8Increment x512

Offset increment the low 64-bit integer of a set of 64-bit pairs of a little-endian integers. Takes a pointer to the 512-bit integer counter.

```
void qsc_intutils_leincrement_x512(__m512i* counter)
```

Le8To16

Convert an 8-bit integer array to a 16-bit little-endian integer. Takes the input byte array as a parameter, and returns a 16-bit unsigned integer.

uint16_t qsc_intutils_le8to16(const uint8_t* input)

Le8To32

Convert an 8-bit integer array to a 32-bit little-endian integer. Takes the input byte array as a parameter, and returns a 32-bit unsigned integer.

uint32_t qsc_intutils_le8to32(const uint8_t* input)

Le8To64

Convert an 8-bit integer array to a 64-bit little-endian integer. Takes the input byte array as a parameter, and returns a 64-bit unsigned integer.

uint64_t qsc_intutils_le8to64(const uint8_t* input)

Le16To8

Convert a 16-bit little-endian integer to an 8-bit unsigned integer array. Takes a pointer to the output array, and the 16-bit value as parameters.

void qsc_intutils_be16to8(uint8_t* output, uint16_t value)

Le32To8

Convert a 32-bit little-endian integer to a 8-bit unsigned integer array. Takes a pointer to the output array, and the 32-bit value as parameters.

void qsc_intutils_be32to8(uint8_t* output, uint32_t value)

Le64To8

Convert a 64-bit integer to a little-endian 8-bit unsigned integer array. Takes a pointer to the output array, and the 64-bit value as parameters.

Historical function name: qsc_intutils_b64to8. Revision 1.4 uses qsc_intutils_le64to8; refer to its source-exact declaration and contract in this chapter.

Max

Return the larger of two integers. Takes the two comparison integers as parameters, and returns the largest value.

size_t qsc_intutils_max(size_t a, size_t b)

Min

Return the smaller of two integers. Takes the two comparison integers as parameters, and returns the smallest value.

size_t qsc_intutils_min(size_t a, size_t b)

Reverse Bytes X128

Reverse a 128-bit integer. Takes the input and output integers as parameters.

void qsc_intutils_reverse_bytes_x128(const __m128i* input, __m128i* output)

Reverse Bytes X512

Reverse a 512-bit integer. Takes the input and output integers as parameters.

void qsc_intutils_reverse_bytes_x512(const __m512i* input, __m512i* output)

RotL32

Rotate an unsigned 32-bit integer to the left. Takes the value and the number of bits to rotate that value as parameters, and returns the rotated integer.

uint32_t qsc_intutils_rotl32(uint32_t value, size_t shift)

RotL64

Rotate an unsigned 64-bit integer to the left. Takes the value and the number of bits to rotate that value as parameters, and returns the rotated integer.

uint64_t qsc_intutils_rotl64(uint64_t value, size_t shift)

RotR32

Rotate an unsigned 32-bit integer to the right. Takes the value and the number of bits to rotate that value as parameters, and returns the rotated integer.

uint32_t qsc_intutils_rotr32(uint32_t value, size_t shift)

RotR64

Rotate an unsigned 64-bit integer to the right. Takes the value and the number of bits to rotate that value as parameters, and returns the rotated integer.

uint64_t qsc_intutils_rotr64(uint64_t value, size_t shift)

Verify

Constant time comparison of two arrays of unsigned 8-bit integers. Takes pointers to the two arrays, and the number of bytes to compare as parameters. Returns zero in the arrays are equivalent.

int32_t qsc_intutils_verify(const uint8_t* a, const uint8_t* b, size_t length)

16.7 String Utilities

Header:

stringutils.h

Description:

String utilities; common string support functions.

Constants:

Constant Name	Value	Purpose
QSC_STRINGUTILS_TOKEN_NOT_FOUND	-1	The search token was not found.
QSC_STRINGUTILS_HEX_EXTENSION_SIZE	2	The char size of the hex extension
QSC_STRINGUTILS_HEX_BYTE_SIZE	2	The char size of a hexadecimal byte

Table 16.7 string return constants

API:

Formatting Count

Counts all whitespaces, line stops, and return characters from a string. Takes a pointer to the destination string, and the destination length as parameters, and returns the new string length.

```
size_t qsc_stringutils_formatting_count(const char* dest, size_t dstlen)
```

Formatting Filter

Remove all whitespaces, line stops, and return characters from a string. Takes pointers to the source and destination strings, and the source length as parameters, and returns the size of the new string.

```
size_t qsc_stringutils_formatting_filter(const char* source, size_t srclen, char* dest);
```

Add Line Breaks

Add line breaks to a string at a line length interval. Takes pointers to the destination buffer and the source string, the destination length, the line length, and the source length as parameters, returns the size of the new string.

size_t qsc_stringutils_add_line_breaks(char* dest, size_t dstlen, size_t linelen, const char* source, size_t srclen)

Remove Line Breaks

Removes all line breaks from a string. Takes pointers to the destination buffer and source strings, and the destination length and source length as parameters, returns the size of the new string.

size_t qsc_stringutils_remove_line_breaks(char* dest, size_t dstlen, const char* source, size_t srclen)

Clear String

Clear a string of data. Takes a pointer to the source array as a parameter.

void qsc_stringutils_clear_string(char* source)

Clear SubString

Clear a length of data from a string. Takes a pointer to the source array and the number of characters to clear, as parameters.

void qsc_stringutils_clear_substring(char* dest, size_t length)

Compare Strings

Compare two strings for equivalence. Takes pointers for the two strings to compare, and the number of characters to compare as parameters, returns true if the arrays are equal.

bool qsc_stringutils_compare_strings(const char* str1, const char* str2, size_t length)

Concat Strings

Concatenate two strings. Takes the pointers to the destination buffer and input strings, and the destination length as parameters, and returns the size of the new string.

size_t qsc_stringutils_concat_strings(char* dest, size_t dstlen, const char* source)

Concat and Copy

Concatenate two strings and copy them to a third string. Takes the pointers to the destination buffer, the first and second substrings, and the destination length parameters, and returns the length of the new string.

size_t qsc_stringutils_concat_and_copy(char* dest, size_t dstlen, const char* str1, const char* str2)

Copy SubString

Copy a length of one string to another. Takes pointers to the destination buffer and source string, and the destination and source lengths as parameters, and returns the size of the new strings.

size_t qsc_stringutils_copy_substring(char* dest, size_t dstlen, const char* source, size_t srclen)

Copy String

Copy a source string to a destination string. Takes pointers to the source and destination arrays, and the destination length as parameters, and returns the new string size.

size_t qsc_stringutils_copy_string(char* dest, size_t dstlen, const char* source)

Find Char

Find a character position within a string. Takes a pointer to the source array, and the search token string as parameters, and returns the token position withing the source or -1 if the token is not found.

int64_t qsc_stringutils_find_char(const char* source, const char token)

Find String

Find a substrings position within a string. Takes a pointer to the source array, and the search token string as parameters, and returns the token position withing the source or -1 if the token is not found.

```
int64_t qsc_stringutils_find_string(const char* source, const char* token);
```

Byte To Hex

Convert a byte to a hexadecimal string.

```
int64_t qsc_stringutils_find_string(const char* source, const char* token);
```

Hex To Byte

Convert a hexadecimal string to a byte.

```
uint8_t qsc_stringutils_hex_to_byte(const char* hex)
```

Insert String

Inserts a substring into a string. Takes pointers to the destination and source strings, the destination length, and the insert offset as parameters, returns the new string length or -1 on failure.

```
int64_t qsc_stringutils_insert_string(char* dest, size_t dstlen, const char* source, size_t offset);
```

Is Alpha Numeric

Check that a string contains only alpha numeric ascii characters. Takes the source and source length as parameters, and returns true if the string is alpha-numeric.

No dedicated alphanumeric-validator function is exported; use the currently supported string utility interfaces.

Is Empty

Check if a string contains and characters.

bool qsc_stringutils_is_empty(const char* source)

Is Hex

Check that a string contains only hexadecimal ascii characters. Takes the source and source length as parameters, and returns true if the string is in hexadecimal.

bool qsc_stringutils_is_hex(const char* source, size_t srclen)

Is Numeric

Check that a string contains only numeric ascii characters. Takes the source and source length as parameters, and returns true if the string is in hexadecimal.

bool qsc_stringutils_is_numeric(const char* source, size_t srclen)

Join String

Join an array of strings to form one string. Takes an array of sub-strings and the array count as parameters, and returns the joined string.

The legacy join_string call is not exported in revision 1.4; use the documented string utility interface.

Register String

Join an array of strings to form one string.

char* qsc_stringutils_register_string(char** source, size_t count)

Remove Null Chars

Remove null characters from an array.

size_t qsc_stringutils_remove_null_chars(char* source, size_t srclen)

Reverse Find String

Find the position of a substring within a string, searching in reverse.

int64_t qsc_stringutils_reverse_find_string(const char* source, const char* token, size_t start)

Reverse SubString

Find a substring within a string, searching in reverse. Takes a pointer to the source string, and the search token as parameters, and returns the sub-string.

const char* qsc_stringutils_reverse_sub_string(const char* source, const char* token);

String Contains

Test if the string contains a substring. Takes a pointer to the source string, and the search token as parameters, and returns true if the source contains token.

bool qsc_stringutils_string_contains(const char* source, const char* token)

Strings Equal

Compare two strings for equality.

bool qsc_stringutils_strings_equal(const char* str1, const char* str2)

Split String

Split a string into a substring 2-dimensional array. Takes pointers to the source array, the delimiter, and the sub-string count, and returns the split string, or NULL on failure.

char** qsc_stringutils_split_string(char* source, const char* delim, size_t* count)

Split Strings

Split a string into two substrings.

void qsc_stringutils_split_strings(char* dest1, char* dest2, size_t destlen, const char* source, const char* token)

Sub String

Find a substring within a string. Takes a pointer to the source string, and the search token as parameters, and returns a pointer to the substring.

```
char* qsc_stringutils_sub_string(const char* source, const char* token)
```

String To Int

Convert a string to a 32-bit integer. Takes a pointer to the source string as a parameter, and returns a 32-bit signed integer.

```
int32_t qsc_stringutils_string_to_int(const char* source)
```

String Size

Get the character length of a string. Takes a pointer to the source string as a parameter, and returns the string length.

```
size_t qsc_stringutils_string_size(const char* source)
```

Int To String

Convert a 32-bit signed integer to a string. Takes the integer, a pointer to the destination string, and the destination length as parameters.

```
void qsc_stringutils_int_to_string(int32_t num, char* dest, size_t dstlen)
```

UInt32 To String

Convert a 32-bit unsigned integer to a string.

```
void qsc_stringutils_uint32_to_string(uint32_t num, char* dest, size_t destlen)
```

Int64 To String

Convert a 64-bit signed integer to a string.

```
void qsc_stringutils_int64_to_string(int64_t num, char* dest, size_t dstlen);
```

UInt64 To String

Convert a 64-bit unsigned integer to a string.

```
void qsc_stringutils_uint64_to_string(uint64_t num, char* dest, size_t dstlen);
```

To Lowercase

Convert a string to all lower -characters. Takes a pointer to the source string as a parameter.

```
void qsc_stringutils_to_lowercase(char* source)
```

To Uppercase

Convert a string to all upper-case characters. Takes a pointer to the source string as a parameter.

```
void qsc_stringutils_to_uppercase(char* source)
```

Trim Newline

Trim null and newline characters from a string. Takes a pointer to the source string as a parameter.

```
void qsc_stringutils_trim_newline(char* source)
```

Trim Spaces

Trim null and newline characters from a string. Takes a pointer to the source string as a parameter.

```
void qsc_stringutils_trim_spaces(char* source)
```

Whitespace Count

Count all the whitespaces in a string. Takes a pointer to the source string, and the source length as parameters.

size_t qsc_stringutils_whitespace_count(const char* source, size_t srclen)

Whitespace Filter

Remove all the whitespaces from a string. Takes pointers to the source and destination strings, and the source length as parameters.

size_t qsc_stringutils_whitespace_filter(const char* source, size_t srclen, char* dest);

16.8 System Utilities

Header:

sysutils.h

Description:

System specific functions; provides system specific statistics, counters, and feature availability information.

Constants:

Constant Name	Value	Purpose
QSC_SYSUTILS_SYSTEM_NAME_MAX	256	The system maximum name length.

Table 16.8a system constants

Structures:

The qsc_sysutils_drive_space_state structure.

Data Name	Data Type	Bit Length	Function
free	Uint64	0x40	The free drive space.

total	Uint64	0x40	The total drive space.
avail	Uint64	0x40	The available drive space.

Table 16.8b drive space structure

The qsc_sysutils_memory_statistics_state structure.

Data Name	Data Type	Bit Length	Function
phystotal	Uint64	0x40	The total physical memory.
physavail	Uint64	0x40	The available physical memory.
virttotal	Uint64	0x40	The total virtual memory.
virtavail	Uint64	0x40	The available virtual memory.

Table 16.1c drive space structure

API:

Computer Name

Get the computer string name. Takes a pointer to the name string buffer as a parameter, and returns the length of the computer name.

```
size_t qsc_sysutils_computer_name(char* name)
```

Drive Space

Get the system drive space statistics. Takes a pointer to the drive string, and the drive space state structure as parameters.

```
void qsc_sysutils_drive_space(const char* drive, qsc_sysutils_drive_space_state* state)
```

RdRand Available

Check if the system supports Intel RDRAND random seed generator. Returns true if the service is available on the system.

Check CPU instruction availability with qsc_cpuidex_features_set and the CPU features structure.

RdSeed Available

Check if the system supports Intel RDSEED random seed generator. Returns true if the service is available on the system.

Check CPU instruction availability with qsc_cpuidex_features_set and the CPU features structure.

RdSeed Available

Check if the system supports Intel RDSEED random seed generator. Returns true if the service is available on the system.

Check CPU instruction availability with qsc_cpuidex_features_set and the CPU features structure.

RdTsc Available

Check if the system has a high resolution RDTSC timer.

```
bool qsc_sysutils_rdtsc_available(void);
```

Memory Statistics

Get the memory statistics from the system. Takes a pointer to the memory statistics state as a parameter.

```
void qsc_sysutils_memory_statistics(qsc_sysutils_memory_statistics_state* state)
```

Process Id

Get the current process id. Returns the process id.

```
uint32_t qsc_sysutils_process_id(void);
```

User Name

Get the systems logged-on user name string. Takes the name buffer string as a parameter.

size_t qsc_sysutils_user_name(char* name)

System Uptime

Get the system uptime since boot.

uint64_t qsc_sysutils_system_uptime(void);

System Timestamp

Get the current high-resolution time-stamp. Returns the 64-bit timestamp.

uint64_t qsc_sysutils_system_timestamp(void);

User Identity

Get the users identity string. Takes pointers to the user-name string, and the output id strings as parameters.

void qsc_sysutils_user_identity(const char* name, char* id)

16.9 TimerEx

Header:

timerex.h

Constants:

Constant Name	Value	Purpose
QSC_TIMEREX_TIMESTAMP_MAX	80	The timestamp maximum byte size.

Table 16.9 timestamp maximum size

Description:

Timer functions.

Get Date

Get the calendar date from the current locale. Writes the time string to the output pameter.

```
void qsc_timerex_get_date(char output[QSC_TIMEREX_TIMESTAMP_MAX]);
```

Get DateTime

Get the calendar date and the time from the current locale. Writes the time string to the output pameter.

```
void qsc_timerex_get_datetime(char output[QSC_TIMEREX_TIMESTAMP_MAX]);
```

Get Time

Get the time from the current locale. Writes the time string to the output pameter.

```
void qsc_timerex_get_time(char output[QSC_TIMEREX_TIMESTAMP_MAX]);
```

Stopwatch Start

Returns the clock time at the start of a timed operation. Returns a clock_t time structure.

```
uint64_t qsc_timerex_stopwatch_start(void);
```

StopWatch Elapsed

Returns the time difference between the start and current time in milliseconds. Takes a clock_t structure as a parameter, and returns the elapsed milliseconds.

```
uint64_t qsc_timerex_stopwatch_elapsed(uint64_t start);
```

16.10 Time Stamp

Header:

timestamp.h

Description:

Timestamp function definitions.

Constants:

Constant Name	Value	Purpose
QSC_TIMESTAMP_EPOCH_START	1900	The system maximum name length.
QSC_TIMESTAMP_STRING_SIZE	20	The size of the timestamp string.
QSC_TIMESTAMP_SECONDS_PER_MINUTE	60	The seconds in a minute.
QSC_TIMESTAMP_SECONDS_PER_HOUR	3600	The seconds in an hour.
QSC_TIMESTAMP_SECONDS_PER_DAY	86400	The seconds in a day.

Table 16.10 timestamp constants

API:

Current Date

Get the calendar date from the current locale. Takes the output string as a parameter.

```
void qsc_timestamp_current_date(char output[QSC_TIMESTAMP_STRING_SIZE])
```

Current DateTime

Get the calendar date and time from the current locale. Timestamp string format is YYYY-MM-DD HH-MM-SS. Takes the output string as a parameter.

```
void qsc_timestamp_current_datetime(char output[QSC_TIMESTAMP_STRING_SIZE])
```

Current Time

Get the local time. Takes the output string as a parameter.

```
void qsc_timestamp_current_time(char output[QSC_TIMESTAMP_STRING_SIZE])
```

EpochTime Seconds

Get the date and time from the current locale in seconds from epoch. Returns the 64-bit seconds since epoch.

```
uint64_t qsc_timestamp_epochtime_seconds(void);
```

Time Struct To String

Convert a time structure to a date and time string. Timestamp string format is YYYY-MM-DD HH-MM-SS. Takes the output string and a pointer to the time struct as parameters.

```
void qsc_timestamp_time_struct_to_string(char output[QSC_TIMESTAMP_STRING_SIZE],
const struct tm* tstruct)
```

String To Time Struct

Convert a date and time string to a time structure. Timestamp string format must be YYYY-MM-DD HH-MM-SS. Takes the output string and a pointer to the time struct as parameters.

```
void qsc_timestamp_string_to_time_struct(struct tm* tstruct, const char
input[QSC_TIMESTAMP_STRING_SIZE])
```

DateTime Seconds Remaining

Compare a base datetime with another future datetime string, and return the difference in seconds. Takes the base-time string, and the comparison string as parameters, and returns the number of seconds in the date-time string.

```
uint64_t qsc_timestamp_datetime_seconds_remaining(const char
basetime[QSC_TIMESTAMP_STRING_SIZE], const char
comptime[QSC_TIMESTAMP_STRING_SIZE])
```

DateTime To Seconds

Convert the date-time string to a seconds from epoch unsigned 64-bit integer. Pass the datetime string in the input parameter, retiurns the time in seconds..

```
uint64_t qsc_timestamp_datetime_to_seconds (const char input[QSC_TIMESTAMP_STRING_SIZE])
```

Seconds To DateTime

Convert a second's count from epoch-time to a date-time string. Takes the number of seconds between the clock epoch time and now, and the output time string as parameters.

```
void qsc_timestamp_seconds_to_datetime(uint64_t tsec, char output[QSC_TIMESTAMP_STRING_SIZE])
```

16.11 Transpose

Header:

transpose.h

Description:

String and array transposition functions.

API:

Bytes To Native

Convert 32-bit integers in big-endian format to 8-bit integers. Takes a pointer to the 32-bit integer output and 8-bit input arrays, and the length as parameters.

```
void qsc_transpose_bytes_to_native(uint32_t* output, const uint8_t* input, size_t length)
```

Hex To Bin

Convert a hexadecimal string to a decimal 8-bit array. Takes pointers to the output and input arrays, and the length as parameters.

```
void qsc_transpose_hex_to_bin(uint8_t* output, const char* input, size_t length)
```

Native To Bytes

Convert 8-bit integers to 32-bit integers in big-endian format. Takes a pointer to the 8-bit integer output and32-bit input arrays, and the length as parameters.

```
void qsc_transpose_native_to_bytes(uint8_t* output, const uint32_t* input, size_t length)
```

String To Scalar

Convert an 8-bit character array to zero padded 32-bit scalar integers. Takes a pointer to the 32-bit integer output and 8-bit input arrays, and the length as parameters.

```
void qsc_transpose_string_to_scalar(uint32_t* output, const char* input, size_t length);
```

16.12 WinUtils

Header:

winutils.h

Description:

Windows operating system functions.

Enumerations:

The **qsc_winutils_registry_value_types** enumeration

Enumeration	Purpose
REG_SZ_TYPE	String value type.
REG_DWORD_TYPE	DWORD value type.

REG_QWORD_TYPE	QWORD value type.
REG_BINARY_TYPE	Binary value type.

Table 16.12a registry values enumeration

The qsc_winutils_service_states enumeration

Enumeration	Purpose
QSC_WINUTILS_SERVICE_START	Start the service.
QSC_WINUTILS_SERVICE_STOP	Stop the service..
QSC_WINUTILS_SERVICE_PAUSE	Pause the service.
QSC_WINUTILS_SERVICE_RESUME	Resume the service.

Table 16.12b service states enumeration

Constants:

Constant Name	Value	Purpose
QSC_WINTOOLS_ATTRIBUTES_BUFFER_SIZE	256	The file attributes buffer size.
QSC_WINTOOLS_NETSTAT_BUFFER_SIZE	1024	The network statistics buffer size.
QSC_WINTOOLS_NETSTAT_NAME_SIZE	256	The network statistics name size.
QSC_WINTOOLS_PROCESS_LIST_SIZE	16384	The process list buffer size.
QSC_WINTOOLS_REGISTRY_BUFFER_SIZE	1024	The registry buffer size.
QSC_WINTOOLS_REGISTRY_LIST_SIZE	8192	The registry list buffer size.
QSC_WINTOOLS_RUNAS_BUFFER_SIZE	260	The runas buffer size.
QSC_WINTOOLS_SERVICE_LIST_SIZE	16384	The service list buffer size
QSC_WINTOOLS_SERVICE_BUFFER_SIZE	512	The service buffer size.
QSC_WINTOOLS_SERVICE_LIST_DESCRIPTION	NA	Include the service descriptions in the service list output.
QSC_WINTOOLS_SERVICE_LIST_ACTIVE_ONLY	NA	Only include running services when listing services

Table 16.12c timestamp constants

API:

File Get Attributes

Get a string of file attributes. Takes the path string as a parameter, and outputs the string to the result array.

```
size_t qsc_winutils_file_get_attributes(char* result, size_t reslen, const char* path)
```

File Set Attributes

Set a file attribute. Takes the path string as a parameter, and the attribute string. Valid attributes are readonly, hidden, system, archive, normal, temporary, offline, noindex, encrypted.

```
bool qsc_winutils_file_set_attribute(const char* path, const char* attr)
```

Network Statistics

Get a list of network statistics including address and adaptor information. The result string receives the formatted printable list. The function returns the size of the list string.

```
size_t qsc_winutils_network_statistics(char* result, size_t reslen)
```

Process List

Get a list of running system processes. The result string receives the formatted printable list. The function returns the size of the list string.

```
size_t qsc_winutils_process_list(char* result, size_t reslen)
```

Process Terminate

Terminate a process. Pass the process name parameter. The function returns true if the process was terminated.

```
bool qsc_winutils_process_terminate(const char* name)
```

Registry Value Add

Add a value to a registry subkey. The keypath specifies the full string path including the root key. The value string contains the value, and the vtype specifies the type of value. The function returns true if successful.

```
bool qsc_winutils_registry_key_add(const char* keypath, const char* value,
qsc_winutils_registry_value_types vtype)
```

Registry Key Delete

Delete a subkey and its values from the registry. The function returns true if successful.

```
bool qsc_winutils_registry_key_delete(const char* keypath)
```

Registry Key List

Copy a list of subkeys under a registry key. The keypath contains the full path to the key including the root key. The result string receives the formatted printable list. The function returns the size of the list string.

```
size_t qsc_winutils_registry_key_list(char* result, size_t reslen, const char* keypath)
```

Run Executable

Run an executable. The path specifies the full path to the executable. The function returns true if successful.

```
bool qsc_winutils_run_executable(const char* path)
```

Run As

Run an executable under a user account. The expath specifies the full path to the executable. The user and password are the users credentials. The function returns true if successful.

bool qsc_winutils_run_as_user(const char* user, const char* password, const char* expath)

Service List

Get a list of running system services. The result string receives the formatted printable list. The function returns the size of the list string.

size_t qsc_winutils_service_list(char* result, size_t reslen)

Service List Size

Get the service list size.

size_t qsc_winutils_service_list_size(void);

Service List State

Change a service state. Pass the service name and the desired state in as parameters. The function returns true on success.

bool qsc_winutils_service_state(const char* name, qsc_winutils_service_states state)

User List

Get a list of system user accounts. The result string receives the formatted printable list. The function returns the size of the list string.

size_t qsc_winutils_user_list(char* result, size_t reslen)

User Current

Get the name of the logged-in user account. The result string receives the list name and group. The function returns the size of the string.

size_t qsc_winutils_current_user(char* result, size_t reslen)

16.12 Additional Utility, Encoding and System APIs

Header: arrayutils.h

qsc_arrayutils_try_parse_uint16

Purpose: Strict 16-bit decimal conversion.

bool qsc_arrayutils_try_parse_uint16(const char* str, size_t slen, uint16_t* value);

Parameters: str: Caller-owned input containing decimal digits and optional final NUL.; slen: Accessible input byte count.; value: Caller-owned output set only on success..

Return: True for a complete uint16_t value; false for overflow, invalid or empty input.

Notes: Allocation-free and thread-safe; does not consume or retain input.

qsc_arrayutils_try_parse_uint32

Purpose: Strict 32-bit decimal conversion.

bool qsc_arrayutils_try_parse_uint32(const char* str, size_t slen, uint32_t* value);

Parameters: str: Caller-owned input containing decimal digits and optional final NUL.; slen: Accessible input byte count.; value: Caller-owned output set only on success..

Return: True for a complete uint32_t value; false for overflow, invalid or empty input.

Notes: Allocation-free and thread-safe; does not consume or retain input.

qsc_arrayutils_try_parse_uint64

Purpose: Strict 64-bit decimal conversion.

bool qsc_arrayutils_try_parse_uint64(const char* str, size_t slen, uint64_t* value);

Parameters: str: Caller-owned input containing decimal digits and optional final NUL.; slen: Accessible input byte count.; value: Caller-owned output set only on success..

Return: True for a complete uint64_t value; false for overflow, invalid or empty input.

Notes: Allocation-free and thread-safe; does not consume or retain input.

qsc_arrayutils_try_parse_uint8

Purpose: Strictly parse a decimal unsigned integer without overflow or partial acceptance.

bool qsc_arrayutils_try_parse_uint8(const char* str, size_t slen, uint8_t* value);

Parameters: str: Input data, not necessarily terminated; a single final NUL is permitted.; slen: Available input bytes, including an optional final NUL.; value: Output on success; unchanged on failure; caller-owned storage..

Return: True only when all digits are valid and the result fits the destination.

Notes: No allocations; thread safe. Failure includes NULL pointers and empty input.

qsc_arrayutils_uint16_to_hex

Purpose: Convert a 16-bit unsigned integer to a hexadecimal string. Converts the given uint16_t value into a four-digit hexadecimal representation, writing the result to the provided output buffer.

void qsc_arrayutils_uint16_to_hex(char* output, size_t otplen, uint16_t value);

Parameters: output: [char*] Pointer to the output character array.; otplen: [size_t] The length of the output buffer in bytes.; value: [uint16_t] The 16-bit unsigned integer to convert..

Return: None (void).

qsc_arrayutils_uint32_to_hex

Purpose: Convert a 32-bit unsigned integer to a hexadecimal string. Converts the given uint32_t value into an eight-digit hexadecimal representation, writing the result to the provided output buffer.

void qsc_arrayutils_uint32_to_hex(char* output, size_t otplen, uint32_t value);

Parameters: output: [char*] Pointer to the output character array.; otplen: [size_t] The length of the output buffer in bytes.; value: [uint32_t] The 32-bit unsigned integer to convert..

Return: None (void).

qsc_arrayutils_uint64_to_hex

Purpose: Convert a 64-bit unsigned integer to a hexadecimal string. Converts the given uint64_t value into a sixteen-digit hexadecimal representation, writing the result to the provided output buffer.

void qsc_arrayutils_uint64_to_hex(char* output, size_t otplen, uint64_t value);

Parameters: output: [char*] Pointer to the output character array.; otplen: [size_t] The length of the output buffer in bytes.; value: [uint64_t] The 64-bit unsigned integer to convert..

Return: None (void).

Header: asn1.h

qsc_asn1_child_at

Purpose: Gets a child element by index.

const qsc_encoding_ber_element* qsc_asn1_child_at(const qsc_encoding_ber_element* element, size_t index);

Parameters: element: [const qsc_encoding_ber_element*] The parent constructed element.; index: [size_t] The child index..

Return: [const qsc_encoding_ber_element*] Returns a pointer to the requested child, or NULL on error.

qsc_asn1_child_count

Purpose: Gets the number of child elements in a constructed ASN.1 element.

size_t qsc_asn1_child_count(const qsc_encoding_ber_element* element);

Parameters: element: [const qsc_encoding_ber_element*] The constructed element..

Return: [size_t] Returns the number of child elements, or 0 if the input is invalid.

qsc_asn1_decode_bit_string

Purpose: Decodes a BIT STRING to a normalized descriptor.

qsc_asn1_status qsc_asn1_decode_bit_string(const qsc_encoding_ber_element* element, qsc_asn1_bit_string* value);

Parameters: element: [const qsc_encoding_ber_element*] The BIT STRING element.; value: [qsc_asn1_bit_string*] Receives the normalized BIT STRING descriptor..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_decode_boolean

Purpose: Decodes a BOOLEAN value.

qsc_asn1_status qsc_asn1_decode_boolean(const qsc_encoding_ber_element* element, bool* value);

Parameters: element: [const qsc_encoding_ber_element*] The BOOLEAN element.; value: [bool*] Receives the decoded boolean value..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_decode_integer_u64

Purpose: Decodes an ASN.1 INTEGER into a 64-bit unsigned value. This function is a compatibility wrapper around qsc_asn1_decode_uint64(). The INTEGER must be non-negative and must fit within a 64-bit unsigned integer.

qsc_asn1_status qsc_asn1_decode_integer_u64(const qsc_encoding_ber_element* element, uint64_t* value);

Parameters: element: [const qsc_encoding_ber_element*] Pointer to the INTEGER element.; value: [uint64_t*] Receives the decoded integer value..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS if decoding succeeds.

qsc_asn1_decode_null

Purpose: Validates that an ASN.1 NULL element is correctly encoded.

qsc_asn1_status qsc_asn1_decode_null(const qsc_encoding_ber_element* element);

Parameters: element: [const qsc_encoding_ber_element*] The NULL element..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS if the NULL encoding is valid.

qsc_asn1_decode_octet_string

Purpose: Copies the contents of an OCTET STRING into a caller-provided buffer.

qsc_asn1_status qsc_asn1_decode_octet_string(const qsc_encoding_ber_element* element, uint8_t* output, size_t otplen, size_t* outlen);

Parameters: element: [const qsc_encoding_ber_element*] The OCTET STRING element.; output: [uint8_t*] The destination byte array.; otplen: [size_t] The size of the destination byte array.; outlen: [size_t*] Receives the number of bytes copied..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_decode_oid

Purpose: Decodes an OBJECT IDENTIFIER value.

Details: Decodes every subidentifier using minimal base-128 form, including a multi-octet first combined subidentifier for large second arcs under arc 2.

qsc_asn1_status qsc_asn1_decode_oid(const qsc_encoding_ber_element* element, qsc_asn1_oid* oid);

Parameters: element: [const qsc_encoding_ber_element*] The OBJECT IDENTIFIER element.; oid: [qsc_asn1_oid*] Receives the decoded object identifier..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_decode_string

Purpose: Copies the contents of a supported ASN.1 string type into a character buffer.

Details: This function supports the common string types used by X.509 distinguished names, including PrintableString, UTF8String, IA5String, T61String, VisibleString, BMPString, and UniversalString. For multi-byte encodings such as BMPString and UniversalString, this helper accepts only values in the 7-bit ASCII subset and rejects non-ASCII code points.

qsc_asn1_status qsc_asn1_decode_string(const qsc_encoding_ber_element* element, char* output, size_t otplen, size_t* outlen);

Parameters: element: [const qsc_encoding_ber_element*] The ASN.1 string element.; output: [char*] The destination character array.; otplen: [size_t] The size of the

destination character array.; outlen: [size_t*] Receives the length of the copied string, excluding the terminator..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_decode_time

Purpose: Decodes a UTCTime or GeneralizedTime value to a normalized structure.

qsc_asn1_status qsc_asn1_decode_time(const qsc_encoding_ber_element* element, qsc_asn1_time* value);

Parameters: element: [const qsc_encoding_ber_element*] The ASN.1 time element.; value: [qsc_asn1_time*] Receives the normalized time value..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_decode_uint64

Purpose: Decodes a non-negative INTEGER into a 64-bit unsigned integer.

qsc_asn1_status qsc_asn1_decode_uint64(const qsc_encoding_ber_element* element, uint64_t* value);

Parameters: element: [const qsc_encoding_ber_element*] The INTEGER element.; value: [uint64_t*] Receives the decoded integer value..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_der_decode_exact

Purpose: Decode a complete DER element and reject trailing octets.

Details: This helper wraps qsc_encoding_der_decode_element() and requires the input buffer to contain exactly one complete DER element. Inputs with trailing bytes are rejected so callers can fail closed on ambiguous or concatenated encodings.

qsc_asn1_status qsc_asn1_der_decode_exact(const uint8_t* der, size_t derlen, qsc_encoding_ber_element** element);

Parameters: der: [const uint8_t*] Pointer to the DER input buffer.; derlen: [size_t] Length of the DER input buffer in bytes.; element: [qsc_encoding_ber_element**] Receives the decoded element tree on success..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_der_get_child_region

Purpose: Returns the exact encoded DER slice of a child element within a parent element.

Details: The parent buffer must contain exactly one DER element. The function walks the encoded child sequence inside the parent content and returns the exact byte range for the requested child element. This is intended for preserving signed substructures such as TBSCertificate, CertificationRequestInfo, and TBSCertList without reconstructing them from normalized fields.

qsc_asn1_status qsc_asn1_der_get_child_region(const uint8_t* der, size_t derlen, size_t index, const uint8_t** child, size_t* childlen);

Parameters: der: [const uint8_t*] Pointer to the complete parent DER element.; derlen: [size_t] Length of the parent DER element in bytes.; index: [size_t] Zero-based child index inside the parent content.; child: [const uint8_t**] Receives a pointer to the exact child DER encoding.; childlen: [size_t*] Receives the length of the child DER encoding in bytes..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_der_size

Purpose: Computes the encoded DER size of a decoded ASN.1 element.

Details: This helper calculates the size of the element as it would appear in DER, including the tag, length field, and content octets. For decoded BER trees, this is useful when reconstructing exact spans of explicit wrappers or when checking structure sizes during certificate parsing.

qsc_asn1_status qsc_asn1_der_size(const qsc_encoding_ber_element* element, size_t* size);

Parameters: element: [const qsc_encoding_ber_element*] The ASN.1 element.; size: Receives the calculated byte length of the DER encoding..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_element_is_tag

Purpose: Tests whether an ASN.1 element matches an expected tag description.

bool qsc_asn1_element_is_tag(const qsc_encoding_ber_element* element, uint8_t tagclass, bool constructed, uint32_t tagnumber);

Parameters: element: [const qsc_encoding_ber_element*] The element to test.; tagclass: [uint8_t] The expected tag class.; constructed: [bool] The expected constructed flag.; tagnumber: [uint32_t] The expected tag number..

Return: [bool] Returns true if the element matches the requested tag description.

qsc_asn1_find_context_child

Purpose: Gets an optional context-specific child by tag number.

const qsc_encoding_ber_element* qsc_asn1_find_context_child(const qsc_encoding_ber_element* element, uint32_t tagnumber);

Parameters: element: [const qsc_encoding_ber_element*] The parent constructed element.; tagnumber: [uint32_t] The requested context-specific tag number..

Return: [const qsc_encoding_ber_element*] Returns a pointer to the requested child, or NULL if no matching child exists.

qsc_asn1_get_child

Purpose: Retrieves a child element from a constructed ASN.1 element. This function returns the child element at the specified index. It is a compatibility alias for qsc_asn1_child_at().

const qsc_encoding_ber_element* qsc_asn1_get_child(const qsc_encoding_ber_element* element, size_t index);

Parameters: element: [const qsc_encoding_ber_element*] Pointer to the parent element.; index: [size_t] The zero-based child index..

Return: [const qsc_encoding_ber_element*] Returns a pointer to the child element, or NULL if the index is invalid.

qsc_asn1_get_explicit_child

Purpose: Gets the sole child of an explicitly tagged context-specific element.

qsc_asn1_status qsc_asn1_get_explicit_child(const qsc_encoding_ber_element* element, const qsc_encoding_ber_element** child);

Parameters: element: [const qsc_encoding_ber_element*] The context-specific wrapper element.; child: [const qsc_encoding_ber_element**] Receives the wrapped child element..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_is_boolean

Purpose: Tests whether an ASN.1 element is a BOOLEAN. The function verifies that the supplied element has the universal BOOLEAN tag and is encoded as a primitive value.

bool qsc_asn1_is_boolean(const qsc_encoding_ber_element* element);

Parameters: element: [const qsc_encoding_ber_element*] Pointer to the ASN.1 element..

Return: [bool] Returns true if the element is a BOOLEAN.

qsc_asn1_is_integer

Purpose: Tests whether an ASN.1 element is an INTEGER. The function verifies that the supplied element has the universal INTEGER tag and is encoded as a primitive value.

bool qsc_asn1_is_integer(const qsc_encoding_ber_element* element);

Parameters: element: [const qsc_encoding_ber_element*] Pointer to the ASN.1 element..

Return: [bool] Returns true if the element is an INTEGER.

qsc_asn1_is_null

Purpose: Tests whether an ASN.1 element is a NULL value. The function verifies that the supplied element has the universal NULL tag and is encoded as a primitive value.

bool qsc_asn1_is_null(const qsc_encoding_ber_element* element);

Parameters: element: [const qsc_encoding_ber_element*] Pointer to the ASN.1 element..

Return: [bool] Returns true if the element is a NULL.

qsc_asn1_is_oid

Purpose: Tests whether an ASN.1 element is an OBJECT IDENTIFIER. The function verifies that the supplied element has the universal OBJECT IDENTIFIER tag and is encoded as a primitive value.

bool qsc_asn1_is_oid(const qsc_encoding_ber_element* element);

Parameters: element: [const qsc_encoding_ber_element*] Pointer to the ASN.1 element..

Return: [bool] Returns true if the element is an OBJECT IDENTIFIER.

qsc_asn1_oid_are_equal

Purpose: Compares two decoded OBJECT IDENTIFIER values for equality.

bool qsc_asn1_oid_are_equal(const qsc_asn1_oid* a, const qsc_asn1_oid* b);

Parameters: a: [const qsc_asn1_oid*] The first object identifier.; b: [const qsc_asn1_oid*] The second object identifier..

Return: [bool] Returns true if the object identifiers are equal.

qsc_asn1_oid_compare

Purpose: Compares two decoded ASN.1 OBJECT IDENTIFIER values. This function performs an exact comparison of two OID structures. It is a compatibility wrapper around qsc_asn1_oid_are_equal() used by the X.509 implementation.

bool qsc_asn1_oid_compare(const qsc_asn1_oid* a, const qsc_asn1_oid* b);

Parameters: a: [const qsc_asn1_oid*] Pointer to the first OID.; b: [const qsc_asn1_oid*] Pointer to the second OID..

Return: [bool] Returns true if both OIDs are identical.

qsc_asn1_oid_to_string

Purpose: Converts a decoded OBJECT IDENTIFIER to dotted-decimal text.

qsc_asn1_status qsc_asn1_oid_to_string(const qsc_asn1_oid* oid, char* output, size_t otplen);

Parameters: oid: [const qsc_asn1_oid*] The decoded object identifier.; output: [char*] The destination character array.; otplen: [size_t] The size of the destination character array..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_require_sequence

Purpose: Validates that an element is a constructed ASN.1 SEQUENCE. The function verifies that the supplied element has the universal SEQUENCE tag and that the number of child elements lies within the specified inclusive bounds. This helper is used by the X.509 parser to validate DER structures before accessing their child elements.

qsc_asn1_status qsc_asn1_require_sequence(const qsc_encoding_ber_element* element, size_t minchildren, size_t maxchildren);

Parameters: element: [const qsc_encoding_ber_element*] Pointer to the ASN.1 element.; minchildren: [size_t] Minimum allowed number of children.; maxchildren: [size_t] Maximum allowed number of children..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS if validation succeeds.

qsc_asn1_require_tag

Purpose: Validates that an ASN.1 element matches an expected tag description.

qsc_asn1_status qsc_asn1_require_tag(const qsc_encoding_ber_element* element, uint8_t tagclass, bool constructed, uint32_t tagnumber);

Parameters: element: [const qsc_encoding_ber_element*] The element to validate.; tagclass: [uint8_t] The expected tag class.; constructed: [bool] The expected constructed flag.; tagnumber: [uint32_t] The expected tag number..

Return: [qsc_asn1_status] Returns QSC_ASN1_STATUS_SUCCESS on success.

qsc_asn1_time_compare

Purpose: Compares two normalized ASN.1 times.

int32_t qsc_asn1_time_compare(const qsc_asn1_time* a, const qsc_asn1_time* b);

Parameters: a: [const qsc_asn1_time*] The first time value.; b: [const qsc_asn1_time*] The second time value..

Return: [int32_t] Returns -1 if a is earlier than b, 0 if equal, and 1 if later.

Header: cjp.h

qsc_cjp_available

Purpose: Test whether the platform exposes a usable high-resolution timer.

bool qsc_cjp_available(void);

Return: [bool] Returns true when the CJP timing source passes the timer test.

qsc_cjp_generate

Purpose: Generate an array of random bytes using the CPU jitter entropy provider.

bool qsc_cjp_generate(uint8_t* output, size_t length);

Parameters: output: [uint8_t*] Pointer to the output byte array.; length: [size_t] The number of bytes to generate..

Return: [bool] Returns true if the entropy generation was successful; false otherwise.

qsc_cjp_uint16

Purpose: Generate a random 16-bit unsigned integer using the CPU jitter entropy provider.

uint16_t qsc_cjp_uint16(void);

Return: [uint16_t] Returns a random 16-bit unsigned integer.

qsc_cjp_uint32

Purpose: Generate a random 32-bit unsigned integer using the CPU jitter entropy provider.

uint32_t qsc_cjp_uint32(void);

Return: [uint32_t] Returns a random 32-bit unsigned integer.

qsc_cjp_uint64

Purpose: Generate a random 64-bit unsigned integer using the CPU jitter entropy provider.

uint64_t qsc_cjp_uint64(void);

Return: [uint64_t] Returns a random 64-bit unsigned integer.

Header: consoleutils.h

qsc_consoleutils_get_quoted_string

Purpose: Extract a quoted string from input. Searches the input for a quoted substring (using either double or single quotes) and copies it to the output array.

size_t qsc_consoleutils_get_quoted_string(char* output, const char* input, size_t maxlen);

Parameters: output: [char*] A pointer to the destination array for the extracted string.; input: [const char*] A pointer to the input string.; maxlen: [size_t] The maximum number of characters to extract..

Return: [size_t] Returns the number of characters in the quoted string.

qsc_consoleutils_line_equals

Purpose: Compare two lines of text for equivalence.

bool qsc_consoleutils_line_equals(const char* line1, const char* line2);

Parameters: line1: [const char*] A pointer to the first constant input string.; line2: [const char*] A pointer to the second constant input string..

Return: [bool] Returns true if the strings are equal; otherwise, false.

qsc_consoleutils_print_array

Purpose: Print a byte array to the console. Prints the array in a formatted manner with a specified number of characters per line.

void qsc_consoleutils_print_array(const uint8_t* input, size_t inplen, size_t linelen);

Parameters: input: [const uint8_t*] A pointer to the constant byte array.; inplen: [size_t] The number of bytes in the array.; linelen: [size_t] The number of bytes to print per line..

Return: None (void).

qsc_consoleutils_send_enter

Purpose: Send an "enter" (newline) command to the console.

void qsc_consoleutils_send_enter(void);

Return: None (void).

Header: encoding.h

qsc_encoding_base64_encoded_size

Purpose: Compute the character count required to hold the Base64 encoding of length input bytes, including trailing '=' padding. The returned value does not include the null terminator; allocate qsc_encoding_base64_encoded_size(length) + 1 bytes for the output buffer passed to qsc_encoding_base64_encode.

size_t qsc_encoding_base64_encoded_size(size_t length);

Parameters: length: [size_t] Number of raw input bytes..

Return: [size_t] Number of Base64 characters required (always a multiple of four), or 0 if length is zero.

qsc_encoding_base64_stream_decode_final

Purpose: Finalize an incremental Base64 decoding stream.

bool qsc_encoding_base64_stream_decode_final(qsc_encoding_base64_stream_state* state);

Parameters: state: Mutable initialized decoder state..

Return: True if no incomplete quartet remains, false otherwise.

Notes: On success pending bytes are cleared and further updates are rejected.

qsc_encoding_base64_stream_decode_update

Purpose: Decode a chunk of Base64, emitting only complete quartets.

bool qsc_encoding_base64_stream_decode_update(qsc_encoding_base64_stream_state* state, uint8_t* output, size_t otplen, const char* input, size_t inplen, size_t* written);

Parameters: state: Mutable initialized decoder state.; output: Caller-owned destination; may be NULL if no bytes are produced.; otplen: Available destination capacity.; input: Length-bounded Base64 input, may be NULL if inplen is zero.; inplen: Number of input characters.; written: Receives decoded byte count, or zero on failure..

Return: True on success; false on invalid syntax, capacity, or state.

Notes: Each call validates all new input before writing or advancing state. The stream accepts the same padding-bit conventions as the legacy decoder. Once a padded quartet arrives, subsequent nonempty updates are rejected. Stateless across independent state instances; no allocations.

qsc_encoding_base64_stream_encode_final

Purpose: Write the remaining padded Base64 quartet and finalize encoding.

bool qsc_encoding_base64_stream_encode_final(qsc_encoding_base64_stream_state* state, char* output, size_t otplen, size_t* written);

Parameters: state: Mutable initialized encoder state.; output: Destination for up to four characters; may be NULL when empty.; otplen: Destination capacity in bytes.; written: Receives zero or four output characters..

Return: True on success; false for invalid state or insufficient space.

Notes: On failure, output and state are unchanged; no null terminator is written.

qsc_encoding_base64_stream_encode_update

Purpose: Encode a chunk of bytes to complete Base64 quartets.

bool qsc_encoding_base64_stream_encode_update(qsc_encoding_base64_stream_state* state, char* output, size_t otplen, const uint8_t* input, size_t inplen, size_t* written);

Parameters: state: Mutable initialized encoder state, owned by caller.; output: Caller-owned destination; NULL is allowed if no output is produced.; otplen: Available destination bytes; no null terminator is written.; input: Source bytes; may be NULL only when inplen is zero. No overlap with output.; inplen: Source byte count.; written: Receives output byte count, or zero on failure..

Return: True on success. On failure the state and output remain unchanged.

Notes: Stateful, allocation-free, not safe for concurrent access to one state.

qsc_encoding_base64_stream_initialize

Purpose: Initialize an incremental Base64 encoding or decoding stream.

void qsc_encoding_base64_stream_initialize(qsc_encoding_base64_stream_state* state);

Parameters: state: Caller-owned stream state; must not be NULL..

Return: None (void).

Notes: Clears prior buffered data, does not allocate, and is not thread safe when the same state is shared between threads.

qsc_encoding_ber_free_element

Purpose: Recursively free a decoded BER element tree. Releases all memory associated with element and its descendants: for constructed elements the children array and each child element are freed depth-first; for primitive elements the value buffer is freed. The element itself is then freed. Passing NULL is safe and is a no-op.

void qsc_encoding_ber_free_element(qsc_encoding_ber_element* element);

Parameters: element: [qsc_encoding_ber_element*] Root of the element tree to free..

Return: None (void).

qsc_encoding_der_validate

Purpose: Validate the first complete DER element without allocating a tree.

bool qsc_encoding_der_validate(const uint8_t* buffer, size_t buflen, size_t* consumed);

Parameters: buffer: Caller-owned, length-bounded DER input (read only).; buflen: Number of accessible bytes.; consumed: Receives the validated element size, or zero on failure..

Return: True for structurally valid definite-length DER with minimally encoded tags and lengths; false for malformed input. ASN.1 type-specific canonical restrictions and schema validation are not performed.

Notes: Trailing bytes are not rejected; require *consumed == buflen for an exact document. The function performs no allocations and is thread safe.

Header: fileutils.h

qsc_fileutils_close

Purpose: Close a file.

```
void qsc_fileutils_close(FILE* fp);
```

Parameters: fp: [FILE*] The file pointer..

Return: None (void).

qsc_fileutils_create

Purpose: Create a new file.

```
bool qsc_fileutils_create(const char* fpath);
```

Parameters: fpath: [const char*] The full fpath to the file to be created..

Return: [bool] Returns true for success.

qsc_fileutils_delete

Purpose: Delete a file.

```
bool qsc_fileutils_delete(const char* fpath);
```

Parameters: fpath: [const char*] The full fpath to the file to be deleted..

Return: [bool] Returns true for success.

qsc_fileutils_erase

Purpose: Erase a file's contents.

bool qsc_fileutils_erase(const char* fpath);

Parameters: fpath: [const char*] The full fpath to the file..

Return: [bool] Returns true for success.

qsc_fileutils_exists

Purpose: Test to see if a file exists.

bool qsc_fileutils_exists(const char* fpath);

Parameters: fpath: [const char*] The fully qualified fpath to the file..

Return: [bool] Returns true if the file exists.

qsc_fileutils_flush

Purpose: Flush a file's contents to disk.

void qsc_fileutils_flush(FILE* fp);

Parameters: fp: [FILE*] The file pointer..

Return: None (void).

qsc_fileutils_get_access

Purpose: Test a user's access right to a file.

bool qsc_fileutils_get_access(const char* fpath, qsc_fileutils_access_rights level);

Parameters: fpath: [const char*] The fully qualified fpath to the file.; level: [qsc_fileutils_access_rights] The access level to check..

Return: [bool] Returns true if the specified access level is present.

qsc_fileutils_get_extension

Purpose: Get the file extension.

size_t qsc_fileutils_get_extension(char* extension, size_t extlen, const char* fpath);

Parameters: extension: [char*] The output extension buffer.; extlen: [size_t] The length of the extension buffer.; fpath: [const char*] The full fpath to the file..

Return: [size_t] Returns the length of the file extension.

qsc_fileutils_get_line

Purpose: Reads a line of text from a formatted file.

int64_t qsc_fileutils_get_line(char** line, size_t* length, FILE* fp);

Parameters: line: [char**] Receives a newly allocated NUL-terminated line, without LF; caller frees using qsc_memutils_alloc_free().; length: [size_t*] Receives allocated capacity in bytes; set to zero on failure/EOF.; fp: [FILE*] Open stream owned by the caller; its current position advances through the line..

Return: [int64_t] The line's byte count, including the consumed LF when present, or -1 on EOF/error. Output pointers are reset on failure. The input stream must not be used concurrently.

Notes: The line buffer must be freed after the last call.

qsc_fileutils_get_size

Purpose: Get the file size in bytes.

size_t qsc_fileutils_get_size(const char* fpath);

Parameters: fpath: [const char*] The full fpath to the file..

Return: [size_t] Returns the size of the file in bytes.

qsc_fileutils_get_working_directory

Purpose: Get the working directory fpath.

bool qsc_fileutils_get_working_directory(char* fpath, size_t flen);

Parameters: fpath: [char*] The output buffer for the current working directory.; flen: [size_t] The length of the fpath string..

Return: [bool] True only if the complete NUL-terminated working directory fits in the supplied buffer. The function does not allocate storage and is safe for independent concurrent callers.

qsc_fileutils_open

Purpose: Open a file and return the handle.

FILE* qsc_fileutils_open(const char* fpath, qsc_fileutils_mode mode, bool binary);

Parameters: fpath: [const char*] The fully qualified file fpath.; mode: [qsc_fileutils_mode] The file access mode.; binary: [bool] Open the file in binary mode (true) or ANSI mode (false)..

Return: [FILE*] Returns the file handle, or NULL on failure.

qsc_fileutils_read

Purpose: Read data from a file into an output stream.

size_t qsc_fileutils_read(char* output, size_t otplen, size_t position, FILE* fp);

Parameters: output: [char*] The output buffer.; otplen: [size_t] The size of the output buffer.; position: [size_t] The starting position within the file.; fp: [FILE*] The file pointer..

Return: [size_t] Returns the number of bytes read.

qsc_fileutils_sync

Purpose: Flush and synchronize a file's contents to stable storage.

bool qsc_fileutils_sync(FILE* fp);

Parameters: fp: [FILE*] The file pointer..

Return: [bool] Returns true when both the C stream and operating-system file handle were synchronized.

Header: folderutils.h

qsc_folderutils_create_directory_tree

Purpose: Create a new folder tree. Creates a directory path.

bool qsc_folderutils_create_directory_tree(const char path[QSC_SYSTEM_MAX_PATH]);

Parameters: path: [const char*] The full path including the new folder names..

Return: [bool] Returns true if the folder is created.

qsc_folderutils_directory_list

Purpose: Write a list of directories to a string, delineated by newline characters. Constructs a string listing the directories within the specified starting directory.

size_t qsc_folderutils_directory_list(char* result, size_t reslen, const char* directory);

Parameters: result: [char*] The output result string.; reslen: [size_t] The length of the output string buffer.; directory: [const char*] The starting directory..

Return: [size_t] The number of entry bytes written, excluding the trailing NUL. If an entry does not fit it is omitted and enumeration stops. Output is NUL terminated when reslen is nonzero. Operates on the caller's buffer without heap allocation. Entries are in filesystem order.

qsc_folderutils_enumerate

Purpose: Enumerate directory entries without creating a concatenated result buffer. Skips the dot and dot-dot entries. Entries are reported in operating-system enumeration order. No user-owned memory is allocated by this function. Multiple independent calls have no shared state.

bool qsc_folderutils_enumerate(const char* directory, qsc_folderutils_entry_callback callback, void* context);

Parameters: directory: NUL-terminated directory path; not retained.; callback: Non-NULL function receiving each entry name, type, and context.; context: Caller-owned context; may be NULL. Ownership is not transferred..

Return: True if enumeration completes, or false on open/enumeration failure or callback cancellation.

Header: intutils.h

qsc_intutils_be8increment_rfc3686

Purpose: Increment only the low 32 bits (last 4 bytes) of a BE counter block, per RFC 3686: CTRBLK := NONCE(32) ? IV(64) ? COUNT(32) COUNT ? (COUNT + 1) mod 2^32

Details: Increment only the low 32 bits (last 4 bytes) of a BE counter block, per RFC 3686: CTRBLK := NONCE(32) ? IV(64) ? COUNT(32) COUNT ? (COUNT + 1) mod 2^32

void qsc_intutils_be8increment_rfc3686(uint8_t* output, size_t otplen);

Parameters: output: [uint8_t*] The pointer to the counter block (length ? 4); otplen: [size_t] The total length of the counter buffer in bytes.

Return: None (void).

qsc_intutils_bit_reverse

Purpose: Reverse the bits of an integer.

size_t qsc_intutils_bit_reverse(size_t x, uint32_t bits);

Parameters: x: The integer.; bits: [size_t] The number of bits to reverse..

Return: [size_t] Returns the integer with its bits reversed.

qsc_intutils_bit_reverse_u16

Purpose: Reverse the bits of a 16-bit integer.

uint16_t qsc_intutils_bit_reverse_u16(uint16_t x);

Parameters: x: [uint16_t] The 16-bit integer..

Return: [uint16_t] Returns the 16-bit integer with its bits reversed.

qsc_intutils_bit_reverse_u32

Purpose: Reverse the bits of a 32-bit integer.

uint32_t qsc_intutils_bit_reverse_u32(uint32_t x);

Parameters: x: [uint32_t] The 32-bit integer..

Return: [uint32_t] Returns the 32-bit integer with its bits reversed.

qsc_intutils_bit_reverse_u64

Purpose: Reverse the bits of a 64-bit integer.

uint64_t qsc_intutils_bit_reverse_u64(uint64_t x);

Parameters: x: [uint64_t] The 64-bit integer..

Return: [uint64_t] Returns the 64-bit integer with its bits reversed.

qsc_intutils_calculate_abs

Purpose: Computes the absolute value of a double.

double qsc_intutils_calculate_abs(double a);

Parameters: a: [double] The input value..

Return: [double] The absolute value of a.

qsc_intutils_calculate_exp

Purpose: Computes the exponential function exp(x) using a Taylor series. Special cases: - If x is NaN, the function returns NaN. - If x is greater than approximately 709.782712893384, the function returns positive infinity. - If x is less than approximately -745.133219101941, the function returns 0. The function computes exp(x) via the Taylor series: exp(x) = 1 + x/1! + x^2/2! + x^3/3! + ... and stops when the absolute value of the term is less than a relative tolerance.

double qsc_intutils_calculate_exp(double x);

Parameters: x: [double] The exponent..

Return: [double] The computed exp(x) value.

qsc_intutils_calculate_fabs

Purpose: Return the absolute value of a double.

double qsc_intutils_calculate_fabs(double x);

Parameters: x: [double] The input double..

Return: [double] Returns the absolute value.

qsc_intutils_calculate_log

Purpose: Computes the natural logarithm ln(x) without using math.h. Special cases: - If x is NaN, returns NaN. - If x < 0, returns NaN. - If x == 0, returns negative infinity. - If x is extremely large, returns positive infinity. For 0 < x < 1e300, this function first scales x into the interval [1,2) by repeatedly dividing or multiplying by 2, and then computes ln(x) using the series expansion: ln(x) = 2 * (y + y^3/3 + y^5/5 + ...) where y = (x - 1)/(x + 1). The series continues until the current term falls below a fixed tolerance.

double qsc_intutils_calculate_log(double x);

Parameters: x: [double] The input value..

Return: [double] The natural logarithm of x.

qsc_intutils_calculate_sqrt

Purpose: Computes the square root of a nonnegative number without using math.h. Special cases: - If x is negative, the function returns NaN. - If x is 0, the function returns 0. For x > 0, Newton-Raphson iteration is used: guess_{n+1} = 0.5 * (guess_n + x /

guess_n) The iteration stops when the absolute difference between successive guesses is less than a small fraction of the guess (relative tolerance).

double qsc_intutils_calculate_sqrt(double x);

Parameters: x: [double] The input value..

Return: [double] The square root of x, or NaN if x is negative.

qsc_intutils_hex_to_bin_checked

Purpose: Strictly decode a bounded hexadecimal string into bytes.

bool qsc_intutils_hex_to_bin_checked(const char* hexstr, size_t hexlen, uint8_t* output, size_t outlen);

Parameters: hexstr: ASCII input containing exactly hexlen bytes, without ownership transfer.; hexlen: Exact accessible input byte count; must equal twice outlen.; output: Caller-owned buffer with capacity for outlen bytes; must not overlap hexstr.; outlen: Number of output bytes..

Return: True when every input character is hexadecimal and lengths match; otherwise false.

Notes: On failure output is unchanged. Stateless and thread safe.

qsc_intutils_is_power_of_two

Purpose: Check if the integer is a power of two.

bool qsc_intutils_is_power_of_two(size_t x);

Parameters: x: [size_t] The base integer..

Return: [bool] Returns true if x is a power of two.

qsc_intutils_le16to8

Purpose: Convert a 16-bit integer to a little-endian 8-bit integer array.

void qsc_intutils_le16to8(uint8_t* output, uint16_t value);

Parameters: output: [uint8_t*] The destination array.; value: [uint16_t] The 16-bit integer..

Return: None (void).

qsc_intutils_le32to8

Purpose: Convert a 32-bit integer to a little-endian 8-bit integer array.

void qsc_intutils_le32to8(uint8_t* output, uint32_t value);

Parameters: output: [uint8_t*] The destination array.; value: [uint32_t] The 32-bit integer..

Return: None (void).

qsc_intutils_le64to8

Purpose: Convert a 64-bit integer to a little-endian 8-bit integer array.

void qsc_intutils_le64to8(uint8_t* output, uint64_t value);

Parameters: output: [uint8_t*] The destination array.; value: [uint64_t] The 64-bit integer..

Return: None (void).

qsc_intutils_lsb_is_set

Purpose: Check if the integer has the lsb set.

bool qsc_intutils_lsb_is_set(size_t x);

Parameters: x: [size_t] The base integer..

Return: [bool] Returns true if x has the lsb set.

qsc_intutils_next_power_of_2

Purpose: Find the next power of two.

size_t qsc_intutils_next_power_of_2(size_t x);

Parameters: x: [size_t] The base integer..

Return: [size_t] The smallest power of two greater than or equal to x, or 0 if that power cannot be represented by size_t.

qsc_intutils_popcount32

Purpose: Count the number of bits set in a 32-bit unsigned integer.

uint32_t qsc_intutils_popcount32(uint32_t v);

Parameters: v: [uint32_t] The 32-bit integer..

Return: [uint32_t] Returns the number of bits set.

qsc_intutils_popcount64

Purpose: Count the number of set bits in a 64-bit unsigned integer.

uint32_t qsc_intutils_popcount64(uint64_t value);

Parameters: value: Value to count; no pointer ownership or allocation..

Return: Number of set bits (0 to 64).

Notes: Stateless and thread safe; portable scalar implementation.

Header: oid.h

qsc_oid_equals_id

Purpose: Tests whether a decoded object identifier matches a registry identifier.

bool qsc_oid_equals_id(const qsc_asn1_oid* oid, qsc_oid_id id);

Parameters: oid: [const qsc_asn1_oid*] The decoded object identifier.; id: [qsc_oid_id] The registry identifier..

Return: [bool] Returns true if the decoded object identifier matches the registry entry.

qsc_oid_get_dotted

Purpose: Gets the dotted-decimal text form of a registry identifier.

const char* qsc_oid_get_dotted(qsc_oid_id id);

Parameters: id: [qsc_oid_id] The registry identifier..

Return: [const char*] Returns the dotted-decimal text form, or NULL if the identifier is invalid.

qsc_oid_get_encoded

Purpose: Gets the encoded value octets for a registry identifier.

bool qsc_oid_get_encoded(qsc_oid_id id, const uint8_t** data, size_t* length);

Parameters: id: [qsc_oid_id] The registry identifier.; data: [const uint8_t**] Receives a pointer to the encoded OID value octets.; length: [size_t*] Receives the number of encoded OID value octets..

Return: [bool] Returns true on success.

qsc_oid_get_entry

Purpose: Gets a registry entry by internal identifier.

const qsc_oid_entry* qsc_oid_get_entry(qsc_oid_id id);

Parameters: id: [qsc_oid_id] The internal registry identifier..

Return: [const qsc_oid_entry*] Returns a pointer to the matching registry entry, or NULL if not found.

qsc_oid_get_name

Purpose: Gets the descriptive registry name of a registry identifier.

const char* qsc_oid_get_name(qsc_oid_id id);

Parameters: id: [qsc_oid_id] The registry identifier..

Return: [const char*] Returns the descriptive name, or NULL if the identifier is invalid.

qsc_oid_get_name_from_oid

Purpose: Gets the descriptive registry name of a decoded object identifier.

const char* qsc_oid_get_name_from_oid(const qsc_asn1_oid* oid);

Parameters: oid: [const qsc_asn1_oid*] The decoded object identifier..

Return: [const char*] Returns the descriptive name, or NULL if the OID is not recognized.

qsc_oid_identify

Purpose: Identifies a decoded object identifier against the static registry.

qsc_oid_id qsc_oid_identify(const qsc_asn1_oid* oid);

Parameters: oid: [const qsc_asn1_oid*] The decoded object identifier..

Return: [qsc_oid_id] Returns the matching registry identifier, or QSC_OID_ID_NONE if not found.

qsc_oid_registry_at

Purpose: Gets a registry entry by zero-based table index.

const qsc_oid_entry* qsc_oid_registry_at(size_t index);

Parameters: index: [size_t] The registry entry index..

Return: [const qsc_oid_entry*] Returns a pointer to the registry entry, or NULL on error.

qsc_oid_registry_count

Purpose: Gets the number of entries in the static object identifier registry.

size_t qsc_oid_registry_count(void);

Return: [size_t] Returns the number of known registry entries.

qsc_oid_to_asn1

Purpose: Copies the encoded value octets of a registry identifier to a decoded ASN.1 OID structure.

bool qsc_oid_to_asn1(qsc_oid_id id, qsc_asn1_oid* oid);

Parameters: id: [qsc_oid_id] The registry identifier.; oid: [qsc_asn1_oid*] Receives the decoded OID structure..

Return: [bool] Returns true on success.

Header: selftest.h

qsc_selftest_aes_test

Purpose: Tests AES modes, HBA, NIST GCM and independent multi-block GCM KATs.

bool qsc_selftest_aes_test(void);

Return: [bool] Returns true if the AES test passes.

qsc_selftest_chacha_test

Purpose: Tests the ChaCha cipher for correct operation.

bool qsc_selftest_chacha_test(void);

Return: [bool] Returns true if the ChaCha test passes.

qsc_selftest_csx_test

Purpose: Tests ten authenticated CSX-512 reference KATs for the active round profile.

bool qsc_selftest_csx_test(void);

Return: [bool] Returns true if the CSX test passes.

qsc_selftest_rcs_test

Purpose: Tests ten authenticated KATs each for RCS-256 and RCS-512.

bool qsc_selftest_rcs_test(void);

Return: [bool] Returns true if the RCS test passes.

qsc_selftest_sha2_test

Purpose: Tests the SHA2 digests, HKDF and HMAC for correct operation.

bool qsc_selftest_sha2_test(void);

Return: [bool] Returns true if the SHA2 test passes.

qsc_selftest_sha3_test

Purpose: Tests the SHA3 digests, SHAKE, cSHAKE, and KMAC for correct operation.

bool qsc_selftest_sha3_test(void);

Return: [bool] Returns true if the SHA3 test passes.

qsc_selftest_symmetric_run

Purpose: Runs the library self tests. Tests the symmetric primitives with a set of known-answer tests.

bool qsc_selftest_symmetric_run(void);

Return: [bool] Returns true if all tests pass successfully.

Header: stringutils.h

qsc_stringutils_byte_to_hex

Purpose: Convert a byte to a hexidecimal string

void qsc_stringutils_byte_to_hex(char* hex, uint8_t input);

Parameters: hex: [char*] The hex string output; input: [uint8_t] The byte to be converted.

Return: None (void).

qsc_stringutils_string_comparison

Purpose: Test if the string contains a substring

int32_t qsc_stringutils_string_comparison(const char* source, const char* token);

Parameters: source: [const char*] The string to check for the substring; token: [const char*] The substring to search for.

Return: [int32_t] Returns zero if the strings are equal

Header: sysutils.h

qsc_system_values_print

Purpose: Print the output of system function calls

void qsc_system_values_print(void);

Return: None (void).

qsc_sysutils_computer_name_bounded

Purpose: Retrieve the computer name with an explicit output capacity.

size_t qsc_sysutils_computer_name_bounded(char* name, size_t namelen);

Parameters: name: Destination character buffer, NUL terminated on success.; namelen: Size of the destination in bytes, including the terminator..

Return: Characters stored, excluding NUL; zero for invalid input or lookup failure.

Notes: Truncates output safely. Thread-safe, no caller-owned allocations.

qsc_sysutils_cpu_count

Purpose: Get the CPU count

size_t qsc_sysutils_cpu_count(void);

Return: Returns the number of cpus on the system

qsc_sysutils_get_os_drive_letter

Purpose: On Windows returns the drive letter containing the OS, on macOS/BSD/Linux returns '/'

char qsc_sysutils_get_os_drive_letter(void);

Return: [char] 'A'..'Z' on success (Windows), '/' on *nix, or \0

qsc_sysutils_user_name_bounded

Purpose: Retrieve the current user name with an explicit output capacity.

size_t qsc_sysutils_user_name_bounded(char* name, size_t namelen);

Parameters: name: Destination character buffer, NUL terminated on success.; namelen: Size of the destination in bytes, including the terminator..

Return: Characters stored, excluding NUL; zero for invalid input or lookup failure.

Notes: Truncates output safely. Thread-safe, no caller-owned allocations.

Header: timerex.h

qsc_timerex_print_values

Purpose: Print timer function values

void qsc_timerex_print_values(void);

Return: None (void).

qsc_timerex_stopwatch_elapsed_microseconds

Purpose: Return elapsed monotonic time in microseconds since the provided stopwatch start.

uint64_t qsc_timerex_stopwatch_elapsed_microseconds(uint64_t start);

Parameters: start: Opaque counter returned by qsc_timerex_stopwatch_start; not Unix time..

Return: Whole elapsed microseconds, or zero on clock failure or backwards time.

Notes: Thread safe; no allocation. Values from previous builds must not be reused.

qsc_timerex_stopwatch_elapsed_nanoseconds

Purpose: Return elapsed monotonic time in nanoseconds since the provided stopwatch start.

uint64_t qsc_timerex_stopwatch_elapsed_nanoseconds(uint64_t start);

Parameters: start: Opaque counter returned by qsc_timerex_stopwatch_start; not Unix time..

Return: Whole elapsed nanoseconds, or zero on clock failure or backwards time.

Notes: Thread safe; no allocation. Values from previous builds must not be reused.

Header: timestamp.h

qsc_timestamp_datetime_utc

Purpose: Get the calendar date and time for utc time.

uint64_t qsc_timestamp_datetime_utc(void);

Return: [uint64_t] The number of seconds

qsc_timestamp_epochtime_microseconds

Purpose: Get the date and time from the current locale in microseconds from epoch

uint64_t qsc_timestamp_epochtime_microseconds(void);

Return: [uint64_t] Return the date/time in seconds from epoch

qsc_timestamp_epochtime_milliseconds

Purpose: Get the date and time from the current locale in milliseconds from epoch

uint64_t qsc_timestamp_epochtime_milliseconds(void);

Return: [uint64_t] Return the date/time in seconds from epoch

qsc_timestamp_print_values

Purpose: Print time-stamp function values

void qsc_timestamp_print_values(void);

Return: None (void).

qsc_timestamp_string_to_time_struct_checked

Purpose: Parse the fixed-length QSC local date-time representation with validation.

bool qsc_timestamp_string_to_time_struct_checked(struct tm* tstruct, const char input[QSC_TIMESTAMP_STRING_SIZE]);

Parameters: tstruct: Destination structure; cleared on failure. Caller owns storage.; input: NUL-terminated 19-character date-time string. Caller owns storage..

Return: True if the complete input is canonical and calendar-valid; false otherwise.

Notes: The function performs no allocations and is thread safe. Leap-second value 60 is accepted.

Header: winutils.h

qsc_winutils_process_token_elevate

Purpose: Elevate the token access

```
bool qsc_winutils_process_token_elevate(void);
```

Return: [bool] Returns true if successful

17: Memory and Processor

17.1 CPUID

Header:

cpuid.h

Description:

Contains the CPU feature availability functions.

Structures:

The qsc_cpu_features structure.

Data Name	Data Type	Bit Length	Function
aesni	Bool	0x08	The AESNI flag.
avx	Bool	0x08	The AVX flag.
avx2	Bool	0x08	The AVX2 flag.
avx512	Bool	0x08	The AVX512 flag.
hyperthread	Bool	0x08	The hyper thread flag.
pcmul	Bool	0x08	The PCMUL flag.

rdtcsp	Bool	0x08	The RDTCSP flag.
cacheline	Uint32	0x20	The number of cache lines.
cores	Uint32	0x20	The number of cores.
cpus	Uint32	0x20	The number of CPUs.
freqbase	Uint32	0x20	The frequency base.
l1cache	Uint32	0x20	The L1 cache size.
l2cache	Uint32	0x20	The L2 cache size.
serial	Char Array	0x40	The CPU serial number.
vendor	Char Array	0x60	The CPU vendor name.

Table 17.1 cpu features structure

API:

Runtime Features

Get a list of supported CPU features. Takes a pointer to a CPU features structure as a parameter, and returns true for success.

Historical function name: qsc_runtime_features. Revision 1.4 uses qsc_cpuidex_features_set; refer to its source-exact declaration and contract in this chapter.

17.2 SIMD Memory Utilities

Header:

memutils.h

Description:

Memory utilities; contains common memory related functions, implemented with AVX/AVX2/AVX512 SIMD instructions.

API:

FlushCacheLine

Flush a cache line.

```
void qsc_memutils_flush_cache_line(void *address);
```

PrefetchL1

Pre-fetch memory to L1 cache.

```
void qsc_memutils_prefetch_l1(uint8_t* address, size_t length);
```

PrefetchL2

Pre-fetch memory to L2 cache.

```
void qsc_memutils_prefetch_l2(uint8_t* address, size_t length);
```

PrefetchL3

Pre-fetch memory to L3 cache.

```
void qsc_memutils_prefetch_l3(uint8_t* address, size_t length);
```

Malloc

Allocate a block of memory.

```
void* qsc_memutils_malloc(size_t length);
```

Realloc

Resize a block of memory.

```
void* qsc_memutils_realloc(void* block, size_t length);
```

AllocFree

Free a memory block created with alloc.

```
void qsc_memutils_alloc_free(void* block);
```

AlignedAlloc

Allocate an aligned 8-bit integer array.

```
void* qsc_memutils_aligned_alloc(int32_t align, size_t length);
```

AlignedRealloc

Reallocate an aligned 8-bit integer array.

```
void* qsc_memutils_aligned_realloc(void* block, size_t length);
```

AlignedFree

Free an aligned memory block.

```
void qsc_memutils_aligned_free(void* block);
```

Clear

Erase a block of memory.

```
void qsc_memutils_clear(void* output, size_t length);
```

ArrayUniform

Check if all array members are the same.

```
bool qsc_memutils_array_uniform(const uint8_t* input, size_t length);
```

AreEqual
Compare two byte arrays for equality.

```
bool qsc_memutils_are_equal(const uint8_t* a, const uint8_t* b, size_t length);
```

AreEqual128

Compare two 16 byte arrays for equality.

```
bool qsc_memutils_are_equal_128(const uint8_t* a, const uint8_t* b);
```

AreEqual256

Compare two 32 byte arrays for equality.

```
bool qsc_memutils_are_equal_256(const uint8_t* a, const uint8_t* b);
```

AreEqual512

Compare two 64 byte arrays for equality.

```
bool qsc_memutils_are_equal_512(const uint8_t* a, const uint8_t* b);
```

Copy

Copy a block of memory.

```
void qsc_memutils_copy(void* output, const void* input, size_t length);
```

CMul64x128

Emulates the _mm_clmulepi64_si128 intrinsic.

```
void qsc_memutils_clmulepi64_si128(uint64_t r[2U], const uint64_t a[2U], const uint64_t b[2U], int32_t imm8);
```

CMul64x256AVX

Multiply two 256-bit field elements (each represented as two 128-bit integers).

```
void qsc_memutils_clmulepi64_si256_avx(__m128i r[4U], const __m128i a[2U], const __m128i b[2U]);
```

CMul64x256

Multiply two 256-bit field elements (each represented as two 128-bit integers).

qsc_memutils_clmulepi64_si256(uint64_t r[8], const uint64_t a[4], const uint64_t b[4]);

GreaterThanBE128

Compare two 16 byte arrays as 128-bit big endian integers as A is greater than B.

bool qsc_memutils_greater_than_be128(const uint8_t* a, const uint8_t* b);

GreaterThanBE256

Compare two 32 byte arrays as 256-bit big endian integers as A is greater than B.

bool qsc_memutils_greater_than_be256(const uint8_t* a, const uint8_t* b);

GreaterThanBE512

Compare two 64 byte arrays as 512-bit big endian integers as A is greater than B.

bool qsc_memutils_greater_than_be512(const uint8_t* a, const uint8_t* b);

GreaterThanLE128

Compare two 16 byte arrays as 128-bit little endian integers as A is greater than B.

bool qsc_memutils_greater_than_le128(const uint8_t* a, const uint8_t* b);

GreaterThanLE256

Compare two 32 byte arrays as 256-bit little endian integers as A is greater than B.

bool qsc_memutils_greater_than_le256(const uint8_t* a, const uint8_t* b);

GreaterThanLE512

Compare two 64 byte arrays as 512-bit little endian integers as A is greater than B.

bool qsc_memutils_greater_than_le512(const uint8_t* a, const uint8_t* b);

Move

Move a block of memory, erasing the previous location.

```
void qsc_memutils_move(void* output, const void* input, size_t length);
```

SecureErase

Erase a memory block securely.

```
void qsc_memutils_secure_erase(void* block, size_t length);
```

SecureFree

Free a secure memory block.

```
void qsc_memutils_secure_free(void* block, size_t length);
```

SecureMalloc

Allocate an secure 8-bit integer array.

```
void* qsc_memutils_secure_malloc(size_t length);
```

SetValue

Set a block of memory to a value.

```
void qsc_memutils_set_value(void* output, size_t length, uint8_t value);
```

XOR

Bitwise XOR two blocks of memory.

```
void qsc_memutils_xor(uint8_t* output, const uint8_t* input, size_t length);
```

XORV

Bitwise XOR a block of memory with a byte value.

```
void qsc_memutils_xorv(uint8_t* output, const uint8_t value, size_t length);
```

Zeroed

Tests an array for all zeroed elements.

```
bool qsc_memutils_zeroed(const void* input, size_t length);
```

17.3 Secure Memory Functions

Header:

memutils.h

Description:

Secure memory functions in the current public API are provided through memutils.h. The exposed secure allocation, erase, and free functions provide centralized memory handling for sensitive buffers; no separate secmem.h header is present in the current source tree.

API:

Secmem Alloc

Allocate a block of secure memory. Takes the number of bytes to allocate as a parameter, and returns a pointer to the allocated memory.

```
void* qsc_memutils_secure_malloc(size_t length)
```

Secmem Release

Erase a length of secure memory. Takes a pointer to the memory to erase. And the number of bytes to erase as parameters.

```
void qsc_memutils_secure_erase(void* block, size_t length)
```

Secmem Free

Erase and free a block of secure memory.

```
void qsc_memutils_secure_free(void* block, size_t length)
```

Secure Memory API Boundary

The current public secure-memory API is limited to secure allocation, secure erase, and secure free through memutils.h. No exported page-size function is present in the current source tree.

18: Validation, Build, and Wrapper Assets

17.4 Additional Memory and CPU APIs

Header: cpuidex.h

qsc_cpuidex_features_set

Purpose: Populate the CPU features structure with detected CPU capabilities.

```
bool qsc_cpuidex_features_set(qsc_cpuidex_cpu_features* const features);
```

Parameters: features: [qsc_cpuidex_cpu_features* const] Pointer to a qsc_cpuidex_cpu_features structure to populate..

Return: [bool] Returns true if the features were successfully detected; false otherwise.

qsc_cpuidex_features_set_cached

Purpose: Copy an immutable, thread-safe process-lifetime snapshot of CPU capabilities.

Details: Initializes once and avoids repeated CPUID and topology queries. This is a startup snapshot and is NOT suitable where CPU hotplug, VM migration, or dynamic OS

processor-feature changes must be detected. In those cases use qsc_cpuidex_features_set() to query the current capabilities.

bool qsc_cpuidex_features_set_cached(qsc_cpuidex_cpu_features* features);

Parameters: features: Destination for the fully initialized capabilities structure..

Return: True when the snapshot is available; false for NULL, unsupported OS, or one-time initialization failure. Does not allocate caller-owned memory.

Notes: Safe to call concurrently; output must not be shared without synchronization.

qsc_cpuidex_print_stats

Purpose: Print the detected CPU features to the console. This function outputs the CPU capabilities (for example, AVX, AESNI, cache size, vendor) to the console.

void qsc_cpuidex_print_stats(void);

Return: None (void).

Header: memutils.h

qsc_memutils_aligned_resize

Purpose: Resize an aligned allocation with an explicitly supplied old size and alignment.

void* qsc_memutils_aligned_resize(void* block, size_t old_length, size_t length, int32_t align);

Parameters: block: [void*] Original aligned allocation, or NULL to allocate.; old_length: [size_t] Readable byte count in block, zero if block is NULL.; length: [size_t] Requested allocation size, nonzero.; align: [int32_t] Power-of-two alignment not less than sizeof(void*)..

Return: [void*] Resized aligned block, or NULL on failure. A successful allocation releases the original block; a failure leaves it unchanged.

Notes: This operation is not safe while other threads access block.

qsc_memutils_calloc

Purpose: Allocate and zero - initialize an array of elements. Allocates storage for an array of count elements, each length bytes in size, and initializes the allocated memory to zero.

void* qsc_memutils_calloc(size_t count, size_t length);

Parameters: count: [size_t] The number of elements to allocate.; length: [size_t] The size, in bytes, of each element..

Return: [void*] Returns a pointer to the zero - initialized memory block, or NULL on failure.

qsc_memutils_clmulepi64_si256

Purpose: Multiply two 256-bit field elements (each represented as two 128-bit integers) to produce a 512-bit product.

void qsc_memutils_clmulepi64_si256(uint64_t r[8U], const uint64_t a[4U], const uint64_t b[4U]);

Parameters: r: [uint64_t[8]] 512-bit product (r[0] = least-significant 64 bits, r[7] = most-significant 64 bits).; a: [const uint64_t[4]] First 256-bit operand (a[0] = lower 64 bits, a[3] = upper 64 bits).; b: [const uint64_t[4]] Second 256-bit operand (same ordering)..

Return: None (void).

qsc_memutils_page_size

Purpose: Get the system oage size.

size_t qsc_memutils_page_size(void);

Return: [void*] Returns the system page size.

18.1 Test Projects

Header and project paths:

QSC, QSCTest, CAVP, QSCSDKC, QSCCPP, QSCSDKCPP, QSCABI, QSCCS, QSCSDKCS

The QSC source and SDK package is organized around a native C implementation, native validation projects, a stable C ABI bridge, and language-specific wrapper and example projects. The native QSC library remains the authoritative cryptographic implementation. QSCTest provides the primary native known-answer, operational, negative, stress, and regression test suite. The CAVP-oriented validation project provides algorithm-vector processing for certification-readiness workflows. QSCSDKC provides native C example coverage and command-line usage demonstrations. QSCCPP and

QSCSDKCPP provide the C++ wrapper and C++ example harness. QSCABI exposes the stable native C ABI boundary used by managed and foreign-language bindings. QSCCS and QSCSDKCS provide the C# wrapper, tests, and example harness.

These assets are validation, integration, portability, and certification-readiness resources. They support known-answer testing, operational self-tests, wrapper conformance checks, example-driven integration, validation-vector processing, and regression testing across the native and wrapper ecosystem. They do not, by themselves, constitute FIPS, CAVP, ACVP, CMVP, NIST, or third-party certification. Certification status must be represented only where an applicable certificate, validation listing, laboratory report, or formal attestation exists.

18.2 Validation Assets

The validation package includes native and wrapper-level test assets. QSCTest is the authoritative native correctness and regression harness for the QSC implementation. The CAVP project contains vector-processing code for algorithms including AES, SHA2, SHA3, Kyber / ML-KEM, Dilithium / ML-DSA, and SPHINCS+ / SLH-DSA, together with response-vector files and AESAVS material used by the validation workflow. QSCABITest validates the stable ABI layer and verifies that ABI exports, status handling, buffer ownership rules, and native bridge behavior remain suitable for external language bindings.

Each wrapper family also includes its own test project. QSCCS, the Java wrapper, the Rust wrapper, and the Python wrapper are validated against QSCABI behavior rather than against duplicated cryptographic implementations. Wrapper tests verify interop correctness, size constants, exception or status-code mapping, buffer handling, negative cases, and representative known-answer behavior. The wrapper tests are conformance assets for the language bindings and should be interpreted as wrapper-level assurance, not as independent validation of separate cryptographic engines.

18.3 Packaged Build Assets

The source package is organized around Visual Studio solution and project files for Windows development, with native portability branches and platform-selection macros for supported operating systems and processor features. QSC builds as the native cryptographic library. QSCABI builds as the shared native bridge used by higher-level language wrappers. QSCSDKC, QSCCPP, QSCSDKCPP, QSCCS, QSCSDKCS, and the Java, Rust, and Python wrapper/example projects are packaged as separate integration assets that consume the native library or ABI boundary according to their role.

Packaged build-system coverage should be distinguished from implementation portability. The presence of project files, build scripts, runtime folders, or wrapper packaging layouts indicates supported integration paths; it does not by itself imply certification, operational deployment qualification, or platform validation unless those claims are separately tested and documented. Native QSC and QSCABI builds remain the foundation for all wrapper packages.

18.4 Wrapper and ABI Assets

QSCABI is the stable C ABI bridge over the native QSC implementation. It is the designated boundary for C#, Java, Rust, Python, and other external language integrations. Wrapper projects must bind to QSCABI and must not duplicate QSC internals or depend on the C++ wrapper. This model preserves a single authoritative cryptographic implementation while allowing each language ecosystem to expose idiomatic APIs, examples, tests, packaging conventions, and runtime-loading behavior.

QSCCS provides the C# wrapper over QSCABI, with QSCSDKCS supplying the C# example harness and QSCCSTest supplying wrapper-level validation. The Java, Rust, and Python wrapper families follow the same pattern: a language-specific wrapper library, a language-specific SDK example project, and a wrapper-specific test project. Wrapper coverage should be described only where the wrapper declaration, ABI binding, example coverage, and corresponding test asset are present. The wrapper layer is an integration and usability layer over QSCABI; it is not a separate cryptographic implementation.

18.5 Optimized Compilation Profiles (Revision 1.4)

The planned and separately delivered modular CMake integration supports seven source-selection profiles: FULL, CLASSICAL, PQC3, PQC5, HYBRID, DEVICE_IDENTITY, and BOOT_VERIFY. Profile selection restricts the objects and capabilities compiled into a target. Platform-specific hardware adapters may require additional SDKs, provider services and hardware qualification.

The uploaded library archive contains public hardware headers but not the separately developed modular Source/QSC CMake integration. Verify exact cache variable names, CMake targets and generated feature manifest paths against the installed CMake files before use.

19: Hardware Security and Protected-Key Interfaces

Hardware-provider facilities are optional. The provider and each native backend are subject to platform dependencies, key-authorization policies and compile-time feature guards. A callable provider interface does not establish physical hardware key custody or FIPS validation.

19.1 Hardware Provider and Protected-Key API Reference

Header: hwatecc.h

qsc_hw_atecc_dispose

Purpose: Dispose callback state without modifying or clearing any hardware slot.

void qsc_hw_atecc_dispose(qsc_hw_atecc_context* context);

Parameters: context: Caller-owned context; must have no live provider/key references..

Return: None (void).

Notes: Does not call atcab_release; the OEM retains vendor-device ownership.

qsc_hw_atecc_get_operations

Purpose: Return the common versioned provider callback table for this ATECC adapter.

qsc_hw_status qsc_hw_atecc_get_operations(qsc_hw_provider_ops* operations);

Parameters: operations: Caller-owned output descriptor, cleared on failure..

Return: Success or invalid parameter; no vendor initialization is performed.

qsc_hw_atecc_initialize

Purpose: Initialize one pre-provisioned ATECC slot without changing device configuration.

qsc_hw_status qsc_hw_atecc_initialize(qsc_hw_atecc_context* context, const qsc_hw_atecc_callbacks* callbacks, void* device, uint8_t slot, bool physical_confirmed);

Parameters: context: Caller-owned zero-initialized context.; callbacks: Complete versioned OEM backend callbacks; copied on success.; device: Non-NULL OEM-owned initialized device handle.; slot: Pre-provisioned P-256 signing slot (0..15).; physical_confirmed: True only after out-of-band physical-device qualification..

Return: Success or typed failure. Invalid or unlocked slots fail closed.

qsc_hw_atecc_locator

Purpose: Encode a persistent device-bound locator for the configured slot.

qsc_hw_status qsc_hw_atecc_locator(qsc_hw_atecc_context* context, char* output, size_t* outputlen);

Parameters: context: Initialized caller-owned context.; output: Caller buffer, or NULL to query required size.; outputlen: In/out byte capacity and required size, including NUL..

Return: Success or buffer-too-small; no secrets in the locator.

Header: hwncrypt.h

qsc_hw_ncrypt_algorithm_available

Purpose: Check a specific signature curve against the opened TPM KSP.

bool qsc_hw_ncrypt_algorithm_available(const qsc_hw_ncrypt_context* context, qsc_hw_algorithm algorithm);

Parameters: context: Initialized provider context.; algorithm: ECDSA P-256, P-384 or P-521; other algorithms return false..

Return: True only when the platform provider explicitly supports the algorithm.

Notes: This is mechanism availability, not physical-operation or FIPS qualification.

qsc_hw_ncrypt_dispose

Purpose: Dispose native provider storage after all key references have been released.

void qsc_hw_ncrypt_dispose(qsc_hw_ncrypt_context* context);

Parameters: context: Initialized context, or a zeroed context for idempotent cleanup..

Return: None (void).

Notes: qsc_hw_provider_close performs this automatically when registered. Do not call while a provider wrapper or key reference is still active.

qsc_hw_ncrypt_get_operations

Purpose: Populate a versioned hardware-provider operations descriptor.

qsc_hw_status qsc_hw_ncrypt_get_operations(qsc_hw_provider_ops* operations);

Parameters: operations: Caller-owned output; initialized only on success..

Return: Success, invalid parameter, or unsupported on non-Windows platforms.

Notes: The caller passes the returned descriptor and initialized context to qsc_hw_provider_open; closing that provider disposes the native context.

qsc_hw_ncrypt_initialize

Purpose: Open the TPM-only Windows key storage provider for one persistent identity name.

qsc_hw_status qsc_hw_ncrypt_initialize(qsc_hw_ncrypt_context* context, const char* name, bool machine_key);

Parameters: context: Caller-owned zeroed context; initialized on success.; name: UTF-8 NUL-terminated persistent key name, 1..127 bytes, no embedded NUL.; machine_key: True for machine-wide storage, false for current-user scope..

Return: Success or typed error. Unsupported on non-Windows systems or without TPM KSP.

Notes: Caller must close any qsc_hw_provider using this context before disposal. Never invoke from a Windows service StartService callback.

Header: hwncrypttls.h

qsc_hw_ncrypt_tls_client_config_set_identity

Purpose: Configure TLS 1.3 client authentication using a non-exportable TPM key.

qsc_tls_status qsc_hw_ncrypt_tls_client_config_set_identity(qsc_tls_client_config* config, const qsc_tls_certificate_view* chain, size_t chainlength, qsc_hw_ncrypt_tls_signer* signer);

Parameters: config: Existing client configuration; no other field is modified on failure.; chain: Nonempty leaf-first DER certificate chain, borrowed until handshake finishes.; chainlength: Number of chain elements, max QSC_TLS_CERTIFICATE_LIST_MAX_ENTRIES.; signer: Live TPM signer; certificate leaf public key must match TPM key..

Return: TLS status; invalid or mismatched identity is rejected.

Notes: No private-key bytes are copied or stored; signer is caller-owned.

qsc_hw_ncrypt_tls_server_config_add_identity

Purpose: Add a host-specific TLS 1.3 server identity for SNI selection.

qsc_tls_status qsc_hw_ncrypt_tls_server_config_add_identity(qsc_tls_server_config* config, const char* hostname, const qsc_tls_certificate_view* chain, size_t chainlength, qsc_hw_ncrypt_tls_signer* signer);

Parameters: config: Existing server configuration; caller owns selected certificate buffers.; hostname: Bounded, NUL-terminated hostname, as required by QSC server config.; chain: Nonempty leaf-first DER certificate chain, borrowed through handshake.; chainlength: Number of entries in chain.; signer: Live TPM signer, retained by pointer only..

Return: TLS status, including error from QSC's existing SNI identity setter.

qsc_hw_ncrypt_tls_server_config_set_identity

Purpose: Configure TLS 1.3 server certificate identity using a protected TPM key.

qsc_tls_status qsc_hw_ncrypt_tls_server_config_set_identity(qsc_tls_server_config* config, const qsc_tls_certificate_view* chain, size_t chainlength, qsc_hw_ncrypt_tls_signer* signer);

Parameters: config: Existing server configuration, changed only on successful verification.; chain: Nonempty leaf-first DER certificate chain, borrowed through handshake.; chainlength: Number of chain elements.; signer: Live non-exportable TPM signer, borrowed through all handshakes..

Return: Typed TLS status; rejects certificate/key mismatch and malformed chain.

qsc_hw_ncrypt_tls_sign_callback

Purpose: Generate TLS CertificateVerify DER ECDSA signature from a TPM-protected key.

bool qsc_hw_ncrypt_tls_sign_callback(qsc_tls_signature_scheme scheme, const uint8_t* input, size_t inputlen, uint8_t* signature, size_t* signaturelen, void* state);

Parameters: scheme: Negotiated TLS signature scheme; must match signer scheme exactly.; input: Formatted CertificateVerify bytes; hashed once by QSC SHA-2.; inputlen: Input length in bytes; zero or NULL input is refused.; signature: Caller output buffer for DER ECDSA signature; never partial on failure.; signaturelen: In/out output capacity and encoded signature size, in bytes.; state: Caller-owned qsc_hw_ncrypt_tls_signer; borrowed for operation..

Return: True only on successful hardware operation and DER encoding.

Notes: Synchronous; never silently falls back to software. Caller synchronizes operations.

qsc_hw_ncrypt_tls_signer_initialize

Purpose: Initialize a signer from an existing protected Windows TPM/NCrypt ECDSA key.

qsc_hw_status qsc_hw_ncrypt_tls_signer_initialize(qsc_hw_ncrypt_tls_signer* signer, qsc_hw_keyref* key, qsc_tls_signature_scheme scheme);

Parameters: signer: Caller-owned signer initialized on success; cleared on error.; key: Live protected TPM private key, borrowed for signer lifetime.; scheme: One of TLS ECDSA P-256/SHA-256, P-384/SHA-384 or P-521/SHA-512..

Return: Typed status. No key ownership is transferred; no private material is read.

Notes: Not thread-safe with concurrent release, removal or mutation of the same signer.

qsc_hw_ncrypt_x509_csr_sign

Purpose: Encode and sign a PKCS#10 CSR using a protected Windows TPM/NCrypt private key.

qsc_asn1_status qsc_hw_ncrypt_x509_csr_sign(const qsc_x509_csr* csr, qsc_hw_ncrypt_tls_signer* signer, uint8_t* output, size_t* outputlen);

Parameters: csr: Fully configured CSR with TPM-matching subjectPublicKeyInfo and ECDSA algorithm.; signer: Live protected TPM signer; signer key must match CSR SPKI.; output: Caller-owned DER output buffer; no private bytes are returned.; outputlen: In/out DER capacity and encoded length, in bytes..

Return: Typed ASN.1 status; mismatched public key or unsupported scheme fails closed.

qsc_hw_ncrypt_x509_sign_callback

Purpose: Sign X.509/PKCS#10 TBS input via the protected Windows TPM/NCrypt key.

qsc_asn1_status qsc_hw_ncrypt_x509_sign_callback(qsc_x509_signature_algorithm algorithm, const uint8_t* tbs, size_t tbslen, uint8_t* signature, size_t* signaturelen, void* context);

Parameters: algorithm: X.509 ECDSA/SHA-2 algorithm; must correspond exactly to signer.; tbs: DER TBSCertificate or CertificationRequestInfo bytes, prehash input.; tbslen: Input length, in bytes.; signature: Caller output buffer for DER ECDSA signature; erased on error.; signaturelen: In/out buffer capacity and DER signature length, in bytes.; context: Borrowed qsc_hw_ncrypt_tls_signer, updated with backend status..

Return: ASN.1 success only if TPM signing and DER encoding succeed.

Header: hwpkcs11.h

qsc_hw_pkcs11_key_agree

Purpose: Perform token-backed ECDH with an explicitly authorized export of the derived secret.

qsc_hw_status qsc_hw_pkcs11_key_agree(qsc_hw_pkcs11_key* key, const uint8_t* peer, size_t peerlen, uint8_t* secret, size_t* secretlen, bool allow_export);

Parameters: key: Live ECDH-only protected private key reference.; peer: Uncompressed SEC1 peer public EC point; caller must authenticate the peer.; peerlen: Exact peer point length for this key's curve, in bytes.; secret: Caller-owned raw output buffer, or NULL to query supported length.; secretlen: In/out output size; 32, 48 or 66 bytes for P-256/P-384/P-521.; allow_export: Explicit authorization to export derived secret into application memory..

Return: Typed result; a temporary token-derived object is always destroyed before success.

Notes: Only the private key remains non-exportable; output bytes are NOT hardware-resident.

qsc_hw_pkcs11_key_find

Purpose: Resolve one protected private key by exact CKA_ID and elliptic-curve algorithm.

qsc_hw_status qsc_hw_pkcs11_key_find(qsc_hw_pkcs11_key** key, qsc_hw_pkcs11_session* session, const uint8_t* id, size_t idlen, qsc_hw_algorithm algorithm);

Parameters: key: Output reference, NULL on error.; session: Active Cryptoki session, including token authorization where required.; id: Nonempty binary CKA_ID bytes, at most QSC_HW_PKCS11_MAX_KEY_ID bytes.; idlen: Length of id in bytes.; algorithm: Exact ECDSA curve (P-256/P-384/P-521), or supported ECDH curve..

Return: Success only for exactly one sensitive, nonextractable, token-resident matching key.

Notes: This interface reports token attributes, not independent hardware attestation.

qsc_hw_pkcs11_key_generate

Purpose: Generate one token-resident nonextractable EC keypair with unique CKA_ID.

qsc_hw_status qsc_hw_pkcs11_key_generate(qsc_hw_pkcs11_key** key, qsc_hw_pkcs11_session* session, const uint8_t* id, size_t idlen, qsc_hw_algorithm algorithm);

Parameters: key: Receives live private key reference, NULL on failure.; session: Active read-write token session.; id: Unique nonempty caller-selected CKA_ID; never a secret.; idlen: Identifier length in bytes.; algorithm: ECDSA P-256/P-384/P-521 or ECDH P-256/P-384/P-521..

Return: Typed generation, policy, or rollback failure. Generated objects persist until explicitly destroyed by provisioning software.

Notes: Token must enforce CKA_SENSITIVE=true and CKA_EXTRACTABLE=false. Creation failure attempts rollback; treat failed rollback as provisioning incident.

qsc_hw_pkcs11_key_public

Purpose: Retrieve a token private-key reference's matched public EC point.

qsc_hw_status qsc_hw_pkcs11_key_public(qsc_hw_pkcs11_key* key, uint8_t* output, size_t* outputlen);

Parameters: key: Live protected key reference.; output: Caller buffer for SEC1 uncompressed point (0x04 || X || Y), or NULL for size query.; outputlen: In/out buffer capacity and exact point length, in bytes..

Return: Success, buffer-too-small, stale-key or typed backend failure; no private data is exported.

qsc_hw_pkcs11_key_release

Purpose: Release an ephemeral key reference without deleting token objects.

qsc_hw_status qsc_hw_pkcs11_key_release(qsc_hw_pkcs11_key** key);

Parameters: key: Address of reference; set NULL on release..

Return: Typed status. Owning session must remain live; caller synchronizes concurrent calls.

qsc_hw_pkcs11_key_sign

Purpose: Sign a precomputed digest using CKM_ECDSA, returning raw fixed-width r||s.

qsc_hw_status qsc_hw_pkcs11_key_sign(qsc_hw_pkcs11_key* key, const uint8_t* digest, size_t digestlen, uint8_t* signature, size_t* signaturelen);

Parameters: key: Live ECDSA private-key reference; ECDH-only keys are rejected.; digest: Precomputed digest bytes, exactly 32, 48 or 64 bytes for P-256/P-384/P-521 respectively.; digestlen: Digest length in bytes.; signature: Writable raw r||s buffer, or NULL for required-length query.; signaturelen: In/out capacity and raw signature length (64, 96, 132 bytes)..

Return: Typed status; signature buffer is erased on failed backend execution.

Notes: Digest construction and TLS DER conversion are caller responsibilities.

qsc_hw_pkcs11_mechanism_info_get

Purpose: Obtain mechanism limits and attributes from the token.

qsc_hw_status qsc_hw_pkcs11_mechanism_info_get(qsc_hw_pkcs11_module* module, uint64_t slot, uint64_t mechanism, qsc_hw_pkcs11_mechanism_info* info);

Parameters: module: Loaded module.; slot: Slot identifier.; mechanism: Cryptoki mechanism identifier.; info: Caller-owned output..

Return: Typed status; unsupported mechanism is explicit.

qsc_hw_pkcs11_mechanism_list

Purpose: Enumerate actual available Cryptoki mechanisms for a slot.

qsc_hw_status qsc_hw_pkcs11_mechanism_list(qsc_hw_pkcs11_module* module, uint64_t slot, uint64_t* mechanisms, size_t* count);

Parameters: module: Loaded module.; slot: Slot identifier obtained from enumeration.; mechanisms: Caller buffer or NULL for size query.; count: In/out entry capacity and reported count..

Return: Success, buffer-too-small, or typed failure.

qsc_hw_pkcs11_module_close

Purpose: Close one client reference, finalizing/unloading only when owned and unused.

qsc_hw_status qsc_hw_pkcs11_module_close(qsc_hw_pkcs11_module** module);

Parameters: module: In/out live module reference; set NULL on successful release..

Return: Busy if any open session exists, otherwise typed status.

Notes: Module references are process-local; do not inherit them across fork.

qsc_hw_pkcs11_module_open

Purpose: Open or share one initialized module in the current process.

qsc_hw_status qsc_hw_pkcs11_module_open(qsc_hw_pkcs11_module** module, const char* absolute_path);

Parameters: module: Output receives one shared module reference, NULL on failure.; absolute_path: NUL-terminated absolute filesystem path; no relative or traversal paths..

Return: Typed status. When CKR_CRYPTOKI_ALREADY_INITIALIZED is returned, QSC does not own Finalize.

Notes: Thread-safe module registry; application must trust the path and protect module integrity.

qsc_hw_pkcs11_session_close

Purpose: Close a session without implicitly logging out other consumers.

qsc_hw_status qsc_hw_pkcs11_session_close(qsc_hw_pkcs11_session** session);

Parameters: session: In/out open handle, set NULL on successful close or device loss..

Return: Typed result. A lost session is discarded; other module sessions remain live.

Notes: Caller serializes close against calls using the same session.

qsc_hw_pkcs11_session_login

Purpose: Log in as CKU_USER, with a transient copy of PIN bytes securely erased.

qsc_hw_status qsc_hw_pkcs11_session_login(qsc_hw_pkcs11_session* session, const uint8_t* pin, size_t pinlen);

Parameters: session: Open session.; pin: PIN bytes; NULL with zero length requests a token protected-authentication path.; pinlen: PIN byte count, bounded by QSC_HW_PKCS11_MAX_PIN_LENGTH..

Return: Typed authentication, lockout or token status. No retry is performed.

Notes: Does not cache credentials; QSC never performs Security Officer login.

qsc_hw_pkcs11_session_logout

Purpose: Explicitly log out the USER token login state.

qsc_hw_status qsc_hw_pkcs11_session_logout(qsc_hw_pkcs11_session* session);

Parameters: session: Open session..

Return: Typed status; logout may affect other sessions to the same token.

qsc_hw_pkcs11_session_open

Purpose: Open one isolated serial Cryptoki session.

qsc_hw_status qsc_hw_pkcs11_session_open(qsc_hw_pkcs11_session** session, qsc_hw_pkcs11_module* module, uint64_t slot, bool read_write);

Parameters: session: Output handle, NULL on failure.; module: Loaded module; cannot be closed while any session is live.; slot: Slot identifier from enumeration.; read_write: Request read-write access; false opens a read-only session..

Return: Success or a typed module/session/authorization failure.

Notes: Shared token-wide login state may affect other sessions; caller coordinates access.

qsc_hw_pkcs11_slot_list

Purpose: Enumerate slot identifiers without exposing Cryptoki-dependent public types.

qsc_hw_status qsc_hw_pkcs11_slot_list(qsc_hw_pkcs11_module* module, bool token_present, uint64_t* slots, size_t* count);

Parameters: module: Loaded module.; token_present: Filter for slots reporting inserted tokens.; slots: Caller buffer of capacity *count entries, or NULL for count-only query.; count: In/out capacity and reported count in entries; zeroed on errors except insufficient capacity..

Return: Success or typed failure, including buffer-too-small and malicious oversized enumeration.

Notes: Query and copy are serialized with this module's other QSC adapter operations.

qsc_hw_pkcs11_token_info_get

Purpose: Query public metadata of a physical/logical token.

qsc_hw_status qsc_hw_pkcs11_token_info_get(qsc_hw_pkcs11_module* module, uint64_t slot, qsc_hw_pkcs11_token_info* info);

Parameters: module: Loaded module.; slot: Numeric slot identifier returned from slot_list.; info: Caller-owned output; zeroed before querying..

Return: Typed status; token absence reported distinctly.

Header: hwpkcs11tls.h

qsc_hw_pkcs11_tls_client_config_set_identity

Purpose: Configure TLS 1.3 client authentication using a non-exportable token key.

qsc_tls_status qsc_hw_pkcs11_tls_client_config_set_identity(qsc_tls_client_config* config, const qsc_tls_certificate_view* chain, size_t chainlength, qsc_hw_pkcs11_tls_signer* signer);

Parameters: config: Existing client configuration; no other field is modified on failure.; chain: Nonempty leaf-first DER certificate chain, borrowed until handshake finishes.; chainlength: Number of chain elements, max QSC_TLS_CERTIFICATE_LIST_MAX_ENTRIES.; signer: Live token signer; certificate leaf public key must match token key..

Return: TLS status; invalid or mismatched identity is rejected.

Notes: No private-key bytes are copied or stored; signer is caller-owned.

qsc_hw_pkcs11_tls_server_config_add_identity

Purpose: Add a host-specific TLS 1.3 server identity for SNI selection.

qsc_tls_status qsc_hw_pkcs11_tls_server_config_add_identity(qsc_tls_server_config* config, const char* hostname, const qsc_tls_certificate_view* chain, size_t chainlength, qsc_hw_pkcs11_tls_signer* signer);

Parameters: config: Existing server configuration; caller owns selected certificate buffers.; hostname: Bounded, NUL-terminated hostname, as required by QSC server config.; chain: Nonempty leaf-first DER certificate chain, borrowed through handshake.; chainlength: Number of entries in chain.; signer: Live token signer, retained by pointer only..

Return: TLS status, including error from QSC's existing SNI identity setter.

qsc_hw_pkcs11_tls_server_config_set_identity

Purpose: Configure TLS 1.3 server certificate identity using a protected token key.

qsc_tls_status qsc_hw_pkcs11_tls_server_config_set_identity(qsc_tls_server_config* config, const qsc_tls_certificate_view* chain, size_t chainlength, qsc_hw_pkcs11_tls_signer* signer);

Parameters: config: Existing server configuration, changed only on successful verification.; chain: Nonempty leaf-first DER certificate chain, borrowed through handshake.; chainlength: Number of chain elements.; signer: Live non-exportable token signer, borrowed through all handshakes..

Return: Typed TLS status; rejects certificate/key mismatch and malformed chain.

qsc_hw_pkcs11_tls_sign_callback

Purpose: Generate TLS CertificateVerify DER ECDSA signature from a token-protected key.

bool qsc_hw_pkcs11_tls_sign_callback(qsc_tls_signature_scheme scheme, const uint8_t* input, size_t inputlen, uint8_t* signature, size_t* signaturelen, void* state);

Parameters: scheme: Negotiated TLS signature scheme; must match signer scheme exactly.; input: Formatted CertificateVerify bytes; hashed once by QSC SHA-2.; inputlen: Input length in bytes; zero or NULL input is refused.; signature: Caller output buffer for DER ECDSA signature; never partial on failure.; signaturelen: In/out output capacity and encoded signature size, in bytes.; state: Caller-owned qsc_hw_pkcs11_tls_signer; borrowed for operation..

Return: True only on successful hardware operation and DER encoding.

Notes: Synchronous; never silently falls back to software. Caller synchronizes operations.

qsc_hw_pkcs11_tls_signer_initialize

Purpose: Initialize a signer from an existing protected PKCS#11 ECDSA key.

qsc_hw_status qsc_hw_pkcs11_tls_signer_initialize(qsc_hw_pkcs11_tls_signer* signer, qsc_hw_pkcs11_key* key, qsc_tls_signature_scheme scheme);

Parameters: signer: Caller-owned signer initialized on success; cleared on error.; key: Live protected token private key, borrowed for signer lifetime.; scheme: One of TLS ECDSA P-256/SHA-256, P-384/SHA-384 or P-521/SHA-512..

Return: Typed status. No key ownership is transferred; no private material is read.

Notes: Not thread-safe with concurrent release, removal or mutation of the same signer.

qsc_hw_pkcs11_x509_csr_sign

Purpose: Encode and sign a PKCS#10 CSR using a protected PKCS#11 private key.

qsc_asn1_status qsc_hw_pkcs11_x509_csr_sign(const qsc_x509_csr* csr, qsc_hw_pkcs11_tls_signer* signer, uint8_t* output, size_t* outputlen);

Parameters: csr: Fully configured CSR with token-matching subjectPublicKeyInfo and ECDSA algorithm.; signer: Live protected token signer; signer key must match CSR SPKI.; output: Caller-owned DER output buffer; no private bytes are returned.; outputlen: In/out DER capacity and encoded length, in bytes..

Return: Typed ASN.1 status; mismatched public key or unsupported scheme fails closed.

qsc_hw_pkcs11_x509_sign_callback

Purpose: Sign X.509/PKCS#10 TBS input via the protected PKCS#11 key.

qsc_asn1_status qsc_hw_pkcs11_x509_sign_callback(qsc_x509_signature_algorithm algorithm, const uint8_t* tbs, size_t tbslen, uint8_t* signature, size_t* signaturelen, void* context);

Parameters: algorithm: X.509 ECDSA/SHA-2 algorithm; must correspond exactly to signer.; tbs: DER TBSCertificate or CertificationRequestInfo bytes, prehash input.; tbslen: Input length, in bytes.; signature: Caller output buffer for DER ECDSA signature; erased on error.; signaturelen: In/out buffer capacity and DER signature length, in bytes.; context: Borrowed qsc_hw_pkcs11_tls_signer, updated with backend status..

Return: ASN.1 success only if token signing and DER encoding succeed.

Header: hwplatform.h

qsc_hw_platform_dispose

Purpose: Clear adapter metadata without deleting persistent platform key.

void qsc_hw_platform_dispose(qsc_hw_platform_context* context);

Parameters: context: Caller-owned object with no outstanding provider/key references..

Return: None (void).

Notes: Native service remains caller-owned; do not use this before provider_close.

qsc_hw_platform_get_operations

Purpose: Return generic provider operations wrapping this OS-backed P-256 identity.

qsc_hw_status qsc_hw_platform_get_operations(qsc_hw_provider_ops* operations);

Parameters: operations: Caller-owned descriptor populated on success..

Return: Success, or invalid_parameter when NULL.

qsc_hw_platform_initialize

Purpose: Initialize one portable platform signing context without creating keys.

qsc_hw_status qsc_hw_platform_initialize(qsc_hw_platform_context* context, const qsc_hw_platform_callbacks* callbacks, void* service, qsc_hw_platform_family family, const char* alias, bool require_strongbox, bool hardware_verified);

Parameters: context: Zero-initialized caller-owned context.; callbacks: Native bridge callback table; copied and validated.; service: Non-NULL caller-owned native service instance.; family: Apple or Android, never none.; alias: Nonempty ASCII alias with only letters, digits, '.', '_' or '-'.; require_strongbox: Valid only for Android, forces StrongBox custody.; hardware_verified: Device-specific qualification decision, not inferred from API..

Return: Typed status. No key/device operation occurs during initialization.

qsc_hw_platform_locator

Purpose: Format canonical apple:// or android:// persistent key locator.

qsc_hw_status qsc_hw_platform_locator(const qsc_hw_platform_context* context, char* output, size_t* outputlen);

Parameters: context: Initialized platform identity scope.; output: Output bytes or NULL for size query.; outputlen: In/out capacity and required count including NUL..

Return: Success or buffer_too_small; no writes outside supplied capacity.

Header: hwprovider.h

qsc_hw_key_agree

Purpose: Derive an explicitly exportable shared secret using a protected key.

qsc_hw_status qsc_hw_key_agree(qsc_hw_keyref* key, const uint8_t* peer, size_t peerlen, uint8_t* secret, size_t* secretlen);

Parameters: key: Live key reference.; peer: Public peer key bytes.; peerlen: Peer public key bytes.; secret: Caller-owned output or NULL for length query.; secretlen: In/out capacity/required bytes..

Return: Success or typed error; never claim the resulting exported bytes are hardware-resident.

qsc_hw_key_attest

Purpose: Request key-bound attestation evidence without verifying its trust chain.

qsc_hw_status qsc_hw_key_attest(qsc_hw_keyref* key, const uint8_t* challenge, size_t challengelen, uint8_t* evidence, size_t* evidencelen);

Parameters: key: Live key reference.; challenge: Required nonempty fresh challenge bytes.; challengelen: Challenge length in bytes.; evidence: Caller-owned output or NULL for length query.; evidencelen: In/out capacity/required length..

Return: Success or typed failure. Caller must independently verify freshness and trust.

Notes: Backend error (except buffer_too_small) clears the supplied evidence capacity.

qsc_hw_key_generate

Purpose: Generate and reference a device-protected key under explicit policy.

qsc_hw_status qsc_hw_key_generate(qsc_hw_keyref** key, qsc_hw_provider* provider, const qsc_hw_key_policy* policy);

Parameters: key: Output receives transient key reference.; provider: Live provider.; policy: Requested algorithm, usage and custody; backend must enforce before generation..

Return: Typed error or success; persistent key rollback is backend responsibility on failure.

qsc_hw_key_get_info

Purpose: Retrieve public key metadata, not private key material.

qsc_hw_status qsc_hw_key_get_info(const qsc_hw_keyref* key, qsc_hw_key_info* info);

Parameters: key: Live key reference.; info: Caller-owned versioned output (size/version set before calling)..

Return: Success or typed error.

qsc_hw_key_open

Purpose: Open a key with explicit custody and usage requirements.

qsc_hw_status qsc_hw_key_open(qsc_hw_keyref** key, qsc_hw_provider* provider, const char* locator, const qsc_hw_key_policy* policy);

Parameters: key: Output receives allocated transient reference; no private bytes.; provider: Live provider.; locator: NUL-terminated persistent key locator, not a session handle.; policy: Requested immutable policy with size/version..

Return: Typed error or success; backend object released on policy mismatch.

qsc_hw_key_public

Purpose: Retrieve public key bytes from an opened protected key.

qsc_hw_status qsc_hw_key_public(qsc_hw_keyref* key, uint8_t* output, size_t* outputlen);

Parameters: key: Live key reference, including a current generation.; output: Writable public bytes or NULL for a size query.; outputlen: In/out capacity and actual/required byte length..

Return: Success, unsupported, stale-key or backend error.

qsc_hw_key_release

Purpose: Release one transient key handle without deleting persistent hardware objects.

qsc_hw_status qsc_hw_key_release(qsc_hw_keyref** key);

Parameters: key: Address of live key; set to NULL on successful release..

Return: Typed status; caller synchronizes release with in-flight calls.

qsc_hw_key_sign

Purpose: Sign using an opened key, with enforced usage and custody policy.

qsc_hw_status qsc_hw_key_sign(qsc_hw_keyref* key, const uint8_t* message, size_t msglen, uint8_t* signature, size_t* signaturelen);

Parameters: key: Live key reference.; message: Input bytes or NULL when msglen is zero.; msglen: Input byte length.; signature: Output or NULL for length query.; signaturelen: In/out capacity and actual/required bytes..

Return: Success or typed failure; unsupported operations never invoke the backend.

Notes: Backend error (except buffer_too_small) clears the supplied signature capacity.

qsc_hw_provider_capabilities

Purpose: Obtain a current capability snapshot.

qsc_hw_status qsc_hw_provider_capabilities(qsc_hw_provider* provider, qsc_hw_capabilities* capabilities);

Parameters: provider: Live provider handle.; capabilities: Caller initializes size/version; output available on success..

Return: Typed error or success; no device attestation implied.

qsc_hw_provider_close

Purpose: Close a provider without invalidating outstanding handles.

qsc_hw_status qsc_hw_provider_close(qsc_hw_provider** provider);

Parameters: provider: Address of live provider; set to NULL on success..

Return: busy while any key references remain, otherwise success.

Notes: Caller synchronizes close with all active operations. Backend context remains caller-owned.

qsc_hw_provider_invalidate

Purpose: Invalidate all previously opened keys on device reconnect or replacement.

qsc_hw_status qsc_hw_provider_invalidate(qsc_hw_provider* provider);

Parameters: provider: Live provider; caller serializes against concurrent key operations..

Return: Success or typed error. Invalidated keys must still be released.

qsc_hw_provider_open

Purpose: Construct an isolated provider wrapper around an externally managed backend context.

qsc_hw_status qsc_hw_provider_open(qsc_hw_provider** provider, const qsc_hw_provider_ops* operations, void* context);

Parameters: provider: Output receives allocated handle; set to NULL before validation when non-NULL.; operations: Descriptor with recognized ABI version, minimum mandatory callbacks and size.; context: Backend context; caller guarantees lifetime through successful close..

Return: Success or typed failure. Failure transfers no ownership and does not invoke dispose.

Notes: Caller must synchronize close against concurrent operations.

qsc_hw_provider_random

Purpose: Request random data from a provider's optional RNG.

qsc_hw_status qsc_hw_provider_random(qsc_hw_provider* provider, uint8_t* output, size_t length);

Parameters: provider: Live provider.; output: Writable buffer, required for nonzero length.; length: Requested output bytes; zero is permitted..

Return: Typed status; not an entropy quality or FIPS approval claim.

Header: hwpsa.h

qsc_hw_psa_dispose

Purpose: Erase bridge state only, never remove PSA persistent keys or close OEM service.

void qsc_hw_psa_dispose(qsc_hw_psa_context* context);

Parameters: context: Caller-owned context with no outstanding provider/key handles..

Return: None (void).

qsc_hw_psa_get_operations

Purpose: Construct versioned callback operations for the generic QSC provider.

qsc_hw_status qsc_hw_psa_get_operations(qsc_hw_provider_ops* operations);

Parameters: operations: Caller-owned versioned descriptor, cleared before population..

Return: Success or invalid parameter.

qsc_hw_psa_initialize

Purpose: Initialize a caller-owned PSA provider scope without creating/destroying keys.

qsc_hw_status qsc_hw_psa_initialize(qsc_hw_psa_context* context, const qsc_hw_psa_callbacks* callbacks, void* service, uint32_t key_id, bool hardware_verified);

Parameters: context: Zero-initialized caller-owned context.; callbacks: Versioned OEM PSA service operations.; service: Non-NULL OEM-owned initialized PSA service context.; key_id: Nonzero persistent PSA ID; pin one identity per provider instance.; hardware_verified: True only after device-specific independent custody proof..

Return: Typed error or success; no authorization prompts or vendor calls.

qsc_hw_psa_locator

Purpose: Format a canonical persistent PSA ID locator, no secret material.

qsc_hw_status qsc_hw_psa_locator(qsc_hw_psa_context* context, char* output, size_t* outputlen);

Parameters: context: Initialized PSA context.; output: Caller output or NULL for size query.; outputlen: In/out byte capacity, including final NUL..

Return: Success or buffer-too-small; outputs exactly QSC_HW_PSA_LOCATOR_SIZE bytes.

Header: hwpsanative.h

qsc_hw_psa_native_initialize

Purpose: Initialize a native PSA client and bind a fixed persistent ID to QSC.

qsc_hw_status qsc_hw_psa_native_initialize(qsc_hw_psa_context* context, qsc_hw_psa_native_service* service, uint32_t key_id, uint32_t location, bool hardware_verified);

Parameters: context: Zero-initialized caller-owned PSA bridge state.; service: Zero-initialized caller-owned native PSA service state.; key_id: Nonzero persistent PSA key identifier.; location: Expected PSA storage location returned by platform key attributes.; hardware_verified: Independently qualified physical key-operation path..

Return: Typed status. Does not take device ownership or delete keys.

Notes: A local-storage PSA key can never be claimed as hardware-backed here.

Header: hwse050.h

qsc_hw_se050_preflight

Purpose: Preflight the NXP middleware token before opening QSC PKCS#11 keys.

qsc_hw_status qsc_hw_se050_preflight(qsc_hw_pkcs11_module* module, uint64_t slot, const qsc_hw_se050_policy* policy);

Parameters: module: Loaded vendor Cryptoki module, managed by existing QSC PKCS#11 API.; slot: Current slot enumerated from that module; not a persistent identity.; policy: Versioned caller requirements including exact expected serial..

Return: Success only for initialized token with exact serial and requested mechanisms.

Notes: Does not certify device make, applet, SCP03, residency, secure boot or FIPS. Key custody is checked later by QSC's existing PKCS#11 protected-key functions.

Header: hwtpm.h

qsc_hw_tpm_dispose

Purpose: Release only local ESAPI/TCTI state without deleting persistent TPM objects.

void qsc_hw_tpm_dispose(qsc_hw_tpm_context* context);

Parameters: context: Backend context; may be NULL or already disposed..

Return: None (void).

Notes: Must be called only after the generic provider wrapper and keys close.

qsc_hw_tpm_get_ak_identity

Purpose: Retrieve the opened AK's public key and TPM Name for out-of-band trust enrollment.

qsc_hw_status qsc_hw_tpm_get_ak_identity(const qsc_hw_tpm_context* context, uint8_t* publickey, uint8_t* akname);

Parameters: context: Initialized context with a persistent AK handle.; publickey: Caller-owned 64-byte P-256 Qx||Qy output.; akname: Caller-owned 34-byte TPM Name (SHA-256 name algorithm and digest)..

Return: success, unsupported when no AK configured, or typed failure.

Notes: Outputs are untrusted metadata until checked against OEM trust anchors.

qsc_hw_tpm_get_operations

Purpose: Populate the generic QSC hardware provider callback descriptor.

qsc_hw_status qsc_hw_tpm_get_operations(qsc_hw_provider_ops* operations);

Parameters: operations: Caller-owned output descriptor..

Return: success only when compiled for Linux with TPM2-TSS ESAPI support; otherwise unsupported, with a cleared descriptor.

qsc_hw_tpm_initialize

Purpose: Open a pre-provisioned non-exportable P-256 signing identity on a Linux TPM.

qsc_hw_status qsc_hw_tpm_initialize(qsc_hw_tpm_context* context, const char* tcti, uint32_t identityhandle, uint32_t akhandle, const uint8_t* pcrselect, bool physical_device_confirmed);

Parameters: context: Caller-owned zeroed context initialized only on success.; tcti: Optional installed TCTI configuration, or NULL for system default.; identityhandle: TPM persistent identity handle in 0x81000000..0x81FFFFFF.; akhandle: TPM persistent P-256 attestation-key handle, or zero to disable quotes.; pcrselect: Optional SHA-256 bank 3-byte PCR bitset, or NULL for PCR 0.; physical_device_confirmed: Explicit OEM-confirmed physical TPM classification. False labels key custody as software for simulators, subject to caller fallback policy. True requires external evidence of real TPM transport, not inferred from key flags..

Return: success only for a verified TPM-resident sign-capable key, unsupported when TSS is not compiled or the TPM/handle cannot be accessed; typed failures otherwise.

Notes: Handles must have empty authorization and be provisioned by authorized tooling. Never clears, reassigns or changes TPM ownership. Does not attest platform origin.

Header: hwtpmquote.h

qsc_hw_tpm_quote_envelope_verify

Purpose: Decode the QSC TPM2-TSS quote envelope and verify its signed claims.

qsc_hw_status qsc_hw_tpm_quote_envelope_verify(const uint8_t* envelope, size_t envelopelen, const qsc_hw_tpm_quote_policy* policy);

Parameters: envelope: Caller-owned bytes containing big-endian u16 quote size, quote, big-endian u16 TPMT_SIGNATURE size and signature bytes.; envelopelen: Exact encoded envelope size; trailing bytes are rejected.; policy: Trusted verifier policy for signature, nonce, AK and PCR measurements..

Return: success only for canonical envelope and verified quote; typed error otherwise.

Notes: This is a convenience parser; it does not supply a hardware trust anchor.

qsc_hw_tpm_quote_verify

Purpose: Verify a bounded TPM2 Quote with a trusted AK and expected PCR policy.

qsc_hw_status qsc_hw_tpm_quote_verify(const uint8_t* quote, size_t quotelen, const uint8_t* signature, size_t signaturelen, const qsc_hw_tpm_quote_policy* policy);

Parameters: quote: Raw serialized TPMS_ATTEST bytes (without outer TPM2B size word).; quotelen: Number of quote bytes, 1..QSC_HW_TPM_QUOTE_MAX_SIZE.; signature: Serialized TPMT_SIGNATURE ECDSA/SHA-256 (big-endian TPM wire format).; signaturelen: Exact signature byte count, maximum 76 for ECDSA P-256.; policy: Required expected nonce, AK Name, AK public key, PCR selection/digest..

Return: success only for valid signed content and matching policy; unsupported for non-P256/SHA-256 signature/selection; invalid_parameter for malformed wire input; policy for mismatched verified claims or failed cryptographic signature.

Notes: Does not consult the TPM, validate AK certificate chains, verify PCR event logs, or provide cross-quote replay tracking. Caller must enforce fresh one-use challenges. Thread-safe for independent immutable arguments; all buffers remain caller-owned.

Annex A: Design Specifications

Design Objectives

The QSC library was designed to be a lightweight, intuitive, and powerful set of cryptographic tools. We took many different considerations into account while designing this library; it had to be easy to use, so as to avoid implementation mistakes. It had to be compact, so as to fit on a large range of hardware implementations. The library has to be fast, to be competitive in high-performance environments. Most of all though, we have designed QSC to be powerful; this library contains some of the most advanced, and future-secure cryptographic primitives available in the world. We developed a library that will withstand the technological changes that are certain to come, as we venture into the quantum age.

We chose to make the library MISRA compliant, a very strict and comprehensive set of secure C programming-language guidelines. We upgraded existing codes to this format, including asymmetric ciphers and signature schemes, and have followed a strict regimen of secure programming practices throughout the library. Many industries now insist on code that conforms to these rule-sets, including the automobile industry, military and government organizations, and the aeronautics industry. Source codes must conform to these standards, or they will not be accepted by a rapidly growing number of industries. These rules were designed by experts in computer security, and MISRA has become the standard of computer-code conformance when using the C or C++ language in a secure context.

We have integrated SIMD instructions throughout the library, spanning many different supporting functions and cryptographic primitives. Most server hardware is now SIMD capable, and we have implemented those instructions as optional implementations for compatible CPUs. We have used the modern subsets of SIMD instructions; AVX, AVX2, and the newest version AVX512, in functions integrated into the library, from common memory functions, to entire primitives like Falcon, being implemented in SIMD instructions. The library also contains support for asynchronous threading, parallel loops, event callbacks, and a variety of multi-threading support functions.

We have implemented secure IPv4 and IPv6 networking stacks. These TCP/IP networking functions, cover a wide range of network operations, including synchronous and asynchronous sockets, complete IPv6 support, and parallel implementations of **Winsock** for Windows devices, or **Berkely Sockets** for Unix and Linux. We have added simplified server and client functionality, including asynchronous server *accept* and *receive* operations. We have also added a queuing implementation, and many peripheral network support operations, all in a straightforward, easy-to-use format.

A thorough testing framework is critical to developing secure code, and we have implemented a comprehensive set of testing functions, that perform operation wellness tests, known answer tests, and stress tests on every cryptographic primitive. We use only the official and most-current known answer tests (KATs), including the NIST PQ3 official tests for the asymmetric primitives. In addition to this are custom stress and wellness tests, and official testing frameworks like the NIST AESAVS tests, and the official NIST SHA3 tests.

The library organization and naming conventions take on a natural language approach, where the API and function purpose are synonymous, making the implementation as obvious as possible. We re-use many keywords throughout the library like *initialize*, *update*, and *generate*, to create a cohesive set of instruction meanings, and define a memorable repetition-based function language.

API Naming Patterns

- All public functions are prefixed with the library name **qsc_**
- The API naming convention begins with the library-name prefix, followed by the primitive or function-family name, and ending with the function-name, ex. `void qsc_aes_dispose(qsc_aes_state* ctx);`
- Pointer types shall place the pointer character (*) alongside the integer type ex. **void qsc_stringutils_clear_string(char* source)**
- The function portion of a function name should not contain more than 3 words, and ideally, use only a single word, ex. **qsc_kyber_generate(...)**, the 'generate' portion is the function name, 'kyber' is the function-family name, and 'qsc' is the root library name
- A function state should always be the first parameter in a function call, even if it is declared a constant. The function parameters should always have output receiving parameters before constant parameters, but after a function state, ex. `void qsc_aes_cbc_encrypt_block(qsc_aes_state* ctx, uint8_t* output, const uint8_t* input);`
- A function that returns a binary value (0/1), shall use the **bool** integer type
- A function that returns from a range of condition values (-1/0/1), shall return an **enumeration** type
- A function that returns a length or size value, shall return a **size_t** type
- Function variables should be declared at the top function, or at the top of an inner scope

- Function variables shall not be initialized in the declaration line
- Header files should only contain public functions, whenever possible all other functions should be declared static in the implementation file.
- Each primitives functions should be contained in a single implementation file whenever possible. Exceptions should be in files with the format *primitive-base*, ex. **kyberbase.h**
- To differentiate static functions in an implementation file, the library prefix should be excluded from static functions in the implementation file to denote locality, ex. **static void aes_beincrement_x128(__m128i* counter).**
- Use overlapping keywords in the API for similar functionality, ex: **qsc_kyber_generate_keypair(...), or qsc_hkdf256_extract(...).**
- Global constants should be declared in the standard C language form of all uppercase letters, using words separated by underscores, ex. QSC_KYBER_PRIVATEKEY_SIZE
- A constant should have no fewer than 2 words, and no more than 5 words
- Function local constants should be 6 letters, one word in uppercase ex. **CTRLEN**
- Public file names should strive to be a single word ex. **sha2**, helper files can use 2 words.
- Every public file, function, structure, enumeration, definition and type, must be documented

Rules

- No macros; macros are forbidden
- Excepting memory functions and callbacks, the type **void*** shall not be used
- Use standard integer types throughout (stdint)
- Pass only the defined integer types to a function
- Use the **size_t** integer type for every length and size parameter
- No casting to a different base integer or structure type
- Limit define-trees as much as is possible, and primarily to file includes
- Constants declared with correct integer-type and made static whenever possible
- Integrate memory functions using **SIMD**, as alternatives to *malloc*, *memcpy*, and *memset*
- Any function that can leak secret information must be constant time
- Enforce all **MISRA** rules, strive for no exceptions
- Documentation includes an overview and brief description, with all operations explained, a working example and reference links

- Document every public function, structure, enumeration, type, and constant
- Use extended function documentation flags used for auto-documentation
- Testing must be thorough and regimented, and every cryptographic primitive must be tested
- Create a separate test library that employs a testing framework, applied to all primitives and functions in the library
- Use random sampling tests, all relevant KAT tests, stress, authentication, and wellness tests for everything, including supporting tool functions